

pvtrend

Price and Volume Trend (PVT)

Syntax

```
trend = pvtrend(Data)
```

Description

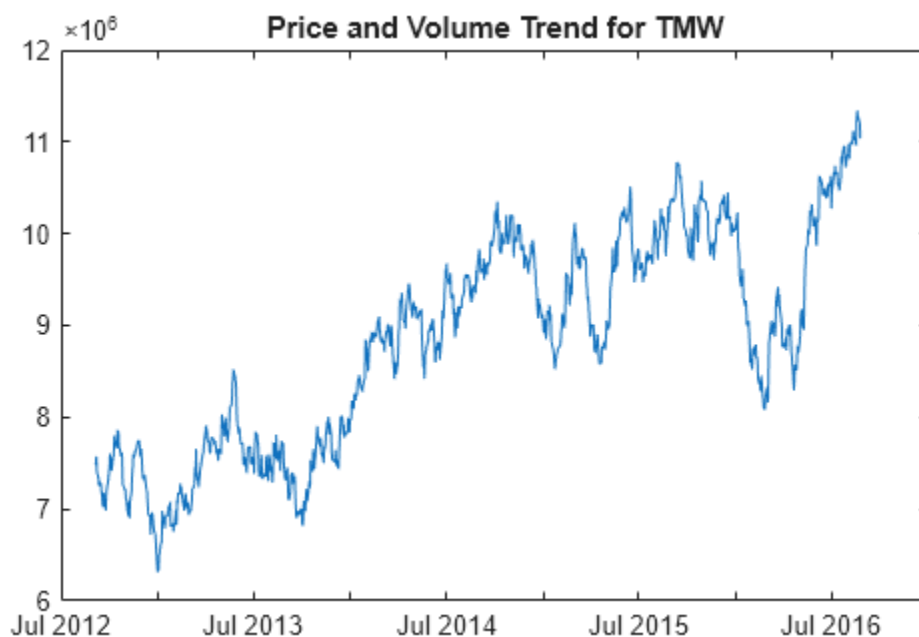
`trend = pvtrend(Data)` calculates the Price and Volume Trend (PVT) from the series of closing stock prices and trade volume.

Examples

Calculate the Price and Volume Trend for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
trend = pvtrend(TMW);  
plot(trend.Time,trend.PriceVolumeTrend)  
title('Price and Volume Trend for TMW')
```



Input Arguments

Data — Data for closing prices and trade volume

matrix | table | timetable

Data for closing prices and trade volume, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-2 matrix of closing prices and trade volume. Timetables and tables with M rows must contain variables named 'Close' and 'Volume' (case insensitive).

Data Types: double | table | timetable

Output Arguments

trend — Price and volume trend

matrix | table | timetable

Price and volume trend (PVT), returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

Price and Volume Trend Chart

The price and volume trend (PVT) chart combines price movements and trading volume into a single indicator, allowing traders and investors to analyze the relationship between price changes and volume in a more integrated manner.

The PVT chart is a cumulative indicator that adds or subtracts volume based on the price movement. It reflects the overall direction of price trends while accounting for volume, providing insights into market strength.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 239-240.

See Also

timetable | table | onbalvol | volroc | tsaccel

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

pvvar

Present value of varying cash flow

Syntax

```
PresentVal = pvvar(CashFlow,Rate)
PresentVal = pvvar( ____,CFDates)
```

Description

`PresentVal = pvvar(CashFlow,Rate)` calculates present value of a varying cash flow.

`PresentVal = pvvar( ____,CFDates)` adds an optional argument for `CFDates`.

Examples

Calculate Present Value for Regular and Irregular Cash Flow

Calculate the net present value for a regular and irregular cash flow.

Regular Cash Flow

This cash flow represents the yearly income from an initial investment of \$10,000. The annual interest rate is 8%.

Year 1 - \$2000

Year 2 - \$1500

Year 3 - \$3000

Year 4 - \$3800

Year 5 - \$5000

To calculate the net present value of this regular cash flow:

```
PresentVal = pvvar([-10000 2000 1500 3000 3800 5000], 0.08)
```

```
PresentVal =  
1.7154e+03
```

Irregular Cash Flow

An investment of \$10,000 returns this irregular cash flow. The original investment and its date are included. The periodic interest rate is 9%.

January 12, 1987 - (\$10000)

February 14, 1988 - \$1500

March 3, 1988 - \$2000

June 14, 1988 - \$3000

December 1, 1988 - \$4000

To calculate the net present value of this irregular cash flow:

```
CashFlow = [-10000, 1500, 2000, 3000, 4000];
```

```
CFDates = [ '01/12/1987'
             '02/14/1988'
             '03/03/1988'
             '06/14/1988'
             '12/01/1988' ];
```

```
PresentVal = pvvar(CashFlow, 0.09, CFDates)
```

```
PresentVal =
-768.1461
```

The net present value of the same investment under different discount rates of 7%, 9%, and 11% is obtained by:

```
PresentVal = pvvar(repmat(CashFlow,3,1)', [.07 .09 .11], CFDates)
```

```
PresentVal = 1×3
103 ×
    -0.5099    -0.7681    -1.0146
```

Input Arguments

CashFlow — Cash flow amounts

vector

Cash flow amounts, specified as a vector of varying cash flows. Include the initial investment as the initial cash flow value (a negative number). If **CashFlow** is a matrix, each column is treated as a separate cash-flow stream.

Data Types: double

Rate — Periodic interest rate

decimal

Periodic interest rate, specified as a decimal. If **CashFlow** is a matrix, a scalar **Rate** is allowed when the same rate applies to all cash-flow streams in **CashFlow**. When multiple cash-flow streams require different discount rates, **Rate** must be a vector whose length equals the number of columns in **CashFlow**.

Data Types: double

CFDates — Indicates irregular cash flow

datetime array | string array | cell array of date character vectors

(Optional) Indicates irregular cash flow, specified as a `datetime` array, string array, or cell array of date character vectors on which the cash flows occur.

Specify `CFDates` when there are irregular (nonperiodic) cash flows. The default assumes that `CashFlow` contains regular (periodic) cash flows. If `CashFlow` is a matrix, and all cash-flow streams share the same dates, `CFDates` can be a vector whose length matches the number of rows in `CashFlow`. When different cash-flow streams have different payment dates, specify `CFDates` as a matrix the same size as `CashFlow`.

To support existing code, `pvvar` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `cell` | `string` | `char` | `datetime`

Output Arguments

PresentVal — Present value

numeric

Present value, returns the net present value of a varying cash flow. Present value is calculated at the time the first cash flow occurs.

More About

Present Value

Present value is a fundamental financial concept that refers to the current worth of a sum of money or a stream of cash flows that will be received or paid in the future, discounted at a specific interest rate.

The idea behind present value is that a certain amount of money today is worth more than the same amount in the future due to its potential earning capacity. This concept is crucial in finance for valuing investments, making financial decisions, and assessing the worth of future cash flows.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `pvvar` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`fvfix` | `fvvar` | `irr` | `payuni` | `pvfix` | `datetime`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

pyld2zero

Zero curve given par yield curve

Note In R2017b, the specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the new optional name-value pair inputs: `InputCompounding`, `InputBasis`, `OutputCompounding`, and `OutputBasis`.

Syntax

```
[ZeroRates, CurveDates] = pyld2zero(ParRates, CurveDates, Settle)
[ZeroRates, CurveDates] = pyld2zero( ___, Name, Value)
```

Description

`[ZeroRates, CurveDates] = pyld2zero(ParRates, CurveDates, Settle)` returns a zero curve given a par yield curve and its maturity dates. If either input for `CurveDates` or `Settle` is a datetime array, `CurveDates` is returned as a datetime array. Otherwise, `CurveDates` is returned as a serial date number.

`[ZeroRates, CurveDates] = pyld2zero( ___, Name, Value)` adds optional name-value pair arguments

Examples

Compute Zero Curve Given Par Yield Curve

Define the settlement date, maturity, and zero rates.

```
Settle = datenum('01-Feb-2013');
CurveDates = datemnth(Settle, 12*[1 2 3 5 7 10 20 30]');
ZeroRates = [.11 0.30 0.64 1.44 2.07 2.61 3.29 3.55]'/100;
InputCompounding = 2;
InputBasis = 1;
OutputCompounding = 2;
OutputBasis = 1;
```

Compute par yield curve from zero rates.

```
ParRates = zero2pyld(ZeroRates, CurveDates, Settle, 'InputCompounding', 2, ...
    'InputBasis', 1, 'OutputCompounding', 2, 'OutputBasis', 1)
```

```
ParRates = 8×1
```

```
0.0011
0.0030
0.0064
0.0142
0.0201
```



```
0.0251
0.0309
0.0330
```

Compute zero curve from the par yield curve.

```
ZeroRates = pyld2zero(ParRates, CurveDates, Settle, 'InputCompounding', 2, ...
    'InputBasis', 1, 'OutputCompounding', 2, 'OutputBasis', 1)
```

```
ZeroRates = 8×1
```

```
0.0011
0.0030
0.0064
0.0144
0.0207
0.0261
0.0329
0.0355
```

Compute Zero Curve Given Par Yield Curve Using datetime Inputs

Use datetime inputs to compute the zero curve given the par yield curve.

```
Settle = datetime(2013,2,1);
```

```
CurveDates = [datetime(2014,2,1)
    datetime(2015,2,1)
    datetime(2016,2,1)
    datetime(2018,2,1)
    datetime(2020,2,1)
    datetime(2023,2,1)
    datetime(2033,2,1)
    datetime(2043,2,1)];
```

```
OriginalParRates = [0.11 0.30 0.64 1.42 2.02 2.51 3.10 3.31]'/100;
```

```
InputCompounding = 1;
InputBasis = 0;
OutputCompounding = 1;
OutputBasis = 0;
```

```
[ZeroRates Dates] = pyld2zero(OriginalParRates, CurveDates, Settle, ...
    'OutputCompounding', OutputCompounding, 'OutputBasis', OutputBasis, ...
    'InputCompounding', InputCompounding, 'InputBasis', InputBasis)
```

```
ZeroRates = 8×1
```

```
0.0011
0.0030
0.0064
0.0144
0.0207
0.0261
```

```
0.0329
0.0356
```

```
Dates = 8×1 datetime
01-Feb-2014
01-Feb-2015
01-Feb-2016
01-Feb-2018
01-Feb-2020
01-Feb-2023
01-Feb-2033
01-Feb-2043
```

Demonstrate a Roundtrip From pyld2zero to zero2pyld

Given the following a par yield curve and its maturity dates, return the ZeroRates.

```
Settle = datetime(2013,2,1);

CurveDates = [datetime(2014,2,1)
datetime(2015,2,1)
datetime(2016,2,1)
datetime(2018,2,1)
datetime(2020,2,1)
datetime(2023,2,1)
datetime(2033,2,1)
datetime(2043,2,1)];

OriginalParRates = [0.11 0.30 0.64 1.42 2.02 2.51 3.10 3.31]'/100;

InputCompounding = 1;
InputBasis = 0;
OutputCompounding = 1;
OutputBasis = 0;

ZeroRates = pyld2zero(OriginalParRates, CurveDates, Settle, ...
'OutputCompounding', OutputCompounding, 'OutputBasis', OutputBasis, ...
'InputCompounding', InputCompounding, 'InputBasis', InputBasis)

ZeroRates = 8×1

0.0011
0.0030
0.0064
0.0144
0.0207
0.0261
0.0329
0.0356
```

With the ZeroRates, use the zero2pyld function to return the ParRatesOut and determine the roundtrip error.

```
ParRatesOut = zero2pyld(ZeroRates, CurveDates, Settle, ...
    'OutputCompounding', OutputCompounding, 'OutputBasis', OutputBasis, ...
    'InputCompounding', InputCompounding, 'InputBasis', InputBasis)
```

```
ParRatesOut = 8×1
```

```
0.0011
0.0030
0.0064
0.0142
0.0202
0.0251
0.0310
0.0331
```

```
max(abs(OriginalParRates - ParRatesOut)) % Roundtrip error
```

```
ans =
1.2750e-16
```

Input Arguments

ParRates — Annualized par yields

decimal fraction

Annualized par yields (coupon rates), specified as a **NUMBONDS**-by-1 vector using decimal fractions. In aggregate, the rates constitute an implied zero curve for the investment horizon represented by **CurveDates**.

Data Types: double

CurveDates — Maturity dates

datetime array | string array | date character vector

Maturity dates which correspond to the input **ParRates**, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, **pyld2zero** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Settle — Common settlement date for **ZeroRates**

datetime scalar | string scalar | date character vector

Common settlement date for input **ParRates**, specified as a scalar datetime, string, or data character vector.

To support existing code, **pyld2zero** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[ZeroRates, CurveDates] =
pyld2zero(ParRates, CurveDates, Settle, 'OutputCompounding', 3, 'OutputBasis', 5, 'InputCompounding', 4, 'InputBasis', 5)`

OutputCompounding — Compounding frequency of output ZeroRates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of output `ZeroRates`, specified using the allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Note

- If `OutputCompounding` is set to 0 (simple), -1 (continuous), or 365 (daily), the `InputCompounding` must also be specified using a valid value.
 - If `OutputCompounding` is not specified, then `OutputCompounding` is assigned the value specified for `InputCompounding`.
 - If either `OutputCompounding` or `InputCompounding` are not specified, the default is 2 (semiannual) for both.
-

Data Types: `double`

OutputBasis — Day-count basis of output ZeroRates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of output `ZeroRates`, specified using allowed values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365

- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If `OutputBasis` is not specified, then `OutputBasis` is assigned the value specified for `InputBasis`. If either `InputBasis` or `OutputBasis` are not specified, the default is 0 (actual/actual) for both.

Data Types: double

InputCompounding — Compounding frequency of input ParRates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of input `ParRates`, specified using allowed values:

- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note

- If `OutputCompounding` is 1, 2, 3, 4, 6, or 12 and `InputCompounding` is not specified, the value of `OutputCompounding` is used.
 - If `OutputCompounding` is 0 (simple), -1 (continuous), or 365 (daily), a valid `InputCompounding` value must also be specified.
 - If either `InputCompounding` or `OutputCompounding` are not specified, the default is 2 (semiannual) for both.
-

Data Types: double

InputBasis — Day-count basis of input ParRates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of the input `ParRates`, specified using allowed values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If `InputBasis` is not specified, then `InputBasis` is assigned the value specified for `OutputBasis`. If either `InputBasis` or `OutputBasis` are not specified, the default is 0 (actual/actual) for both.

Data Types: `double`

Output Arguments

ZeroRates — Zero rates

`numeric`

Zero rates, returned as a `NUMBONDS`-by-1 numeric vector. In aggregate, the rates in `ZeroRates` constitute a zero curve for the investment horizon represented by `CurveDates`. `ZeroRates` are ordered by ascending maturity.

CurveDates — Maturity dates that correspond to ZeroRates

`datetime` | `serial date number`

Maturity dates that correspond to the `ZeroRates`, returned as a `NUMBONDS`-by-1 vector of maturity dates that correspond to each par rate contained in `ZeroRates`. `CurveDates` are ordered by ascending maturity.

If either input for `CurveDates` or `Settle` is a datetime array, `CurveDates` is returned as a datetime array. Otherwise, `CurveDates` are returned as a serial date numbers. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

More About

Par Yield

Par yield is a financial term that refers to the coupon rate of a bond that is trading at par value, which means it is priced at its face value.

The par yield is the interest rate at which the present value of the bond's future cash flows (coupon payments and the face value at maturity) equals its current market price.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although pyld2zero supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`fwd2zero` | `zero2fwd` | `getForwardRates` | `disc2zero` | `zero2disc` | `zero2pyld` | `zbtprice` | `zbtyield` | `datetime`

Topics

"Term Structure of Interest Rates" on page 2-29

"Sensitivity of Bond Prices to Interest Rates" on page 10-2

"Bond Prices and Yield Curve Parallel Shifts" on page 10-9

"Bond Prices and Yield Curve Nonparallel Shifts" on page 10-12

"Term Structure Analysis and Interest-Rate Swaps" on page 10-18

"Fixed-Income Terminology" on page 2-15

renko

Renko

Syntax

```
renko(Data)
renko(Data,Threshold,)
h = renko(ax, ____)
```

Description

`renko(Data)` plots a Renko chart from a series of prices of a security.

`renko(Data,Threshold,)` adds an optional argument for `Threshold`.

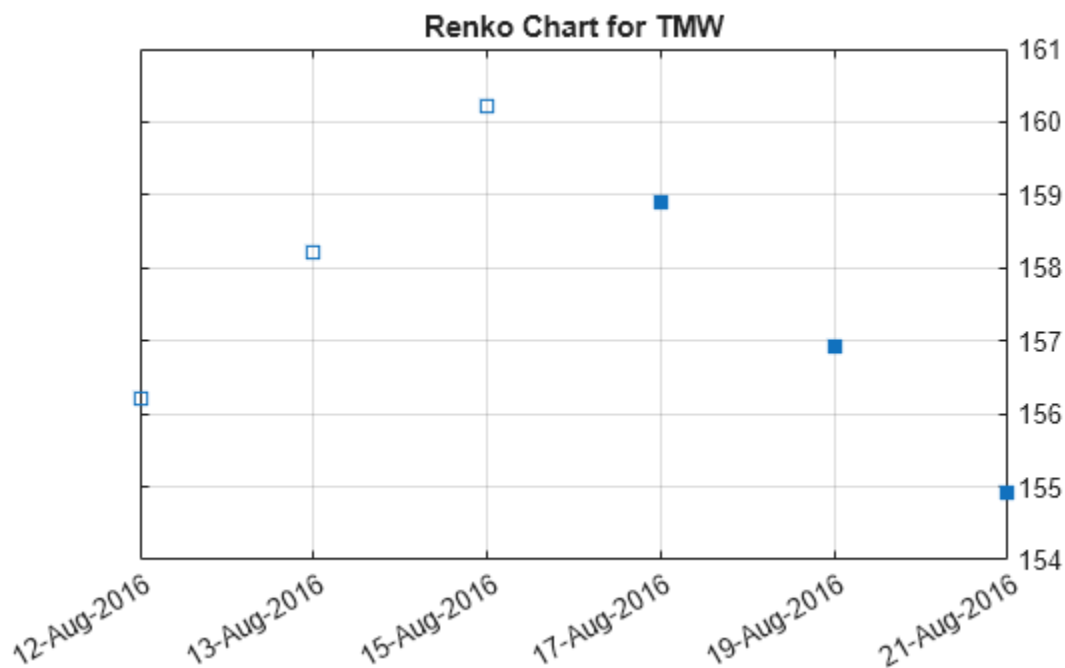
`h = renko(ax, ____)` adds an optional argument for `ax`.

Examples

Generate a Renko Chart for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This example shows how to plot a Renko chart for the most recent 21 days. Note that the variable name of asset price is renamed to 'Price' (case insensitive).

```
load SimulatedStock.mat
TMW.Properties.VariableNames{'Close'} = 'Price';
renko(TMW(end-8:end,:),2)
title('Renko Chart for TMW')
```

Input Arguments

Data — Data for a series of prices

matrix | table | timetable

Data for a series of prices, specified as a matrix, table, or timetable. Timetables and tables with *M* rows must contain a variable named 'Price' (case insensitive).

Data Types: double | table | timetable

Threshold — (Optional) Least price change value when adding a new box

1 (default) | positive numeric

Least price change value when adding a new box, specified as a scalar positive numeric value.

Data Types: double

ax — Valid axis object

current axes (ax = gca) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The renko plot is created in the axes specified by *ax* instead of in the current axes (ax = gca). The option *ax* can precede any of the input argument combinations.

Data Types: object

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Renko Chart

A Renko chart is a type of financial chart used in technical analysis to represent price movements of an asset without considering time intervals.

Instead of plotting price changes over time, Renko charts focus solely on significant price movements, filtering out minor fluctuations. This makes them particularly useful for identifying trends and reversals in the market.

Version History

Introduced in R2008a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

renko accepts Data input as a matrix, timetable, or table.

The syntax for renko has changed. Previously, when using table input, the first column of dates could be a datetime array, date character vectors, or serial date numbers, and you were required to have specific number of columns.

When using table input, the new syntax for renko supports:

- No need for time information. If you want to pass in date information, use timetable input.
- No requirement of specific number of columns. However, you must provide valid column names. renko must contain a column named 'price' (case insensitive).

See Also

timetable | table | movavg | linebreak | highlow | kagi | priceandvol | volarea | candle | pointfig

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

ret2tick

Convert return series to price series

Syntax

```
[TickSeries, TickTimes] = ret2tick(Data)
[TickSeries, TickTimes] = ret2tick( ___, Name, Value)
```

Description

[TickSeries, TickTimes] = ret2tick(Data) computes prices from the start prices of NASSET assets and NUMOBS return observations.

[TickSeries, TickTimes] = ret2tick(___, Name, Value) adds optional name-value pair arguments.

Examples

Convert a Return Series to a Price Series

Compute the price increase of two stocks over a year's time based on three incremental return observations.

```
RetSeries = [0.10 0.12
             0.05 0.04
            -0.05 0.05];
```

```
RetIntervals = [182
                91
                92];
```

```
StartTime = datetime('18-Dec-2000', 'Locale', 'en_US');
```

```
[TickSeries, TickTimes] = ret2tick(RetSeries, 'ReturnIntervals', RetIntervals, ...
    'StartTime', StartTime)
```

```
TickSeries = 4×2
```

```
1.0000    1.0000
1.1000    1.1200
1.1550    1.1648
1.0973    1.2230
```

```
TickTimes = 4×1 datetime
18-Dec-2000
18-Jun-2001
17-Sep-2001
18-Dec-2001
```

Convert a Price Series to a Return Series Using `timetable` Input

Use `timetable` input to convert a price series to a return series, given periodic returns of two stocks observed in the first, second, third, and fourth quarters.

```
RetSeries = [0.10 0.12
             0.05 0.04
            -0.05 0.05];
```

```
RetTimes = datetime({'6/18/2001','9/17/2001','12/18/2001'},'InputFormat','MM/dd/yyyy','Locale','en_US');
RetSeries = array2timetable(RetSeries,'RowTimes',RetTimes);
StartTime = datetime('12/18/2000','InputFormat','MM/dd/yyyy','Locale','en_US');
```

```
[TickSeries, TickTimes] = ret2tick(RetSeries, 'StartTime', StartTime)
```

```
TickSeries=4x2 timetable
      Time      RetSeries1      RetSeries2
      _____      _____      _____
18-Dec-2000           1           1
18-Jun-2001          1.1          1.12
17-Sep-2001         1.155         1.1648
18-Dec-2001         1.0973         1.223
```

```
TickTimes = 4x1 datetime
18-Dec-2000
18-Jun-2001
17-Sep-2001
18-Dec-2001
```

Input Arguments

Data — Data for asset returns

matrix | table | timetable

Data for asset returns, specified as a `NUMOBSN`ASSETS matrix, table, or timetable. The returns are not normalized by the time increments between successive price observations.

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `[TickSeries, TickTimes] = ret2tick(RetSeries, 'StartTime', StartTime)`

StartPrice — Initial prices for each asset

1 for all assets (default) | numeric

Initial prices for each asset, specified as the comma-separated pair consisting of 'StartPrice' and a NASSETS-by-1 vector indicating initial prices for each asset, or a single scalar initial price applied to all assets.

Data Types: double

ReturnIntervals — Return interval between prices

1 for all assets (default) | numeric | duration | calendar duration

Return interval between prices, specified as the comma-separated pair consisting of 'ReturnIntervals' and a scalar return interval applied to all returns, or a vector of length NUMOBS return intervals between successive returns. ReturnIntervals is defined as:

$$\text{ReturnIntervals}(t) = \text{TickTimes}(t) - \text{TickTimes}(t-1).$$

Note If the type of Data is a timetable, ReturnIntervals is ignored.

Data Types: double

StartTime — Starting time for first observation applied to the prices of all assets

0 if ReturnIntervals is numeric (default) | numeric | duration | calendar duration

Starting time for first observation applied to the price series of all assets, specified as the comma-separated pair consisting of 'StartTime' and a scalar string, character vector, double, or datetime.

Note If ReturnIntervals is a duration or calendar duration value, the default for StartTime is datetime('today').

If Data is a timetable and StartTime is not specified, the resulting asset prices in the first period are not reported.

Data Types: double | string | char | datetime

Method — Method to convert asset returns to prices

'Simple' (default) | character vector with value of 'Simple' or 'Continuous' | string with value of "Simple" or "Continuous"

Method to convert asset returns to prices, specified as the comma-separated pair consisting of 'Method' and a string or character vector indicating the method to convert asset prices to returns.

If the method is 'Simple', then simple periodic returns are used:

$$\text{TTickSeries}(t) = \text{TickSeries}(t-1) * (1 + \text{ReturnSeries}(t)).$$

If the method is 'Continuous', then continuous returns are used:

$$\text{TickSeries}(t) = \text{TickSeries}(t-1) * \exp(\text{ReturnSeries}(t)).$$

Data Types: char | string

Output Arguments

TickSeries — Time series array of asset prices

matrix | table | timetable

Time series array of asset prices, returned as NUMBOBS+1-by-NASSETS time series of asset prices of the same type (matrix, table, or timetable) as the input `Data`. The first row contains the oldest prices and the last row contains the most recent. Prices across a given row are assumed to occur at the same time for all columns, and each column is a price series of an individual asset.

TickTimes — Observation times associated with prices in TickSeries

vector

Observation times associated with the prices in `TickSeries`, returned as a NUMBOBS+1 length column vector of monotonically increasing observation times associated with the prices in `TickSeries`. The initial time is `StartTime`. For matrix and table `Data`, sequential observations occur at increments specified in `ReturnIntervals` and for `Data` timetables, sequential observations are derived from times and dates in `Data`.

Version History

Introduced before R2006a

R2022b: Support for negative price data

Behavior changed in R2022b

The `Data` input accepts negative prices.

Extended Capabilities

Tall Arrays

Calculate with arrays that have more rows than fit in memory.

This function supports input `Data` that is specified as a tall column vector, a tall table, or a tall timetable. For more information, see `tall` and “Tall Arrays”.

See Also

`tick2ret` | `datetime` | `timetable` | `table`

Topics

“Use Timetables in Finance” on page 11-7

“Returns with Negative Prices” on page 2-32

rsindex

Relative Strength Index (RSI)

Syntax

```
index = rsindex(Data)
index = rsindex( ___,Name,Value)
```

Description

`index = rsindex(Data)` calculates the Relative Strength Index (RSI) from the series of closing stock prices.

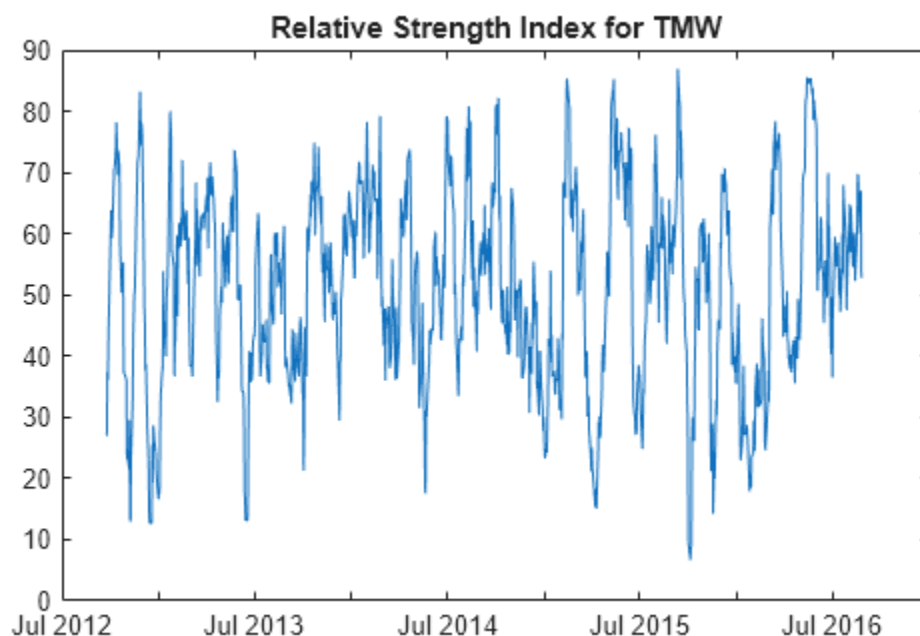
`index = rsindex( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Relative Strength Index for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
index = rsindex(TMW);
plot(index.Time,index.RelativeStrengthIndex)
title('Relative Strength Index for TMW')
```



Input Arguments

Data — Data with closing prices

matrix | table | timetable

Data with closing prices, specified as a matrix, table, or timetable. For matrix input, Data is M-by-1 with closing prices. Timetables and tables with M rows must contain a variable named 'Close' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `index = rsindex(TMW,'WindowSize',10)`

WindowSize — Moving window size for relative strength index

14 (default) | positive integer

Moving window size for relative strength index, specified as the comma-separated pair consisting of 'WindowSize' and a scalar positive integer.

Data Types: double

Output Arguments

index — Relative strength index

matrix | table | timetable

Relative strength index, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

Relative Strength Index

Relative strength index (RSI) is a momentum oscillator used in technical analysis to measure the speed and change of price movements.

RSI is primarily used to identify overbought or oversold conditions in a market, helping traders assess potential reversal points or continuation of trends.

RSI is calculated by dividing the average of the gains by the average of the losses within a specified period.

$$RS = (\text{average gains}) / (\text{average losses})$$

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Murphy, John J. *Technical Analysis of the Futures Market*. New York Institute of Finance, 1986, pp. 295-302.

See Also

timetable | table | negvolidx | posvolidx

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

selectreturn

Portfolio configurations from 3-D efficient frontier

Syntax

```
PortConfigs = selectreturn(AllMean,AllCovariance,Target)
```

Description

`PortConfigs = selectreturn(AllMean,AllCovariance,Target)` returns the portfolio configurations (`PortConfigs`) for a target return given the average return and covariance for a rolling efficient frontier.

Input Arguments

AllMean — Expected asset returns used to generate each curve on surface

vector

Expected asset returns used to generate each curve on the surface, specified as a number of curves (NCURVES-by-1 cell array), where each element is a 1-by-NASSETS (number of assets) vector

Data Types: double

AllCovariance — Covariance matrix used to generate each curve on surface

cell array

Covariance matrix used to generate each curve on the surface, specified as an NCURVES-by-1 cell array where each element is an NASSETS-by-NASSETS vector.

Data Types: cell

Target — Target return value for each curve in the frontier

numeric

Target return value for each curve in the frontier, specified as a numeric.

Data Types: double

Output Arguments

PortConfigs — Asset allocation weights needed to obtain target rate of return

matrix

Asset allocation weights needed to obtain the target rate of return, returned as an NASSETS-by-NCURVES matrix.

Version History

Introduced before R2006a

See Also

targetreturn | frontier

Topics

"Portfolio Construction Examples" on page 3-5

"Portfolio Optimization Functions" on page 3-3

setAssetList

Set up list of identifiers for assets

Syntax

```
obj = setAssetList(obj,AssetList)
obj = setAssetList(obj,'asset1','asset2',asset3',...)
```

Description

`obj = setAssetList(obj,AssetList)` sets up the list of identifiers for assets for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setAssetList(obj,'asset1','asset2',asset3',...)` sets up a list of asset identifiers, specified as a comma-separated list of character vectors, a cell array of character vectors, or string array where each character vector or string is an asset identifier.

Note

- If an asset list is entered as an input, this function overwrites an existing asset list in the object if one exists.
 - If no asset list is entered as an input, three actions can occur:
 - If `NumAssets` is nonempty and `AssetList` is empty, `AssetList` becomes a numbered list of assets with default names according to the hidden property in `defaultforAssetList` ('Asset').
 - If `NumAssets` is nonempty and `AssetList` is nonempty, nothing happens.
 - If `NumAssets` is empty and `AssetList` is empty, the default `NumAssets = 1` is set and a default asset list is created ('Asset1').
-

Examples

Create a Default List of Asset Names with Three Assets for a Portfolio Object

Create a default list of asset names with three assets.

```
p = Portfolio('NumAssets',3);
p = setAssetList(p);
disp(p.AssetList);

{'Asset1'}    {'Asset2'}    {'Asset3'}
```

Create an Explicitly Named List of Asset Names with Three Assets for a Portfolio Object

Create a list of asset names for three equities AGG, EEM, and VEU.

```
p = Portfolio;
p = setAssetList(p, 'AGG', 'EEM', 'VEU');
disp(p.AssetList);

{'AGG'}    {'EEM'}    {'VEU'}
```

Create a Default List of Asset Names with Three Assets for a PortfolioCvAR Object

Create a default list of asset names with three assets.

```
p = PortfolioCvAR('NumAssets',3);
p = setAssetList(p);
disp(p.AssetList);

{'Asset1'}    {'Asset2'}    {'Asset3'}
```

Create an Explicitly Named List of Asset Names with Three Assets for a PortfolioCvAR Object

Create a list of asset names for three equities AGG, EEM, and VEU.

```
p = PortfolioCvAR;
p = setAssetList(p, 'AGG', 'EEM', 'VEU');
disp(p.AssetList);

{'AGG'}    {'EEM'}    {'VEU'}
```

Create a Default List of Asset Names with Three Assets for a PortfolioMAD Object

Create a default list of asset names with three assets.

```
p = PortfolioMAD('NumAssets',3);
p = setAssetList(p);
disp(p.AssetList);

{'Asset1'}    {'Asset2'}    {'Asset3'}
```

Create an Explicitly Named List of Asset Names with Three Assets for a PortfolioMAD Object

Create a list of asset names for three equities AGG, EEM, and VEU.

```
p = PortfolioMAD;  
p = setAssetList(p, 'AGG', 'EEM', 'VEU');  
disp(p.AssetList);  
  
{'AGG'} {'EEM'} {'VEU'}
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

AssetList — List of assets

character vector | cell array of character vectors | string array

List of assets, specified using a character vector, cell array of character vectors, or string array where each character vector or string is an asset identifier.

Data Types: `char` | `cell` | `string`

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

The underlying object (`obj`) has a number of public hidden properties to format the asset list:

- `defaultforAssetList` — Default name for assets ('Asset'). Change this name to create default asset names such as 'ETF', 'Bond'.
- `sortAssetList` — Reserved for future implementation.
- `uppercaseAssetList` — If `true`, make all asset identifiers uppercase character vectors. Otherwise do nothing. Default is `false`.

Tips

- You can also use dot notation to set up list of identifiers for assets.

```
obj = obj.setAssetList(AssetList);
```

- To clear an `AssetList`, call this function with `[]` or `{[]}`.

Version History

Introduced in R2011a

See Also

`estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk` |
`estimateFrontierLimits`

Topics

“Common Operations on the Portfolio Object” on page 4-33

“Common Operations on the PortfolioCVaR Object” on page 5-29

“Common Operations on the PortfolioMAD Object” on page 6-29

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

setAssetMoments

Set moments (mean and covariance) of asset returns for Portfolio object

Syntax

```
obj = setAssetMoments(obj, AssetMean)
obj = setAssetMoments(obj, AssetMean, AssetCovar, NumAssets)
```

Description

`obj = setAssetMoments(obj, AssetMean)` obtains mean and covariance of asset returns for a Portfolio object. For details on the workflow, see “Portfolio Object Workflow” on page 4-18.

`obj = setAssetMoments(obj, AssetMean, AssetCovar, NumAssets)` obtains mean and covariance of asset returns for a Portfolio object with additional options for `AssetCovar` and `NumAssets`.

Examples

Set Asset Moments for a Portfolio Object

Set the asset moment properties, given the mean and covariance of asset returns in the variables `m` and `C`.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

p = Portfolio;
p = setAssetMoments(p, m, C);
[assetmean, assetcovar] = getAssetMoments(p)
```

```
assetmean = 4×1
```

```
0.0042
0.0083
0.0100
0.0150
```

```
assetcovar = 4×4
```

```
0.0005    0.0003    0.0002         0
0.0003    0.0024    0.0017    0.0010
0.0002    0.0017    0.0048    0.0028
         0    0.0010    0.0028    0.0102
```


Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object. For more information on creating a portfolio object, see

- `Portfolio`

Data Types: `object`

AssetMean — Mean of asset returns

vector

Mean of asset returns, specified as a vector.

Note If `AssetMean` is a scalar and the number of assets is known, scalar expansion occurs. If the number of assets cannot be determined, this method assumes that `NumAssets = 1`.

Data Types: `double`

AssetCovar — Covariance of asset returns

symmetric positive semidefinite matrix

Covariance of asset returns, specified as a symmetric positive semidefinite matrix.

Note

- If `AssetCovar` is a scalar and the number of assets is known, a diagonal matrix is formed with the scalar value along the diagonals. If it is not possible to determine the number of assets, this method assumes that `NumAssets = 1`.
 - If `AssetCovar` is a vector, a diagonal matrix is formed with the vector along the diagonal.
 - If `AssetCovar` is not a symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix for a correlation matrix.
-

Data Types: `double`

NumAssets — Number of assets

integer

Number of assets, specified as an integer.

Note If `NumAssets` is not already set in the object, `NumAssets` can be entered to resolve array expansions with `AssetMean` or `AssetCovar`.

Data Types: `double`

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio` object. For more information on creating a portfolio object, see

- `Portfolio`

Tips

- You can also use dot notation to set moments (mean and covariance) of the asset returns.

```
obj = obj.setAssetMoments(obj, AssetMean, AssetCovar, NumAssets);
```

- To clear `NumAssets` and `AssetCovar`, use this function to set these respective inputs to `[]`.

Version History

Introduced in R2011a

See Also

`estimateAssetMoments` | `estimateFrontierByRisk` | `nearcorr` | `covarianceShrinkage` | `covarianceDenoising`

Topics

“Asset Returns and Moments of Asset Returns Using Portfolio Object” on page 4-42

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization Theory” on page 4-4

setBounds

Set up bounds for portfolio weights for portfolio

Syntax

```
obj = setBounds(obj, LowerBound)
obj = setBounds( ____, Name, Value)

obj = setBounds(obj, LowerBound, UpperBound)
obj = setBounds( ____, Name, Value)
```

Description

`obj = setBounds(obj, LowerBound)` sets up bounds for portfolio weights for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setBounds( ____, Name, Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax, including `BoundType` as 'Simple' or 'Conditional'.

`obj = setBounds(obj, LowerBound, UpperBound)` sets up bounds for portfolio weights for portfolio objects with an additional option for `UpperBound`.

Given bound constraints `LowerBound` and `UpperBound` and 'Simple' `BoundType`, every weight in a portfolio `Port` must satisfy the following:

$$\text{LowerBound} \leq \text{Port} \leq \text{UpperBound}$$

Given bound constraints `LowerBound` and `UpperBound`, and 'Conditional' `BoundType`, every weight in a portfolio `Port` must satisfy the following:

$$\text{Port} = 0 \text{ or } \text{LowerBound} \leq \text{Port} \leq \text{UpperBound}$$

`obj = setBounds( ____, Name, Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax, including `BoundType` as 'Simple' or 'Conditional'.

Examples

Set Bound Constraints for a Portfolio Object

Suppose you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. Use `setBounds` to set the bound constraints for a balanced fund. Note that this sets default 'Simple' `BoundType`, which enforces $0.5 \leq x_1 \leq 0.75$, $0.25 \leq x_2 \leq 0.5$.

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];
```

```
p = Portfolio;
p = setBounds(p, lb, ub);
disp(p.NumAssets);
```

```
2
```

```
disp(p.LowerBound);
```

```
0.5000
0.2500
```

```
disp(p.UpperBound);
```

```
0.7500
0.5000
```

Set Bound Constraints to Define 'Conditional' BoundType Constraints for a Portfolio Object

Suppose you have the mean and covariance of the asset returns for a three asset portfolio:

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
```

The following uses `setBounds` with 'Conditional' BoundType (semicontinuous) constraints to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i = 1, \dots, \text{NumAssets}$.

```
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3)
```

```
p =
Portfolio with properties:
```

```
BuyCost: []
SellCost: []
RiskFreeRate: []
AssetMean: [3×1 double]
AssetCovar: [3×3 double]
TrackingError: []
TrackingPort: []
Turnover: []
BuyTurnover: []
SellTurnover: []
Name: []
NumAssets: 3
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
LowerBound: [3×1 double]
UpperBound: [3×1 double]
```

```

        LowerBudget: []
        UpperBudget: []
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
            GroupA: []
            GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: [3×1 categorical]

```

```
disp(p.LowerBound);
```

```

0.0200
0.0200
0.0200

```

```
disp(p.UpperBound);
```

```

0.5000
0.5000
0.5000

```

```
disp(p.BoundType);
```

```

conditional
conditional
conditional

```

Set Bound Constraints to Define Mixed BoundTypes for a Portfolio Object

Suppose you have the mean and covariance of the asset returns for a three asset portfolio:

```

AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

```

The following uses `setBounds` with both 'Simple' and 'Conditional' BoundType constraints for all $i = 1, \dots, \text{NumAssets}$.

```

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setBounds(p, 0.1, 0.5, 'BoundType', ["simple"; "conditional"; "conditional"])

p =
    Portfolio with properties:

```

```

        BuyCost: []
        SellCost: []
    RiskFreeRate: []
        AssetMean: [3×1 double]
        AssetCovar: [3×3 double]

```

```
TrackingError: []
TrackingPort: []
  Turnover: []
  BuyTurnover: []
  SellTurnover: []
  Name: []
  NumAssets: 3
  AssetList: []
  InitPort: []
  AInequality: []
  bInequality: []
  AEquality: []
  bEquality: []
  LowerBound: [3×1 double]
  UpperBound: [3×1 double]
  LowerBudget: []
  UpperBudget: []
  GroupMatrix: []
  LowerGroup: []
  UpperGroup: []
  GroupA: []
  GroupB: []
  LowerRatio: []
  UpperRatio: []
  MinNumAssets: []
  MaxNumAssets: []
  ConditionalBudgetThreshold: []
  ConditionalUpperBudget: []
  BoundType: [3×1 categorical]
```

```
disp(p.LowerBound);
```

```
0.1000
0.1000
0.1000
```

```
disp(p.UpperBound);
```

```
0.5000
0.5000
0.5000
```

```
disp(p.BoundType);
```

```
simple
conditional
conditional
```

You can use supply lower and upper bounds as vectors, which defines different values for each asset. The following have $-0.8 \leq x_1 \leq 0.2$; $x_2 = 0$ or $0.1 \leq x_2 \leq 0.5$; $x_3 = 0$ or $0.1 \leq x_3 \leq 0.5$. Note that as 'Simple' BoundType, the assets can be held as short or long positions. However, when using 'Conditional' BoundType, the assets can only be long positions.

```
p = setBounds(p, [-0.8, 0.1, 0.1], [-0.2,0.5,0.5], 'BoundType',["simple"; "conditional"; "conditional"]);
disp(p.LowerBound);
```

```

-0.8000
 0.1000
 0.1000

disp(p.UpperBound);

-0.2000
 0.5000
 0.5000

disp(p.BoundType);

simple
conditional
conditional

```

Set MinNumAssets Constraint for a Portfolio Object

Set the minimum cardinality constraint for a three-asset portfolio for which you have the mean and covariance values of the asset returns.

```

AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);

```

When working with a `Portfolio` object, the `setMinNumAssets` function enables you to set up limits on the number of assets invested. These limits are also known as cardinality constraints. When managing a portfolio, it is common that you want to invest in at least a certain number of assets. In addition, you should also clearly define the weight requirement for each invested asset. You can do this using `setBounds` with a 'Conditional' `BoundType`. If you do not specify a 'Conditional' `BoundType`, the optimizer cannot understand which assets are invested assets and cannot formulate the `MinNumAssets` constraint.

The following example specifies that at least two assets should be invested and the investments should be greater than 16%.

```

p = setMinNumAssets(p, 2, []);
p = setBounds(p, 0.16, 'BoundType', 'conditional');

```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```

pwgt = estimateFrontierByReturn(p,[ 0.008, 0.01 ])

pwgt = 3×2

    0.2861    0.3967
    0.5001    0.2437
    0.2138    0.3595

```

Set Bound Constraints for a PortfolioCVaR Object

Suppose you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. To set the bound constraints for a balanced fund.

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];

p = PortfolioCVaR;
p = setBounds(p, lb, ub);
disp(p.NumAssets);
```

2

```
disp(p.LowerBound);
```

```
0.5000
0.2500
```

```
disp(p.UpperBound);
```

```
0.7500
0.5000
```

Set Bound Constraints to Define 'Conditional' BoundType Constraints for a PortfolioCVaR Object

Suppose you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. To set the bound constraints for a balanced fund with semicontinuous constraints, use `setBounds` with 'Conditional' BoundType constraints to set $x_i = 0.25$ or $0.5 \leq x_i \leq 0.5$ or 0.75 for all $i = 1, \dots, \text{NumAssets}$.

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];

p = PortfolioCVaR;
p = setBounds(p, lb, ub, 'BoundType', ["conditional"; "conditional"]);
disp(p.NumAssets);
```

2

```
disp(p.LowerBound);
```

```
0.5000
0.2500
```

```
disp(p.UpperBound);
```

```
0.7500
0.5000
```


Set Bound Constraints for a PortfolioMAD Object

Suppose you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. To set the bound constraints for a balanced fund.

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];

p = PortfolioMAD;
p = setBounds(p, lb, ub);
disp(p.NumAssets);
```

2

```
disp(p.LowerBound);
```

```
0.5000
0.2500
```

```
disp(p.UpperBound);
```

```
0.7500
0.5000
```

Set Bound Constraints to Define 'Conditional' BoundType Constraints for a PortfolioMAD Object

Suppose you have a balanced fund with stocks that can range from 50% to 75% of your portfolio and bonds that can range from 25% to 50% of your portfolio. To set the bound constraints for a balanced fund with semicontinuous constraints, use `setBounds` with 'Conditional' `BoundType` constraints to set $x_i = 0.25$ or $0.5 \leq x_i \leq 0.5$ or 0.75 for all $i = 1, \dots, \text{NumAssets}$.

```
lb = [ 0.5; 0.25 ];
ub = [ 0.75; 0.5 ];

p = PortfolioMAD;
p = setBounds(p, lb, ub, 'BoundType', ["conditional"; "conditional"]);
disp(p.NumAssets);
```

2

```
disp(p.LowerBound);
```

```
0.5000
0.2500
```

```
disp(p.UpperBound);
```

```
0.7500
0.5000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

LowerBound — Lower-bound weight for each asset

scalar | vector

Lower-bound weight for each asset, specified as a scalar or vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note

- If either `LowerBound` or `UpperBound` are input as empties with `[]`, the corresponding attributes in the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object are cleared and set to `[]`.
 - If `LowerBound` or `UpperBound` are specified as scalars and `NumAssets` exists or can be computed, then they undergo scalar expansion. The default value for `NumAssets` is 1.
 - If both `LowerBound` and `UpperBound` exist and they are not ordered correctly, the `setBounds` function switches bounds if necessary.
 - If `'Conditional' BoundType` is specified, the `LowerBound` cannot be a negative value.
-

Data Types: `double`

UpperBound — Upper-bound weight for each asset

scalar | vector

(Optional) Upper-bound weight for each asset, specified as a scalar or vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note

- If either `LowerBound` or `UpperBound` are input as empties with `[]`, the corresponding attributes in the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object are cleared and set to `[]`.
 - If `LowerBound` or `UpperBound` are specified as scalars and `NumAssets` exists or can be computed, then they undergo scalar expansion. The default value for `NumAssets` is 1.
 - If both `LowerBound` and `UpperBound` exist and they are not ordered correctly, the `setBounds` function switches bounds if necessary.
 - If `'Conditional' BoundType` is specified, the `UpperBound` cannot be a negative value.
-

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `obj = setBounds(p,0.02,'BoundType','Conditional');`

BoundType — Type of bounds for each asset weight

'Simple' (default) | character vector with value 'Simple' or 'Conditional' | string with value "Simple" or "Conditional" | cell array of character vectors with values 'Simple' or 'Conditional' | string array with values "Simple" or "Conditional"

Type of bounds for each asset weight, specified as the comma-separated pair consisting of 'BoundType' and a scalar character vector or string with a value of 'Simple' or 'Conditional' or a cell array of character vectors with values of 'Simple' or 'Conditional'.

- 'Simple' is $\text{LowerBound} \leq \text{AssetWeight} \leq \text{UpperBound}$.
- 'Conditional' is $\text{LowerBound} \leq \text{AssetWeight} \leq \text{UpperBound}$ or $\text{AssetWeight} = 0$.

Warning If you specify the Bound range to be inclusive of zero (using either a 'Simple' or 'Conditional' BoundType), when you use `setMinNumAssets` to specify the `MinNumAssets` constraint, and then use one of the `estimate` functions, it is ambiguous for the optimizer to define the minimum requirement for an allocated asset. In this case, the optimizer considers that an asset with zero weight is a valid allocated asset and the optimization proceeds, but with the warning that the allocation has less than the `MinNumAssets` required. For more information, see "Troubleshooting for Setting 'Conditional' BoundType, `MinNumAssets`, and `MaxNumAssets` Constraints" on page 4-148.

Data Types: char | cell | string

NumAssets — Number of assets in portfolio

1 (default) | scalar numeric

Number of assets in portfolio, specified as the comma-separated pair consisting of 'NumAssets' and a scalar numeric value.

Note `NumAssets` cannot be used to change the dimension of a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object.

- If either `LowerBound` or `UpperBound` are input as empties with `[]`, the corresponding attributes in the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object are cleared and set to `[]`.
 - If `LowerBound` or `UpperBound` are specified as scalars and `NumAssets` exists or can be imputed, then they undergo scalar expansion. The default value for `NumAssets` is 1.
 - If both `LowerBound` and `UpperBound` exist and they are not ordered correctly, the `setBounds` function switches bounds if necessary.
-

Data Types: double

Output Arguments

obj — Updated portfolio object
object

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object.

More About

Upper and Lower Bounds

Upper and lower bounds in portfolio optimization are critical constraints that help define the limits of asset allocation, ensuring that the resulting portfolio aligns with the investor's goals for risk tolerance.

Tips

You can also use dot notation to set up the bounds for portfolio weights.

```
obj = obj.setBounds(LowerBound, UpperBound, Name, Value);
```

If any of `LowerBound`, `UpperBound`, or `BoundType` are input as empties with `[]`, the corresponding attributes in the portfolio object are cleared and set to `[]`. If `BoundType` is cleared as `[]`, the bound type defaults to `'Simple'`.

```
p = setBounds(p, LowerBound, [], 'BoundType', []);
```

To reset a portfolio object to be a continuous problem, run the following:

```
p = setMinNumAssets(p, [], []);  
p = setBounds(p, p.LowerBound, p.UpperBound, 'BoundType', 'Simple');
```

Version History

Introduced in R2011a

See Also

`getBounds` | `setMinNumAssets` | `setSolverMINLP` | `estimateAssetMoments` |
`estimateFrontier` | `estimateFrontierByReturn` | `estimateFrontierByRisk` |
`estimateFrontierLimits` | `estimateMaxSharpeRatio` | `estimatePortSharpeRatio` |
`estimatePortMoments` | `estimatePortReturn` | `estimatePortRisk`

Topics

“Working with 'Simple' Bound Constraints Using Portfolio Object” on page 4-62

“Working with 'Simple' Bound Constraints Using PortfolioCVaR Object” on page 5-55

“Working with 'Simple' Bound Constraints Using PortfolioMAD Object” on page 6-54

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioCVaR Objects” on page 5-72

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects” on page 6-71

“Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190

“Adding Constraints to Satisfy UCITS Directive” on page 4-457

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setBudget

Set up budget constraints for portfolio

Syntax

```
obj = setBudget(obj,LowerBudget)
obj = setBudget(obj,LowerBudget,UpperBudget)
```

Description

`obj = setBudget(obj,LowerBudget)` sets up budget constraints for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Note `setConditionalBudget` supports the Undertakings for Collective Investment in Transferable Securities (UCITS) directive for `Portfolio`, `PortfolioCVaR`, and `PortfolioMAD` objects. The UCITS asset allocation regulation states that funds can only invest up to 10% in a single issuer, and that investments in excess of 5% must not exceed 40% of the total portfolio. This is known as the 5/10/40 rule.

`obj = setBudget(obj,LowerBudget,UpperBudget)` sets up budget constraints for portfolio objects with an additional option for `UpperBudget`.

Examples

Set Budget Constraint for a Portfolio Object

Assume you have a fund that permits up to 10% leverage, which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `Portfolio` object `p`, set the budget constraint.

```
p = Portfolio;
p = setBudget(p, 1, 1.1);
disp(p.LowerBudget);
```

```
1
```

```
disp(p.UpperBudget);
```

```
1.1000
```

Set Budget Constraint for a PortfolioCVaR Object

Assume you have a fund that permits up to 10% leverage, which means that your portfolio can be from 100% to 110% invested in risky assets. Given a `CVaR` portfolio object `p`, set the budget constraint.

```
p = PortfolioCVaR;
p = setBudget(p, 1, 1.1);
disp(p.LowerBudget);
```

```
1
```

```
disp(p.UpperBudget);
```

```
1.1000
```

Set Budget Constraint for a PortfolioMAD Object

Assume you have a fund that permits up to 10% leverage, which means that your portfolio can be from 100% to 110% invested in risky assets. Given PortfolioMAD object p, set the budget constraint.

```
p = PortfolioMAD;
p = setBudget(p, 1, 1.1);
disp(p.LowerBudget);
```

```
1
```

```
disp(p.UpperBudget);
```

```
1.1000
```

Control the Allocation of a Risk-Free Asset Using setBudget

Define mean and covariance of risk asset returns.

```
m = [ 0.05; 0.1; 0.12; 0.18; ];
C = [ 0.0064 0.00408 0.00192 0,;
      0.00408 0.0289 0.0204 0.0119,;
      0.00192 0.0204 0.0576 0.0336,;
      0 0.0119 0.0336 0.1225];
```

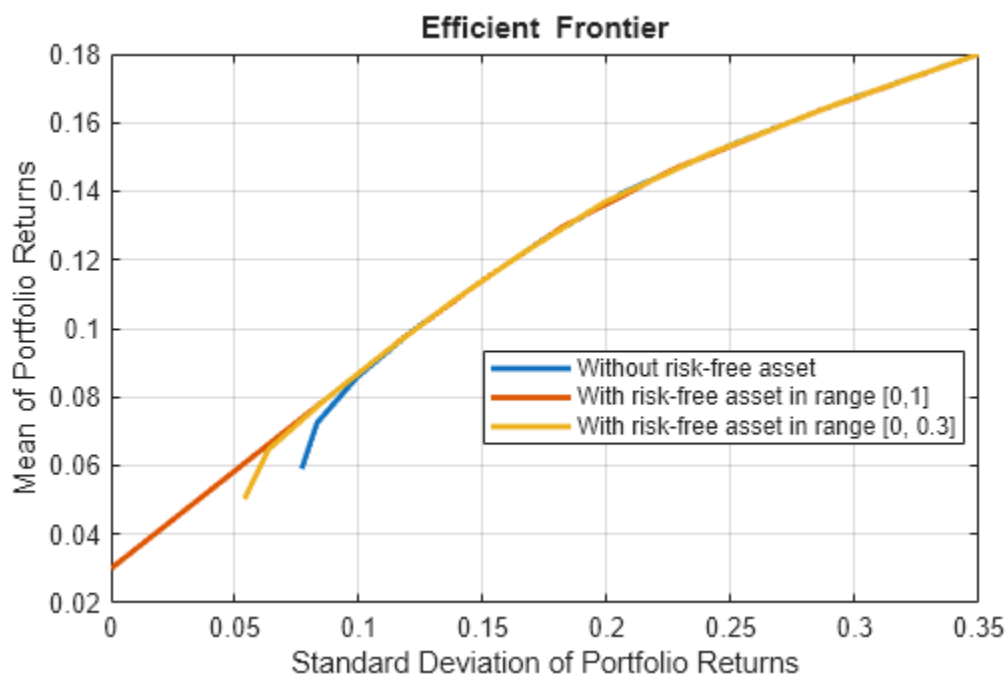
Create a Portfolio object defining the risk-free rate.

```
p = Portfolio('RiskFreeRate',0.03, 'assetmean', m, 'assetcovar', C, ...
'lowerbudget', 1, 'upperbudget', 1, 'lowerbound', 0);
```

Create multiple Portfolio objects with different budgets on risky assets. By defining the risky assets, you can control how much is invested in a risk-free asset.

```
p = setBudget(p, 1, 1); % allow 0% risk-free asset allocation, meaning fully invested in risky
p1 = setBudget(p, 0, 1); % allow 0 to 100% risk-free asset allocation
p2 = setBudget(p, 0.7, 1); % allow 0 to 30% risk-free asset allocation
```

```
plotFrontier(p); hold on;
plotFrontier(p1);hold on;
plotFrontier(p2);
legend('Without risk-free asset', 'With risk-free asset in range [0,1]', 'With risk-free asset in
```



`setBudget` defines the bound for total weights for the allocated risky assets, and the remaining is automatically the bound for a risk-free asset. Use `setBudget` to control the level of allowed allocation to a risk-free asset. For additional information on using `setBudget` with a risk-free asset, see “Leverage in Portfolio Optimization with a Risk-Free Asset” on page 4-217.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

LowerBudget — Lower-bound for budget constraint

scalar

Lower-bound for budget constraint, specified as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

Note Given bounds for a budget constraint in either `LowerBudget` or `UpperBudget`, budget constraints require any portfolio in `Port` to satisfy:

`LowerBudget <= sum(Port) <= UpperBudget`

One or both constraints may be specified. The usual budget constraint for a fully invested portfolio is to have `LowerBudget = UpperBudget = 1`. However, if the portfolio has allocations in cash, the budget constraints can be used to specify the cash constraints. For example, if the portfolio can hold between 0% and 10% in cash, the budget constraint would be set up with

```
obj = setBudget(obj, 0.9, 1)
```

Data Types: double

UpperBudget — Upper-bound for budget constraint

scalar

Upper-bound for budget constraint, specified as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note Given bounds for a budget constraint in either `LowerBudget` or `UpperBudget`, budget constraints require any portfolio in `Port` to satisfy:

$$\text{LowerBudget} \leq \text{sum}(\text{Port}) \leq \text{UpperBudget}$$

One or both constraints may be specified. The usual budget constraint for a fully invested portfolio is to have `LowerBudget = UpperBudget = 1`. However, if the portfolio has allocations in cash, the budget constraints can be used to specify the cash constraints. For example, if the portfolio can hold between 0% and 10% in cash, the budget constraint would be set up with

```
obj = setBudget(obj, 0.9, 1)
```

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Budget Constraints

Budget constraints are the limitations placed on the total amount of capital that can be allocated to different assets within a portfolio.

Budget constraints ensure that the total investment does not exceed the available capital and that the portfolio adheres to the investor's financial guidelines.

Tips

You can also use dot notation to set up the budget constraints.

```
obj = obj.setBudget(LowerBudget, UpperBudget);
```

Version History

Introduced in R2011a

See Also

`getBudget` | `setConditionalBudget`

Topics

“Working with Budget Constraints Using Portfolio Object” on page 4-65

“Working with Budget Constraints Using PortfolioCVaR Object” on page 5-58

“Working with Budget Constraints Using PortfolioMAD Object” on page 6-57

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Leverage in Portfolio Optimization with a Risk-Free Asset” on page 4-217

“Adding Constraints to Satisfy UCITS Directive” on page 4-457

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setConditionalBudget

Set up conditional budget constraints for portfolio

Syntax

```
obj = setConditionalBudget(obj,ConditionalBudgetThreshold,  
ConditionalUpperBudget)
```

Description

`obj = setConditionalBudget(obj,ConditionalBudgetThreshold,ConditionalUpperBudget)` sets up conditional budget constraints for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. If the weight of an asset exceeds the `ConditionalBudgetThreshold` value, the weight of that asset is added to the aggregate sum that is bound by the `ConditionalUpperBudget` value. For more information, see “Conditional Budget Constraints” on page 4-12 .

This constraint supports the Undertakings for Collective Investment in Transferable Securities (UCITS) Directive. The UCITS asset regulation states that any investments in excess of 5% must not exceed 40% of the total portfolio. This constraint is a conditional budget constraint. For more information, see “Undertakings for Collective Investment in Transferable Securities” on page 15-1452 and “Adding Constraints to Satisfy UCITS Directive” on page 4-457.

For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Add Conditional Budget Constraints

This example shows the workflow to add a conditional budget constraint for a portfolio optimization problem. The conditional budget constraint enforces a limit on the total proportion that you can invest in assets that pass a prescribed threshold.

In mathematical terms, the conditional budget constraint has the following form:

$$\sum_{i \in I} w_i \leq \beta$$

where $I = \{i | w_i \geq \alpha_i\}$ and w represents the asset weights.

Create Portfolio Object

To solve a portfolio problem with this constraint, start by creating a `Portfolio` object with default constraints. In addition to using a `Portfolio` object, you can also perform this workflow using a `PortfolioCVaR` or `PortfolioMAD` object.

```
% Load data  
load CAPMuniverse.mat
```

```
tol = 1e-8;
p = Portfolio('AssetList',Assets(1:12));
p = estimateAssetMoments(p,Data(:,1:12));
```

Using `setConditionalBudget`, set a conditional budget constraint that limits the aggregate weight that can be invested in assets that exceed 10% to 50%.

```
% Assets with weights above 10% must not exceed 50% of the total portfolio
p = setConditionalBudget(p,0.1,0.5);
```

At this point, the portfolio problem is unbounded and computing any optimal portfolio results in an error. Use `setBounds` to add bounds to the portfolio weights.

```
% Weights must be between -1 and 1
p = setBounds(p,-1,1);
```

Compute Efficient Frontier

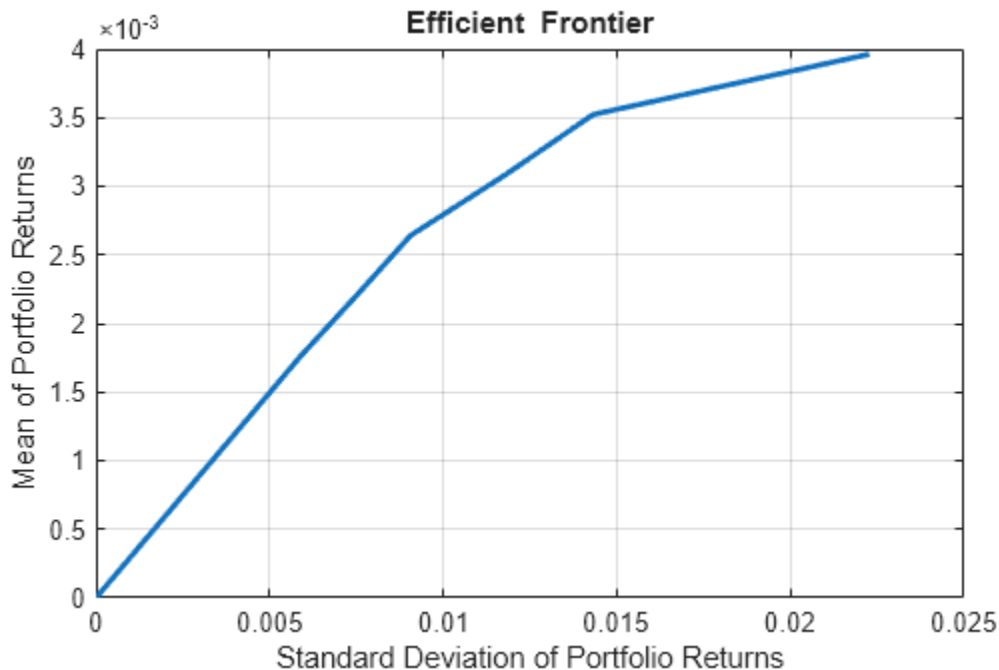
The most common workflow is to estimate a set of portfolios on the efficient frontier. The efficient frontier is the curve that shows the tradeoff between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level.

```
% Compute efficient frontier
w = estimateFrontier(p)
```

```
w = 12×10
```

0.0000	0.0368	0.0737	0.1105	0.1473	0.1979	0.2408	0.2686	0.5000	0.5000
0.0000	0.0008	0.0016	0.0023	0.0031	0.0082	0.0108	0.0762	0.0317	0.0317
0.0000	-0.0222	-0.0444	-0.0665	-0.0887	-0.1084	-0.1294	-0.2283	-0.2164	-1.0000
-0.0000	-0.0218	-0.0436	-0.0654	-0.0871	-0.1144	-0.1386	-0.4499	-0.4361	-1.0000
0.0000	-0.0087	-0.0175	-0.0262	-0.0350	-0.0448	-0.0539	-0.0260	-0.0300	0.0000
-0.0000	0.0398	0.0796	0.1194	0.1592	0.2122	0.2578	0.2314	0.1000	0.1000
0.0000	0.0366	0.0732	0.1098	0.1464	0.1000	0.1000	0.1000	0.1000	0.1000
-0.0000	-0.0282	-0.0565	-0.0847	-0.1129	-0.1424	-0.1712	-0.1291	-0.2717	0.0000
0.0000	0.0060	0.0120	0.0180	0.0240	0.0506	0.0658	0.1000	0.1000	0.1000
-0.0000	0.0111	0.0222	0.0333	0.0444	0.0746	0.0941	0.1000	0.1000	0.1000
0.0000	0.0131	0.0263	0.0394	0.0526	0.0706	0.0859	0.1000	0.1000	0.1000
0.0000	-0.0127	-0.0254	-0.0381	-0.0508	-0.0707	-0.0866	0.1000	0.1000	0.1000
:									

```
plotFrontier(p,w)
```



Check that none of the assets that exceed 10% go above 50%.

```
% Keep weights above 10% threshold
```

```
auxW = w;
```

```
auxW(w <= 0.1 + tol) = 0;
```

```
conditionalBudget = sum(auxW)
```

```
conditionalBudget = 1×10
```

```
0 0 0 0.3397 0.4529 0.4101 0.4986 0.5000 0.5000 0.5
```

Set Different Conditional Budget Thresholds for Different Assets

This example shows how to set a different conditional budget threshold for one of the assets in the portfolio. `setConditionalBudget` supports a vector input for the `ConditionalBudgetThreshold` argument, where each entry represents the conditional threshold for each asset. In this example, assume that the conditional budget threshold for all assets is 10% except for IBM. The conditional budget threshold for IBM is 15%.

To solve a portfolio problem with this constraint, start by creating a `PortfolioCVaR` object with default constraints. In addition to using a `PortfolioCVaR` object, you can also perform this workflow using a `Portfolio` or `PortfolioMAD` object.

```
% Load data
```

```
load CAPMuniverse.mat
```

```
tol = 1e-8;
```

```
startRow = find(~isnan(Data(:,6)),1);
```

```
p = PortfolioCVaR(AssetList=Assets(1:12), ...
```

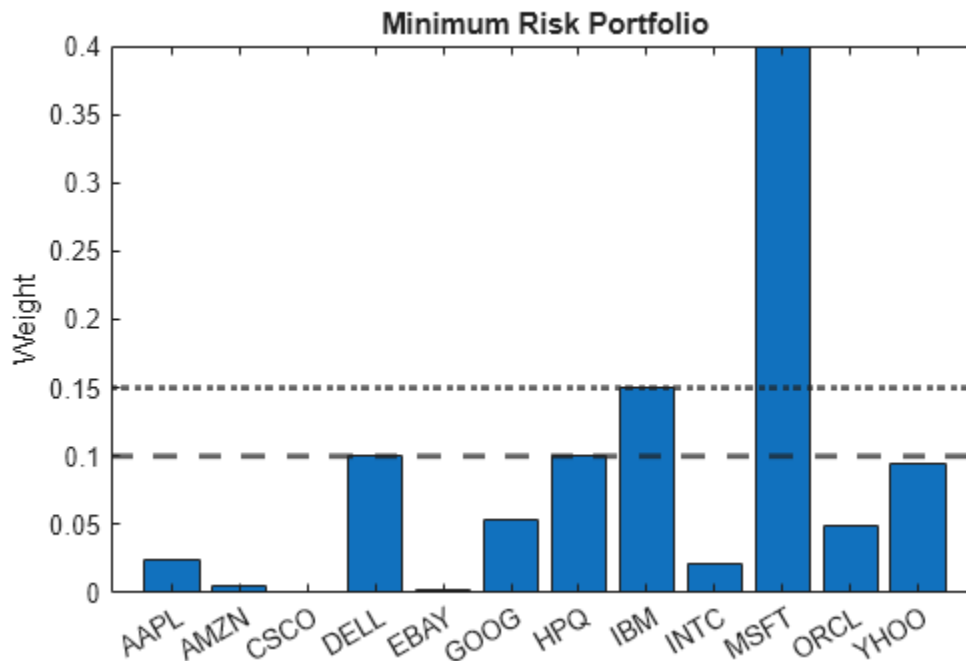
```
AssetScenarios=Data(startRow:end,1:12),ProbabilityLevel=0.95);
```

```
p = setDefaultConstraints(p);
```

The objective of the minimum variance problem is to find the weights (properties of the total investment) allocated to each asset in the portfolio that result in the lowest possible risk.

```
threshold = 0.1*ones(p.NumAssets,1);
threshold(strcmp('IBM',p.AssetList)) = 0.15;
p = setConditionalBudget(p,threshold,0.4);
```

```
% Solve the minimum risk problem
wMin = estimateFrontierLimits(p,'min');
figure
bar(Assets(1:12),wMin);
hold on
yline(0.1,'--',LineWidth=2)
yline(0.15,':',LineWidth=2)
title('Minimum Risk Portfolio')
ylabel('Weight')
```



All assets except IBM and MSFT do not exceed the 10% threshold, IBM does not exceed the 15% threshold, and the sum of all the assets above their conditional threshold (in this case only MSFT) is less than 40%.

Excluding One Asset from Conditional Budget Constraint

This example shows how to exclude individual assets from the conditional budget constraint.

Create a PortfolioMAD object. In addition to using a PortfolioMAD object, you can also perform this workflow using a Portfolio or PortfolioCVaR object.

```
% Load data
load CAPMuniverse.mat
```

```

tol = 1e-8;
% Create a PortfolioMAD object with default constraints
startRow = find(~isnan(Data(:,6)),1);
q = PortfolioMAD(AssetList=Assets(1:12), ...
    AssetScenarios=Data(startRow:end,1:12));

% Nonnegative and fully invested portfolios
q = setDefaultConstraints(q);

```

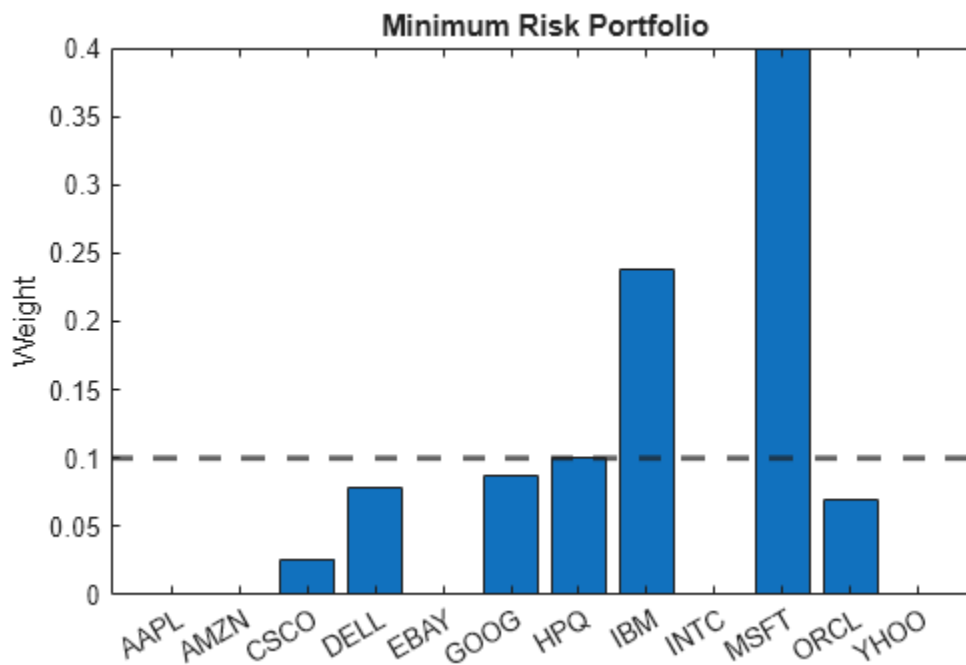
To exclude any asset from the conditional budget constraint, set the `ConditionalBudgetThreshold` of that asset to `Inf`. In this example, you can exclude IBM from the conditional budget constraint.

```

% Conditional budget
threshold = 0.1*ones(q.NumAssets,1);
threshold(strcmpi('IBM',q.AssetList)) = Inf;
q2 = setConditionalBudget(q,threshold,0.4);

% Solve minimum risk problem
wMin2 = estimateFrontierLimits(q2,'min');
figure
bar(Assets(1:12),wMin2);
hold on
yline(0.1,'--',LineWidth=2)
title('Minimum Risk Portfolio')
ylabel('Weight')

```



Working with Conditional Budget and Other Constraints

This example shows how to combine the conditional budget constraint with other types of constraints. Specifically, the example shows how to add semicontinuous bounds on the assets together with a conditional budget constraint, while ensuring that the portfolio is fully invested.

Create a mean-variance Portfolio object with default constraints.

```
% Load data
load CAPMuniverse.mat
tol = 1e-8;
p = Portfolio('AssetList',Assets(1:12));
p = estimateAssetMoments(p,Data(:,1:12));
```

Use `setBounds` to ensure that any asset that is chosen in the portfolio has a weight of at least 8% and at most 15%. Then, use `setBudget` to set the fully invested constraint.

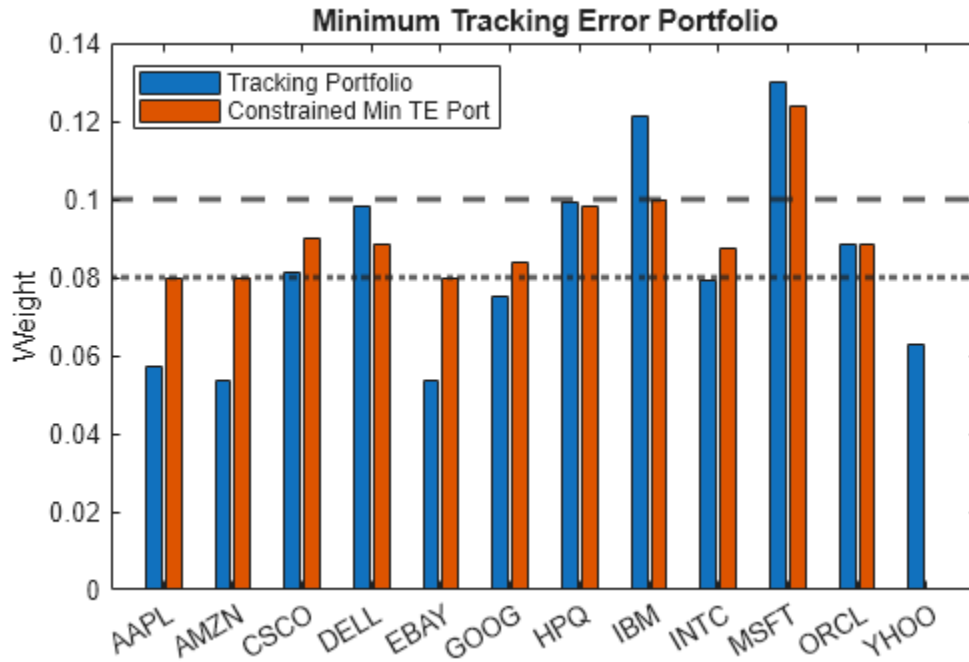
```
% Assets must be either 0 or between 8% and 15%
p = setBounds(p,0.08,0.15,BoundType="conditional");
```

```
% Assets must sum to 1
p = setBudget(p,1,1);
```

```
% Conditional budget
p = setConditionalBudget(p,0.1,0.2);
```

To find the minimum tracking error portfolio that satisfies all the previously specified constraints, use the risk parity portfolio as the benchmark portfolio.

```
% Solve minimum tracking error portfolio
trackingPort = riskBudgetingPortfolio(p.AssetCovar);
TE = @(w) (w-trackingPort)'*p.AssetCovar*(w-trackingPort);
wTE = estimateCustomObjectivePortfolio(p,TE);
figure
bar(Assets(1:12),[trackingPort wTE]);
hold on
yline(0.1,'--',LineWidth=2)
yline(0.08,':',LineWidth=2)
title('Minimum Tracking Error Portfolio')
ylabel('Weight')
legend('Tracking Portfolio','Constrained Min TE Port', ...
    Location='northwest')
```

All assets for the constrained portfolio are either zero or have at least a 0.08 weight, which means that the semicontinuous bound is satisfied. Furthermore, the sum of the assets above the 0.1 threshold is less than 0.2, and this result is also consistent with the conditional budget.

Input Arguments

obj — Object for portfolio

Portfolio object | PortfolioCVaR object | PortfolioMAD object

Object for portfolio, specified using the Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see:

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

ConditionalBudgetThreshold — Weights threshold above which asset is accounted for in conditional budget

scalar numeric | numeric vector

Weights threshold above which the asset is accounted for in the conditional budget, specified as a scalar or a NumAssets-by-1 vector for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj). The ConditionalBudgetThreshold value can be a positive or negative value. However, this value cannot be NaN or -Inf.

The ConditionalBudgetThreshold value indicates whether an asset should be accounted for in the aggregate sum or not. If the weight of an asset exceeds the ConditionalBudgetThreshold

value, the weight of that asset is added to the aggregate sum that is bound by the `ConditionalBudgetThreshold` value.

Note All assets for the `ConditionalBudgetThreshold` value are larger than or equal to the `ConditionalUpperBudget` value, then the conditional budget constraint is always feasible, so Financial Toolbox issues the following message: `Warning: Redundant conditional budget constraint.`

Data Types: double

ConditionalUpperBudget — Maximum aggregate weight of portfolio that can be invested in assets that exceed ConditionalBudgetThreshold

scalar numeric

Maximum aggregate weight of the portfolio that can be invested in assets that exceed the `ConditionalBudgetThreshold` value, specified as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`). The `ConditionalUpperBudget` value can be positive or negative. However, this value cannot be NaN or -Inf.

Data Types: double

Output Arguments

obj — Updated portfolio object

Updated `Portfolio` object | Updated `PortfolioCVaR` object | Updated `PortfolioMAD` object

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see:

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Undertakings for Collective Investment in Transferable Securities

Undertakings for Collective Investment in Transferable Securities (UCITS) is a regulatory framework of the European Union that creates a harmonized regime throughout Europe for the management and sale of mutual funds.

UCITS funds are subject to strict rules regarding investment diversification and risk management to protect investors against concentration risk. One of the core principles of UCITS is investment diversification. This principle reduces the risk associated with concentrating investments in a single issuer or asset class. By spreading investments across various assets, UCITS funds aim to mitigate individual asset volatility and reduce the overall risk profile of the portfolio.

The diversification rules for a UCITS fund state that a fund cannot hold more than 10% of its assets in securities or money market instruments issued by a single body. Additionally, any investments in excess of 5% must not exceed 40% of the total portfolio. This rule is known as the 5/10/40 rule.

Tips

You can also use dot notation to set up the conditional budget constraints.

```
obj = obj.setConditionalBudget(ConditionalBudgetThreshold,ConditionalUpperBudget);
```

Version History

Introduced in R2024b

See Also

setBounds | setBudget

Topics

“Adding Constraints to Satisfy UCITS Directive” on page 4-457

“Conditional Budget Constraints” on page 4-12

“Conditional Budget Constraints” on page 5-11

“Conditional Budget Constraints” on page 6-11

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setCosts

Set up proportional transaction costs for portfolio

Syntax

```
obj = setCosts(obj,BuyCost)
```

```
obj = setCosts(obj,BuyCost,SellCost,InitPort,NumAssets)
```

Description

`obj = setCosts(obj,BuyCost)` sets up proportional transaction costs for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setCosts(obj,BuyCost,SellCost,InitPort,NumAssets)` sets up proportional transaction costs for portfolio objects with additional options specified for `SellCost`, `InitPort`, and `NumAssets`.

Given proportional transaction costs and an initial portfolio in the variables `BuyCost`, `SellCost`, and `InitPort`, the transaction costs for any portfolio `Port` reduce expected portfolio return by:

$$\text{BuyCost}' * \max\{0, \text{Port} - \text{InitPort}\} + \text{SellCost}' * \max\{0, \text{InitPort} - \text{Port}\}$$

Examples

Set Up Transaction Costs for a Portfolio Object

Given a `Portfolio` object `p` with an initial portfolio already set, use the `setCosts` function to set up transaction costs.

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = Portfolio('InitPort', x0);
p = setCosts(p, bc, sc);
```

```
disp(p.NumAssets);
```

```
5
```

```
disp(p.BuyCost);
```

```
0.0013
0.0013
0.0013
0.0013
0.0013
```

```
disp(p.SellCost);
```

```

0.0013
0.0070
0.0013
0.0013
0.0024

disp(p.InitPort);

0.4000
0.2000
0.2000
0.1000
0.1000

```

Set Up Transaction Costs for a PortfolioCVaR Object

Given a CVaR portfolio object `p` with an initial portfolio already set, use the `setCosts` function to set up transaction costs.

```

bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];

```

```

p = PortfolioCVaR('InitPort', x0);
p = setCosts(p, bc, sc);

```

```

disp(p.NumAssets);

```

```

5

```

```

disp(p.BuyCost);

```

```

0.0013
0.0013
0.0013
0.0013
0.0013

```

```

disp(p.SellCost);

```

```

0.0013
0.0070
0.0013
0.0013
0.0024

```

```

disp(p.InitPort);

```

```

0.4000
0.2000
0.2000
0.1000
0.1000

```

Set Up Transaction Costs for a PortfolioMAD Object

Given PortfolioMAD object `p` with an initial portfolio already set, use the `setCosts` function to set up transaction costs.

```
bc = [ 0.00125; 0.00125; 0.00125; 0.00125; 0.00125 ];
sc = [ 0.00125; 0.007; 0.00125; 0.00125; 0.0024 ];
x0 = [ 0.4; 0.2; 0.2; 0.1; 0.1 ];
```

```
p = PortfolioMAD('InitPort', x0);
p = setCosts(p, bc, sc);
```

```
disp(p.NumAssets);
```

```
5
```

```
disp(p.BuyCost);
```

```
0.0013
0.0013
0.0013
0.0013
0.0013
```

```
disp(p.SellCost);
```

```
0.0013
0.0070
0.0013
0.0013
0.0024
```

```
disp(p.InitPort);
```

```
0.4000
0.2000
0.2000
0.1000
0.1000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

BuyCost — Proportional transaction cost to purchase each asset

vector

Proportional transaction cost to purchase each asset, specified as a vector for a **Portfolio**, **PortfolioCVar**, or **PortfolioMAD** input object (obj).

Note

- If **BuyCost**, **SellCost**, or **InitPort** are specified as scalars and **NumAssets** exists or can be imputed, then these values undergo scalar expansion. The default value for **NumAssets** is 1.
 - Transaction costs in **BuyCost** and **SellCost** are positive valued if they introduce a cost to trade. In some cases, they can be negative valued, which implies trade credits.
-

Data Types: double

SellCost — Proportional transaction cost to sell each asset

vector

Proportional transaction cost to sell each asset, specified as a vector for a **Portfolio**, **PortfolioCVar**, or **PortfolioMAD** input object (obj).

Note

- If **BuyCost**, **SellCost**, or **InitPort** are specified as scalars and **NumAssets** exists or can be imputed, then these values undergo scalar expansion. The default value for **NumAssets** is 1.
 - Transaction costs in **BuyCost** and **SellCost** are positive valued if they introduce a cost to trade. In some cases, they can be negative valued, which implies trade credits.
-

Data Types: double

InitPort — Initial or current portfolio weights

vector

Initial or current portfolio weights, specified as a vector for a **Portfolio**, **PortfolioCVar**, or **PortfolioMAD** input object (obj).

Note If no **InitPort** is specified, that value is assumed to be 0.

- If **BuyCost**, **SellCost**, or **InitPort** are specified as scalars and **NumAssets** exists or can be imputed, then these values undergo scalar expansion. The default value for **NumAssets** is 1.
 - Transaction costs in **BuyCost** and **SellCost** are positive valued if they introduce a cost to trade. In some cases, they can be negative valued, which implies trade credits.
-

Data Types: double

NumAssets — Number of assets in portfolio

scalar

Number of assets in portfolio, specified as a scalar for a **Portfolio**, **PortfolioCVar**, or **PortfolioMAD** input object (obj).

Note NumAssets cannot be used to change the dimension of a portfolio object.

- If BuyCost, SellCost, or InitPort are specified as scalars and NumAssets exists or can be imputed, then these values undergo scalar expansion. The default value for NumAssets is 1.
 - Transaction costs in BuyCost and SellCost are positive valued if they introduce a cost to trade. In some cases, they can be negative valued, which implies trade credits.
-

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

More About

Transaction Costs

Transaction costs are the expenses incurred when buying or selling securities within an investment portfolio.

Transaction costs can significantly impact the overall performance of an investment strategy, and understanding them is crucial for effective portfolio management.

Tips

- You can also use dot notation to set up proportional transaction costs.

```
obj = obj.setCosts(BuyCost, SellCost, InitPort, NumAssets);
```
- If BuyCost or SellCost are input as empties with [], the corresponding attributes in the portfolio object are cleared and set to []. If InitPort is set to empty with [], it will only be cleared and set to [] if BuyCost, SellCost, and Turnover are also empty. Otherwise, it is an error.

Version History

Introduced in R2011a

See Also

getCosts | setInitPort

Topics

"Working with Transaction Costs" on page 4-54

"Working with Transaction Costs" on page 5-47

"Working with Transaction Costs" on page 6-46

"Portfolio Analysis with Turnover Constraints" on page 4-211

"Portfolio Optimization Examples Using Financial Toolbox" on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setDefaultConstraints

Set up portfolio constraints with nonnegative weights that sum to 1

Syntax

```
obj = setDefaultConstraints(obj)
obj = setDefaultConstraints(obj,NumAssets)
```

Description

`obj = setDefaultConstraints(obj)` sets up portfolio constraints with nonnegative weights that sum to 1 for `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVar Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setDefaultConstraints(obj,NumAssets)` sets up portfolio constraints with nonnegative weights that sum to 1 with an additional option for `NumAssets`.

A "default" portfolio set has `LowerBound = 0` and `LowerBudget = UpperBudget = 1` such that a portfolio `Port` must satisfy `sum(Port) = 1` with `Port >= 0`.

Examples

Define Default Constraints for the Portfolio Object

Assuming you have 20 assets, you can define the "default" portfolio set.

```
p = Portfolio('NumAssets', 20);
p = setDefaultConstraints(p);
disp(p);
```

Portfolio with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
AssetMean: []
AssetCovar: []
TrackingError: []
TrackingPort: []
Turnover: []
BuyTurnover: []
SellTurnover: []
Name: []
NumAssets: 20
AssetList: []
InitPort: []
AInequality: []
bInequality: []
AEquality: []
bEquality: []
```

```

        LowerBound: [20×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: [20×1 categorical]

```

Define Default Constraints for the PortfolioCVaR Object

Assuming you have 20 assets, you can define the "default" portfolio set.

```

p = PortfolioCVaR('NumAssets', 20);
p = setDefaultConstraints(p);
disp(p);

```

PortfolioCVaR with properties:

```

        BuyCost: []
        SellCost: []
        RiskFreeRate: []
        ProbabilityLevel: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        NumScenarios: []
        Name: []
        NumAssets: 20
        AssetList: []
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [20×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []

```

```
        MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
        BoundType: [20×1 categorical]
```

Define Default Constraints for the PortfolioMAD Object

Assuming you have 20 assets, you can define the "default" portfolio set.

```
p = PortfolioMAD('NumAssets', 20);
p = setDefaultConstraints(p);
disp(p);
```

PortfolioMAD with properties:

```
        BuyCost: []
        SellCost: []
RiskFreeRate: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
NumScenarios: []
        Name: []
        NumAssets: 20
        AssetList: []
        InitPort: []
AInequality: []
bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [20×1 double]
        UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
        BoundType: [20×1 categorical]
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

NumAssets — Number of assets in portfolio

scalar

Number of assets in portfolio, specified as a scalar for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

Note NumAssets cannot be used to change the dimension of a portfolio object. The default for NumAssets is 1.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Tips

- You can also use dot notation to set up the default portfolio set.

```
obj = obj.setDefaultConstraints(NumAssets);
```

- This function does not modify any existing constraints in a portfolio object other than the bound (see setBounds) and budget (see setBudget) constraints. If an UpperBound constraint exists, it is cleared and set to [].

Version History

Introduced in R2011a

See Also

setBounds | getBounds | setBudget

Topics

“Setting Default Constraints for Portfolio Weights Using Portfolio Object” on page 4-58

“Setting Default Constraints for Portfolio Weights Using PortfolioCVaR Object” on page 5-51

“Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object” on page 6-50

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setEquality

Set up linear equality constraints for portfolio weights

Syntax

```
obj= setEquality(obj,AEquality,bEquality)
```

Description

`obj= setEquality(obj,AEquality,bEquality)` sets up linear equality constraints for portfolio weights for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Given linear equality constraint matrix `AEquality` and vector `bEquality`, every weight in a portfolio `Port` must satisfy the following:

$$AEquality * Port = bEquality$$

Examples

Set Linear Equality Constraints for a Portfolio Object

Suppose you have a portfolio of five assets, and you want to ensure that the first three assets are 50% of your portfolio. Given a `Portfolio` object `p`, set the linear equality constraints with the following.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio;
p = setEquality(p, A, b);

disp(p.NumAssets);

5

disp(p.AEquality);

1      1      1      0      0

disp(p.bEquality);

0.5000
```

To add additional Linear Equality constraints, use `addEquality`.

Set Linear Equality Constraints for a PortfolioCvAR Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are 50% of your portfolio. Given a PortfolioCvAR object *p*, set the linear equality constraints and obtain the values for AEquality and bEquality:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioCvAR;
p = setEquality(p, A, b);
disp(p.NumAssets);

5

disp(p.AEquality);

1      1      1      0      0

disp(p.bEquality);

0.5000
```

To add additional Linear Equality constraints, use `addEquality`.

Set Linear Equality Constraints for a PortfolioMAD Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are 50% of your portfolio. Given a PortfolioMAD object *p*, set the linear equality constraints and obtain the values for AEquality and bEquality:

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD;
p = setEquality(p, A, b);
[AEquality, bEquality] = getEquality(p)

AEquality = 1×5

1      1      1      0      0

bEquality =
0.5000
```

To add additional Linear Equality constraints, use `addEquality`.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCvAR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

AEquality — Matrix to form linear equality constraints

matrix

Matrix to form linear equality constraints, returned as a matrix for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

Note An error results if AEquality is empty and bEquality is nonempty.

Data Types: double

bEquality — Vector to form linear equality constraints

vector

Vector to form linear equality constraints, returned as a vector for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

Note An error results if AEquality is nonempty and bEquality is empty.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

More About

Equality Constraint

An equality constraint requires certain conditions to be met exactly, rather than allowing for a range of values.

Tips

- You can also use dot notation to set up linear equality constraints for portfolio weights.

```
obj = obj.setEquality(AEquality, bEquality);
```

- Linear equality constraints can be removed from a portfolio object by entering [] for each property you want to remove.

Version History

Introduced in R2011a

See Also

`addEquality` | `getEquality`

Topics

“Working with Linear Equality Constraints Using Portfolio Object” on page 4-75

“Working with Linear Equality Constraints Using PortfolioCVaR Object” on page 5-68

“Working with Linear Equality Constraints Using PortfolioMAD Object” on page 6-67

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setGroupRatio

Set up group ratio constraints for portfolio weights

Syntax

```
obj = setGroupRatio(obj,GroupA,GroupB,LowerRatio)
obj = setGroupRatio( ___,UpperRatio)
```

Description

`obj = setGroupRatio(obj,GroupA,GroupB,LowerRatio)` sets up group ratio constraints for portfolio weights for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setGroupRatio( ___,UpperRatio)` sets up group ratio constraints for portfolio weights for portfolio objects with an additional optional argument for `UpperRatio`.

Given base and comparison group matrices `GroupA` and `GroupB` and `LowerRatio` or `UpperRatio` bounds, group ratio constraints require any portfolio in `Port` to satisfy the following:

$$(\text{GroupB} * \text{Port}) .* \text{LowerRatio} \leq \text{GroupA} * \text{Port} \leq (\text{GroupB} * \text{Port}) .* \text{UpperRatio}$$

Caution This collection of constraints usually requires that portfolio weights be nonnegative and that the products `GroupA * Port` and `GroupB * Port` are always nonnegative. Although negative portfolio weights and non-Boolean group ratio matrices are supported, use with caution.

Examples

Set Group Ratio Constraints for a Portfolio Object

Suppose you want to ensure that the ratio of financial to nonfinancial companies in your portfolio never exceeds 50%. Assume you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). Group ratio constraints can be set with:

```
GA = [ true true true false false false ];    % financial companies
GB = [ false false false true true true ];    % nonfinancial companies
p = Portfolio;
p = setGroupRatio(p, GA, GB, [], 0.5);

disp(p.NumAssets);

6

disp(p.GroupA);

1    1    1    0    0    0

disp(p.GroupB);
```

```
0      0      0      1      1      1
disp(p.UpperRatio);
0.5000
```

To add additional Group ratio constraints, use `addGroupRatio`.

Set Group Ratio Constraints for a PortfolioCVaR Object

Suppose you want to ensure that the ratio of financial to nonfinancial companies in your portfolio never exceeds 50%. Assume you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). Group ratio constraints can be set with:

```
GA = [ true true true false false false ];    % financial companies
GB = [ false false false true true true ];    % nonfinancial companies
p = PortfolioCVaR;
p = setGroupRatio(p, GA, GB, [], 0.5);

disp(p.NumAssets);
6

disp(p.GroupA);
1      1      1      0      0      0

disp(p.GroupB);
0      0      0      1      1      1

disp(p.UpperRatio);
0.5000
```

To add additional Group ratio constraints, use `addGroupRatio`.

Set Group Ratio Constraints for a PortfolioMAD Object

Suppose you want to ensure that the ratio of financial to nonfinancial companies in your portfolio never exceeds 50%. Assume you have six assets with three financial companies (assets 1-3) and three nonfinancial companies (assets 4-6). Group ratio constraints can be set with:

```
GA = [ true true true false false false ];    % financial companies
GB = [ false false false true true true ];    % nonfinancial companies
p = PortfolioMAD;
p = setGroupRatio(p, GA, GB, [], 0.5);

disp(p.NumAssets);
6

disp(p.GroupA);
1      1      1      0      0      0
```

```
disp(p.GroupB);
    0    0    0    1    1    1
disp(p.UpperRatio);
    0.5000
```

To add additional Group ratio constraints, use `addGroupRatio`.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

GroupA — Matrix that forms base groups for comparison

matrix

Matrix that forms base groups for comparison, specified as a matrix for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note The group matrices `GroupA` and `GroupB` are usually indicators of membership in groups, which means that their elements are usually either 0 or 1. Because of this interpretation, `GroupA` and `GroupB` matrices can be either logical or numerical arrays.

Data Types: `double` | `logical`

GroupB — Matrix that forms comparison groups

matrix

Matrix that forms comparison groups, specified as a matrix `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note The group matrices `GroupA` and `GroupB` are usually indicators of membership in groups, which means that their elements are usually either 0 or 1. Because of this interpretation, `GroupA` and `GroupB` matrices can be either logical or numerical arrays.

Data Types: `double` | `logical`

LowerRatio — Lower bound for ratio of GroupB groups to GroupA groups

vector

Lower bound for ratio of GroupB groups to GroupA groups, specified as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If input is scalar, `LowerRatio` undergoes scalar expansion to be conformable with the group matrices.

Data Types: `double`

UpperRatio — Upper bound for ratio of GroupB groups to GroupA groups vector

Upper bound for ratio of GroupB groups to GroupA groups, specified as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If input is scalar, `UpperRatio` undergoes scalar expansion to be conformable with the group matrices.

Data Types: `double`

Output Arguments

obj — Updated portfolio object object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Group Ratio Constraints

Group ratio constraints specify that the total weight or allocation of assets within a particular group must satisfy certain conditions relative to the total portfolio.

Tips

- You can also use dot notation to set up group ratio constraints for portfolio weight.
`obj = obj.setGroupRatio(GroupA, GroupB, LowerRatio, UpperRatio);`
- To remove group ratio constraints, enter empty arrays for the corresponding arrays. To add to existing group ratio constraints, use `addGroupRatio`.

Version History Introduced in R2011a

See Also

[addGroupRatio](#) | [getGroupRatio](#)

Topics

[“Working with Group Ratio Constraints Using Portfolio Object” on page 4-72](#)

[“Working with Group Constraints Using PortfolioCVaR Object” on page 5-62](#)

[“Working with Group Constraints Using PortfolioMAD Object” on page 6-61](#)

[“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161](#)

[Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9](#)

[Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8](#)

[Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8](#)

setGroups

Set up group constraints for portfolio weights

Syntax

```
obj = setGroups(obj,GroupMatrix,LowerGroup)
obj = setGroups(obj,GroupMatrix,LowerGroup,UpperGroup)
```

Description

`obj = setGroups(obj,GroupMatrix,LowerGroup)` sets up group constraints for portfolio weights for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setGroups(obj,GroupMatrix,LowerGroup,UpperGroup)` sets up group constraints for portfolio weights for portfolio objects with an additional option specified for `UpperGroup`.

Given `GroupMatrix` and either `LowerGroup` or `UpperGroup`, a portfolio `Port` must satisfy the following:

$$\text{LowerGroup} \leq \text{GroupMatrix} * \text{Port} \leq \text{UpperGroup}$$

Examples

Set Group Constraints for a Portfolio Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 30% of your portfolio. Given a `Portfolio` object `p`, set the group constraints with the following.

```
G = [ true true true false false ];
p = Portfolio;
p = setGroups(p, G, [], 0.3);
```

```
disp(p.NumAssets);
```

```
5
```

```
disp(p.GroupMatrix);
```

```
1    1    1    0    0
```

```
disp(p.UpperGroup);
```

```
0.3000
```

To add more group constraints, see `addGroups`.

Set Group Constraints for a PortfolioCvAR Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 30% of your portfolio. Given a CVaR portfolio object `p`, set the group constraints with the following.

```
G = [ true true true false false ];
p = PortfolioCvAR;
p = setGroups(p, G, [], 0.3);

disp(p.NumAssets);

5

disp(p.GroupMatrix);

1      1      1      0      0

disp(p.UpperGroup);

0.3000
```

To add more group constraints, see `addGroups`.

Set Group Constraints for a PortfolioMAD Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets constitute at most 30% of your portfolio. Given PortfolioMAD object `p`, set the group constraints with the following.

```
G = [ true true true false false ];
p = PortfolioMAD;
p = setGroups(p, G, [], 0.3);

disp(p.NumAssets);

5

disp(p.GroupMatrix);

1      1      1      0      0

disp(p.UpperGroup);

0.3000
```

To add more group constraints, see `addGroups`.

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCvAR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

GroupMatrix — Group constraint matrix

logical or numeric matrix

Group constraint matrix, specified as a matrix for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note The group matrix `GroupMatrix` is usually an indicator of membership in groups, which means that its elements are usually either 0 or 1. Because of this interpretation, `GroupMatrix` can be either a logical or numerical matrix.

Data Types: double

LowerGroup — Lower bound for group constraints

vector

Lower bound for group constraints, specified as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If input is scalar, `LowerGroup` undergoes scalar expansion to be conformable with `GroupMatrix`.

Data Types: double

UpperGroup — Upper bound for group constraints

vector

Upper bound for group constraints, returned as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If input is scalar, `UpperGroup` undergoes scalar expansion to be conformable with `GroupMatrix`.

Data Types: double

Output Arguments**obj — Updated portfolio object**

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

More About

Group Constraints

Group constraints specify that the total weight or allocation of assets within a particular group must satisfy certain conditions relative to the total portfolio.

Group constraints help maintain diversification by preventing excessive concentration in specific asset groups. They also align the portfolio with the investor's risk tolerance and investment strategy.

Tips

- You can also use dot notation to set up group constraints for portfolio weights.
`obj = obj.setGroups(GroupMatrix, LowerGroup, UpperGroup);`
- To remove group constraints, enter empty arrays for the corresponding arrays. To add to existing group constraints, use `addGroups`.

Version History

Introduced in R2011a

See Also

`getGroups` | `addGroups`

Topics

"Working with Group Constraints Using Portfolio Object" on page 4-69

"Working with Group Constraints Using PortfolioCVaR Object" on page 5-62

"Working with Group Constraints Using PortfolioMAD Object" on page 6-61

"Constraint Specification Using a Portfolio Object" on page 3-19

"Portfolio Optimization Examples Using Financial Toolbox" on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setInequality

Set up linear inequality constraints for portfolio weights

Syntax

```
obj = setInequality(obj,AInequality,bInequality)
```

Description

`obj = setInequality(obj,AInequality,bInequality)` sets up linear inequality constraints for portfolio weights for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Given a linear inequality constraint matrix `AInequality` and vector `bInequality`, every weight in a portfolio `Port` must satisfy the following:

$$\text{AInequality} * \text{Port} \leq \text{bInequality}$$

Examples

Set Linear Inequality Constraints for a Portfolio Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. Given a `Portfolio` object `p`, set the linear inequality constraints with the following.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = Portfolio;
p = setInequality(p, A, b);

disp(p.NumAssets);

5

disp(p.AInequality);

1    1    1    0    0

disp(p.bInequality);

0.5000
```

To add additional Linear Inequality constraints, use `addInequality`.

Set Linear Inequality Constraints for a PortfolioCVaR Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. Given a CVaR portfolio object `p`, set the linear inequality constraints with the following.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioCVaR;
p = setInequality(p, A, b);

disp(p.NumAssets);

5

disp(p.AInequality);

1      1      1      0      0

disp(p.bInequality);

0.5000
```

To add additional Linear Inequality constraints, use `addInequality`.

Set Linear Inequality Constraints for a PortfolioMAD Object

Suppose you have a portfolio of five assets and you want to ensure that the first three assets are no more than 50% of your portfolio. Given PortfolioMAD object `p`, set the linear inequality constraints with the following.

```
A = [ 1 1 1 0 0 ];
b = 0.5;
p = PortfolioMAD;
p = setInequality(p, A, b);

disp(p.NumAssets);

5

disp(p.AInequality);

1      1      1      0      0

disp(p.bInequality);

0.5000
```

To add additional Linear Inequality constraints, use `addInequality`.

Input Arguments

obj — Object for portfolio
object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

AInequality — Matrix to form linear inequality constraints

`matrix`

Matrix to form linear inequality constraints, specified as a matrix for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note An error results if `AInequality` is empty and `bInequality` is nonempty.

Data Types: `double`

bInequality — Vector to form linear inequality constraints

`vector`

Vector to form linear inequality constraints, specified as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note An error results if `AInequality` is nonempty and `bInequality` is empty.

Data Types: `double`

Output Arguments

obj — Updated portfolio object

`object for portfolio`

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Inequality Constraint

An inequality constraint specifies a range of acceptable values rather than requiring an exact value.

Use inequality constraints to impose limits on certain parameters of the portfolio to ensure that the portfolio adheres to specific guidelines or preferences set by the investor or portfolio manager.

Tips

- You can also use dot notation to set up linear inequality constraints for portfolio weights.

```
obj = obj.setInequality(AInequality, bInequality);
```

- To remove inequality constraints, enter empty arguments. To add to existing inequality constraints, use `addInequality`.

Version History

Introduced in R2011a

See Also

`getInequality` | `addInequality`

Topics

“Working with Linear Inequality Constraints Using Portfolio Object” on page 4-78

“Working with Linear Inequality Constraints Using PortfolioCVaR Object” on page 5-70

“Working with Linear Inequality Constraints Using PortfolioMAD Object” on page 6-69

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setInitPort

Set up initial or current portfolio

Syntax

```
obj = setInitPort(obj,InitPort)
obj = setInitPort(obj,InitPort,NumAssets)
```

Description

`obj = setInitPort(obj,InitPort)` sets up initial or current portfolio for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setInitPort(obj,InitPort,NumAssets)` sets up initial or current portfolio for portfolio objects with an additional options specified for `NumAssets`.

Examples

Set the InitPort Property for a Portfolio Object

Given an initial portfolio in `x0`, use the `setInitPort` function to set the `InitPort` property.

```
p = Portfolio('NumAssets', 4);
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setInitPort(p, x0);
disp(p.InitPort);

    0.3000
    0.2000
    0.2000
         0
```

Set InitPort to Create an Equally-Weighted Portfolio of Four Assets for a Portfolio Object

Create an equally weighted portfolio of four assets using the `setInitPort` function.

```
p = Portfolio('NumAssets', 4);
p = setInitPort(p, 1/4, 4);
disp(p.InitPort);

    0.2500
    0.2500
    0.2500
    0.2500
```


Set the InitPort Property for a PortfolioCvAR Object

Given an initial portfolio in `x0`, use the `setInitPort` function to set the `InitPort` property.

```
p = PortfolioCvAR('NumAssets', 4);
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setInitPort(p, x0);
disp(p.InitPort);
```

```
0.3000
0.2000
0.2000
0
```

Set InitPort to Create an Equally-Weighted Portfolio of Four Assets for a PortfolioCvAR Object

Create an equally weighted portfolio of four assets using the `setInitPort` function.

```
p = PortfolioCvAR('NumAssets', 4);
p = setInitPort(p, 1/4, 4);
disp(p.InitPort);
```

```
0.2500
0.2500
0.2500
0.2500
```

Set the InitPort Property for a PortfolioMAD Object

Given an initial portfolio in `x0`, use the `setInitPort` function to set the `InitPort` property.

```
p = PortfolioMAD('NumAssets', 4);
x0 = [ 0.3; 0.2; 0.2; 0.0 ];
p = setInitPort(p, x0);
disp(p.InitPort);
```

```
0.3000
0.2000
0.2000
0
```

Set InitPort to Create an Equally-Weighted Portfolio of Four Assets for a PortfolioMAD Object

Create an equally weighted portfolio of four assets using the `setInitPort` function.

```
p = PortfolioMAD('NumAssets', 4);
p = setInitPort(p, 1/4, 4);
disp(p.InitPort);
```

```
0.2500
0.2500
0.2500
0.2500
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

InitPort — Initial or current portfolio weights

vector

Initial or current portfolio weights, specified as a vector for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If `InitPort` is specified as a scalar and `NumAssets` exists, then `InitPort` undergoes scalar expansion.

Data Types: `double`

NumAssets — Number of assets in portfolio

1 (default) | scalar

Number of assets in portfolio, specified as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If it is not possible to obtain a value for `NumAssets`, it is assumed that `NumAssets` is 1.

Data Types: `double`

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`

- PortfolioCVaR
- PortfolioMAD

Tips

- You can also use dot notation to set up an initial or current portfolio.
`obj = obj.setInitPort(InitPort, NumAssets);`
- To remove an initial portfolio, call this function with an empty argument `[]` for `InitPort`.

Version History

Introduced in R2011a

See Also

`setTurnover` | `setCosts`

Topics

“Common Operations on the Portfolio Object” on page 4-33

“Common Operations on the PortfolioCVaR Object” on page 5-29

“Common Operations on the PortfolioMAD Object” on page 6-29

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setMinMaxNumAssets

Set cardinality constraints on the number of assets invested in a portfolio

Syntax

```
obj = setMinMaxNumAssets(obj, MinNumAssets, MaxNumAssets)
```

Description

`obj = setMinMaxNumAssets(obj, MinNumAssets, MaxNumAssets)` sets cardinality constraints for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object.

`MinNumAssets` and `MaxNumAssets` are the minimum and maximum number of assets invested in the portfolio, respectively. The total number of allocated assets satisfying the Bound constraints is between `[MinNumAssets, MaxNumAssets]`. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

Examples

Set MaxNumAssets Constraint for a Portfolio Object

Set the maximum cardinality constraint for a three-asset portfolio for which you have the mean and covariance values of the asset returns.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up the limits on the number of assets invested. Use `setMinMaxNumAssets` to limit the total number of allocated assets to no more than two.

```
p = setMinMaxNumAssets(p, [], 2);
```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```
pwgt = estimateFrontierByReturn(p, [ 0.008, 0.01 ])
```

```
pwgt = 3×2
```

```
    0    0.0000
0.6101    0.3962
0.3899    0.6038
```

Set MinNumAssets Constraint for a Portfolio Object

Set the minimum cardinality constraint for a three-asset portfolio for which you have the mean and covariance values of the asset returns.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up limits on the number of assets invested. These limits are also known as cardinality constraints. When managing a portfolio, it is common that you want to invest in at least a certain number of assets. In addition, you should also clearly define the weight requirement for each invested asset. You can do this using `setBounds` with a 'Conditional' `BoundType`. If you do not specify a 'Conditional' `BoundType`, the optimizer cannot understand which assets are invested assets and cannot formulate the `MinNumAssets` constraint.

The following example specifies that at least two assets should be invested and the investments should be greater than 16%.

```
p = setMinMaxNumAssets(p, 2, []);
p = setBounds(p, 0.16, 'BoundType', 'conditional');
```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```
pwgt = estimateFrontierByReturn(p,[ 0.008, 0.01 ])
pwgt = 3×2
```

```
    0.2861    0.3967
    0.5001    0.2437
    0.2138    0.3595
```

Set 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints for a Portfolio Object

Set the minimum and maximum cardinality constraints and a 'Conditional' `BoundType` for a three-asset portfolio for which you have the mean and covariance values of the asset returns.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];

p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
p = setDefaultConstraints(p);
```

When working with a `Portfolio` object, the `setMinMaxNumAssets` function enables you to set up the limits on the number of assets invested. The following example specifies that exactly two assets

should be invested using `setMinMaxNumAssets` and the investment should be equally allocated among the two assets using `setBounds`.

```
p = setMinMaxNumAssets(p, 2, 2);
p = setBounds(p, 0.5, 0.5, 'BoundType', 'conditional');
```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```
pwgt = estimateFrontierByReturn(p,[ 0.008, 0.01 ])
```

```
pwgt = 3×2
```

```
      0      0.5000
0.5000      0
0.5000      0.5000
```

Set 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints for a PortfolioCVaR Object

Suppose you have a universe of 12 stocks where you want to find the optimal portfolios with targeted returns and you want to set semicontinuous and cardinality constraints for the portfolio.

```
load CAPMuniverse
p = PortfolioCVaR('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000, 'missingdata',true);
p = setProbabilityLevel(p, 0.80);
```

When working with a `PortfolioCVaR` object, the `setMinMaxNumAssets` function enables you to set up the limits on the number of assets invested. The following example specifies that a minimum of five assets and a maximum of 10 assets should be invested using `setMinMaxNumAssets` and the investments should be greater than 4% and less than 45% using `setBounds`.

```
p = setMinMaxNumAssets(p, 5, 10);
p = setBounds(p, 0.04, 0.45, 'BoundType', 'conditional');
```

Use `estimateFrontierByReturn` to estimate optimal portfolios with targeted portfolio returns.

```
pwgt = estimateFrontierByReturn(p,[ 0.00026, 0.00038 ])
```

```
pwgt = 12×2
```

```
0.0400      0.0400
      0      0
      0      0
      0      0
      0      0
0.0507      0.0786
0.0400      0.0400
0.0400      0.0400
      0      0
0.0400      0.0400
      0      0
      0      0
      :
```

Set 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints for a PortfolioMAD Object

Suppose you have a universe of 12 stocks where you want to find the optimal portfolios with targeted returns and you want to set semicontinuous and cardinality constraints for the portfolio.

```
load CAPMuniverse
p = PortfolioMAD('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12), 20000 , 'missingdata', true);
```

When working with a PortfolioMAD object, the setMinMaxNumAssets function enables you to set up the limits on the number of assets invested. The following example specifies that a minimum of five assets and a maximum of 10 assets should be invested using setMinMaxNumAssets and the investments should be greater than 4% and less than 45% using setBounds.

```
p = setMinMaxNumAssets(p, 5, 10);
p = setBounds(p, 0.04, 0.45, 'BoundType', 'conditional');
```

Use estimateFrontierByReturn to estimate optimal portfolios with targeted portfolio returns.

```
pwgt = estimateFrontierByReturn(p,[ 0.0005, 0.0010 ])
```

```
pwgt = 12×2
```

```
    0.0400    0.0400
         0         0
         0         0
         0         0
         0         0
    0.1064    0.2225
    0.0400    0.0400
    0.0400    0.0400
         0         0
    0.0400    0.0400
         0         0
         0         0
         :
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

MinNumAssets — Minimum number of assets allocated in portfolio

scalar numeric

Minimum number of assets allocated in the portfolio, specified using a scalar numeric.

Data Types: double

MaxNumAssets — Maximum number of assets allocated in portfolio

scalar numeric

Maximum number of assets allocated in the portfolio, specified using a scalar numeric.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a Portfolio, PortfolioCVaR, or PortfolioMAD object.

Tips

- You can also use dot notation to set up a list of identifiers for assets.

```
obj = obj.setMinMaxNumAssets(MinNumAssets,MaxNumAssets);
```

- Specifying empty values ([]) for MinNumAssets and MaxNumAssets removes limit constraints from the Portfolio, PortfolioCVaR, or PortfolioMAD object.

Version History

Introduced in R2018b

See Also

setBounds | setSolverMINLP | estimateAssetMoments | estimateFrontier |
estimateFrontierByReturn | estimateFrontierByRisk | estimateFrontierLimits |
estimateMaxSharpeRatio | estimatePortSharpeRatio | estimatePortMoments |
estimatePortReturn | estimatePortRisk

Topics

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects” on page 4-81

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioCVaR Objects” on page 5-72

“Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects” on page 6-71

“Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints” on page 4-148

“Common Operations on the Portfolio Object” on page 4-33

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Optimization with Semicontinuous and Cardinality Constraints” on page 4-190

“Mixed-Integer Quadratic Programming Portfolio Optimization: Problem-Based”

“Role of Convexity in Portfolio Problems” on page 4-157

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setOneWayTurnover

Set up one-way portfolio turnover constraints

Syntax

```
obj = setOneWayTurnover(obj,BuyTurnover)
obj = setOneWayTurnover(obj,BuyTurnover,SellTurnover,InitPort,NumAssets)
```

Description

`obj = setOneWayTurnover(obj,BuyTurnover)` sets up one-way portfolio turnover constraints for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setOneWayTurnover(obj,BuyTurnover,SellTurnover,InitPort,NumAssets)` sets up one-way portfolio turnover constraints for portfolio objects with additional options specified for `SellTurnover`, `InitPort`, and `NumAssets`.

Given an initial portfolio in `InitPort` and an upper bound for portfolio turnover on purchases in `BuyTurnover` or sales in `SellTurnover`, the one-way turnover constraints require any portfolio `Port` to satisfy the following:

```
1' * max{0, Port - InitPort} <= BuyTurnover
1' * max{0, InitPort - Port} <= SellTurnover
```

Note If `Turnover = BuyTurnover = SellTurnover`, the constraint is not equivalent to:

```
1' * | Port - InitPort | <= Turnover
```

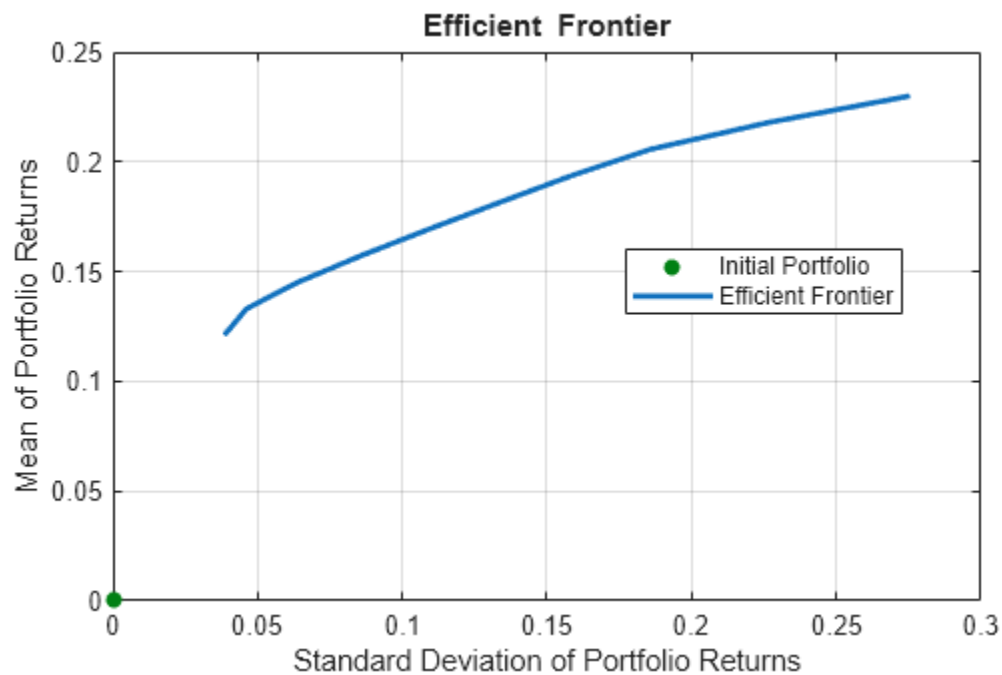
To set this constraint, use `setTurnover`.

Examples

Set One-Way Turnover Constraints for a Portfolio Object

Set one-way turnover constraints.

```
p = Portfolio('AssetMean',[0.1, 0.2, 0.15], 'AssetCovar',...
[ 0.005, -0.010, 0.004; -0.010, 0.040, -0.002; 0.004, -0.002, 0.023]);
p = setBudget(p, 1, 1);
p = setOneWayTurnover(p, 1.3, 0.3, 0); %130-30 portfolio
plotFrontier(p);
```



Set One-Way Turnover Constraints for a PortfolioCVaR Object

Set one-way turnover constraints.

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCVaR('InitPort', x0);
p = setOneWayTurnover(p, 0.3, 0.2);
disp(p.NumAssets);

10

disp(p.BuyTurnover)

0.3000

disp(p.SellTurnover)

0.2000

disp(p.InitPort);

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

Set One-Way Turnover Constraints for a PortfolioMAD Object

Set one-way turnover constraints.

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD('InitPort', x0);
p = setOneWayTurnover(p, 0.3, 0.2);
disp(p.NumAssets);

10

disp(p.BuyTurnover)

0.3000

disp(p.SellTurnover)

0.2000

disp(p.InitPort);

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

BuyTurnover — Turnover constraint on purchases

nonnegative and finite scalar

Turnover constraint on purchases, specified as a nonnegative and finite scalar for a Portfolio, PortfolioCVaR, or PortfolioMAD input object (obj).

SellTurnover — Turnover constraint on sales

nonnegative and finite scalar

Turnover constraint on sales, specified as a nonnegative and finite scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

InitPort — Initial or current portfolio weights

0 (default) | finite vector with `NumAssets > 0` elements

Initial or current portfolio weights, specified as a finite vector with `NumAssets > 0` elements for a `Portfolio`, `PortfolioCVaR`, `PortfolioMAD` input object (`obj`).

Note If no `InitPort` is specified, that value is assumed to be 0.

If `InitPort` is specified as a scalar and `NumAssets` exists, then `InitPort` undergoes scalar expansion.

Data Types: double

NumAssets — Number of assets in portfolio

1 (default) | scalar

Number of assets in portfolio, specified as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (`obj`).

Note If it is not possible to obtain a value for `NumAssets`, it is assumed that `NumAssets` is 1.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

One-way Turnover Constraint

One-way turnover constraint is a specific measure of trading activity in a portfolio that captures the extent to which assets are bought or sold, but only considers the purchases or sales in one direction (either buys or sells).

One-way turnover constraints ensure that estimated optimal portfolios differ from an initial portfolio by no more than specified amounts according to whether the differences are purchases or sales.

The constraints take the form

$$1^T \max\{0, x - x_0\} \leq \tau_B$$

$$1^T \max\{0, x_0 - x\} \leq \tau_S$$

with

- x — The portfolio (*NumAssets* vector)
- x_0 — Initial portfolio (*NumAssets* vector)
- τ_B — Upper-bound for turnover constraint on purchases (scalar)
- τ_S — Upper-bound for turnover constraint on sales (scalar)

Specify one-way turnover constraints using the following properties in a supported portfolio object: `BuyTurnover` for τ_B , `SellTurnover` for τ_S , and `InitPort` for x_0 .

Note The average turnover constraint (which is set using `setTurnover`) is not just the combination of the one-way turnover constraints with the same value for the constraint.

Tips

You can also use dot notation to set up one-way portfolio turnover constraints.

```
obj = obj.setOneWayTurnover(BuyTurnover,SellTurnover,InitPort,NumAssets)
```

Version History

Introduced in R2011a

See Also

`getOneWayTurnover` | `setTurnover` | `setInitPort` | `setCosts`

Topics

“Working with One-Way Turnover Constraints Using Portfolio Object” on page 4-88

“Working with One-Way Turnover Constraints Using PortfolioCVaR Object” on page 5-78

“Working with One-Way Turnover Constraints Using PortfolioMAD Object” on page 6-77

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

setProbabilityLevel

Set probability level for VaR and CVaR calculations

Syntax

```
obj = setProbabilityLevel(obj,ProbabilityLevel)
```

Description

`obj = setProbabilityLevel(obj,ProbabilityLevel)` sets probability level for VaR and CVaR calculations for a `PortfolioCVaR` object. For details on the workflow, see “PortfolioCVaR Object Workflow” on page 5-16.

Examples

Set Probability Level for a PortfolioCVaR Object

Set the `ProbabilityLevel` for a CVaR portfolio object.

```
p = PortfolioCVaR;
p = setProbabilityLevel(p, 0.95);
disp(p.ProbabilityLevel)
```

```
0.9500
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `PortfolioCVaR` object.

For more information on creating a `PortfolioCVaR` object, see

- `PortfolioCVaR`

Data Types: `object`

ProbabilityLevel — Probability level which is 1 minus the probability of losses greater than the value-at-risk

scalar with value from 0 to 1

Probability level which is 1 minus the probability of losses greater than the value-at-risk, specified as a scalar with value from 0 to 1.

Note `ProbabilityLevel` must be a value from 0 to 1 and, in most cases, should be a value from 0.9 to 0.99.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a PortfolioCVaR object. For more information on creating a portfolio object, see

- PortfolioCVaR

Tips

You can also use dot notation to set the probability level for VaR and CVaR calculations:

```
obj = obj.setProbabilityLevel(ProbabilityLevel)
```

Version History

Introduced in R2012b

See Also

setScenarios

Topics

“What Are Scenarios?” on page 5-36

“Conditional Value-at-Risk” on page 5-5

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

setSolver

Choose main solver and specify associated solver options for portfolio optimization

Syntax

```
obj = setSolver(obj,solverType)
obj = setSolver(obj,solverType,Name,Value)
obj = setSolver(obj,solverType,optimoptions)
```

Description

`obj = setSolver(obj,solverType)` selects the main solver and enables you to specify associated solver options for portfolio optimization for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setSolver(obj,solverType,Name,Value)` selects the main solver and enables you to specify associated solver options for portfolio optimization for portfolio objects with additional options specified by using one or more `Name,Value` arguments.

`obj = setSolver(obj,solverType,optimoptions)` selects the main solver and enables you to specify associated solver options for portfolio optimization for portfolio objects with an `optimoptions` object.

Examples

Set Solver Type for a Portfolio Object

If you use the `quadprog` function as the `solverType`, the default is the interior-point-convex version of `quadprog`.

```
load CAPMuniverse
p = Portfolio('AssetList',Assets(1:12));
p = setDefaultConstraints(p);
p = setSolver(p, 'quadprog');
display(p.solverType);
```

```
quadprog
```

You can switch back to `lcprog` with:

```
p = setSolver(p, 'lcprog');
display(p.solverType);
```

```
lcprog
```

Set the Solver Type as 'fmincon' for a PortfolioCVaR Object

Use 'fmincon' as the solverType.

```
p = PortfolioCVaR;  
p = setSolver(p, 'fmincon');  
display(p.solverType);
```

```
fmincon
```

Set the Solver Type as 'fmincon' and Use Name-Value Pair Arguments to Set the Algorithm for a PortfolioCVaR Object

Use 'fmincon' as the solverType and use name-value pair arguments to set the algorithm to 'interior-point' and to turn off the display.

```
p = PortfolioCVaR;  
p = setSolver(p, 'fmincon', 'Algorithm', 'interior-point', 'Display', 'off');  
display(p.solverOptions.Algorithm);
```

```
interior-point
```

```
display(p.solverOptions.Display);
```

```
off
```

Set the Solver Type as 'fmincon' and Use an optimoptions Object to Set the Algorithm for a PortfolioCVaR Object

Use 'fmincon' as the solverType and use an optimoptions object to set the algorithm to 'interior-point' and to turn off the display.

```
p = PortfolioCVaR;  
options = optimoptions('fmincon', 'Algorithm', 'interior-point', 'Display', 'off');  
p = setSolver(p, 'fmincon', options);  
display(p.solverOptions.Algorithm);
```

```
interior-point
```

```
display(p.solverOptions.Display);
```

```
off
```

Set 'TrustRegionCP' as the Solver Type with Default Options for a PortfolioCVaR Object

Use 'TrustRegionCP' as the solverType with default options.

```
p = PortfolioCVaR;  
p = setSolver(p, 'TrustRegionCP');  
display(p.solverType);
```

```
trustregioncp
display(p.solverOptions);
    struct with fields:
        MaxIterations: 1000
        AbsoluteGapTolerance: 1.0000e-07
        RelativeGapTolerance: 1.0000e-05
        NonlinearScalingFactor: 1000
        ObjectiveScalingFactor: 1000
        MainSolverOptions: [1x1 optim.options.Linprog]
        Display: 'off'
        CutGeneration: 'basic'
        MaxIterationsInactiveCut: 30
        ActiveCutTolerance: 1.0000e-07
        ShrinkRatio: 0.7500
        TrustRegionStartIteration: 2
        InitialDelta: 0.5000
        DeltaLimit: 1000000
```

Set 'TrustRegionCP' as the Solver Type with 'ShrinkRatio' for a PortfolioCVaR Object

Use the name-value pair 'ShrinkRatio' to shrink the size of the trust region by 0.75.

```
p = PortfolioCVaR;
p = setSolver(p,'TrustRegionCP','ShrinkRatio',0.75);
display(p.solverType);
trustregioncp
display(p.solverOptions);
    struct with fields:
        MaxIterations: 1000
        AbsoluteGapTolerance: 1.0000e-07
        RelativeGapTolerance: 1.0000e-05
        NonlinearScalingFactor: 1000
        ObjectiveScalingFactor: 1000
        MainSolverOptions: [1x1 optim.options.Linprog]
        Display: 'off'
        CutGeneration: 'basic'
        MaxIterationsInactiveCut: 30
        ActiveCutTolerance: 1.0000e-07
        ShrinkRatio: 0.7500
        TrustRegionStartIteration: 2
        InitialDelta: 0.5000
        DeltaLimit: 1000000
```

Set 'TrustRegionCP' as the Solver Type and Change the Main Solver Option for a PortfolioCvAR Object

For the main solver, continue using the dual-simplex algorithm with no display, but tighten its termination tolerance to $1e8$.

```
p = PortfolioCvAR;
options = optimoptions('linprog','Algorithm','Dual-Simplex','Display','off','OptimalityTolerance',1e8);
p = setSolver(p,'TrustRegionCP','MainSolverOptions',options);
display(p.solverType)

trustregioncp

display(p.solverOptions)

    struct with fields:

        MaxIterations: 1000
        AbsoluteGapTolerance: 1.0000e-07
        RelativeGapTolerance: 1.0000e-05
        NonlinearScalingFactor: 1000
        ObjectiveScalingFactor: 1000
        MainSolverOptions: [1x1 optim.options.Linprog]
        Display: 'off'
        CutGeneration: 'basic'
        MaxIterationsInactiveCut: 30
        ActiveCutTolerance: 1.0000e-07
        ShrinkRatio: 0.7500
        TrustRegionStartIteration: 2
        InitialDelta: 0.5000
        DeltaLimit: 1000000

display(p.solverOptions.MainSolverOptions.Algorithm)

dual-simplex-highs

display(p.solverOptions.MainSolverOptions.Display)

off

display(p.solverOptions.MainSolverOptions.TolFun)

1000000000
```

For the main solver, use the interior-point algorithm with no display and with a termination tolerance of $1e7$.

```
p = PortfolioCvAR;
options = optimoptions('linprog','Algorithm','interior-point','Display','off','OptimalityTolerance',1e7);
p = setSolver(p,'TrustRegionCP','MainSolverOptions',options);
display(p.solverType)

trustregioncp

display(p.solverOptions)

    struct with fields:

        MaxIterations: 1000
        AbsoluteGapTolerance: 1.0000e-07
```

```

        RelativeGapTolerance: 1.0000e-05
        NonlinearScalingFactor: 1000
        ObjectiveScalingFactor: 1000
        MainSolverOptions: [1x1 optim.options.Linprog]
            Display: 'off'
            CutGeneration: 'basic'
        MaxIterationsInactiveCut: 30
        ActiveCutTolerance: 1.0000e-07
        ShrinkRatio: 0.7500
        TrustRegionStartIteration: 2
        InitialDelta: 0.5000
        DeltaLimit: 1000000

display(p.solverOptions.MainSolverOptions.Algorithm)

interior-point

display(p.solverOptions.MainSolverOptions.Display)

off

display(p.solverOptions.MainSolverOptions.TolFun)

10000000

```

Set Solver Type as 'fmincon' for a PortfolioMAD Object

Use 'fmincon' as the solverType.

```

p = PortfolioMAD;
p = setSolver(p, 'fmincon');
display(p.solverType);

fmincon

```

Set the Solver Type as 'fmincon' and Use Name-Value Pair Arguments to Set the Algorithm for a PortfolioMAD Object

Use 'fmincon' as the solverType and use name-value pair arguments to set the algorithm to 'sqp' and to turn on the display.

```

p = PortfolioMAD;
p = setSolver(p, 'fmincon', 'Algorithm', 'sqp', 'Display', 'final');
display(p.solverOptions.Algorithm);

sqp

display(p.solverOptions.Display);

final

```

Set Solver Type as 'fmincon' and Use an optimoptions Structure to Set the Algorithm for a PortfolioMAD Object

Use 'fmincon' as the solverType and use an optimoptions object to set the algorithm to 'trust-region-reflective' and to turn off the display.

```
p = PortfolioMAD;
options = optimoptions('fmincon', 'Algorithm', 'trust-region-reflective', 'Display', 'off');
p = setSolver(p, 'fmincon', options);
display(p.solverOptions.Algorithm);

trust-region-reflective

display(p.solverOptions.Display);

off
```

Set the Solver Type as 'fmincon' and Use an optimoptions Structure to Set the Algorithm and Use of Gradients for a PortfolioMAD Object

Use 'fmincon' as the solverType and use an optimoptions object to set the algorithm to 'active-set' and to set the gradients flag 'on' for 'GradObj' and turn off the display.

```
p = PortfolioMAD;
options = optimoptions('fmincon','algorithm','active-set','display','off','gradobj','on');
p = setSolver(p, 'fmincon', options);
display(p.solverOptions.Algorithm);

active-set

display(p.solverOptions.Display);

off
```

Input Arguments**obj — Object for portfolio**

object

Object for portfolio, specified using Portfolio, PortfolioCVaR, or PortfolioMAD object. For more information on creating a portfolio object, see

- Portfolio
- PortfolioCVaR
- PortfolioMAD

Data Types: object

solverType — Solver to use for portfolio optimization

character vector | string

Solver to use for portfolio optimization, specified using a character vector or string for the supported solverType.

The `solverType` input argument depends on which type of object (`obj`) is being used for a portfolio optimization.

Using Portfolio Object

For a `Portfolio` object, the supported `solverType` are:

- `'lcprog'` (Default).
 - The `'lcprog'` solver uses linear complementary programming with Lemke's algorithm with control variables name-value arguments for `'maxiter'`, `'tiebreak'`, `'tolpiv'`. For more information about `'lcprog'` name-value options, see “Portfolio Object Name-Value Arguments” on page 15-1507.
- `'fmincon'`
 - The default algorithm for `'fmincon'` is `'sqp'`. For more information about `'fmincon'` name-value options, see “Portfolio Object Name-Value Arguments” on page 15-1507.
- `'quadprog'`
 - The default algorithm for `'quadprog'` is `interior-point-convex`. For more information about `'quadprog'` name-value options, see “Portfolio Object Name-Value Arguments” on page 15-1507.

Using PortfolioCVaR Object

For a `PortfolioCVaR` object, the supported `solverType` are:

- `'TrustRegionCP'` (Default)
 - `'TrustRegionCP'` is an implementation of Kelley's [1] cutting-plane method for convex optimization. For more information about `'TrustRegionCP'` name-value options, see “Name-Value Arguments for `'TrustRegionCP'` and `'ExtendedCP'`” on page 15-1508.
- `'ExtendedCP'`
 - `'ExtendedCP'` is an implementation of Kelley's [1] cutting-plane method for convex optimization. For more information about `'ExtendedCP'` name-value options, see “Name-Value Arguments for `'TrustRegionCP'` and `'ExtendedCP'`” on page 15-1508.
- `'fmincon'`
 - The default algorithm for `'fmincon'` is `'sqp'`. For more information about `'fmincon'` name-value options, see “PortfolioCVaR Object Name-Value Arguments” on page 15-1507.
- `'cuttingplane'`
 - The `'cuttingplane'` solver is an implementation of Kelley's [1] cutting-plane method for convex optimization with name-value arguments for `'MaxIter'`, `'Abstol'`, `'Reltol'` and `'MainSolverOptions'`. For more information about `'cuttingplane'` name-value options, see “PortfolioCVaR Object Name-Value Arguments” on page 15-1507.

Using PortfolioMAD Object

For a `PortfolioMAD` object, the supported `solverType` are:

- `'TrustRegionCP'` (Default)

- 'TrustRegionCP' is an implementation of Kelley's [1] cutting-plane method for convex optimization. For more information about 'TrustRegionCP' name-value options, see “Name-Value Arguments for 'TrustRegionCP' and 'ExtendedCP'” on page 15-1508.
- 'ExtendedCP'
- 'ExtendedCP' is an implementation of Kelley's [1] cutting-plane method for convex optimization. For more information about 'ExtendedCP' name-value options, see “Name-Value Arguments for 'TrustRegionCP' and 'ExtendedCP'” on page 15-1508.
- 'fmincon'
- The default algorithm for 'fmincon' is the 'sqp' algorithm and 'GradObj' set to 'on'. For more information about 'fmincon' name-value options, see “PortfolioMAD Object Name-Value Arguments” on page 15-1508.

Note `setSolver` can also configure solver options for 'linprog'. `linprog` is a helper solver used in estimating efficient frontier problems for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. The default algorithm for 'linprog' is 'dual-simplex-highs'. For more information about 'linprog' name-value options, see “Name-Value Arguments” on page 15-1506. For more details on using a helper solver, see “Solver Guidelines for Portfolio Objects” on page 4-115, “Solver Guidelines for PortfolioCVaR Objects” on page 5-100, or “Solver Guidelines for PortfolioMAD Objects” on page 6-101.

Data Types: `char` | `string`

optimoptions — optimoptions object

object

(Optional) `optimoptions` object, specified as an `optimoptions` object that is created using `optimoptions` from Optimization Toolbox. For example:

```
p = setSolver(p,'fmincon',optimoptions('fmincon','Display','iter'));
```

Note `optimoptions` is the default and recommended method to set solver options, however `optimset` is also supported.

Data Types: `object`

Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `p = setSolver(p,'cuttingplane','MainSolverOptions',options)` sets `cuttingplane` options for a `PortfolioCVaR` object.

Depending on the `obj` type (`Portfolio`, `PortfolioCVaR`, or `PortfolioMAD`) and the specified `solverType`, the options for the associated name-value arguments are different.

Portfolio Object Name-Value Arguments

- For a Portfolio object using a solverType of `lcprog`, choose a name-value value in this table.

Value	Description
'maxiter'	Maximum number of iterations, specified as the comma-separated pair consisting of 'MaxIter' and a positive integer. The default value is $1 + n^3$, where n is the dimension of the input.
'tiebreak'	Method to break ties for pivot selection, specified as the comma-separated pair consisting of 'tiebreak' and one of the following options: <code>first</code> - Select pivot with lowest index. <code>last</code> - Select pivot with highest index. <code>random</code> - Select a pivot at random. The default value is <code>first</code> .
'tolpiv'	Pivot tolerance below which a number is considered to be zero, specified as the comma-separated pair consisting of 'tolpiv' and a numeric value. The default value is $1.0e-9$.

- For a Portfolio object using a solverType of `fmincon`, see “options” to choose name-value arguments.
- For a Portfolio object using a solverType of `linprog`, see “options” to choose name-value arguments.
- For a Portfolio object using a solverType of `quadprog`, see “options” to choose name-value arguments.

PortfolioCVaR Object Name-Value Arguments

- For a PortfolioCVaR object using a solverType of `fmincon`, see “options” to choose name-value arguments.
- For a PortfolioCVaR object using a solverType of 'TrustRegionCP' or 'ExtendedCP', see “Name-Value Arguments for 'TrustRegionCP' and 'ExtendedCP'” on page 15-1508 to choose name-value arguments.
- For a PortfolioCVaR object using a solverType of 'cuttingplane', choose a name-value value in this table.

Value	Description
'MaxIter'	Maximum number of iterations, specified as the comma-separated pair consisting of 'MaxIter' and a positive integer. The default value is 1000.
'AbsTol'	Absolute stopping tolerance, specified as the comma-separated pair consisting of 'AbsTol' and a positive scalar. The default value is $1e6$.
'RelTol'	Relative stopping tolerance, specified as the comma-separated pair consisting of 'RelTol' and a positive scalar. The default value is $1e5$.

Value	Description
'MainSolverOptions'	Options for the main solver linprog, specified as the comma-separated pair consisting of 'MainSolverOptions' and an optimoptions object. The default is optimoptions('linprog','Algorithm','dual-simplex-highs','Display','off').

- For a PortfolioCVaR object using a solverType of linprog, see “options” to choose name-value arguments.

PortfolioMAD Object Name-Value Arguments

- For a PortfolioMAD object using a solverType of fmincon, see “options” to choose name-value arguments.
- For a PortfolioMAD object using a solverType of 'TrustRegionCP' or 'ExtendedCP', see “Name-Value Arguments for 'TrustRegionCP' and 'ExtendedCP'” on page 15-1508 to choose name-value arguments.
- For a PortfolioMAD object using a solverType of linprog, see “options” to choose name-value arguments.

Name-Value Arguments for 'TrustRegionCP' and 'ExtendedCP'

For a PortfolioCVaR or PortfolioMAD object using a solverType of 'TrustRegionCP' or 'ExtendedCP', choose a name-value pair in this table.

Value	Description
'MaxIterations'	Maximum number of iterations, specified as the comma-separated pair consisting of 'MaxIterations' and a positive real number. The default value is 1e3.
'NonlinearScalingFactor'	Scales the nonlinear function and its gradient by a factor, specified as the comma-separated pair consisting of 'NonlinearScalingFactor' and a positive real number. The default value is 1e3.
'ObjectiveScalingFactor'	Scales the objective function by a factor, specified as the comma-separated pair consisting of 'ObjectiveScalingFactor' and a positive real number. The default value is 1e3.
'AbsoluteGapTolerance'	Solver stops if the absolute difference between the approximated nonlinear function value and its true value is less than or equal to AbsoluteGapTolerance, specified as the comma-separated pair consisting of 'AbsoluteGapTolerance' and a positive real number. The default value is 1e7.
'RelativeGapTolerance'	Solver stops if the relative difference between the approximated nonlinear function value and its true value is less than or equal to RelativeGapTolerance, specified as the comma-separated pair consisting of 'RelativeGapTolerance' and a positive real number. The default value is 1e5.

Value	Description
'Display'	Level of display, specified as the comma-separated pair consisting of 'Display' and a supported value of: 'iter' displays output at each iteration and gives the technical exit message. 'final' displays just the final output and gives the final technical exit message. 'off' is the default and displays no output.
'CutGeneration'	Method to add the cut, specified as the comma-separated pair consisting of 'CutGeneration' and a supported value of: 'basic' is the default and the new cut is added at the latest solution found. 'midway' is where the new cut is added at the mid point between the latest and previous solution found.
'MaxIterationsInactiveCut'	Removes constraints that are not active for the last MaxIterationsInactiveCut iterations, specified as the comma-separated pair consisting of 'MaxIterationsInactiveCut' and a positive integer. The default value is 30.
'ActiveCutTolerance'	Determines if the cuts are active and is used together with MaxIterationsInactiveCut to decide which cuts to remove from the LP subproblem, specified as the comma-separated pair consisting of 'ActiveCutTolerance' and a real number. The default value is 1e7.
'MainSolverOptions'	Options for the main solver linprog, specified as the comma-separated pair consisting of 'MainSolverOptions' and an optimoptions object. The default is optimoptions('linprog','Algorithm','dual-simplex-highs','Display','off').
'TrustRegionStartIteration'	Use this parameter only for a solverType of 'TrustRegionCP'. Solver starts to apply the trust region heuristic at TrustRegionStartIteration. Nonnegative integer. Default is 2.
'ShrinkRatio'	Use this parameter only for a solverType of 'TrustRegionCP'. If the approximated functions are not agreeing well in the previous iterations, the algorithm will shrink the size of trust region by the ShrinkRatio. Nonnegative real between 0 and 1. Default is 0.75.
'InitialDelta'	Use this parameter only for a solverType of 'TrustRegionCP'. Value to initialize trust region. Nonnegative real. Default is 0.5.

Value	Description
'DeltaLimit'	Use this parameter only for a <code>solverType</code> of 'TrustRegionCP'. The trust region of the approximated functions is bounded by <code>DeltaLimit</code> during the iterations. The <code>DeltaLimit</code> value is a nonnegative real and the default value is <code>1e6</code>. Note Modifying 'DeltaLimit' might result in the solver not being able to find a solution. If you modify 'DeltaLimit' without specifying a value for 'InitialDelta', the 'InitialDelta' is automatically set to <code>InitialDelta = 'DeltaLimit'/2</code>.
'DeltaLowerBound'	Use this parameter only for a <code>solverType</code> of 'TrustRegionCP'. Use 'DeltaLowerBound' to set the lower bound for the trust-region radius. The 'DeltaLowerBound' numeric value must be in <code>[0,1]</code> and you can include 0 and 1. The default value is <code>0.01</code>.

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Tips

You can also use dot notation to choose the solver and specify associated solver options.

```
obj = obj.setSolver(solverType,Name,Value);
```

Algorithms

To solve the efficient frontier of a portfolio, one version of the portfolio optimization problem minimizes the portfolio risk $Risk(x)$, subject to a target return, and other linear constraints specified for the `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For the definition of portfolio risk and return, see “Risk Proxy” on page 4-6 and “Return Proxy” on page 4-5.

$$\text{Minimize}_x Risk(x)$$

$$\text{subject to } Return(x) \geq TargetReturn$$

$$Ax \leq b$$

$$A_{eq}x = b_{eq}$$

$$lb \leq x \leq ub$$

An alternative version of the portfolio optimization problem maximizes the expected return of the portfolio, subject to a target risk and other linear constraints specified for the Portfolio, PortfolioCVaR, or PortfolioMAD object.

$$\begin{aligned} & \text{Maximize}_x \text{Return}(x) \\ & \text{subject to } \text{Risk}(x) \leq \text{TargetRisk} \\ & Ax \leq b \\ & A_{eq}x = b_{eq} \\ & lb \leq x \leq ub \end{aligned}$$

The return proxy is always a linear function. Therefore, depending on the risk proxy and whether it is used as the objective or constraints, the problem needs to be solved by different solvers. For example, quadprog is appropriate for problems with a quadratic function as the objective and only linear constraints, and fmincon is appropriate for problems with nonlinear objective or constraints. In addition, there are solvers in Financial Toolbox suitable for certain special types of problems, such as the solverType lcprog, 'TrustRegionCP', or 'ExtendedCP'.

Version History

Introduced in R2011a

R2024a: Default for name-value argument MainSolverOptions is changed

The default for the name-value argument MainSolverOptions is changed to `optimoptions('linprog','Algorithm','dual-simplex-highs','Display','off')`.

R2023a: Added name-value argument 'DeltaLowerBound' for solverType of 'TrustRegionCP'

When using a solverType of 'TrustRegionCP', you can use the name-value argument 'DeltaLowerBound' to set the lower bound for the trust-region radius.

R2023a: Renamed 'MasterSolverOptions' name-value argument to 'MainSolverOptions'

The 'MasterSolverOptions' name-value argument is renamed to 'MainSolverOptions'. The use of 'MasterSolverOptions' name-value argument is discouraged.

References

- [1] Kelley, J. E. "The Cutting-Plane Method for Solving Convex Programs." *Journal of the Society for Industrial and Applied Mathematics*. Vol. 8, No. 4, December 1960, pp. 703-712.
- [2] Rockafellar, R. T. and S. Uryasev "Optimization of Conditional Value-at-Risk." *Journal of Risk*. Vol. 2, No. 3, Spring 2000, pp. 21-41.
- [3] Rockafellar, R. T. and S. Uryasev "Conditional Value-at-Risk for General Loss Distributions." *Journal of Banking and Finance*. Vol. 26, 2002, pp. 1443-1471.

See Also

`getOneWayTurnover` | `setTurnover` | `setInitPort` | `setCosts` | `setSolverMINLP`

Topics

“Working with One-Way Turnover Constraints Using Portfolio Object” on page 4-88
“Working with One-Way Turnover Constraints Using PortfolioCVaR Object” on page 5-78
“Working with One-Way Turnover Constraints Using PortfolioMAD Object” on page 6-77
“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161
Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8
“Solver Guidelines for Portfolio Objects” on page 4-115
“Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization” on page 4-114
“Choosing and Controlling the Solver for PortfolioCVaR Optimizations” on page 5-98
“Choosing and Controlling the Solver for PortfolioMAD Optimizations” on page 6-99

setSolverMINLP

Choose mixed integer nonlinear programming (MINLP) solver for portfolio optimization

Syntax

```
obj = setSolverMINLP(obj,solverTypeMINLP)
```

```
obj = setSolverMINLP( ___,Name,Value)
```

Description

`obj = setSolverMINLP(obj,solverTypeMINLP)` selects the mixed integer nonlinear programming (MINLP) solver and enables you to specify associated solver options for portfolio optimization for a `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object.

When any one or any combination of 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active, the portfolio problem is formulated by adding `NumAssets` binary variables. The binary variable 0 indicates that an asset is not invested and the binary variable 1 indicates that an asset is invested. For more information on using 'Conditional' `BoundType`, see `setBounds`. For more information on specifying `MinNumAssets` and `MaxNumAssets`, see `setMinMaxNumAssets`.

If you use the Financial Toolbox `estimate` functions with a `Portfolio`, `PortfolioCVar`, or `PortfolioMAD` object for which any of the 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active, MINLP solver is automatically used. For details on MINLP, see “Algorithms” on page 15-1531 and “Choose MINLP Solvers for Portfolio Problems” on page 4-140.

`obj = setSolverMINLP( ___,Name,Value)` specifies options using one or more name-value arguments in addition to the input arguments in the previous syntax.

Examples

Configure MINLP Solver and Options When Using a Portfolio Object

Configure the MINLP solver for a three-asset portfolio for which you have the mean and covariance values of the asset returns.

```
AssetMean = [ 0.0101110; 0.0043532; 0.0137058 ];
AssetCovar = [ 0.00324625 0.00022983 0.00420395;
               0.00022983 0.00049937 0.00019247;
               0.00420395 0.00019247 0.00764097 ];
p = Portfolio('AssetMean', AssetMean, 'AssetCovar', AssetCovar);
```

When working with a `Portfolio` object, use `setBounds` with a 'Conditional' `BoundType` constraint to set $x_i = 0$ or $0.02 \leq x_i \leq 0.5$ for all $i = 1, \dots, \text{NumAssets}$.

```
p = setBounds(p, 0.02, 0.5, 'BoundType', 'Conditional', 'NumAssets', 3);
```

When working with a `Portfolio` object, use `setMinMaxNumAssets` function to set up `MinNumAssets` and `MaxNumAssets` constraints for a portfolio. This sets limit constraints for the

Portfolio object, where the total number of allocated assets satisfying the constraints is between MinNumAssets and MaxNumAssets. Setting MinNumAssets = MaxNumAssets = 2, specifies that only two of the three assets are invested in the portfolio.

```
p = setMinMaxNumAssets(p, 2, 2);
```

Three different MINLP solvers (OuterApproximation, ExtendedCP, TrustRegionCP) use the cutting plane method. Use the setSolverMINLP function to configure the OuterApproximation solver and options.

```
pint = setSolverMINLP(p, 'OuterApproximation', 'NonlinearScalingFactor', 1e4, 'Display', 'iter',  
pint.solverTypeMINLP
```

```
ans =  
'OuterApproximation'
```

```
pint.solverOptionsMINLP
```

```
ans = struct with fields:
```

```
             MaxIterations: 1000  
             AbsoluteGapTolerance: 1.0000e-07  
             RelativeGapTolerance: 1.0000e-05  
             NonlinearScalingFactor: 10000  
             ObjectiveScalingFactor: 1000  
             Display: 'iter'  
             CutGeneration: 'basic'  
             MaxIterationsInactiveCut: 30  
             ActiveCutTolerance: 1.0000e-07  
             IntMainSolverOptions: [1x1 optim.options.Intlinprog]  
             NumIterationsEarlyIntegerConvergence: 30  
             ExtendedFormulation: 0  
             NumInnerCuts: 10  
             NumInitialOuterCuts: 10
```

You can also configure the options for intlinprog, which is the Main solver for mixed integer linear programming problems in the MINLP solver.

```
pint = setSolverMINLP(p, 'OuterApproximation', 'IntMainSolverOptions', optimoptions('intlinprog',  
pint.solverOptionsMINLP.IntMainSolverOptions
```

```
ans =  
intlinprog options:
```

```
Options used by current Algorithm ('highs'):  
(Other available algorithms: 'legacy')
```

```
Set properties:  
    Display: 'off'
```

```
Default properties:  
    AbsoluteGapTolerance: 1.0000e-06  
    Algorithm: 'highs'  
    ConstraintTolerance: 1.0000e-06  
    MaxFeasiblePoints: 2.1475e+09  
    MaxNodes: 10000000  
    MaxTime: 7200  
    ObjectiveCutOff: Inf
```



```

OutputFcn: []
PlotFcn: []
RelativeGapTolerance: 1.0000e-04

```

Show options not used by current Algorithm ('highs')

Configure MINLP Solver and Options When Using a PortfolioCVaR Object

Configure the MINLP solver for a 12 asset portfolio that is using semicontinuous and cardinality constraints.

```

load CAPMuniverse
p = PortfolioCVaR('AssetList',Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:,1:12),20000,'missingdata',true);
p = setProbabilityLevel(p, 0.95);

```

When working with a `PortfolioCVaR` object, the `setMinNumAssets` function enables you to set up the limits on the number of assets invested. The following example specifies that a minimum of five assets and a maximum of 10 assets should be invested using `setMinNumAssets` and the investments should be greater than 4% and less than 45% using `setBounds`.

```

p = setMinNumAssets(p, 5, 10);
p = setBounds(p, 0.04, 0.45, 'BoundType','conditional');

```

Three different MINLP solvers (`OuterApproximation`, `ExtendedCP`, `TrustRegionCP`) use the cutting plane method. Use the `setSolverMINLP` function to configure the `OuterApproximation` solver and options.

```

pint = setSolverMINLP(p,'OuterApproximation','NonlinearScalingFactor', 1e4, 'Display', 'iter', '0');
pint.solverTypeMINLP

```

```

ans =
'OuterApproximation'

```

```

pint.solverOptionsMINLP

```

```

ans = struct with fields:

```

```

    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 10000
    ObjectiveScalingFactor: 1000
        Display: 'iter'
    CutGeneration: 'basic'
MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1x1 optim.options.Intlinprog]
NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
        NumInnerCuts: 10
    NumInitialOuterCuts: 10

```

You can also configure the options for `intlinprog`, which is the Main solver for mixed integer linear programming problems in the MINLP solver.

```

pint = setSolverMINLP(p, 'OuterApproximation', 'IntMainSolverOptions', optimoptions('intlinprog', 'pint.solverOptionsMINLP.IntMainSolverOptions

ans =
    intlinprog options:

    Options used by current Algorithm ('highs'):
    (Other available algorithms: 'legacy')

    Set properties:
        Display: 'off'

    Default properties:
        AbsoluteGapTolerance: 1.0000e-06
            Algorithm: 'highs'
        ConstraintTolerance: 1.0000e-06
        MaxFeasiblePoints: 2.1475e+09
        MaxNodes: 10000000
        MaxTime: 7200
        ObjectiveCutOff: Inf
        OutputFcn: []
        PlotFcn: []
        RelativeGapTolerance: 1.0000e-04

    Show options not used by current Algorithm ('highs')

```

Configure MINLP Solver and Options When Using a PortfolioMAD Object

Configure the MINLP solver for a 12 asset portfolio that is using semicontinuous and cardinality constraints.

```

load CAPMuniverse
p = PortfolioMAD('AssetList', Assets(1:12));
p = simulateNormalScenariosByData(p, Data(:, 1:12), 20000, 'missingdata', true);

```

When working with a PortfolioMAD object, the setMinMaxNumAssets function enables you to set up the limits on the number of assets invested. The following example specifies that a minimum of five assets and a maximum of 10 assets should be invested using setMinMaxNumAssets and the investments should be greater than 4% and less than 45% using setBounds.

```

p = setMinMaxNumAssets(p, 5, 10);
p = setBounds(p, 0.04, 0.45, 'BoundType', 'conditional');

```

Three different MINLP solvers (OuterApproximation, ExtendedCP, TrustRegionCP) use the cutting plane method. Use the setSolverMINLP function to configure the OuterApproximation solver and options.

```

pint = setSolverMINLP(p, 'OuterApproximation', 'NonlinearScalingFactor', 1e4, 'Display', 'iter',
pint.solverTypeMINLP

ans =
    'OuterApproximation'

pint.solverOptionsMINLP

```

```
ans = struct with fields:
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 10000
    ObjectiveScalingFactor: 1000
    Display: 'iter'
    CutGeneration: 'basic'
    MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1x1 optim.options.Intlinprog]
    NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

You can also configure the options for `intlinprog`, which is the Main solver for mixed integer linear programming problems in the MINLP solver.

```
pint = setSolverMINLP(p, 'OuterApproximation', 'IntMainSolverOptions', optimoptions('intlinprog',
pint.solverOptionsMINLP.IntMainSolverOptions
```

```
ans =
    intlinprog options:

    Options used by current Algorithm ('highs'):
    (Other available algorithms: 'legacy')

    Set properties:
        Display: 'off'

    Default properties:
    AbsoluteGapTolerance: 1.0000e-06
        Algorithm: 'highs'
    ConstraintTolerance: 1.0000e-06
    MaxFeasiblePoints: 2.1475e+09
    MaxNodes: 10000000
    MaxTime: 7200
    ObjectiveCutOff: Inf
    OutputFcn: []
    PlotFcn: []
    RelativeGapTolerance: 1.0000e-04

    Show options not used by current Algorithm ('highs')
```

Use Extended Formulation of OuterApproximation Solver Type

This example shows how to solve a portfolio optimization problem with constraints in the number of selected assets and conditional (semi-continuous) bounds. This example demonstrates `setSolverMINLP` support for extended formulation, using the name-value argument `ExtendedFormulation`, for the `OuterApproximation` solverTypeMINLP. The extended formulation reformulates problems with quadratic functions to work in an extended space to decrease the computation time.

Create Mean-Variance Portfolio

Load the returns data in `CAPMuniverse.mat`. Then, create a standard mean-variance `Portfolio` object with default constraints, a long-only portfolio whose weights sum to 1. For this example, you can define the feasible region of weights X as

$$X = \left\{ x \mid \sum_{i=1}^n x_i = 1, x_i \geq 0 \right\}.$$

```
% Load data
load CAPMuniverse.mat

% Create a mean-variance Portfolio object with default constraints
p = Portfolio(AssetList=Assets(1:12));
p = estimateAssetMoments(p,Data(:,1:12));
p = setDefaultConstraints(p);
p.solverOptionsMINLP

ans = struct with fields:
    MaxIterations: 1000
    AbsoluteGapTolerance: 1.0000e-07
    RelativeGapTolerance: 1.0000e-05
    NonlinearScalingFactor: 1000
    ObjectiveScalingFactor: 1000
    Display: 'off'
    CutGeneration: 'basic'
    MaxIterationsInactiveCut: 30
    ActiveCutTolerance: 1.0000e-07
    IntMainSolverOptions: [1x1 optim.options.Intlinprog]
    NumIterationsEarlyIntegerConvergence: 30
    ExtendedFormulation: 0
    NumInnerCuts: 10
    NumInitialOuterCuts: 10
```

Include binary variables for this scenario by setting conditional (semicontinuous) bounds using `setBounds`. Conditional bounds are those such that $x_i = 0$ or $x_i \geq \ell_i$. In this example, $\ell_i = 0.1$ for all assets.

```
% Set conditional bounds
condLB = 0.1;
condUB = 0.5;
p = setBounds(p,condLB,condUB,BoundType="conditional")

p =
    Portfolio with properties:
        BuyCost: []
        SellCost: []
        RiskFreeRate: []
        AssetMean: [12x1 double]
        AssetCovar: [12x12 double]
        TrackingError: []
        TrackingPort: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
```

```

        Name: []
        NumAssets: 12
        AssetList: {'AAPL' 'AMZN' 'CSCO' 'DELL' 'EBAY' 'GOOG' 'HPQ' 'IBM' 'JPM' 'KO' 'MSFT' 'NFLX' 'ORCL' 'PFE' 'TSLA' 'VZ'}
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: [12x1 double]
        UpperBound: [12x1 double]
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: [12x1 categorical]

```

Set ExtendedFormulation of OuterApproximation

Use `setSolverMINLP` to apply the extended formulation to the problem by setting the `solverTypeMINLP` argument for `OuterApproximation` and the `ExtendedFormulation` name-value argument to `true`. The extended formulation reformulates problems with quadratic functions to work in an extended space to decrease the computation time. You can use the name-value arguments `NumInnerCuts` and `NumInitialOuterCuts` to control the extended formulation algorithm:

- `NumInnerCuts` — Number of cuts per asset added to the inner approximation problem to obtain an initial feasible point and upper bound
- `NumInitialOuterCuts` — Number of cuts per asset added to the initial outer approximation problem.

% Increase the number of inner cuts and initial outer cuts

```
pEOA = setSolverMINLP(p,'OuterApproximation',ExtendedFormulation=true,NumInnerCuts=20,NumInitialOuterCuts=20);
pEOA.solverOptionsMINLP
```

ans = struct with fields:

```

        MaxIterations: 1000
        AbsoluteGapTolerance: 1.0000e-07
        RelativeGapTolerance: 1.0000e-05
        NonlinearScalingFactor: 1000
        ObjectiveScalingFactor: 1000
        Display: 'off'
        CutGeneration: 'basic'
        MaxIterationsInactiveCut: 30
        ActiveCutTolerance: 1.0000e-07
        IntMainSolverOptions: [1x1 optim.options.Intlinprog]
        NumIterationsEarlyIntegerConvergence: 30
        ExtendedFormulation: 1

```

```

NumInnerCuts: 20
NumInitialOuterCuts: 20

```

Estimate Set of Portfolios on Efficient Frontier

Use `estimateFrontier` to compute the efficient frontier, which is the curve that shows the trade-off between the return and risk achieved by Pareto-optimal portfolios. For a given return level, the portfolio on the efficient frontier is the one that minimizes the risk while maintaining the desired return. Conversely, for a given risk level, the portfolio on the efficient frontier is the one that maximizes return while maintaining the desired risk level. Compute risks and returns of the portfolios on the efficient frontier.

```
% Find the efficient frontier
```

```
wEOA = estimateFrontier(pEOA)
```

```
wEOA = 12×10
```

```

0      0      0.1000      0.1253      0.1745      0.2236      0.2715      0.3327      0.4111      0.5
0      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0
0.1350      0      0      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0
0.1000      0.1450      0.1406      0.1910      0.2344      0.2778      0.3200      0.3726      0.4415      0.5
0.1000      0.1609      0.1642      0.2121      0.2415      0.2709      0.3085      0.2947      0.1474
0.2354      0.1875      0.1290      0      0      0      0      0      0
0      0      0      0      0      0      0      0      0
0.4296      0.4066      0.3662      0.3717      0.2496      0.1277      0      0      0
0      0.1000      0.1000      0.1000      0.1000      0.1000      0.1000      0      0
0      0      0      0      0      0      0      0      0
:

```

```
% Compute risk and returns of the portfolios on the efficient frontier
```

```
[rskEOA,retEOA] = estimatePortMoments(p,wEOA)
```

```
rskEOA = 10×1
```

```

0.0076
0.0080
0.0085
0.0094
0.0105
0.0117
0.0132
0.0147
0.0168
0.0193

```

```
retEOA = 10×1
```

```

0.0008
0.0012
0.0017
0.0021
0.0026
0.0030

```

```

0.0034
0.0039
0.0043
0.0048

```

Maximize Return by Risk

Problems modeled to obtain the efficient frontier are of the type that minimizes risk subject to a return level. Another common portfolio problem is one that maximizes return for a fixed risk level. The risk level selected should be achievable by the portfolio. That is, the risk level should be between the risk attained by the minimum risk portfolio and the maximum return portfolio. The following code first checks if the target risk level is achievable and, if it is, computes the portfolio that maximizes the return subject to the target risk level.

```

% Set a risk level that is "achievable"
targetRisk_A = mean(rskEOA);
wAchievRisk = estimateFrontierByRisk(p,targetRisk_A)

wAchievRisk = 12x1

    0.2322
    0.0000
    0.0000
    0.0000
    0.0000
    0.2853
    0.2760
    0.0000
    0.0000
    0.1065
    0.1000
    0.0000
    ⋮

```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

solverTypeMINLP — MINLP solver for portfolio optimization

character vector with a value of `'OuterApproximation'`, `'ExtendedCP'`, or `'TrustRegionCP'` | string with a value of `"OuterApproximation"`, `"ExtendedCP"`, or `"TrustRegionCP"`

MINLP solver for portfolio optimization when any one or any combination of `'Conditional'` `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints are active. Specify `solverTypeMINLP`

using a character vector or string with a value of 'OuterApproximation', or 'ExtendedCP', 'TrustRegionCP'.

Using Portfolio Object

- For a Portfolio object, when using the default value of the solverTypeMINLP as 'OuterApproximation', the following name-value arguments, with their default values, are supported for setSolverMINLP:
 - MaxIterations — 1000
 - AbsoluteGapTolerance — 1.0000e-07
 - RelativeGapTolerance — 1.0000e-05
 - Display — 'off'
 - NonlinearScalingFactor — 1000
 - ObjectiveScalingFactor — 1000
 - ExtendedFormulation — false
 - CutGeneration — 'basic'
 - MaxIterationsInactiveCut — 30
 - NumIterationsEarlyIntegerConvergence — 30
 - ActiveCutTolerance — 1.0000e-07
 - IntMainSolverOptions —
optimoptions('intlinprog','Algorithm','legacy','Display','off','ConstraintTolerance',1e-5,'MaxTime',1000,'IPPreprocess','none','CutGeneration','advanced','Heuristics','rins','IntegerTolerance',1e-5,'NodeSelection','mininfeas','LPPreprocess','none')
- For the extended formulation with a Portfolio object, when you specify the value of the solverTypeMINLP as 'OuterApproximation' and set the name-value argument ExtendedFormulation to true, the following name-value arguments, with their default values, are supported for setSolverMINLP:
 - MaxIterations — 1000
 - AbsoluteGapTolerance — 1.0000e-07
 - RelativeGapTolerance — 1.0000e-05
 - Display — 'off'
 - NonlinearScalingFactor — 1000
 - ObjectiveScalingFactor — 1000
 - ExtendedFormulation — true
 - NumInnerCuts — 10
 - NumInitialOuterCuts — 10
 - MaxIterationsInactiveCut — 30
 - NumIterationsEarlyIntegerConvergence — 30
 - ActiveCutTolerance — 1.0000e-07
 - IntMainSolverOptions —
optimoptions('intlinprog','Algorithm','legacy','Display','off','ConstraintTolerance',1e-5,'MaxTime',1000,'IPPreprocess','none','CutGeneration','


```
advanced','Heuristics','rins','IntegerTolerance',1e-5,'NodeSelection','mininfeas','LPPreprocess','none')
```

For more information on the extended formulation solver, see “Extended Formulation” on page 15-1529. For examples of using extended formulation, see “Use Extended Formulation of OuterApproximation Solver Type” on page 15-1517 and “Mixed-Integer Mean-Variance Portfolio Optimization Problem” on page 4-402.

Using PortfolioCVaR or PortfolioMAD Object

For a PortfolioCVaR and PortfolioMAD object, the default value of the solverTypeMINLP is 'TrustRegionCP' with the following default settings for name-value pairs for setSolverMINLP:

- MaxIterations — 1000
- AbsoluteGapTolerance — 1.0000e-07
- RelativeGapTolerance — 1.0000e-05
- Display — 'off'
- NonlinearScalingFactor — 1000
- ObjectiveScalingFactor — 1000
- CutGeneration — 'basic'
- MaxIterationsInactiveCut — 30
- NumIterationsEarlyIntegerConvergence — 30
- ActiveCutTolerance — 1.0000e-07
- TrustRegionStartIteration - 2
- ShrinkRatio — 0.75
- InitialDelta — 0.5
- DeltaLimit — 1e6
- DeltaLowerBound — 0.01
- IntMainSolverOptions —
optimoptions('intlinprog','Algorithm','legacy','Display','off','ConstraintTolerance',1e-5,'MaxTime',1000,'IPPreprocess','none','CutGeneration','advanced','Heuristics','rins','IntegerTolerance',1e-5,'NodeSelection','mininfeas','LPPreprocess','none')

Data Types: char | string

Name-Value Arguments or optimoptions Object

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: p =
setSolverMINLP(p,'ExtendedCP','MaxIterations',10000,'NonlinearScalingFactor',1000)

MaxIterations — Maximum number of iterations

1000 (default) | nonnegative integer

Maximum number of iterations, specified as the comma-separated pair consisting of 'MaxIterations' and a nonnegative integer value.

Data Types: double

NonlinearScalingFactor — Scaling factor for nonlinear function and gradient

1000 (default) | nonnegative real

Scaling factor for nonlinear function and gradient, specified as the comma-separated pair consisting of 'NonlinearScalingFactor' and a nonnegative real value.

Data Types: double

ObjectiveScalingFactor — Scales the objective function used by MainSolver by a factor

1000 (default) | nonnegative real

Scales the objective function used by the MainSolver by a factor, specified as the comma-separated pair consisting of 'ObjectiveScalingFactor' and a nonnegative real value.

Data Types: double

AbsoluteGapTolerance — Solver stops if absolute difference between approximated nonlinear function value and its true value is less than or equal to AbsoluteGapTolerance

0.0000001 (default) | nonnegative real

The solver stops if the absolute difference between the approximated nonlinear function value and its true value is less than or equal to AbsoluteGapTolerance. AbsoluteGapTolerance is specified as the comma-separated pair consisting of 'AbsoluteGapTolerance' and a nonnegative real value.

Data Types: double

RelativeGapTolerance — Solver stops if the relative difference between approximated nonlinear function value and its true value is less than or equal to RelativeGapTolerance

0.00001 (default) | nonnegative real

The solver stops if the relative difference between the approximated nonlinear function value and its true value is less than or equal to RelativeGapTolerance. RelativeGapTolerance is specified as the comma-separated pair consisting of 'RelativeGapTolerance' and a nonnegative real value.

Data Types: double

Display — Display output format

'off' (default) | character vector with value of 'iter', 'final', or 'off'

Display output format, specified as the comma-separated pair consisting of 'Display' and a character vector with a value of:

- 'off' — Display no output
- 'iter' — Display output at each iteration and the technical exit message
- 'final' — Display only the final output and the final technical exit message

Data Types: char

CutGeneration — Cut specification

'basic' (default) | character vector with value of 'basic', 'midway', or 'advanced'

Cut specification, specified as the comma-separated pair consisting of 'CutGeneration' and a character vector with one of these values:

- 'basic' — Add the new cut at the latest solution found.
- 'midway' — Add the new cut at the midpoint between the latest and previous solutions found.

Data Types: char

ExtendedFormulation — Extend problem if all functions are linear or quadratic for Portfolio object

false (default) | logical with value true or false

Extend problem if all functions are linear or quadratic for a Portfolio object, specified as the comma-separated pair consisting of 'ExtendedFormulation' and a logical value:

- true — If possible, use extended formulation. When set to true, you can specify a value for NumInnerCuts and NumInitialOuterCuts.
- false — Use original formulation.

For more information on the extended formulation solver, see “Extended Formulation” on page 15-1529. For an example of using extended formulation, see “Use Extended Formulation of OuterApproximation Solver Type” on page 15-1517 and “Mixed-Integer Mean-Variance Portfolio Optimization Problem” on page 4-402.

Data Types: logical

NumInnerCuts — Number of inner cuts (secant lines) added per asset to find a feasible point

10 (default) | positive numeric value < 20

Number of inner cuts (secant lines) added per asset to find a feasible point, specified as the comma-separated pair consisting of 'NumInnerCuts' and a positive numeric value ≤ 20 . Consider decreasing NumInnerCuts when the number of assets is greater than 100.

Note NumInnerCuts is only applied when ExtendedFormulation is set to true. If NumInnerCuts = 0, the solver does not solve an inner approximation to find an initial feasible point.

For more information on the extended formulation solver, see “Extended Formulation” on page 15-1529. For an example of using extended formulation, see “Use Extended Formulation of OuterApproximation Solver Type” on page 15-1517 and “Mixed-Integer Mean-Variance Portfolio Optimization Problem” on page 4-402.

Data Types: double

NumInitialOuterCuts — Number outer cuts added at beginning of the algorithm to start the iterations with a more accurate approximation

10 (default) | numeric

Number outer cuts added at the beginning of the algorithm to start the iterations with a more accurate approximation, specified as the comma-separated pair consisting of

'NumInitialOuterCuts' and a positive numeric value ≤ 20 . Consider decreasing NumInitialOuterCuts when the number of assets is greater than 100

Note NumInitialOuterCuts is only applied when ExtendedFormulation is set to true. If NumInitialOuterCuts = 0, the initial MILP approximation starts without outer cuts.

For more information on the extended formulation solver, see “Extended Formulation” on page 15-1529. For an example of using extended formulation, see “Use Extended Formulation of OuterApproximation Solver Type” on page 15-1517.

Data Types: double

MaxIterationsInactiveCut — Removes constraints that are not active for last MaxIterationsInactiveCut iterations

30 (default) | nonnegative integer

Removes constraints that are not active for the last MaxIterationsInactiveCut iterations, specified as the comma-separated pair consisting of 'MaxIterationsInactiveCut' and a nonnegative integer value. Generally, the MaxIterationsInactiveCut value is larger than 10.

Data Types: double

NumIterationsEarlyIntegerConvergence — When integer variable solution is stable for the last NumIterationsEarlyIntegerConvergence iterations, the solver computes a final NLP by using latest integer variable solution in the MILP

30 (default) | nonnegative integer

When the integer variable solution is stable for the last NumIterationsEarlyIntegerConvergence iterations, the solver computes a final NLP by using the latest integer variable solution in the MILP. NumIterationsEarlyIntegerConvergence is specified as the comma-separated pair consisting of 'NumIterationsEarlyIntegerConvergence' and a nonnegative integer value.

Data Types: double

ActiveCutTolerance — Determines if the cuts are active

0.0000001 (default) | nonnegative real

Determines if the cuts are active, specified as the comma-separated pair consisting of 'ActiveCutTolerance' and a nonnegative real value. ActiveCutTolerance is used together with MaxIterationsInactiveCut to decide which cuts to remove from the MILP subproblem.

Data Types: double

TrustRegionStartIteration — Solver starts to apply trust region heuristic at TrustRegionStartIteration

2 (default) | nonnegative integer

Solver starts to apply the trust region heuristic at TrustRegionStartIteration, specified as the comma-separated pair consisting of 'TrustRegionStartIteration' and a nonnegative integer.

Note The TrustRegionStartIteration name-value pair argument can only be used with a solverTypeMINLP of 'TrustRegionCP'.

Data Types: double

ShrinkRatio — Ratio to shrink size of trust region

0.75 (default) | nonnegative real between 0 and 1

Ratio to shrink size of trust region, specified as the comma-separated pair consisting of 'ShrinkRatio' and a nonnegative real value between 0 and 1. If the approximated functions do not have good agreement in the previous iterations, the algorithm uses this ratio to shrink the trust-region size.

Note The ShrinkRatio name-value pair argument can only be used with a solverTypeMINLP of 'TrustRegionCP'.

Data Types: double

InitialDelta — Value to initialize trust region

0.5 (default) | nonnegative real

Value to initialize trust region, specified as the comma-separated pair consisting of 'InitialDelta' and a nonnegative real value.

Note The InitialDelta name-value pair argument can only be used with a solverTypeMINLP of 'TrustRegionCP'.

Data Types: double

DeltaLimit — Trust region of the approximated functions is bounded by DeltaLimit during iterations

1e6 (default) | nonnegative real

Trust region of the approximated functions is bounded by DeltaLimit during the iterations, specified as the comma-separated pair consisting of 'DeltaLimit' and a nonnegative real value.

Note The DeltaLimit name-value pair argument can only be used with a solverTypeMINLP of 'TrustRegionCP'.

Modifying 'DeltaLimit' might result in the solver not being able to find a solution.

If you modify 'DeltaLimit' without specifying a value for 'InitialDelta', the 'InitialDelta' is automatically set to InitialDelta' = 'DeltaLimit'/2.

Data Types: double

DeltaLowerBound — Set lower bound for trust-region radius

0.01 (default) | numeric value in [0, 1]

Set lower bound for the trust-region radius, specified as the comma-separated pair consisting of 'DeltaLowerBound' and a numeric value in [0, 1] that can include 0 and 1.

Note The `DeltaLowerBound` name-value pair argument can only be used with a `solverType` `MINLP` or `'TrustRegionCP'`.

Data Types: double

QuadprogOptions — quadprog options to solve the continuous version of the mixed-integer quadratic problem

`optimoptions('quadprog','LinearSolver','sparse','Display','off','maxiter',10000,'ConstraintTolerance',1.0e-8,'OptimalityTolerance',1.0e-8,'StepTolerance',1.0e-8)` (default) | `quadprog` `optimoptions` object

`quadprog` options to solve the continuous version of the mixed-integer quadratic problem (MIQP), specified as the comma-separated pair consisting of `'QuadprogOptions'` and an `quadprog` `optimoptions` object.

Data Types: `optim.options.Quadprog`

FminconOptions — fmincon options to solve the continuous version of the mixed-integer non-linear problem

`optimoptions('fmincon','Display','off','Algorithm','sqp','SpecifyObjectiveGradient',true,'SpecifyConstraintGradient',true,'ConstraintTolerance',1.0e-8,'OptimalityTolerance',1.0e-8,'StepTolerance',1.0e-8)` (default) | `fmincon` `optimoptions` object

`fmincon` options to solve the continuous version of the mixed-integer non-linear problem (MINLP), specified as the comma-separated pair consisting of `'FminconOptions'` and a `fmincon` `optimoptions` object.

Data Types: `optim.options.Fmincon`

ObjectiveBoundRelaxation — Nonnegative value subtracted from (when minimizing) or added to (when maximizing) the objective bound

`1e-4` (default) | nonnegative numeric

Nonnegative value subtracted from (when minimizing) or added to (when maximizing) the objective bound to account for numerical inaccuracies in the objective bound, specified as the comma-separated pair consisting of `'ObjectiveBoundRelaxation'` and a nonnegative numeric value.

Data Types: double

MaxIterationsGapStall — Number of iterations allowed for the objective lower and upper bounds difference to remain the same before stopping the algorithm

`25` (default) | integer

Number of iterations allowed for the objective lower and upper bounds difference to remain the same before stopping the algorithm, specified as the comma-separated pair consisting of `'MaxIterationsGapStall'` and an integer.

Data Types: double

IntMainSolverOptions — Options for main solver

`optimoptions('intlinprog','Algorithm','legacy','Display','off','ConstraintTolerance',1e-5,'MaxTime',1000,'IPPreprocess','none','CutGeneration','advanced',`

```
'Heuristics','rins','IntegerTolerance',1e-5,'NodeSelection','mininfeas','LPPr  
eprocess','none') (default) | optimoptions object
```

Options for the main solver `intlinprog`, specified as the comma-separated pair consisting of `'IntMainSolverOptions'` and an `optimoptions` object.

Example: `'IntMainSolverOptions', optimoptions('intlinprog','Display','off')`

Data Types: object

Output Arguments

obj — Updated portfolio object

object

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object.

More About

MINLP Solvers

All three MINLP solvers (`'OuterApproximation'`, `'ExtendedCP'`, and `'TrustRegionCP'`) defined by `solverTypeMINLP` rely on the cutting plane concept.

These MINLP solvers approximate the nonlinear convex function $f(x)$ by a piecewise linear approximation, which is a sequence of linear cuts around the original function. In this way, the original MINLP is reduced to a sequence of MILP subproblems, each one with a more refined approximation to $f(x)$ than previous MILPs, and yields a more optimal solution. The process continues until the solution found from MILP converges to the true function value within a certain tolerance.

- The `'ExtendedCP'` solver iteratively adds a linear cut at the latest solution found to approximate $f(x)$.
- The `'OuterApproximation'` solver is similar to `'ExtendedCP'`, but they differ in where to add the cut. Instead of using the solution from the latest MILP, `OuterApproximation` uses the values of integer variables from the latest MILP solution and fixes them to reduce the MINLP to a nonlinear programming (NLP) problem. The cut is added at the solution from this NLP problem.
- The `'TrustRegionCP'` solver is a version of `'ExtendedCP'` that is modified to speed up the optimization process. In general, the trust region method uses a model to approximate the true function within a region at each iteration. In the context of the MINLP solver, the model is the maximum of all the added cuts. The true function is the nonlinear function $f(x)$ in the optimization problem. The region of the model is updated based on how well the model approximates the true function for the iteration. This approximation is the comparison of the predicted reduction of the objective function using the model vs. the true reduction.

Extended Formulation

The extended formulation solver can only be used with a `Portfolio` object to solve problems to with quadratic functions, such as variance and tracking error.

Assume the following problem

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & g(x) \leq 0 \\ & x \in X \end{aligned}$$

In the extended formulation, nonlinear functions must have separable terms, that is, each term must only depend on one variable. For example:

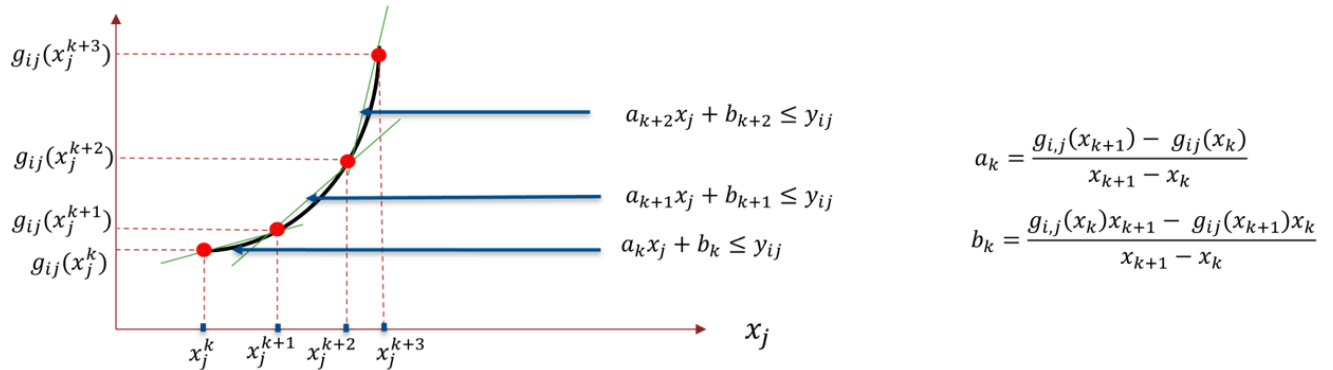
$$g(x) = x_1^2 + \frac{1}{x_2} - \alpha \leq 0$$

$$g(x) = g_1(x_1) + g_2(x_2) - \alpha \leq 0, \quad g_1(y) = y^2, \quad g_2(y) = \frac{1}{y} - \alpha$$

This allows the problem to be reformulated as

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & g_i(x_i) \leq y_i \\ & \sum_{i=1}^n y_i \leq 0 \\ & x \in X \end{aligned}$$

This reformulation obtains tighter approximations because, instead of adding just one cut at each iteration, you are adding `numVars` cuts per iteration. Furthermore, the univariate functions allow you to build a better initial approximation that helps to obtain an initial feasible point. The initial approximation is achieved with the inner cuts. Unlike the outer cuts that linearly approximate the nonlinear function with tangent lines, the inner cuts use secant lines.



Quadratic functions are not originally separable. By introducing two sets of auxiliary variables (y and t), you can rewrite all quadratic functions in a problem as separable functions as long as the matrix of the quadratic coefficients is positive semidefinite.

$$g(x) = x^T Q x + q^T x + q_0 \leq 0,$$

$$g(x) = x^T (S^T S) x + q^T x + q_0 \leq 0, \quad Q = S^T S$$

$$g(x) = y^T y + q^T x + q_0 \leq 0, \quad y = Sx$$

$$g(x) = \sum_{i=1}^n (t_i + q_i x_i) \leq -q_0, \quad y_i^2 \leq t_i$$

Tips

You can also use dot notation to specify associated name-value options.


```
obj = obj.setSolverMINLP(Name,Value);
```

Note The `solverTypeMINLP` and `solverOptionsMINLP` properties cannot be set using dot notation because they are hidden properties. To set the `solverTypeMINLP` and `solverOptionsMINLP` properties, use the `setSolverMINLP` function directly.

Algorithms

When any one, or any combination of 'Conditional' `BoundType`, `MinNumAssets`, or `MaxNumAssets` constraints is active, the portfolio problem is formulated by adding `NumAssets` binary variables. The binary variable 0 indicates that an asset is not invested and the binary variable 1 indicates that an asset is invested.

The `MinNumAssets` and `MaxNumAssets` constraints narrow down the number of active positions in a portfolio to the range of $[minN, maxN]$. In addition, the 'Conditional' `BoundType` constraint is to set a lower and upper bound so that the position is either 0 or lies in the range $[minWgt, maxWgt]$. These two types of constraints are incorporated into the portfolio optimization model by introducing n variables, v_i , which only take binary values 0 and 1 to indicate whether the corresponding asset is invested (1) or not invested (0). Here n is the total number of assets and the constraints can be formulated as the following linear inequality constraints:

$$\begin{aligned} minN &\leq \sum_{i=1}^n v_i \leq maxN \\ minWgt * v_i &\leq x_i \leq maxWgt * v_i \\ 0 &\leq v_i \leq 1 \\ v_i &\text{ are integers} \end{aligned}$$

In this equation, $minN$ and $maxN$ are representations for `MinNumAsset` and `MaxNumAsset` that are set using `setMinMaxNumAssets`. Also, $minWgt$ and $maxWgt$ are representations for `LowerBound` and `UpperBound` that are set using `setBounds`.

The portfolio optimization problem to minimize the variance of the portfolio, subject to achieving a target expected return and some additional linear constraints on the portfolio weights, is formulated as

$$\begin{aligned} &\text{minimize}_x \quad x^T H x \\ &s.t. \quad m^T x \geq TargetReturn \\ &\quad Ax \leq b \\ &\quad A_{eq} x = b_{eq} \\ &\quad lb \leq x \leq ub \end{aligned}$$

In this equation, H represents the covariance and m represents the asset returns.

The portfolio optimization problem to maximize the return, subject to an upper limit on the variance of the portfolio return and some additional linear constraints on the portfolio weights, is formulated as

$$\begin{aligned} & \text{maximize}_{\mathbf{x}} \quad m^T \mathbf{x} \\ & \text{s.t.} \quad \mathbf{x}^T H \mathbf{x} \leq \text{TargetRisk} \\ & \quad A \mathbf{x} \leq \mathbf{b} \\ & \quad A_{eq} \mathbf{x} = \mathbf{b}_{eq} \\ & \quad \mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub} \end{aligned}$$

When the 'Conditional' `BoundType`, `MinNumAssets`, and `MaxNumAssets` constraints are added to the two optimization problems, the problems become:

$$\begin{aligned} & \text{minimize}_{\mathbf{x}, \mathbf{v}} \quad \mathbf{x}^T H \mathbf{x} \\ & \text{s.t.} \quad m^T \mathbf{x} \geq \text{TargetReturn} \\ & \quad A'[\mathbf{x}; \mathbf{v}] \leq \mathbf{b}' \\ & \quad A_{eq} \mathbf{x} = \mathbf{b}_{eq} \\ & \quad \min N \leq \sum_{i=1}^n v_i \leq \max N \\ & \quad \min Wgt * v_i \leq x_i \leq \max Wgt * v_i \\ & \quad \mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub} \\ & \quad 0 \leq v \leq 1 \\ & \quad v_i \text{ are integers} \end{aligned}$$

$$\begin{aligned} & \text{maximize}_{\mathbf{x}, \mathbf{v}} \quad m^T \mathbf{x} \\ & \text{s.t.} \quad \mathbf{x}^T H \mathbf{x} \leq \text{TargetRisk} \\ & \quad A'[\mathbf{x}; \mathbf{v}] \leq \mathbf{b}' \\ & \quad A_{eq} \mathbf{x} = \mathbf{b}_{eq} \\ & \quad \min N \leq \sum_{i=1}^n v_i \leq \max N \\ & \quad \min Wgt * v_i \leq x_i \leq \max Wgt * v_i \\ & \quad 0 \leq v \leq 1 \\ & \quad v_i \text{ are integers} \end{aligned}$$

Version History

Introduced in R2018b

R2024a: Default for name-value argument `IntMainSolverOptions` is changed

The default for the name-value argument `IntMainSolverOptions` is changed to `optimoptions('intlinprog','Algorithm','legacy','Display','off','ConstraintTolerance',1e-5,'MaxTime',1000,'IPPreprocess','none','CutGeneration','advanced','Heuristics','rins','IntegerTolerance',1e-5,'NodeSelection','mininfeas','LPPreprocess','none')`.

R2023b: Added ExtendedFormulation for solverTypeMINLP of OuterApproximation and name-value arguments for NumInnerCuts and NumInitialOuterCuts

To use the extended formulation solver with setSolverMINLP for a Portfolio object, you set the solverTypeMINLP argument to OuterApproximation and specify the name-value arguments for ExtendedFormulation, NumInnerCuts, and NumInitialOuterCuts.

R2023b: Added name-value arguments for QuadprogOptions, FminconOptions, ObjectiveBoundRelaxation, and MaxIterationsGapStall

When using a Portfolio, PortfolioCVaR, or PortfolioMAD object, you can use the setSolverMINLP name-value arguments for QuadprogOptions, FminconOptions, ObjectiveBoundRelaxation, and MaxIterationsGapStall.

R2023a: Added name-value argument 'DeltaLowerBound' for solverTypeMINLP of 'TrustRegionCP'

When using a solverTypeMINLP of 'TrustRegionCP', you can use the 'DeltaLowerBound' name-value argument to set the lower bound for the trust-region radius.

R2023a: Renamed 'IntMasterSolverOptions' name-value argument to 'IntMainSolverOptions'

The 'IntMasterSolverOptions' name-value argument is renamed to 'IntMainSolverOptions'. The use of 'IntMasterSolverOptions' name-value argument is discouraged.

References

- [1] Bonami, P., Kilinc, M., and J. Linderoth. "Algorithms and Software for Convex Mixed Integer Nonlinear Programs." Technical Report #1664. Computer Sciences Department, University of Wisconsin-Madison, 2009.
- [2] Kelley, J. E. "The Cutting-Plane Method for Solving Convex Programs." *Journal of the Society for Industrial and Applied Mathematics*. Vol. 8, Number 4, 1960, pp. 703-712.
- [3] Linderoth, J. and S. Wright. "Decomposition Algorithms for Stochastic Programming on a Computational Grid." *Computational Optimization and Applications*. Vol. 24, Issue 2-3, 2003, pp. 207-250.
- [4] Nocedal, J., and S. Wright. *Numerical Optimization*. New York: Springer-Verlag, 1999.

See Also

optimoptions | setBounds | setMinMaxNumAssets | estimateFrontier |
estimateFrontierByReturn | estimateFrontierByRisk | estimateFrontierLimits |
estimateMaxSharpeRatio | setSolver

Topics

"Portfolio Optimization with Semicontinuous and Cardinality Constraints" on page 4-190

"Mixed-Integer Quadratic Programming Portfolio Optimization: Problem-Based"
"Mixed-Integer CVaR Portfolio Optimization Problem" on page 5-141
"Mixed-Integer MAD Portfolio Optimization Problem" on page 6-89
"Mixed-Integer Mean-Variance Portfolio Optimization Problem" on page 4-402
Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9
Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8
Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8
"Solver Guidelines for Portfolio Objects" on page 4-115
"Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization" on page 4-114
"Choosing and Controlling the Solver for PortfolioCVaR Optimizations" on page 5-98
"Choosing and Controlling the Solver for PortfolioMAD Optimizations" on page 6-99
"Choose MINLP Solvers for Portfolio Problems" on page 4-140

setScenarios

Set asset returns scenarios by direct matrix

Note Using a `fints` object for the `AssetScenarios` argument of `setScenarios` is not recommended. Use `timetable` instead for financial time series. For more information, see “Convert Financial Time Series Objects (`fints`) to Timetables”.

Syntax

```
obj = setScenarios(obj,AssetScenarios)
obj = setScenarios(obj,AssetScenarios,Name,Value)
```

Description

`obj = setScenarios(obj,AssetScenarios)` sets asset returns scenarios by direct matrix for `PortfolioCVaR` or `PortfolioMAD` objects. For details on the workflows, see “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setScenarios(obj,AssetScenarios,Name,Value)` set asset returns scenarios by direct matrix for `PortfolioCVaR` or `PortfolioMAD` objects using additional options specified by one or more `Name,Value` pair arguments.

Examples

Set Asset Returns Scenarios for a PortfolioCVaR Object

Given a `PortfolioCVaR` object `p`, use the `setScenarios` function to set asset return scenarios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);
disp(p)

PortfolioCVaR with properties:

    BuyCost: []
    SellCost: []
RiskFreeRate: []
```

```
ProbabilityLevel: 0.9500
  Turnover: []
  BuyTurnover: []
  SellTurnover: []
  NumScenarios: 20000
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
  AInequality: []
  bInequality: []
  AEquality: []
  bEquality: []
  LowerBound: [4×1 double]
  UpperBound: []
  LowerBudget: 1
  UpperBudget: 1
  GroupMatrix: []
  LowerGroup: []
  UpperGroup: []
    GroupA: []
    GroupB: []
  LowerRatio: []
  UpperRatio: []
  MinNumAssets: []
  MaxNumAssets: []
  ConditionalBudgetThreshold: []
  ConditionalUpperBudget: []
    BoundType: [4×1 categorical]
```

Set Asset Returns Scenarios for a PortfolioMAD Object

Given PortfolioMAD object p, use the setScenarios function to set asset return scenarios.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
disp(p)
```

PortfolioMAD with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
Turnover: []
```

```

    BuyTurnover: []
    SellTurnover: []
    NumScenarios: 20000
        Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: [4×1 categorical]

```

Set Asset Returns Scenarios for a PortfolioCVaR Object Using Timetable Data

To illustrate using the `setScenarios` function with `AssetScenarios` data continued in a `timetable` object, use the `CAPMuniverse.mat` which contains a `timetable` object (`AssetTimeTable`) for returns data.

```

load CAPMuniverse;
AssetsTimeTable.Properties;
head(AssetsTimeTable,5)

```

Time	AAPL	AMZN	CSCO	DELL	EBAY	GOOG	HP
03-Jan-2000	0.088805	0.1742	0.008775	-0.002353	0.12829	NaN	0.0
04-Jan-2000	-0.084331	-0.08324	-0.05608	-0.08353	-0.093805	NaN	-0.07
05-Jan-2000	0.014634	-0.14877	-0.003039	0.070984	0.066875	NaN	-0.00
06-Jan-2000	-0.086538	-0.060072	-0.016619	-0.038847	-0.012302	NaN	-0.00
07-Jan-2000	0.047368	0.061013	0.0587	-0.037708	-0.000964	NaN	0.02

`setScenarios` accepts a name-value pair argument name `'DataFormat'` with a corresponding value set to `'prices'` to indicate that the input to the function is in the form of asset prices and not returns (the default value for the `'DataFormat'` argument is `'returns'`).

```

r = PortfolioCVaR;
r = setScenarios(r,AssetsTimeTable,'dataformat','returns');

```

In addition, the `setScenarios` function also extracts asset names or identifiers from a `timetable` object when the name-value argument `'GetAssetList'` set to `true` (its default value is `false`). If

the 'GetAssetList' value is true, the timetable column identifiers are used to set the `AssetList` property of the `PortfolioCVaR` object. To show this, the formation of the `PortfolioCVaR` object `r` is repeated with the 'GetAssetList' flag set to true.

```
r = setScenarios(r,AssetsTimeTable,'GetAssetList',true);
disp(r.AssetList')

{'AAPL' }
{'AMZN' }
{'CSCO' }
{'DELL' }
{'EBAY' }
{'GOOG' }
{'HPQ' }
{'IBM' }
{'INTC' }
{'MSFT' }
{'ORCL' }
{'YHOO' }
{'MARKET' }
{'CASH' }
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `PortfolioCVaR` or `PortfolioMAD` object.

For more information on creating a `PortfolioCVaR` or `PortfolioMAD` object, see

- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: `object`

AssetScenarios — Scenarios for asset returns or prices

matrix | table | timetable

Scenarios for asset returns or prices, specified as a matrix, table, or timetable that contains asset data that can be converted into asset returns ([`NumSamples`-by-`NumAssets`] matrix).

AssetReturns data can be:

- `NumSamples`-by-`NumAssets` matrix.
- Table of `NumSamples` prices or returns at a given periodicity for an underlying single-period investment horizon for a collection of `NumAssets` assets
- Timetable object with `NumSamples` observations and `NumAssets` time series

If the input data are prices, they can be converted into returns with the `DataFormat` name-value pair argument, where the default format is assumed to be 'Returns'. Be careful using price data because portfolio optimization usually requires total returns and not simply price returns.

This function sets up a function handle to indirectly access input `AssetScenarios` without needing to make a copy of the data.

Data Types: `double` | `table` | `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `p = setScenarios(p, AssetScenarios, 'DataFormat', 'Returns', 'GetAssetList', false);`

DataFormat — Flag to convert input data as prices into returns

'Returns' (default) | character vector with values 'Returns' or 'Prices'

Flag to convert input data as prices into returns, specified as the comma-separated pair consisting of 'DataFormat' and a character vector with the values:

- 'Returns' — Data in `AssetReturns` contains asset total returns.
- 'Prices' — Data in `AssetReturns` contains asset total return prices.

Data Types: `char`

GetAssetList — Flag indicating which asset names to use for the asset list

false (default) | logical with value true or false

Flag indicating which asset names to use for the asset list, specified as the comma-separated pair consisting of 'GetAssetList' and a logical with a value of true or false. Acceptable values for `GetAssetList` are:

- false — Do not extract or create asset names.
- true — Extract or create asset names from table or timetable.

If a table or timetable is passed into this function as `AssetScenarios` and the `GetAssetList` flag is true, the column names from the table or timetable are used as asset names in `obj.AssetList`.

If a matrix is passed and the `GetAssetList` flag is true, default asset names are created based on the `AbstractPortfolio` property `defaultforAssetList`, which is currently 'Asset'.

If the `GetAssetList` flag is false, no action occurs, which is the default behavior.

Data Types: `logical`

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `PortfolioCVaR` or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `PortfolioCVaR`

- PortfolioMAD

Tips

You can also use dot notation to set asset return scenarios.

```
obj = obj.setScenarios(AssetScenarios);
```

Version History

Introduced in R2012b

See Also

getScenarios

Topics

“Asset Returns and Scenarios Using PortfolioCVaR Object” on page 5-36

“Asset Returns and Scenarios Using PortfolioMAD Object” on page 6-35

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

setTrackingError

Set up maximum portfolio tracking error constraint

Syntax

```
obj = setTrackingError(obj,TrackingError)
obj = setTrackingError( ____,TrackingPort,NumAssets)
```

Description

`obj = setTrackingError(obj,TrackingError)` sets up a maximum portfolio tracking error constraint for a `Portfolio` object. For details on the workflow when using a `Portfolio` object, see “Portfolio Object Workflow” on page 4-18.

`obj = setTrackingError( ____,TrackingPort,NumAssets)` sets up a maximum portfolio tracking error constraint using optional arguments for `TrackingPort` and `NumAssets`.

Examples

Set up a Tracking Error Constraint

Create a `Portfolio` object.

```
AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064 0.00408 0.00192 0;
               0.00408 0.0289 0.0204 0.0119;
               0.00192 0.0204 0.0576 0.0336;
               0 0.0119 0.0336 0.1225 ];

p = Portfolio('mean', AssetMean, 'covar', AssetCovar, 'lb', 0, 'budget', 1)

p =
    Portfolio with properties:

        BuyCost: []
        SellCost: []
    RiskFreeRate: []
        AssetMean: [4×1 double]
        AssetCovar: [4×4 double]
    TrackingError: []
    TrackingPort: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
```

```
        bEquality: []
        LowerBound: [4×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: []
```

Estimate the Sharpe ratio for the Portfolio object p and define the tracking error.

```
x0 = estimateMaxSharpeRatio(p);
te = 0.08;
p = setTrackingError(p, te, x0);
display(p.NumAssets);
```

```
4
```

```
display(p.TrackingError);
```

```
0.0800
```

```
display(p.TrackingPort);
```

```
0.6608
```

```
0.1622
```

```
0.0626
```

```
0.1143
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object. For more information on creating a portfolio object, see `Portfolio`.

Data Types: object

TrackingError — Upper bound for portfolio tracking error

nonnegative and finite scalar

Upper bound for portfolio tracking error, specified using a nonnegative and finite scalar.

Given an upper bound for portfolio tracking error in `TrackingError` and a tracking portfolio in `TrackingPort`, the tracking error constraint requires any portfolio in `Port` to satisfy

$$(\text{Port} - \text{TrackingPort})' * \text{AssetCovar} * (\text{Port} - \text{TrackingPort}) \leq \text{TrackingError}^2.$$

For more information, see “Tracking Error Constraints” on page 4-15.

Data Types: double

TrackingPort — Tracking portfolio weights

finite vector

Tracking portfolio weights, specified using a vector. `TrackingPort` must be a finite vector with `NumAssets > 0` elements.

If no `TrackingPort` is specified, it is assumed to be 0. If `TrackingPort` is specified as a scalar and `NumAssets` exists, then `TrackingPort` undergoes scalar expansion.

Data Types: double

NumAssets — Number of assets in portfolio

scalar

Number of assets in portfolio, specified using a scalar. If it is not possible to obtain a value for `NumAssets`, it is assumed that `NumAssets` is 1.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio` object. For more information on creating a portfolio object, see `Portfolio`.

Note The tracking error constraints can be used with any of the other supported constraints in the `Portfolio` object without restrictions. However, since the portfolio set necessarily and sufficiently must be a non-empty compact set, the application of a tracking error constraint can result in an empty portfolio set. Use `estimateBounds` to confirm that the portfolio set is non-empty and compact.

More About

Tracking Error

Tracking error is a measure used in portfolio management to assess how closely a portfolio's performance aligns with that of a benchmark index.

Tracking error is the standard deviation of the difference between the returns of the portfolio and the returns of the benchmark index. It provides a statistical measure of the variability of the portfolio's excess returns relative to the benchmark.

Tips

You can also use dot notation to set up a maximum portfolio tracking error constraint.

```
obj = obj.setTrackingError(TrackingError, NumAssets);
```

To remove a tracking portfolio, call this function with an empty argument (`[]`) for `TrackingError`.

```
obj = setTrackingError(obj, [ 1]);
```

Version History

Introduced in R2015b

See Also

[setTrackingPort](#) | [Portfolio](#)

Topics

[“Working with Tracking Error Constraints Using Portfolio Object”](#) on page 4-91

[“Portfolio Optimization Examples Using Financial Toolbox”](#) on page 4-161

[“Add Tracking Error Constraints to a Custom Objective Problem”](#) on page 15-922

[Supported Constraints for Portfolio Optimization Using Portfolio Objects](#) on page 4-9

[“Setting Up a Tracking Portfolio”](#) on page 4-40

setTrackingPort

Set up benchmark portfolio for tracking error constraint

Syntax

```
obj = setTrackingPort(obj,TrackingPort)
obj = setTrackingPort( ____,NumAssets)
```

Description

`obj = setTrackingPort(obj,TrackingPort)` sets up a benchmark portfolio for a tracking error constraint for a `Portfolio` object. For details on the workflow when using a `Portfolio` object, see “Portfolio Object Workflow” on page 4-18.

`obj = setTrackingPort( ____,NumAssets)` sets up a benchmark portfolio for a tracking error constraint using an optional input argument for `NumAssets`.

Examples

Set up a Tracking Port

Create a `Portfolio` object.

```
AssetMean = [ 0.05; 0.1; 0.12; 0.18 ];
AssetCovar = [ 0.0064 0.00408 0.00192 0;
               0.00408 0.0289 0.0204 0.0119;
               0.00192 0.0204 0.0576 0.0336;
               0 0.0119 0.0336 0.1225 ];

p = Portfolio('mean', AssetMean, 'covar', AssetCovar, 'lb', 0, 'budget', 1)

p =
    Portfolio with properties:

        BuyCost: []
        SellCost: []
    RiskFreeRate: []
        AssetMean: [4x1 double]
        AssetCovar: [4x4 double]
    TrackingError: []
    TrackingPort: []
        Turnover: []
        BuyTurnover: []
        SellTurnover: []
        Name: []
        NumAssets: 4
        AssetList: []
        InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
```

```
        bEquality: []
        LowerBound: [4×1 double]
        UpperBound: []
        LowerBudget: 1
        UpperBudget: 1
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
        BoundType: []
```

Estimate the Sharpe ratio for the Portfolio object `p` and define the tracking port.

```
x0 = estimateMaxSharpeRatio(p);
p = setTrackingPort(p, x0);
```

```
display(p.NumAssets);
```

```
4
```

```
display(p.TrackingPort);
```

```
0.6608
0.1622
0.0626
0.1143
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a `Portfolio` object. For more information on creating a portfolio object, see `Portfolio`.

Data Types: `object`

TrackingPort — Tracking portfolio weights

vector

Tracking portfolio weights, specified using a vector. If `TrackingPort` is specified as a scalar and `NumAssets` exists, then `TrackingPort` undergoes scalar expansion.

Data Types: `double`

NumAssets — Number of assets in portfolio

scalar

Number of assets in portfolio, specified using a scalar. If it is not possible to obtain a value for NumAssets, it is assumed that NumAssets is 1.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a Portfolio object. For more information on creating a portfolio object, see Portfolio.

Note The tracking error constraints can be used with any of the other supported constraints in the Portfolio object without restrictions. However, since the portfolio set necessarily and sufficiently must be a non-empty compact set, the application of a tracking error constraint can result in an empty portfolio set. Use estimateBounds to confirm that the portfolio set is non-empty and compact.

More About

Tracking Error

Tracking error is a measure used in portfolio management to assess how closely a portfolio's performance aligns with that of a benchmark index.

Tracking error is the standard deviation of the difference between the returns of the portfolio and the returns of the benchmark index. It provides a statistical measure of the variability of the portfolio's excess returns relative to the benchmark.

Tips

You can also use dot notation to set up a benchmark portfolio for tracking error constraint.

```
obj = obj.setTrackingPort(TrackingPort, NumAssets);
```

To remove a tracking portfolio, call this function with an empty argument ([]) for TrackingPort.

```
obj = setTrackingPort(obj, [ ]);
```

Version History

Introduced in R2015b

See Also

setTrackingError | Portfolio

Topics

“Working with Tracking Error Constraints Using Portfolio Object” on page 4-91

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

“Setting Up a Tracking Portfolio” on page 4-40

setTurnover

Set up maximum portfolio turnover constraint

Syntax

```
obj = setTurnover(obj,Turnover)
obj = setTurnover(obj,Turnover,InitPort,NumAssets)
```

Description

`obj = setTurnover(obj,Turnover)` sets up maximum portfolio turnover constraint for `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` objects. For details on the respective workflows when using these different objects, see “Portfolio Object Workflow” on page 4-18, “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = setTurnover(obj,Turnover,InitPort,NumAssets)` sets up maximum portfolio turnover constraint for portfolio objects with additional options specified for `Turnover`, `InitPort`, and `NumAssets`.

Given an upper bound for portfolio turnover in `Turnover` and an initial portfolio in `InitPort`, the turnover constraint requires any portfolio in `Port` to satisfy the following:

$$| \text{Port} - \text{InitPort} | \leq \text{Turnover}$$

Examples

Set Turnover Constraint for a Portfolio Object

Given a `Portfolio` object `p`, to ensure that average turnover is no more than 30% with an initial portfolio of 10 assets in a variable `x0`, use the `setTurnover` method to set the turnover constraint.

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = Portfolio('InitPort', x0);
p = setTurnover(p, 0.3);
```

```
disp(p.NumAssets);
```

```
10
```

```
disp(p.Turnover);
```

```
0.3000
```

```
disp(p.InitPort);
```

```
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
```

```
0.1500
0.1100
0.0800
0.1000
```

Set Turnover Constraint for a CVaR Portfolio Object

Given a CVaR portfolio object `p`, to ensure that average turnover is no more than 30% with an initial portfolio of 10 assets in a variable `x0`, use the `setTurnover` method to set the turnover constraint.

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioCVaR('InitPort', x0);
p = setTurnover(p, 0.3);
```

```
disp(p.NumAssets);
```

```
10
```

```
disp(p.Turnover);
```

```
0.3000
```

```
disp(p.InitPort);
```

```
0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000
```

Set Turnover Constraint for a MAD Portfolio Object

Given PortfolioMAD object `p`, to ensure that average turnover is no more than 30% with an initial portfolio of 10 assets in a variable `x0`, use the `setTurnover` method to set the turnover constraint.

```
x0 = [ 0.12; 0.09; 0.08; 0.07; 0.1; 0.1; 0.15; 0.11; 0.08; 0.1 ];
p = PortfolioMAD('InitPort', x0);
p = setTurnover(p, 0.3);
```

```
disp(p.NumAssets);
```

```
10
```

```
disp(p.Turnover);
```

```
0.3000
```

```
disp(p.InitPort);
```

```

0.1200
0.0900
0.0800
0.0700
0.1000
0.1000
0.1500
0.1100
0.0800
0.1000

```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

Data Types: object

Turnover — Portfolio turnover constraint

nonnegative and finite scalar

Portfolio turnover constraint, specified as a nonnegative and finite scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

InitPort — Initial or current portfolio weights

0 (default) | finite vector with `NumAssets > 0` elements.

Initial or current portfolio weights, specified as a finite vector with `NumAssets > 0` elements for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

Note If no `InitPort` is specified, that value is assumed to be 0.

If `InitPort` is specified as a scalar and `NumAssets` exists, then `InitPort` undergoes scalar expansion.

Data Types: double

NumAssets — Number of assets in portfolio

1 (default) | scalar

Number of assets in portfolio, specified as a scalar for a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` input object (obj).

Note If it is not possible to obtain a value for `NumAssets`, it is assumed that `NumAssets` is 1.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `Portfolio`, `PortfolioCVaR`, or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `Portfolio`
- `PortfolioCVaR`
- `PortfolioMAD`

More About

Turnover Constraint

Turnover constraint is a limitation placed on the amount of buying and selling activity that can occur within a portfolio.

Turnover measures how frequently assets in a portfolio are replaced or traded, and it is expressed as a percentage of the total portfolio value.

Tips

You can also use dot notation to set up the maximum portfolio turnover constraint.

```
obj = obj.setTurnover(Turnover, InitPort, NumAssets);
```

Version History

Introduced in R2011a

See Also

`getOneWayTurnover` | `setOneWayTurnover` | `setInitPort`

Topics

“Working with Average Turnover Constraints Using Portfolio Object” on page 4-85

“Working with Average Turnover Constraints Using PortfolioCVaR Object” on page 5-75

“Portfolio Optimization Examples Using Financial Toolbox” on page 4-161

“Portfolio Analysis with Turnover Constraints” on page 4-211

Supported Constraints for Portfolio Optimization Using Portfolio Objects on page 4-9

Supported Constraints for Portfolio Optimization Using PortfolioCVaR Object on page 5-8

Supported Constraints for Portfolio Optimization Using PortfolioMAD Object on page 6-8

simulateNormalScenariosByData

Simulate multivariate normal asset return scenarios from data

Syntax

```
obj = simulateNormalScenariosByData(obj,AssetReturns)
```

```
obj = simulateNormalScenariosByData(obj,AssetReturns,NumScenarios,Name,Value)
```

Description

`obj = simulateNormalScenariosByData(obj,AssetReturns)` simulates multivariate normal asset return scenarios from data for portfolio object for `PortfolioCVaR` or `PortfolioMAD` objects. For details on the workflows, see “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = simulateNormalScenariosByData(obj,AssetReturns,NumScenarios,Name,Value)` simulates multivariate normal asset return scenarios from data for portfolio object for `PortfolioCVaR` or `PortfolioMAD` objects using additional options specified by one or more `Name,Value` pair arguments.

This function estimates the mean and covariance of asset returns from either price or return data and then uses these estimates to generate the specified number of scenarios with the function `mvnrnd`.

Data can be in a `NumSamples-by-NumAssets` matrix of `NumSamples` prices or returns at a given periodicity for a collection of `NumAssets` assets, a `table` or a `timetable`.

Note If you want to use the method multiple times and you want to simulate identical scenarios each time the function is called, precede each function call with `rng(seed)` using a specified integer seed.

Examples

Simulate Multivariate Normal Asset Return Scenarios from Data for a PortfolioCVaR Object

Given a `PortfolioCVaR` object `p`, use the `simulateNormalScenariosByData` function to simulate multivariate normal asset return scenarios from data.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

RawData = mvnrnd(m, C, 240);
NumScenarios = 2000;
```

```
p = PortfolioCVaR;  
p = simulateNormalScenariosByData(p, RawData, NumScenarios)
```

```
p =  
PortfolioCVaR with properties:  
  
    BuyCost: []  
    SellCost: []  
    RiskFreeRate: []  
    ProbabilityLevel: []  
    Turnover: []  
    BuyTurnover: []  
    SellTurnover: []  
    NumScenarios: 2000  
    Name: []  
    NumAssets: 4  
    AssetList: []  
    InitPort: []  
    AInequality: []  
    bInequality: []  
    AEquality: []  
    bEquality: []  
    LowerBound: []  
    UpperBound: []  
    LowerBudget: []  
    UpperBudget: []  
    GroupMatrix: []  
    LowerGroup: []  
    UpperGroup: []  
    GroupA: []  
    GroupB: []  
    LowerRatio: []  
    UpperRatio: []  
    MinNumAssets: []  
    MaxNumAssets: []  
    ConditionalBudgetThreshold: []  
    ConditionalUpperBudget: []  
    BoundType: []
```

```
p = setDefaultConstraints(p);  
p = setProbabilityLevel(p, 0.9);
```

```
disp(p);  
  
PortfolioCVaR with properties:  
  
    BuyCost: []  
    SellCost: []  
    RiskFreeRate: []  
    ProbabilityLevel: 0.9000  
    Turnover: []  
    BuyTurnover: []  
    SellTurnover: []  
    NumScenarios: 2000  
    Name: []  
    NumAssets: 4  
    AssetList: []  
    InitPort: []
```



```

AInequality: []
bInequality: []
  AEquality: []
  bEquality: []
  LowerBound: [4×1 double]
  UpperBound: []
LowerBudget: 1
UpperBudget: 1
GroupMatrix: []
  LowerGroup: []
  UpperGroup: []
    GroupA: []
    GroupB: []
  LowerRatio: []
  UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
  BoundType: [4×1 categorical]

```

Simulate Multivariate Normal Asset Return Scenarios from Data for PortfolioCVaR Object Using Financial Timetable Data

To illustrate using the `simulateNormalScenariosByData` function with `AssetReturns` data continued in a timetable object, use the `CAPMuniverse.mat` which contains a timetable object (`AssetTimeTable`) for returns data.

```
load CAPMuniverse;
AssetsTimeTable.Properties
```

```
ans =
  TimetableProperties with properties:
```

```

    Description: ''
      UserData: []
DimensionNames: {'Time' 'Variables'}
VariableNames: {'AAPL' 'AMZN' 'CSCO' 'DELL' 'EBAY' 'GOOG' 'HPQ' 'IBM' 'INTC'}
VariableTypes: ["double" "double" "double" "double" "double" "double"]
VariableDescriptions: {}
  VariableUnits: {}
VariableContinuity: []
      RowTimes: [1471×1 datetime]
      StartTime: 03-Jan-2000
      SampleRate: NaN
      TimeStep: NaN
      Events: []
CustomProperties: No custom properties are set.
Use addprop and rmprop to modify CustomProperties.

```

```
head(AssetsTimeTable,5)
```

Time	AAPL	AMZN	CSCO	DELL	EBAY	GOOG	HPQ

03-Jan-2000	0.088805	0.1742	0.008775	-0.002353	0.12829	NaN	0.0
04-Jan-2000	-0.084331	-0.08324	-0.05608	-0.08353	-0.093805	NaN	-0.0
05-Jan-2000	0.014634	-0.14877	-0.003039	0.070984	0.066875	NaN	-0.0
06-Jan-2000	-0.086538	-0.060072	-0.016619	-0.038847	-0.012302	NaN	-0.0
07-Jan-2000	0.047368	0.061013	0.0587	-0.037708	-0.000964	NaN	0.0

Notice that GOOG has missing data (NaN) because it was not listed before Aug 2004. The `simulateNormalScenariosByData` function has a name-value pair argument `'MissingData'` that indicates with a Boolean value whether to use the missing data capabilities of Financial Toolbox™ software. The default value for `'MissingData'` is `false` which removes all samples with NaN values. If, however, `'MissingData'` is set to `true`, the `estimateAssetMoments` function uses the ECM algorithm to estimate asset moments. The `simulateNormalScenariosByData` function also accepts a name-value pair argument `'DataFormat'` with a corresponding value set to `'prices'` to indicate that the input to the function is in the form of asset prices and not returns (the default value for the `'DataFormat'` argument is `'returns'`).

```
NumScenarios = 100;
r = PortfolioCVaR;
r = simulateNormalScenariosByData(r,AssetsTimeTable,NumScenarios,'DataFormat','Returns','MissingData',true);
```

In addition, `simulateNormalScenariosByData` extracts asset names or identifiers from a timetable object when the name-value argument `'GetAssetList'` is set to `true` (its default value is `false`). If the `'GetAssetList'` value is `true`, the timetable column identifiers are used to set the `AssetList` property of the `PortfolioCVaR` object. To show this, the formation of the `PortfolioCVaR` object `r` is repeated with the `'GetAssetList'` flag set to `true`.

```
r = simulateNormalScenariosByData(r,AssetsTimeTable,NumScenarios,'GetAssetList',true);
disp(r.AssetList)

{'AAPL'}    {'AMZN'}    {'CSCO'}    {'DELL'}    {'EBAY'}    {'GOOG'}    {'HPQ'}    {'IBM'}
```

Estimate Mean and Covariance of Asset Returns from Market Data for a PortfolioCVaR Object

Create a `PortfolioCVaR` object `p` and use the `simulateNormalScenariosByData` function with market data loaded from `CAPMuniverse.mat` to simulate multivariate normal asset return scenarios. The market data, `AssetsTimeTable`, is a timetable of asset returns.

```
load CAPMuniverse

p = PortfolioCVaR('AssetList',Assets);
disp(p);
```

PortfolioCVaR with properties:

```
BuyCost: []
SellCost: []
RiskFreeRate: []
ProbabilityLevel: []
Turnover: []
BuyTurnover: []
SellTurnover: []
NumScenarios: []
```

```

        Name: []
        NumAssets: 14
        AssetList: {'AAPL' 'AMZN' 'CSCO' 'DELL' 'EBAY' 'GOOG' 'HPQ' 'IBM' 'JNJ' 'KO' 'MSFT' 'PFE' 'TSLA'}
        InitPort: []
        AInequality: []
        bInequality: []
        AEquality: []
        bEquality: []
        LowerBound: []
        UpperBound: []
        LowerBudget: []
        UpperBudget: []
        GroupMatrix: []
        LowerGroup: []
        UpperGroup: []
        GroupA: []
        GroupB: []
        LowerRatio: []
        UpperRatio: []
        MinNumAssets: []
        MaxNumAssets: []
        ConditionalBudgetThreshold: []
        ConditionalUpperBudget: []
        BoundType: []

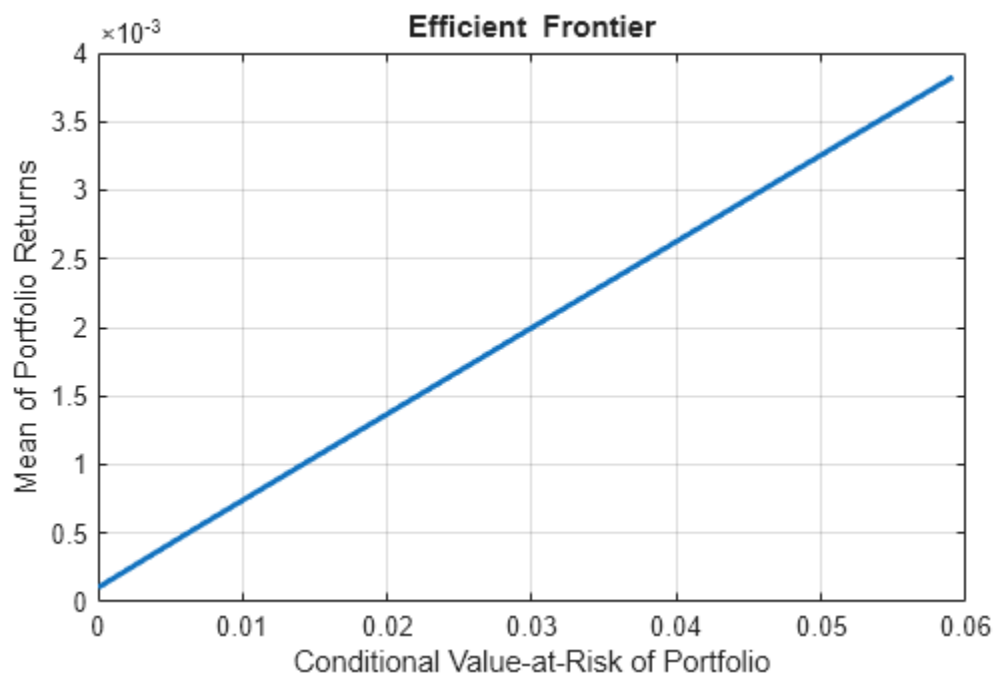
```

Simulate the scenarios from the timetable data for each of the assets from `CAPMuniverse.mat` and plot the efficient frontier.

```

p = simulateNormalScenariosByData(p,AssetsTimeTable,10000,'missingdata',true);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.9);
plotFrontier(p);

```



Estimate Mean and Covariance of Asset Returns from Data for a PortfolioMAD Object

Given a PortfolioMAD object `p`, use the `simulateNormalScenariosByData` function to simulate multivariate normal asset return scenarios from data.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

RawData = mvnrnd(m, C, 240);
NumScenarios = 2000;

p = PortfolioMAD;
p = simulateNormalScenariosByData(p, RawData, NumScenarios);
p = setDefaultConstraints(p);

disp(p);
```

PortfolioMAD with properties:

```
          BuyCost: []
          SellCost: []
    RiskFreeRate: []
          Turnover: []
        BuyTurnover: []
        SellTurnover: []
    NumScenarios: 2000
           Name: []
        NumAssets: 4
        AssetList: []
         InitPort: []
    AInequality: []
    bInequality: []
        AEquality: []
        bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
        GroupA: []
        GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
          BoundType: [4×1 categorical]
```

Estimate Mean and Covariance of Asset Returns from Market Data for a PortfolioMAD Object

Create a PortfolioMAD object `p` and use the `simulateNormalScenariosByData` function with market data loaded from `CAPMuniverse.mat` to simulate multivariate normal asset return scenarios. The market data, `AssetsTimeTable`, is a timetable of asset returns.

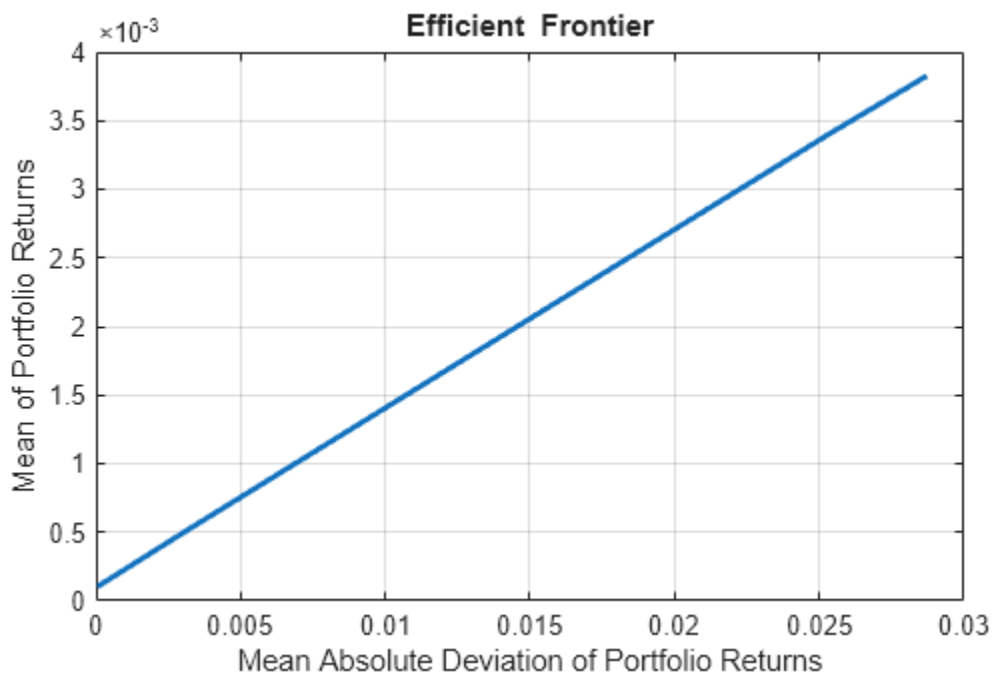
```
load CAPMuniverse

p = PortfolioMAD('AssetList',Assets);
disp(p.AssetList');

{'AAPL' }
{'AMZN' }
{'CSCO' }
{'DELL' }
{'EBAY' }
{'GOOG' }
{'HPQ' }
{'IBM' }
{'INTC' }
{'MSFT' }
{'ORCL' }
{'YHOO' }
{'MARKET'}
{'CASH' }
```

Simulate the scenarios from the timetable data for each of the assets from `CAPMuniverse.mat` and plot the efficient frontier.

```
p = simulateNormalScenariosByData(p,AssetsTimeTable,10000,'missingdata',true);
p = setDefaultConstraints(p);
plotFrontier(p);
```



Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a PortfolioCVaR or PortfolioMAD object.

For more information on creating a PortfolioCVaR or PortfolioMAD object, see

- PortfolioCVaR
- PortfolioMAD

Data Types: object

AssetReturns — Asset data that can be converted into asset returns

matrix | table | timetable

Asset data that can be converted into asset returns ([NumSamples-by-NumAssets] matrix), specified as a matrix, table, or timetable.

AssetReturns data can be:

- NumSamples-by-NumAssets matrix.
- Table of NumSamples prices or returns at a given periodicity for a collection of NumAssets assets
- Timetable object with NumSamples observations and NumAssets time series

Data Types: double | table | timetable

NumScenarios — Number of scenarios to simulate

positive integer

Number of scenarios to simulate, specified as a positive integer.

Data Types: `double`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `p =`

```
simulateNormalScenariosByData(p,RawData,NumScenarios,'DataFormat','Returns','MissingData',true,'GetAssetList',true)
```

DataFormat — Flag to convert input data as prices into returns

`'Returns'` (default) | character vector with values `'Returns'` or `'Prices'`

Flag to convert input data as prices into returns, specified as the comma-separated pair consisting of `'DataFormat'` and a character vector with the values:

- `'Returns'` — Data in `AssetReturns` contains asset total returns.
- `'Prices'` — Data in `AssetReturns` contains asset total return prices.

Data Types: `char`

MissingData — Flag to use ECM algorithm to handle NaN values

`false` (default) | logical with values `true` or `false`

Flag to use ECM algorithm to handle NaN values, specified as the comma-separated pair consisting of `'MissingData'` and a logical with a value of `true` or `false`.

- `false` — Do not use ECM algorithm to handle NaN values (exclude NaN values).
- `true` — Use ECM algorithm to handle NaN values.

Data Types: `logical`

GetAssetList — Flag indicating which asset names to use for the asset list

`false` (default) | logical with values `true` or `false`

Flag indicating which asset names to use for the asset list, specified as the comma-separated pair consisting of `'GetAssetList'` and a logical with a value of `true` or `false`.

- `false` — Do not extract or create asset names.
- `true` — Extract or create asset names from the table or timetable.

If a `table` or `timetable` is passed into this function using the `AssetReturns` argument and the `GetAssetList` flag is `true`, the column names from the `table` or `timetable` are used as asset names in `obj.AssetList`.

If a `matrix` is passed and the `GetAssetList` flag is `true`, default asset names are created based on the `AbstractPortfolio` property `defaultforAssetList`, which is `'Asset'`.

If the `GetAssetList` flag is `false`, no action occurs, which is the default behavior.

Data Types: `logical`

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `PortfolioCVaR` or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `PortfolioCVaR`
- `PortfolioMAD`

Tips

You can also use dot notation to simulate multivariate normal asset return scenarios from data for a `PortfolioCVaR` or `PortfolioMAD` object.

```
obj = obj.simulateNormalScenariosByData(AssetReturns,NumScenarios,Name,Value);
```

Version History

Introduced in R2012b

R2023a: `fints` support for `AssetReturns` argument removed

Behavior changed in R2023a

The `AssetReturns` argument no longer supports a `fints` object as a datatype.

See Also

`simulateNormalScenariosByMoments` | `rng`

Topics

“Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36

“Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

simulateNormalScenariosByMoments

Simulate multivariate normal asset return scenarios from mean and covariance of asset returns

Syntax

```
obj = simulateNormalScenariosByMoments(obj,AssetMean,AssetCovar,NumScenarios)
obj = simulateNormalScenariosByMoments(obj,AssetMean,AssetCovarNumScenarios,
NumAssets)
```

Description

`obj = simulateNormalScenariosByMoments(obj,AssetMean,AssetCovar,NumScenarios)` simulates multivariate normal asset return scenarios from mean and covariance of asset returns for `PortfolioCVaR` or `PortfolioMAD` objects. For details on the workflows, see “PortfolioCVaR Object Workflow” on page 5-16, and “PortfolioMAD Object Workflow” on page 6-16.

`obj = simulateNormalScenariosByMoments(obj,AssetMean,AssetCovarNumScenarios,NumAssets)` simulates multivariate normal asset return scenarios from mean and covariance of asset returns for `PortfolioCVaR` or `PortfolioMAD` objects using the optional input `NumScenarios`.

Note This function overwrites existing scenarios associated with `PortfolioCVaR` or `PortfolioMAD` objects, and also, possibly, `NumScenarios`.

If you want to use the function multiple times and you want to simulate identical scenarios each time the function is called, precede each function call with `rng(seed)` using a specified integer seed.

Examples

Simulate Multivariate Normal Asset Return Scenarios from Moments for a PortfolioCVaR Object

Given `PortfolioCVaR` object `p`, use the `simulateNormalScenariosByMoments` function to simulate multivariate normal asset return scenarios from moments.

```
m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioCVaR;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);
p = setProbabilityLevel(p, 0.95);
```

```
AssetMean = [.5]

AssetMean =
0.5000

AssetCovar = [.5]

AssetCovar =
0.5000

NumScenarios = 100

NumScenarios =
100

p = simulateNormalScenariosByMoments(p, AssetMean, AssetCovar, NumScenarios)

p =
PortfolioCVaR with properties:

    BuyCost: []
    SellCost: []
    RiskFreeRate: []
    ProbabilityLevel: 0.9500
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: 100
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4×1 double]
    UpperBound: []
    LowerBudget: 1
    UpperBudget: 1
    GroupMatrix: []
    LowerGroup: []
    UpperGroup: []
    GroupA: []
    GroupB: []
    LowerRatio: []
    UpperRatio: []
    MinNumAssets: []
    MaxNumAssets: []
    ConditionalBudgetThreshold: []
    ConditionalUpperBudget: []
    BoundType: [4×1 categorical]
```

Simulate Multivariate Normal Asset Return Scenarios from Moments for a PortfolioMAD Object

Given PortfolioMAD object p, use the simulateNormalScenariosByMoments function to simulate multivariate normal asset return scenarios from moments.

```

m = [ 0.05; 0.1; 0.12; 0.18 ];
C = [ 0.0064 0.00408 0.00192 0;
      0.00408 0.0289 0.0204 0.0119;
      0.00192 0.0204 0.0576 0.0336;
      0 0.0119 0.0336 0.1225 ];
m = m/12;
C = C/12;

AssetScenarios = mvnrnd(m, C, 20000);

p = PortfolioMAD;
p = setScenarios(p, AssetScenarios);
p = setDefaultConstraints(p);

AssetMean = [.5]

AssetMean =
0.5000

AssetCovar = [.5]

AssetCovar =
0.5000

NumScenarios = 100

NumScenarios =
100

p = simulateNormalScenariosByMoments(p, AssetMean, AssetCovar, NumScenarios)

p =
PortfolioMAD with properties:
    BuyCost: []
    SellCost: []
RiskFreeRate: []
    Turnover: []
    BuyTurnover: []
    SellTurnover: []
    NumScenarios: 100
    Name: []
    NumAssets: 4
    AssetList: []
    InitPort: []
    AInequality: []
    bInequality: []
    AEquality: []
    bEquality: []
    LowerBound: [4x1 double]
    UpperBound: []
    LowerBudget: 1

```

```
UpperBudget: 1
GroupMatrix: []
LowerGroup: []
UpperGroup: []
GroupA: []
GroupB: []
LowerRatio: []
UpperRatio: []
MinNumAssets: []
MaxNumAssets: []
ConditionalBudgetThreshold: []
ConditionalUpperBudget: []
BoundType: [4x1 categorical]
```

Input Arguments

obj — Object for portfolio

object

Object for portfolio, specified using a PortfolioCVaR or PortfolioMAD object.

For more information on creating a PortfolioCVaR or PortfolioMAD object, see

- PortfolioCVaR
- PortfolioMAD

Data Types: object

AssetMean — Mean of asset returns

vector

Mean of asset returns, specified as a vector.

Note If AssetMean is a scalar and the number of assets is known, scalar expansion occurs. If the number of assets cannot be determined, this function assumes that NumAssets = 1.

Data Types: double

AssetCovar — Covariance of asset returns

symmetric positive semidefinite matrix

Covariance of asset returns, specified as a symmetric positive semidefinite matrix.

Note

- If AssetCovar is a scalar and the number of assets is known, a diagonal matrix is formed with the scalar value along the diagonals. If it is not possible to determine the number of assets, this method assumes that NumAssets = 1.
- If AssetCovar is a vector, a diagonal matrix is formed with the vector along the diagonal.

- If `AssetCovar` is not a symmetric positive semidefinite matrix, use `nearcorr` to create a positive semidefinite matrix for a correlation matrix.

Data Types: double

NumScenarios — Number of scenarios to simulate

positive integer

Number of scenarios to simulate, specified as a positive integer.

Data Types: double

NumAssets — Number of assets

scalar

Number of assets, specified as a scalar.

Data Types: double

Output Arguments

obj — Updated portfolio object

object for portfolio

Updated portfolio object, returned as a `PortfolioCVaR` or `PortfolioMAD` object. For more information on creating a portfolio object, see

- `PortfolioCVaR`
- `PortfolioMAD`

Tips

You can also use dot notation to simulate multivariate normal asset return scenarios from a mean and covariance of asset returns for a `PortfolioCVaR` or `PortfolioMAD` object.

```
obj = obj.simulateNormalScenariosByMoments(AssetMean, AssetCovar, NumScenarios, NumAssets);
```

Version History

Introduced in R2012b

See Also

`simulateNormalScenariosByData` | `rng` | `nearcorr` | `covarianceShrinkage` | `covarianceDenoising`

Topics

“Asset Returns and Scenarios Using `PortfolioCVaR` Object” on page 5-36

“Asset Returns and Scenarios Using `PortfolioMAD` Object” on page 6-35

External Websites

Getting Started with Portfolio Optimization (4 min 13 sec)

CVaR Portfolio Optimization (4 min 56 sec)

sharpe

Compute Sharpe ratio for one or more assets

Syntax

```
sharpe(Asset)
sharpe(Asset,Cash)
Ratio = sharpe(Asset,Cash)
```

Description

`sharpe(Asset)` computes Sharpe ratio for each asset.

`sharpe(Asset,Cash)` computes Sharpe ratio for each asset including the optional argument `Cash`.

`Ratio = sharpe(Asset,Cash)` computes Sharpe ratio for each asset including the optional argument `Cash`.

Examples

Compute Sharpe Ratio

This example shows how to compute the Sharpe ratio using the mean return of a cash asset as the return for the riskless asset.

Given asset return data and the riskless asset return, the Sharpe ratio is calculated:

```
load FundMarketCash
Returns = tick2ret(TestData);
Riskless = mean>Returns(:,3))

Riskless =
0.0017

Sharpe = sharpe>Returns, Riskless)

Sharpe = 1×3
    0.0886    0.0315    0
```

The Sharpe ratio of the example fund is significantly higher than the Sharpe ratio of the market. As is demonstrated with `portalalpha`, this translates into a strong risk-adjusted return. Since the `Cash` asset is the same as `Riskless`, it makes sense that its Sharpe ratio is 0. The Sharpe ratio is calculated with the mean of cash returns. The Sharpe ratio can also be calculated with the cash return series as input for the riskless asset.

```
Sharpe = sharpe>Returns, Returns(:,3))

Sharpe = 1×3
```

0.0886 0.0315 0

When using the `Portfolio` object, you can use the `estimateMaxSharpeRatio` function to estimate an efficient portfolio that maximizes the Sharpe ratio. For more information, see “Efficient Portfolio That Maximizes Sharpe Ratio” on page 4-111.

Input Arguments

Asset — Asset returns

matrix

Asset returns, specified as a `NUMSAMPLES` × `NUMSERIES` matrix with `NUMSAMPLES` observations of asset returns for `NUMSERIES` asset return series.

Data Types: `double`

Cash — Riskless asset

0 (default) | numeric | vector

(Optional) Riskless asset, specified as either a scalar return for a riskless asset or a vector of asset returns to be a proxy for a “riskless” asset. In either case, the periodicity must be the same as the periodicity of `Asset`. For example, if `Asset` is monthly data, then `Cash` must be monthly returns. If no value is supplied, the default value for `Cash` returns is 0.

Data Types: `double`

Output Arguments

Ratio — Sharpe ratios

vector

Sharpe ratios, returned as a 1-by-`NUMSERIES` row vector of Sharpe ratios for each series in `Asset`. Any series in `Asset` with standard deviation of returns equal to 0 has a `NaN` value for its Sharpe ratio.

Note If `Cash` is a vector, `Asset` and `Cash` need not have the same number of returns but must have the same periodicity of returns. The classic Sharpe ratio assumes that `Cash` is riskless. In reality, a short-term cash rate is not necessarily riskless. `NaN` values in the data are ignored.

More About

Sharpe Ratio

The Sharpe ratio is a widely used financial metric that measures the risk-adjusted return of an investment or portfolio.

Use the Sharpe ratio to evaluate the performance of an investment relative to its risk. The Sharpe ratio helps investors understand how much excess return they are receiving for the additional volatility or risk taken compared to a risk-free asset.

Version History

Introduced in R2006b

References

[1] Sharpe, W. F. "Mutual Fund Performance." *Journal of Business*. Vol. 39, No. 1, Part 2, January 1966, pp. 119-138.

See Also

inforatio | portalpha

Topics

"Performance Metrics Illustration" on page 7-3

"Performance Metrics Overview" on page 7-2

stochosc

Stochastic oscillator

Syntax

```
percentKnD = stochosc(Data)
percentKnD = stochosc( ___,Name,Value)
```

Description

`percentKnD = stochosc(Data)` calculates the stochastic oscillator.

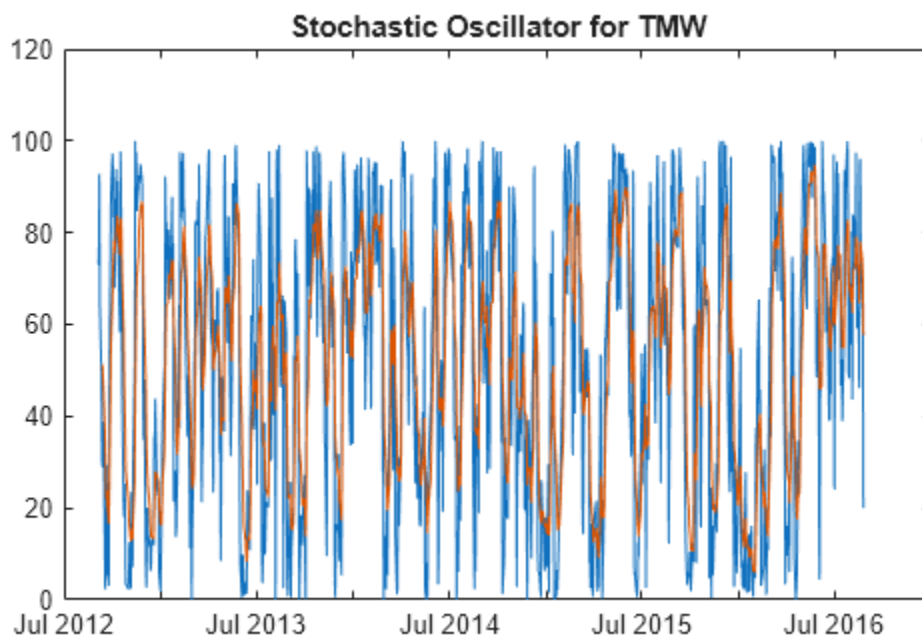
`percentKnD = stochosc( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Stochastic Oscillator for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
oscillator = stochosc(TMW,'NumPeriodsD',7,'NumPeriodsK',10,'Type','exponential');
plot(oscillator.Time,oscillator.FastPercentK,oscillator.Time,oscillator.FastPercentD)
title('Stochastic Oscillator for TMW')
```



Input Arguments

Data — Data with high, low, open, close information

matrix | table | timetable

Data with high, low, open, close information, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-3 matrix of high, low, and closing prices stored in the corresponding columns, respectively. Timetables and tables with M rows must contain variables named 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `percentKnD =
stochosc(TMW, 'NumPeriodsD', 10, 'NumPeriodsK', 3, 'Type', 'exponential')`

NumPeriodsK — Period difference for PercentK

10 (default) | positive integer

Period difference for PercentK, specified as the comma-separated pair consisting of 'NumPeriodsK' and a scalar positive integer.

Data Types: double

NumPeriodsD — Length of moving average in periods for PercentD

3 (default) | positive integer

Length of moving average in periods for PercentD, specified as the comma-separated pair consisting of 'NumPeriodsD' and a scalar positive integer.

Data Types: double

Type — Moving average method for PercentD calculation

'e' (exponential) (default) | character vector with values 'exponential' or 'triangular'

Moving average method for PercentD calculation, specified as the comma-separated pair consisting of 'Type' and a character vector with a value of:

- 'exponential' - Exponential moving average is a weighted moving average. Exponential moving averages reduce the lag by applying more weight to recent prices. For example, a 10 period exponential moving average weights the most recent price by 18.18%.
- 'triangular' - Triangular moving average is a double-smoothing of the data. The first simple moving average is calculated and then a second simple moving average is calculated on the first moving average with the same window size.

Data Types: char

Output Arguments

percentKnD — PercentK and PercentD

matrix | table | timetable

PercentK and PercentD, returned with the same number of rows (M) and type (matrix, table, or timetable) as the input Data.

More About

Stochastic Oscillator

The stochastic oscillator is a momentum indicator used in technical analysis to measure the level of a security's closing price relative to its price range over a specific period.

The stochastic oscillator calculates the Fast PercentK (F%K), Fast PercentD (F%D), Slow PercentK (S%K), and Slow PercentD (S%D) from the series of high, low, and closing stock prices.

By default, the stochastic oscillator is based on 10-period difference for PercentK and a 3-period exponential moving average for PercentD.

The the stochastic oscillator is primarily used to identify overbought or oversold conditions in the market, helping traders make informed decisions about buying or selling assets.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 268-271.

See Also

timetable | table | chaikosc

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

targetreturn

Portfolio weight accuracy

Syntax

```
return = targetreturn(Universe,Window,Offset,Weights)
```

Description

`return = targetreturn(Universe,Window,Offset,Weights)` computes target return values for each Window of data and given portfolio weights. These values should match the input target return used with `selectreturn`.

Input Arguments

Universe — Total return data for a group of securities

array

Total return data for a group of securities, specified as a number of observations (NUMOBS) by number of assets plus one (NASSETS + 1) array. Each row represents an observation. Column 1 contains MATLAB serial date numbers. The remaining columns contain the total return data for each security.

Data Types: double

Window — Number of data periods used to calculate frontier

integer

Number of data periods used to calculate frontier, specified as an integer.

Data Types: double

Offset — Number of periods to increment when each frontier is generated

integer

Number of periods to increment when each frontier is generated , specified as an integer.

Data Types: double

Weights — Asset allocation weights needed to obtain the target rate of return

matrix

Asset allocation weights needed to obtain the target rate of return, specified as a number of assets (NASSETS) by number of curves (NCURVES) matrix.

Data Types: double

Output Arguments

return — Target return

numeric

Target return, returned as a numeric value for each Window of data and given portfolio Weights.

Version History

Introduced before R2006a

See Also

`selectreturn` | `frontier` | `portopt`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

taxedrr

After-tax rate of return

Syntax

Return = taxedrr(PreTaxReturn,TaxRate)

Description

Return = taxedrr(PreTaxReturn,TaxRate) calculates the after-tax rate of return.

Examples

Calculate the After-Tax Rate of Return

This example shows how to calculate the after-tax rate of return, given an investment that has a 12% nominal rate of return and is taxed at a 30% rate.

```
Return = taxedrr(0.12, 0.30)
```

```
Return =  
0.0840
```

Input Arguments

PreTaxReturn — Normal rate of return

decimal

Normal rate of return, specified as a decimal.

Data Types: double

TaxRate — Tax rate

decimal

Tax rate, specified as a decimal.

Data Types: double

Output Arguments

Return — After-tax rate of return

decimal

After-tax rate of return, returned as a decimal.

Version History

Introduced before R2006a

See Also

`effrr` | `irr` | `mirr` | `nomrr` | `xirr`

Topics

“Analyzing and Computing Cash Flows” on page 2-11

tbilldisc2yield

Convert Treasury bill discount to equivalent yield

Syntax

```
[BEYield,MMYield] = tbilldisc2yield(Discount,Settle,Maturity)
```

Description

[BEYield,MMYield] = tbilldisc2yield(Discount,Settle,Maturity) converts the discount rate on Treasury bills into their respective money-market or bond-equivalent yields.

Examples

Convert the Discount Rate on Treasury Bills

This example shows how to convert the discount rate on Treasury bills into their respective money-market or bond-equivalent yields, given a Treasury bill with the following characteristics.

```
Discount = 0.0497;
Settle = '01-Oct-02';
Maturity = '31-Mar-03';
```

```
[BEYield MMYield] = tbilldisc2yield(Discount, Settle, Maturity)
```

```
BEYield =
0.0517
```

```
MMYield =
0.0510
```

Convert the Discount Rate on Treasury Bills Using datetime Inputs

This example shows how to use `datetime` inputs to convert the discount rate on Treasury bills into their respective money-market or bond-equivalent yields, given a Treasury bill with the following characteristics.

```
Discount = 0.0497;
Settle = datetime(2002,10,1);
Maturity = datetime(2003,3,31);
[BEYield MMYield] = tbilldisc2yield(Discount, Settle, Maturity)
```

```
BEYield =
0.0517
```

```
MMYield =
0.0510
```

Input Arguments

Discount — Discount rate of Treasury bills

decimal

Discount rate of the Treasury bills, specified as a scalar of a NTBILLS-by-1 vector of decimal values. The discount rate basis is actual/360.

Data Types: double

Settle — Settlement date of Treasury bill

datetime array | string array | date character vector

Settlement date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, `tbilldisc2yield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of Treasury bill

datetime array | string array | date character vector

Maturity date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `tbilldisc2yield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

BEYield — Bond equivalent yields of Treasury bills

numeric

Bond equivalent yields of the Treasury bills, returned as a NTBILLS-by-1 vector. The bond-equivalent yield basis is actual/365.

MMYield — Money-market yields of Treasury bills

numeric

Money-market yields of the Treasury bills, returned as a NTBILLS-by-1 vector. The money-market yield basis is actual/360.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tbilldisc2yield` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] *SIA Fixed Income Securities Formulas for Price, Yield, and Accrued Interest*. Volume 1, 3rd edition, pp. 44-45.
- [2] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`tbillyield2disc` | `zeroyield` | `datetime`

Topics

- "Computing Treasury Bill Price and Yield" on page 2-26
- "Bond Portfolio Optimization Using Portfolio Object" on page 10-30
- "Treasury Bills Defined" on page 2-25

tbillprice

Price Treasury bill

Syntax

```
Price = tbillprice(Rate,Settle,Maturity)
Price = tbillprice( ____,Type)
```

Description

Price = tbillprice(Rate,Settle,Maturity) computes the price of a Treasury bill given a yield or discount rate.

Price = tbillprice(____,Type) adds an optional argument for Type.

Examples

Compute the Price of a Treasury Bill Using the Bond-Equivalent Yield

Given a Treasury bill with the following characteristics, compute the price of the Treasury bill using the bond-equivalent yield (Type = 2) as input.

```
Rate = 0.045;
Settle = '01-Oct-02';
Maturity = '31-Mar-03';
```

```
Type = 2;
```

```
Price = tbillprice(Rate, Settle, Maturity, Type)
```

```
Price =
97.8172
```

Price a Portfolio of Treasury Bills

Use tbillprice to price a portfolio of Treasury bills.

```
Rate = [0.045; 0.046];
Settle = {'02-Jan-02'; '01-Mar-02'};
Maturity = {'30-June-02'; '30-June-02'};
Type = [2 3];
```

```
Price = tbillprice(Rate, Settle, Maturity, Type)
```

```
Price = 2×1
```

```
97.8408
98.4539
```

Price a Portfolio of Treasury Bills Using datetime Input

Use `tbillprice` to price a portfolio of Treasury bills using `datetime` input.

```
Rate = [0.045; 0.046];
Type = [2 3];

Settle = [datetime(2002,1,2);datetime(2002,3,1)];
Maturity = [datetime(2002,6,30);datetime(2002,6,30)];
Price = tbillprice(Rate, Settle, Maturity, Type)

Price = 2×1

    97.8408
    98.4539
```

Input Arguments

Rate — Bond-equivalent yield, money-market yield, or discount rate

decimal

Bond-equivalent yield, money-market yield, or discount rate (defined by the input `Type`), specified as a scalar or a `NTBILLS`-by-1 vector of decimal values.

Data Types: `double`

Settle — Settlement date of Treasury bill

`datetime` array | `string` array | `date` character vector

Settlement date of the Treasury bill, specified as a scalar or a `NTBILLS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `tbillprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of Treasury bill

`datetime` array | `string` array | `date` character vector

Maturity date of the Treasury bill, specified as a scalar or a `NTBILLS`-by-1 vector using a `datetime` array, `string` array, or `date` character vectors.

To support existing code, `tbillprice` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Type — (Optional) Rate type

2 (default) | numeric with values 1 = money market, 2 = bond-equivalent, 3 = discount rate

Rate type (determines how to interpret values entered in `Rate`), specified as a numeric value of 1, 2, or 3 using a scalar or a `NTBILLS`-by-1 vector.

Note The bond-equivalent yield basis is actual/365. The money-market yield basis is actual/360. The discount rate basis is actual/360.

Data Types: double

Output Arguments

Price — Treasury bill price for every \$100 face

numeric

Treasury bill prices for every \$100 face, returned as a NTBILLS-by-1 vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tbillprice` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] *SIA Fixed Income Securities Formulas for Price, Yield, and Accrued Interest*. Volume 1, 3rd edition, pp. 44–45.
- [2] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`tbillyield` | `zeroprice` | `datetime`

Topics

“Computing Treasury Bill Price and Yield” on page 2-26

“Bond Portfolio Optimization Using Portfolio Object” on page 10-30

“Treasury Bills Defined” on page 2-25

tbillrepo

Break-even discount of repurchase agreement

Syntax

```
TBEDiscount = tbillrepo(RepoRate,InitialDiscount,PurchaseDate,SaleDate,
Maturity)
```

Description

TBEDiscount = tbillrepo(RepoRate,InitialDiscount,PurchaseDate,SaleDate,Maturity) computes the true break-even discount of a repurchase agreement.

Examples

Compute the True Break-Even Discount

This example shows how to compute the true break-even discount of a Treasury bill repurchase agreement.

```
RepoRate = [0.045; 0.0475];
InitialDiscount = 0.0475;
PurchaseDate = '3-Jan-2002';
SaleDate = '3-Feb-2002';
Maturity = '3-Apr-2002';

TBEDiscount = tbillrepo(RepoRate, InitialDiscount,...
PurchaseDate, SaleDate, Maturity)

TBEDiscount = 2×1

    0.0491
    0.0478
```

Compute the True Break-Even Discount Using datetime Inputs

This example shows how to use datetime inputs to compute the true break-even discount of a Treasury bill repurchase agreement.

```
RepoRate = [0.045; 0.0475];
InitialDiscount = 0.0475;
PurchaseDate = datetime(2002,1,3);
SaleDate = datetime(2002,2,3);
Maturity = datetime(2002,4,3);
TBEDiscount = tbillrepo(RepoRate, InitialDiscount,...
PurchaseDate, SaleDate, Maturity)
```

```
TBEDiscount = 2×1
```

```
0.0491  
0.0478
```

Input Arguments

RepoRate — Annualized, 360-day based repurchase rate

decimal

Annualized, 360-day based repurchase rate, specified as a scalar of a NTBILLS-by-1 vector of decimal values.

Data Types: double

InitialDiscount — Discount on Treasury bill on day of purchase

decimal

Discount on the Treasury bill on the day of purchase, specified as a scalar of a NTBILLS-by-1 vector of decimal values.

Data Types: double

PurchaseDate — Date Treasury bill is purchased

datetime array | string array | date character vector

Date the Treasury bill is purchased, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `tbillrepo` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

SaleDate — Date Treasury bill repurchase term is due

datetime array | string array | date character vector

Date the Treasury bill repurchase term is due, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `tbillrepo` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of Treasury bill

datetime array | string array | date character vector

Maturity date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `tbillrepo` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

TBEDiscount — True break-even discount of repurchase agreement

numeric

True break-even discount of a repurchase agreement, returned as a scalar or NTBILLS-by-1 vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tbillrepo` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] *SIA Fixed Income Securities Formulas for Price, Yield, and Accrued Interest*. Volume 1, 3rd edition, pp. 44–45.
- [2] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`tbillyield` | `tbillprice` | `tbillval01` | `datetime`

Topics

- “Computing Treasury Bill Price and Yield” on page 2-26
- “Bond Portfolio Optimization Using Portfolio Object” on page 10-30
- “Treasury Bills Defined” on page 2-25

tbillval01

Value of one basis point

Syntax

```
[Val01Disc,Val01MMY,Val01BEY] = tbillval01(Settle,Maturity)
```

Description

`[Val01Disc,Val01MMY,Val01BEY] = tbillval01(Settle,Maturity)` calculates the value of one basis point of \$100 Treasury bill face value on the discount rate, money-market yield, or bond-equivalent yield.

Examples

Compute the Value of One Basis Point

This example shows how to compute the value of one basis point, given a Treasury bill with the following settle and maturity dates.

```
Settle = '01-Mar-03';  
Maturity = '30-June-03';  
[Val01Disc, Val01MMY, Val01BEY] = tbillval01(Settle, Maturity)  
  
Val01Disc =  
0.0034  
  
Val01MMY =  
0.0034  
  
Val01BEY =  
0.0033
```

Compute the Value of One Basis Point Using datetime Inputs

This example shows how to use `datetime` inputs to compute the value of one basis point, given a Treasury bill with the following settle and maturity dates.

```
Settle = datetime(2003,3,1);  
Maturity = datetime(2003,6,30);  
[Val01Disc, Val01MMY, Val01BEY] = tbillval01(Settle, Maturity)  
  
Val01Disc =  
0.0034  
  
Val01MMY =  
0.0034
```

```
Val01BEY =  
0.0033
```

Input Arguments

Settle — Settlement date of Treasury bill

datetime array | string array | date character vector

Settlement date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, `tbillval01` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of Treasury bill

datetime array | string array | date character vector

Maturity date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `tbillval01` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

Val01Disc — Value of one basis point of discount rate for every \$100 face

numeric

Value of one basis point of discount rate for every \$100 face, returned as a NTBILLS-by-1 vector.

Val01MMY — Value of one basis point of money-market yield for every \$100 face

numeric

Value of one basis point of money-market yield for every \$100 face, returned as a NTBILLS-by-1 vector.

Val01BEY — Value of one basis point of bond-equivalent yield for every \$100 face

numeric

Value of one basis point of bond-equivalent yield for every \$100 face, returned as a NTBILLS-by-1 vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tbillval01` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =  
  
2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] *SIA Fixed Income Securities Formulas for Price, Yield, and Accrued Interest*. Volume 1, 3rd edition, pp. 44-45.
- [2] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`tbillyield2disc` | `zeroyield` | `datetime`

Topics

- "Computing Treasury Bill Price and Yield" on page 2-26
- "Bond Portfolio Optimization Using Portfolio Object" on page 10-30
- "Treasury Bills Defined" on page 2-25

tbillyield

Yield on Treasury bill

Syntax

```
[MMYield,BEYield,Discount] = tbillyield(Price,Settle,Maturity)
```

Description

[MMYield,BEYield,Discount] = tbillyield(Price,Settle,Maturity) computes the yield of US Treasury bills given Price, Settle, and Maturity.

Examples

Compute the Yield of U.S. Treasury Bills

This example shows how to compute the yield of U.S. Treasury bills, given a Treasury bill with the following characteristics.

```
Price = 98.75;
Settle = '01-Oct-02';
Maturity = '31-Mar-03';

[MMYield, BEYield, Discount] = tbillyield(Price, Settle,...
Maturity)

MMYield =
0.0252

BEYield =
0.0255

Discount =
0.0249
```

Compute the Yield of U.S. Treasury Bills Using datetime Inputs

This example shows how to use datetime inputs to compute the yield of U.S. Treasury bills, given a Treasury bill with the following characteristics.

```
Price = 98.75;
Settle = datetime('01-Oct-2002','Locale','en_US');
Maturity = datetime('31-Mar-2003','Locale','en_US');
[MMYield, BEYield, Discount] = tbillyield(Price, Settle,Maturity)

MMYield =
0.0252
```

```
BEYield =  
0.0255
```

```
Discount =  
0.0249
```

Input Arguments

Price — Price of Treasury bills for every \$100 face value

numeric

Price of Treasury bills for every \$100 face value, specified as a scalar of a NTBILLS-by-1 vector of decimal values.

Data Types: double

Settle — Settlement date of Treasury bill

datetime array | string array | date character vector

Settlement date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors. **Settle** must be earlier than **Maturity**.

To support existing code, `tbillyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of Treasury bill

datetime array | string array | date character vector

Maturity date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `tbillyield` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Output Arguments

MMYield — Money-market yields of Treasury bills

numeric

Money-market yields of the Treasury bills, returned as a NTBILLS-by-1 vector.

BEYield — Bond equivalent yields of Treasury bills

numeric

Bond equivalent yields of the Treasury bills, returned as a NTBILLS-by-1 vector.

Discount — Discount rates of Treasury bills

numeric

Discount rates of the Treasury bills, returned as a NTBILLS-by-1 vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tbillyield` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =

    2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.
- [2] Mayle, J. "Standard Securities Calculations Methods: Fixed Income Securities Formulas for Price, Yield, and Accrued Interest." SIA, Vol 1, 3rd edition, Jan 1993, pp. 44-45.
- [3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`zeroyield` | `tbillprice` | `tbilldisc2yield` | `tbillyield2disc` | `datetime`

Topics

- "Computing Treasury Bill Price and Yield" on page 2-26
- "Bond Portfolio Optimization Using Portfolio Object" on page 10-30
- "Treasury Bills Defined" on page 2-25

tbillyield2disc

Convert Treasury bill yield to equivalent discount

Syntax

```
Discount = tbillyield2disc(Yield,Settle,Maturity)
Discount = tbillyield2disc( ____,Type)
```

Description

`Discount = tbillyield2disc(Yield,Settle,Maturity)` converts the yield on some Treasury bills into their respective discount rates.

`Discount = tbillyield2disc( ____,Type)` adds an optional argument for Type.

Examples

Compute the Discount Rate on a Money-Market Basis

Given a Treasury bill with these characteristics, compute the discount rate.

```
Yield = 0.0497;
Settle = '01-Oct-02';
Maturity = '31-Mar-03';

Discount = tbillyield2disc(Yield,Settle,Maturity)

Discount =
0.0485
```

Compute the Discount Rate on a Money-Market Basis Using datetime Inputs

Given a Treasury bill with these characteristics, compute the discount rate using datetime inputs.

```
Yield = 0.0497;
Settle = datetime(2002,10,1);
Maturity = datetime(2003,3,31);

Discount = tbillyield2disc(Yield,Settle,Maturity)

Discount =
0.0485
```

Input Arguments

Yield — Yield of Treasury bills
decimal

Yield of Treasury bills, specified as a scalar or a NTBILLS-by-1 vector of decimal values.

Data Types: `double`

Settle — Settlement date of Treasury bill

`datetime array` | `string array` | `date character vector`

Settlement date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a `datetime` array, `string array`, or `date character vectors`.

To support existing code, `tbillyield2disc` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of Treasury bill

`datetime array` | `string array` | `date character vector`

Maturity date of the Treasury bill, specified as a scalar or a NTBILLS-by-1 vector using a `datetime` array, `string array`, or `date character vectors`.

To support existing code, `tbillyield2disc` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Type — Yield type

1 (default) | numeric with values 1 = money market, 2 = bond-equivalent

(Optional) Yield type (determines how to interpret values entered in `Yield`), specified as a numeric value of 1 or 2 using a scalar or a NTBILLS-by-1 vector.

Note The bond-equivalent yield basis is actual/365. The money-market yield basis is actual/360.

Data Types: `double`

Output Arguments

Discount — Discount rates of Treasury bills

`numeric`

Discount rates of the Treasury bills, returned as a NTBILLS-by-1 vector.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tbillyield2disc` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

[1] *SIA Fixed Income Securities Formulas for Price, Yield, and Accrued Interest*. Volume 1, 3rd edition, pp. 44–45.

[2] Krgin, D. *Handbook of Global Fixed Income Calculations*. Wiley, 2002.

[3] Stigum, M., Robinson, F. *Money Market and Bond Calculation*. McGraw-Hill, 1996.

See Also

`tbilldisc2yield` | `datetime`

Topics

“Computing Treasury Bill Price and Yield” on page 2-26

“Bond Portfolio Optimization Using Portfolio Object” on page 10-30

“Treasury Bills Defined” on page 2-25

tbl2bond

Treasury bond parameters given Treasury bill parameters

Syntax

```
[TBondMatrix,Settle] = tbl2bond(TBillMatrix)
```

Description

[TBondMatrix,Settle] = tbl2bond(TBillMatrix) restates US Treasury bill market parameters in US Treasury bond form as zero-coupon bonds. This function makes Treasury bills directly comparable to Treasury bonds and notes.

Examples

Restate U.S. Treasury Bill in U.S. Treasury Bond Form

This example shows how to restate U.S. Treasury bill market parameters in U.S. Treasury bond form, given published Treasury bill market parameters for December 22, 1997.

```
TBill = [datenum('jan 02 1998') 10 0.0526 0.0522 0.0530
         datenum('feb 05 1998') 44 0.0537 0.0533 0.0544
         datenum('mar 05 1998') 72 0.0529 0.0527 0.0540];
```

```
TBond = tbl2bond(TBill)
```

```
TBond = 3×5
105 ×
```

```
    0    7.2976    0.0010    0.0010    0.0000
    0    7.2979    0.0010    0.0010    0.0000
    0    7.2982    0.0010    0.0010    0.0000
```

Restate U.S. Treasury Bill in U.S. Treasury Bond Form Using datetime Input

This example shows how to use `datetime` input to restate U.S. Treasury bill market parameters in U.S. Treasury bond form, given published Treasury bill market parameters for December 22, 1997.

```
TBill = [datenum('jan 02 1998') 10 0.0526 0.0522 0.0530
         datenum('feb 05 1998') 44 0.0537 0.0533 0.0544
         datenum('mar 05 1998') 72 0.0529 0.0527 0.0540];
```

```
dates = datetime(TBill(:,1), 'ConvertFrom', 'datenum','Locale','en_US');
data = TBill(:,2:end);
t=[table(dates) array2table(data)];
[TBond, Settle] = tbl2bond(t)
```

TBond=3×5 *table*

CouponRate	Maturity	Bid	Asked	AskYield
0	02-Jan-1998	99.854	99.855	0.053
0	05-Feb-1998	99.344	99.349	0.0544
0	05-Mar-1998	98.942	98.946	0.054

Settle = 3×1 *datetime*
 22-Dec-1997
 22-Dec-1997
 22-Dec-1997

Input Arguments

TBillMatrix — Treasury bill parameters

table | matrix

Treasury bill parameters, specified as a 5-column table or a N-by-5 matrix of bond information where the table columns or matrix columns contains:

- **Maturity** (Required) Maturity date of Treasury bills, specified as a datetime, string, date character vector, or serial date number when using a matrix. If the input TBillMatrix is a table, the Maturity dates can be a datetime array, string array, date character vectors, or serial date numbers. You can use `datetime` to convert serial date numbers to datetime arrays.
- **DaysMaturity** (Required) Days to maturity, specified as an integer. Days to maturity are quoted on a skip-day basis; the actual number of days from settlement to maturity is `DaysMaturity + 1`.
- **Bid** (Required) Bid bank-discount rate (the percentage discount from face value at which the bill could be bought, annualized on a simple-interest basis), specified as a decimal fraction.
- **Asked** (Required) Asked bank-discount rate, specified as a decimal fraction.
- **AskYield** (Required) Asked yield (the bond-equivalent yield from holding the bill to maturity, annualized on a simple-interest basis and assuming a 365-day year), specified as a decimal fraction.

Data Types: double | char | string | datetime | table

Output Arguments

TBondMatrix — Treasury bond parameters

table | matrix

Treasury bond parameters, returned as a table or matrix depending on the TBillMatrix input.

When TBillMatrix is a table, TBondMatrix is also a table, and the variable type for the Maturity dates in TBondMatrix (column 1) matches the variable type for Maturity in TBillMatrix. For example, if Maturity dates are datetime arrays in TBillMatrix, they will also be datetime arrays in TBondMatrix.

When TBillMatrix input is a N-by-5 matrix, then each row describes a Treasury bond.

The parameters or columns returned for `TBondMatrix` are:

- **CouponRate** (Column 1) Coupon rate, which is always 0 since the Treasury bills are, by definition, a zero coupon instrument.
- **Maturity** (Column 2) Maturity date for each bond. The format of the dates matches the format used for **Maturity** in `TBillMatrix` (datetime array, string array, date character vector, or serial date number).
- **Bid** (Column 3) Bid price based on \$100 face value.
- **Asked** (Column 4) Asked price based on \$100 face value.
- **AskYield** (Column 5) Asked yield to maturity: the effective return from holding the bond to maturity, annualized on a compound-interest basis.

Settle — Settlement dates implied by maturity dates and number of days to maturity quote

serial date number | datetime

Settlement dates implied by the maturity dates and the number of days to maturity quote, returned as a N-by-5 vector containing serial date numbers, by default. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. **Settle** is returned as a datetime array only if the input `TBillMatrix` is a table containing datetime arrays for **Maturity** in the first column.

Version History

Introduced before R2006a

See Also

`tr2bonds` | `datetime`

Topics

“Term Structure of Interest Rates” on page 2-29

“Computing Treasury Bill Price and Yield” on page 2-26

“Bond Portfolio Optimization Using Portfolio Object” on page 10-30

“Treasury Bills Defined” on page 2-25

thirdwednesday

Find third Wednesday of month

Syntax

```
[BeginDates,EndDates] = thirdwednesday(Month,Year)
[BeginDates,EndDates] = thirdwednesday( ___,outputType)
```

Description

[BeginDates,EndDates] = thirdwednesday(Month,Year) computes the beginning and end period date for a LIBOR contract (third Wednesdays of delivery months).

[BeginDates,EndDates] = thirdwednesday(___,outputType), using optional input arguments, computes the beginning and end period date for a LIBOR contract (third Wednesdays of delivery months).

The type of the outputs depends on the input outputType. If this variable is 'datenum', BeginDates and EndDates are serial date numbers. If outputType is 'datetime', then BeginDates and EndDates are datetime arrays. By default, outputType is set to 'datenum'.

Examples

Determine the Third Wednesday for Given Months and Years

Find the third Wednesday dates for swaps commencing in the month of October in the years 2002, 2003, and 2004.

```
Months = [10; 10; 10];
Year = [2002; 2003; 2004];
[BeginDates, EndDates] = thirdwednesday(Months, Year);
datestr(BeginDates)
```

```
ans = 3×11 char array
    '16-Oct-2002'
    '15-Oct-2003'
    '20-Oct-2004'
```

```
datestr(EndDates)
```

```
ans = 3×11 char array
    '16-Jan-2003'
    '15-Jan-2004'
    '20-Jan-2005'
```

Find the third Wednesday dates for swaps commencing in the month of October in the years 2002, 2003, and 2004 using an outputType of 'datetime'.

```
Months = [10; 10; 10];
Year = [2002; 2003; 2004];
[BeginDates, EndDates] = thirdwednesday(Months, Year, 'datetime')
```

```
BeginDates = 3×1 datetime
    16-Oct-2002
    15-Oct-2003
    20-Oct-2004
```

```
EndDates = 3×1 datetime
    16-Jan-2003
    15-Jan-2004
    20-Jan-2005
```

Input Arguments

Month — Month of delivery for Eurodollar futures

integer from 1 through 12 | vector of integers from 1 through 12

Month of delivery for Eurodollar futures, specified as an N-by-1 vector of integers from 1 through 12.

Duplicate dates are returned when identical months and years are supplied.

Data Types: single | double

Year — Delivery year for Eurodollar futures/Libor contracts corresponding to Month

four-digit nonnegative integer | vector of four-digit nonnegative integers

Delivery year for Eurodollar futures/Libor contracts corresponding to Month, specified as an N-by-1 vector of four-digit nonnegative integers.

Duplicate dates are returned when identical months and years are supplied.

Data Types: single | double

outputType — Output date format

'datenum' (default) | character vector with values 'datenum' or 'datetime'

Output date format, specified as a character vector with values 'datenum' or 'datetime'. If outputType is 'datenum', then BeginDates and EndDates are serial date numbers. However, if outputType is 'datetime', then BeginDates and EndDates are datetime arrays.

Data Types: char

Output Arguments

BeginDates — Third Wednesday of given month and year

serial date number | date character vector

Third Wednesday of given month and year, returned as serial date numbers or date character vectors, or datetime arrays. This is also the beginning of the 3-month period contract.

The type of the outputs depends on the input `outputType`. If this variable is 'datenum', `BeginDates` and `EndDates` are serial date numbers. If `outputType` is 'datetime', then `BeginDates` and `EndDates` are datetime arrays. By default, `outputType` is set to 'datenum'.

EndDates — End of three-month period contract for given month and year

serial date number | date character vector

End of three-month period contract for given month and year, returned as serial date numbers or date character vectors, or datetime arrays.

The type of the outputs depends on the input `outputType`. If this variable is 'datenum', `BeginDates` and `EndDates` are serial date numbers. If `outputType` is 'datetime', then `BeginDates` and `EndDates` are datetime arrays. By default, `outputType` is set to 'datenum'.

Version History

Introduced before R2006a

See Also

`tr2bonds` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

thirtytwo2dec

Thirty-second quotation to decimal

Syntax

```
OutNumber = thirtytwo2dec(InNumber,InFraction)
```

Description

`OutNumber = thirtytwo2dec(InNumber,InFraction)` changes the price quotation for a bond or bond future from a fraction with a denominator of 32 to a decimal.

Examples

Change the Price Quotation for a Bond or Bond Future From a Fraction

This example shows how to change the price quotation for a bond or bond future from a fraction with a denominator of 32 to a decimal, given two bonds that are quoted as 101-25 and 102-31.

```
InNumber = [101; 102];
InFraction = [25; 31];
```

```
OutNumber = thirtytwo2dec(InNumber, InFraction)
```

```
OutNumber = 2×1
```

```
101.7812
102.9688
```

Input Arguments

InNumber — Input number

integer

Input number, specified as a scalar or an N-by-1 vector of integers representing price without the fractional components.

Data Types: double

InFraction — Fractional portions of each element in InNumber

numeric decimal fraction

Fractional portions of each element in `InNumber`, specified as a scalar or an N-by-1 vector of numeric decimal fractions.

Data Types: double

Output Arguments

OutNumber — Output number that represents sum of InNumber and InFraction
decimal

Output number that represents sum of InNumber and InFraction, returned as a decimal.

Version History

Introduced before R2006a

See Also

`dec2thirtytwo`

tick2ret

Convert price series to return series

Syntax

```
[Returns,Intervals] = tick2ret(Data)
[Returns,Intervals] = tick2ret( ___,Name,Value)
```

Description

[Returns,Intervals] = tick2ret(Data) computes asset returns for NUMOBS price observations of NASSETS assets.

[Returns,Intervals] = tick2ret(___,Name,Value) adds optional name-value pair arguments.

Examples

Convert Price Series to Return Series

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. Then convert a price series to a return series, given the first 10 periodic returns of TMW.

```
load SimulatedStock.mat
```

```
TMW_Close = TMW(1:10,'Close');
[Returns,Intervals] = tick2ret(TMW_Close)
```

```
Returns=9x1 timetable
      Time      Close
      _____  _____
05-Sep-2012    0.0017955
06-Sep-2012    0.013741
07-Sep-2012   -0.022591
10-Sep-2012   -0.011557
11-Sep-2012   -0.014843
12-Sep-2012   -0.0012384
13-Sep-2012    0.0081628
14-Sep-2012   -0.00051245
17-Sep-2012   -0.02902
```

```
Intervals = 9x1 duration
24:00:00
24:00:00
24:00:00
72:00:00
24:00:00
24:00:00
24:00:00
```

```
24:00:00
72:00:00
```

Convert Price Series to Return Series Using `datetime` Input

Use `datetime` input to convert a price series to a return series, given periodic returns of two stocks observed in the first, second, third, and fourth quarters.

```
TickSeries = [100 80
110 90
115 88
110 91];
```

```
TickTimes = datetime({'1/1/2015','1/7/2015','1/16/2015','1/28/2015'},'InputFormat','MM/dd/yyyy')
[Returns,Intervals] = tick2ret(TickSeries,'TickTimes',TickTimes)
```

```
Returns = 3×2
```

```
    0.1000    0.1250
    0.0455   -0.0222
   -0.0435    0.0341
```

```
Intervals = 3×1 duration
```

```
144:00:00
216:00:00
288:00:00
```

Input Arguments

Data — Data for asset prices

matrix | table | timetable

Data for asset prices, specified as a `NUMOBS``NASSETS` matrix, table, or timetable. Prices across a given row are assumed to occur at the same time for all columns, and each column is a price series of an individual asset.

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `[Returns,Intervals] = tick2ret(TickSeries,'TickTimes',TickTimes)`

`TickTimes` — Observation times associated with prices

sequential observation times from 1,2,...`NUMOBS` assumed for all assets (default) | vector

Observation times associated with prices, specified as the comma-separated pair consisting of 'TickTimes' and a NUMOBS element column vector of monotonically increasing observation times associated with the prices in Data. Times are taken either as serial date numbers (day units), date strings, datetime arrays, or as decimal numbers in arbitrary units (for example, yearly).

Note If the input Data type is a timetable, the row times information in the timetable overwrites the TickTimes input.

Data Types: double | datetime | string

Method — Method to convert asset prices to returns

'Simple' (default) | character vector with value of 'Simple' or 'Continuous' | string with value of "Simple" or "Continuous"

Method to convert asset prices to returns, specified as the comma-separated pair consisting of 'Method' and a string or character vector indicating the method to convert asset prices to returns.

If the method is 'Simple', then simple periodic returns at time t are computed as:

$$\text{Returns}(t) = \text{Data}(t)/\text{Data}(t-1) - 1.$$

If the method is 'Continuous', the continuous returns are computed as:

$$\text{Returns}(t) = \log(\text{Data}(t)/\text{Data}(t-1)).$$

Data Types: char | string

Output Arguments

Returns — Time series array of asset returns

matrix | table | timetable

Time series array of asset returns, returned as a NUMOBS - 1-by-NASSETS array of asset returns with the same type (matrix, table, or timetable) as the input Data. The first row contains the oldest returns and the last row contains the most recent. Returns across a given row are assumed to occur at the same time for all columns, and each column is a return series of an individual asset.

Intervals — Interval times between successive prices

vector

Interval times between successive prices, returned as a NUMOBS - 1 length column vector where $\text{Intervals}(t) = \text{TickTimes}(t) - \text{TickTimes}(t - 1)$.

Version History

Introduced before R2006a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

Extended Capabilities

Tall Arrays

Calculate with arrays that have more rows than fit in memory.

This function supports input `Data` that is specified as a tall column vector, a tall table, or a tall timetable. For more information, see `tall` and “Tall Arrays”.

See Also

`ret2tick` | `timetable` | `table` | `datetime`

Topics

“Use Timetables in Finance” on page 11-7

“Returns with Negative Prices” on page 2-32

time2date

Dates from time and frequency

Syntax

```
Dates = time2date(Settle,TFactors)
Dates = time2date(____,Compounding,Basis,EndMonthRule)
```

Description

Dates = time2date(Settle,TFactors) computes Dates corresponding to compounded rate quotes between Settle and TFactors. time2date is the inverse of date2time.

Dates = time2date(____,Compounding,Basis,EndMonthRule) computes Dates corresponding to compounded rate quotes between Settle and TFactors using optional input arguments for Compounding, Basis, and EndMonthRule. time2date is the inverse of date2time.

Examples

Calculate Dates Using time2date

Show that date2time and time2date are the inverse of each other. First compute the time factors using date2time.

```
Settle = datetime(2002,9,1);
Dates = [datetime(2005,8,31) ; datetime(2006,2,28) ; datetime(2006,6,15) ; datetime(2006,12,31)]
Compounding = 2;
Basis = 0;
EndMonthRule = 1;
TFactors = date2time(Settle, Dates, Compounding, Basis,...
EndMonthRule)
```

```
TFactors = 4×1
```

```
5.9945
6.9945
7.5738
8.6576
```

Now use the calculated TFactors in time2date and compare the calculated dates with the original set.

```
Dates_calc = time2date(Settle, TFactors, Compounding, Basis,...
EndMonthRule)
```

```
Dates_calc = 4×1 datetime
31-Aug-2005
28-Feb-2006
15-Jun-2006
```

31-Dec-2006

Input Arguments

Settle — Settlement date

datetime scalar | string scalar | date character vector

Settlement date, specified as a scalar datetime, string, or date character vector.

To support existing code, `time2date` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

TFactors — Time factors

vector

Time factors, corresponding to the compounding value, specified as a vector. `TFactors` must be equal to or greater than zero.

Data Types: double

Compounding — Rate at which input zero rates are compounded when annualized

2 (Semiannual compounding) (default) | scalar with numeric values of 0, 1, 2, 3, 4, 5, 6, 12, 365, -1

Rate at which input zero rates are compounded when annualized, specified as a scalar with numeric values of: 0, 1, 2, 3, 4, 5, 6, 12, 365, or -1. Allowed values are defined as:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

The optional `Compounding` argument determines the formula for the discount factors (`Disc`):

- `Compounding = 1, 2, 3, 4, 6, 12`
 - $\text{Disc} = (1 + Z/F)^{-T}$, where F is the compounding frequency, Z is the zero rate, and T is the time in periodic units, for example, $T = F$ is one year.
- `Compounding = 365`
 - $\text{Disc} = (1 + Z/F)^{-T}$, where F is the number of days in the basis year and T is a number of days elapsed computed by basis.
- `Compounding = -1`

- $\text{Disc} = \exp(-T \cdot Z)$, where T is time in years.

Basis — Day-count basis

0 (actual/actual) (default) | numeric with value 0 through 13 | vector of numerics with values 0 through 13

Day-count basis, specified as an integer with a value of 0 through 13 or a N-by-1 vector of integers with values 0 through 13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: single | double

EndMonthRule — End-of-month rule flag for month having 30 or fewer days

1 (in effect) (default) | nonnegative integer [0, 1]

End-of-month rule flag for month having 30 or fewer days, specified as scalar nonnegative integer [0, 1] or a using a N-by-1 vector of values. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days.

- 0 = Ignore rule, meaning that a payment date is always the same numerical day of the month.
- 1 = Set rule on, meaning that a payment date is always the last actual day of the month.

Data Types: logical

Output Arguments

Dates — Dates corresponding to compounded rate quotes between **Settle and **TFactors****

serial date number | datetime array

Dates corresponding to compounded rate quotes between **Settle** and **TFactors**, returned as a scalar or a N-by-1 vector using serial date numbers or a datetime array.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `time2date` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`cftimes` | `date2time` | `datetime` | `cfamounts`

Topics

“Handle and Convert Dates” on page 2-2

tmfactor

Time factors of arbitrary dates

Syntax

```
TMFactors = tmfactor(Settle,Maturity)
```

Description

`TMFactors = tmfactor(Settle,Maturity)` determines the time factors from a vector of Settlement dates to a vector of Maturity dates.

Examples

Find Time Factors of Settle and Maturity Dates

Find the TFactors for Settle and Maturity dates.

```
TFactors = tmfactor('1-Jan-2015','1-Jan-2016')
```

```
TFactors =  
2
```

Find the TFactors for Settle and Maturity dates using a datetimes.

```
TFactors = tmfactor(datetime(2015,1,1),datetime(2016,1,1))
```

```
TFactors =  
2
```

Input Arguments

Settle — Settlement date

datetime array | string array | date character vector

Settlement date, specified as a scalar or a NINST-by-1 vector using a datetime array, string array, or date character vectors.

Note Settle must be earlier than Maturity.

To support existing code, `tmfactor` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date

datetime array | string array | date character vector

Maturity date, specified as a scalar or a NINST-by-1 vector using a `datetime` array, string array, or date character vectors.

Data Types: `char` | `string` | `datetime`

Output Arguments

TMFactors — Time factors from a vector of Settle dates to a vector of Maturity dates

scalar numeric

Time factors from a vector of Settle dates to a vector of Maturity dates, returned as a scalar numeric.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `tmfactor` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

`cfamounts` | `cftimes` | `datetime`

Topics

"Analyzing and Computing Cash Flows" on page 2-11

todecimal

Fractional to decimal conversion

Syntax

```
usddec = todecimal(quote)
usddec = todecimal(____,fracpart)
```

Description

`usddec = todecimal(quote)` returns the decimal equivalent, `usddec`, of a security whose price is normally quoted as a whole number and a fraction (`quote`).

`usddec = todecimal(____,fracpart)` returns the decimal equivalent, `usddec`, of a security whose price is normally quoted as a whole number and a fraction (`quote`). `fracpart` indicates the fractional base (denominator) with which the security is normally quoted (default = 32).

Examples

Convert Stock Quote from Fractinal to Decimal

Often securty prices are quoted in fractional form based on a denominator of 32. For example, if you see the quoted price is 100:05 it means 100 5/32. To find the equivalent decimal value, enter:

```
usddec = todecimal(100.05)
```

```
usddec =
100.1562
```

You can change the default denominator of 32 by using the `fracpart` optional argument with a value of 16.

```
usddec = todecimal(97.04, 16)
```

```
usddec =
97.2500
```

Input Arguments

quote — Quoted security price

scalar numeric

Quoted security price, specified as a scalar numeric or an NPORTS-by-1 vector.

Note The convention of using . (period) as a substitute for : (colon) in the input is adopted from Excel software.

Data Types: double

fracpart — Fractional base (denominator) with which the security is normally quoted

32 (default) | scalar numeric

(Optional) Fractional base (denominator) with which the security is normally quoted, specified as a scalar numeric.

Data Types: double

Output Arguments

usddec — Fractional to decimal conversion

numeric

Fractional to decimal conversion, returned as a numeric decimal.

Version History

Introduced before R2006a

See Also

toquoted

toquoted

Decimal to fractional conversion

Syntax

```
quote = toquoted(usddec,fracpart)
```

Description

`quote = toquoted(usddec,fracpart)` returns the fractional equivalent, `quote`, of the decimal figure, `usddec`, based on the fractional base (denominator), `fracpart`. The fractional bases are the ones used for quoting equity prices in the United States (denominator 2, 4, 8, 16, or 32). If `fracpart` is not entered, the denominator 32 is assumed.

Examples

Convert Decimal to Fraction

A United States equity price in decimal form is 101.625. To convert this to fractional form in eighths of a dollar:

```
quote = toquoted(101.625, 8)
```

```
quote =  
101.0500
```

The answer is interpreted as 101 5/8.

The convention of using . (period) as a substitute for : (colon) in the output is adopted from Excel ® software.

Input Arguments

usddec — Equity price

numeric

Equity price, specified as a numeric decimal.

Data Types: double

fracpart — Fractional bases used for quoting equity prices in the United States

32 (default) | numeric with denominator value of 2, 4, 8, 16, or 32

Fractional bases used for quoting equity prices in the United States, specified as a scalar numeric.

Data Types: double

Output Arguments

quote — Fractional equivalent

numeric

Fractional equivalent, returned as a numeric. For example, bond prices are quoted in fractional form based on a 32nd. If you see the quoted price is `100:05`, it means 100 5/32 which is equivalent to 100.15625.

Version History

Introduced before R2006a

See Also

`todecimal`

convert2daily

Aggregate timetable data to daily periodicity

Syntax

```
TT2 = convert2daily(TT1)
TT2 = convert2daily(TT1,Name,Value)
```

Description

TT2 = convert2daily(TT1) aggregates data (for example, high-frequency and intra-day data) to a daily periodicity.

TT2 = convert2daily(TT1,Name,Value) uses additional options specified by one or more name-value arguments.

Examples

Aggregate Prices and Logarithmic Returns to Daily Periodicity

Apply separate aggregation methods to related variables in a timetable while maintaining consistency between aggregated results for a daily periodicity.

Load a timetable (DataTimeTable) of simulated stock price data and corresponding logarithmic returns. The data stored in DataTimeTable is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable DataTimeTable also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using addBusinessCalendar first.

```
load("SimulatedStockSeries.mat","DataTimeTable");
head(DataTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

Aggregate prices and logarithmic returns to a daily periodicity. To maintain consistency between prices and returns, for any given trading day, aggregate the prices by reporting the last recorded price by using "lastvalue" and aggregate the returns by summing all logarithmic returns by using "sum".

```
DTT = convert2daily(DataTimeTable,Aggregation=["lastvalue" "sum"]);
head(DTT)
```

Time	Price	Log_Return
01-Jan-2018	100.15	-0.023876
02-Jan-2018	99.72	-0.0043028
03-Jan-2018	105.57	0.057008
04-Jan-2018	109.01	0.032065
05-Jan-2018	110.69	0.015294
06-Jan-2018	110.48	-0.001899
07-Jan-2018	113.83	0.029872
08-Jan-2018	116.41	0.022412

To verify consistency, examine the input and output timetables for January 2 and 3, 2018.

```
dt = datetime(2018,1,4);
DataTimeTable(DataTimeTable.Time < dt,:) % Input data through 03-Jan-2018
```

```
ans=9x2 timetable
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929
03-Jan-2018 14:45:20	105.57	-0.013922

```
DTT(1:3,:) % Return aggregated results
```

```
ans=3x2 timetable
```

Time	Price	Log_Return
01-Jan-2018	100.15	-0.023876
02-Jan-2018	99.72	-0.0043028
03-Jan-2018	105.57	0.057008

```
logLastPriceJan2 = log(99.72);
logLastPriceJan3 = log(105.57);
aggLogRetJan3 = logLastPriceJan3 - logLastPriceJan2

aggLogRetJan3 =
0.0570
```

For each business day in DTT, notice that the output aggregated price is the last price of the day and that the aggregated return is the sum of all logarithmic returns. Also, the aggregated returns are consistent with aggregated prices.

For example, the aggregated return for January 3, 2018, is 0.0570, which is the logarithmic return associated with the last prices recorded on January 2 and 3, 2018.

The dates of the aggregated results are whole dates that indicate the dates for which aggregated results are reported.

Input Arguments

TT1 — Data to aggregate to daily periodicity

timetable

Data to aggregate to a daily periodicity, specified as a timetable.

Each variable can be a numeric vector (univariate series) or numeric matrix (multivariate series).

Note

- NaNs indicate missing values.
 - Timestamps must be in ascending or descending order.
-

By default, all days are business days. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first. For example, the following command adds business calendar logic to include only NYSE business days.

```
TT = addBusinessCalendar(TT);
```

Data Types: `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = convert2daily(TT1,'Aggregation',{'lastvalue' 'sum'})`

Aggregation — Intra-day aggregation method for data in TT1

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Intra-day aggregation method for TT1 defining how data is aggregated over business days, specified as one of the following methods, a string vector of methods, or a length `numVariables` cell vector of methods, where `numVariables` is the number of variables in TT1.

- "sum" — Sum the values in each year or day.
- "mean" — Calculate the mean of the values in each year or day.
- "prod" — Calculate the product of the values in each year or day.
- "min" — Calculate the minimum of the values in each year or day.
- "max" — Calculate the maximum of the values in each year or day.
- "firstvalue" — Use the first value in each year or day.

- "lastvalue" — Use the last value in each year or day.
- @customfcn — A custom aggregation method that accepts a timetable and returns a numeric scalar (for univariate series) or row vector (for multivariate series). The function must accept empty inputs [].

If you specify a single method, `convert2daily` applies the specified method to all time series in `TT1`. If you specify a string vector or cell vector `aggregation`, `convert2daily` applies `aggregation(j)` to `TT1(:,j)`; `convert2daily` applies each aggregation method one at a time (for more details, see `retime`). For example, consider a daily timetable representing `TT1` with three variables.

Time	AAA	BBB	CCC	
01-Jan-2018 09:45:47	100.00	200.00	300.00	400.00
01-Jan-2018 12:48:09	100.03	200.06	300.09	400.12
02-Jan-2018 10:27:32	100.07	200.14	300.21	400.28
02-Jan-2018 12:46:09	100.08	200.16	300.24	400.32
02-Jan-2018 14:14:13	100.25	200.50	300.75	401.00
02-Jan-2018 15:52:31	100.19	200.38	300.57	400.76
03-Jan-2018 09:47:11	100.54	201.08	301.62	402.16
03-Jan-2018 11:24:23	100.59	201.18	301.77	402.36
03-Jan-2018 14:41:17	101.40	202.80	304.20	405.60
03-Jan-2018 16:00:00	101.94	203.88	305.82	407.76
04-Jan-2018 09:55:51	102.53	205.06	307.59	410.12
04-Jan-2018 10:07:12	103.35	206.70	310.05	413.40
04-Jan-2018 14:26:23	103.40	206.80	310.20	413.60
05-Jan-2018 13:13:12	103.91	207.82	311.73	415.64
05-Jan-2018 14:57:53	103.89	207.78	311.67	415.56

The corresponding default daily results representing `TT2` (where the 'lastvalue' is reported for each day) are as follows.

Time	AAA	BBB	CCC	
01-Jan-2018	100.03	200.06	300.09	400.12
02-Jan-2018	100.19	200.38	300.57	400.76
03-Jan-2018	101.94	203.88	305.82	407.76
04-Jan-2018	103.40	206.80	310.20	413.60
05-Jan-2018	103.89	207.78	311.67	415.56

All methods omit missing data (NaNs) in direct aggregation calculations on each variable. However, for situations in which missing values appear in the first row of `TT1`, missing values can also appear in the aggregated results `TT2`. To address missing data, write and specify a custom aggregation method (function handle) that supports missing data.

Data Types: char | string | cell | function_handle

Output Arguments

TT2 — Daily data

timetable

Daily data, returned as a timetable. The time arrangement of `TT1` and `TT2` are the same.

If a variable of `TT1` has no records for a business day within the sampling time span, `convert2daily` returns a NaN for that variable and business day in `TT2`.

The first date in TT2 is the first business date on or after the first date in TT1. The last date in TT2 is the last business date on or before the last date in TT1.

Version History

Introduced in R2021a

See Also

[convert2weekly](#) | [convert2monthly](#) | [convert2quarterly](#) | [convert2semiannual](#) | [convert2annual](#) | [timetable](#) | [addBusinessCalendar](#)

Topics

[“Resample and Aggregate Data in Timetable”](#)

[“Combine Timetables and Synchronize Their Data”](#)

[“Retime and Synchronize Timetable Variables Using Different Methods”](#)

convert2weekly

Aggregate timetable data to weekly periodicity

Syntax

```
TT2 = convert2weekly(TT1)
TT2 = convert2weekly( ___, Name, Value)
```

Description

TT2 = convert2weekly(TT1) aggregates data (for example, data recorded daily) to a weekly periodicity.

TT2 = convert2weekly(___, Name, Value) uses additional options specified by one or more name-value arguments.

Examples

Aggregate Timetable Daily Data to Weekly Periodicity

Apply separate aggregation methods to related variables in a `timetable` while maintaining consistency between aggregated results when converting from a daily to a weekly periodicity. You can use `convert2weekly` to aggregate both intra-daily data and aggregated daily data. These methods result in equivalent weekly aggregates.

Load a timetable (`DateTimeTable`) of simulated stock price data and corresponding logarithmic returns. The data stored in `DateTimeTable` is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable `DateTimeTable` also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first.

```
load("SimulatedStockSeries.mat","DateTimeTable");
head(DateTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

Use `convert2daily` to aggregate intra-daily prices and returns to daily periodicity. To maintain consistency between prices and returns, for any given trading day, aggregate prices by reporting the

last recorded price with "lastvalue" and aggregate returns by summing all logarithmic returns with "sum".

```
DTTDaily = convert2daily(DataTimeTable,Aggregation=["lastvalue" "sum"]);
head(DTTDaily)
```

Time	Price	Log_Return
01-Jan-2018	100.15	-0.023876
02-Jan-2018	99.72	-0.0043028
03-Jan-2018	105.57	0.057008
04-Jan-2018	109.01	0.032065
05-Jan-2018	110.69	0.015294
06-Jan-2018	110.48	-0.001899
07-Jan-2018	113.83	0.029872
08-Jan-2018	116.41	0.022412

Use `convert2weekly` to aggregate the data to a weekly periodicity and compare the results of two different aggregation approaches. The first approach computes weekly results by aggregating the daily aggregates and the second approach computes weekly results by directly aggregating the original intra-daily data.

```
DTTWeekly1 = convert2weekly(DTTDaily,Aggregation=["lastvalue" "sum"]);           % Daily to weekly
DTTWeekly2 = convert2weekly(DataTimeTable,Aggregation=["lastvalue" "sum"]); % Intra-daily to weekly
```

```
head(DTTWeekly1)
```

Time	Price	Log_Return
05-Jan-2018	110.69	0.076188
12-Jan-2018	119.91	0.080008
19-Jan-2018	116.6	-0.027992
26-Jan-2018	118.51	0.016248
02-Feb-2018	120.03	0.012744
09-Feb-2018	117.07	-0.02497
16-Feb-2018	117.06	-8.5423e-05
23-Feb-2018	116.72	-0.0029087

```
head(DTTWeekly2)
```

Time	Price	Log_Return
05-Jan-2018	110.69	0.076188
12-Jan-2018	119.91	0.080008
19-Jan-2018	116.6	-0.027992
26-Jan-2018	118.51	0.016248
02-Feb-2018	120.03	0.012744
09-Feb-2018	117.07	-0.02497
16-Feb-2018	117.06	-8.5423e-05
23-Feb-2018	116.72	-0.0029087

Notice that the results of the two approaches are the same and that `convert2weekly` reports on Fridays by default. For weeks in which Friday is not an NYSE trading day, the function reports results on the previous business day. In addition, you can use the `convert2weekly` optional name-value argument `EndOfWeekDay` to specify a different day of the week that ends business weeks.

Input Arguments

TT1 — Data to aggregate to weekly periodicity

timetable

Data to aggregate to a weekly periodicity, specified as a timetable.

Each variable can be a numeric vector (univariate series) or numeric matrix (multivariate series).

Note

- NaNs indicate missing values.
 - Timestamps must be in ascending or descending order.
-

By default, all days are business days. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first. For example, the following command adds business calendar logic to include only NYSE business days.

```
TT = addBusinessCalendar(TT);
```

Data Types: timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = convert2weekly(TT1, 'Aggregation', ["lastvalue" "sum"])`

Aggregation — Aggregation method for TT1 data for intra-week or inter-day aggregation

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Aggregation method for TT1 defining how data is aggregated over business days in an intra-week or inter-day periodicity, specified as one of the following methods, a string vector of methods, or a length `numVariables` cell vector of methods, where `numVariables` is the number of variables in TT1.

- "sum" — Sum the values in each year or day.
- "mean" — Calculate the mean of the values in each year or day.
- "prod" — Calculate the product of the values in each year or day.
- "min" — Calculate the minimum of the values in each year or day.
- "max" — Calculate the maximum of the values in each year or day.
- "firstvalue" — Use the first value in each year or day.
- "lastvalue" — Use the last value in each year or day.

- `@customfcn` — A custom aggregation method that accepts a table variable and returns a numeric scalar (for univariate series) or row vector (for multivariate series). The function must accept empty inputs `[]`.

If you specify a single method, `convert2weekly` applies the specified method to all time series in `TT1`. If you specify a string vector or cell vector `aggregation`, `convert2weekly` applies `aggregation(j)` to `TT1(:,j)`; `convert2weekly` applies each aggregation method one at a time (for more details, see `retime`). For example, consider a daily timetable representing `TT1` with three variables.

Time	AAA	BBB	CCC	
01-Jan-2018	100.00	200.00	300.00	400.00
02-Jan-2018	100.03	200.06	300.09	400.12
03-Jan-2018	100.07	200.14	300.21	400.28
04-Jan-2018	100.08	200.16	300.24	400.32
05-Jan-2018	100.25	200.50	300.75	401.00
06-Jan-2018	100.19	200.38	300.57	400.76
07-Jan-2018	100.54	201.08	301.62	402.16
08-Jan-2018	100.59	201.18	301.77	402.36
09-Jan-2018	101.40	202.80	304.20	405.60
10-Jan-2018	101.94	203.88	305.82	407.76
11-Jan-2018	102.53	205.06	307.59	410.12
12-Jan-2018	103.35	206.70	310.05	413.40
13-Jan-2018	103.40	206.80	310.20	413.60
14-Jan-2018	103.91	207.82	311.73	415.64
15-Jan-2018	103.89	207.78	311.67	415.56
16-Jan-2018	104.44	208.88	313.32	417.76
17-Jan-2018	104.44	208.88	313.32	417.76
18-Jan-2018	104.04	208.08	312.12	416.16
19-Jan-2018	104.94	209.88	314.82	419.76

The corresponding default weekly results representing `TT2` (in which all days are business days and the `'lastvalue'` is reported on Fridays) are as follows.

Time	AAA	BBB	CCC	
05-Jan-2018	100.25	200.50	300.75	401.00
12-Jan-2018	103.35	206.70	310.05	413.40
19-Jan-2018	104.94	209.88	314.82	419.76

The default `'lastvalue'` returns the latest observed value in a given week for all variables in `TT1`.

All methods omit missing data (NaNs) in direct aggregation calculations on each variable. However, for situations in which missing values appear in the first row of `TT1`, missing values can also appear in the aggregated results `TT2`. To address missing data, write and specify a custom aggregation method (function handle) that supports missing data.

Data Types: `char` | `string` | `cell` | `function_handle`

Daily — Intra-day aggregation method for `TT1`

`"lastvalue"` (default) | `"sum"` | `"prod"` | `"mean"` | `"min"` | `"max"` | `"firstvalue"` | character vector | function handle | string vector | cell vector of character vectors or function handles

Intra-day aggregation method for `TT1`, specified as an aggregation method, a string vector of methods, or a length `numVariables` cell vector of methods. For more details on supported methods and behaviors, see the `'Aggregation'` name-value argument.

Data Types: `char` | `string` | `cell` | `function_handle`

EndOfWeekDay — Day of week that ends business weeks

"Friday" (weeks end on Friday) (default) | scalar integer with value 1 through 7 | "Sunday" | "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday" | "Saturday" | character vector

Day of the week that ends business weeks, specified as a value in the table.

Value	Day Ending Each Week
"Sunday" or 1	Sunday
"Monday" or 2	Monday
"Tuesday" or 3	Tuesday
"Wednesday" or 4	Wednesday
"Thursday" or 5	Thursday
"Friday" or 6	Friday
"Saturday" or 7	Saturday

If the specified end-of-week day in a given week is not a business day, the preceding business day ends that week.

Data Types: `double` | `char` | `string`

Output Arguments

TT2 — Weekly data

timetable

Weekly data, returned as a timetable. The time arrangement of TT1 and TT2 are the same.

If a variable of TT1 has no business-day records during an annual period within the sampling time span, `convert2weekly` returns a NaN for that variable and annual period in TT2.

If the first week (week1) of TT1 contains at least one business day, the first date in TT2 is the last business date of week1. Otherwise, the first date in TT2 is the next end-of-week business date of TT1.

If the last week (weekT) of TT1 contains at least one business day, the last date in TT2 is the last business date of weekT. Otherwise, the last date in TT2 is the previous end-of-week business date of TT1.

Version History

Introduced in R2021a

See Also

`convert2daily` | `convert2quarterly` | `convert2semiannual` | `convert2monthly` | `convert2annual` | `timetable` | `addBusinessCalendar`

Topics

"Resample and Aggregate Data in Timetable"
 "Combine Timetables and Synchronize Their Data"

“Retime and Synchronize Timetable Variables Using Different Methods”

convert2monthly

Aggregate timetable data to monthly periodicity

Syntax

```
TT2 = convert2monthly(TT1)
TT2 = convert2monthly(TT1,Name,Value)
```

Description

`TT2 = convert2monthly(TT1)` aggregates data (for example, data recorded daily or weekly) to monthly periodicity.

`TT2 = convert2monthly(TT1,Name,Value)` uses additional options specified by one or more name-value arguments.

Examples

Aggregate Timetable Data to Monthly Periodicity

Apply separate aggregation methods to related variables in a `timetable` while maintaining consistency between aggregated results when converting to a monthly periodicity. You can use `convert2monthly` to aggregate both intra-daily data and aggregated daily data. These methods result in equivalent monthly aggregates. Lastly, you can aggregate results on a specific day of each month (for example, the 15th), rather than the default end of the month.

Load a timetable (`DataTimeTable`) of simulated stock price data and corresponding logarithmic returns. The data stored in `DataTimeTable` is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable `DataTimeTable` also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first.

```
load("SimulatedStockSeries.mat","DataTimeTable");
head(DataTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

First aggregate intra-daily prices and returns to daily periodicity. To maintain consistency between prices and returns, for any given trading day aggregate prices by reporting the last recorded price using "lastvalue" and aggregate returns by summing all logarithmic returns using "sum".

```
DTTDaily = convert2daily(DataTimeTable,Aggregation=["lastvalue" "sum"]);
head(DTTDaily)
```

Time	Price	Log_Return
01-Jan-2018	100.15	-0.023876
02-Jan-2018	99.72	-0.0043028
03-Jan-2018	105.57	0.057008
04-Jan-2018	109.01	0.032065
05-Jan-2018	110.69	0.015294
06-Jan-2018	110.48	-0.001899
07-Jan-2018	113.83	0.029872
08-Jan-2018	116.41	0.022412

Use `convert2monthly` to aggregate the data to a monthly periodicity and compare the results of two different approaches. The first approach computes monthly results by aggregating the daily aggregates and the second approach computes monthly results by directly aggregating the original intra-daily data. Note that although `convert2monthly` reports results on the last business day of each month by default, you can report monthly results on the 15th of each month by using the optional name-value pair argument 'EndOfMonthDay'.

```
DTTMonthly1 = convert2monthly(DTTDaily,Aggregation=["lastvalue" "sum"], ...
    EndOfMonthDay=15); % Daily to monthly
DTTMonthly2 = convert2monthly(DataTimeTable,Aggregation=["lastvalue" "sum"], ...
    EndOfMonthDay=15); % Intra-daily to monthly
```

```
head(DTTMonthly1)
```

Time	Price	Log_Return
15-Jan-2018	114.72	0.11195
15-Feb-2018	117.27	0.021985
15-Mar-2018	105.3	-0.10767
15-Apr-2018	106.4	0.010392
15-May-2018	100.29	-0.05914
15-Jun-2018	98.72	-0.015778
15-Jul-2018	103.18	0.044187
15-Aug-2018	108.07	0.046304

```
head(DTTMonthly2)
```

Time	Price	Log_Return
15-Jan-2018	114.72	0.11195
15-Feb-2018	117.27	0.021985
15-Mar-2018	105.3	-0.10767
15-Apr-2018	106.4	0.010392
15-May-2018	100.29	-0.05914
15-Jun-2018	98.72	-0.015778
15-Jul-2018	103.18	0.044187
15-Aug-2018	108.07	0.046304

Notice that the results of the two approaches are the same. For months in which the 15th is not an NYSE trading day, the function reports results on the previous business day.

Use Custom Aggregation Method to Convert Timetable Daily Data to Monthly Periodicity

You can apply custom aggregation methods using function handles. Specify a function handle to aggregate related variables in a `timetable` while maintaining consistency between aggregated results when converting from a daily to a monthly periodicity.

Load a timetable (`DateTimeTable`) of simulated stock price data and corresponding logarithmic returns. The data stored in `DateTimeTable` is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable `DateTimeTable` also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first.

```
load("SimulatedStockSeries.mat","DateTimeTable");
head(DateTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

First add another variable to `DateTimeTable` that contains the simple (proportional) returns associated with the prices in `DateTimeTable` and examine the first few rows.

```
DateTimeTable.Simple_Return = exp(DateTimeTable.Log_Return) - 1; % Log returns to simple returns
head(DateTimeTable)
```

Time	Price	Log_Return	Simple_Return
01-Jan-2018 11:52:48	100	-0.025375	-0.025056
01-Jan-2018 13:23:13	101.14	0.011336	0.0114
01-Jan-2018 14:45:09	101.5	0.0035531	0.0035594
01-Jan-2018 15:30:30	100.15	-0.01339	-0.0133
02-Jan-2018 10:43:37	99.72	-0.0043028	-0.0042936
03-Jan-2018 10:02:21	100.11	0.0039033	0.003911
03-Jan-2018 11:22:37	103.96	0.037737	0.038458
03-Jan-2018 13:42:27	107.05	0.02929	0.029723

Create a function to aggregate simple returns and compute the monthly aggregates. To maintain consistency between prices and returns, for any given month, aggregate prices by reporting the last recorded price by using `"lastvalue"` and report logarithmic returns by summing all intervening logarithmic returns by using `"sum"`.

Notice that the aggregation function for simple returns operates along the first (row) dimension and omits missing data (NaNs). For more information on custom aggregation functions, see `timetable` and `retime`. When aggregation methods are a mix of supported methods and user-supplied functions, the `Aggregation` optional name-value argument must be specified as a cell vector of methods enclosed in curly braces.

```
f = @(x)(prod(1 + x,1,"omitnan") - 1); % Aggregate simple returns
DTTMonthly = convert2monthly(DataTimeTable,Aggregation={'lastvalue' 'sum' f});
head(DTTMonthly)
```

Time	Price	Log_Return	Simple_Return
31-Jan-2018	117.35	0.13462	0.1441
28-Feb-2018	113.52	-0.033182	-0.032637
31-Mar-2018	110.74	-0.024794	-0.024489
30-Apr-2018	105.58	-0.047716	-0.046596
31-May-2018	97.88	-0.075727	-0.07293
30-Jun-2018	99.29	0.014303	0.014405
31-Jul-2018	102.72	0.033962	0.034545
31-Aug-2018	124.99	0.19623	0.2168

Input Arguments

TT1 — Data to aggregate to monthly periodicity

`timetable`

Data to aggregate to a monthly periodicity, specified as a `timetable`.

Each variable can be a numeric vector (univariate series) or numeric matrix (multivariate series).

Note

- NaNs indicate missing values.
 - Timestamps must be in ascending or descending order.
-

By default, all days are business days. If your `timetable` does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first. For example, the following command adds business calendar logic to include only NYSE business days.

```
TT = addBusinessCalendar(TT);
```

Data Types: `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = convert2monthly(TT1,'Aggregation',{'lastvalue' 'sum'})`

Aggregation — Aggregation method for TT1

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Aggregation method for TT1 defining how to aggregate data over business days in an intra-month or inter-day periodicity, specified as one of the following methods, a string vector of methods, or a length numVariables cell vector of methods, where numVariables is the number of variables in TT1.

- "sum" — Sum the values in each year or day.
- "mean" — Calculate the mean of the values in each year or day.
- "prod" — Calculate the product of the values in each year or day.
- "min" — Calculate the minimum of the values in each year or day.
- "max" — Calculate the maximum of the values in each year or day.
- "firstvalue" — Use the first value in each year or day.
- "lastvalue" — Use the last value in each year or day.
- @customfcn — A custom aggregation method that accepts a table variable and returns a numeric scalar (for univariate series) or row vector (for multivariate series). The function must accept empty inputs [].

If you specify a single method, `convert2monthly` applies the specified method to all time series in TT1. If you specify a string vector or cell vector `aggregation`, `convert2monthly` applies `aggregation(j)` to `TT1(:,j)`; `convert2monthly` applies each aggregation method one at a time (for more details, see `retime`). For example, consider a daily timetable representing TT1 with three variables.

Time	AAA	BBB	CCC	
01-Jan-2018	100.00	200.00	300.00	400.00
02-Jan-2018	100.03	200.06	300.09	400.12
03-Jan-2018	100.07	200.14	300.21	400.28
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
31-Jan-2018	114.65	229.3	343.95	458.60
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
28-Feb-2018	129.19	258.38	387.57	516.76
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
31-Mar-2018	162.93	325.86	488.79	651.72
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
30-Apr-2018	171.72	343.44	515.16	686.88
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
31-May-2018	201.24	402.48	603.72	804.96
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
30-Jun-2018	223.22	446.44	669.66	892.88

The corresponding default monthly results representing TT2 (in which all days are business days and the 'lastvalue' is reported on the last business day of each month) are as follows.

Time	AAA	BBB	CCC	
31-Jan-2018	114.65	229.30	343.95	458.60
28-Feb-2018	129.19	258.38	387.57	516.76
31-Mar-2018	162.93	325.86	488.79	651.72
30-Apr-2018	171.72	343.44	515.16	686.88
31-May-2018	201.24	402.48	603.72	804.96
30-Jun-2018	223.22	446.44	669.66	892.88

All methods omit missing data (NaNs) in direct aggregation calculations on each variable. However, for situations in which missing values appear in the first row of TT1, missing values can also appear in the aggregated results TT2. To address missing data, write and specify a custom aggregation method (function handle) that supports missing data.

Data Types: char | string | cell | function_handle

Daily — Intra-day aggregation method for TT1

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Intra-day aggregation method for TT1, specified as an aggregation method, a string vector of methods, or a length numVariables cell vector of methods. For more details on supported methods and behaviors, see the 'Aggregation' name-value argument.

Data Types: char | string | cell | function_handle

EndOfMonthDay — Day of the month that ends months

last business day of month (default) | integer with value 1 to 31

Day of the month that ends months, specified as a scalar integer with value 1 to 31. For months with fewer days than EndOfMonthDay, convert2monthly reports aggregation results on the last business day of the month.

Data Types: double

Output Arguments

TT2 — Monthly data

timetable

Monthly data, returned as a timetable. The time arrangement of TT1 and TT2 are the same.

If a variable of TT1 has no business-day records during a month within the sampling time span, convert2monthly returns a NaN for that variable and month in TT2.

If the first month (month1) of TT1 contains at least one business day, the first date in TT2 is the last business date of month1. Otherwise, the first date in TT2 is the next end-of-month business date of TT1.

If the last month (monthT) of TT1 contains at least one business day, the last date in TT2 is the last business date of monthT. Otherwise, the last date in TT2 is the previous end-of-month business date of TT1.

Version History

Introduced in R2021a

See Also

`convert2daily` | `convert2weekly` | `convert2quarterly` | `convert2semiannual` | `convert2annual` | `addBusinessCalendar` | `timetable`

Topics

“Resample and Aggregate Data in Timetable”

“Combine Timetables and Synchronize Their Data”

“Retime and Synchronize Timetable Variables Using Different Methods”

convert2quarterly

Aggregate timetable data to quarterly periodicity

Syntax

```
TT2 = convert2quarterly(TT1)
TT2 = convert2quarterly(TT1,Name,Value)
```

Description

TT2 = convert2quarterly(TT1) aggregates data (for example, data recorded daily or weekly) to a quarterly periodicity.

TT2 = convert2quarterly(TT1,Name,Value) uses additional options specified by one or more name-value arguments.

Examples

Aggregate Timetable Data to Quarterly Periodicity

Apply separate aggregation methods to related variables in a `timetable` while maintaining consistency between aggregated results when converting to a quarterly periodicity. You can use `convert2quarterly` to aggregate both intra-daily data and aggregated monthly data. These methods result in equivalent quarterly aggregates.

Load a timetable (`DateTimeTable`) of simulated stock price data and corresponding logarithmic returns. The data stored in `DateTimeTable` is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable `DateTimeTable` also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first.

```
load("SimulatedStockSeries.mat","DateTimeTable");
head(DateTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

Use `convert2monthly` to aggregate intra-daily prices and returns to a monthly periodicity. To maintain consistency between prices and returns for any given month, aggregate prices by reporting

the last recorded price using "lastvalue" and aggregate returns by summing all logarithmic returns using "sum".

```
DTTMonthly1 = convert2monthly(DataTimeTable,Aggregation=["lastvalue" "sum"]);
head(DTTMonthly1)
```

Time	Price	Log_Return
31-Jan-2018	117.35	0.13462
28-Feb-2018	113.52	-0.033182
31-Mar-2018	110.74	-0.024794
30-Apr-2018	105.58	-0.047716
31-May-2018	97.88	-0.075727
30-Jun-2018	99.29	0.014303
31-Jul-2018	102.72	0.033962
31-Aug-2018	124.99	0.19623

Use `convert2quarterly` to aggregate the data to a quarterly periodicity and compare the results of two different approaches. The first approach computes quarterly results by aggregating the monthly aggregates and the second approach computes quarterly results by directly aggregating the original intra-daily data. Note that `convert2quarterly` reports results on the last business day of each quarter.

```
DTTQuarterly1 = convert2quarterly(DTTMonthly1,Aggregation=["lastvalue" "sum"]); % Monthly to
DTTQuarterly2 = convert2quarterly(DataTimeTable,Aggregation=["lastvalue" "sum"]); % Intra-daily
head(DTTQuarterly1)
```

Time	Price	Log_Return
31-Mar-2018	110.74	0.07664
30-Jun-2018	99.29	-0.10914
30-Sep-2018	105.42	0.059908
31-Dec-2018	84.26	-0.22405
31-Mar-2019	112.93	0.29286
30-Jun-2019	169.77	0.40768
30-Sep-2019	148.97	-0.1307
31-Dec-2019	153.22	0.02813

```
head(DTTQuarterly2)
```

Time	Price	Log_Return
31-Mar-2018	110.74	0.07664
30-Jun-2018	99.29	-0.10914
30-Sep-2018	105.42	0.059908
31-Dec-2018	84.26	-0.22405
31-Mar-2019	112.93	0.29286
30-Jun-2019	169.77	0.40768
30-Sep-2019	148.97	-0.1307
31-Dec-2019	153.22	0.02813

The results of the two approaches are the same because each quarter contains exactly three calendar months.

Input Arguments

TT1 — Data to aggregate to quarterly periodicity

timetable

Data to aggregate to a quarterly periodicity, specified as a timetable.

Each variable can be a numeric vector (univariate series) or numeric matrix (multivariate series).

Note

- NaNs indicate missing values.
 - Timestamps must be in ascending or descending order.
-

By default, all days are business days. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first. For example, the following command adds business calendar logic to include only NYSE business days.

```
TT = addBusinessCalendar(TT);
```

Data Types: `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = convert2quarterly(TT1,'Aggregation',{'lastvalue' 'sum'})`

Aggregation — Aggregation method for TT1 data for intra-quarter or inter-day aggregation

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Aggregation method for TT1 data defining how to aggregate data over business days in an intra-quarter or inter-day periodicity, specified as one of the following methods, a string vector of methods, or a length `numVariables` cell vector of methods, where `numVariables` is the number of variables in TT1.

- "sum" — Sum the values in each year or day.
- "mean" — Calculate the mean of the values in each year or day.
- "prod" — Calculate the product of the values in each year or day.
- "min" — Calculate the minimum of the values in each year or day.
- "max" — Calculate the maximum of the values in each year or day.
- "firstvalue" — Use the first value in each year or day.
- "lastvalue" — Use the last value in each year or day.

- `@customfcn` — A custom aggregation method that accepts a table variable and returns a numeric scalar (for univariate series) or row vector (for multivariate series). The function must accept empty inputs `[]`.

If you specify a single method, `convert2quarterly` applies the specified method to all time series in `TT1`. If you specify a string vector or cell vector `aggregation`, `convert2quarterly` applies `aggregation(j)` to `TT1(:,j)`; `convert2quarterly` applies each aggregation method one at a time (for more details, see `retime`). For example, consider a daily timetable representing `TT1` with three variables.

Time	AAA	BBB	CCC	
01-Jan-2018	100.00	200.00	300.00	400.00
02-Jan-2018	100.03	200.06	300.09	400.12
03-Jan-2018	100.07	200.14	300.21	400.28
.	.	.	.	.
.	.	.	.	.
31-Mar-2018	162.93	325.86	488.79	651.72
.	.	.	.	.
.	.	.	.	.
30-Jun-2018	223.22	446.44	669.66	892.88
.	.	.	.	.
.	.	.	.	.
30-Sep-2018	232.17	464.34	696.51	928.68
.	.	.	.	.
.	.	.	.	.
31-Dec-2018	243.17	486.34	729.51	972.68

The corresponding default quarterly results representing `TT2` (in which all days are business days and the 'lastvalue' is reported on the last business day of each quarter) are as follows.

Time	AAA	BBB	CCC	
31-Mar-2018	162.93	325.86	488.79	651.72
30-Jun-2018	223.22	446.44	669.66	892.88
30-Sep-2018	232.17	464.34	696.51	928.68
31-Dec-2018	243.17	486.34	729.51	972.68

All methods omit missing data (NaNs) in direct aggregation calculations on each variable. However, for situations in which missing values appear in the first row of `TT1`, missing values can also appear in the aggregated results `TT2`. To address missing data, write and specify a custom aggregation method (function handle) that supports missing data.

Data Types: `char` | `string` | `cell` | `function_handle`

Daily — Intra-day aggregation method for `TT1`

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Intra-day aggregation method for `TT1`, specified as an aggregation method, a string vector of methods, or a length `numVariables` cell vector of methods. For more details on supported methods and behaviors, see the 'Aggregation' name-value argument.

Data Types: `char` | `string` | `cell` | `function_handle`

Output Arguments

TT2 — Quarterly data

timetable

Quarterly data, returned as a timetable. The time arrangement of TT1 and TT2 are the same.

convert2quarterly reports quarterly aggregation results on the last business day of March, June, September, and December.

If a variable of TT1 has no business-day records during a quarter within the sampling time span, convert2quarterly returns a NaN for that variable and quarter in TT2.

If the first quarter (Q1) of TT1 contains at least one business day, the first date in TT2 is the last business date of Q1. Otherwise, the first date in TT2 is the next end-of-quarter business date of TT1.

If the last quarter (QT) of TT1 contains at least one business day, the last date in TT2 is the last business date of QT. Otherwise, the last date in TT2 is the previous end-of-quarter business date of TT1.

Version History

Introduced in R2021a

See Also

convert2daily | convert2weekly | convert2semiannual | convert2monthly |
convert2annual | timetable | addBusinessCalendar

Topics

“Resample and Aggregate Data in Timetable”

“Combine Timetables and Synchronize Their Data”

“Retime and Synchronize Timetable Variables Using Different Methods”

convert2semiannual

Aggregate timetable data to semiannual periodicity

Syntax

```
TT2 = convert2semiannual(TT1)
TT2 = convert2semiannual(TT1,Name,Value)
```

Description

`TT2 = convert2semiannual(TT1)` aggregates data (for example, data recorded daily or weekly) to a semiannual periodicity.

`TT2 = convert2semiannual(TT1,Name,Value)` uses additional options specified by one or more name-value arguments.

Examples

Aggregate Timetable Data to Semiannual Periodicity

Apply separate aggregation methods to related variables in a `timetable` while maintaining consistency between aggregated results when converting to a semiannual periodicity. You can use `convert2semiannual` to aggregate both intra-daily data and aggregated quarterly data. These methods result in equivalent semiannual aggregates.

Load a timetable (`DateTimeTable`) of simulated stock price data and corresponding logarithmic returns. The data stored in `DateTimeTable` is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable `DateTimeTable` also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first.

```
load("SimulatedStockSeries.mat","DateTimeTable");
head(DateTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

Use `convert2quarterly` to aggregate intra-daily prices and returns to a quarterly periodicity. To maintain consistency between prices and returns, for any given quarter, aggregate prices by

reporting the last recorded price using "lastvalue" and aggregate returns by summing all logarithmic returns using "sum".

```
DTTQuarterly1 = convert2quarterly(DataTimeTable,Aggregation=["lastvalue" "sum"])
```

```
DTTQuarterly1=12×2 timetable
      Time      Price      Log_Return
      -----
31-Mar-2018    110.74      0.07664
30-Jun-2018     99.29     -0.10914
30-Sep-2018    105.42      0.059908
31-Dec-2018     84.26     -0.22405
31-Mar-2019    112.93      0.29286
30-Jun-2019    169.77      0.40768
30-Sep-2019    148.97     -0.1307
31-Dec-2019    153.22      0.02813
31-Mar-2020    229.88      0.40568
30-Jun-2020    224.29     -0.024618
30-Sep-2020    246.77      0.095517
31-Dec-2020    301.04      0.19879
```

Use `convert2semiannual` to aggregate the data to a semiannual periodicity and compare the results of two different approaches. The first approach computes semiannual results by aggregating the quarterly aggregates and the second approach computes semiannual results by directly aggregating the original intra-daily data. Note that `convert2semiannual` reports results on the last business day of June and December.

```
DTTSemi1 = convert2semiannual(DTTQuarterly1,Aggregation=["lastvalue" "sum"]) % Quarterly to semiannual
```

```
DTTSemi1=6×2 timetable
      Time      Price      Log_Return
      -----
30-Jun-2018     99.29     -0.032501
31-Dec-2018     84.26     -0.16414
30-Jun-2019    169.77      0.70054
31-Dec-2019    153.22     -0.10257
30-Jun-2020    224.29      0.38107
31-Dec-2020    301.04      0.2943
```

```
DTTSemi2 = convert2semiannual(DataTimeTable,Aggregation=["lastvalue" "sum"]) % Intra-daily to semiannual
```

```
DTTSemi2=6×2 timetable
      Time      Price      Log_Return
      -----
30-Jun-2018     99.29     -0.032501
31-Dec-2018     84.26     -0.16414
30-Jun-2019    169.77      0.70054
31-Dec-2019    153.22     -0.10257
30-Jun-2020    224.29      0.38107
31-Dec-2020    301.04      0.2943
```

The results of the two approaches are the same because each semiannual period contains exactly two calendar quarters.

Input Arguments

TT1 — Data to aggregate to semiannual periodicity

timetable

Data to aggregate to a semiannual periodicity, specified as a timetable.

Each variable can be a numeric vector (univariate series) or numeric matrix (multivariate series).

Note

- NaNs indicate missing values.
 - Timestamps must be in ascending or descending order.
-

By default, all days are business days. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first. For example, the following command adds business calendar logic to include only NYSE business days.

```
TT = addBusinessCalendar(TT);
```

Data Types: `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = convert2semiannual(TT1, 'Aggregation', ["lastvalue" "sum"])`

Aggregation — Aggregation method for semiannual period to semiannual periodicity (inter-day aggregation)

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Aggregation method for `TT1` defining how data is aggregated over business days in a semiannual period to semiannual periodicity (inter-day aggregation), specified as one of the following methods, a string vector of methods, or a length `numVariables` cell vector of methods, where `numVariables` is the number of variables in `TT1`.

- "sum" — Sum the values in each year or day.
- "mean" — Calculate the mean of the values in each year or day.
- "prod" — Calculate the product of the values in each year or day.
- "min" — Calculate the minimum of the values in each year or day.
- "max" — Calculate the maximum of the values in each year or day.
- "firstvalue" — Use the first value in each year or day.
- "lastvalue" — Use the last value in each year or day.

- `@customfcn` — A custom aggregation method that accepts a table variable and returns a numeric scalar (for univariate series) or row vector (for multivariate series). The function must accept empty inputs `[]`.

If you specify a single method, `convert2semiannual` applies the specified method to all time series in `TT1`. If you specify a string vector or cell vector `aggregation`, `convert2semiannual` applies `aggregation(j)` to `TT1(:,j)`; `convert2semiannual` applies each aggregation method one at a time (for more details, see `retime`). For example, consider a daily timetable representing `TT1` with three variables.

Time	AAA	BBB	CCC	
01-Jan-2018	100.00	200.00	300.00	400.00
02-Jan-2018	100.02	200.04	300.06	400.08
03-Jan-2018	99.96	199.92	299.88	399.84
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
28-Jun-2018	69.63	139.26	208.89	278.52
29-Jun-2018	70.15	140.3	210.45	280.60
30-Jun-2018	75.77	151.54	227.31	303.08
01-Jul-2018	75.68	151.36	227.04	302.72
02-Jul-2018	71.34	142.68	214.02	285.36
03-Jul-2018	69.25	138.50	207.75	277.00
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
29-Dec-2018	249.16	498.32	747.48	996.64
30-Dec-2018	250.21	500.42	750.63	1000.84
31-Dec-2018	256.75	513.50	770.25	1027.00

The corresponding default semiannual results representing `TT2` (in which all days are business days and the 'lastvalue' is reported on the last business day of each semiannual period) are as follows.

Time	AAA	BBB	CCC	
30-Jun-2018	75.77	151.54	227.31	303.08
31-Dec-2018	256.75	513.50	770.25	1027.00

All methods omit missing data (NaNs) in direct aggregation calculations on each variable. However, for situations in which missing values appear in the first row of `TT1`, missing values can also appear in the aggregated results `TT2`. To address missing data, write and specify a custom aggregation method (function handle) that supports missing data.

Data Types: `char` | `string` | `cell` | `function_handle`

Daily — Intra-day aggregation method for `TT1`

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Intra-day aggregation method for `TT1`, specified as an aggregation method, a string vector of methods, or a length `numVariables` cell vector of methods. For more details on supported methods and behaviors, see the 'Aggregation' name-value argument.

Data Types: `char` | `string` | `cell` | `function_handle`

Output Arguments

TT2 — Semiannual data

timetable

Semiannual data, returned as a timetable. `convert2semiannual` reports semiannual aggregation results on the last business day of June and December. The time arrangement of TT1 and TT2 are the same.

If a variable of TT1 has no business-day records during an annual period within the sampling time span, `convert2semiannual` returns a NaN for that variable and annual period in TT2.

The first date in TT2 is the last business date of the semiannual period in which the first date in TT1 occurs, provided TT1 has business dates in that semiannual period. Otherwise the first date in TT2 is the next end-of-semiannual-period business date.

The last date in TT2 is the last business date of the semiannual period in which the last date in TT1 occurs, provided TT1 has business dates in that semiannual period. Otherwise the last date in TT2 is the previous end-of-semiannual-period business date.

Version History

Introduced in R2021a

See Also

`convert2daily` | `convert2weekly` | `convert2quarterly` | `convert2monthly` | `convert2annual` | `timetable` | `addBusinessCalendar`

Topics

“Resample and Aggregate Data in Timetable”

“Combine Timetables and Synchronize Their Data”

“Retime and Synchronize Timetable Variables Using Different Methods”

convert2annual

Aggregate timetable data to annual periodicity

Syntax

```
TT2 = convert2annual(TT1)
TT2 = convert2annual(TT1,Name,Value)
```

Description

`TT2 = convert2annual(TT1)` aggregates data (for example, recorded daily or weekly data) to annual periodicity.

`TT2 = convert2annual(TT1,Name,Value)` uses additional options specified by one or more name-value arguments.

Examples

Aggregate Timetable Data to Annual Periodicity

Apply separate aggregation methods to related variables in a `timetable` while maintaining consistency between aggregated results when converting to an annual periodicity. You can use `convert2annual` to aggregate both intra-daily data and aggregated monthly data. These methods result in equivalent annual aggregates.

Load a timetable (`DateTimeTable`) of simulated stock price data and corresponding logarithmic returns. The data stored in `DateTimeTable` is recorded at various times throughout the day on New York Stock Exchange (NYSE) business days from January 1, 2018, to December 31, 2020. The timetable `DateTimeTable` also includes NYSE business calendar awareness. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first.

```
load("SimulatedStockSeries.mat","DateTimeTable");
head(DateTimeTable)
```

Time	Price	Log_Return
01-Jan-2018 11:52:48	100	-0.025375
01-Jan-2018 13:23:13	101.14	0.011336
01-Jan-2018 14:45:09	101.5	0.0035531
01-Jan-2018 15:30:30	100.15	-0.01339
02-Jan-2018 10:43:37	99.72	-0.0043028
03-Jan-2018 10:02:21	100.11	0.0039033
03-Jan-2018 11:22:37	103.96	0.037737
03-Jan-2018 13:42:27	107.05	0.02929

First, aggregate intra-daily prices and returns to a monthly periodicity. To maintain consistency between prices and returns, for any given month aggregate prices by reporting the last recorded price using `"lastvalue"` and aggregate returns by summing all logarithmic returns using `"sum"`.

```
DTTMonthly1 = convert2monthly(DataTimeTable,Aggregation=["lastvalue" "sum"]);
head(DTTMonthly1)
```

Time	Price	Log_Return
31-Jan-2018	117.35	0.13462
28-Feb-2018	113.52	-0.033182
31-Mar-2018	110.74	-0.024794
30-Apr-2018	105.58	-0.047716
31-May-2018	97.88	-0.075727
30-Jun-2018	99.29	0.014303
31-Jul-2018	102.72	0.033962
31-Aug-2018	124.99	0.19623

Use `convert2annual` to aggregate the data to an annual periodicity and compare the results of the two different aggregation approaches. The first approach computes annual results by aggregating the monthly aggregates and the second approach computes annual results by directly aggregating the original intra-daily data. Notice that by default, `convert2annual` reports results on the last business day of December. To change the month that ends annual periods, use the `EndOfYearMonth` optional name-value argument for `convert2annual`.

```
DTTAnnual1 = convert2annual(DTTMonthly1,Aggregation=["lastvalue" "sum"]) % Monthly to annual
```

DTTAnnual1=3×2 timetable

Time	Price	Log_Return
31-Dec-2018	84.26	-0.19664
31-Dec-2019	153.22	0.59797
31-Dec-2020	301.04	0.67537

```
DTTAnnual2 = convert2annual(DataTimeTable,Aggregation=["lastvalue" "sum"]) % Intra-daily to semi-annual
```

DTTAnnual2=3×2 timetable

Time	Price	Log_Return
31-Dec-2018	84.26	-0.19664
31-Dec-2019	153.22	0.59797
31-Dec-2020	301.04	0.67537

The results of the two approaches are the same because each annual period contains exactly 12 calendar months.

Input Arguments

TT1 — Data to aggregate to annual periodicity

timetable

Data to aggregate to an annual periodicity, specified as a timetable.

Each variable can be a numeric vector (univariate series) or numeric matrix (multivariate series).

Note

- NaNs indicate missing values.
 - Timestamps must be in ascending or descending order.
-

By default, all days are business days. If your timetable does not account for nonbusiness days (weekends, holidays, and market closures), add business calendar awareness by using `addBusinessCalendar` first. For example, the following command adds business calendar logic to include only NYSE business days.

```
TT = addBusinessCalendar(TT);
```

Data Types: `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = convert2annual(TT1,'Aggregation',["lastvalue" "sum"])`

Aggregation — Aggregation method for TT1

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Aggregation method for `TT1` defining how to aggregate data over business days in a year to an annual periodicity, specified as one of the following methods, a string vector of methods, or a length `numVariables` cell vector of methods, where `numVariables` is the number of variables in `TT1`.

- "sum" — Sum the values in each year or day.
- "mean" — Calculate the mean of the values in each year or day.
- "prod" — Calculate the product of the values in each year or day.
- "min" — Calculate the minimum of the values in each year or day.
- "max" — Calculate the maximum of the values in each year or day.
- "firstvalue" — Use the first value in each year or day.
- "lastvalue" — Use the last value in each year or day.
- `@customfcn` — A custom aggregation method that accepts a table variable and returns a numeric scalar (for univariate series) or row vector (for multivariate series). The function must accept empty inputs `[]`.

If you specify a single method, `convert2annual` applies the specified method to all time series in `TT1`. If you specify a string vector or cell vector `aggregation`, `convert2annual` applies `aggregation(j)` to `TT1(:,j)`; `convert2annual` applies each aggregation method one at a time (for more details, see `retime`). For example, consider an input daily timetable with three variables.

Time	AAA	BBB	CCC	
01-Jan-2018	100.00	200.00	300.00	400.00
02-Jan-2018	100.03	200.06	300.09	400.12

03-Jan-2018	100.07	200.14	300.21	400.28
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
29-Dec-2018	249.16	498.32	747.48	996.64
30-Dec-2018	250.21	500.42	750.63	1000.84
31-Dec-2018	256.75	513.50	770.25	1027.00

By default, `convert2annual` applies the aggregation method "lastvalue", which reports for each variable the values of the last business day of each year. The aggregated annual results are as follows:

```
TT2 = convert2annual(TT1)
```

```
TT2 =
```

```
1×3 timetable
```

Time	AAA	BBB	CCC
31-Dec-2018	256.75	513.50	770.25 1027.00

All methods omit missing data (NaNs) in direct aggregation calculations on each variable. However, for situations in which missing values appear in the first row of `TT1`, missing values can also appear in the aggregated results `TT2`. To address missing data, write and specify a custom aggregation method (function handle) that supports missing data.

Data Types: char | string | cell | function_handle

Daily — Intra-day aggregation method for TT1

"lastvalue" (default) | "sum" | "prod" | "mean" | "min" | "max" | "firstvalue" | character vector | function handle | string vector | cell vector of character vectors or function handles

Intra-day aggregation method for `TT1`, specified as an aggregation method, a string vector of methods, or a length `numVariables` cell vector of methods. For more details on supported methods and behaviors, see the 'Aggregation' name-value argument.

Data Types: char | string | cell | function_handle

EndOfYearMonth — Month that ends annual periods

"December" (weeks end on Friday) (default) | integer with value 1 to 12 | "January" | "February" | "March" | "April" | "May" | "June" | "July" | "August" | "September" | "October" | "November" | "December" | character vector

Month that ends annual periods, specified as a value in this table.

Value	Month Ending Each Year
"January" or 1	January
"February" or 2	February
"March" or 3	March
"April" or 4	April
"May" or 5	May
"June" or 6	June

Value	Month Ending Each Year
"July" or 7	July
"August" or 8	August
"September" or 9	September
"October" or 10	October
"November" or 11	November
"December" or 12	December

Data Types: double | char | string

Output Arguments

TT2 — Annual data

timetable

Annual data, returned as a timetable. The time arrangement of TT1 and TT2 are the same.

If a variable of TT1 has no business-day records during an annual period within the sampling time span, `convert2annual` returns a NaN for that variable and annual period in TT2.

If the first annual period (`year1`) of TT1 contains at least one business day, the first date in TT2 is the last business date of `year1`. Otherwise, the first date in TT2 is the next end-of-year-period business date of TT1.

If the last annual period (`yearT`) of TT1 contains at least one business day, the last date in TT2 is the last business date of `yearT`. Otherwise, the last date in TT2 is the previous end-of-year-period business date of TT1.

Version History

Introduced in R2021a

See Also

`convert2daily` | `convert2weekly` | `convert2quarterly` | `convert2semiannual` | `convert2monthly` | `timetable` | `addBusinessCalendar`

Topics

"Resample and Aggregate Data in Timetable"

"Combine Timetables and Synchronize Their Data"

"Retime and Synchronize Timetable Variables Using Different Methods"

totalreturnprice

Total return price time series

Note totalreturnprice will be removed in a future release. Use adjustedClosingPrices instead.

Syntax

Return = totalreturnprice(Price,Action,Dividend)

Description

Return = totalreturnprice(Price,Action,Dividend) generates a total return price time series given price data, action or split data, and dividend data. All input data is unadjusted.

Input Arguments

Price — Price of security

matrix | table

Price of security, specified as a table or an NUMOBS-by-2 matrix. If Price is a table, the dates can either be a datetime array, string array, date character vectors, or serial date numbers. If Price is an NUMOBS-by-2 matrix of price data, column 1 contains MATLAB serial date numbers and column 2 contains price values.

Data Types: double | char | string | datetime | table

Action — Action or split data

matrix | table

Action or split data, specified as a table or an NUMOBS-by-2 matrix. If Action is a table, the dates can either be a datetime array, string array, date character vectors, or serial date numbers. If Action is an NUMOBS-by-2 matrix of price data, column 1 contains MATLAB serial date numbers and column 2 contains split ratios.

Data Types: double | char | string | datetime | table

Dividend — Dividend payouts

matrix | table

Dividend payouts, specified as a table or an NUMOBS-by-2 matrix. If Dividend is a table, the dates can either be serial date numbers, date character vectors, or datetime arrays. If Dividend is a NUMOBS-by-2 matrix of price data, column 1 contains MATLAB serial date numbers and column 2 contains dividend payouts.

Data Types: double | table

Output Arguments

Return — Total return price time series

matrix | table

Total return price time series, returned as a NUMOBS-by-2 matrix (if all inputs are matrices) or table (if any inputs are tables) of price data, where column 1 is dates and column 2 is total return price values. Dates in column 1 are in datetime format if any inputs specify dates in datetime format. Dates in column 1 are in date character vector format if no inputs specify dates in datetime format, but any of them use date character vector format. Otherwise, dates in column 1 are specified as serial date numbers. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

Version History

Introduced before R2006a

See Also

`adjustedClosingPrices` | `periodicreturns` | `datetime`

Topics

“Portfolio Construction Examples” on page 3-5

“Portfolio Optimization Functions” on page 3-3

adjustedClosingPrices

Adjust closing stock prices for splits and cash dividends

Syntax

```
TT2 = adjustedclosingprices(TT1)
TT2 = adjustedclosingprices( ___,Name=Value)
```

Description

TT2 = adjustedclosingprices(TT1) adjusts raw closing prices for splits and cash dividends.

adjustedclosingprices adjusts raw closing prices for stock splits by dividing by the split ratio (for example, dividing by 2 for a 2:1 split or 1.5 for a 3:2 split). adjustedclosingprices adjusts closing prices for cash dividends by scaling them by a dividend multiplier in which it expresses the dividend as a fraction of the last closing price preceding the ex-dividend date. To avoid negative prices, this scaling operation adheres to the Center for Research in Security Prices (CRSP) standard.

TT2 = adjustedclosingprices(___,Name=Value) specifies options using one or more name-value arguments in addition to the input arguments in the previous syntax.

Examples

Adjust Closing Stock Prices for Splits and Cash Dividends

This example shows how to adjust a time series of closing stock prices that includes a 2-for-1 split and pays a 0.17 cash dividend. First, you store the closing prices, split ratios, and cash dividends in timetable TT1 and then pass them to adjustedClosingPrices.

Create a timetable (TT1) for a single stock. TT1 contains raw closing prices, split ratios, and cash dividends, but TT1 can also contain other variables such as Open, High, Low, Volume, and so on.

```
dates = [(datetime(2023,2,18) + caldays(0:4)') ; datetime(2023,2,[25 26]')];
close = [100 102.79 53.12 53.01 53.10 52.20 52.22]';
split = [1 1 2 1 1 1 1]';
dividend = [0 0 0 0 0 0.17 0]';
TT1 = timetable(close,split,dividend,RowTimes=dates,VariableNames=["Close" "Split" "Dividend"])
```

TT1=7×3 timetable

Time	Close	Split	Dividend
18-Feb-2023	100	1	0
19-Feb-2023	102.79	1	0
20-Feb-2023	53.12	2	0
21-Feb-2023	53.01	1	0
22-Feb-2023	53.1	1	0
25-Feb-2023	52.2	1	0.17
26-Feb-2023	52.22	1	0

To adjust the closing stock prices for the splits and cash dividends in TT1, use `adjustedClosingPrices` and output the results in TT2.

```
TT2 = adjustedClosingPrices(TT1)
```

```
TT2=7x1 timetable
      Time      Adjusted_Close
      _____
18-Feb-2023      49.84
19-Feb-2023      51.23
20-Feb-2023      52.95
21-Feb-2023      52.84
22-Feb-2023      52.93
25-Feb-2023       52.2
26-Feb-2023      52.22
```

The output timetable TT2 has the same dates as TT1, but TT2 contains *only* the adjusted closing prices. However, since TT1 and TT2 share the same dates, you can combine the data of both timetables.

```
[TT1 TT2]
```

```
ans=7x4 timetable
      Time      Close      Split      Dividend      Adjusted_Close
      _____
18-Feb-2023      100         1         0         49.84
19-Feb-2023     102.79        1         0         51.23
20-Feb-2023      53.12        2         0         52.95
21-Feb-2023      53.01        1         0         52.84
22-Feb-2023      53.1         1         0         52.93
25-Feb-2023      52.2         1        0.17         52.2
26-Feb-2023      52.22        1         0         52.22
```

Input Arguments

TT1 — Timetable containing raw closing stock prices to adjust for splits and cash dividends
timetable

Timetable containing the raw closing stock prices to adjust for splits and cash dividends, specified as a timetable.

Note TT1 must have datetime stamps in strictly ascending or descending order. All observations in TT1 are associated with whole dates that you specify as datetime values but have no HH:MM:SS time component.

Optionally, TT1 can also contain split ratios and cash dividends per share.

Data Types: timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `TT2 = adjustedClosingPrices(TT1, ClosingPriceVariable="closing_price_variable")`

ClosingPriceVariable — Variable specification to select raw closing price variable from TT1

variable that contains "Close" (case insensitive) (default) | positive scalar numeric index | scalar variable name (string or character vector) | logical vector with one true element

Variable specification to select the raw closing price variable from `TT1`, specified as a positive scalar numeric index, a scalar variable name (string or character vector), or a logical vector with one true element.

Note If no variable contains "Close", then this function issues an error.

Data Types: double | char | string | logical

DividendVariable — Variable specification to select cash dividend variable from TT1

variable that contains "Dividend" (case insensitive) (default) | positive scalar numeric index | scalar variable name (string or character vector) | logical vector with one true element

Variable specification to select the cash dividend variable from `TT1`, specified as a positive scalar numeric index, a scalar variable name (string or character vector), or a logical vector with one true element.

Note If no variable contains "Dividend", then this function does not make a dividend adjustment.

Data Types: double | char | string | logical

SplitVariable — Variable specification to select split ratio variable from TT1

variable that contains "Split" (case insensitive) (default) | positive scalar numeric index | scalar variable name (string or character vector) | logical vector with one true element

Variable specification to select the split ratio variable from `TT1`, specified as a positive scalar numeric index, a scalar variable name (string or character vector), or a logical vector with one true element.

Note If no variable contains "Split", then this function makes split adjustment.

Data Types: double | char | string | logical

Output Arguments

TT2 — Adjusted closing prices associated with raw closing prices of variable ClosingPriceVariable

timetable

Timetable with the same dates as TT1 containing adjusted closing prices associated with the raw closing prices of variable ClosingPriceVariable, returned as a timetable.

If the raw closing price variable in TT1 contains "Close", then the function obtains the adjusted closing price variable by inserting "Adjusted_" immediately before "Close". Otherwise, the function obtains the adjusted closing price variable by prepending "Adjusted_" to the variable in TT1 containing the raw closing prices. If the function makes no adjustments, it copies the raw closing prices of TT1 to the adjusted closing prices of TT2. If the dates of TT1 are in ascending or descending order, then the dates of TT2 are in ascending or descending order, respectively.

The naming convention of the adjusted closing price variable in TT2 is:

- If the closing price variable in TT1 is "XYZ_Close" (TT1.XYZ_Close), then the adjusted closing price variable in TT2 is "XYZ_Adjusted_Close" (TT2.XYZ_Adjusted_Close).
- If the closing price variable in TT1 is "XYZ" (TT1.XYZ), then the adjusted closing price variable in TT2 is "Adjusted_XYZ" (TT2.Adjusted_XYZ).

Version History

Introduced in R2024a

See Also

timetable | rollingreturns | datetime

tr2bonds

Term-structure parameters given Treasury bond parameters

Syntax

```
[Bonds,Prices,Yields] = tr2bonds(TreasuryMatrix,Settle)
[Bonds,Prices,Yields] = tr2bonds( ___,Settle)
```

Description

[Bonds,Prices,Yields] = tr2bonds(TreasuryMatrix,Settle) returns term-structure parameters (Bonds, Prices, and Yields) sorted by ascending maturity date, given Treasury bond parameters. The formats of the output matrix and vectors meet requirements for input to the zbtprice and zbtyield zero-curve bootstrapping functions.

[Bonds,Prices,Yields] = tr2bonds(___,Settle) adds an optional argument for Settle.

Examples

Return Term-Structure Parameters Given Treasury Bond Parameters

This example shows how to return term-structure parameters (bond information, prices, and yields) sorted by ascending maturity date, given Treasury bond market parameters for December 22, 1997.

```
Matrix =[0.0650 datenum('15-apr-1999') 101.03125 101.09375 0.0564
         0.05125 datenum('17-dec-1998') 99.4375 99.5 0.0563
         0.0625 datenum('30-jul-1998') 100.3125 100.375 0.0560
         0.06125 datenum('26-mar-1998') 100.09375 100.15625 0.0546];
```

```
[Bonds, Prices, Yields] = tr2bonds(Matrix)
```

```
Bonds = 4×6
105 ×
```

7.2984	0.0000	0.0010	0.0000	0	0.0000
7.2997	0.0000	0.0010	0.0000	0	0.0000
7.3011	0.0000	0.0010	0.0000	0	0.0000
7.3022	0.0000	0.0010	0.0000	0	0.0000

```
Prices = 4×1
```

```
100.1562
100.3750
99.5000
101.0938
```

```
Yields = 4×1
```

```
0.0546
```



```
0.0560
0.0563
0.0564
```

Return Term-Structure Parameters Given Treasury Bond Parameters Using datetime Input

This example shows how to use `datetime` input to return term-structure parameters (bond information, prices, and yields) sorted by ascending maturity date, given Treasury bond market parameters for December 22, 1997.

```
Matrix =[0.0650 datenum('15-apr-1999') 101.03125 101.09375 0.0564
         0.05125 datenum('17-dec-1998') 99.4375 99.5 0.0563
         0.0625 datenum('30-jul-1998') 100.3125 100.375 0.0560
         0.06125 datenum('26-mar-1998') 100.09375 100.15625 0.0546];
```

```
t=array2table(Matrix);
t.Matrix2=datetime(t{:,2},'ConvertFrom','datenum','Locale','en_US');
[Bonds, Prices, Yields] = tr2bonds(t,datetime(1997,1,1))
```

Bonds=4×6 table

Maturity	CouponRate	Face	Period	Basis	EndMonthRule
26-Mar-1998	0.06125	100	2	0	1
30-Jul-1998	0.0625	100	2	0	1
17-Dec-1998	0.05125	100	2	0	1
15-Apr-1999	0.065	100	2	0	1

Prices = 4×1

```
100.1562
100.3750
99.5000
101.0938
```

Yields = 4×1

```
0.0598
0.0599
0.0540
0.0598
```

Input Arguments

TreasuryMatrix — Treasury bond parameters

table | matrix

Treasury bond parameters, specified as a 5-column table or a NumBonds-by-5 matrix of bond information where the table columns or matrix columns contains:

- **CouponRate** (Required) Coupon rate of the Treasury bond, specified as a decimal indicating the coupon rates for each bond in the portfolio.
- **Maturity** (Required) Maturity date of the Treasury bond, specified as a serial date number when using a matrix. Use `datenum` to convert date character vectors to serial date numbers. If the input `TreasuryMatrix` is a table, the **Maturity** dates can be a datetime array, string array, or date character vectors.
- **Bid** (Required) Bid prices, specified using an integer-decimal form for each bond in the portfolio.
- **Asked** (Required) Asked prices, specified using an integer-decimal form for each bond in the portfolio.
- **AskYield** (Required) Quoted ask yield, specified using a decimal form for each bond in the portfolio.

Data Types: `double` | `char` | `string` | `datetime` | `table`

Settle — Settlement date of Treasury bond

`datetime array` | `string array` | `date character vector`

(Optional) Settlement date of the Treasury bond, specified as a scalar or a `NINST-by-1` vector using a datetime array, string array, or date character vectors. The **Settle** date must be before the **Maturity** date.

To support existing code, `tr2bonds` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Output Arguments

Bonds — Coupon bond information

`table` | `matrix`

Coupon bond information, returned as a table or matrix depending on the `TreasuryMatrix` input.

When `TreasuryMatrix` is a table, **Bonds** is also a table, and the variable type for the **Maturity** dates in **Bonds** (column 1) matches the variable type for **Maturity** in `TreasuryMatrix`.

When `TreasuryMatrix` input is a `n-by-5` matrix, then each row describes a bond.

The parameters or columns returned for **Bonds** are:

- **Maturity** (Column 1) Maturity date for each bond in the portfolio. The format of the dates matches the format used for **Maturity** in `TreasuryMatrix` (datetime array, string array, date character vector, or serial date number).
- **CouponRate** (Column 2) Coupon rate for each bond in the portfolio in decimal form.
- **Face** (Column 3, Optional) Face or par value for each bond in the portfolio. The default is 100.
- **Period** (Column 4, Optional) Number of coupon payments per year for each bond in the portfolio with allowed values: 1, 2, 3, 4, 6, and 12. The default is 2, unless you are dealing with zero coupons, then **Period** is 0 instead of 2.
- **Basis** (Column 5, Optional) Day-count basis for each bond in the portfolio with possible values:
 - 0 = actual/actual (default)

- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252
- For more information, see “Basis” on page 2-16.
- **EndMonthRule** (Column 6, Optional) End-of-month rule flag for each bond in the portfolio. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days. 0 = ignore rule, meaning that a bond's coupon payment date is always the same numerical day of the month. 1 = set rule on, meaning that a bond's coupon payment date is always the last actual day of the month. The default is 1.

Prices — Bond prices

numeric

Bond prices, returned as a column vector containing the price of each bond in **Bonds**, respectively. The number of rows (*n*) matches the number of rows in **Bonds**.

Yields — Bond yields

numeric

Bond yields, returned as a column vector containing the yield to maturity of each bond in **Bonds**, respectively. The number of rows (*n*) matches the number of rows in **Bonds**.

If the optional input argument **Settle** is used, **Yields** is computed as a semiannual yield to maturity. If the input **Settle** is not used, the quoted input yields are used.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although **tr2bonds** supports serial date numbers, **datetime** values are recommended instead. The **datetime** data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to **datetime** values, use the **datetime** function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");  
y = year(t)  
  
y =  
  
2021
```

There are no plans to remove support for serial date number inputs.

See Also

tbl2bond | zbtprice | zbtyield | datetime

Topics

"Term Structure of Interest Rates" on page 2-29

"Treasury Bills Defined" on page 2-25

transprob

Estimate transition probabilities from credit ratings data

Syntax

```
[transMat,sampleTotals,idTotals] = transprob(data)
[transMat,sampleTotals,idTotals] = transprob( ___,Name,Value)
```

Description

[transMat,sampleTotals,idTotals] = transprob(data) constructs a transition matrix from historical data of credit ratings.

[transMat,sampleTotals,idTotals] = transprob(___,Name,Value) adds optional name-value pair arguments.

Examples

Construct a Transition Matrix From a Table of Historical Data of Credit Ratings

Using the historical credit rating table as input data from Data_TransProb.mat display the first ten rows and compute the transition matrix:

```
load Data_TransProb
data(1:10,:)
```

```
ans=10x3 table
      ID      Date      Rating
      ---      ---      ---
{'00010283'} {'10-Nov-1984'} {'CCC'}
{'00010283'} {'12-May-1986'} {'B'  }
{'00010283'} {'29-Jun-1988'} {'CCC'}
{'00010283'} {'12-Dec-1991'} {'D'  }
{'00013326'} {'09-Feb-1985'} {'A'  }
{'00013326'} {'24-Feb-1994'} {'AA' }
{'00013326'} {'10-Nov-2000'} {'BBB'}
{'00014413'} {'23-Dec-1982'} {'B'  }
{'00014413'} {'20-Apr-1988'} {'BB' }
{'00014413'} {'16-Jan-1998'} {'B'  }
```

```
% Estimate transition probabilities with default settings
transMat = transprob(data)
```

```
transMat = 8x8

    93.1170    5.8428    0.8232    0.1763    0.0376    0.0012    0.0001    0.0017
    1.6166    93.1518    4.3632    0.6602    0.1626    0.0055    0.0004    0.0396
    0.1237    2.9003    92.2197    4.0756    0.5365    0.0661    0.0028    0.0753
```

```

0.0236    0.2312    5.0059    90.1846    3.7979    0.4733    0.0642    0.2193
0.0216    0.1134    0.6357    5.7960    88.9866    3.4497    0.2919    0.7050
0.0010    0.0062    0.1081    0.8697    7.3366    86.7215    2.5169    2.4399
0.0002    0.0011    0.0120    0.2582    1.4294    4.2898    81.2927    12.7167
      0          0          0          0          0          0          0    100.0000

```

Using the historical credit rating table input data from `Data_TransProb.mat`, compute the transition matrix using the cohort algorithm:

```

%Estimate transition probabilities with 'cohort' algorithm
transMatCoh = transprob(data,'algorithm','cohort')

```

```
transMatCoh = 8×8
```

```

93.1345    5.9335    0.7456    0.1553    0.0311         0         0         0
 1.7359   92.9198    4.5446    0.6046    0.1560         0         0    0.0390
 0.1268    2.9716   91.9913    4.3124    0.4711    0.0544         0    0.0725
 0.0210    0.3785    5.0683   89.7792    4.0379    0.4627    0.0421    0.2103
 0.0221    0.1105    0.6851    6.2320   88.3757    3.6464    0.2873    0.6409
      0          0    0.0761    0.7230    7.9909   86.1872    2.7397    2.2831
      0          0          0    0.3094    1.8561    4.5630   80.8971   12.3743
      0          0          0          0          0          0         0    100.0000

```

Using the historical credit rating data with ratings investment grade ('IG'), speculative grade ('SG'), and default ('D'), from `Data_TransProb.mat` display the first ten rows and compute the transition matrix:

```
dataIGSG(1:10,:)
```

```
ans=10×3 table
```

ID	Date	Rating
{ '00011253' }	{ '04-Apr-1983' }	{ 'IG' }
{ '00012751' }	{ '17-Feb-1985' }	{ 'SG' }
{ '00012751' }	{ '19-May-1986' }	{ 'D' }
{ '00014690' }	{ '17-Jan-1983' }	{ 'IG' }
{ '00012144' }	{ '21-Nov-1984' }	{ 'IG' }
{ '00012144' }	{ '25-Mar-1992' }	{ 'SG' }
{ '00012144' }	{ '07-May-1994' }	{ 'IG' }
{ '00012144' }	{ '23-Jan-2000' }	{ 'SG' }
{ '00012144' }	{ '20-Aug-2001' }	{ 'IG' }
{ '00012937' }	{ '07-Feb-1984' }	{ 'IG' }

```
transMatIGSG = transprob(dataIGSG,'labels',{'IG','SG','D'})
```

```
transMatIGSG = 3×3
```

```

98.6719    1.2020    0.1261
 3.5781   93.3318    3.0901
      0          0    100.0000

```

Using the historical credit rating data with numeric ratings for investment grade (1), speculative grade (2), and default (3), from `Data_TransProb.mat` display the first ten rows and compute the transition matrix:

```
dataIGSGnum(1:10,:)
```

```
ans=10x3 table
      ID      Date      Rating
      ---      ---      ---
{'00011253'} {'04-Apr-1983'}    1
{'00012751'} {'17-Feb-1985'}    2
{'00012751'} {'19-May-1986'}    3
{'00014690'} {'17-Jan-1983'}    1
{'00012144'} {'21-Nov-1984'}    1
{'00012144'} {'25-Mar-1992'}    2
{'00012144'} {'07-May-1994'}    1
{'00012144'} {'23-Jan-2000'}    2
{'00012144'} {'20-Aug-2001'}    1
{'00012937'} {'07-Feb-1984'}    1
```

```
transMatIGSGnum = transprob(dataIGSGnum,'labels',{1,2,3})
```

```
transMatIGSGnum = 3x3
```

```
98.6719    1.2020    0.1261
 3.5781   93.3318    3.0901
      0         0 100.0000
```

Create a Transition Matrix Using a Cell Array for Historical Data of Credit Ratings

Using a MATLAB® table containing the historical credit rating cell array input data (dataCellFormat) from Data_TransProb.mat, estimate the transition probabilities with default settings.

```
load Data_TransProb
transMat = transprob(dataCellFormat)
```

```
transMat = 8x8
```

```
93.1170    5.8428    0.8232    0.1763    0.0376    0.0012    0.0001    0.0017
 1.6166   93.1518    4.3632    0.6602    0.1626    0.0055    0.0004    0.0396
 0.1237    2.9003   92.2197    4.0756    0.5365    0.0661    0.0028    0.0753
 0.0236    0.2312    5.0059   90.1846    3.7979    0.4733    0.0642    0.2193
 0.0216    0.1134    0.6357    5.7960   88.9866    3.4497    0.2919    0.7050
 0.0010    0.0062    0.1081    0.8697    7.3366   86.7215    2.5169    2.4399
 0.0002    0.0011    0.0120    0.2582    1.4294    4.2898   81.2927   12.7167
      0         0         0         0         0         0         0 100.0000
```

Using the historical credit rating cell array input data (dataCellFormat), compute the transition matrix using the cohort algorithm:

```
%Estimate transition probabilities with 'cohort' algorithm
transMatCoh = transprob(dataCellFormat,'algorithm','cohort')
```

```
transMatCoh = 8x8
```

```

93.1345    5.9335    0.7456    0.1553    0.0311         0         0         0
 1.7359   92.9198    4.5446    0.6046    0.1560         0         0    0.0390
 0.1268    2.9716   91.9913    4.3124    0.4711    0.0544         0    0.0725
 0.0210    0.3785    5.0683   89.7792    4.0379    0.4627    0.0421    0.2103
 0.0221    0.1105    0.6851    6.2320   88.3757    3.6464    0.2873    0.6409
      0         0    0.0761    0.7230    7.9909   86.1872    2.7397    2.2831
      0         0         0    0.3094    1.8561    4.5630   80.8971   12.3743
      0         0         0         0         0         0         0   100.0000

```

Visualize Transitions Data for transprob

This example shows how to visualize credit rating transitions that are used as an input to the `transprob` function. The example also describes how the `transprob` function treats rating transitions when the company data starts after the start date of the analysis, or when the end date of the analysis is after the last transition observed.

Sample Data

Set up fictitious sample data for illustration purposes.

```

data = {'ABC', '17-Feb-2015', 'AA';
        'ABC', '6-Jul-2017', 'A';
        'LMN', '12-Aug-2014', 'B';
        'LMN', '9-Nov-2015', 'CCC';
        'LMN', '7-Sep-2016', 'D';
        'XYZ', '14-May-2013', 'BB';
        'XYZ', '21-Jun-2016', 'BBB'};
data = cell2table(data, 'VariableNames', {'ID', 'Date', 'Rating'});
disp(data)

```

ID	Date	Rating
{ 'ABC' }	{ '17-Feb-2015' }	{ 'AA' }
{ 'ABC' }	{ '6-Jul-2017' }	{ 'A' }
{ 'LMN' }	{ '12-Aug-2014' }	{ 'B' }
{ 'LMN' }	{ '9-Nov-2015' }	{ 'CCC' }
{ 'LMN' }	{ '7-Sep-2016' }	{ 'D' }
{ 'XYZ' }	{ '14-May-2013' }	{ 'BB' }
{ 'XYZ' }	{ '21-Jun-2016' }	{ 'BBB' }

The `transprob` function understands that this panel-data format indicates the dates when a new rating is assigned to a given company. `transprob` assumes that such ratings remain unchanged, unless a subsequent row explicitly indicates a rating change. For example, for company 'ABC', `transprob` understands that the 'A' rating is unchanged for any date after '6-Jul-2017' (indefinitely).

Compute Transition Matrix and Transition Counts

The `transprob` function returns a transition probability matrix as the primary output. There are also optional outputs that contain additional information for how many transitions occurred. For more information, see `transprob` for information on the optional outputs for both the 'cohort' and the 'duration' methods.

For illustration purposes, this example allows you to pick the `StartYear` (limited to 2014 or 2015 for this example) and the `EndYear` (2016 or 2017). This example also uses the `hDisplayTransitions` helper function (see the Local Functions on page 15-1672 section) to format the transitions information for ease of reading.

```
StartYear = 2014 ;
EndYear = 2017 ;
startDate = datetime(StartYear,12,31,'Locale','en_US');
endDate = datetime(EndYear,12,31,'Locale','en_US');
RatingLabels = ["AAA","AA","A","BBB","BB","B","CCC","D"];

[tm,st,it] = transprob(data,'startDate',startDate,'endDate',endDate,'algorithm','cohort','labels'
```

The transition probabilities of the `TransMat` output indicate the probability of migrating between ratings. The probabilities are expressed in %, that is, they are multiplied by 100.

```
hDisplayTransitions(tm,RatingLabels,"Transition Matrix")
```

Transition Matrix

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	100	0	0	0	0	0	0	0
AA	0	50	50	0	0	0	0	0
A	0	0	100	0	0	0	0	0
BBB	0	0	0	100	0	0	0	0
BB	0	0	0	50	50	0	0	0
B	0	0	0	0	0	0	100	0
CCC	0	0	0	0	0	0	0	100
D	0	0	0	0	0	0	0	100

The transition counts are stored in the `sampleTotals` optional output and indicate how many transitions occurred between ratings for the entire sample (that is, all companies).

```
hDisplayTransitions(st.totalsMat,RatingLabels,"Transition counts, all companies")
```

Transition counts, all companies

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	0	0	0	0	0	0	0	0
AA	0	1	1	0	0	0	0	0
A	0	0	0	0	0	0	0	0
BBB	0	0	0	1	0	0	0	0
BB	0	0	0	1	1	0	0	0
B	0	0	0	0	0	0	1	0
CCC	0	0	0	0	0	0	0	1
D	0	0	0	0	0	0	0	1

The third output of `transprob` is `idTotals` that contains information about transitions at an ID level, company by company (in the same order that the companies appear in the input data).

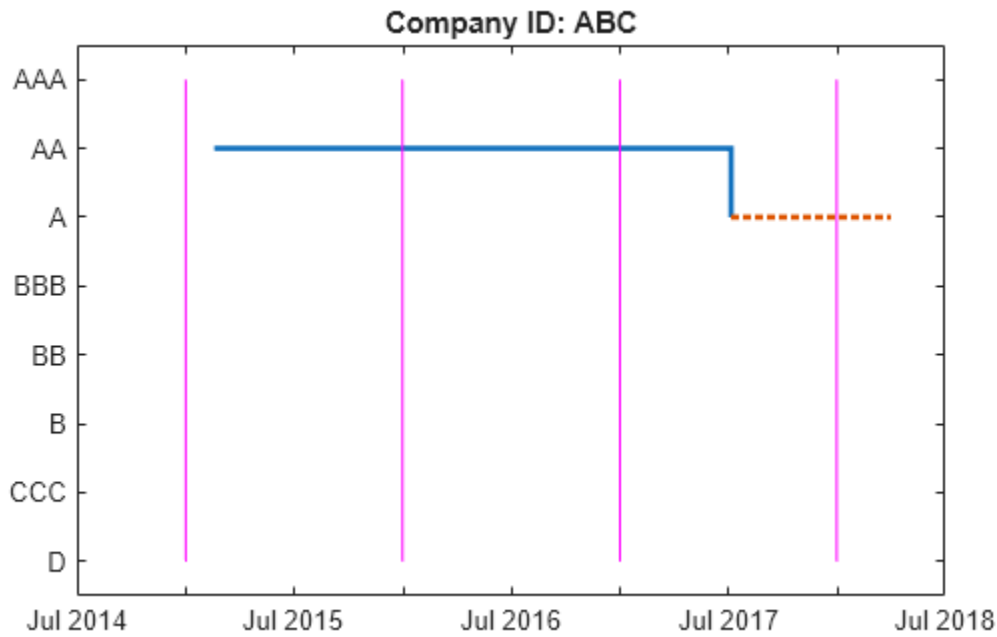
Select a company to display the transition counts and a corresponding visualization of the transitions. The `hPlotTransitions` helper function (see the Local Functions on page 15-1672 section) shows the transitions history for a company.

```
CompanyID = ;  
UniqueIDs = unique(data.ID,'stable');  
[~,CompanyIndex] = ismember(CompanyID,UniqueIDs);  
hDisplayTransitions(it(CompanyIndex).totalsMat,RatingLabels, strcat("Transition counts, company ID: ABC"))
```

Transition counts, company ID: ABC

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	0	0	0	0	0	0	0	0
AA	0	1	1	0	0	0	0	0
A	0	0	0	0	0	0	0	0
BBB	0	0	0	0	0	0	0	0
BB	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
CCC	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0

```
hPlotTransitions(CompanyID,startDate,endDate,data,RatingLabels)
```



To understand how `transprob` handles data when the first observed date is after the start date of the analysis, or whose last observed date occurs before the end date of the analysis, consider the following example. For company 'ABC' suppose that the analysis has a start date of 31-Dec-2014 and end date of 31-Dec-2017. There are only two transitions reported for this company for that analysis time window. The first observation for 'ABC' happened on 17-Feb-2015. So the 31-Dec-2015 snapshot is the first time the company is observed. By 31-Dec-2016, the company remained in the original 'AA' rating. By 31-Dec-2017, a downgrade to 'A' is recorded. Consistent

with this, the transition counts show one transition from 'AA' to 'AA' (from the end of 2015 to the end of 2016), and one transition from 'AA' to 'A' (from the end of 2016 to the end of 2017). The plot shows the last rating as a dotted red line to emphasize that the last rating in the data is extrapolated indefinitely into the future. There is no extrapolation into the past; the company's history is ignored until a company rating is known for an entire transition period (31-Dec-2015 through 31-Dec-2016 in the case of 'ABC').

Compute Transition Matrix Containing NR (Not Rated) Rating

Consider a different sample data containing only a single company 'DEF'. The data contains transitions of company 'DEF' from 'A' to 'NR' rating and a subsequent transition from 'NR' to 'BBB'.

```
dataNR = {'DEF', '17-Mar-2011', 'A';
          'DEF', '24-Mar-2014', 'NR';
          'DEF', '26-Sep-2016', 'BBB'};
dataNR = cell2table(dataNR, 'VariableNames', {'ID', 'Date', 'Rating'});
disp(dataNR)
```

ID	Date	Rating
{'DEF'}	{'17-Mar-2011'}	{'A' }
{'DEF'}	{'24-Mar-2014'}	{'NR' }
{'DEF'}	{'26-Sep-2016'}	{'BBB'}

transprob treats 'NR' as another rating. The transition matrix below shows the estimated probability of transitioning into and out of 'NR'.

```
StartYearNR = 2010;
EndYearNR = 2018;
startDateNR = datetime(StartYearNR,12,31,'Locale','en_US');
endDateNR = datetime(EndYearNR,12,31,'Locale','en_US');
CompanyID_NR = "DEF";
```

```
RatingLabelsNR = ["AAA", "AA", "A", "BBB", "BB", "B", "CCC", "D", "NR"];
```

```
[tmNR,~,itNR] = transprob(dataNR,'startDate',startDateNR,'endDate',endDateNR,'algorithm','cohort');
hDisplayTransitions(tmNR,RatingLabelsNR,"Transition Matrix")
```

Transition Matrix

	AAA	AA	A	BBB	BB	B	CCC	D	NR
AAA	100	0	0	0	0	0	0	0	0
AA	0	100	0	0	0	0	0	0	0
A	0	0	66.667	0	0	0	0	0	33.333
BBB	0	0	0	100	0	0	0	0	0
BB	0	0	0	0	100	0	0	0	0
B	0	0	0	0	0	100	0	0	0
CCC	0	0	0	0	0	0	100	0	0
D	0	0	0	0	0	0	0	100	0
NR	0	0	0	50	0	0	0	0	50

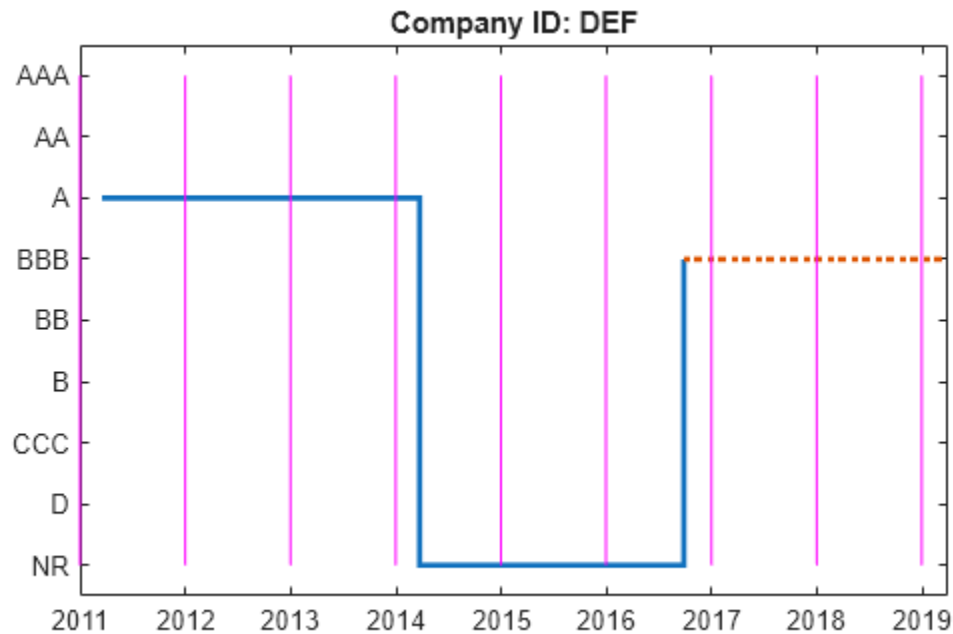
Display the transition counts and corresponding visualization of the transitions.

```
hDisplayTransitions(itNR.totalsMat,RatingLabelsNR, strcat("Transition counts, company ID: ", CompanyID_NR))
```

Transition counts, company ID: DEF

	AAA	AA	A	BBB	BB	B	CCC	D	NR
AAA	0	0	0	0	0	0	0	0	0
AA	0	0	0	0	0	0	0	0	0
A	0	0	2	0	0	0	0	0	1
BBB	0	0	0	2	0	0	0	0	0
BB	0	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0	0
CCC	0	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0	0
NR	0	0	0	1	0	0	0	0	1

hPlotTransitions(CompanyID_NR,startDateNR,endDateNR,dataNR,RatingLabelsNR)



To remove the 'NR' from the transition matrix, use the 'excludeLabels' name-value input argument in transprob. The list of labels to exclude may or may not be specified in the name-value pair argument labels. For example, both RatingLabels and RatingLabelsNR generate the same output from transprob.

```
[tmNR,stNR,itNR] = transprob(dataNR,'startDate',startDateNR,'endDate',endDateNR,'algorithm','coh  
hDisplayTransitions(tmNR,RatingLabels,"Transition Matrix")
```

Transition Matrix

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	100	0	0	0	0	0	0	0
AA	0	100	0	0	0	0	0	0
A	0	0	100	0	0	0	0	0

BBB	0	0	0	100	0	0	0	0
BB	0	0	0	0	100	0	0	0
B	0	0	0	0	0	100	0	0
CCC	0	0	0	0	0	0	100	0
D	0	0	0	0	0	0	0	100

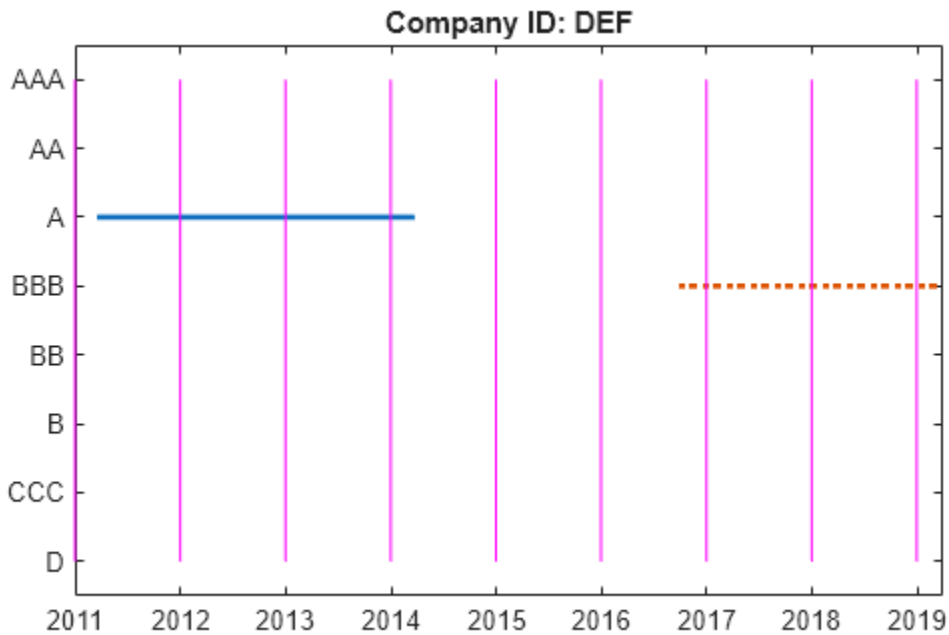
Display the transition counts and corresponding visualization of the transitions.

```
hDisplayTransitions(itNR.totalsMat,RatingLabels, strcat("Transition counts, company ID: ",CompanyID))
```

Transition counts, company ID: DEF

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	0	0	0	0	0	0	0	0
AA	0	0	0	0	0	0	0	0
A	0	0	2	0	0	0	0	0
BBB	0	0	0	2	0	0	0	0
BB	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
CCC	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0

```
hPlotTransitions(CompanyID_NR,startDateNR,endDateNR,dataNR,RatingLabels)
```



Consistent with the previous plot, the transition counts still show two transitions from 'A' to 'A' (from the end of 2012 to the end of 2014), and two transitions from 'BBB' to 'BBB' (from the end of 2017 to the end of 2019).

However, different from the previous plot, specifying 'NR' using the 'excludeLabels' name-value input argument of `transprob` removes any transitions into and out of the 'NR' rating.

Local Functions

```
function hDisplayTransitions(TransitionsData,RatingLabels,Title)
% Helper function to format transition information outputs

TransitionsAsTable = array2table(TransitionsData,...
    'VariableNames',RatingLabels,'RowNames',RatingLabels);

fprintf('\n%s\n\n',Title)
disp(TransitionsAsTable)

end

function hPlotTransitions(CompanyID,startDate,endDate,data,RatingLabels)
% Helper function to visualize transitions between ratings

Ind = string(data.ID)==CompanyID;
DatesOriginal = datetime(data.Date(Ind),'Locale','en_US');
RatingsOriginal = categorical(data.Rating(Ind),flipud(RatingLabels(:)),flipud(RatingLabels(:))

stairs(DatesOriginal,RatingsOriginal,'LineWidth',2)
hold on;

% Indicate rating extrapolated into the future (arbitrarily select 91
% days after endDate as the last date on the plot)
endDateExtrap = endDate+91;
if endDateExtrap>DatesOriginal(end)
    DatesExtrap = [DatesOriginal(end); endDateExtrap];
    RatingsExtrap = [RatingsOriginal(end); RatingsOriginal(end)];
    stairs(DatesExtrap,RatingsExtrap,'LineWidth',2,'LineStyle',':')
end
hold off;

% Add lines to indicate the snapshot dates
% transprob uses 1 as the default for 'snapsPerYear', hardcoded here for simplicity
% The call to cfdates generates the exact same snapshot dates that transprob uses
snapsPerYear = 1;
snapDates = cfdates(startDate-1,endDate,snapsPerYear)';
yLimits = ylim;
for ii=1:length(snapDates)
    line([snapDates(ii) snapDates(ii)],yLimits,'Color','m')
end
title(strcat("Company ID: ",CompanyID))
end
```

Visualize Transitions Data for transprob

This example shows how to visualize credit rating transitions that are used as an input to the `transprob` function. The example also describes how the `transprob` function treats rating transitions when the company data starts after the start date of the analysis, or when the end date of the analysis is after the last transition observed.

Sample Data

Set up fictitious sample data for illustration purposes.

```
data = {'ABC', '17-Feb-2015', 'AA';
        'ABC', '6-Jul-2017', 'A';
        'LMN', '12-Aug-2014', 'B';
        'LMN', '9-Nov-2015', 'CCC';
        'LMN', '7-Sep-2016', 'D';
        'XYZ', '14-May-2013', 'BB';
        'XYZ', '21-Jun-2016', 'BBB'};
data = cell2table(data, 'VariableNames', {'ID', 'Date', 'Rating'});
disp(data)
```

ID	Date	Rating
{ 'ABC' }	{ '17-Feb-2015' }	{ 'AA' }
{ 'ABC' }	{ '6-Jul-2017' }	{ 'A' }
{ 'LMN' }	{ '12-Aug-2014' }	{ 'B' }
{ 'LMN' }	{ '9-Nov-2015' }	{ 'CCC' }
{ 'LMN' }	{ '7-Sep-2016' }	{ 'D' }
{ 'XYZ' }	{ '14-May-2013' }	{ 'BB' }
{ 'XYZ' }	{ '21-Jun-2016' }	{ 'BBB' }

The `transprob` function understands that this panel-data format indicates the dates when a new rating is assigned to a given company. `transprob` assumes that such ratings remain unchanged, unless a subsequent row explicitly indicates a rating change. For example, for company 'ABC', `transprob` understands that the 'A' rating is unchanged for any date after '6-Jul-2017' (indefinitely).

Compute Transition Matrix and Transition Counts

The `transprob` function returns a transition probability matrix as the primary output. There are also optional outputs that contain additional information for how many transitions occurred. For more information, see `transprob` for information on the optional outputs for both the 'cohort' and the 'duration' methods.

For illustration purposes, this example allows you to pick the `StartYear` (limited to 2014 or 2015 for this example) and the `EndYear` (2016 or 2017). This example also uses the `hDisplayTransitions` helper function (see the Local Functions on page 15-1672 section) to format the transitions information for ease of reading.

```
StartYear = 2014 ;
EndYear = 2017 ;
startDate = datetime(StartYear,12,31,'Locale','en_US');
endDate = datetime(EndYear,12,31,'Locale','en_US');
RatingLabels = ["AAA","AA","A","BBB","BB","B","CCC","D"];

[tm,st,it] = transprob(data,'startDate',startDate,'endDate',endDate,'algorithm','cohort','labels'
```

The transition probabilities of the `TransMat` output indicate the probability of migrating between ratings. The probabilities are expressed in %, that is, they are multiplied by 100.

```
hDisplayTransitions(tm,RatingLabels,'Transition Matrix')
```

Transition Matrix

AAA	AA	A	BBB	BB	B	CCC	D
-----	----	---	-----	----	---	-----	---

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	100	0	0	0	0	0	0	0
AA	0	50	50	0	0	0	0	0
A	0	0	100	0	0	0	0	0
BBB	0	0	0	100	0	0	0	0
BB	0	0	0	50	50	0	0	0
B	0	0	0	0	0	0	100	0
CCC	0	0	0	0	0	0	0	100
D	0	0	0	0	0	0	0	100

The transition counts are stored in the `sampleTotals` optional output and indicate how many transitions occurred between ratings for the entire sample (that is, all companies).

```
hDisplayTransitions(st.totalsMat,RatingLabels,"Transition counts, all companies")
```

Transition counts, all companies

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	0	0	0	0	0	0	0	0
AA	0	1	1	0	0	0	0	0
A	0	0	0	0	0	0	0	0
BBB	0	0	0	1	0	0	0	0
BB	0	0	0	1	1	0	0	0
B	0	0	0	0	0	0	1	0
CCC	0	0	0	0	0	0	0	1
D	0	0	0	0	0	0	0	1

The third output of `transprob` is `idTotals` that contains information about transitions at an ID level, company by company (in the same order that the companies appear in the input data).

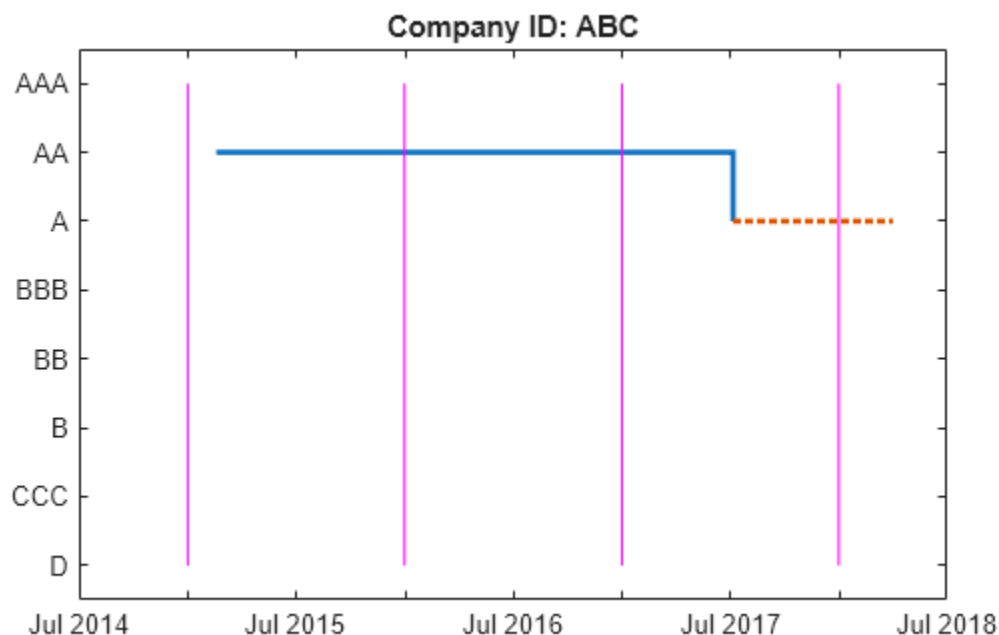
Select a company to display the transition counts and a corresponding visualization of the transitions. The `hPlotTransitions` helper function (see the Local Functions on page 15-1672 section) shows the transitions history for a company.

```
CompanyID = ;
UniqueIDs = unique(data.ID,'stable');
[~,CompanyIndex] = ismember(CompanyID,UniqueIDs);
hDisplayTransitions(it(CompanyIndex).totalsMat,RatingLabels, strcat("Transition counts, company ID: ", CompanyID))
```

Transition counts, company ID: ABC

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	0	0	0	0	0	0	0	0
AA	0	1	1	0	0	0	0	0
A	0	0	0	0	0	0	0	0
BBB	0	0	0	0	0	0	0	0
BB	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
CCC	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0

```
hPlotTransitions(CompanyID,startDate,endDate,data,RatingLabels)
```

To understand how `transprob` handles data when the first observed date is after the start date of the analysis, or whose last observed date occurs before the end date of the analysis, consider the following example. For company 'ABC' suppose that the analysis has a start date of 31-Dec-2014 and end date of 31-Dec-2017. There are only two transitions reported for this company for that analysis time window. The first observation for 'ABC' happened on 17-Feb-2015. So the 31-Dec-2015 snapshot is the first time the company is observed. By 31-Dec-2016, the company remained in the original 'AA' rating. By 31-Dec-2017, a downgrade to 'A' is recorded. Consistent with this, the transition counts show one transition from 'AA' to 'AA' (from the end of 2015 to the end of 2016), and one transition from 'AA' to 'A' (from the end of 2016 to the end of 2017). The plot shows the last rating as a dotted red line to emphasize that the last rating in the data is extrapolated indefinitely into the future. There is no extrapolation into the past; the company's history is ignored until a company rating is known for an entire transition period (31-Dec-2015 through 31-Dec-2016 in the case of 'ABC').

Compute Transition Matrix Containing NR (Not Rated) Rating

Consider a different sample data containing only a single company 'DEF'. The data contains transitions of company 'DEF' from 'A' to 'NR' rating and a subsequent transition from 'NR' to 'BBB'.

```
dataNR = {'DEF', '17-Mar-2011', 'A';
          'DEF', '24-Mar-2014', 'NR';
          'DEF', '26-Sep-2016', 'BBB'};
dataNR = cell2table(dataNR, 'VariableNames', {'ID', 'Date', 'Rating'});
disp(dataNR)
```

ID	Date	Rating
{'DEF'}	{'17-Mar-2011'}	{'A' }
{'DEF'}	{'24-Mar-2014'}	{'NR' }
{'DEF'}	{'26-Sep-2016'}	{'BBB'}

transprob treats 'NR' as another rating. The transition matrix below shows the estimated probability of transitioning into and out of 'NR'.

```
StartYearNR = 2010;
EndYearNR = 2018;
startDateNR = datetime(StartYearNR,12,31,'Locale','en_US');
endDateNR = datetime(EndYearNR,12,31,'Locale','en_US');
CompanyID_NR = "DEF";

RatingLabelsNR = ["AAA","AA","A","BBB","BB","B","CCC","D","NR"];

[tmNR,~,itNR] = transprob(dataNR,'startDate',startDateNR,'endDate',endDateNR,'algorithm','cohort');
hDisplayTransitions(tmNR,RatingLabelsNR,"Transition Matrix")
```

Transition Matrix

	AAA	AA	A	BBB	BB	B	CCC	D	NR
AAA	100	0	0	0	0	0	0	0	0
AA	0	100	0	0	0	0	0	0	0
A	0	0	66.667	0	0	0	0	0	33.333
BBB	0	0	0	100	0	0	0	0	0
BB	0	0	0	0	100	0	0	0	0
B	0	0	0	0	0	100	0	0	0
CCC	0	0	0	0	0	0	100	0	0
D	0	0	0	0	0	0	0	100	0
NR	0	0	0	50	0	0	0	0	50

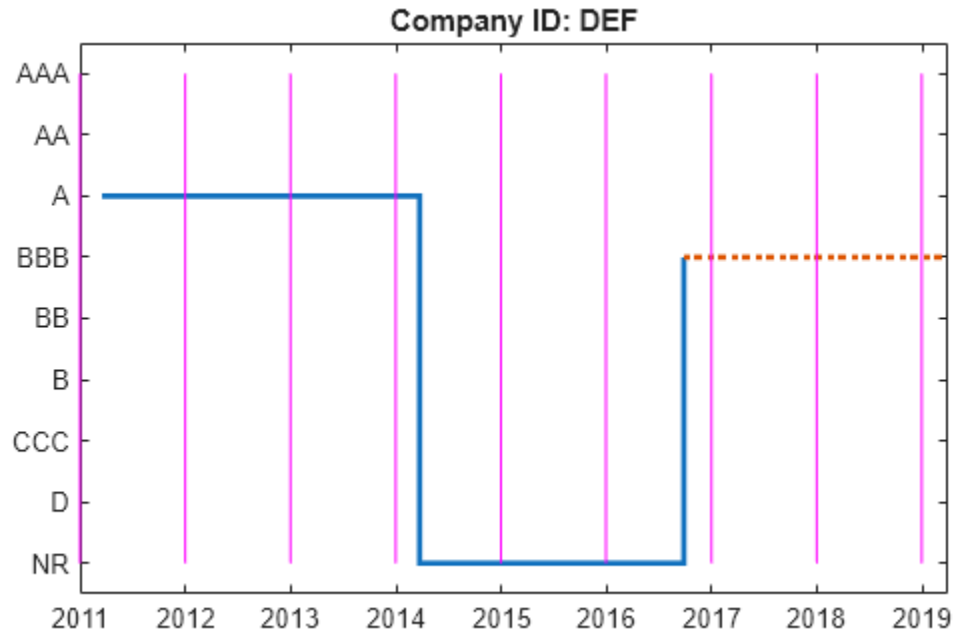
Display the transition counts and corresponding visualization of the transitions.

```
hDisplayTransitions(itNR.totalsMat,RatingLabelsNR, strcat("Transition counts, company ID: ", CompanyID_NR))
```

Transition counts, company ID: DEF

	AAA	AA	A	BBB	BB	B	CCC	D	NR
AAA	0	0	0	0	0	0	0	0	0
AA	0	0	0	0	0	0	0	0	0
A	0	0	2	0	0	0	0	0	1
BBB	0	0	0	2	0	0	0	0	0
BB	0	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0	0
CCC	0	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0	0
NR	0	0	0	1	0	0	0	0	1

```
hPlotTransitions(CompanyID_NR,startDateNR,endDateNR,dataNR,RatingLabelsNR)
```



To remove the 'NR' from the transition matrix, use the 'excludeLabels' name-value input argument in `transprob`. The list of labels to exclude may or may not be specified in the name-value pair argument `labels`. For example, both `RatingLabels` and `RatingLabelsNR` generate the same output from `transprob`.

```
[tmNR,stNR,itNR] = transprob(dataNR,'startDate',startDateNR,'endDate',endDateNR,'algorithm','coh  
hDisplayTransitions(tmNR,RatingLabels,"Transition Matrix")
```

Transition Matrix

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	100	0	0	0	0	0	0	0
AA	0	100	0	0	0	0	0	0
A	0	0	100	0	0	0	0	0
BBB	0	0	0	100	0	0	0	0
BB	0	0	0	0	100	0	0	0
B	0	0	0	0	0	100	0	0
CCC	0	0	0	0	0	0	100	0
D	0	0	0	0	0	0	0	100

Display the transition counts and corresponding visualization of the transitions.

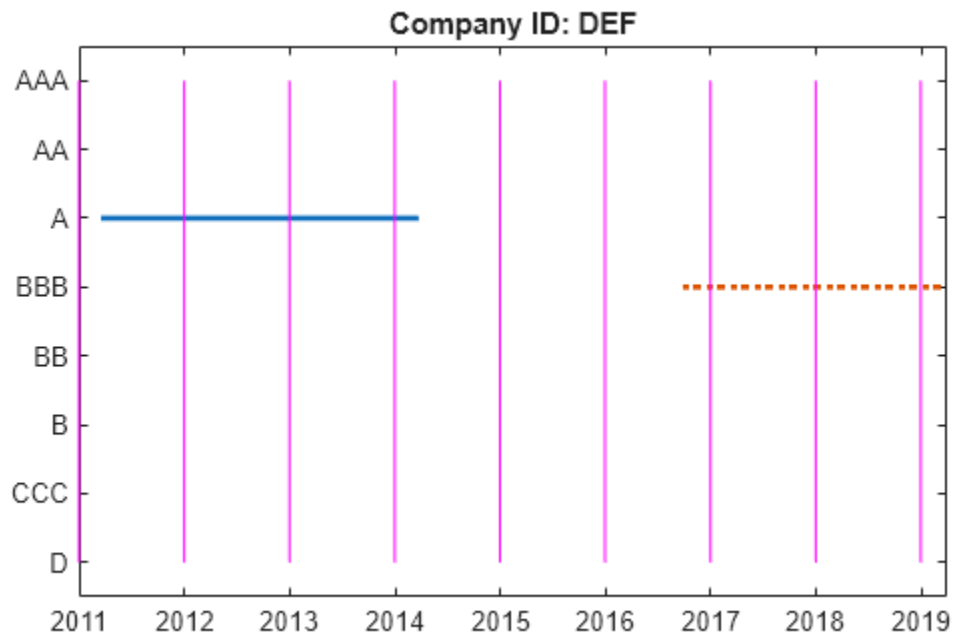
```
hDisplayTransitions(itNR.totalsMat,RatingLabels, strcat("Transition counts, company ID: ",CompanyID))
```

Transition counts, company ID: DEF

	AAA	AA	A	BBB	BB	B	CCC	D
AAA	0	0	0	0	0	0	0	0
AA	0	0	0	0	0	0	0	0

A	0	0	2	0	0	0	0	0
BBB	0	0	0	2	0	0	0	0
BB	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
CCC	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0

```
hPlotTransitions(CompanyID_NR,startDateNR,endDateNR,dataNR,RatingLabels)
```



Consistent with the previous plot, the transition counts still show two transitions from 'A' to 'A' (from the end of 2012 to the end of 2014), and two transitions from 'BBB' to 'BBB' (from the end of 2017 to the end of 2019).

However, different from the previous plot, specifying 'NR' using the 'excludeLabels' name-value input argument of `transprob` removes any transitions into and out of the 'NR' rating.

Local Functions

```
function hDisplayTransitions(TransitionsData,RatingLabels,Title)
% Helper function to format transition information outputs

TransitionsAsTable = array2table(TransitionsData,...
    'VariableNames',RatingLabels,'RowNames',RatingLabels);

fprintf('\n%s\n\n',Title)
disp(TransitionsAsTable)

end

function hPlotTransitions(CompanyID,startDate,endDate,data,RatingLabels)
% Helper function to visualize transitions between ratings

Ind = string(data.ID)==CompanyID;
```

```

DatesOriginal = datetime(data.Date(Ind),'Locale','en_US');
RatingsOriginal = categorical(data.Rating(Ind),flipud(RatingLabels(:)),flipud(RatingLabels(:))

stairs(DatesOriginal,RatingsOriginal,'LineWidth',2)
hold on;

% Indicate rating extrapolated into the future (arbitrarily select 91
% days after endDate as the last date on the plot)
endDateExtrap = endDate+91;
if endDateExtrap>DatesOriginal(end)
    DatesExtrap = [DatesOriginal(end); endDateExtrap];
    RatingsExtrap = [RatingsOriginal(end); RatingsOriginal(end)];
    stairs(DatesExtrap,RatingsExtrap,'LineWidth',2,'LineStyle',':')
end
hold off;

% Add lines to indicate the snapshot dates
% transprob uses 1 as the default for 'snapsPerYear', hardcoded here for simplicity
% The call to cfdates generates the exact same snapshot dates that transprob uses
snapsPerYear = 1;
snapDates = cfdates(startDate-1,endDate,snapsPerYear)';
ylimits = ylim;
for ii=1:length(snapDates)
    line([snapDates(ii) snapDates(ii)],ylimits,'Color','m')
end
title(strcat("Company ID: ",CompanyID))
end

```

Create Exposure-Based Transition Matrix From Historical Data of Credit Ratings with Exposures

This example shows how to load a historical credit rating table and then use `transprob` to compute the exposure-based transition matrix. The sample totals and ID totals are weighted by the exposure. For more information on the computation of transition probabilities with general weights, which specializes to exposure-based probabilities, see “Algorithms” on page 15-1688.

```

load Data_TransProb
dataExposures(1:10,:)

```

```

ans=10x4 table
      ID      Date      Rating      Exposure
      ---      -
    10283    10-Nov-1984    {'CCC'}      8500
    10283    12-May-1986    {'B'  }      8500
    10283    29-Jun-1988    {'CCC'}      8500
    10283    12-Dec-1991    {'D'  }      8500
    13326     09-Feb-1985    {'A'  }      7500
    13326    24-Feb-1994    {'AA' }      7500
    13326    10-Nov-2000    {'BBB'}      8500
    14413    23-Dec-1982    {'B'  }      8500
    14413    20-Apr-1988    {'BB' }      8500
    14413    16-Jan-1998    {'B'  }      8500

```

The "Weight" column is the fourth column, and in this example, it is the loan's exposure on an observation date. Note that the `transprob` function also supports more general weights that are only required to be nonnegative and real.

Use `transprob` With duration Algorithm

Use `transprob` to estimate the exposure based transition probabilities with default settings.

```
[transMatExposures, sampleTotalsExposures, idTotalsExposures] = transprob(dataExposures);
```

Display the exposure-based transition matrix using the default settings.

```
transMatExposures
```

```
transMatExposures = 8×8
```

92.9124	6.1143	0.7937	0.1300	0.0470	0.0013	0.0001	0.0011
1.6083	93.2741	4.2951	0.6416	0.1552	0.0056	0.0005	0.0197
0.1205	3.1292	92.0483	4.0680	0.4639	0.0845	0.0034	0.0822
0.0190	0.2259	5.0466	90.1037	3.8386	0.4605	0.0819	0.2239
0.0219	0.1085	0.5943	5.9012	89.2276	3.2159	0.2776	0.6530
0.0010	0.0057	0.0654	1.0355	7.7249	85.9825	2.6259	2.5591
0.0002	0.0012	0.0131	0.3450	1.4889	4.1707	81.6593	12.3218
0	0	0	0	0	0	0	100.0000

Display the exposure-based sample totals with default settings that use the duration algorithm.

```
sampleTotalsExposures.totalsVec
```

```
ans = 1×8
```

```
107 ×
```

1.6488	2.8735	2.8788	2.5613	2.4690	1.3158	0.6614	2.1678
--------	--------	--------	--------	--------	--------	--------	--------

```
sampleTotalsExposures.totalsMat
```

```
ans = 8×8
```

0	1081500	116000	17000	7000	0	0	0
496000	0	1326500	170000	42000	0	0	5000
29000	971000	0	1280500	118500	22000	0	22500
4000	38500	1417500	0	1090000	113000	21000	53500
5500	25500	116500	1620500	0	902500	65500	153000
0	0	3000	116000	1156500	0	411000	332500
0	0	0	21500	100500	327500	0	896000
0	0	0	0	0	0	0	0

```
sampleTotalsExposures.algorithm
```

```
ans =
```

```
'duration'
```

Display the exposure-based ID totals for the second obligor (ID 13326) with default settings that use the duration algorithm.

```
idTotalsExposures(2).totalsVec
```

```
ans = 1×8 sparse double row vector (3 nonzeros)
1.0e+04 *
```

```
(1,2)      5.0328
(1,3)      6.7808
(1,4)      3.6445
```

```
idTotalsExposures(2).totalsMat
```

```
ans = 8×8 sparse double matrix (2 nonzeros)
(3,2)      7500
(2,4)      7500
```

```
idTotalsExposures(2).algorithm
```

```
ans =
'duration'
```

Use transprob With Cohort Algorithm

Use `transprob` to estimate the exposure-based transition probabilities with the cohort algorithm.

```
[transMatCohExposures,sampleTotalsCohExposures,idTotalsCohExposures] = transprob(dataExposures,a
```

Display the exposure-based transition matrix when using the cohort algorithm.

```
transMatCohExposures
```

```
transMatCohExposures = 8×8
```

```
92.9468    6.1934    0.7124    0.1044    0.0430         0         0         0
 1.7148   93.0587    4.4778    0.5811    0.1497         0         0    0.0178
 0.1393    3.1653   91.8358    4.2990    0.4017    0.0786         0    0.0803
 0.0160    0.4148    5.1063   89.7052    4.0382    0.4529    0.0521    0.2144
 0.0227    0.1054    0.5992    6.3851   88.6102    3.4198    0.2707    0.5868
         0         0    0.0231    0.8706    8.3320   85.5894    2.7311    2.4538
         0         0         0    0.4250    1.9731    4.3181   81.1793   12.1044
         0         0         0         0         0         0         0  100.0000
```

Display the exposure-based sample totals when using the cohort algorithm.

```
sampleTotalsCohExposures.totalsVec
```

```
ans = 1×8
```

```
16283500    28049500    28006500    24949500    24197000    12980000    6588500    20952000
```

```
sampleTotalsCohExposures.totalsMat
```

```
ans = 8×8
```

```
15135000    1008500    116000    17000    7000         0         0         0
 481000    26102500    1256000    163000    42000         0         0    5000
 39000     886500    25720000    1204000    112500    22000         0    22500
 4000     103500    1274000    22381000    1007500    113000    13000    53500
 5500     25500    145000    1545000    21441000    827500    65500    142000
```

0	0	3000	113000	1081500	11109500	354500	318500
0	0	0	28000	130000	284500	5348500	797500
0	0	0	0	0	0	0	20952000

```
sampleTotalsCohExposures.algorithm
```

```
ans =  
'cohort'
```

Display the exposure-based ID totals for the second obligor (ID 13326) when using the cohort algorithm

```
idTotalsCohExposures(2).totalsVec
```

```
ans = 1x8 sparse double row vector (3 nonzeros)  
(1,2)      45000  
(1,3)      75000  
(1,4)      34000
```

```
idTotalsCohExposures(2).totalsMat
```

```
ans = 8x8 sparse double matrix (5 nonzeros)  
(2,2)      37500  
(3,2)      7500  
(3,3)      67500  
(2,4)      7500  
(4,4)      34000
```

```
idTotalsCohExposures(2).algorithm
```

```
ans =  
'cohort'
```

The `duration` algorithm and the `cohort` algorithm produce similar transition matrices. In each case, the totals are weighted by the exposures. For additional details, see “Algorithms” on page 15-1688.

Input Arguments

data — Credit migration data

table | cell array of character vectors | preprocessed data structure

Using `transprob` to estimate transition probabilities given credit ratings historical data (that is, credit migration data), the `data` input can be one of the following:

- An `nRecords-by-3` MATLAB table containing the historical credit ratings data of the form:

ID	Date	Rating
'00010283'	'10-Nov-1984'	'CCC'
'00010283'	'12-May-1986'	'B'
'00010283'	'29-Jun-1988'	'CCC'
'00010283'	'12-Dec-1991'	'D'
'00013326'	'09-Feb-1985'	'A'
'00013326'	'24-Feb-1994'	'AA'


```
'00013326' '10-Nov-2000' 'BBB'
'00014413' '23-Dec-1982' 'B'
```

Or an `nRecords`-by-4 MATLAB table containing weights and the historical credit ratings data of the form:

ID	Date	Rating	Weight
'00010283'	'10-Nov-1984'	'CCC'	1
'00010283'	'12-May-1986'	'B'	1.4
'00010283'	'29-Jun-1988'	'CCC'	1.8
'00010283'	'12-Dec-1991'	'D'	0.2
'00013326'	'09-Feb-1985'	'A'	0
'00013326'	'24-Feb-1994'	'AA'	2
'00013326'	'10-Nov-2000'	'BBB'	1.7
'00014413'	'23-Dec-1982'	'B'	1.1

where each row contains an ID (column 1), a date (column 2), a credit rating (column 3), and an optional weight (column 4). Column 3 is the rating assigned to the corresponding ID on the corresponding date. All information corresponding to the same ID must be stored in contiguous rows. Sorting this information by date is not required, but recommended for efficiency. When using a MATLAB table input, the names of the columns are irrelevant, but the ID, date, rating information, and weights are assumed to be in the first, second, third, and fourth columns, respectively. Also, when using a table input, the first and third columns can be categorical arrays, and the second can be a datetime array. The following summarizes the supported data types for table input:

Data Input Type	ID (1st Column)	Date (2nd Column)	Rating (3rd Column)	Weight (Optional 4th Column)
Table	- Numeric array - Cell array of character vectors - String array - Categorical array	- Numeric array - Cell array of character vectors - String array - Datetime array	- Numeric array - Cell array of character vectors - String array - Categorical array	Numeric array with nonnegative values

For an example of using the `data` input argument with an optional fourth column for `Weight`, see “Create Exposure-Based Transition Matrix From Historical Data of Credit Ratings with Exposures” on page 15-1679.

Note If no weights are provided in a fourth column of the `data`, the default is to set all weights equal to 1. In this case, the weighted transition matrix output agrees with the ordinary, count-based transition matrix.

- An `nRecords`-by-3 cell array of character vectors with the historical credit ratings data of the form:

```
'00010283' '10-Nov-1984' 'CCC'
'00010283' '12-May-1986' 'B'
'00010283' '29-Jun-1988' 'CCC'
'00010283' '12-Dec-1991' 'D'
'00013326' '09-Feb-1985' 'A'
```

```
'00013326' '24-Feb-1994' 'AA'
'00013326' '10-Nov-2000' 'BBB'
'00014413' '23-Dec-1982' 'B'
```

Or an `nRecords-by-4` cell array of character vectors if weights are included with the historical credit ratings data of the form:

```
'00010283' '10-Nov-1984' 'CCC' '1.2'
'00010283' '12-May-1986' 'B' '1'
'00010283' '29-Jun-1988' 'CCC' '1.2'
'00010283' '12-Dec-1991' 'D' '0.2'
'00013326' '09-Feb-1985' 'A' '1.7'
'00013326' '24-Feb-1994' 'AA' '1.3'
'00013326' '10-Nov-2000' 'BBB' '1'
'00014413' '23-Dec-1982' 'B' '1.8'
```

where each row contains an ID (column 1), a date (column 2), a credit rating (column 3), and an optional weight (Column 4). Column 3 is the rating assigned to the corresponding ID on the corresponding date. All information corresponding to the same ID must be stored in contiguous rows. Sorting this information by date is not required, but recommended for efficiency. IDs, dates, and ratings are stored in character vector format, but they can also be entered in numeric format. The following summarizes the supported data types for cell array input:

Data Input Type	ID (1st Column)	Date (2nd Column)	Rating (3rd Column)	Weight (Optional 4th Column)
Cell	- Numeric elements - Character vector elements	- Numeric elements - Character vector elements	- Numeric elements - Character vector elements	Numeric elements with nonnegative values

Note If no weights are provided in a fourth column of the data, the default is to set all weights equal to 1. In this case, the weighted transition matrix output agrees with the ordinary, count-based transition matrix.

- A preprocessed data structure obtained using `transprobprep`. This data structure contains the fields `'idStart'`, `'numericDates'`, `'numericRatings'`, `'Weights'` (optional), and `'ratingsLabels'`.

Data Types: `table` | `cell` | `struct`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `transMat = transprob(data,'algorithm','cohort')`

algorithm — Estimation algorithm

`'duration'` (default) | character vector with values are `'duration'` or `'cohort'`

Estimation algorithm, specified as the comma-separated pair consisting of 'algorithm' and a character vector with a value of 'duration' or 'cohort'.

Data Types: char

endDate — End date of the estimation time window

latest date in data (default) | datetime array | string array | date character vector | serial date number

End date of the estimation time window, specified as the comma-separated pair consisting of 'endDate' and a scalar datetime, string, date character vector, or serial date number. The endDate cannot be a date before the startDate.

Data Types: char | double | string | datetime

labels — Credit-rating scale

{'AAA', 'AA', 'A', 'BBB', 'BB', 'B', 'CCC', 'D'} (default) | cell array of character vectors

Credit-rating scale, specified as the comma-separated pair consisting of 'labels' and a nRatings-by-1, or 1-by-nRatings cell array of character vectors.

labels must be consistent with the ratings labels used in the third column of data. Use a cell array of numbers for numeric ratings, and a cell array for character vectors for categorical ratings.

Note When the input argument data is a preprocessed data structure obtained from a previous call to transproprep, this optional input for 'labels' is unused because the labels in the 'ratingsLabels' field of transproprep take priority.

Data Types: cell

snapsPerYear — Number of credit-rating snapshots per year

1 (default) | numeric values are 1, 2, 3, 4, 6, or 12

Number of credit-rating snapshots per year to be considered for the estimation, specified as the comma-separated pair consisting of 'snapsPerYear' and a numeric value of 1, 2, 3, 4, 6, or 12.

Note This parameter is only used with the 'cohort' algorithm.

Data Types: double

startDate — Start date of the estimation time window

earliest date in data (default) | datetime array | string array | date character vector | serial date number

Start date of the estimation time window, specified as the comma-separated pair consisting of 'startDate' and a scalar datetime, string, date character vector, or serial date number.

Data Types: char | double | string | datetime

transInterval — Length of the transition interval in years

1 (one year transition probability) (default) | numeric

Length of the transition interval, in years, specified as the comma-separated pair consisting of 'transInterval' and a numeric value.

Data Types: double

excludeLabels — Label that is excluded from the transition probability computation

' ' (do not exclude any label) (default) | numeric | character vector | string

Label that is excluded from the transition probability computation, specified as the comma-separated pair consisting of 'excludeLabels' and a character vector, string, or numerical rating.

If multiple labels are to be excluded, 'excludeLabels' must be a cell array containing all of the labels for exclusion. The type of the labels given in 'excludeLabels' must be consistent with the data type specified in the labels input.

The list of labels to exclude may or may not be specified in labels.

Data Types: double | char | string

Output Arguments

transMat — Matrix of transition probabilities in percent

matrix

Matrix of transition probabilities in percent, returned as a nRatings-by-nRatings transition matrix.

sampleTotals — Structure with sample totals

structure

Structure with sample totals, returned with fields:

- totalsVec — A vector of size 1-by-nRatings.
- totalsMat — A matrix of size nRatings-by-nRatings.
- algorithm — A character vector with values 'duration' or 'cohort'.

For the 'duration' algorithm, totalsMat(*i,j*) contains the total weight which transitioned out of rating *i* into rating *j* (all the diagonal elements are zero). The total weighted time spent on rating *i* is stored in totalsVec(*i*). If the default weights are used, totalsMat(*i,j*) contains the total transitions out of rating *i* into rating *j* and totalsVec(*i*) stores the total time spent on rating *i*.

For example, if there are three rating categories, Investment Grade (IG), Speculative Grade (SG), and Default (D), and the following information:

Total time spent	IG	SG	D
in rating:	4859.09	1503.36	1162.05

Transitions		IG	SG	D
out of (row)	IG	0	89	7
into (column):	SG	202	0	32
	D	0	0	0

Then

```
totals.totalsVec = [4859.09 1503.36 1162.05]
totals.totalsMat = [ 0 89 7
```

```

      202    0    32
      0     0     0]
totals.algorithm = 'duration'

```

For the 'cohort' algorithm, `totalsMat(i,j)` contains the total weight which is transitioned out of rating i to rating j , and `totalsVec(i)` is the initial weight in rating i . If the default weights are used, then `totalsMat(i,j)` contains the total transitions out of rating i into rating j and `totalsVec(i)` is the initial count in rating i .

For example, given the following information:

```

Initial count      IG      SG      D
in rating:        4808    1572    1145

Transitions
from (row)      IG      SG      D
to (column):   IG  4721     80     7
               SG   193    1347    32
               D     0       0    1145

```

Then

```

totals.totalsVec = [4808    1572    1145]
totals.totalsMat = [4721     80     7
                   193    1347    32
                   0       0    1145]
totals.algorithm = 'cohort'

```

idTotals — IDs totals

struct array

IDs totals, returned as a struct array of size `nIDs-by-1`, where `nIDs` is the number of distinct IDs in column 1 of `data` when this is a table or cell array or, equivalently, equal to the length of the `idStart` field minus 1 when `data` is a preprocessed data structure from `transprobbprep`. For each ID in the sample, `idTotals` contains one structure with the following fields:

- `totalsVec` — A sparse vector of size `1-by-nRatings`.
- `totalsMat` — A sparse matrix of size `nRatings-by-nRatings`.
- `algorithm` — A character vector with values 'duration' or 'cohort'.

These fields contain the same information described for the output `sampleTotals`, but at an ID level. For example, for 'duration', `idTotals(k).totalsVec` contains the total time that the k -th company spent on each rating.

More About

Transition Probabilities

Transition probabilities quantify the probability of transitioning from one state (or condition) to another within a defined time frame.

Cohort Estimation

The cohort algorithm estimates the transition probabilities based on a sequence of snapshots of credit ratings at regularly spaced points in time.

If the credit rating of a company changes twice between two snapshot dates, the intermediate rating is overlooked and only the initial and final ratings influence the estimates.

Duration Estimation

Unlike the cohort method, the duration algorithm estimates the transition probabilities based on the full credit ratings history, looking at the exact dates on which the credit rating migrations occur.

There is no concept of snapshots in this method, and all credit rating migrations influence the estimates, even when a company's rating changes twice within a short time.

Algorithms

Cohort Estimation

The algorithm first determines a sequence $t_0, \dots, t_K$ of snapshot dates. The elapsed time, in years, between two consecutive snapshot dates t_{k-1} and t_k is equal to $1 / ns$, where ns is the number of snapshots per year. These $K + 1$ dates determine K transition periods.

The algorithm computes N_i^n , the number of transition periods in which obligor n starts at rating i . These are added up over all obligors to get N_i , the number of obligors in the sample that start a period at rating i . The number periods in which obligor n starts at rating i and ends at rating j , or migrates from i to j , denoted by N_{ij}^n , is also computed. These are also added up to get N_{ij} , the total number of migrations from i to j in the sample.

The estimate of the transition probability from i to j in one period, denoted by P_{ij} , is

$$P_{ij} = \frac{N_{ij}}{N_i}$$

These probabilities are arranged in a one-period transition matrix P_0 , where the i,j entry in P_0 is P_{ij} .

If the number of snapshots per year ns is 4 (quarterly snapshots), the probabilities in P_0 are 3-month (or 0.25-year) transition probabilities. You may, however, be interested in 1-year or 2-year transition probabilities. The latter time interval is called the transition interval, Δt , and it is used to convert P_0 into the final transition matrix, P , according to the formula:

$$P = P_0^{ns * \Delta t}$$

For example, if $ns = 4$ and $\Delta t = 2$, P contains the two-year transition probabilities estimated from quarterly snapshots.

When weights are provided, the calculation is similar. In this case, the number N_i^n is equal to the sum of the starting weights of transition periods in which obligor n starts at rating i . These are added up over all obligors to get N_i . The quantity N_{ij}^n is computed as the sum of the starting weights of transition periods in which obligor n starts at rating i and ends at rating j . These are added up to get N_{ij} . The remainder of the computation proceeds as above.

Note For the cohort algorithm, optional output arguments `idTotals` and `sampleTotals` from `transprob` contain the following information:

- $\text{idTotals}(n).\text{totalsVec} = (N_i^n) \forall i$
- $\text{idTotals}(n).\text{totalsMat} = (N_{i,j}^n) \forall i,j$
- $\text{idTotals}(n).\text{algorithm} = \text{'cohort'}$
- $\text{sampleTotals}.\text{totalsVec} = (N_i) \forall i$
- $\text{sampleTotals}.\text{totalsMat} = (N_{i,j}) \forall i,j$
- $\text{sampleTotals}.\text{algorithm} = \text{'cohort'}$

For efficiency, the vectors and matrices in `idTotals` are stored as sparse arrays.

When ratings must be excluded (see the `excludeLabels` name-value input argument), all transitions involving the excluded ratings are removed from the sample. For example, if the 'NR' rating must be excluded, any transitions into 'NR' and out of 'NR' are excluded from the sample. The total counts for all other ratings are adjusted accordingly. For more information, see “Visualize Transitions Data for transprob” on page 8-114.

Duration Estimation

The algorithm computes T_i^n , the total time that obligor n spends in rating i within the estimation time window. These quantities are added up over all obligors to get T_i , the total time spent in rating i , collectively, by all obligors in the sample. The algorithm also computes T_{ij}^n , the number times that obligor n migrates from rating i to rating j , with i not equal to j , within the estimation time window. And it also adds them up to get T_{ij} , the total number of migrations, by all obligors in the sample, from the rating i to j , with i not equal to j .

To estimate the transition probabilities, the duration algorithm first computes a generator matrix Λ . Each off-diagonal entry of this matrix is an estimate of the transition rate out of rating i into rating j , and is

$$\lambda_{ij} = \frac{T_{ij}}{T_i}, i \neq j$$

The diagonal entries are computed as:

$$\lambda_{ii} = - \sum_{j \neq i} \lambda_{ij}$$

With the generator matrix and the transition interval Δt (e.g., $\Delta t = 2$ corresponds to two-year transition probabilities), the transition matrix is obtained as $P = \exp(\Delta t \Lambda)$, where *exp* denotes matrix exponentiation (`expm` in MATLAB).

When weights are provided, the calculation is similar. In this case, the number T_i^n is the total weighted time that obligor n spends in rating i within the estimation window. In general, T_i^n will be a sum of terms, each of which is the length of a period the obligor spent in rating i times the obligor's starting weight during that period. The quantities T_i^n are added up over all obligors to get T_i , the total weighted time spent in rating i , collectively, by all obligors in the sample. The quantity T_{ij}^n is computed as the sum of the weights of periods in which obligor n starts at rating i and ends at rating j , with i not equal to j . The remainder of the computation proceeds as above.

Note For the duration algorithm, optional output arguments `idTotals` and `sampleTotals` from `transprob` contain the following information:

- `idTotals(n).totalsVec = (T_i^n) \forall i`
- `idTotals(n).totalsMat = (T_{i,j}^n) \forall ij`
- `idTotals(n).algorithm = 'duration'`
- `sampleTotals.totalsVec = (T_i) \forall i`
- `sampleTotals.totalsMat = (T_{i,j}) \forall ij`
- `sampleTotals.algorithm = 'duration'`

For efficiency, the vectors and matrices in `idTotals` are stored as sparse arrays.

When ratings must be excluded (see the `excludeLabels` name-value input argument), all transitions involving the exclude ratings are removed from the sample. For example, if the 'NR' rating must be excluded, any transitions into 'NR' and out of 'NR' are excluded from the sample. The total time spent in 'NR' (or any other excluded rating) is also removed.

Version History

Introduced in R2010b

R2024a: Support for weights as optional fourth column in data

`transprob` supports an optional fourth column for weights in the data input.

References

- [1] Hanson, S., T. Schuermann. "Confidence Intervals for Probabilities of Default." *Journal of Banking & Finance*. Vol. 30(8), Elsevier, August 2006, pp. 2281-2301.
- [2] Löffler, G., P. N. Posch. *Credit Risk Modeling Using Excel and VBA*. West Sussex, England: Wiley Finance, 2007.
- [3] Schuermann, T. "Credit Migration Matrices." in E. Melnick, B. Everitt (eds.), *Encyclopedia of Quantitative Risk Analysis and Assessment*. Wiley, 2008.

See Also

`transprobytotals` | `transprobbprep` | `table`

Topics

"Estimation of Transition Probabilities" on page 8-2

"Estimate Transition Probabilities for Different Rating Scales" on page 8-4

External Websites

Credit Risk Modeling with MATLAB (53 min 09 sec)

Forecasting Corporate Default Rates with MATLAB (54 min 36 sec)

transprobytotals

Estimate transition probabilities using `totals` structure input

Syntax

```
[transMat,sampleTotals] = transprobytotals(totals)
[transMat,sampleTotals] = transprobytotals( ___,Name,Value)
```

Description

`[transMat,sampleTotals] = transprobytotals(totals)` estimates transition probabilities using a `totals` structure input. `transprobytotals` is useful for removing outlier information, obtaining bootstrapped confidence intervals, or computing transition probability estimates for different periodicity parameters (1-year transitions, 2-year transitions, and so on) efficiently.

`[transMat,sampleTotals] = transprobytotals( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Estimate Transition Probabilities Using a totals Structure Input

Use historical credit rating input data from `Data_TransProb.mat` and `transprob` to generate input for `transprobytotals`:

```
load Data_TransProb
```

```
% Call TRANSPROB with three output arguments
```

```
[transMat, sampleTotals, idTotals] = transprob(data);
transMat
```

```
transMat = 8×8
```

93.1170	5.8428	0.8232	0.1763	0.0376	0.0012	0.0001	0.0017
1.6166	93.1518	4.3632	0.6602	0.1626	0.0055	0.0004	0.0396
0.1237	2.9003	92.2197	4.0756	0.5365	0.0661	0.0028	0.0753
0.0236	0.2312	5.0059	90.1846	3.7979	0.4733	0.0642	0.2193
0.0216	0.1134	0.6357	5.7960	88.9866	3.4497	0.2919	0.7050
0.0010	0.0062	0.1081	0.8697	7.3366	86.7215	2.5169	2.4399
0.0002	0.0011	0.0120	0.2582	1.4294	4.2898	81.2927	12.7167
0	0	0	0	0	0	0	100.0000

Suppose companies 4 and 27 are outliers and you want to remove them from the pre-processed `idTotals` struct array and estimate the new transition probabilities.

```
idTotals([4 27]) = [];
[transMat1, sampleTotals1] = transprobytotals(idTotals);
transMat1
```

```
transMat1 = 8×8
```

```

93.1172    5.8427    0.8231    0.1763    0.0377    0.0012    0.0001    0.0017
 1.6213   93.1501    4.3584    0.6614    0.1631    0.0055    0.0004    0.0397
 0.1239    2.9027   92.2297    4.0628    0.5367    0.0661    0.0028    0.0753
 0.0236    0.2313    5.0070   90.1825    3.7986    0.4734    0.0642    0.2193
 0.0216    0.1134    0.6357    5.7959   88.9866    3.4497    0.2920    0.7050
 0.0010    0.0062    0.1081    0.8697    7.3367   86.7217    2.5171    2.4395
 0.0002    0.0011    0.0120    0.2591    1.4340    4.3034   81.3027   12.6875
         0         0         0         0         0         0         0   100.0000

```

Obtain the 1-year, 2-year, 3-year, 4-year, and 5-year default probabilities, without the outlier information (i.e., using `sampleTotals1`).

```

DefProb = zeros(7,5);
for t = 1:5
    transMatTemp = transprobytotals(sampleTotals1,'transInterval',t);
    DefProb(:,t) = transMatTemp(1:7,8);
end
DefProb

```

DefProb = 7×5

```

0.0017    0.0070    0.0159    0.0285    0.0450
0.0397    0.0828    0.1299    0.1813    0.2377
0.0753    0.1606    0.2567    0.3640    0.4831
0.2193    0.4675    0.7430    1.0445    1.3700
0.7050    1.4668    2.2759    3.1232    4.0000
2.4395    4.9282    7.4071    9.8351   12.1847
12.6875   23.1184   31.7177   38.8282   44.7266

```

Input Arguments

totals — Total transitions observed

structure | struct array

Total transitions observed, specified as a structure, or a struct array of length `nTotals`, with fields:

- **totalsVec** — A sparse vector of size 1-by-`nRatings1`.
- **totalsMat** — A sparse matrix of size `nRatings1`-by-`nRatings2` with `nRatings1` ≤ `nRatings2`.
- **algorithm** — A character vector with values 'duration' or 'cohort'.

For the 'duration' algorithm, `totalsMat(i,j)` contains the total transitions observed out of rating *i* into rating *j* (all the diagonal elements are 0). The total time spent on rating *i* is stored in `totalsVec(i)`. For example, you have three rating categories, Investment Grade (IG), Speculative Grade (SG), and Default (D), and the following information:

```

Total time spent    IG      SG      D
in rating:         4859.09  1503.36  1162.05

```

```

Transitions
out of (row)      IG      SG      D
into (column):   SG  202      0     32
                  D    0      0      0

```

Then:

```
totals.totalsVec = [4859.09 1503.36 1162.05]
totals.totalsMat = [ 0 89 7
                   202 0 32
                   0 0 0]
totals.algorithm = 'duration'
```

For the 'cohort' algorithm, `totalsMat(i,j)` contains the total transitions observed from rating i to rating j , and `totalsVec(i)` is the initial count in rating i . For example, given the following information:

Initial count		IG	SG	D
in rating:		4808	1572	1145

Transitions		IG	SG	D
from (row)	IG	4721	80	7
to (column):	SG	193	1347	32
	D	0	0	1145

Then:

```
totals.totalsVec = [4808 1572 1145]
totals.totalsMat = [4721 80 7
                   193 1347 32
                   0 0 1145]
totals.algorithm = 'cohort'
```

Common totals structures are the optional output arguments from `transprob`:

- `sampleTotals` — A single structure summarizing the totals information for the whole dataset.
- `idTotals` — A struct array with the totals information at the ID level.

Data Types: `struct` | `structure`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `transMat = transprobytotals(Totals1,'transInterval',5)`

snapsPerYear — Number of credit-rating snapshots per year

1 (default) | numeric values are 1, 2, 3, 4, 6, or 12

Number of credit-rating snapshots per year to be considered for the estimation, specified as the comma-separated pair consisting of 'snapsPerYear' and a numeric value of 1, 2, 3, 4, 6, or 12.

Note This parameter is only used with the 'cohort' algorithm.

Data Types: `double`

transInterval — Length of the transition interval in years

1 (one year transition probability) (default) | numeric

Length of the transition interval, in years, specified as the comma-separated pair consisting of 'transInterval' and a numeric value.

Data Types: `double`

Output Arguments

transMat — Matrix of transition probabilities in percent

`matrix`

Matrix of transition probabilities in percent, returned as a `nRatings1`-by-`nRatings2` transition matrix.

sampleTotals — Structure with sample totals

`structure`

Structure with sample totals, returned with fields:

- `totalsVec` — A vector of size 1-by-`nRatings1`.
- `totalsMat` — A matrix of size `nRatings1`-by-`nRatings2` with $nRatings1 \leq nRatings2$.
- `algorithm` — A character vector with values 'duration' or 'cohort'.

If `totals` is a struct array, `sampleTotals` contains the aggregated information. That is, `sampleTotals.totalsVec` is the sum of `totals(k).totalsVec` over all k , and similarly for `totalsMat`. When `totals` is itself a single structure, `sampleTotals` and `totals` are the same.

More About

Transition Probabilities

Transition probabilities quantify the probability of transitioning from one state (or condition) to another within a defined time frame.

Cohort Estimation

The cohort algorithm estimates the transition probabilities based on a sequence of snapshots of credit ratings at regularly spaced points in time.

If the credit rating of a company changes twice between two snapshot dates, the intermediate rating is overlooked and only the initial and final ratings influence the estimates.

Duration Estimation

Unlike the cohort method, the duration algorithm estimates the transition probabilities based on the full credit ratings history, looking at the exact dates on which the credit rating migrations occur.

There is no concept of snapshots in this method, and all credit rating migrations influence the estimates, even when a company's rating changes twice within a short time.

Version History

Introduced in R2010b

References

- [1] Hanson, S., T. Schuermann. "Confidence Intervals for Probabilities of Default." *Journal of Banking & Finance*. Vol. 30(8), Elsevier, August 2006, pp. 2281-2301.
- [2] Löffler, G., P. N. Posch. *Credit Risk Modeling Using Excel and VBA*. West Sussex, England: Wiley Finance, 2007.
- [3] Schuermann, T. "Credit Migration Matrices." in E. Melnick, B. Everitt (eds.), *Encyclopedia of Quantitative Risk Analysis and Assessment*. Wiley, 2008.

See Also

transprobgrouptotals | transprob

Topics

"Estimation of Transition Probabilities" on page 8-2

"Estimate Transition Probabilities for Different Rating Scales" on page 8-4

External Websites

Credit Risk Modeling with MATLAB (53 min 09 sec)

Forecasting Corporate Default Rates with MATLAB (54 min 36 sec)

transprobfromthresholds

Convert from credit quality thresholds to transition probabilities

Syntax

```
trans = transprobfromthresholds(thresh)
```

Description

`trans = transprobfromthresholds(thresh)` transforms credit quality thresholds into transition probabilities

Examples

Transform Credit Quality Thresholds Into Transition Probabilities

Use historical credit rating input data from `Data_TransProb.mat`, estimate transition probabilities with default settings.

```
load Data_TransProb
```

```
% Estimate transition probabilities with default settings
```

```
transMat = transprob(data)
```

```
transMat = 8×8
```

93.1170	5.8428	0.8232	0.1763	0.0376	0.0012	0.0001	0.0017
1.6166	93.1518	4.3632	0.6602	0.1626	0.0055	0.0004	0.0396
0.1237	2.9003	92.2197	4.0756	0.5365	0.0661	0.0028	0.0753
0.0236	0.2312	5.0059	90.1846	3.7979	0.4733	0.0642	0.2193
0.0216	0.1134	0.6357	5.7960	88.9866	3.4497	0.2919	0.7050
0.0010	0.0062	0.1081	0.8697	7.3366	86.7215	2.5169	2.4399
0.0002	0.0011	0.0120	0.2582	1.4294	4.2898	81.2927	12.7167
0	0	0	0	0	0	0	100.0000

Obtain the credit quality thresholds.

```
thresh = transprobt thresholds(transMat)
```

```
thresh = 8×8
```

Inf	-1.4846	-2.3115	-2.8523	-3.3480	-4.0083	-4.1276	-4.1413
Inf	2.1403	-1.6228	-2.3788	-2.8655	-3.3166	-3.3523	-3.3554
Inf	3.0264	1.8773	-1.6690	-2.4673	-2.9800	-3.1631	-3.1736
Inf	3.4963	2.8009	1.6201	-1.6897	-2.4291	-2.7663	-2.8490
Inf	3.5195	2.9999	2.4225	1.5089	-1.7010	-2.3275	-2.4547
Inf	4.2696	3.8015	3.0477	2.3320	1.3838	-1.6491	-1.9703
Inf	4.6241	4.2097	3.6472	2.7803	2.1199	1.5556	-1.1399
Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf

Recover the transition probabilities.

```
trans = transprobfromthresholds(thresh)
```

```
trans = 8×8
```

93.1170	5.8428	0.8232	0.1763	0.0376	0.0012	0.0001	0.0017
1.6166	93.1518	4.3632	0.6602	0.1626	0.0055	0.0004	0.0396
0.1237	2.9003	92.2197	4.0756	0.5365	0.0661	0.0028	0.0753
0.0236	0.2312	5.0059	90.1846	3.7979	0.4733	0.0642	0.2193
0.0216	0.1134	0.6357	5.7960	88.9866	3.4497	0.2919	0.7050
0.0010	0.0062	0.1081	0.8697	7.3366	86.7215	2.5169	2.4399
0.0002	0.0011	0.0120	0.2582	1.4294	4.2898	81.2927	12.7167
0	0	0	0	0	0	0	100.0000

Input Arguments

thresh — Credit quality thresholds

matrix

Credit quality thresholds, specified as a M-by-N matrix of credit quality thresholds.

In each row, the first element must be `Inf` and the entries must satisfy the following monotonicity condition:

$$\text{thresh}(i,j) \geq \text{thresh}(i,j+1), \text{ for } 1 \leq j < N$$

The M-by-N input `thresh` and the M-by-N output `trans` are related as follows. The thresholds `thresh(i,j)` are critical values of a standard normal distribution z , such that:

$$\text{trans}(i,N) = P[z < \text{thresh}(i,N)],$$

$$\text{trans}(i,j) = P[z < \text{thresh}(i,j)] - P[z < \text{thresh}(i,j+1)], \text{ for } 1 \leq j < N$$

Any given row in the output matrix `trans` determines a probability distribution over a discrete set of N ratings 'R1', ..., 'RN', so that for any row i `trans(i,j)` is the probability of migrating into 'Rj'. `trans` can be a standard transition matrix, with $M \leq N$, in which case row i contains the transition probabilities for issuers with rating 'Ri'. But `trans` does not have to be a standard transition matrix. `trans` can contain individual transition probabilities for a set of M -specific issuers, with $M > N$.

For example, suppose that there are only $N=3$ ratings, 'High', 'Low', and 'Default', with these credit quality thresholds:

	High	Low	Default
High	Inf	-2.0814	-3.1214
Low	Inf	2.4044	-1.7530

The matrix of transition probabilities is then:

	High	Low	Default
High	98.13	1.78	0.09
Low	0.81	95.21	3.98

This means the probability of default for 'High' is equivalent to drawing a standard normal random number smaller than -3.1214 , or 0.09%. The probability that a 'High' ends up the period with a

rating of 'Low' or lower is equivalent to drawing a standard normal random number smaller than -2.0814 , or 1.87%. From here, the probability of ending with a 'Low' rating is:

$$P[z < -2.0814] - P[z < -3.1214] = 1.87\% - 0.09\% = 1.78\%$$

And the probability of ending with a 'High' rating is:

$$100\% - 1.87\% = 98.13\%$$

where 100% is the same as:

$$P[z < \text{Inf}]$$

Data Types: double

Output Arguments

trans — Matrix of transition probabilities in percent

matrix

Matrix of transition probabilities in percent, returned as a M-by-N matrix.

More About

Credit Quality Thresholds

Credit quality thresholds are specific levels or benchmarks used to evaluate the creditworthiness of borrowers, issuers, or financial instruments.

Transition Probabilities

Transition probabilities quantify the probability of transitioning from one state (or condition) to another within a defined time frame.

Version History

Introduced in R2011b

References

- [1] Gupton, G. M., C. C. Finger, and M. Bhatia. “*CreditMetrics*.” Technical Document, RiskMetrics Group, Inc., 2007.

See Also

transprobt thresholds | transprob | transprobbytotals

Topics

“Estimation of Transition Probabilities” on page 8-2

“Estimate Transition Probabilities for Different Rating Scales” on page 8-4

“Estimate Probabilities for Different Segments” on page 8-19

External Websites

Credit Risk Modeling with MATLAB (53 min 09 sec)

Forecasting Corporate Default Rates with MATLAB (54 min 36 sec)

transprobgrouptotals

Aggregate credit ratings information into fewer rating categories

Syntax

```
totalsGrouped = transprobgrouptotals(totals,groupingEdges)
```

Description

`totalsGrouped = transprobgrouptotals(totals,groupingEdges)` aggregates the credit ratings information stored in the `totals` input into fewer ratings categories, which are defined by the `groupingEdges` argument.

Examples

Aggregate the Credit Ratings Information Stored in the totals Input

Use historical credit rating input data from `Data_TransProb.mat`. Load input data from file `Data_TransProb.mat`.

```
load Data_TransProb
```

```
% Call TRANSPROB with two output arguments
[transMat, sampleTotals] = transprob(data);
transMat
```

```
transMat = 8×8
```

93.1170	5.8428	0.8232	0.1763	0.0376	0.0012	0.0001	0.0017
1.6166	93.1518	4.3632	0.6602	0.1626	0.0055	0.0004	0.0396
0.1237	2.9003	92.2197	4.0756	0.5365	0.0661	0.0028	0.0753
0.0236	0.2312	5.0059	90.1846	3.7979	0.4733	0.0642	0.2193
0.0216	0.1134	0.6357	5.7960	88.9866	3.4497	0.2919	0.7050
0.0010	0.0062	0.1081	0.8697	7.3366	86.7215	2.5169	2.4399
0.0002	0.0011	0.0120	0.2582	1.4294	4.2898	81.2927	12.7167
0	0	0	0	0	0	0	100.0000

Group into investment grade (ratings 1-4) and speculative grade (ratings 5-7); note, the default is the last rating (number 8).

```
edges = [4 7 8];
sampleTotalsGrp = transprobgrouptotals(sampleTotals,edges);
```

```
% Transition matrix at investment grade / speculative grade level
transMatIGSG = transprobbytotals(sampleTotalsGrp)
```

```
transMatIGSG = 3×3
```

98.5336	1.3608	0.1056
3.9155	92.9692	3.1153

```
0      0  100.0000
```

Obtain the 1-year, 2-year, 3-year, 4-year, and 5-year default probabilities at investment grade and speculative grade level.

```
DefProb = zeros(2,5);
for t = 1:5
    transMatTemp = transprobbytotals(sampleTotalsGrp,'transInterval',t);
    DefProb(:,t) = transMatTemp(1:2,3);
end
DefProb
```

```
DefProb = 2×5
```

```
0.1056    0.2521    0.4359    0.6537    0.9027
3.1153    6.0157    8.7179   11.2373   13.5881
```

Input Arguments

totals — Total transitions observed

structure | struct array

Total transitions observed, specified as a structure, or a struct array of length nTotals, with fields:

- **totalsVec** — A sparse vector of size 1-by-nRatings1.
- **totalsMat** — A sparse matrix of size nRatings1-by-nRatings2 with nRatings1 ≤ nRatings2.
- **algorithm** — A character vector with values 'duration' or 'cohort'.

For the 'duration' algorithm, **totalsMat**(*i,j*) contains the total transitions observed out of rating *i* into rating *j* (all the diagonal elements are 0). The total time spent on rating *i* is stored in **totalsVec**(*i*). For example, you have three rating categories, Investment Grade (IG), Speculative Grade (SG), and Default (D), and the following information:

Total time spent		IG	SG	D
in rating:		4859.09	1503.36	1162.05

Transitions		IG	SG	D
out of (row)	IG	0	89	7
into (column):	SG	202	0	32
	D	0	0	0

Then:

```
totals.totalsVec = [4859.09 1503.36 1162.05]
totals.totalsMat = [ 0 89 7
                   202 0 32
                   0 0 0]
totals.algorithm = 'duration'
```

For the 'cohort' algorithm, **totalsMat**(*i,j*) contains the total transitions observed from rating *i* to rating *j*, and **totalsVec**(*i*) is the initial count in rating *i*. For example, given the following information:

Initial count		IG	SG	D
in rating:		4808	1572	1145

Transitions		IG	SG	D
from (row)	IG	4721	80	7
to (column):	SG	193	1347	32
	D	0	0	1145

Then:

```
totals.totalsVec = [4808  1572  1145]
totals.totalsMat = [4721    80    7
                   193   1347   32
                    0     0   1145]
totals.algorithm = 'cohort'
```

Common totals structures are the optional output arguments from `transprob`:

- `sampleTotals` — A single structure summarizing the totals information for the whole dataset.
- `idTotals` — A struct array with the totals information at the ID level.

Data Types: `struct` | `structure`

groupingEdges — Indicator for grouping credit ratings into categories

numeric array

Indicator for grouping credit ratings into categories, specified as a numeric array.

This table illustrates how to group a list of whole ratings into investment grade (IG) and speculative grade (SG) categories. Eight ratings are in the original list. Ratings 1 to 4 are IG, ratings 5 to 7 are SG, and rating 8 is a category of its own. In this example, the array of grouping edges is `[4 7 8]`.

Original ratings:	'AAA'	'AA'	'A'	'BBB'	'BB'	'B'	'CCC'	'D'
Relative ordering:	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)
Grouped ratings:	'IG'			'SG'			'D'	
Grouping edges:				(4)				(7) (8)

In general, if `groupingEdges` has K elements `edge1 < edge2 < ... < edgeK`, ratings 1 to `edge1` (inclusive) are grouped in the first category, ratings `edge1+1` to `edge2` in the second category, and so forth.

Regarding the last element, `edgeK`:

- If `nRatings1` equals `nRatings2`, then `edgeK` must equal `nRatings1`. This leads to K groups, and `nRatingsGrouped1 = nRatingsGrouped2 = K`.
- If `nRatings1 < nRatings2`, then either:
 - `edgeK` equals `nRatings1`, in which case ratings `edgeK+1,...,nRatings2` are treated as categories of their own. This results in $K+(nRatings2-edgeK)$ groups, with `nRatingsGrouped1 = K` and `nRatingsGrouped2 = K + (nRatings2 - edgeK)`; or
 - `edgeK` equals `nRatings2`, in which case there must be a j th edge element, `edgej`, such that `edgej` equals `nRatings1`. This leads to K groups, and `nRatingsGrouped1 = j` and `nRatingsGrouped2 = K`.

Data Types: `double`

Output Arguments

totalsGrouped — Aggregated information by categories

structure | struct array

Aggregated information by categories, returned as a structure, or a struct array of length `nTotals`, with fields:

- `totalsVec` — A vector of size 1-by-`nRatingsGrouped1`.
- `totalsMat` — A matrix of size `nRatingsGrouped1`-by-`nRatingsGrouped2`.
- `algorithm` — A character vector, 'duration' or 'cohort'.

`nRatingsGrouped1` and `nRatingsGrouped2` are defined in the description of `groupingEdges`. Each structure contains aggregated information by categories, based on the information provided in the corresponding structure in `totals`, according to the grouping of ratings defined by `groupingEdges` and consistent with the `algorithm` choice.

Following the examples in the description of the `totals` input, suppose IG and SG are grouped into a single ND (Not-Defaulted) category, using the edges[2 3]. For the 'cohort' algorithm, the output is:

```
totalsGrouped.totalsVec = [6380    1145]
totalsGrouped.totalsMat = [6341     39
                           0     1145]
totalsGrouped.algorithm = 'cohort'
```

and for the 'duration' algorithm:

```
totalsGrouped.totalsVec = [6362.45  1162.05]
totalsGrouped.totalsMat = [0    39
                           0    0]
totalsGrouped.algorithm = 'duration'
```

More About

Cohort Estimation

The cohort algorithm estimates the transition probabilities based on a sequence of snapshots of credit ratings at regularly spaced points in time.

If the credit rating of a company changes twice between two snapshot dates, the intermediate rating is overlooked and only the initial and final ratings influence the estimates. For more information, see the Algorithms section of `transprob`.

Duration Estimation

Unlike the cohort algorithm, the duration algorithm estimates the transition probabilities based on the full credit ratings history, looking at the exact dates on which the credit rating migrations occur.

There is no concept of snapshots in this method, and all credit rating migrations influence the estimates, even when a company's rating changes twice within a short time. For more information, see the Algorithms section of `transprob`.

Version History

Introduced in R2011b

References

- [1] Hanson, S., T. Schuermann. "Confidence Intervals for Probabilities of Default." *Journal of Banking & Finance*. Vol. 30(8), Elsevier, August 2006, pp. 2281-2301.
- [2] Löffler, G., P. N. Posch. *Credit Risk Modeling Using Excel and VBA*. West Sussex, England: Wiley Finance, 2007.
- [3] Schuermann, T. "Credit Migration Matrices." in E. Melnick, B. Everitt (eds.), *Encyclopedia of Quantitative Risk Analysis and Assessment*. Wiley, 2008.

See Also

transprob | transprobbytotals

Topics

- "Group Credit Ratings" on page 8-15
- "Estimation of Transition Probabilities" on page 8-2
- "Estimate Transition Probabilities for Different Rating Scales" on page 8-4
- "Estimate Probabilities for Different Segments" on page 8-19

External Websites

- Credit Risk Modeling with MATLAB (53 min 09 sec)
- Forecasting Corporate Default Rates with MATLAB (54 min 36 sec)

transprobprep

Preprocess credit ratings data to estimate transition probabilities

Syntax

```
[prepData] = transprobprep(data)
[prepData] = transprobprep( ___, Name, Value)
```

Description

[prepData] = transprobprep(data) preprocesses credit ratings historical data (that is, credit migration data) for the subsequent estimation of transition probabilities.

[prepData] = transprobprep(___, Name, Value) adds optional name-value pair arguments.

Examples

Aggregate the Credit Ratings Information Stored in the totals Input

Load input data from the file Data_TransProb.mat and display the first ten rows. In this example, the inputs are provided in character vector format.

```
load Data_TransProb

% Preprocess credit ratings data.
prepData = transprobprep(data)

prepData = struct with fields:
    idStart: [1506x1 double]
    numericDates: [4315x1 double]
    numericRatings: [4315x1 double]
    ratingsLabels: {'AAA' 'AA' 'A' 'BBB' 'BB' 'B' 'CCC' 'D'}
    weights: []
```

Estimate transition probabilities with the default settings.

```
transMat = transprob(prepData)
```

```
transMat = 8x8
```

93.1170	5.8428	0.8232	0.1763	0.0376	0.0012	0.0001	0.0017
1.6166	93.1518	4.3632	0.6602	0.1626	0.0055	0.0004	0.0396
0.1237	2.9003	92.2197	4.0756	0.5365	0.0661	0.0028	0.0753
0.0236	0.2312	5.0059	90.1846	3.7979	0.4733	0.0642	0.2193
0.0216	0.1134	0.6357	5.7960	88.9866	3.4497	0.2919	0.7050
0.0010	0.0062	0.1081	0.8697	7.3366	86.7215	2.5169	2.4399
0.0002	0.0011	0.0120	0.2582	1.4294	4.2898	81.2927	12.7167
0	0	0	0	0	0	0	100.0000

Estimate transition probabilities with the 'cohort' algorithm.

```
transMatCoh = transprob(prepData, 'algorithm', 'cohort')
```

```
transMatCoh = 8×8
```

```

93.1345    5.9335    0.7456    0.1553    0.0311         0         0         0
 1.7359   92.9198    4.5446    0.6046    0.1560         0         0    0.0390
 0.1268    2.9716   91.9913    4.3124    0.4711    0.0544         0    0.0725
 0.0210    0.3785    5.0683   89.7792    4.0379    0.4627    0.0421    0.2103
 0.0221    0.1105    0.6851    6.2320   88.3757    3.6464    0.2873    0.6409
         0         0    0.0761    0.7230    7.9909   86.1872    2.7397    2.2831
         0         0         0    0.3094    1.8561    4.5630   80.8971   12.3743
         0         0         0         0         0         0         0  100.0000

```

Input Arguments

data — Credit migration data

table | cell array of character vectors | preprocessed data structure

Using `transprob` to estimate transition probabilities given credit ratings historical data (that is, credit migration data), the `data` input can be one of the following:

- An `nRecords-by-3` MATLAB table containing the historical credit ratings data of the form:

ID	Date	Rating
'00010283'	'10-Nov-1984'	'CCC'
'00010283'	'12-May-1986'	'B'
'00010283'	'29-Jun-1988'	'CCC'
'00010283'	'12-Dec-1991'	'D'
'00013326'	'09-Feb-1985'	'A'
'00013326'	'24-Feb-1994'	'AA'
'00013326'	'10-Nov-2000'	'BBB'
'00014413'	'23-Dec-1982'	'B'

Or an `nRecords-by-4` MATLAB table containing weights and the historical credit ratings data of the form:

ID	Date	Rating	Weight
'00010283'	'10-Nov-1984'	'CCC'	1
'00010283'	'12-May-1986'	'B'	1.4
'00010283'	'29-Jun-1988'	'CCC'	1.8
'00010283'	'12-Dec-1991'	'D'	0.2
'00013326'	'09-Feb-1985'	'A'	0
'00013326'	'24-Feb-1994'	'AA'	2
'00013326'	'10-Nov-2000'	'BBB'	1.7
'00014413'	'23-Dec-1982'	'B'	1.1

where each row contains an ID (column 1), a date (column 2), a credit rating (column 3), and an optional weight (column 4). Column 3 is the rating assigned to the corresponding ID on the corresponding date. All information corresponding to the same ID must be stored in contiguous rows. Sorting this information by date is not required, but recommended for efficiency. When using a MATLAB table input, the names of the columns are irrelevant, but the ID, date, rating information, and weights are assumed to be in the first, second, third, and fourth columns, respectively. Also, when using a table input, the first and third columns can be categorical arrays,

and the second can be a datetime array. The following summarizes the supported data types for table input:

Data Input Type	ID (1st Column)	Date (2nd Column)	Rating (3rd Column)	Weight (Optional 4th Column)
Table	- Numeric array - Cell array of character vectors - String array - Categorical array	- Numeric array - Cell array of character vectors - String array - Datetime array	- Numeric array - Cell array of character vectors - String array - Categorical array	Numeric array with nonnegative values

Note For an example of using the `data` input argument with an optional fourth column for `Weight`, see “Create Exposure-Based Transition Matrix From Historical Data of Credit Ratings with Exposures” on page 15-1679. If no weights are provided in a fourth column of the `data`, the default is to set all weights equal to 1. In this case, the weighted transition matrix output agrees with the ordinary, count-based transition matrix.

- An `nRecords-by-3` cell array of character vectors with the historical credit ratings data of the form:

```
'00010283' '10-Nov-1984' 'CCC'
'00010283' '12-May-1986' 'B'
'00010283' '29-Jun-1988' 'CCC'
'00010283' '12-Dec-1991' 'D'
'00013326' '09-Feb-1985' 'A'
'00013326' '24-Feb-1994' 'AA'
'00013326' '10-Nov-2000' 'BBB'
'00014413' '23-Dec-1982' 'B'
```

Or an `nRecords-by-4` cell array of character vectors if weights are included with the historical credit ratings data of the form:

```
'00010283' '10-Nov-1984' 'CCC' '1.2'
'00010283' '12-May-1986' 'B' '1'
'00010283' '29-Jun-1988' 'CCC' '1.2'
'00010283' '12-Dec-1991' 'D' '0.2'
'00013326' '09-Feb-1985' 'A' '1.7'
'00013326' '24-Feb-1994' 'AA' '1.3'
'00013326' '10-Nov-2000' 'BBB' '1'
'00014413' '23-Dec-1982' 'B' '1.8'
```

where each row contains an ID (column 1), a date (column 2), a credit rating (column 3), and an optional weight (Column 4). Column 3 is the rating assigned to the corresponding ID on the corresponding date. All information corresponding to the same ID must be stored in contiguous rows. Sorting this information by date is not required, but recommended for efficiency. IDs, dates, and ratings are stored in character vector format, but they can also be entered in numeric format. The following summarizes the supported data types for cell array input:

Data Input Type	ID (1st Column)	Date (2nd Column)	Rating (3rd Column)	Weight (Optional 4th Column)
Cell	- Numeric elements - Character vector elements	- Numeric elements - Character vector elements	- Numeric elements - Character vector elements	Numeric elements with nonnegative values

Note If no weights are provided in a fourth column of the `data`, the default is to set all weights equal to 1. In this case, the weighted transition matrix output agrees with the ordinary, count-based transition matrix.

Data Types: `table` | `cell` | `struct`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `prepData = transprobbprep(data, 'labels', {'AAA', 'AA', 'A', 'BBB', 'BB', 'B', 'CCC', 'F'})`

labels — Credit-rating scale

`{'AAA', 'AA', 'A', 'BBB', 'BB', 'B', 'CCC', 'D'}` (default) | cell array of character vectors

Credit-rating scale, specified as the comma-separated pair consisting of `'labels'` and a `nRatings-by-1`, or `1-by-nRatings` cell array of character vectors.

`labels` must be consistent with the ratings labels used in the third column of `data`. Use a cell array of numbers for numeric ratings, and a cell array for character vectors for categorical ratings.

Data Types: `cell`

Output Arguments

prepData — Summary where credit ratings information corresponding to each company starts and ends

structure

Summary where the credit ratings information corresponding to each company starts and ends, returned as a structure with the following fields:

- `idStart` — Array of size `(nIDs+1)-by-1`, where `nIDs` is the number of distinct IDs in column 1 of `data`. This array summarizes where the credit ratings information corresponding to each company starts and ends. The dates and ratings corresponding to company j in `data` are stored from row `idStart(j)` to row `idStart(j+1)-1` of `numericDates` and `numericRatings`.
- `numericDates` — Array of size `n Records-by-1`, containing the dates in column 2 of `data`, in numeric format.

- **numericRatings** — Array of size *nRecords-by-1*, containing the ratings in column 3 of **data**, mapped into numeric format.
- **weights** — (Optional) If weights are provided in the **data**, array of size *nRecords-by-1*, containing the weights in column 4 of **data**. If weights are not provided in **data**, **prepData** does not have a **Weights** field.
- **ratingsLabels** — Cell array of size *1-by-nRatings*, containing the credit rating scale.

More About

Transition Probabilities

Transition probabilities quantify the probability of transitioning from one state (or condition) to another within a defined time frame.

Version History

Introduced in R2011b

R2024a: Support for weights as optional fourth column in data

transprobprep supports an optional fourth column for weights in the **data** input.

See Also

transprob | transprobbytotals | table

Topics

“Group Credit Ratings” on page 8-15

“Estimation of Transition Probabilities” on page 8-2

“Estimate Transition Probabilities for Different Rating Scales” on page 8-4

“Estimate Probabilities for Different Segments” on page 8-19

External Websites

Credit Risk Modeling with MATLAB (53 min 09 sec)

Forecasting Corporate Default Rates with MATLAB (54 min 36 sec)

transprobt thresholds

Convert from transition probabilities to credit quality thresholds

Syntax

```
thresh = transprobt thresholds(trans)
```

Description

`thresh = transprobt thresholds(trans)` transforms transition probabilities into credit quality thresholds.

Examples

Transform Transition Probabilities Into Credit Quality Thresholds

Use historical credit rating input data from `Data_TransProb.mat`. Load input data from file `Data_TransProb.mat`.

```
load Data_TransProb
```

```
% Estimate transition probabilities with default settings
```

```
transMat = transprob(data)
```

```
transMat = 8×8
```

93.1170	5.8428	0.8232	0.1763	0.0376	0.0012	0.0001	0.0017
1.6166	93.1518	4.3632	0.6602	0.1626	0.0055	0.0004	0.0396
0.1237	2.9003	92.2197	4.0756	0.5365	0.0661	0.0028	0.0753
0.0236	0.2312	5.0059	90.1846	3.7979	0.4733	0.0642	0.2193
0.0216	0.1134	0.6357	5.7960	88.9866	3.4497	0.2919	0.7050
0.0010	0.0062	0.1081	0.8697	7.3366	86.7215	2.5169	2.4399
0.0002	0.0011	0.0120	0.2582	1.4294	4.2898	81.2927	12.7167
0	0	0	0	0	0	0	100.0000

Obtain the credit quality thresholds.

```
thresh = transprobt thresholds(transMat)
```

```
thresh = 8×8
```

Inf	-1.4846	-2.3115	-2.8523	-3.3480	-4.0083	-4.1276	-4.1413
Inf	2.1403	-1.6228	-2.3788	-2.8655	-3.3166	-3.3523	-3.3554
Inf	3.0264	1.8773	-1.6690	-2.4673	-2.9800	-3.1631	-3.1736
Inf	3.4963	2.8009	1.6201	-1.6897	-2.4291	-2.7663	-2.8490
Inf	3.5195	2.9999	2.4225	1.5089	-1.7010	-2.3275	-2.4547
Inf	4.2696	3.8015	3.0477	2.3320	1.3838	-1.6491	-1.9703
Inf	4.6241	4.2097	3.6472	2.7803	2.1199	1.5556	-1.1399
Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf

Input Arguments

trans — Transition probabilities in percent

matrix

Transition probabilities in percent, specified as a M-by-N matrix. Entries cannot be negative and cannot exceed 100, and all rows must add up to 100.

Any given row in the M-by-N input matrix **trans** determines a probability distribution over a discrete set of N ratings. If the ratings are 'R1', ..., 'RN', then for any row *i* **trans**(*i*,*j*) is the probability of migrating into 'Rj'. If **trans** is a standard transition matrix, then $M \leq N$ and row *i* contains the transition probabilities for issuers with rating 'Ri'. But **trans** does not have to be a standard transition matrix. **trans** can contain individual transition probabilities for a set of M-specific issuers, with $M > N$.

The credit quality thresholds **thresh**(*i*,*j*) are critical values of a standard normal distribution *z*, such that:

$$\text{trans}(i,N) = P[z < \text{thresh}(i,N)],$$

$$\text{trans}(i,j) = P[z < \text{thresh}(i,j)] - P[z < \text{thresh}(i,j+1)], \text{ for } 1 \leq j < N$$

This implies that **thresh**(*i*,1) = Inf, for all *i*. For example, suppose that there are only N=3 ratings, 'High', 'Low', and 'Default', with the following transition probabilities:

	High	Low	Default
High	98.13	1.78	0.09
Low	0.81	95.21	3.98

The matrix of credit quality thresholds is:

	High	Low	Default
High	Inf	-2.0814	-3.1214
Low	Inf	2.4044	-1.7530

This means the probability of default for 'High' is equivalent to drawing a standard normal random number smaller than -3.1214, or 0.09%. The probability that a 'High' ends up the period with a rating of 'Low' or lower is equivalent to drawing a standard normal random number smaller than -2.0814, or 1.87%. From here, the probability of ending with a 'Low' rating is:

$$P[z < -2.0814] - P[z < -3.1214] = 1.87\% - 0.09\% = 1.78\%$$

And the probability of ending with a 'High' rating is:

$$100\% - 1.87\% = 98.13\%$$

where 100% is the same as:

$$P[z < \text{Inf}]$$

Data Types: double

Output Arguments

thresh — Credit quality thresholds

matrix

Credit quality thresholds, returned as a M-by-N matrix.

More About

Transition Probabilities

Transition probabilities quantify the probability of transitioning from one state (or condition) to another within a defined time frame.

Credit Quality Thresholds

Credit quality thresholds are specific levels or benchmarks used to evaluate the creditworthiness of borrowers, issuers, or financial instruments.

Version History

Introduced in R2011b

References

[1] Gupton, G. M., C. C. Finger, and M. Bhatia. “*CreditMetrics*.” Technical Document, RiskMetrics Group, Inc., 2007.

See Also

`transprob` | `transprobbytotals` | `transprobfromthresholds`

Topics

“Credit Quality Thresholds” on page 8-45

“Group Credit Ratings” on page 8-15

“Estimation of Transition Probabilities” on page 8-2

“Estimate Transition Probabilities for Different Rating Scales” on page 8-4

“Estimate Probabilities for Different Segments” on page 8-19

External Websites

Credit Risk Modeling with MATLAB (53 min 09 sec)

Forecasting Corporate Default Rates with MATLAB (54 min 36 sec)

tsaccel

Acceleration between times

Syntax

```
acceleration = tsaccel(Data)  
acceleration = tsaccel( ___,Name,Value)
```

Description

`acceleration = tsaccel(Data)` calculates the acceleration of a data series with time distance of n periods.

n periods.

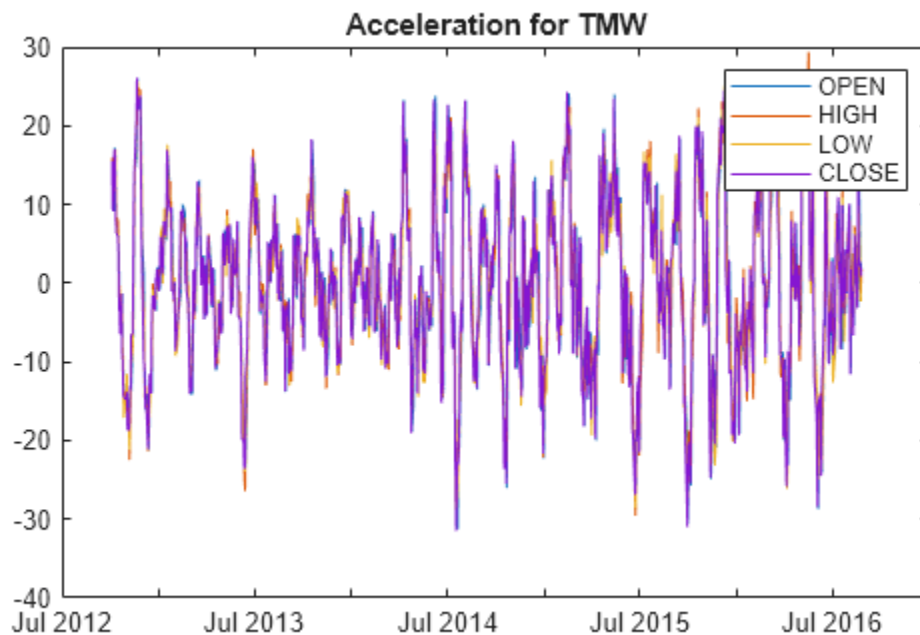
`acceleration = tsaccel( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Acceleration of a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
TMW.Volume = []; % remove VOLUME field  
acceleration = tsaccel(TMW);  
plot(acceleration.Time,acceleration.Variables)  
legend('OPEN','HIGH','LOW','CLOSE')  
title('Acceleration for TMW')
```



Input Arguments

Data — Data with high, low, open, close information

vector | matrix | table | timetable

Data with high, low, open, close information, specified as a vector, matrix, table, or timetable. For vector input, **Data** is a column vector of high, low, open, and closing prices stored in the corresponding columns. For matrix input, **Data** is an M-by-N column-oriented matrix of high, low, open, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'High', 'Low', 'Open', and 'Close' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `acceleartion = tsaccel(TMW,'NumPeriods',10,'Datatype',1)`

NumPeriods — Period difference for acceleration

12 (default) | positive integer

Period difference for acceleration, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Datatype — Indicates if Data contains the data itself or the momentum of the data

0 (Data contains the data itself) (default) | integer with values 0 or 1

Indicates if Data contains the data itself or the momentum of the data, specified as the comma-separated pair consisting of 'Datatype' and a scalar integer with a value of:

- 0 - Data contains the data itself.
- 1 - Data contains the momentum of the data.

Data Types: double

Output Arguments**acceleration — Acceleration series**

vector | matrix | table | timetable

Acceleration series, returned with the same number of rows (M) and columns (N) and the same type (vector, matrix, table, or timetable) as the input Data.

More About**Acceleration**

Acceleration is defined as the difference of two momentum series separated by n periods.

Acceleration is the difference of the current momentum with the momentum n periods ago. By default, acceleration is based on 12-period difference.

Version History**Introduced before R2006a****R2023a: fints support removed for Data input argument**

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Kaufman, P. J. *The New Commodity Trading Systems and Methods*. John Wiley and Sons, New York, 1987.

See Also

timetable | table

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

tsmom

Momentum between times

Syntax

```
momentum = tsmom(Data)
momentum = tsmom( ___,Name,Value)
```

Description

`momentum = tsmom(Data)` calculates the momentum of a data series with time distance of n periods.

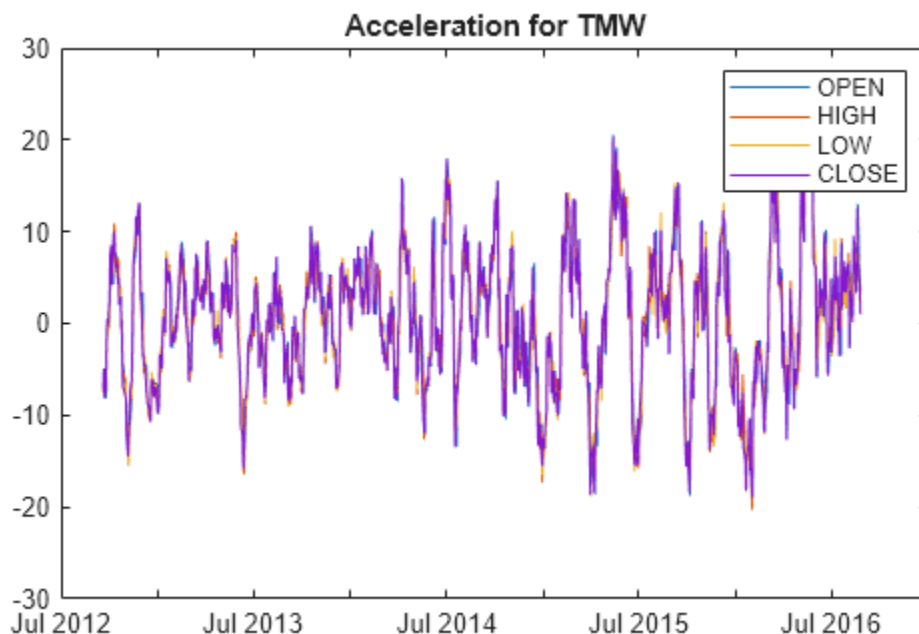
`momentum = tsmom( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Momentum for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
TMW.Volume = []; % remove VOLUME field
momentum = tsmom(TMW);
plot(momentum.Time,momentum.Variables)
legend('OPEN','HIGH','LOW','CLOSE')
title('Acceleration for TMW')
```



Input Arguments

Data — Data with high, low, open, close information

vector | matrix | table | timetable

Data with high, low, open, close information, specified as a vector, matrix, table, or timetable. For vector input, **Data** is a column vector. For matrix input, **Data** is an M-by-N column oriented matrix. Timetables and tables with M rows can contain variables named 'High', 'Low', 'Open', and 'Close' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `momentum = tsmom(TMW,'NumPeriods',15)`

NumPeriods — Period difference for momentum

12 (default) | positive integer

Period difference for momentum, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Output Arguments

momentum — Momentum series

matrix | table | timetable

Momentum series, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

More About

Momentum Series

Momentum series is the difference of the current data with the data *n* periods ago. By default, momentum is based on 12-period difference.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

`fints` object support for the `Data` input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The `Data` input accepts negative prices.

References

[1] Kaufman, P. J. *The New Commodity Trading Systems and Methods*. John Wiley and Sons, New York, 1987.

See Also

`timetable` | `table` | `tsaccel`

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (`fints`) to Timetables" on page 11-2

typprice

Typical price

Syntax

```
TypicalPrice = typprice(Data)
```

Description

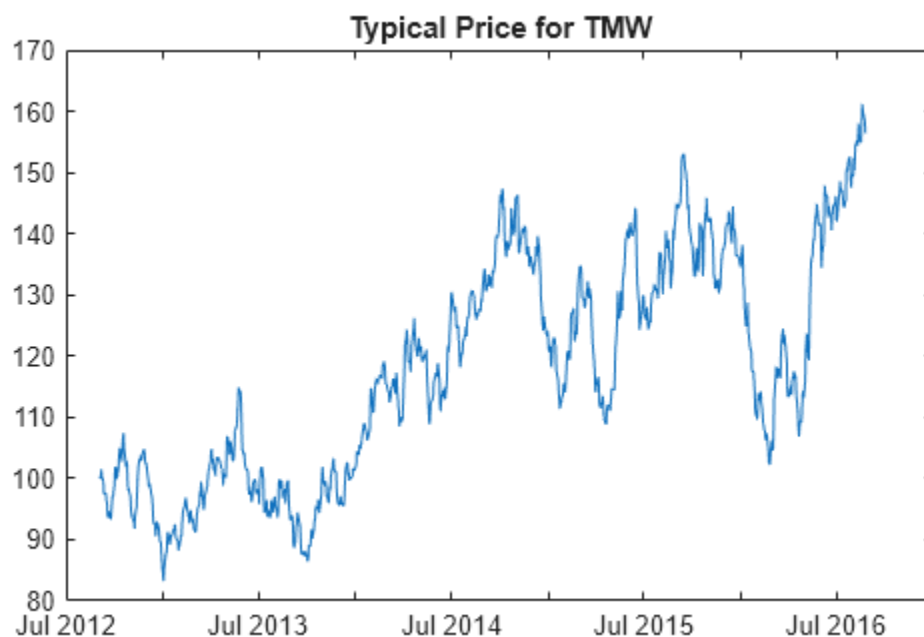
`TypicalPrice = typprice(Data)` calculates the typical prices from the series of high, low, and closing prices. The typical price is the average of the high, low, and closing prices for each period.

Examples

Calculate the Typical Price for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
TypicalPrice = typprice(TMW);
plot(TypicalPrice.Time, TypicalPrice.TypicalPrice)
title('Typical Price for TMW')
```



Input Arguments

Data — Data for high, low, and closing prices

matrix | table | timetable

Data for high, low, and closing prices, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-3 matrix of high, low, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Output Arguments

TypicalPrice — Typical price series

matrix | table | timetable

Typical price series, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

More About

Typical Price

The typical price is a technical analysis indicator used in trading to provide a single price value that represents the average price of a security over a specific period.

Typical price is particularly useful for smoothing out price fluctuations and providing a clearer view of the overall price trend.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 291–292.

See Also

timetable | table | medprice | wclose

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

uicalendar

Graphical calendar

Syntax

`uicalendar(Name,Value)`

Description

`uicalendar(Name,Value)` supports a customizable graphical calendar that interfaces with one or more `uicontrol`. `uicalendar` populates one or more `uicontrol` with user-selected dates.

Note As an alternative to `uicalendar`, you can use `uidatepicker`.

Examples

Use `uicalendar` with an `uicontrol`

Create an `uicontrol`:

```
textH1 = uicontrol('style', 'edit', 'position', [10 10 100 20])
```

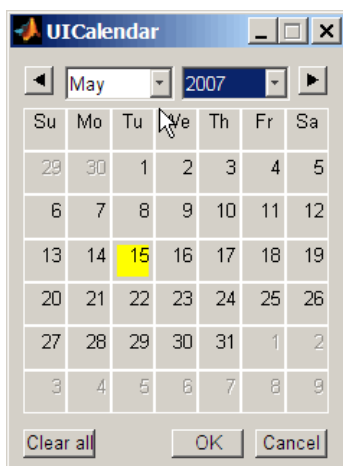
```
textH1 =
```

```
    UIControl with properties:
```

```
        Style: 'edit'
        String: ''
    BackgroundColor: [0.9400 0.9400 0.9400]
        Callback: ''
           Value: 0
      Position: [10 10 100 20]
         Units: 'pixels'
```

Call `UICalendar`:

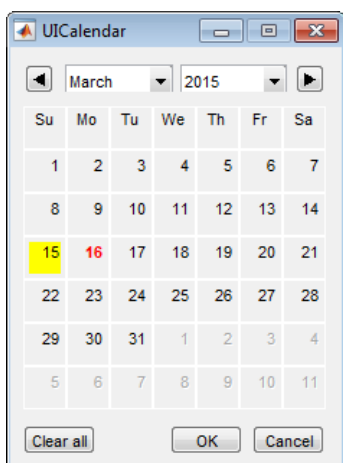
```
uicalendar('DestinationUI', {textH1, 'string'})
```



Select a date and click OK.

Alternatively, you can use datetime arrays for `InitDate` and `Holiday`.

```
uicalendar('InitDate',datetime('15-Mar-2015','Locale','en_US'),'Holiday',datetime('16-Mar-2015','Locale','en_US'))
```



Select a date and click OK. For more information on using `uicalendar` with an application, see “Example of Using UICalendar with an Application” on page 12-4.

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `uicalendar('InitDate',datetime('15-Mar-2015','Locale','en_US'),'Holiday',datetime('16-Mar-2015','Locale','en_US'))`

BusDays — Flag to indicate nonbusiness days

0 (Standard calendar without nonbusiness day indicators) (default) | numeric values of 0 or 1

Flag to indicate nonbusiness days, specified using numeric values of 0 or 1. The values are:

- 0 — (Default) Standard calendar without nonbusiness day indicators.
- 1 — Marks NYSE nonbusiness days in red.

Data Types: `logical`

BusDaySelect — Flag to indicate whether business and nonbusiness days

1 (Allows selections of business and nonbusiness days) (default) | numeric values of 0 or 1

Flag to indicate whether business and nonbusiness days, specified using numeric values of 0 or 1. The values are:

- 0 — Only allow selection of business days. Nonbusiness days are determined from the following parameters:
 - 'BusDays'
 - 'Holiday'
 - 'Weekend'
- 1 — (Default) Allows selections of business and nonbusiness days.

Data Types: `logical`

DateBoxColor — Color of date squares

[date R G B]

Color of date squares, specified using [date R G B], where [R G B] is the color.

Data Types: `double`

DateStrColor — Color of numeric date number in the date square

[date R G B]

Color of numeric date number in the date square, specified using [date R G B], where [R G B] is the color.

Data Types: `double`

DestinationUI — Destination object's handles

'string' (default UI property populated with dates) (default) | values are H or {H, {Prop}}

Destination object's handles, specified with values H or {H, {Prop}}. The values are:

- H — Scalar or vector of the destination object's handles. The default UI property that is populated with the dates is a character vector.
- {H, {Prop}} — Cell array of handles and the destination object's UI properties. H must be a scalar or vector and Prop must be a single property character vector or a cell array of property character vectors.

Data Types: `char` | `cell`

Holiday — Holiday dates in calendar

datetime array | string array | date character vector | serial date number

Holiday dates in calendar, specified using a scalar or vector using a datetime array, string array, date character vectors, or serial date numbers. The corresponding date of the holiday appears Red.

Data Types: double | char | string | datetime

InitDate — Initial start date when calendar is initialized

TODAY (default) | serial date number | datetime array | date character vector

Initial start date when calendar is initialized, specified with date values using a serial date number, datetime array, or date character vector. The values are:

- **Datenum** — Numeric or datetime array date value specifying the initial start date when the calendar is initialized. The default date is TODAY.
- **Datestr** — Date character vector value specifying the initial start date when the calendar is initialized. Datestr must include a Year, Month, and Day (for example, 01-Jan-2006).

Data Types: double | char | datetime

InputDateFormat — Format of initial start date

character vector

Format of initial start date (InitDate), specified using a character vector or datetime.

Data Types: char | datetime

OutputDateFormat — Format of output date

character vector

Format of output date, specified using a character vector or datetime.

Data Types: char | datetime

OutputDateStyle — Style for output date

0 (default) | numeric value of 0, 1, 2, or 3

Style for output date, specified using a value of 0, 1, 2, or 3. The values are:

- **0** — (Default) Returns a single date character vector or a cell array (row) of date character vectors. For example, {'01-Jan-2001', '02-Jan-2001', ...'}.
- **1** — Returns a single date character vector or a cell (column) array of date character vectors. For example, {'01-Jan-2001'; '02-Jan-2001'; ...'}.
- **2** — Returns a character vector representation of a row vector of datenums. For example, '[732758, 732759, 732760, 732761]'.
- **3** — Returns a character vector representation of a column vector of datenums. For example, '[732758; 732759; 732760; 732761]'.

Data Types: double

SelectionType — Flag for date selection

1 (default) | numeric value of 0 or 1

Flag for date selection, specified with using a value of 0 or 1. The values are:

- 0 — Allows multiple date selections.
- 1 — (Default) Allows only a single date selection.

Data Types: `logical`

Weekend — Define weekend days

1 (default) | numeric values of 1 through 7

Define weekend days, specified using a value of 1 through 7. Weekend days are marked in red. `DayOfWeek` can be a vector containing the following numeric values:

- 1 — Sunday
- 2 — Monday
- 3 — Tuesday
- 4 — Wednesday
- 5 — Thursday
- 6 — Friday
- 7 — Saturday

Also this value can be a vector of length 7 containing 0's and 1's. The value 1 indicates a weekend day. The first element of this vector corresponds to Sunday. For example, when Saturday and Sunday are weekend days then `WEEKEND = [1 0 0 0 0 0 1]`.

Data Types: `double`

WindowStyle — Window figure properties

`Normal` (default) | character vector with value of `Normal` or `Modal`

Window figure properties, specified with using a character vector with a value of `Normal` or `Modal`. The values are:

- `Normal` — (Default) Standard figure properties.
- `Modal` — Modal figures remain stacked above all normal figures and the MATLAB Command Window.

Data Types: `char`

Version History

Introduced before R2006a

See Also

`holidays` | `datetime` | `uidatepicker`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

volarea

Price and volume chart

Syntax

```
volarea(Data)
h = volarea(ax,Data)
```

Description

`volarea(Data)` plots a chart from a series of price and traded volume of a security.

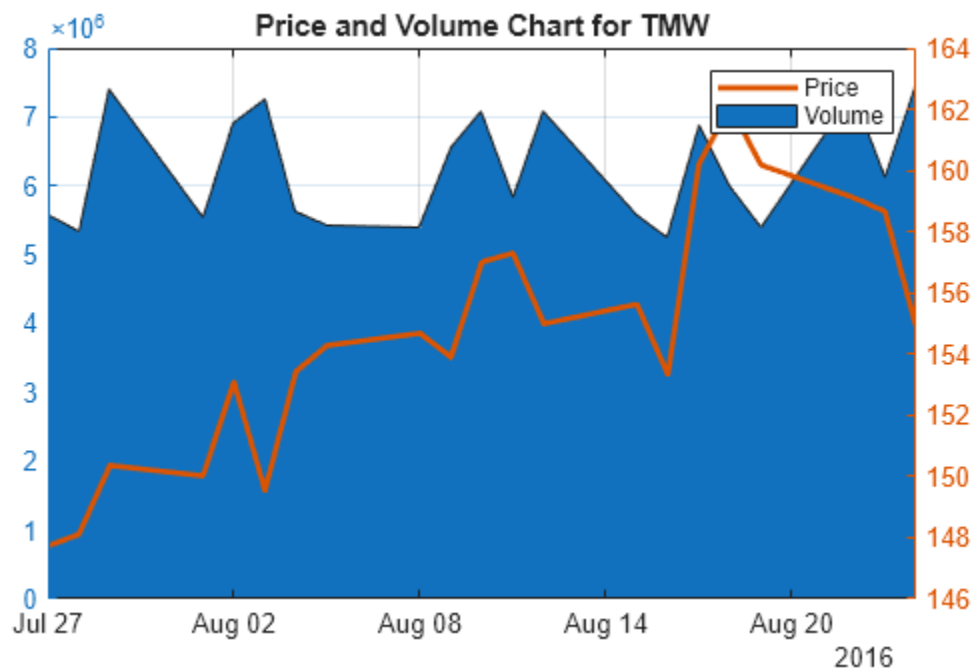
`h = volarea(ax,Data)` adds an optional argument for `ax`.

Examples

Plot Asset Date, Price, and Volume on a Single Axis for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock. This example shows how to plot asset date, price, and volume on a single axis, given asset TMW containing asset price and volume in most recent 21 days. Note that the variable name of asset price is be renamed to 'Price' (case insensitive).

```
load SimulatedStock.mat;
TMW.Properties.VariableNames{'Close'} = 'Price';
volarea(TMW(end-20:end,:))
title('Price and Volume Chart for TMW')
```



Input Arguments

Data — Data for prices and traded volume

matrix | table | timetable

Data for prices and traded volume, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-2 matrix of prices and traded volume. Timetables and tables with M rows must contain variables named 'Price' and 'Volume' (case insensitive).

Data Types: double | table | timetable

ax — Valid axis object

current axes (ax = gca) (default) | axes object

(Optional) Valid axis object, specified as an axes object. The volarea plot is created in the axes specified by ax instead of in the current axes (ax = gca). The option ax can precede any of the input argument combinations.

Data Types: object

Output Arguments

h — Graphic handle of the figure

handle object

Graphic handle of the figure, returned as a handle object.

More About

Price and Volume Chart

A price and volume chart is a graphical representation used in financial markets to display the price movements of a security (such as a stock, commodity, or currency) alongside the volume of trades that occur over a specific period.

Version History

Introduced in R2008a

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

R2018a: Support for matrix, time, or timetable input for data

Behavior changed in R2018a

volarea accepts Data input as a matrix, timetable, or table.

The syntax for volarea has changed. Previously, when using table input, the first column of dates could be a datetime array, date character vectors, or serial date numbers, and you were required to have specific number of columns.

When using table input, the new syntax for `volarea` supports:

- No need for time information. If you want to pass in date information, use timetable input.
- No requirement of specific number of columns. However, you must provide valid column names. `volarea` must contain a column named 'price' (case insensitive).

See Also

`timetable` | `table` | `movavg` | `linebreak` | `highlow` | `kagi` | `candle` | `pointfig`

Topics

"Use Timetables in Finance" on page 11-7

"Convert Financial Time Series Objects (fints) to Timetables" on page 11-2

volroc

Volume rate of change

Syntax

```
volumeChangeRate = volroc(Data)  
volumeChangeRate = volroc( ___,Name,Value)
```

Description

`volumeChangeRate = volroc(Data)` calculates the volume rate-of-change from a data series of volume traded. The volume rate-of-change is calculated between the current volume and the volume n periods ago. By default, the Volume rate of change is based on a 12-period difference.

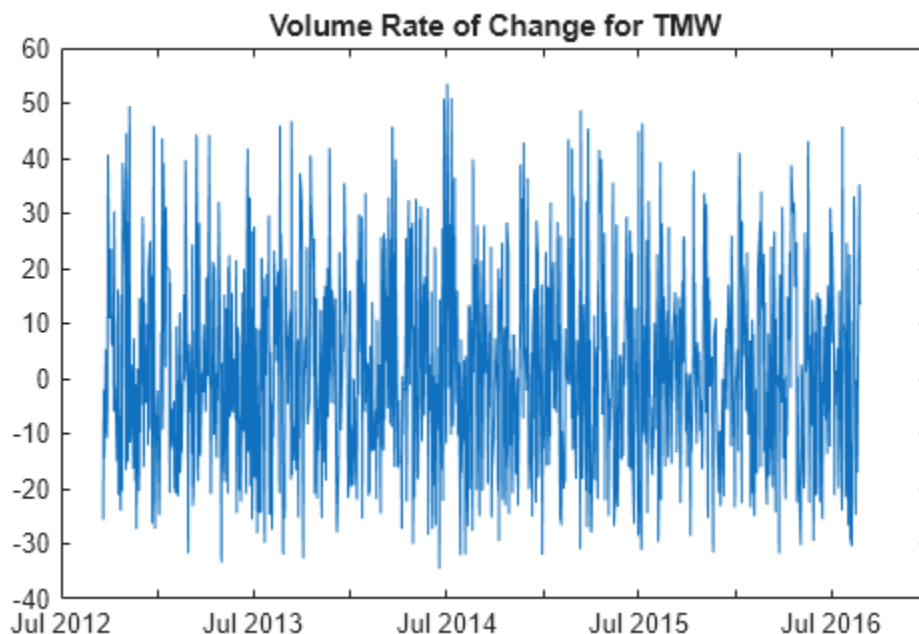
`volumeChangeRate = volroc( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Volume Rate of Change for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
volumeChangeRate = volroc(TMW);  
plot(volumeChangeRate.Time,volumeChangeRate.VolumeChangeRate)  
title('Volume Rate of Change for TMW')
```



Input Arguments

Data — Data for volume traded

vector | table | timetable

Data for volume traded, specified as a vector, table, or timetable. For vector input, **Data** is an M-by-1 column vector of volume traded. Timetables and tables with M rows must contain a variable named 'Volume' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `volumeChangeRate = volroc(TMW,'NumPeriods',18)`

NumPeriods — Period difference for volumeChangeRate

12 (default) | positive integer

Period difference for `volumeChangeRate`, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Output Arguments

volumeChangeRate — Volume rate of change series

vector | table | timetable

Volume rate of change series, returned with the same number of rows (M) and the same type (vector, table, or timetable) as the input **Data**.

More About

Volume Rate of Change Chart

A volume rate of change (VROC) chart is a technical analysis tool that measures the rate of change in trading volume over a specified period.

The VROC chart helps traders and investors assess the momentum of volume trends, which can provide insights into the strength or weakness of price movements in a security. The VROC can be particularly useful for identifying potential reversals, breakouts, or confirmations of existing trends.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument*Behavior changed in R2023a*

fints object support for the Data input argument is removed.

R2022b: Support for negative price data*Behavior changed in R2022b*

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 310-311.

See Also

timetable | table | prcroc

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

wclose

Weighted close

Syntax

```
WeightedClose = wclose(Data)
```

Description

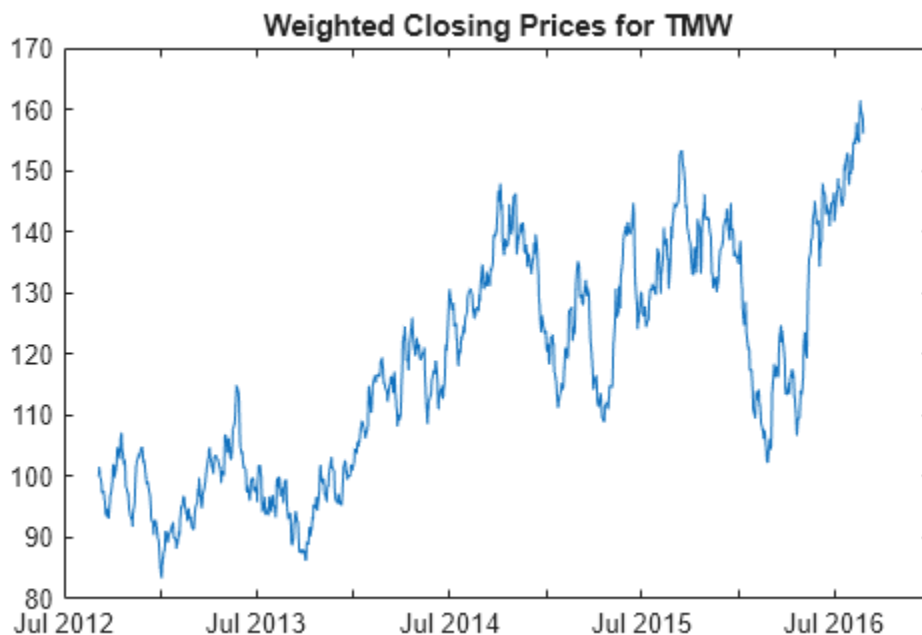
`WeightedClose = wclose(Data)` calculates the weighted closing prices from the series of high, low, and closing prices. The weighted closing price is the average of twice the closing price plus the high and low prices.

Examples

Calculate the Weighted Closing Prices for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
WeightedClose = wclose(TMW);
plot(WeightedClose.Time,WeightedClose.WeightedClose)
title('Weighted Closing Prices for TMW')
```



Input Arguments

Data — Data for high, low, and closing prices

matrix | table | timetable

Data for high, low, and closing prices, specified as a matrix, table, or timetable. For matrix input, Data is an M-by-3 matrix of high, low, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Output Arguments

WeightedClose — Weighted closing price series

matrix | table | timetable

Weighted closing price series, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input Data.

More About

Weighted Close Chart

A weighted close chart is a type of price chart used in technical analysis that calculates the closing price of a security with a greater emphasis on the most recent prices.

The weighted close method is particularly useful for smoothing out price data and providing a more responsive indicator of price trends compared to a simple closing price chart. The weighted close is often employed in trading strategies to better reflect the current market sentiment.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 312–313.

See Also

timetable | table | medprice | typprice

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

weights2holdings

Portfolio values and weights into holdings

Syntax

`Holdings = weights2holdings(Values,Weights,Prices)`

Description

`Holdings = weights2holdings(Values,Weights,Prices)` converts portfolio values and weights into portfolio holdings.

Note `weights2holdings` does not create round-lot positions. The output `Holdings` are floating-point values.

Input Arguments

Values — Portfolio values

scalar numeric | vector

Portfolio values, specified as a scalar or an `NPORTS` vector.

Data Types: `double`

Weights — Portfolio weights

matrix

Portfolio weights, specified as an `NPORTS`-by-`NASSETS` matrix. The weights sum to the value of a `Budget` constraint, which is usually 1. (See `holdings2weights` for information about budget constraints.)

Data Types: `double`

Prices — Asset prices

vector

Asset prices, specified as an `NASSETS` vector.

Data Types: `double`

Output Arguments

Holdings — Holdings value

matrix

Holdings value, returned as an `NPORTS`-by-`NASSETS` matrix that contains the holdings of `NPORTS` portfolios that contain `NASSETS` assets.

Version History

Introduced before R2006a

See Also

holdings2weights

willad

Williams Accumulation/Distribution line

Syntax

```
WADLine = willad(Data)
```

Description

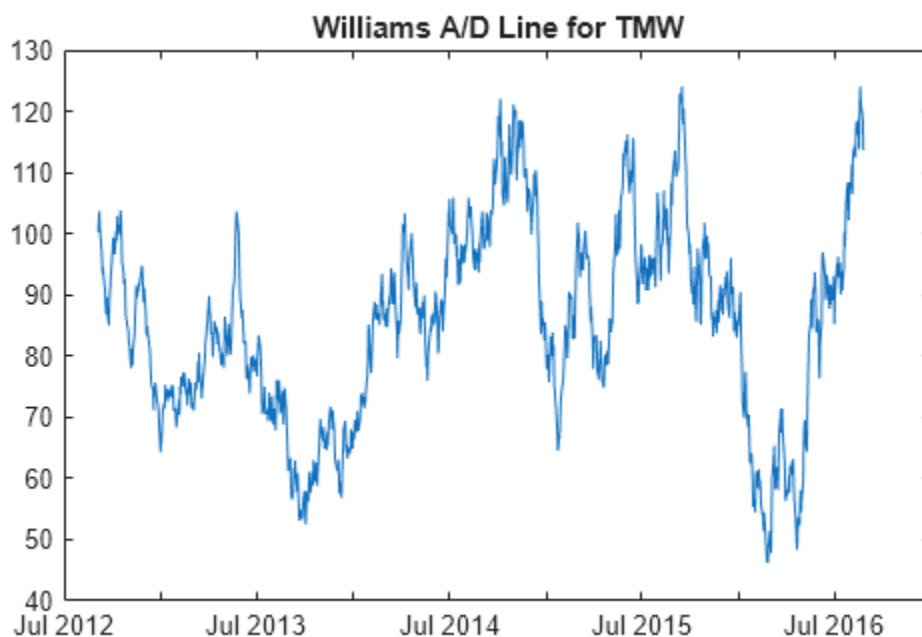
`WADLine = willad(Data)` calculates the Williams Accumulation/Distribution line from the series of high, low, and closing prices.

Examples

Calculate the Williams Accumulation/Distribution Line for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat
WADLine = willad(TMW);
plot(WADLine.Time,WADLine.WillAD)
title('Williams A/D Line for TMW')
```



Input Arguments

Data — Data for high, low, and closing prices

matrix | table | timetable

Data for high, low, and closing prices, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-3 matrix of high, low, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Output Arguments

WADLine — Williams Accumulation/Distribution line

matrix | table | timetable

Williams Accumulation/Distribution line, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

More About

Williams Accumulation/Distribution Line

The Williams Accumulation/Distribution Line is a technical analysis indicator used to assess the buying and selling pressure of a security by analyzing both price movements and volume.

The Williams Accumulation/Distribution Line aims to determine whether a security is being accumulated (bought) or distributed (sold) over a specific period. It helps traders gauge the strength of a price trend and the underlying market dynamics. A rising Accumulation/Distribution Line indicates accumulation (buying pressure), suggesting that the price may continue to rise. Conversely, a declining A/D Line indicates distribution (selling pressure), suggesting potential price declines.

Version History

Introduced before R2006a

R2023a: **fints** support removed for Data input argument

Behavior changed in R2023a

fints object support for the **Data** input argument is removed.

R2022b: **Support for negative price data**

Behavior changed in R2022b

The **Data** input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 314–315.

See Also

`timetable` | `table` | `adline` | `adosc` | `willpctr`

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

willpctr

Williams %R

Syntax

```
PercentR = willpctr(Data)  
PercentR = willpctr( ___,Name,Value)
```

Description

`PercentR = willpctr(Data)` calculates the Williams PercentR (%R) values for a data series of with high, low, and closing prices.

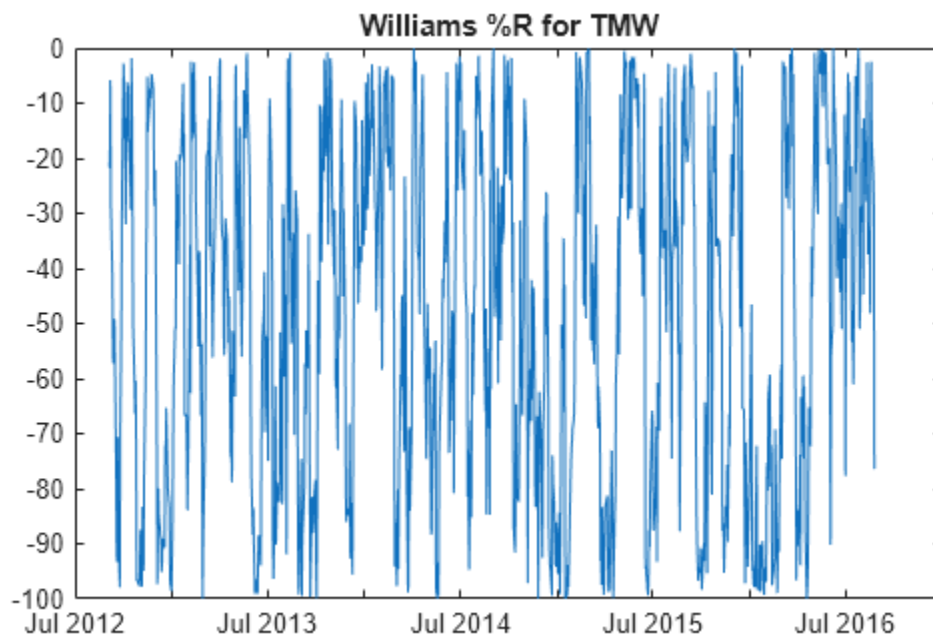
`PercentR = willpctr( ___,Name,Value)` adds optional name-value pair arguments.

Examples

Calculate the Williams %R for a Data Series for a Stock

Load the file `SimulatedStock.mat`, which provides a timetable (TMW) for financial data for TMW stock.

```
load SimulatedStock.mat  
PercentR = willpctr(TMW);  
plot(PercentR.Time,PercentR.WillPercentR)  
title('Williams %R for TMW')
```



Input Arguments

Data — Data with high, low, and close information

matrix | table | timetable

Data with high, low, open, close information, specified as a matrix, table, or timetable. For matrix input, **Data** is an M-by-3 with high, low, and closing prices stored in the corresponding columns. Timetables and tables with M rows must contain variables named 'High', 'Low', and 'Close' (case insensitive).

Data Types: double | table | timetable

Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `PercentR = willpctr(TMW,'NumPeriods',15)`

NumPeriods — Moving window for Williams PercentR

14 (default) | positive integer

Moving window for Williams PercentR, specified as the comma-separated pair consisting of 'NumPeriods' and a scalar positive integer.

Data Types: double

Output Arguments

PercentR — Williams PercentR series

matrix | table | timetable

Williams PercentR series, returned with the same number of rows (M) and the same type (matrix, table, or timetable) as the input **Data**.

More About

Williams %R

Williams %R is a momentum-based technical analysis indicator that measures overbought and oversold levels in a security's price and helps traders identify potential reversal points.

The Williams %R values range from 0 to -100:

- A reading of 0 indicates that the closing price is at the highest high for the lookback period.
- A reading of -100 indicates that the closing price is at the lowest low for the lookback period.
- Readings above -20 are typically considered overbought, while readings below -80 are considered oversold.

By default, Williams %R values are based on 14 periods.

Version History

Introduced before R2006a

R2023a: fints support removed for Data input argument

Behavior changed in R2023a

fints object support for the Data input argument is removed.

R2022b: Support for negative price data

Behavior changed in R2022b

The Data input accepts negative prices.

References

[1] Achelis, S. B. *Technical Analysis from A to Z*. Second Edition. McGraw-Hill, 1995, pp. 316-317.

See Also

timetable | table | willad | stochosc

Topics

“Use Timetables in Finance” on page 11-7

“Convert Financial Time Series Objects (fints) to Timetables” on page 11-2

wrkdydif

Number of working days between dates

Syntax

```
Days = wrkdydif(StartDate,EndDate,Holidays)
```

Description

`Days = wrkdydif(StartDate,EndDate,Holidays)` returns the number of working days between dates `StartDate` and `EndDate` inclusive. `Holidays` is the number of holidays between the given dates, an integer.

Examples

Determine the Number of Working Days Between a StartDate and EndDate

Determine `Days` using date character vectors for `StartDate` and `EndDate`.

```
Days = wrkdydif('9/1/2000', '9/11/2000', 1)
```

```
Days =  
6
```

Determine `Days` using a datetimes for array for `StartDate` and `EndDate`.

```
Days = wrkdydif(datetime(2000,9,1), datetime(2000,9,11), 1)
```

```
Days =  
6
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `wrkdydif` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as an N-by-1 or 1-by-N vector using a datetime array, string array, or date character vectors.

To support existing code, `wrkdydif` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Holidays — Number of holidays between StartDate and EndDate

vector of integers

Number of holidays between the `StartDate` and `EndDate`, specified as an N-by-1 or 1-by-N vector of integers.

Data Types: `double`

Output Arguments

Days — Number of working days between dates StartDate and EndDate inclusive

integer

Number of working days between dates `StartDate` and `EndDate` inclusive, returned an N-by-1 or 1-by-N vector of integers.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `wrkdydif` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

See Also

`busdate` | `datewrkdy` | `days365` | `daysact` | `daysdif` | `holidays` | `yearfrac` | `datetime`

Topics

“Handle and Convert Dates” on page 2-2

“Trading Calendars User Interface” on page 12-2

“UICalendar User Interface” on page 12-4

xirr

Internal rate of return for nonperiodic cash flow

Syntax

```
Return = xirr(CashFlow,CashFlowDates)
Return = xirr( ____,Guess,MaxIterations,Basis)
```

Description

`Return = xirr(CashFlow,CashFlowDates)` returns the internal rate of return for a schedule of nonperiodic cash flows.

`Return = xirr( ____,Guess,MaxIterations,Basis)` adds optional arguments.

Examples

Find Internal Rate of Return for Nonperiodic Cash Flow

Find the internal rate of return for an investment of \$10,000 that returns the following nonperiodic cash flow. The original investment is the first cash flow and is a negative number.

Cash Flow Dates

-10000 12-Jan-2007

2500 14-Feb-2008

2000 03-Mar-2008

3000 14-Jun-2008

4000 01-Dec-2008

Calculate the internal rate of return for this nonperiodic cash flow:

```
CashFlow = [-10000, 2500, 2000, 3000, 4000];
CashFlowDates = ['01/12/2007'
                  '02/14/2008'
                  '03/03/2008'
                  '06/14/2008'
                  '12/01/2008'];
```

```
Return = xirr(CashFlow, CashFlowDates)
```

```
Return =
0.1006
```

Alternatively, you can use datetime input to calculate the internal rate of return for this nonperiodic cash flow:

```
CashFlow = [-10000, 2500, 2000, 3000, 4000];
CashFlowDates = ['01/12/2007'
```

```
        '02/14/2008'  
        '03/03/2008'  
        '06/14/2008'  
        '12/01/2008'];  
CashFlowDates = datetime(CashFlowDates, 'Locale', 'en_US');  
Return = xirr(CashFlow, CashFlowDates)  
  
Return =  
0.1006
```

Input Arguments

CashFlow — Cash flow

vector | matrix

Cash flow, specified as a vector or matrix. The first entry is the initial investment. If **CashFlow** is a matrix, each column represents a separate stream of cash flows whose internal rate of return is calculated. The first cash flow of each stream is the initial investment, usually entered as a negative number.

Data Types: double

CashFlowDates — Cash flow dates

datetime array | string array | date character vector

Cash flow dates, specified as a vector or matrix using a datetime array, string array, or date character vectors. The size of the input date numbers for **CashFlowDates** must be the same size as **CashFlow**. Each column of **CashFlowDates** represents the dates of the corresponding column of **CashFlow**.

To support existing code, **xirr** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | datetime | string

Guess — Initial estimate of the internal rate of return

0.1 (10%) (default) | numeric

(Optional) Initial estimate of the internal rate of return, specified as a scalar or vector. If **Guess** is a scalar, then it is applied to all streams, and if **Guess** is a vector, then it is the same length as the number of streams.

Data Types: double

MaxIterations — Number of iterations used by Newton's method to solve the internal rate of return

50 (default) | positive integer

(Optional) Number of iterations used by Newton's method to solve the internal rate of return, specified as a scalar or vector of positive integers. If **MaxIterations** is a scalar, then it is applied to all streams, and if **MaxIterations** is a vector, then it is the same length as the number of streams.

Data Types: double

Basis — Day-count basis

0 (actual/actual) (default) | positive integers of the set [1...13] | vector of positive integers of the set [1...13]

(Optional) Day-count basis, specified as a positive integer using scalar or a N-by-1 vector. If `Basis` is a scalar, then it is applied to all streams, and if `Basis` is a vector, then it is the same length as the number of streams.

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `double`

Output Arguments

Return — Annualized internal rate of return of each cash flow stream

`numeric`

Annualized internal rate of return of each cash flow stream, returned as a vector. A NaN indicates that a solution was not found.

More About

IRR for Nonperiodic Cash Flows

The IRR is the discount rate at which the present value of the future cash inflows equals the initial investment (cash outflows) and in the case of nonperiodic cash flows, these inflows and outflows occur at irregular intervals.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `xirr` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Brealey and Myers. *Principles of Corporate Finance*. McGraw-Hill Higher Education, Chapter 5, 2003.
- [2] Sharpe, William F., and Gordon J. Alexander. *Investments*. Englewood Cliffs, NJ: Prentice-Hall. 4th ed., 1990.

See Also

`fvvar` | `irr` | `mirr` | `pvvar` | `datetime`

Topics

"Analyzing and Computing Cash Flows" on page 2-11

yeardays

Number of days in year

Syntax

```
Days = yeardays(Year)
Days = yeardays( ___,Basis)
```

Description

`Days = yeardays(Year)` returns the number of days in the given Year.

`Days = yeardays( ___,Basis)` returns the number of days in the given Year, based on the optional argument `Basis` for day-count.

Examples

Determine the Number of Days in a Given Year

Find the number of days in a given Year.

```
Days = yeardays(2000)
```

```
Days =
366
```

Find the number of days in a given Year using the optional argument `Basis`.

```
Days = yeardays(2000, 1)
```

```
Days =
360
```

Input Arguments

Year — Year to determine days

4 digit integer

Data Types: `single` | `double`

Basis — Day-count basis

0 (actual/actual) (default) | vector of integers with values 0,1,2,3,4,5,6,7,8,9,10,11,12,13

Day-count basis, specified as a vector of integers with values 0,1,2,3,4,5,6,7,8,9,10,11,12,13.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360

- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

Output Arguments

Days — Number of days in given Year

`nonnegative integer`

Number of days in given Year, returned as a nonnegative integer.

Version History

Introduced before R2006a

See Also

`days360` | `days365` | `daysact` | `yearfrac`

Topics

“Handle and Convert Dates” on page 2-2

yearfrac

Fraction of year between dates

Syntax

`YearFraction = yearfrac(StartDate,EndDate,Basis)`

Description

`YearFraction = yearfrac(StartDate,EndDate,Basis)` returns a fraction, in years, based on the number of days between dates `StartDate` and `EndDate` using the given day-count `Basis`.

The number of days in a year (365 or 366) is equal to the number of days in the calendar year after the `StartDate`. If `EndDate` is earlier than `StartDate`, `YearFraction` is negative.

All specified arguments must be M-by-NUMINST conforming matrices or scalar arguments.

Examples

Compute yearfrac When the Calendar Year After the StartDate is Not a Leap Year

Given a `Basis` of 0 and a `Basis` of 1, compute `yearfrac`.

Define the `StartDate` and `EndDate` using a `Basis` of 0.

```
YearFraction = yearfrac('14 mar 01', '14 sep 01', 0)
```

```
YearFraction =  
0.5041
```

Define the `StartDate` and `EndDate` using a `Basis` of 1.

```
YearFraction = yearfrac('14 mar 01', '14 sep 01', 1)
```

```
YearFraction =  
0.5000
```

Compute yearfrac When the Calendar Year After the StartDate is a Leap Year

Given a `Basis` of 0, compute `yearfrac` when the calendar after `StartDate` is in a leap year.

Define the `StartDate` and `EndDate` using a `Basis` of 0.

```
yearFraction = yearfrac('14 mar 03', '14 sep 03', 0)
```

```
yearFraction =  
0.5027
```

There are 184 days between March 14 and September 14, and the calendar year after the `StartDate` is a leap year, so `yearfrac` returns $184/366 = 0.5027$.

Compute the Fraction of a Year Using an actual/actual Basis

To get the fraction of a year between '31-Jul-2015' and '30-Sep-2015' using the actual/actual basis:

```
yearfrac('31-Jul-2015', '30-Sep-2015', 0)*2
```

```
ans =  
0.3333
```

For the actual/actual basis, the fraction of a year is calculated as:

(Actual Days between Start Date and End Date)/(Actual Days between Start Date and exactly one year after Start Date)

There are 61 days between 31-Jul-2015 and 30-Sep-2015. Since the next year is a leap year, there are 366 days between 31-Jul-2015 and 31-Jul-2016. So, there is $61/366$ which is exactly $1/6$. So given this, exactly $2/6$ is the expected result for the fraction of the six-month period.

Compute yearfrac When the Date Range Spans a Leap Year

Use a Basis of 12 to compute `yearfrac` when parts of the date range are in leap year and in a non-leap year. The output `YearFraction` is a fraction, in years, based on the number of days between the `StartDate` and `EndDate`.

```
YearFraction = yearfrac('1-Jan-2016', '30-Jan-2017', 12)
```

```
YearFraction =  
1.0795
```

Compute yearfrac When Specifying datetimes

Given a Basis of 9, compute `yearfrac` when the `StartDate` and `EndDate` are specified using datetimes.

```
yearfrac(datetime(2000,1,1), datetime(2001,1,1), 9)
```

```
ans =  
1.0167
```

Input Arguments

StartDate — Start date

datetime array | string array | date character vector

Start date, specified as a scalar or an M-by-N vector using a datetime array, string array, or date character vectors.

Note The dimensions of the `StartDate`, `EndDate`, and `Basis` vectors must be consistent.

To support existing code, `yearfrac` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

EndDate — End date

datetime array | string array | date character vector

End date, specified as a scalar or an M-by-N vector using a datetime array, string array, or date character vectors.

Note The dimensions of the `EndDate`, `StartDate`, and `Basis` vectors must be consistent.

To support existing code, `yearfrac` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Basis — Day-count basis for each set of dates

0 (actual/actual) (default) | vector of numerics with values 0 through 13

Day-count basis for each set of dates, specified as a scalar or an M-by-N matrix of integers with values of 0 through 13

Note The dimensions of the `Basis`, `StartDate`, and `EndDate` matrices must be consistent.

.

- 0 = actual/actual (default)
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (BMA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)

- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: `single` | `double`

Output Arguments

YearFraction — Real numbers identifying interval, in years, between `StartDate` and `EndDate`

vector

Real numbers identifying the interval, in years, between `StartDate` and `EndDate`, returned as a scalar or an M-by-N matrix.

More About

Difference Between `yearfrac` and `date2time`

The difference between `yearfrac` and `date2time` is that `date2time` counts full periods as a whole integer, even if the number of actual days in the periods are different. `yearfrac` does not count full periods.

For example,

```
yearfrac('1/1/2000', '1/1/2001', 9)
```

```
ans =
```

```
1.0167
```

`yearfrac` for Basis 9 (ACT/360 ICMA) calculates $366/360 = 1.0167$. So, even if the dates have the same month and date, with a difference of 1 in the year, the returned value may not be exactly 1. On the other hand, `date2time` calculates one full year period:

```
date2time('1/1/2000', '1/1/2001', 1, 9)
```

```
ans =
```

```
1
```

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `yearfrac` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");  
y = year(t)
```

```
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

See Also

[days360](#) | [date2time](#) | [days365](#) | [daysact](#) | [daysdif](#) | [wrkdydif](#) | [yeardays](#) | [datetime](#)

Topics

"Handle and Convert Dates" on page 2-2

"Trading Calendars User Interface" on page 12-2

"UICalendar User Interface" on page 12-4

ylddisc

Yield of discounted security

Syntax

```
Yield = ylddisc(Settle,Maturity,Face,Price)
Yield = ylddisc(____,Basis)
```

Description

`Yield = ylddisc(Settle,Maturity,Face,Price)` returns the yield of a discounted security.

`Yield = ylddisc(____,Basis)` adds optional an argument for Basis.

Examples

Find the Yield of a Discounted Security

This example shows how to find the yield of the following discounted security.

```
Settle = '10/14/2000';
Maturity = '03/17/2001';
Face = 100;
Price = 96.28;
Basis = 2;
```

```
Yield = ylddisc(Settle, Maturity, Face, Price, Basis)
```

```
Yield =
0.0903
```

Find the Yield of a Discounted Security Using datetime Inputs

This example shows how to use `datetime` inputs to find the yield of the following discounted security.

```
Settle = datetime(2000,10,14);
Maturity = datetime(2001,3,17);
Face = 100;
Price = 96.28;
Basis = 2;
```

```
Yield = ylddisc(Settle, Maturity, Face, Price, Basis)
```

```
Yield =
0.0903
```

Input Arguments

Settle — Settlement date of security

datetime array | string array | date character vector

Settlement date of the security, specified using a datetime array, string array, or date character vectors. The Settle date must be before the Maturity date.

To support existing code, ylddisc also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Maturity — Maturity date of security

datetime array | string array | date character vector

Maturity date of the security, specified using a datetime array, string array, or date character vectors.

To support existing code, ylddisc also accepts serial date numbers as inputs, but they are not recommended.

Data Types: char | string | datetime

Face — Redemption value of security

numeric

Redemption value (par value) of the security, specified as a scalar or vector using numeric values.

Data Types: double

Price — Discount price of security

numeric

Discount price of the security, specified as a scalar or vector using numeric values.

Data Types: double

Basis — (Optional) Day-count basis

0 (actual/actual) (default) | integers of the set [0...13] | vector of integers of the set [0...13]

Day-count basis for the security, specified as a scalar or vector using the following values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)

- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments

Yield — Yield of discounted security

numeric

Yield of discounted security, returned as a scalar or vector of numeric values.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `ylddisc` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Mayle, J. *Standard Securities Calculation Methods*. Volumes I-II, 3rd edition. Formula 1.

See Also

`acrudisc` | `bndprice` | `bndyield` | `prdisc` | `yldmat` | `yldtbill` | `datetime`

Topics

“Yield Functions” on page 2-21

“Yield Conventions” on page 2-21

yldmat

Yield with interest at maturity

Syntax

```
Yield = yldmat(Settle,Maturity,Issue,Face,Price,CouponRate)
Yield = yldmat(____,Basis)
```

Description

`Yield = yldmat(Settle,Maturity,Issue,Face,Price,CouponRate)` returns the yield of a security paying interest at maturity.

`Yield = yldmat(____,Basis)` adds an optional argument for Basis.

Examples

Find the Yield of a Security Paying Interest at Maturity

This example shows how to find the yield of a security paying interest at maturity for the following.

```
Settle = '02/07/2000';
Maturity = '04/13/2000';
Issue = '10/11/1999';
Face = 100;
Price = 99.98;
CouponRate = 0.0608;
Basis = 1;
```

```
Yield = yldmat(Settle, Maturity, Issue, Face, Price, ...
CouponRate, Basis)
```

```
Yield =
0.0607
```

Find the Yield of a Security Paying Interest at Maturity Using datetime Inputs

This example shows how to use `datetime` inputs find the yield of a security paying interest at maturity for the following:

```
Settle = datetime(2000,2,7);
Maturity = datetime(2000,4,13);
Issue = datetime(1999,10,11);
Face = 100;
Price = 99.98;
CouponRate = 0.0608;
Basis = 1;
```

```
Yield = yldmat(Settle, Maturity, Issue, Face, Price, CouponRate, Basis)
```

```
Yield =  
0.0607
```

Input Arguments

Settle — Settlement date of security

datetime array | string array | date character vector

Settlement date of the security, specified using a datetime array, string array, or date character vectors. The **Settle** date must be before the **Maturity** date.

To support existing code, `yldmat` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of security

datetime array | string array | date character vector

Maturity date of the security, specified using a datetime array, string array, or date character vectors.

To support existing code, `yldmat` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Issue — Issue date of security

datetime array | string array | date character vector

Issue date of the security, specified using a datetime array, string array, or date character vectors.

To support existing code, `yldmat` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Face — Redemption value of security

numeric

Redemption value (par value) of the security, specified as a scalar or vector using numeric values.

Data Types: `double`

Price — Price of security

numeric

Price of the security, specified as a scalar or vector using numeric values.

Data Types: `double`

CouponRate — Coupon rate of security

decimal fraction

Coupon rate of the security, specified as a scalar or a vector using decimal fractions.

Data Types: `double`

Basis — Day-count basis

0 (actual/actual) (default) | integers of the set [0 . . . 13] | vector of integers of the set [0 . . . 13]

(Optional) Day-count basis for the security, specified as a scalar or vector using the following values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments**Yield — Yield of a security paying interest at maturity**

numeric

Yield of a security paying interest at maturity, returned as a scalar or vector of numeric values.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although yldmat supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

y =

2021

There are no plans to remove support for serial date number inputs.

References

[1] Mayle, J. *Standard Securities Calculation Methods*. Volumes I-II, 3rd edition. Formula 3.

See Also

`acrudisc` | `bndprice` | `bndyield` | `prmat` | `ylddisc` | `yldtbill` | `datetime`

Topics

“Yield Functions” on page 2-21

“Yield Conventions” on page 2-21

yldtbill

Yield of Treasury bill

Syntax

```
Yield = yldtbill(Settle,Maturity,Face,Price)
```

Description

`Yield = yldtbill(Settle,Maturity,Face,Price)` returns the yield for a Treasury bill.

Examples

Find the Yield for a Treasury Bill

This example shows how to return the yield for a Treasury bill, given the settlement date of a Treasury bill is February 10, 2000, the maturity date is August 6, 2000, the par value is \$1000, and the price is \$981.36.

```
Yield = yldtbill('2/10/2000', '8/6/2000', 1000, 981.36)
```

```
Yield =  
0.0384
```

Find the Yield for a Treasury Bill Using datetime Inputs

This example shows how to use `datetime` inputs to return the yield for a Treasury bill, given the settlement date of a Treasury bill is February 10, 2000, the maturity date is August 6, 2000, the par value is \$1000, and the price is \$981.36.

```
Yield = yldtbill(datetime(2000,2,10), datetime(2000,8,6), 1000, 981.36)
```

```
Yield =  
0.0384
```

Input Arguments

Settle — Settlement date of Treasury bill

`datetime` array | `string` array | `date` character vector

Settlement date of the Treasury bill, specified using a `datetime` array, `string` array, or `date` character vectors. The `Settle` date must be before the `Maturity` date.

To support existing code, `yldtbill` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Maturity — Maturity date of Treasury bill`datetime array | string array | date character vector`

Maturity date of the Treasury bill, specified using a datetime array, string array, or date character vectors.

To support existing code, `yldtbill` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

Face — Redemption value of Treasury bill`numeric`

Redemption value (par value) of the Treasury bill, specified as a scalar or vector using numeric values.

Data Types: `double`

Price — Price of Treasury bill`numeric`

Price of the Treasury bill, specified as a scalar or vector using numeric values.

Data Types: `double`

Output Arguments

Yield — Yield for Treasury bill`numeric`

Yield for Treasury bill, returned as a scalar or vector of numeric values.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `yldtbill` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

References

[1] Bodie, Kane, and Marcus. *Investments*. McGraw-Hill Education, 2013.

See Also

beytbill | bndyield | prtbill | yldmat | datetime

Topics

“Computing Treasury Bill Price and Yield” on page 2-26

“Yield Functions” on page 2-21

“Treasury Bills Defined” on page 2-25

“Yield Conventions” on page 2-21

zbtprice

Zero curve bootstrapping from coupon bond data given price

Syntax

```
[ZeroRates, CurveDates] = zbtprice(Bonds, Prices, Settle)
ZeroRates, CurveDates = zbtprice( ___, OutputCompounding)
```

Description

`[ZeroRates, CurveDates] = zbtprice(Bonds, Prices, Settle)` uses the bootstrap method to return a zero curve given a portfolio of coupon bonds and their prices.

A zero curve consists of the yields to maturity for a portfolio of theoretical zero-coupon bonds that are derived from the input `Bonds` portfolio. The bootstrap method that this function uses does *not* require alignment among the cash-flow dates of the bonds in the input portfolio. It uses theoretical par bond arbitrage and yield interpolation to derive all zero rates; specifically, the interest rates for cash flows are determined using linear interpolation. For best results, use a portfolio of at least 30 bonds evenly spaced across the investment horizon.

`ZeroRates, CurveDates = zbtprice( ___, OutputCompounding)` adds an optional argument for `OutputCompounding`.

Examples

Compute a Zero Curve Given a Portfolio of Coupon Bonds and Their Prices Using `datetime` Inputs

Given data and prices for 12 coupon bonds, two with the same maturity date, and given the common settlement date, use `datetime` inputs to compute a zero curve.

```
Bonds = [datetime('6/1/1998') 0.0475 100 2 0 0;
          datetime('7/1/2000') 0.06 100 2 0 0;
          datetime('7/1/2000') 0.09375 100 6 1 0;
          datetime('6/30/2001') 0.05125 100 1 3 1;
          datetime('4/15/2002') 0.07125 100 4 1 0;
          datetime('1/15/2000') 0.065 100 2 0 0;
          datetime('9/1/1999') 0.08 100 3 3 0;
          datetime('4/30/2001') 0.05875 100 2 0 0;
          datetime('11/15/1999') 0.07125 100 2 0 0;
          datetime('6/30/2000') 0.07 100 2 3 1;
          datetime('7/1/2001') 0.0525 100 2 3 0;
          datetime('4/30/2002') 0.07 100 2 0 0];

Prices = [99.375;
          99.875;
          105.75 ;
          96.875;
          103.625;
          101.125;
```

```

    103.125;
    99.375;
    101.0  ;
    101.25 ;
    96.375;
    102.75 ];

Settle = datetime(1997,12,18);
OutputCompounding = 2;

t=array2table(Bonds);
t.Bonds1 = datetime(t.Bonds1,'ConvertFrom','datenum','Locale','en_US');
[ZeroRates, CurveDates] = zbtprice(t, Prices, Settle, OutputCompounding)

ZeroRates = 11x1

    0.0616
    0.0609
    0.0658
    0.0590
    0.0647
    0.0655
    0.0606
    0.0601
    0.0642
    0.0621
    0.0627
    :

CurveDates = 11x1 datetime
    01-Jun-1998
    01-Sep-1999
    15-Nov-1999
    15-Jan-2000
    30-Jun-2000
    01-Jul-2000
    30-Apr-2001
    30-Jun-2001
    01-Jul-2001
    15-Apr-2002
    30-Apr-2002

```

Input Arguments

Bonds — Coupon bond information to generate zero curve

table | matrix

Coupon bond information to generate zero curve, specified as a 6-column table or a n-by-2 to n-by-6 matrix of bond information, where the table columns or matrix columns contains:

- **Maturity** (Column 1, Required) Maturity date of the bond, specified as a datetime, string, or date character vector.

To support existing code, `zbtprice` also accepts serial date numbers as inputs, but they are not recommended.

If the input **Bonds** is a table, the **Maturity** dates can be a datetime array, string array, or date character vectors.

- **CouponRate** (Column 2, Required) Decimal fraction indicating the coupon rate of the bond.
- **Face** (Column 3, Optional) Redemption or face value of the bond. Default = 100.
- **Period** (Column 4, Optional) Coupons per year of the bond. Allowed values are 0, 1, 2 (default), 3, 4, 6, and 12.
- **Basis** (Column 5, Optional) Day-count basis of the bond. A vector of integers.
 - 0 = actual/actual (default)
 - 1 = 30/360 (SIA)
 - 2 = actual/360
 - 3 = actual/365
 - 4 = 30/360 (BMA)
 - 5 = 30/360 (ISDA)
 - 6 = 30/360 (European)
 - 7 = actual/365 (Japanese)
 - 8 = actual/actual (ICMA)
 - 9 = actual/360 (ICMA)
 - 10 = actual/365 (ICMA)
 - 11 = 30/360E (ICMA)
 - 12 = actual/365 (ISDA)
 - 13 = BUS/252
 - For more information, see “Basis” on page 2-16.
- **EndMonthRule** (Column 6, Optional) End-of-month rule. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days. 0 = ignore rule, meaning that a bond's coupon payment date is always the same numerical day of the month. 1 = set rule on (default), meaning that a bond's coupon payment date is always the last actual day of the month

:

Note

- If **Bonds** is a table, the **Maturity** dates can be a datetime array, string array, or date character vectors.
 - If **Bonds** is a matrix, is an n-by-2 to n-by-6 matrix where each row describes a bond, the first two columns (**Maturity** and **CouponRate**) are required. The remainder of the columns are optional but must be added in order. All rows in **Bonds** must have the same number of columns.
-

Data Types: double | char | string | datetime | table

Prices — Clean price (price without accrued interest) of each bond in Bonds
numeric

Clean price (price without accrued interest) of each bond in **Bonds**, specified as a N-by-1 column vector. The number of rows (*n*) must match the number of rows in **Bonds**.

Data Types: `double`

Settle — Settlement date representing time zero in derivation of zero curve

`datetime scalar` | `string scalar` | `date character vector`

Settlement date representing time zero in derivation of zero curve, specified as scalar `datetime`, `string`, or `date character vector`. **Settle** represents time zero for deriving the zero curve, and it is normally the common settlement date for all the bonds.

To support existing code, **zbtprice** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `char` | `string` | `datetime`

OutputCompounding — Compounding frequency of output ZeroRates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

(Optional) Compounding frequency of output **ZeroRates**, specified using the allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- -1 — Continuous compounding

Data Types: `double`

Output Arguments

ZeroRates — Implied zero rates for each point along the investment horizon defined by maturity date

`vector of decimal fractions`

Implied zero rates for each point along the investment horizon defined by a maturity date, returned as a *m*-by-1 vector of decimal fractions where *m* is the number of bonds of unique maturity dates. In aggregate, the rates in **ZeroRates** constitute a zero curve.

If more than one bond has the same **Maturity** date, **zbtprice** returns the mean zero rate for that **Maturity**. Any rates before the first **Maturity** are assumed to be equal to the rate at the first **Maturity**, that is, the curve is assumed to be flat before the first **Maturity**.

CurveDates — Maturity dates that correspond to ZeroRates

`datetime` | `serial date number`

Maturity dates that correspond to the **ZeroRates**, returned as a *m*-by-1 vector of unique maturity dates, where *m* is the number of bonds of different maturity dates. These dates begin with the earliest **Maturity** date and end with the latest **Maturitydate** in the **Bonds** table or matrix.

If either inputs for `Bonds` or `Settle` have datetime values, then `CurveDates` is datetimes. Otherwise `CurveDates` is serial date numbers. Use the function `datetime` to convert serial date numbers to formatted date character vectors.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `zbtprice` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)  
  
y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

References

- [1] Fabozzi, Frank J. "The Structure of Interest Rates." Ch. 6 in Fabozzi, Frank J. and T. Dossa Fabozzi, eds. *The Handbook of Fixed Income Securities*. 4th ed. New York, Irwin Professional Publishing, 1995.
- [2] McEnally, Richard W. and James V. Jordan. "The Term Structure of Interest Rates." in Ch. 37 in Fabozzi and Fabozzi, *ibid*
- [3] Das, Satyajit. "Calculating Zero Coupon Rates." in *Swap and Derivative Financing*. Appendix to Ch. 8, pp. 219–225. New York, Irwin Professional Publishing, 1994.

See Also

`zbtyield` | `datetime`

Topics

"Term Structure of Interest Rates" on page 2-29
"Fixed-Income Terminology" on page 2-15

zbtyield

Zero curve bootstrapping from coupon bond data given yield

Syntax

```
[ZeroRates, CurveDates] = zbtyield(Bonds, YieldsSettle)
ZeroRates, CurveDates = zbtyield( ___, OutputCompounding)
```

Description

[ZeroRates, CurveDates] = zbtyield(Bonds, YieldsSettle) uses the bootstrap method to return a zero curve given a portfolio of coupon bonds and their yields.

A zero curve consists of the yields to maturity for a portfolio of theoretical zero-coupon bonds that are derived from the input **Bonds** portfolio. The bootstrap method that this function uses does *not* require alignment among the cash-flow dates of the bonds in the input portfolio. It uses theoretical par bond arbitrage and yield interpolation to derive all zero rates; specifically, the interest rates for cash flows are determined using linear interpolation. For best results, use a portfolio of at least 30 bonds evenly spaced across the investment horizon.

ZeroRates, CurveDates = zbtyield(___, OutputCompounding) adds an optional argument for OutputCompounding.

Examples

Compute a Zero Curve Given a Portfolio of Coupon Bonds and Their Yields Using datetime Inputs

Given data and yields to maturity for 12 coupon bonds (two with the same maturity date), and given the common settlement date, compute the zero curve using datetime inputs.

```
Bonds = [datetime('6/1/1998')    0.0475    100    2    0    0;
          datetime('7/1/2000')    0.06       100    2    0    0;
          datetime('7/1/2000')    0.09375   100    6    1    0;
          datetime('6/30/2001')    0.05125   100    1    3    1;
          datetime('4/15/2002')    0.07125   100    4    1    0;
          datetime('1/15/2000')    0.065     100    2    0    0;
          datetime('9/1/1999')     0.08       100    3    3    0;
          datetime('4/30/2001')    0.05875   100    2    0    0;
          datetime('11/15/1999')   0.07125   100    2    0    0;
          datetime('6/30/2000')    0.07       100    2    3    1;
          datetime('7/1/2001')     0.0525    100    2    3    0;
          datetime('4/30/2002')    0.07       100    2    0    0];

Yields = [0.0616
          0.0605
          0.0687
          0.0612
          0.0615
          0.0591]
```

```

0.0603
0.0608
0.0655
0.0646
0.0641
0.0627];

```

```
Settle = datetime(1997,12,18);
```

```
OutputCompounding = 2;
```

```
t = array2table(Bonds, 'VariableNames', {'Maturity', 'CouponRate', 'Face', 'Period', 'Basis', 'EndMonthRule'});
```

Maturity	CouponRate	Face	Period	Basis	EndMonthRule
7.2991e+05	0.0475	100	2	0	0
7.3067e+05	0.06	100	2	0	0
7.3067e+05	0.09375	100	6	1	0
7.3103e+05	0.05125	100	1	3	1
7.3132e+05	0.07125	100	4	1	0
7.305e+05	0.065	100	2	0	0
7.3036e+05	0.08	100	3	3	0
7.3097e+05	0.05875	100	2	0	0
7.3044e+05	0.07125	100	2	0	0
7.3067e+05	0.07	100	2	3	1
7.3103e+05	0.0525	100	2	3	0
7.3134e+05	0.07	100	2	0	0

```
t.Maturity = datetime(t.Maturity, 'ConvertFrom', 'datetime', 'Locale', 'en_US');
```

```
[ZeroRates, CurveDates] = zbtyield(t, Yields, Settle, OutputCompounding)
```

```
ZeroRates = 11x1
```

```

0.0616
0.0603
0.0657
0.0590
0.0649
0.0650
0.0606
0.0611
0.0643
0.0614
0.0627
:

```

```
CurveDates = 11x1 datetime
```

```

01-Jun-1998
01-Sep-1999
15-Nov-1999
15-Jan-2000
30-Jun-2000
01-Jul-2000
30-Apr-2001
30-Jun-2001
01-Jul-2001
15-Apr-2002

```

30-Apr-2002

Compute the Real Zero Rates From the Real Yields of Inflation-Linked Bonds

Use `zbtyield` to compute the real zero rates from the real yields of inflation-linked bonds.

```
% Load the data
load usbond_02Sep2008
Settle = datetime(2008,9,2);
```

Compute the real yields and then compute the real zero rates.

```
RealYields = bndyield(TIPSPPrice,TIPSCoupon,Settle,TIPSMaturity);
TIPSBonds = [TIPSMaturity TIPSCoupon];
[RealZeroRates, CurveDates] = zbtyield(TIPSBonds, RealYields, Settle)
```

```
RealZeroRates = 26×1
```

```
0.0069
0.0094
0.0092
0.0111
0.0110
0.0119
0.0116
0.0128
0.0126
0.0136
0.0137
0.0144
0.0150
0.0158
0.0161
⋮
```

```
CurveDates = 26×1 datetime
```

```
15-Jan-2010
15-Apr-2010
15-Jan-2011
15-Apr-2011
15-Jan-2012
15-Apr-2012
15-Jul-2012
15-Apr-2013
15-Jul-2013
15-Jan-2014
15-Jul-2014
15-Jan-2015
15-Jul-2015
15-Jan-2016
15-Jul-2016
15-Jan-2017
15-Jul-2017
15-Jan-2018
```

```
15-Jul-2018
15-Jan-2025
15-Jan-2026
15-Jan-2027
15-Jan-2028
15-Apr-2028
15-Apr-2029
15-Apr-2032
```

Input Arguments

Bonds — Coupon bond information to generate zero curve

table | matrix

Coupon bond information to generate zero curve, specified as a 6-column table or a n-by-2 to n-by-6 matrix of bond information, where the table columns or matrix columns contains:

- **Maturity** (Column 1, Required) Maturity date of the bond, specified as a datetime, string, or date character vector.

To support existing code, `zbtyield` also accepts serial date numbers as inputs, but they are not recommended.

If the input **Bonds** is a table, the **Maturity** dates can be a datetime array, string array, or date character vectors.

- **CouponRate** (Column 2, Required) Decimal fraction indicating the coupon rate of the bond.
- **Face** (Column 3, Optional) Redemption or face value of the bond. Default = 100.
- **Period** (Column 4, Optional) Coupons per year of the bond. Allowed values are 0, 1, 2 (default), 3, 4, 6, and 12.
- **Basis** (Column 5, Optional) Day-count basis of the bond. A vector of integers.
 - 0 = actual/actual (default)
 - 1 = 30/360 (SIA)
 - 2 = actual/360
 - 3 = actual/365
 - 4 = 30/360 (BMA)
 - 5 = 30/360 (ISDA)
 - 6 = 30/360 (European)
 - 7 = actual/365 (Japanese)
 - 8 = actual/actual (ICMA)
 - 9 = actual/360 (ICMA)
 - 10 = actual/365 (ICMA)
 - 11 = 30/360E (ICMA)
 - 12 = actual/365 (ISDA)
 - 13 = BUS/252

- For more information, see “Basis” on page 2-16.
 - **EndMonthRule** (Column 6, Optional) End-of-month rule. This rule applies only when **Maturity** is an end-of-month date for a month having 30 or fewer days. 0 = ignore rule, meaning that a bond's coupon payment date is always the same numerical day of the month. 1 = set rule on (default), meaning that a bond's coupon payment date is always the last actual day of the month
- :

Note

- If **Bonds** is a table, the **Maturity** dates can be a datetime array, string array, or date character vectors.
 - If **Bonds** is a matrix, is an n-by-2 to n-by-6 matrix where each row describes a bond, the first two columns (**Maturity** and **CouponRate**) are required. The remainder of the columns are optional but must be added in order. All rows in **Bonds** must have the same number of columns.
-

Data Types: double | char | string | datetime | table

Yields — Yield to maturity of each bond in Bonds

numeric

Yield to maturity of each bond in **Bonds**, specified as a N-by-1 column vector. The number of rows (*n*) must match the number of rows in **Bonds**.

Note Yield to maturity must be compounded semiannually.

Data Types: double

Settle — Settlement date representing time zero in derivation of zero curve

datetime scalar | string scalar | date character vector

Settlement date representing time zero in derivation of zero curve, specified as a scalar datetime, string, or date character vector. **Settle** represents time zero for deriving the zero curve, and it is normally the common settlement date for all the bonds.

To support existing code, **zbtyield** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: string | char | datetime

OutputCompounding — Compounding frequency of output ZeroRates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

(Optional) Compounding frequency of output **ZeroRates**, specified using the allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)

- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- -1 — Continuous compounding

Data Types: double

Output Arguments

ZeroRates — Implied zero rates for each point along the investment horizon defined by maturity date

decimal fractions

Implied zero rates for each point along the investment horizon defined by a maturity date, returned as a m-by-1 vector of decimal fractions where m is the number of bonds with unique maturity dates. In aggregate, the rates in ZeroRates constitute a zero curve.

If more than one bond has the same Maturity date, zbtyield returns the mean zero rate for that Maturity. Any rates before the first Maturity are assumed to be equal to the rate at the first Maturity, that is, the curve is assumed to be flat before the first Maturity.

CurveDates — Maturity dates that correspond to ZeroRates

datetime | serial date number

Maturity dates that correspond to the ZeroRates, returned as a m-by-1 vector of unique maturity dates, where m is the number of bonds of different maturity dates. These dates begin with the earliest Maturity date and end with the latest Maturitydate in the Bonds table or matrix.

If either inputs for Bonds or Settle have datetime values, then CurveDatesCurveDates is datetimes. Otherwise CurveDates is serial date numbers. Use the function datetime to convert serial date numbers to formatted date character vectors.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although zbtyield supports serial date numbers, datetime values are recommended instead. The datetime data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to datetime values, use the datetime function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)
```

```
y =
```

```
2021
```


There are no plans to remove support for serial date number inputs.

References

- [1] Fabozzi, Frank J. "The Structure of Interest Rates." Ch. 6 in Fabozzi, Frank J. and T. Dossa Fabozzi, eds. *The Handbook of Fixed Income Securities*. 4th ed. New York, Irwin Professional Publishing, 1995.
- [2] McEnally, Richard W. and James V. Jordan. "The Term Structure of Interest Rates." in Ch. 37 in Fabozzi and Fabozzi, *ibid*
- [3] Das, Satyajit. "Calculating Zero Coupon Rates." in *Swap and Derivative Financing*. Appendix to Ch. 8, pp. 219-225. New York, Irwin Professional Publishing, 1994.

See Also

zbtprice | datetime

Topics

"Term Structure of Interest Rates" on page 2-29
"Fixed-Income Terminology" on page 2-15

zero2disc

Discount curve given zero curve

Syntax

```
[DiscRates, CurveDates] = zero2disc(ZeroRates, CurveDates, Settle)
[DiscRates, CurveDates] = zero2disc( ___, Name, Value)
```

Description

`[DiscRates, CurveDates] = zero2disc(ZeroRates, CurveDates, Settle)` returns a discount curve given a zero curve and its maturity dates. If either inputs for `ZeroRates` or `CurveDates` are a datetime array, `CurveDates` is returned as a datetime array. Otherwise, `CurveDates` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. The `DiscRates` output is the same for any of these input data types.

`[DiscRates, CurveDates] = zero2disc( ___, Name, Value)` adds optional name-value pair arguments

Examples

Compute a Discount Curve Given a Zero Curve and Maturity Dates Using datetime Inputs

Given a zero curve over a set of maturity dates and a settlement date, compute a discount curve using datetime inputs.

```
ZeroRates = [0.0464
             0.0509
             0.0524
             0.0525
             0.0531
             0.0525
             0.0530
             0.0531
             0.0549
             0.0536];
```

```
CurveDates = [datetime(2000,11,6)
              datetime(2000,12,11)
              datetime(2001,1,15)
              datetime(2001,4,2)
              datetime(2001,4,30)
              datetime(2001,6,25)
              datetime(2001,9,4)
              datetime(2001,11,12)
              datetime(2001,2,5)
              datetime(2001,3,4)];
```

```
% Note, the input CurveDates do not need to be in chronological order, ZERO2DISC automatically sorts
% the output CurveDates by the curve dates.
```

```

Settle = datetime(2000,11,3);
Compounding = 365;
Basis = 3;

[DiscRates, CurveDates] = zero2disc(ZeroRates, CurveDates, ...
Settle, Compounding, Basis)

DiscRates = 10×1

    0.9996
    0.9947
    0.9896
    0.9860
    0.9824
    0.9787
    0.9744
    0.9669
    0.9567
    0.9470

CurveDates = 10×1 datetime
    06-Nov-2000
    11-Dec-2000
    15-Jan-2001
    05-Feb-2001
    04-Mar-2001
    02-Apr-2001
    30-Apr-2001
    25-Jun-2001
    04-Sep-2001
    12-Nov-2001

```

Input Arguments

ZeroRates — Annualized zero rates

decimal fraction

Annualized zero rates, specified as a NUMBONDS-by-1 vector using decimal fractions. In aggregate, the zero rates constitute an implied zero curve for the investment horizon represented by CurveDates.

Data Types: double

CurveDates — Maturity dates

datetime array | string array | date character vector

Maturity dates that correspond to the input ZeroRates, specified as NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

Note The input CurveDates does not need to be in chronological order, zero2disc automatically sorts the output CurveDates by the curve dates.

To support existing code, `zero2disc` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `datetime` | `string` | `char`

Settle — Common settlement date for ZeroRates

`datetime` scalar | `string` scalar | `date` character vector

Common settlement date for `ZeroRates`, specified as a scalar `datetime`, `string`, or `date` character vector. `Settle` is the settlement date for the bonds from which the zero curve was bootstrapped.

To support existing code, `zero2disc` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `datetime` | `string` | `char`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[DiscRates, CurveDates] = zero2disc(ZeroRates, CurveDates, Settle, 'Compounding', 4, 'Basis', 6)`

Compounding — Rate at which input ZeroRates zero rates are compounded when annualized

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Rate at which the input `ZeroRates` are compounded when annualized, specified as the comma-separated pair consisting of `'Compounding'` and allowed numeric values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Data Types: `double`

Basis — Day-count basis used for annualizing input ZeroRates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day-count basis used for annualizing the input `ZeroRates`, specified as the comma-separated pair consisting of `'Basis'` and allowed numeric values:

- 0 = actual/actual

- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Data Types: double

Output Arguments

DiscRates — Discount factors

decimal

Discount factors, returned as a `NUMBONDS`-by-1 vector of decimal fractions. In aggregate, the discount factors constitute a discount curve for the investment horizon represented by `CurveDates`.

CurveDates — Maturity dates that correspond to DiscRates

serial date number | datetime

Maturity dates that correspond to the `DiscRates`, returned as a `NUMBONDS`-by-1 vector of maturity dates that correspond to the discount factors. This vector is the same as the input vector `CurveDates`, but is sorted by ascending maturity.

If either inputs for `CurveDates` or `Settle` are a datetime array, `CurveDates` is returned as a datetime array. Otherwise, `CurveDates` is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `zero2disc` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datetime");  
y = year(t)  
  
y =  
  
2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the optional name-value pair inputs: `Compounding` and `Basis`.

See Also

`disc2zero` | `datetime` | `fwd2zero` | `prbyzero` | `pyld2zero` | `zbtprice` | `zbtyield` | `zero2fwd` | `zero2pyld`

Topics

“Term Structure of Interest Rates” on page 2-29

“Fixed-Income Terminology” on page 2-15

zero2fwd

Forward curve given zero curve

Syntax

```
[ForwardRates, CurveDates] = zero2fwd(ZeroRates, CurveDates, Settle)
[ForwardRates, CurveDates] = zero2fwd( ___, Name, Value)
```

Description

[ForwardRates, CurveDates] = zero2fwd(ZeroRates, CurveDates, Settle) returns an implied forward rate curve given a zero curve and its maturity dates. If either input for CurveDates or Settle is a datetime array, CurveDates is returned as a datetime array. Otherwise, CurveDates is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. ForwardRates is the same for any of these input data types.

[ForwardRates, CurveDates] = zero2fwd(___, Name, Value) adds optional name-value pair arguments

Examples

Compute an Implied Forward Rate Curve Given a Zero Curve and Maturity Dates Using datetime Inputs

Given a zero curve over a set of maturity dates, a settlement date, and a compounding rate, use `datetime` compute the forward rate curve.

```
ZeroRates = [0.0458
             0.0502
             0.0518
             0.0519
             0.0524
             0.0519
             0.0523
             0.0525
             0.0541
             0.0529];

CurveDates = [datetime(2000,11,6)
              datetime(2000,12,11)
              datetime(2001,1,15)
              datetime(2001,2,5)
              datetime(2001,3,4)
              datetime(2001,4,2)
              datetime(2001,4,30)
              datetime(2001,6,25)
              datetime(2001,9,4)
              datetime(2001,11,12)];
```

```
Settle = datetime(2000,11,3);
InputCompounding = 1;
InputBasis = 2;
OutputCompounding = 1;
OutputBasis = 2;

[ForwardRates, CurveDates] = zero2fwd(ZeroRates, CurveDates,...
Settle,'InputCompounding',1,'InputBasis',2,'OutputCompounding',1,'OutputBasis',2)

ForwardRates = 10×1

    0.0458
    0.0506
    0.0535
    0.0522
    0.0541
    0.0498
    0.0544
    0.0531
    0.0594
    0.0476

CurveDates = 10×1 datetime
    06-Nov-2000
    11-Dec-2000
    15-Jan-2001
    05-Feb-2001
    04-Mar-2001
    02-Apr-2001
    30-Apr-2001
    25-Jun-2001
    04-Sep-2001
    12-Nov-2001
```

Input Arguments

ZeroRates — Annualized zero rates

decimal fraction

Annualized zero rates, specified as a **NUMBONDS**-by-1 vector using decimal fractions. In aggregate, the rates constitute an implied zero curve for the investment horizon represented by **CurveDates**. The first element pertains to forward rates from the settlement date to the first curve date.

Data Types: double

CurveDates — Maturity dates

datetime array | string array | date character vector

Maturity dates, specified as a **NUMBONDS**-by-1 vector using a datetime array, string array, or date character vectors that correspond to the **ZeroRates**.

To support existing code, **zero2fwd** also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Settle — Common settlement date for ZeroRates

datetime scalar | string scalar | date character vector

Common settlement date for input ZeroRates, specified as scalar datetime, string, or date character vector.

To support existing code, zero2fwd also accepts serial date numbers as inputs, but they are not recommended.

Data Types: datetime | string | char

Name-Value Pair Arguments

Specify optional pairs of arguments as Name1=Value1,...,NameN=ValueN, where Name is the argument name and Value is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: [ForwardRates, CurveDates] =
zero2fwd(ZeroRates, CurveDates, Settle, 'InputCompounding', 3, 'InputBasis', 5, 'OutputCompounding', 4, 'OutputBasis', 5)

InputCompounding — Compounding frequency of input zero rates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of input zero rates, specified as the comma-separated pair consisting of 'InputCompounding' and allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Note If InputCompounding is not specified, then InputCompounding is assigned the value specified for OutputCompounding. If either InputCompounding or OutputCompounding are not specified, the default is 2 (semiannual) for both.

Data Types: double

InputBasis — Day-count basis of input zero rates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of input zero rates, specified as the comma-separated pair consisting of 'InputBasis' and allowed values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If `InputBasis` is not specified, then `InputBasis` is assigned the value specified for `OutputBasis`. If either `InputBasis` or `OutputBasis` are not specified, the default is 0 (actual/actual) for both.

Data Types: double

OutputCompounding — Compounding frequency of output forward rates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of output forward rates, specified as the comma-separated pair consisting of 'OutputCompounding' and allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Note If `OutputCompounding` is not specified, then `OutputCompounding` is assigned the value specified for `InputCompounding`. If either `InputCompounding` or `OutputCompounding` are not specified, the default is 2 (semiannual) for both.

Data Types: double

OutputBasis — Day-count basis of output forward rates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of output forward rates, specified as the comma-separated pair consisting of 'OutputBasis' and allowed values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If OutputBasis is not specified, then OutputBasis is assigned the value specified for InputBasis. If either InputBasis or OutputBasis are not specified, the default is 0 (actual/actual) for both.

Data Types: double

Output Arguments**ForwardRates — Forward curve for investment horizon represented by CurveDates**

numeric

Forward curve for the investment horizon represented by CurveDates, returned as a NUMBONDS-by-1 vector of decimal fractions. In aggregate, the rates in ForwardRates constitute a forward curve over the dates in CurveDates. ForwardRates are ordered by ascending maturity.

CurveDates — Maturity dates that correspond to ForwardRates

datetime | serial date number

Maturity dates that correspond to the ForwardRates, returned as a NUMBONDS-by-1 vector of maturity dates that correspond to the ForwardRates.

ForwardRates are expressed as serial date numbers (default) or datetimes (if CurveDates or Settle are datetime arrays), representing the maturity dates for each rate in ForwardRates. These dates are the same dates as those associated with the input ZeroRates, but are ordered by ascending maturity.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although `zero2fwd` supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");  
y = year(t)
```

```
y =
```

```
    2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the new optional name-value pair inputs: `InputCompounding`, `InputBasis`, `OutputCompounding`, and `OutputBasis`.

See Also

`fwd2zero` | `getForwardRates` | `datetime` | `disc2zero` | `zero2disc` | `zero2pyld` | `pyld2zero` | `zbtprice` | `zbtyield`

Topics

"Term Structure of Interest Rates" on page 2-29

"Fixed-Income Terminology" on page 2-15

zero2pyld

Par yield curve given zero curve

Syntax

```
[ParRates, CurveDates] = zero2pyld(ZeroRates, CurveDates, Settle)
[ParRates, CurveDates] = zero2pyld( ___, Name, Value)
```

Description

[ParRates, CurveDates] = zero2pyld(ZeroRates, CurveDates, Settle) returns a par yield curve given a zero curve and its maturity dates. If either input for CurveDates or Settle is a datetime array, CurveDates is returned as a datetime array. Otherwise, CurveDates is returned as a serial date number. Use the function `datetime` to convert serial date numbers to formatted datetime arrays. ParRates is the same for any of these input data types.

[ParRates, CurveDates] = zero2pyld(___, Name, Value) adds optional name-value pair arguments

Examples

Compute Par Yield Curve Given a Zero Curve and Maturity Dates Using datetime Inputs

Given a zero curve over a set of maturity dates, a settlement date, and annual compounding for the input zero curve and monthly compounding for the output par rates, use `datetime` inputs to compute a par yield curve.

```
ZeroRates = [0.0457
0.0487
0.0506
0.0507
0.0505
0.0504
0.0506
0.0516
0.0539
0.0530];
CurveDates = [datetime(2000,11,6)
datetime(2000,12,11)
datetime(2001,1,15)
datetime(2001,2,5)
datetime(2001,3,4)
datetime(2001,4,2)
datetime(2001,4,30)
datetime(2001,6,25)
datetime(2001,9,4)
datetime(2001,11,12)];
```

```
Settle = datetime(2000,11,3);
InputCompounding = 12;
```

```
InputBasis = 2;
OutputCompounding = 1;
OutputBasis = 2;

[ParRates, CurveDates] = zero2pyld(ZeroRates, CurveDates,...
Settle, 'InputCompounding',12,'InputBasis',2,'OutputCompounding',1,'OutputBasis',2)

ParRates = 10×1

-0.0436
0.0611
0.0579
0.0567
0.0550
0.0543
0.0541
0.0546
0.0565
0.0561

CurveDates = 10×1 datetime
06-Nov-2000
11-Dec-2000
15-Jan-2001
05-Feb-2001
04-Mar-2001
02-Apr-2001
30-Apr-2001
25-Jun-2001
04-Sep-2001
12-Nov-2001
```

Demonstrate a Roundtrip From zero2pyld to pyld2zero

Given the following zero curve and its maturity dates, return the ParRates.

```
Settle = datetime(2013,2,1);

CurveDates = [datetime(2014,2,1)
datetime(2015,2,1)
datetime(2016,2,1)
datetime(2018,2,1)
datetime(2020,2,1)
datetime(2023,2,1)
datetime(2033,2,1)
datetime(2043,2,1) ];

OriginalZeroRates = [.11 0.30 0.64 1.44 2.07 2.61 3.29 3.55]'/100;

OutputCompounding = 1;
OutputBasis = 0;
InputCompounding = 1;
InputBasis = 0;
```

```
ParRates = zero2pyld(OriginalZeroRates, CurveDates, Settle, ...
    'OutputCompounding', OutputCompounding, 'OutputBasis', OutputBasis, ...
    'InputCompounding', InputCompounding, 'InputBasis', InputBasis)
```

```
ParRates = 8×1
```

```
0.0011
0.0030
0.0064
0.0142
0.0202
0.0251
0.0310
0.0331
```

For the ParRates, use the pyld2zero function to return the ZeroRatesOut and determine the roundtrip error.

```
ZeroRatesOut = pyld2zero(ParRates, CurveDates, Settle, ...
    'OutputCompounding', OutputCompounding, 'OutputBasis', OutputBasis, ...
    'InputCompounding', InputCompounding, 'InputBasis', InputBasis)
```

```
ZeroRatesOut = 8×1
```

```
0.0011
0.0030
0.0064
0.0144
0.0207
0.0261
0.0329
0.0355
```

```
max(abs(OriginalZeroRates - ZeroRatesOut)) % Roundtrip error
```

```
ans =
1.4919e-16
```

Input Arguments

ZeroRates — Annualized zero rates

decimal fraction

Annualized zero rates, specified as a NUMBONDS-by-1 vector using decimal fractions. In aggregate, the rates constitute an implied zero curve for the investment horizon represented by CurveDates.

Data Types: double

CurveDates — Maturity dates

datetime array | string array | date character vector

Maturity dates which correspond to the input ZeroRates, specified as a NUMBONDS-by-1 vector using a datetime array, string array, or date character vectors.

To support existing code, `zero2pyld` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `datetime` | `string` | `char`

Settle — Common settlement date for ZeroRates

`datetime` scalar | `string` scalar | `date` character vector

Common settlement date for input `ZeroRates`, specified as a scalar `datetime`, `string`, or `date` character vector.

To support existing code, `zero2pyld` also accepts serial date numbers as inputs, but they are not recommended.

Data Types: `datetime` | `string` | `char`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[ParRates, CurveDates] = zero2pyld(ZeroRates, CurveDates, Settle, 'OutputCompounding', 3, 'OutputBasis', 5, 'InputCompounding', 4, 'InputBasis', 5)`

OutputCompounding — Compounding frequency of output ParRates

2 (default) | numeric values: 0, 1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of output `ParRates`, specified as the comma-separated pair consisting of `'OutputCompounding'` and allowed values:

- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding

Note

- If `InputCompounding` is 1, 2, 3, 4, 6, or 12 and `OutputCompounding` is not specified, the value of `InputCompounding` is used.
 - If `InputCompounding` is 0 (simple), -1 (continuous), or 365 (daily), a valid `OutputCompounding` value must also be specified.
 - If either `InputCompounding` or `OutputCompounding` are not specified, the default is 2 (semiannual) for both.
-

Data Types: `double`

OutputBasis — Day-count basis of output ParRates

0 (default) | numeric values: 0,1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of output ParRates, specified as the comma-separated pair consisting of 'OutputBasis' and allowed values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If OutputBasis is not specified, then OutputBasis is assigned the value specified for InputBasis. If either InputBasis or OutputBasis are not specified, the default is 0 (actual/actual) for both.

Data Types: double

InputCompounding — Compounding frequency of input ZeroRates

2 (default) | numeric values: 0,1, 2, 3, 4, 6, 12, 365, -1

Compounding frequency of input ZeroRates, specified as the comma-separated pair consisting of 'InputCompounding' and allowed values:

- 0 — Simple interest (no compounding)
- 1 — Annual compounding
- 2 — Semiannual compounding (default)
- 3 — Compounding three times per year
- 4 — Quarterly compounding
- 6 — Bimonthly compounding
- 12 — Monthly compounding
- 365 — Daily compounding
- -1 — Continuous compounding

Note

- If `InputCompounding` is set to 0 (simple), -1 (continuous), or 365 (daily), the `OutputCompounding` must also be specified using a valid value.
 - If `InputCompounding` is not specified, then `InputCompounding` is assigned the value specified for `OutputCompounding`.
 - If either `InputCompounding` or `OutputCompounding` are not specified, the default is 2 (semiannual) for both.
-

Data Types: double

InputBasis — Day-count basis of input ZeroRates

0 (default) | numeric values: 0, 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13

Day count basis of the input `ZeroRates`, specified as the comma-separated pair consisting of 'InputBasis' and allowed values:

- 0 = actual/actual
- 1 = 30/360 (SIA)
- 2 = actual/360
- 3 = actual/365
- 4 = 30/360 (PSA)
- 5 = 30/360 (ISDA)
- 6 = 30/360 (European)
- 7 = actual/365 (Japanese)
- 8 = actual/actual (ICMA)
- 9 = actual/360 (ICMA)
- 10 = actual/365 (ICMA)
- 11 = 30/360E (ICMA)
- 12 = actual/365 (ISDA)
- 13 = BUS/252

For more information, see “Basis” on page 2-16.

Note If `InputBasis` is not specified, then `InputBasis` is assigned the value specified for `OutputBasis`. If either `InputBasis` or `OutputBasis` are not specified, the default is 0 (actual/actual) for both.

Data Types: double

Output Arguments**ParRates — Par bond coupon rates**

numeric

Par bond coupon rates, returned as a NUMBONDS-by-1 numeric vector. ParRates are ordered by ascending maturity.

CurveDates — Maturity dates that correspond to ParRates

`datetime` | `serial date number`

Maturity dates that correspond to the ParRates, returned as a NUMBONDS-by-1 vector of maturity dates that correspond to each par rate contained in ParRates.

ParRates are expressed as serial date numbers (default) or datetimes (if CurveDates or Settle are datetime arrays). CurveDates are ordered by ascending maturity.

Version History

Introduced before R2006a

R2022b: Serial date numbers not recommended

Not recommended starting in R2022b

Although zero2pyld supports serial date numbers, `datetime` values are recommended instead. The `datetime` data type provides flexible date and time formats, storage out to nanosecond precision, and properties to account for time zones and daylight saving time.

To convert serial date numbers or text to `datetime` values, use the `datetime` function. For example:

```
t = datetime(738427.656845093, "ConvertFrom", "datenum");
y = year(t)

y =
```

```
2021
```

There are no plans to remove support for serial date number inputs.

R2017b: Support for optional name-value input arguments

Behavior changed in R2017b

The specification of optional input arguments has changed. While the previous ordered inputs syntax is still supported, it may no longer be supported in a future release. Use the new optional name-value pair inputs: `InputCompounding`, `InputBasis`, `OutputCompounding`, and `OutputBasis`.

See Also

`fwd2zero` | `zero2fwd` | `getForwardRates` | `disc2zero` | `zero2disc` | `pyld2zero` | `zbtprice` | `zbtyield` | `datetime`

Topics

“Term Structure of Interest Rates” on page 2-29

“Fixed-Income Terminology” on page 2-15

compact

Create compact credit scorecard

Syntax

```
csc = compact(sc)
```

Description

`csc = compact(sc)` converts an existing `creditscorecard` object to a `compactCreditScorecard` object (`csc`).

Note To use this function, you must have a license for Risk Management Toolbox.

Examples

Convert `creditscorecard` Object into `compactCreditScorecard` Object

Convert a `creditscorecard` object into a `compactCreditScorecard` object to use `displaypoints` (Risk Management Toolbox), `score` (Risk Management Toolbox), and `probdefault` (Risk Management Toolbox) from Risk Management Toolbox™ with the object.

First, create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData.mat
sc = creditscorecard(data)
```

```
sc =
    creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustI
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustI
    Data: [1200×11 table]
```

Before creating a `compactCreditScorecard` object, you must use `autobinning` and `fitmodel` with the `creditscorecard` object.

```
sc = autobinning(sc);
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Convert the `creditscorecard` object into a `compactCreditScorecard` object by using the `compact` function. To use `compact`, you must have a Risk Management Toolbox™ license.

```
csc = compact(sc)
```

```
csc =
```

```
compactCreditScorecard with properties:
```

```

    Description: ''
    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    NumericPredictors: {'CustAge' 'CustIncome' 'TmWBank' 'AMBalance'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    PredictorVars: {'CustAge' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
```

You can then use `displaypoints` (Risk Management Toolbox), `score` (Risk Management Toolbox), and `probdefault` (Risk Management Toolbox) from Risk Management Toolbox™ with the `compactCreditScorecard` object.

Input Arguments

sc — Credit scorecard model

`creditscorecard` object

Credit scorecard model, specified as a `creditscorecard` object.

Note You must use a `creditscorecard` object (`sc`) for input that has been previously processed with `autobinning` and `fitmodel`, or `fitConstrainedModel`. Optionally, you can use `formatpoints` in addition to these functions.

Output Arguments

csc — Compact credit scorecard

`compactCreditScorecard` object

Compact credit scorecard, returned as a `compactCreditScorecard` object. You can then use `displaypoints`, `score`, and `probdefault` from the Risk Management Toolbox with this `csc` object.

Version History

Introduced in R2019a

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `compactCreditScorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `bindata` | `fitmodel` | `formatpoints` | `displaypoints` | `setmodel` | `probdefault` | `validatemodel` | `table`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Credit Scorecard Modeling with Missing Values” on page 8-57

“compactCreditScorecard Object Workflow” (Risk Management Toolbox)

“Troubleshooting Credit Scorecard Results” on page 8-64

“Credit Scorecard Modeling Workflow” on page 8-52

“About Credit Scorecards” on page 8-49

score

Compute credit scores for given data

Syntax

```
Scores = score(sc)
Scores = score(sc, data)

[ Scores, Points ] = score(sc)
[ Scores, Points ] = score(sc, data)
```

Description

`Scores = score(sc)` computes the credit scores for the `creditscorecard` object's training data. This data can be a "training" or a "live" dataset. If the `data` input argument is not explicitly provided, the `score` function determines scores for the existing `creditscorecard` object's data.

`formatpoints` supports multiple alternatives to modify the scaling of the scores and can also be used to control the rounding of points and scores, and whether the base points are reported separately or spread across predictors. Missing data translates into NaN values for the corresponding points, and therefore for the total score. Use `formatpoints` to modify the score behavior for rows with missing data.

`Scores = score(sc, data)` computes the credit scores for the given input data. This data can be a "training" or a "live" dataset.

`formatpoints` supports multiple alternatives to modify the scaling of the scores and can also be used to control the rounding of points and scores, and whether the base points are reported separately or spread across predictors. Missing data translates into NaN values for the corresponding points, and therefore for the total score. Use `formatpoints` to modify the score behavior for rows with missing data.

`[ Scores, Points ] = score(sc)` computes the credit scores and points for the given data. If the `data` input argument is not explicitly provided, the `score` function determines scores for the existing `creditscorecard` object's data.

`formatpoints` supports multiple alternatives to modify the scaling of the scores and can also be used to control the rounding of points and scores, and whether the base points are reported separately or spread across predictors. Missing data translates into NaN values for the corresponding points, and therefore for the total score. Use `formatpoints` to modify the score behavior for rows with missing data.

`[ Scores, Points ] = score(sc, data)` computes the credit scores and points for the given input data. This data can be a "training" or a "live" dataset.

`formatpoints` supports multiple alternatives to modify the scaling of the scores and can also be used to control the rounding of points and scores, and whether the base points are reported separately or spread across predictors. Missing data translates into NaN values for the corresponding points, and therefore for the total score. Use `formatpoints` to modify the score behavior for rows with missing data.

Examples

Obtain Scores for Training Data

This example shows how to use `score` to obtain scores for the training data.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the '`IDVar`' argument in `creditscorecard` to indicate that '`CustID`' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Score training data using the `score` function without an optional input for `data`. By default, it returns unscaled scores. For brevity, only the first ten scores are displayed.

```
Scores = score(sc);
disp(Scores(1:10))
```



```

1.0968
1.4646
0.7662
1.5779
1.4535
1.8944
-0.0872
0.9207
1.0399
0.8252

```

Scale scores and display both points and scores for each individual in the training data (for brevity, only the first ten rows are displayed). For other scaling methods, and other options for formatting points and scores, use the `formatpoints` function.

```

sc = formatpoints(sc, 'WorstAndBestScores', [300 850]);
[Scores, Points] = score(sc);
disp(Scores(1:10))

```

```

602.0394
648.1988
560.5569
662.4189
646.8109
702.1398
453.4572
579.9475
594.9064
567.9533

```

```
disp(Points(1:10,:))
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
95.256	62.421	56.765	121.18	116.05	86.224	64.15
126.46	82.276	105.81	121.18	62.107	86.224	64.15
93.256	62.421	105.81	76.585	116.05	42.287	64.15
95.256	82.276	105.81	121.18	60.719	86.224	110.96
126.46	82.276	105.81	121.18	60.719	86.224	64.15
126.46	82.276	105.81	121.18	116.05	86.224	64.15
48.727	82.276	56.765	53.208	62.107	86.224	64.15
95.256	113.58	105.81	121.18	62.107	42.287	39.729
95.256	62.421	56.765	121.18	62.107	86.224	110.96
95.256	82.276	56.765	121.18	62.107	86.224	64.15

Scores for Missing or Out-of-Range Data When Using the 'BinMissingData' Option

This example describes the assignment of points for missing data when the 'BinMissingData' option is set to `true`.

- Predictors that have missing data in the training set have an explicit bin for `<missing>` with corresponding points in the final scorecard. These points are computed from the Weight-of-Evidence (WOE) value for the `<missing>` bin and the logistic model coefficients. For scoring purposes, these points are assigned to missing values and to out-of-range values.

- Predictors with no missing data in the training set have no <missing> bin, therefore no WOE can be estimated from the training data. By default, the points for missing and out-of-range values are set to NaN, and this leads to a score of NaN when running score. For predictors that have no explicit <missing> bin, use the name-value argument 'Missing' in formatpoints to indicate how missing data should be treated for scoring purposes.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

```
Number of rows: 1200
```

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

```
Number of missing values CustAge: 30
```

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

```
Number of missing values ResStatus: 40
```

Use `creditscorecard` with the name-value argument 'BinMissingData' set to `true` to bin the missing numeric or categorical data in a separate bin. Apply automatic binning.

```
sc = creditscorecard(dataMissing,'IDVar','CustID','BinMissingData',true);
sc = autobinning(sc);
```

```
disp(sc)
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBBalance' 'UtilizationRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 1
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
    Data: [1200x11 table]
```

Set a minimum value of zero for `CustAge` and `CustIncome`. With this, any negative age or income information becomes invalid or "out-of-range". For scoring purposes, out-of-range values are given the same points as missing values.

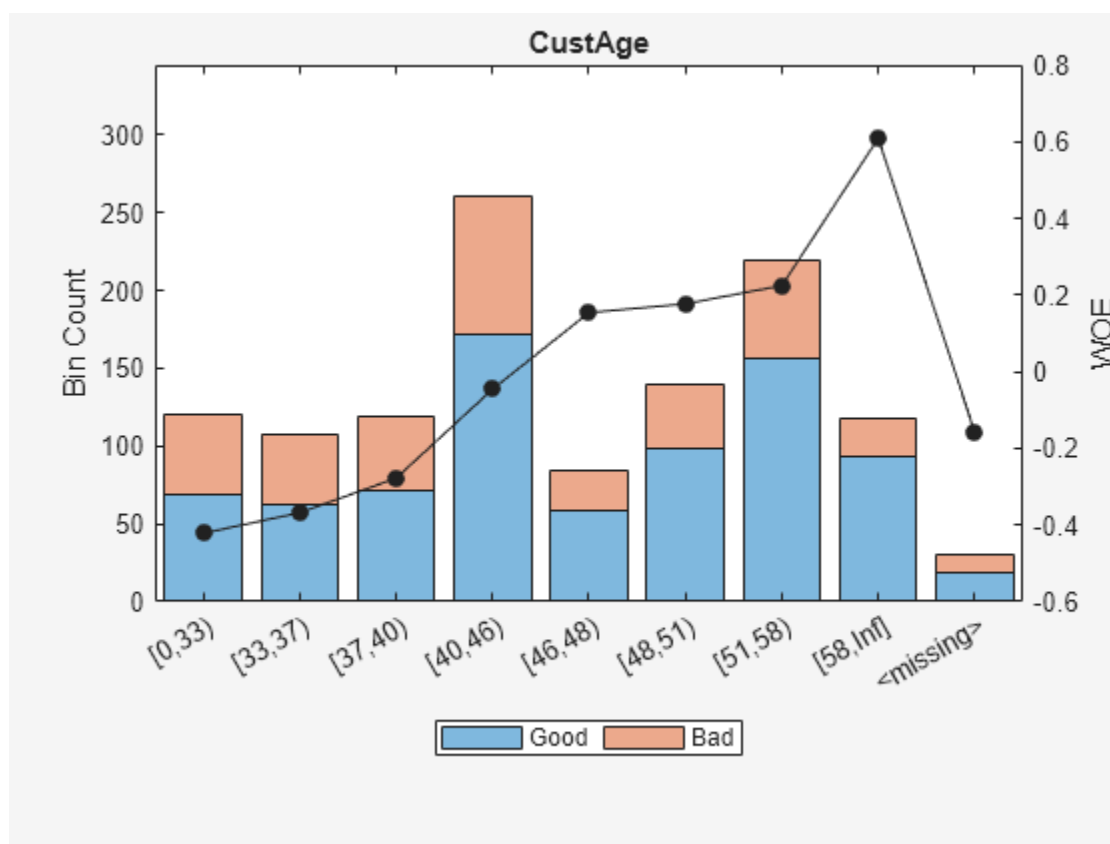
```
sc = modifybins(sc, 'CustAge', 'MinValue', 0);
sc = modifybins(sc, 'CustIncome', 'MinValue', 0);
```

Display and plot bin information for numeric data for 'CustAge' that includes missing data in a separate bin labelled <missing>.

```
[bi,cp] = bininfo(sc, 'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,33)' }	69	52	1.3269	-0.42156	0.018993
{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549
{ '[46,48)' }	59	25	2.36	0.15424	0.0016199
{ '[48,51)' }	99	41	2.4146	0.17713	0.0035449
{ '[51,58)' }	157	62	2.5323	0.22469	0.0088407
{ '[58,Inf]' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.087112

```
plotbins(sc, 'CustAge')
```

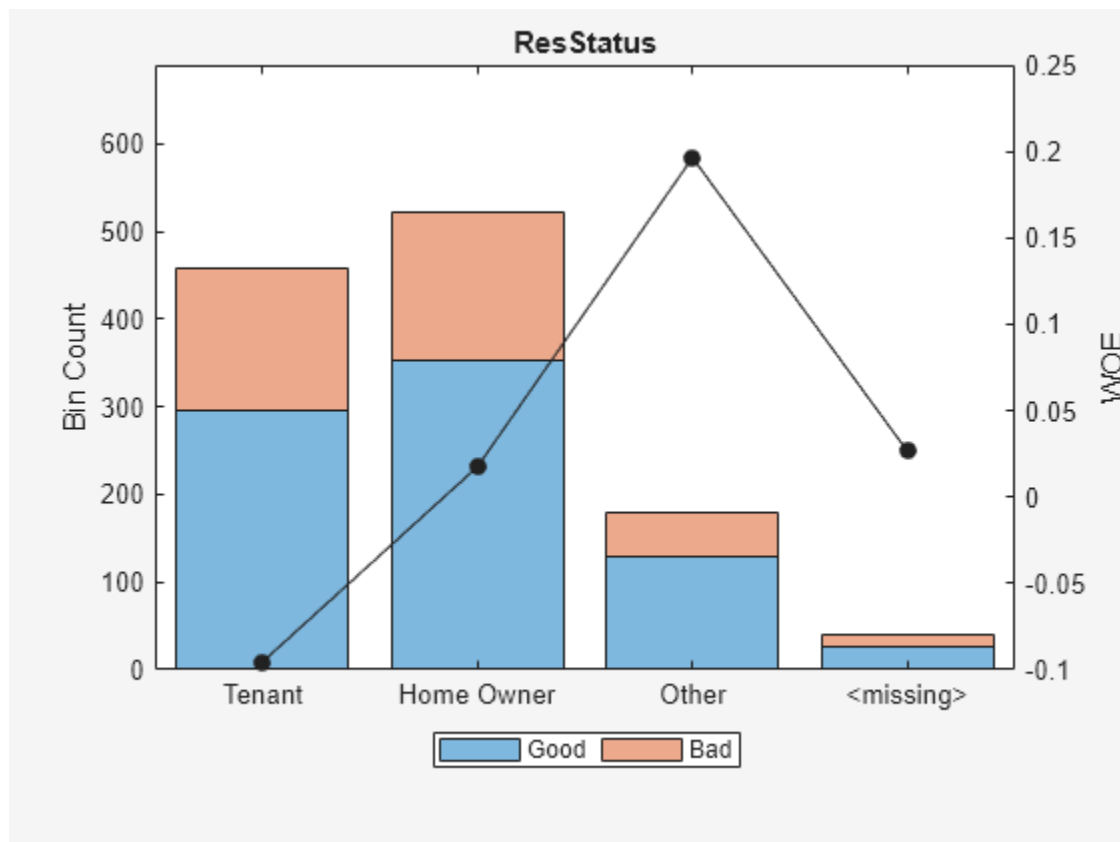


Display and plot bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{'Other' }	128	52	2.4615	0.19637	0.0055808
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



For the 'CustAge' and 'ResStatus' predictors, there is missing data (NaNs and <undefined>) in the training data, and the binning process estimates a WOE value of -0.15787 and 0.026469 respectively for missing data in these predictors, as shown above.

For EmpStatus and CustIncome there is no explicit bin for missing values because the training data has no missing values for these predictors.

```
bi = bininfo(sc, 'EmpStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Unknown' }	396	239	1.6569	-0.19947	0.021715

```

{'Employed'}      407      158      2.5759      0.2418      0.026323
{'Totals' }       803      397      2.0227      NaN       0.048038

```

```

bi = bininfo(sc,'CustIncome');
disp(bi)

```

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,29000)' }	53	58	0.91379	-0.79457	0.06364
{ '[29000,33000)' }	74	49	1.5102	-0.29217	0.0091366
{ '[33000,35000)' }	68	36	1.8889	-0.06843	0.00041042
{ '[35000,40000)' }	193	98	1.9694	-0.026696	0.00017359
{ '[40000,42000)' }	68	34	2	-0.011271	1.0819e-05
{ '[42000,47000)' }	164	66	2.4848	0.20579	0.0078175
{ '[47000,Inf]' }	183	56	3.2679	0.47972	0.041657
{ 'Totals' }	803	397	2.0227	NaN	0.12285

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). For predictors that have missing data, there is an explicit `<missing>` bin, with a corresponding WOE value computed from the data. When using `fitmodel`, the corresponding WOE value for the `<missing>` bin is applied when performing the WOE transformation.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1442.8477, Chi2Stat = 4.4974731, PValue = 0.033944979
6. Adding ResStatus, Deviance = 1438.9783, Chi2Stat = 3.86941, PValue = 0.049173805
7. Adding OtherCC, Deviance = 1434.9751, Chi2Stat = 4.0031966, PValue = 0.045414057

Generalized linear regression model:

```

logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial

```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70229	0.063959	10.98	4.7498e-28
CustAge	0.57421	0.25708	2.2335	0.025513
ResStatus	1.3629	0.66952	2.0356	0.04179
EmpStatus	0.88373	0.2929	3.0172	0.002551
CustIncome	0.73535	0.2159	3.406	0.00065929
TmWBank	1.1065	0.23267	4.7556	1.9783e-06
OtherCC	1.0648	0.52826	2.0156	0.043841
AMBalance	1.0446	0.32197	3.2443	0.0011775

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 88.5, p-value = 2.55e-16

Scale the scorecard points by the "points, odds, and points to double the odds (PDO)" method using the 'PointsOddsAndPDO' argument of `formatpoints`. Suppose that you want a score of 500 points to have odds of 2 (twice as likely to be good than to be bad) and that the odds double every 50 points (so that 550 points would have odds of 4).

Display the scorecard showing the scaled points for predictors retained in the fitting model.

```
sc = formatpoints(sc, 'PointsOddsAndPDO', [500 2 50]);
PointsInfo = displaypoints(sc)
```

```
PointsInfo=38x3 table
    Predictors      Bin      Points
    _____  _____  _____
    {'CustAge' }    {'[0,33)' }    54.062
    {'CustAge' }    {'[33,37)' }    56.282
    {'CustAge' }    {'[37,40)' }    60.012
    {'CustAge' }    {'[40,46)' }    69.636
    {'CustAge' }    {'[46,48)' }    77.912
    {'CustAge' }    {'[48,51)' }    78.86
    {'CustAge' }    {'[51,58)' }    80.83
    {'CustAge' }    {'[58,Inf]' }    96.76
    {'CustAge' }    {'<missing>' }    64.984
    {'ResStatus' }  {'Tenant' }    62.138
    {'ResStatus' }  {'Home Owner' }  73.248
    {'ResStatus' }  {'Other' }    90.828
    {'ResStatus' }  {'<missing>' }    74.125
    {'EmpStatus' }  {'Unknown' }    58.807
    {'EmpStatus' }  {'Employed' }    86.937
    {'EmpStatus' }  {'<missing>' }    NaN
    :
```

Notice that points for the <missing> bin for `CustAge` and `ResStatus` are explicitly shown (as 64.9836 and 74.1250, respectively). These points are computed from the WOE value for the <missing> bin, and the logistic model coefficients.

For predictors that have no missing data in the training set, there is no explicit <missing> bin. By default the points are set to `NaN` for missing data and they lead to a score of `NaN` when running `score`. For predictors that have no explicit <missing> bin, use the name-value argument 'Missing' in `formatpoints` to indicate how missing data should be treated for scoring purposes.

For the purpose of illustration, take a few rows from the original data as test data and introduce some missing data. Also introduce some invalid, or out-of-range values. For numeric data, values below the minimum (or above the maximum) allowed are considered invalid, such as a negative value for age (recall 'MinValue' was earlier set to 0 for `CustAge` and `CustIncome`). For categorical data, invalid values are categories not explicitly included in the scorecard, for example, a residential status not previously mapped to scorecard categories, such as "House", or a meaningless string such as "abc123".

```
tdata = dataMissing(11:18, mdl.PredictorNames); % Keep only the predictors retained in the model
% Set some missing values
tdata.CustAge(1) = NaN;
tdata.ResStatus(2) = missing;
tdata.EmpStatus(3) = missing;
tdata.CustIncome(4) = NaN;
% Set some invalid values
```

```

tdata.CustAge(5) = -100;
tdata.ResStatus(6) = 'House';
tdata.EmpStatus(7) = 'Freelancer';
tdata.CustIncome(8) = -1;
disp(tdata)

```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
NaN	Tenant	Unknown	34000	44	Yes	119.8
48	<undefined>	Unknown	44000	14	Yes	403.62
65	Home Owner	<undefined>	48000	6	No	111.88
44	Other	Unknown	NaN	35	No	436.41
-100	Other	Employed	46000	16	Yes	162.21
33	House	Employed	36000	36	Yes	845.02
39	Tenant	Freelancer	34000	40	Yes	756.26
24	Home Owner	Employed	-1	19	Yes	449.61

Score the new data and see how points are assigned for missing CustAge and ResStatus, because we have an explicit bin with points for <missing>. However, for EmpStatus and CustIncome the score function sets the points to NaN.

```

[Scores,Points] = score(sc,tdata);
disp(Scores)

```

```

481.2231
520.8353
NaN
NaN
551.7922
487.9588
NaN
NaN

```

```

disp(Points)

```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	NaN	96.969	51.132	50.914	89.922
69.636	90.828	58.807	NaN	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	NaN	67.893	61.858	75.622	63.028
54.062	73.248	86.937	NaN	61.061	75.622	89.922

Use the name-value argument 'Missing' in formatpoints to choose how to assign points to missing values for predictors that do not have an explicit <missing> bin. In this example, use the 'MinPoints' option for the 'Missing' argument. The minimum points for EmpStatus in the scorecard displayed above are 58.8072, and for CustIncome the minimum points are 29.3753.

```

sc = formatpoints(sc,'Missing','MinPoints');
[Scores,Points] = score(sc,tdata);
disp(Scores)

```

```

481.2231
520.8353

```

```

517.7532
451.3405
551.7922
487.9588
449.3577
470.2267

```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	58.807	96.969	51.132	50.914	89.922
69.636	90.828	58.807	29.375	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	58.807	67.893	61.858	75.622	63.028
54.062	73.248	86.937	29.375	61.061	75.622	89.922

Score a New Dataset

This example shows how to use `score` to obtain scores for a new dataset (for example, a validation or a test dataset) using the optional `'data'` input in the `score` function.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument in `creditscorecard` to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

Estimate	SE	tStat	pValue
----------	----	-------	--------

(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

For the purpose of illustration, suppose that a few rows from the original data are our "new" data. Use the optional data input argument in the score function to obtain the scores for the newdata.

```
newdata = data(10:20,:);
Scores = score(sc,newdata)
```

Scores = 11×1

```
0.8252
0.6553
1.2443
0.9478
0.5690
1.6192
0.4899
0.3824
0.2945
1.4401
0.8242
⋮
```

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

data — Dataset to be scored

table

(Optional) Dataset to be scored, specified as a MATLAB table where each row corresponds to individual observations. The `data` must contain columns for each of the predictors in the `creditscorecard` object.

Output Arguments

Scores — Scores for each observation

vector

Scores for each observation, returned as a vector.

Points — Points per predictor for each observation

table

Points per predictor for each observation, returned as a table.

Algorithms

The score of an individual i is given by the formula

$$\text{Score}(i) = \text{Shift} + \text{Slope} * (b_0 + b_1 * \text{WOE}_1(i) + b_2 * \text{WOE}_2(i) + \dots + b_p * \text{WOE}_p(i))$$

where b_j is the coefficient of the j -th variable in the model, and $\text{WOE}_j(i)$ is the Weight of Evidence (WOE) value for the i -th individual corresponding to the j -th model variable. Shift and Slope are scaling constants that can be controlled with `formatpoints`.

If the data for individual i is in the i -th row of a given dataset, to compute a score, the $\text{data}(i,j)$ is binned using existing binning maps, and converted into a corresponding Weight of Evidence value $\text{WOE}_j(i)$. Using the model coefficients, the unscaled score is computed as

$$s = b_0 + b_1 * \text{WOE}_1(i) + \dots + b_p * \text{WOE}_p(i).$$

For simplicity, assume in the description above that the j -th variable in the model is the j -th column in the data input, although, in general, the order of variables in a given dataset does not have to match the order of variables in the model, and the dataset could have additional variables that are not used in the model.

The formatting options can be controlled using `formatpoints`.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `bindata` | `fitmodel` | `formatpoints` | `displaypoints` | `setmodel` | `probdefault` | `validatemodel` | `table`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Credit Scorecard Modeling with Missing Values” on page 8-57

"Troubleshooting Credit Scorecard Results" on page 8-64

"Credit Scorecard Modeling Workflow" on page 8-52

"About Credit Scorecards" on page 8-49

formatpoints

Format scorecard points and scaling

Syntax

```
sc = formatpoints(sc,Name,Value)
```

Description

`sc = formatpoints(sc,Name,Value)` modifies the scorecard points and scaling using optional name-value pair arguments. For example, use optional name-value pair arguments to change the scaling of the scores or the rounding of the points.

Examples

Scale Points Using Points, Odds Levels, and PDO

This example shows how to use `formatpoints` to scale by providing the points, odds levels, and PDO (points to double the odds). By using `formatpoints` to scale, you can put points and scores in a desired range that is more meaningful for practical purposes. Technically, this involves a linear transformation from the unscaled to the scaled points by the `formatpoints` function.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the 'IDVar' argument in `creditscorecard` to indicate that 'CustID' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data,'IDVar','CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

Estimate	SE	tStat	pValue
----------	----	-------	--------

(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Display unscaled points for predictors retained in the fitting model and display the minimum and maximum possible unscaled scores.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

PointsInfo=37×3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.15894
{'CustAge' }	{'[33,37)' }	-0.14036
{'CustAge' }	{'[37,40)' }	-0.060323
{'CustAge' }	{'[40,46)' }	0.046408
{'CustAge' }	{'[46,48)' }	0.21445
{'CustAge' }	{'[48,58)' }	0.23039
{'CustAge' }	{'[58,Inf]' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252
{'ResStatus' }	{'Home Owner' }	0.12696
{'ResStatus' }	{'Other' }	0.37641
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.076317
{'EmpStatus' }	{'Employed' }	0.31449
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	-0.45716
:		

```
MinScore =  
-1.3100
```

```
MaxScore =  
3.0726
```

Scale by providing the points, odds levels, and PDO (points to double the odds). Suppose that you want a score of 500 points to have odds of 2 (twice as likely to be good than to be bad) and that the odds double every 50 points (so that 550 points would have odds of 4).

```
sc = formatpoints(sc,'PointsOddsAndPDO',[500 2 50]);  
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

PointsInfo=37×3 table

Predictors	Bin	Points
------------	-----	--------

{'CustAge' }	{'[-Inf,33)' }	52.821
{'CustAge' }	{'[33,37)' }	54.161
{'CustAge' }	{'[37,40)' }	59.934
{'CustAge' }	{'[40,46)' }	67.633
{'CustAge' }	{'[46,48)' }	79.755
{'CustAge' }	{'[48,58)' }	80.905
{'CustAge' }	{'[58,Inf]' }	98.838
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	62.031
{'ResStatus' }	{'Home Owner' }	73.444
{'ResStatus' }	{'Other' }	91.438
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	58.781
{'EmpStatus' }	{'Employed' }	86.971
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	31.309
:		

```
MinScore =
355.5051
```

```
MaxScore =
671.6403
```

Scale Points Using Worst and Best Scores

This example shows how to use `formatpoints` to scale by providing the **Worst** and **Best** score values. By using `formatpoints` to scale, you can put points and scores in a desired range that is more meaningful for practical purposes. Technically, this involves a linear transformation from the unscaled to the scaled points.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument in `creditscorecard` to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding `CustIncome`, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding `TmWBank`, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding `AMBalance`, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding `EmpStatus`, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding `CustAge`, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding `ResStatus`, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078

7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBAL
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Display unscaled points for predictors retained in the fitting model and display the minimum and maximum possible unscaled scores.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

PointsInfo=37x3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33) ' }	-0.15894
{'CustAge' }	{'[33,37) ' }	-0.14036
{'CustAge' }	{'[37,40) ' }	-0.060323
{'CustAge' }	{'[40,46) ' }	0.046408
{'CustAge' }	{'[46,48) ' }	0.21445
{'CustAge' }	{'[48,58) ' }	0.23039
{'CustAge' }	{'[58,Inf] ' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252
{'ResStatus' }	{'Home Owner' }	0.12696
{'ResStatus' }	{'Other' }	0.37641
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.076317
{'EmpStatus' }	{'Employed' }	0.31449
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000) ' }	-0.45716
:		

```
MinScore =
-1.3100
```

```
MaxScore =
3.0726
```

Scale by providing the 'Worst' and 'Best' score values. The range provided below is a common score range. Display the points information again to verify that they are now scaled and also display the scaled minimum and maximum scores.

```
sc = formatpoints(sc, 'WorstAndBestScores', [300 850]);
[PointsInfo, MinScore, MaxScore] = displaypoints(sc)
```

PointsInfo=37×3 table

Predictors	Bin	Points
{ 'CustAge' }	{ '[-Inf, 33)' }	46.396
{ 'CustAge' }	{ '[33, 37)' }	48.727
{ 'CustAge' }	{ '[37, 40)' }	58.772
{ 'CustAge' }	{ '[40, 46)' }	72.167
{ 'CustAge' }	{ '[46, 48)' }	93.256
{ 'CustAge' }	{ '[48, 58)' }	95.256
{ 'CustAge' }	{ '[58, Inf]' }	126.46
{ 'CustAge' }	{ '<missing>' }	NaN
{ 'ResStatus' }	{ 'Tenant' }	62.421
{ 'ResStatus' }	{ 'Home Owner' }	82.276
{ 'ResStatus' }	{ 'Other' }	113.58
{ 'ResStatus' }	{ '<missing>' }	NaN
{ 'EmpStatus' }	{ 'Unknown' }	56.765
{ 'EmpStatus' }	{ 'Employed' }	105.81
{ 'EmpStatus' }	{ '<missing>' }	NaN
{ 'CustIncome' }	{ '[-Inf, 29000)' }	8.9706
:		

```
MinScore =
300.0000
```

```
MaxScore =
850.0000
```

As expected, the values of MinScore and MaxScore correspond to the desired worst and best scores.

Scale Points Using Shift and Slope

This example shows how to use `formatpoints` to scale by providing the `Shift` and `Slope` values. By using `formatpoints` to scale, you can put points and scores in a desired range that is more meaningful for practical purposes. Technically, this involves a linear transformation from the unscaled to the scaled points by the `formatpoints` function.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the '`IDVar`' argument in `creditscorecard` to indicate that '`CustID`' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```


Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Display unscaled points for predictors retained in the fitting model and display the minimum and maximum possible unscaled scores.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

PointsInfo=37x3 table

Predictors	Bin	Points
{ 'CustAge' }	{ '[-Inf,33)' }	-0.15894
{ 'CustAge' }	{ '[33,37)' }	-0.14036
{ 'CustAge' }	{ '[37,40)' }	-0.060323
{ 'CustAge' }	{ '[40,46)' }	0.046408
{ 'CustAge' }	{ '[46,48)' }	0.21445
{ 'CustAge' }	{ '[48,58)' }	0.23039
{ 'CustAge' }	{ '[58,Inf]' }	0.479
{ 'CustAge' }	{ '<missing>' }	NaN
{ 'ResStatus' }	{ 'Tenant' }	-0.031252
{ 'ResStatus' }	{ 'Home Owner' }	0.12696
{ 'ResStatus' }	{ 'Other' }	0.37641
{ 'ResStatus' }	{ '<missing>' }	NaN
{ 'EmpStatus' }	{ 'Unknown' }	-0.076317
{ 'EmpStatus' }	{ 'Employed' }	0.31449
{ 'EmpStatus' }	{ '<missing>' }	NaN

```

{'CustIncome'}    {'[-Inf,29000)'}    -0.45716
:

```

```

MinScore =
-1.3100

```

```

MaxScore =
3.0726

```

Scale by providing the 'Shift' and 'Slope' values. In this example, there is an arbitrary choice of shift and slope. Display the points information again to verify that they are now scaled and also display the scaled minimum and maximum scores.

```

sc = formatpoints(sc, 'ShiftAndSlope', [300 6]);
[PointsInfo, MinScore, MaxScore] = displaypoints(sc)

```

PointsInfo=37×3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33) ' }	41.904
{'CustAge' }	{'[33,37) ' }	42.015
{'CustAge' }	{'[37,40) ' }	42.495
{'CustAge' }	{'[40,46) ' }	43.136
{'CustAge' }	{'[46,48) ' }	44.144
{'CustAge' }	{'[48,58) ' }	44.239
{'CustAge' }	{'[58,Inf] ' }	45.731
{'CustAge' }	{'<missing> ' }	NaN
{'ResStatus' }	{'Tenant' }	42.67
{'ResStatus' }	{'Home Owner' }	43.619
{'ResStatus' }	{'Other' }	45.116
{'ResStatus' }	{'<missing> ' }	NaN
{'EmpStatus' }	{'Unknown' }	42.399
{'EmpStatus' }	{'Employed' }	44.744
{'EmpStatus' }	{'<missing> ' }	NaN
{'CustIncome' }	{'[-Inf,29000) ' }	40.114
:		

```

MinScore =
292.1401

```

```

MaxScore =
318.4355

```

Report Base Points Separately

This example shows how to use `formatpoints` to separate the base points from the rest of the points assigned to each predictor variable. The `formatpoints` name-value pair argument 'BasePoints' serves this purpose.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the 'IDVar' argument in `creditscorecard` to indicate that 'CustID' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Display unscaled points for predictors retained in the fitting model and display the minimum and maximum possible unscaled scores.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

PointsInfo=37x3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.15894
{'CustAge' }	{'[33,37)' }	-0.14036
{'CustAge' }	{'[37,40)' }	-0.060323
{'CustAge' }	{'[40,46)' }	0.046408
{'CustAge' }	{'[46,48)' }	0.21445
{'CustAge' }	{'[48,58)' }	0.23039
{'CustAge' }	{'[58,Inf]' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252

```

{'ResStatus' } {'Home Owner' } 0.12696
{'ResStatus' } {'Other' } 0.37641
{'ResStatus' } {'<missing>' } NaN
{'EmpStatus' } {'Unknown' } -0.076317
{'EmpStatus' } {'Employed' } 0.31449
{'EmpStatus' } {'<missing>' } NaN
{'CustIncome' } {'[-Inf,29000)' } -0.45716
:

```

```

MinScore =
-1.3100

```

```

MaxScore =
3.0726

```

By setting the name-value pair argument `BasePoints` to true, the points information table reports the base points separately in the first row. The minimum and maximum possible scores are not affected by this option.

```

sc = formatpoints(sc,'BasePoints',true);
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)

```

PointsInfo=38×3 table

Predictors	Bin	Points
{'BasePoints' }	{'BasePoints' }	0.70239
{'CustAge' }	{'[-Inf,33)' }	-0.25928
{'CustAge' }	{'[33,37)' }	-0.24071
{'CustAge' }	{'[37,40)' }	-0.16066
{'CustAge' }	{'[40,46)' }	-0.053933
{'CustAge' }	{'[46,48)' }	0.11411
{'CustAge' }	{'[48,58)' }	0.13005
{'CustAge' }	{'[58,Inf]' }	0.37866
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.13159
{'ResStatus' }	{'Home Owner' }	0.026616
{'ResStatus' }	{'Other' }	0.27607
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.17666
{'EmpStatus' }	{'Employed' }	0.21415
{'EmpStatus' }	{'<missing>' }	NaN
:		

```

MinScore =
-1.3100

```

```

MaxScore =
3.0726

```

Round Points

This example shows how to use `formatpoints` to round points. Rounding is usually applied after scaling, otherwise, if the points for a particular predictor are all in a small range, rounding could

cause the rounded points for different bins to be the same. Also, rounding all the points may slightly change the minimum and maximum total points.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument in `creditscorecard` to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Display unscaled points for predictors retained in the fitting model and display the minimum and maximum possible unscaled scores.

```
[PointsInfo, MinScore, MaxScore] = displaypoints(sc)
```

PointsInfo=37×3 table

Predictors	Bin	Points
{ 'CustAge' }	{ '[-Inf,33)' }	-0.15894

```

{'CustAge' } {'[33,37)'} } -0.14036
{'CustAge' } {'[37,40)'} } -0.060323
{'CustAge' } {'[40,46)'} } 0.046408
{'CustAge' } {'[46,48)'} } 0.21445
{'CustAge' } {'[48,58)'} } 0.23039
{'CustAge' } {'[58,Inf]'} } 0.479
{'CustAge' } {'<missing>'} } NaN
{'ResStatus' } {'Tenant' } -0.031252
{'ResStatus' } {'Home Owner' } 0.12696
{'ResStatus' } {'Other' } 0.37641
{'ResStatus' } {'<missing>'} } NaN
{'EmpStatus' } {'Unknown' } -0.076317
{'EmpStatus' } {'Employed' } 0.31449
{'EmpStatus' } {'<missing>'} } NaN
{'CustIncome' } {'[-Inf,29000)'} } -0.45716
:

```

```

MinScore =
-1.3100

```

```

MaxScore =
3.0726

```

Scale points, and display the points information. By default, no rounding is applied.

```

sc = formatpoints(sc, 'WorstAndBestScores', [300 850]);
PointsInfo = displaypoints(sc)

```

```

PointsInfo=37×3 table
    Predictors      Bin      Points
    _____  _
{'CustAge' } {'[-Inf,33)'} } 46.396
{'CustAge' } {'[33,37)'} } 48.727
{'CustAge' } {'[37,40)'} } 58.772
{'CustAge' } {'[40,46)'} } 72.167
{'CustAge' } {'[46,48)'} } 93.256
{'CustAge' } {'[48,58)'} } 95.256
{'CustAge' } {'[58,Inf]'} } 126.46
{'CustAge' } {'<missing>'} } NaN
{'ResStatus' } {'Tenant' } 62.421
{'ResStatus' } {'Home Owner' } 82.276
{'ResStatus' } {'Other' } 113.58
{'ResStatus' } {'<missing>'} } NaN
{'EmpStatus' } {'Unknown' } 56.765
{'EmpStatus' } {'Employed' } 105.81
{'EmpStatus' } {'<missing>'} } NaN
{'CustIncome' } {'[-Inf,29000)'} } 8.9706
:

```

Use the name-value pair argument **Round** to apply rounding for all points and then display the points information again.

```

sc = formatpoints(sc, 'Round', 'AllPoints');
PointsInfo = displaypoints(sc)

```

```

PointsInfo=37×3 table
    Predictors      Bin      Points
    _____  _

```

{ 'CustAge' }	{ '[-Inf,33)' }	46
{ 'CustAge' }	{ '[33,37)' }	49
{ 'CustAge' }	{ '[37,40)' }	59
{ 'CustAge' }	{ '[40,46)' }	72
{ 'CustAge' }	{ '[46,48)' }	93
{ 'CustAge' }	{ '[48,58)' }	95
{ 'CustAge' }	{ '[58,Inf]' }	126
{ 'CustAge' }	{ '<missing>' }	NaN
{ 'ResStatus' }	{ 'Tenant' }	62
{ 'ResStatus' }	{ 'Home Owner' }	82
{ 'ResStatus' }	{ 'Other' }	114
{ 'ResStatus' }	{ '<missing>' }	NaN
{ 'EmpStatus' }	{ 'Unknown' }	57
{ 'EmpStatus' }	{ 'Employed' }	106
{ 'EmpStatus' }	{ '<missing>' }	NaN
{ 'CustIncome' }	{ '[-Inf,29000)' }	9
:		

Rounding and Default Probabilities

This example shows that rounding scorecard points can modify the original risk ranking of a credit scorecard. You can control rounding by using `formatpoints` with the optional name-value pair argument for `'Rounding'`.

Credit scores rank customers by risk. If higher scores are given to better, less risky customers, then higher scores must correspond to lower default probabilities. When you use the name-value pair argument for `'Rounding'`, depending on the value for `'Rounding'`, the rounding behavior is:

- When `'Rounding'` is set to `'None'` (default option), no rounding is applied to points or scores, and the risk ranking is completely consistent with the calibrated model.
- When `'Rounding'` is set to `'FinalScore'`, rounding is only applied to the final scores. In this case: a) Customers with different scores (different risk) may have the same rounded score. b) Customers with the same rounded score may have different default probabilities. c) Customer with higher rounded scores will always have lower default probability than customers with lower scores.
- When `'Rounding'` is set to `'AllPoints'`, rounding is applied to all points in the scorecard (all bins, all predictors). In this case: a) Customers with different scores (different risk) may have the same rounded score, or their ranking may even be reversed (the customer with the lower original score may have a higher rounded score). b) Customers with the same rounded score may have different default probabilities. c) Customer with higher rounded scores may in some cases have *higher* default probabilities than customers with lower scores.

Create a creditcard object

To demonstrate the rounding behavior, first create a `creditcard` object.

```
load CreditCardData
sc = creditcard(data, 'IDVar', 'CustID', 'ResponseVar', 'status');
sc = autobinning(sc);
```

```
sc = modifybins(sc, 'CustIncome', 'CutPoints', 20000:5000:60000);
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1487.9719, Chi2Stat = 35.469392, PValue = 2.5909009e-09
2. Adding TmWBank, Deviance = 1465.7998, Chi2Stat = 22.172089, PValue = 2.4927133e-06
3. Adding AMBalance, Deviance = 1455.206, Chi2Stat = 10.593833, PValue = 0.0011346548
4. Adding EmpStatus, Deviance = 1446.3918, Chi2Stat = 8.8142314, PValue = 0.0029889009
5. Adding CustAge, Deviance = 1440.6825, Chi2Stat = 5.709236, PValue = 0.016875883
6. Adding ResStatus, Deviance = 1436.1363, Chi2Stat = 4.5462043, PValue = 0.032991806
7. Adding OtherCC, Deviance = 1431.9546, Chi2Stat = 4.1817827, PValue = 0.040860699

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70247	0.064046	10.968	5.4345e-28
CustAge	0.60579	0.24405	2.4822	0.013058
ResStatus	1.4463	0.65427	2.2105	0.02707
EmpStatus	0.90501	0.29262	3.0928	0.0019828
CustIncome	0.70869	0.20535	3.4512	0.00055815
TmWBank	1.0839	0.23244	4.6631	3.1145e-06
OtherCC	1.0906	0.52936	2.0602	0.039377
AMBalance	1.0148	0.32273	3.1445	0.0016636

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 91.5, p-value = 6.12e-17

Apply the 'Rounding' Options

Apply each of the three 'Rounding' options to the `creditscorecard` object.

```
sc = formatpoints(sc, 'PointsOddsAndPDO', [500 2 50]); % No Rounding
points1 = displaypoints(sc);
[S1,P1] = score(sc);
defProb1 = probdefault(sc);
```

```
sc = formatpoints(sc, 'Round', 'AllPoints'); % 'AllPoints' Rounding
points2 = displaypoints(sc);
[S2,P2] = score(sc);
defProb2 = probdefault(sc);
```

```
sc = formatpoints(sc, 'Round', 'FinalScore'); % 'FinalScore' Rounding
points3 = displaypoints(sc);
[S3,P3] = score(sc);
defProb3 = probdefault(sc);
```

Compare the 'Rounding' Options

Visualize the default probabilities versus the scores.

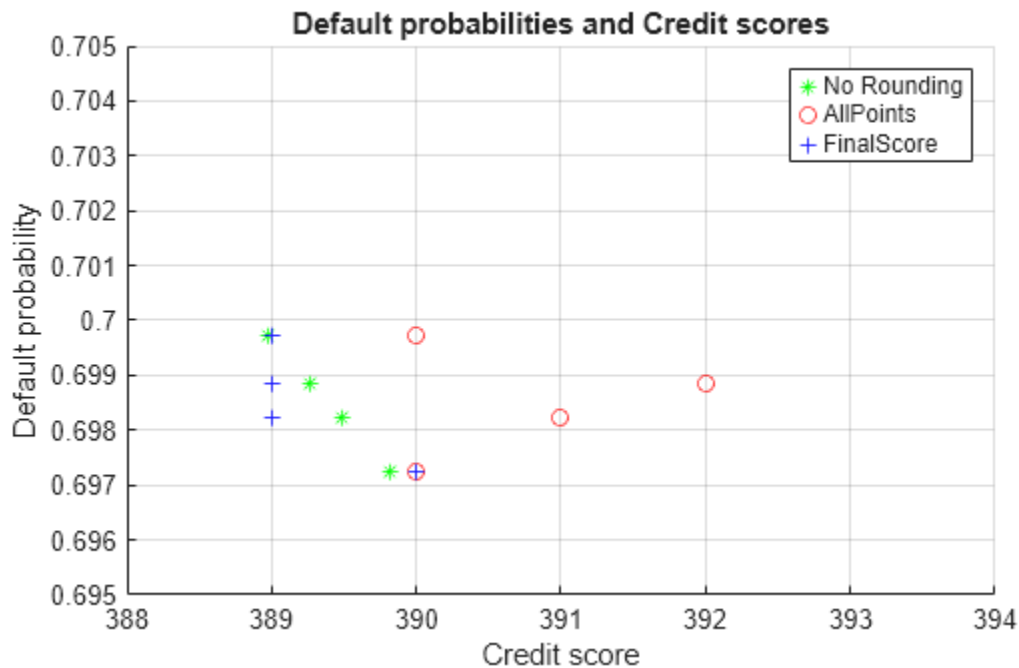
```
figure
hold on
scatter(S1, defProb1, 'g*')
```



```

scatter(S2, defProb2, 'ro')
scatter(S3, defProb3, 'b+')
legend('No Rounding', 'AllPoints', 'FinalScore')
axis([388 394 0.695 0.705])
xlabel('Credit score')
ylabel('Default probability')
title('Default probabilities and Credit scores')
grid

```



Inspect the points and total scores for each 'Rounding' option, in table format.

```

ind = [208 363 694 886];
ProbDefault = defProb1(ind)

```

```

ProbDefault = 4x1

```

```

0.6997
0.6989
0.6982
0.6972

```

```

% ScoreNoRounding = S1(ind)
PointsNoRounding = P1(ind,:);
PointsNoRounding.Total = S1(ind)

```

```

PointsNoRounding=4x8 table

```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance	Total
52.9	61.555	58.503	24.647	51.551	50.416	89.4	388.97
67.65	61.555	58.503	24.647	51.551	75.723	49.64	389.27
54.234	61.555	58.503	24.647	51.551	75.723	63.271	389.48

52.9 92.441 58.503 24.647 61.277 50.416 49.64 389.82

```
% ScoreAllPoints = S2(ind)
PointsAllPoints = P2(ind,:);
PointsAllPoints.Total = S2(ind)
```

PointsAllPoints=4×8 table

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance	Total
53	62	59	25	52	50	89	390
68	62	59	25	52	76	50	392
54	62	59	25	52	76	63	391
53	92	59	25	61	50	50	390

```
% ScoreFinalScore = S3(ind)
PointsFinalScore = P3(ind,:);
PointsFinalScore.Total = S3(ind)
```

PointsFinalScore=4×8 table

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance	Total
52.9	61.555	58.503	24.647	51.551	50.416	89.4	389
67.65	61.555	58.503	24.647	51.551	75.723	49.64	389
54.234	61.555	58.503	24.647	51.551	75.723	63.271	389
52.9	92.441	58.503	24.647	61.277	50.416	49.64	390

The original `creditscorecard` model, without rounding, was calibrated to the data with a logistic regression. The ranking and probabilities have a statistical foundation.

Rounding, however, effectively modifies the `creditscorecard` model. When only the final score is rounded, this leads to some "ties" in rounded scores, but at least the risk ranking *across* scores is preserved (because if $s1 \leq s2$, then $\text{round}(s1) \leq \text{round}(s2)$).

However, when you round all points, a score may gain extra points by chance. For example, in the second row in the table (row 363 of original data), the points for all predictors are rounded up by almost 0.5. The original score is 389.27. Rounding the final score makes it 389. However, rounding all points makes it 392, that is three points higher than rounding the final score.

Scores for Missing or Out-of-Range Data

This example shows how to use `formatpoints` to score missing or out-of-range data. When data is scored, some observations can be either missing (NaN, or undefined) or out of range. You will need to decide whether or not points are assigned to these cases. Use the name-value pair argument `Missing` to do so.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument in `creditscorecard` to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Indicate that the minimum allowed value for 'CustAge' is zero. This makes any negative values for age invalid or out-of-range.

```
sc = modifybins(sc, 'CustAge', 'MinValue', 0);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Suppose there are missing or out of range observations in the data that you want to score. Notice that by default, the points and score assigned to the missing value is NaN.

% Set up a data set with missing and out of range data for illustration purposes

```
newdata = data(1:5,:);
newdata.CustAge(1) = NaN; % missing
newdata.CustAge(2) = -100; % invalid
newdata.ResStatus(3) = missing; % missing
newdata.ResStatus(4) = 'House'; % invalid
disp(newdata)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC
--------	---------	-------------	-----------	-----------	------------	---------	---------

1	NaN	62	Tenant	Unknown	50000	55	Ye
2	-100	22	Home Owner	Employed	52000	25	Ye
3	47	30	<undefined>	Employed	37000	61	Ne
4	50	75	House	Employed	53000	20	Ye
5	68	56	Home Owner	Employed	53000	14	Ye

```
[Scores,Points] = score(sc,newdata);
disp(Scores)
```

```
NaN
NaN
NaN
NaN
1.4535
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
NaN	-0.031252	-0.076317	0.43693	0.39607	0.15842	-0.017472
NaN	0.12696	0.31449	0.43693	-0.033752	0.15842	-0.017472
0.21445	NaN	0.31449	0.081611	0.39607	-0.19168	-0.017472
0.23039	NaN	0.31449	0.43693	-0.044811	0.15842	0.35551
0.479	0.12696	0.31449	0.43693	-0.044811	0.15842	-0.017472

Use the name-value pair argument `Missing` to replace `NaN` with points corresponding to a zero Weight-of-Evidence (WOE).

```
sc = formatpoints(sc,'Missing','ZeroWOE');
[Scores,Points] = score(sc,newdata);
disp(Scores)
```

```
0.9667
1.0859
0.8978
1.5513
1.4535
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
0.10034	-0.031252	-0.076317	0.43693	0.39607	0.15842	-0.017472
0.10034	0.12696	0.31449	0.43693	-0.033752	0.15842	-0.017472
0.21445	0.10034	0.31449	0.081611	0.39607	-0.19168	-0.017472
0.23039	0.10034	0.31449	0.43693	-0.044811	0.15842	0.35551
0.479	0.12696	0.31449	0.43693	-0.044811	0.15842	-0.017472

Alternatively, use the name-value pair argument `Missing` to replace the missing value with the minimum points for the predictors that have the missing values.

```
sc = formatpoints(sc,'Missing','MinPoints');
[Scores,Points] = score(sc,newdata);
disp(Scores)
```

```
0.7074
0.8266
```

```
0.7662
1.4197
1.4535
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
-0.15894	-0.031252	-0.076317	0.43693	0.39607	0.15842	-0.017472
-0.15894	0.12696	0.31449	0.43693	-0.033752	0.15842	-0.017472
0.21445	-0.031252	0.31449	0.081611	0.39607	-0.19168	-0.017472
0.23039	-0.031252	0.31449	0.43693	-0.044811	0.15842	0.35551
0.479	0.12696	0.31449	0.43693	-0.044811	0.15842	-0.017472

As a third alternative, use the name-value pair argument `Missing` to replace the missing value with the maximum points for the predictors that have the missing values.

```
sc = formatpoints(sc, 'Missing', 'MaxPoints');
[Scores, Points] = score(sc, newdata);
disp(Scores)
```

```
1.3454
1.4646
1.1739
1.8273
1.4535
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
0.479	-0.031252	-0.076317	0.43693	0.39607	0.15842	-0.017472
0.479	0.12696	0.31449	0.43693	-0.033752	0.15842	-0.017472
0.21445	0.37641	0.31449	0.081611	0.39607	-0.19168	-0.017472
0.23039	0.37641	0.31449	0.43693	-0.044811	0.15842	0.35551
0.479	0.12696	0.31449	0.43693	-0.044811	0.15842	-0.017472

Verify that the minimum and maximum points assigned to the missing data correspond to the minimum and maximum points for the corresponding predictors. The points for 'CustAge' are reported in the first seven rows of the points information table. For 'ResStatus' the points are in rows 8 through 10.

```
PointsInfo = displaypoints(sc);
PointsInfo(1:7,:)
```

```
ans=7x3 table
Predictors      Bin      Points
_____  
{ 'CustAge' }   { '[0,33)' } -0.15894
{ 'CustAge' }   { '[33,37)' } -0.14036
{ 'CustAge' }   { '[37,40)' } -0.060323
{ 'CustAge' }   { '[40,46)' } 0.046408
{ 'CustAge' }   { '[46,48)' } 0.21445
{ 'CustAge' }   { '[48,58)' } 0.23039
{ 'CustAge' }   { '[58,Inf]' } 0.479
```

```

min(PointsInfo.Points(1:7))

ans =
-0.1589

max(PointsInfo.Points(1:7))

ans =
0.4790

PointsInfo(8:10,:)

ans=3x3 table
    Predictors      Bin      Points
    _____  _____  _____
    {'CustAge'   } {'<missing>' }      0.479
    {'ResStatus'} {'Tenant'   }     -0.031252
    {'ResStatus'} {'Home Owner'}    0.12696

min(PointsInfo.Points(8:10))

ans =
-0.0313

max(PointsInfo.Points(8:10))

ans =
0.4790

```

Scores for Missing or Out-of-Range Data When Using the 'BinMissingData' Option

This example describes the assignment of points for missing data when the 'BinMissingData' option is set to `true`.

- Predictors that have missing data in the training set have an explicit bin for `<missing>` with corresponding points in the final scorecard. These points are computed from the Weight-of-Evidence (WOE) value for the `<missing>` bin and the logistic model coefficients. For scoring purposes, these points are assigned to missing values and to out-of-range values.
- Predictors with no missing data in the training set have no `<missing>` bin, therefore no WOE can be estimated from the training data. By default, the points for missing and out-of-range values are set to `NaN`, and this leads to a score of `NaN` when running `score`. For predictors that have no explicit `<missing>` bin, use the name-value argument 'Missing' in `formatpoints` to indicate how missing data should be treated for scoring purposes.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
1	53	62	<undefined>	Unknown	50000	55	Ye

2	61	22	Home Owner	Employed	52000	25	Ye
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Ye
5	68	56	Home Owner	Employed	53000	14	Ye

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

```
Number of rows: 1200
```

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

```
Number of missing values CustAge: 30
```

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

```
Number of missing values ResStatus: 40
```

Use `creditscorecard` with the name-value argument `'BinMissingData'` set to `true` to bin the missing numeric or categorical data in a separate bin. Apply automatic binning.

```
sc = creditscorecard(dataMissing,'IDVar','CustID','BinMissingData',true);
sc = autobinning(sc);
```

```
disp(sc)
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBBalance' 'UtilizationRate'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBBalance' 'UtilizationRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 1
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBBalance' 'UtilizationRate'}
    Data: [1200x11 table]
```

Set a minimum value of zero for `CustAge` and `CustIncome`. With this, any negative age or income information becomes invalid or "out-of-range". For scoring purposes, out-of-range values are given the same points as missing values.

```
sc = modifybins(sc,'CustAge','MinValue',0);
sc = modifybins(sc,'CustIncome','MinValue',0);
```

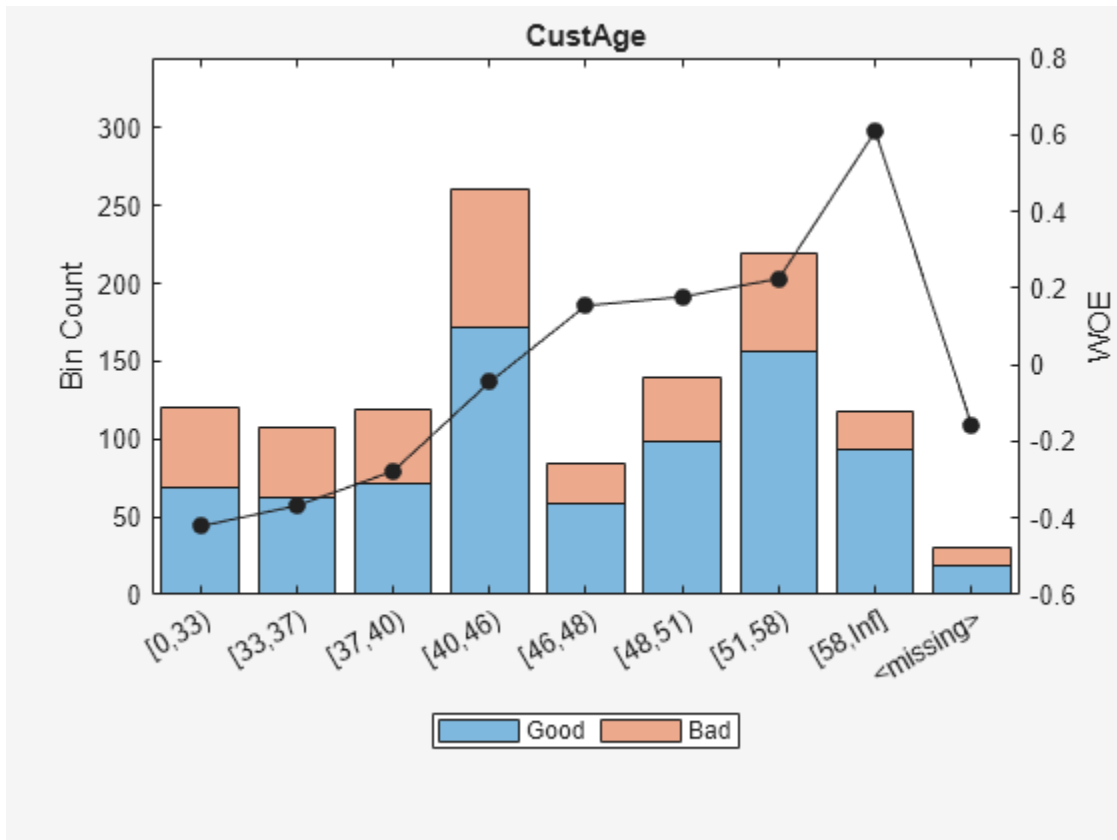
Display and plot bin information for numeric data for `'CustAge'` that includes missing data in a separate bin labelled `<missing>`.

```
[bi,cp] = bininfo(sc,'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,33)' }	69	52	1.3269	-0.42156	0.018993
{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549

{ '[46,48)' }	59	25	2.36	0.15424	0.0016199
{ '[48,51)' }	99	41	2.4146	0.17713	0.0035449
{ '[51,58)' }	157	62	2.5323	0.22469	0.0088407
{ '[58,Inf]' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.087112

```
plotbins(sc, 'CustAge')
```

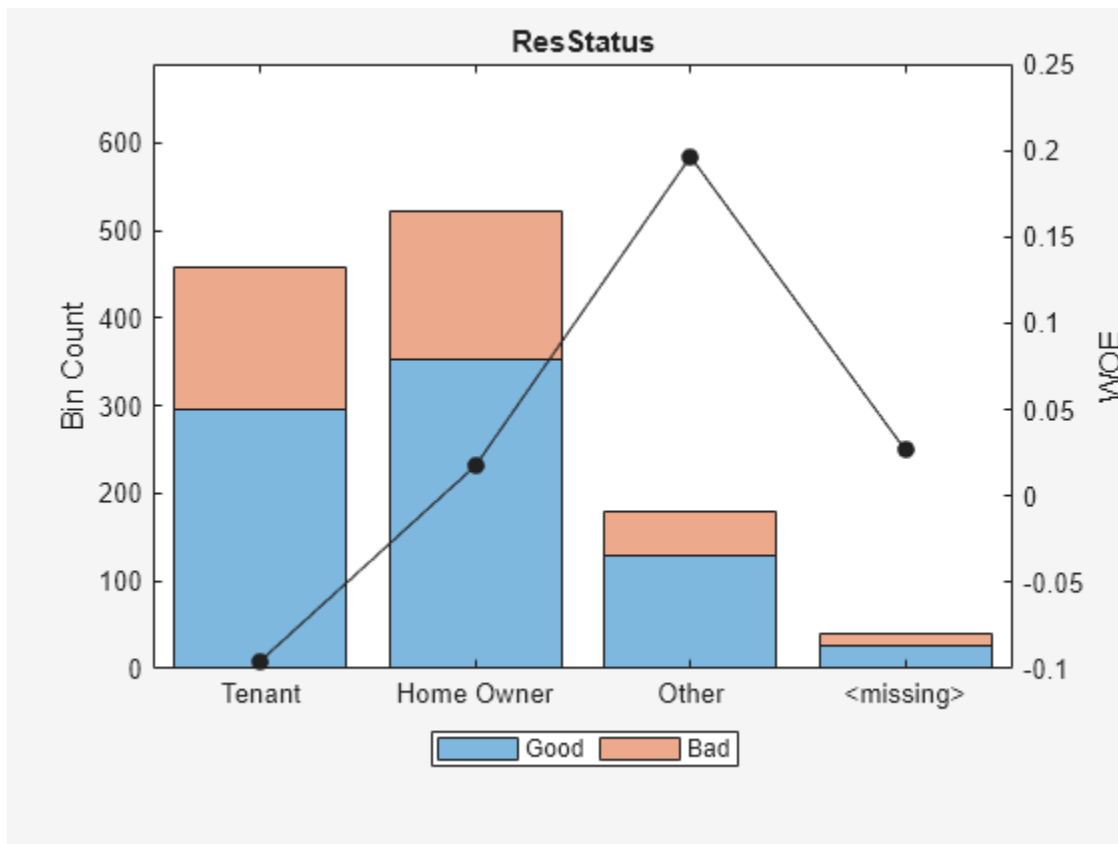


Display and plot bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```

For the 'CustAge' and 'ResStatus' predictors, there is missing data (NaNs and <undefined>) in the training data, and the binning process estimates a WOE value of -0.15787 and 0.026469 respectively for missing data in these predictors, as shown above.

For EmpStatus and CustIncome there is no explicit bin for missing values because the training data has no missing values for these predictors.

```
bi = bininfo(sc,'EmpStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Unknown' }	396	239	1.6569	-0.19947	0.021715
{'Employed' }	407	158	2.5759	0.2418	0.026323
{'Totals' }	803	397	2.0227	NaN	0.048038

```
bi = bininfo(sc,'CustIncome');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[0,29000)' }	53	58	0.91379	-0.79457	0.06364
{'[29000,33000)' }	74	49	1.5102	-0.29217	0.0091366
{'[33000,35000)' }	68	36	1.8889	-0.06843	0.00041042
{'[35000,40000)' }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)' }	68	34	2	-0.011271	1.0819e-05

```

{'[42000,47000)'} 164      66      2.4848      0.20579      0.0078175
{'[47000,Inf]'}   183      56      3.2679      0.47972      0.041657
{'Totals'}        803     397      2.0227      NaN          0.12285

```

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). For predictors that have missing data, there is an explicit `<missing>` bin, with a corresponding WOE value computed from the data. When using `fitmodel`, the corresponding WOE value for the `<missing>` bin is applied when performing the WOE transformation.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1442.8477, Chi2Stat = 4.4974731, PValue = 0.033944979
6. Adding ResStatus, Deviance = 1438.9783, Chi2Stat = 3.86941, PValue = 0.049173805
7. Adding OtherCC, Deviance = 1434.9751, Chi2Stat = 4.0031966, PValue = 0.045414057

Generalized linear regression model:

```

logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial

```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70229	0.063959	10.98	4.7498e-28
CustAge	0.57421	0.25708	2.2335	0.025513
ResStatus	1.3629	0.66952	2.0356	0.04179
EmpStatus	0.88373	0.2929	3.0172	0.002551
CustIncome	0.73535	0.2159	3.406	0.00065929
TmWBank	1.1065	0.23267	4.7556	1.9783e-06
OtherCC	1.0648	0.52826	2.0156	0.043841
AMBalance	1.0446	0.32197	3.2443	0.0011775

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 88.5, p-value = 2.55e-16

Scale the scorecard points by the "points, odds, and points to double the odds (PDO)" method using the `'PointsOddsAndPDO'` argument of `formatpoints`. Suppose that you want a score of 500 points to have odds of 2 (twice as likely to be good than to be bad) and that the odds double every 50 points (so that 550 points would have odds of 4).

Display the scorecard showing the scaled points for predictors retained in the fitting model.

```

sc = formatpoints(sc,'PointsOddsAndPDO',[500 2 50]);
PointsInfo = displaypoints(sc)

```

PointsInfo=38×3 table

Predictors	Bin	Points
{'CustAge' }	{'[0,33) ' }	54.062

```

{'CustAge' } {'[33,37)' } 56.282
{'CustAge' } {'[37,40)' } 60.012
{'CustAge' } {'[40,46)' } 69.636
{'CustAge' } {'[46,48)' } 77.912
{'CustAge' } {'[48,51)' } 78.86
{'CustAge' } {'[51,58)' } 80.83
{'CustAge' } {'[58,Inf]' } 96.76
{'CustAge' } {'<missing>' } 64.984
{'ResStatus'} {'Tenant' } 62.138
{'ResStatus'} {'Home Owner'} 73.248
{'ResStatus'} {'Other' } 90.828
{'ResStatus'} {'<missing>' } 74.125
{'EmpStatus'} {'Unknown' } 58.807
{'EmpStatus'} {'Employed' } 86.937
{'EmpStatus'} {'<missing>' } NaN
:

```

Notice that points for the <missing> bin for CustAge and ResStatus are explicitly shown (as 64.9836 and 74.1250, respectively). These points are computed from the WOE value for the <missing> bin, and the logistic model coefficients.

For predictors that have no missing data in the training set, there is no explicit <missing> bin. By default the points are set to NaN for missing data and they lead to a score of NaN when running score. For predictors that have no explicit <missing> bin, use the name-value argument 'Missing' in formatpoints to indicate how missing data should be treated for scoring purposes.

For the purpose of illustration, take a few rows from the original data as test data and introduce some missing data. Also introduce some invalid, or out-of-range values. For numeric data, values below the minimum (or above the maximum) allowed are considered invalid, such as a negative value for age (recall 'MinValue' was earlier set to 0 for CustAge and CustIncome). For categorical data, invalid values are categories not explicitly included in the scorecard, for example, a residential status not previously mapped to scorecard categories, such as "House", or a meaningless string such as "abc123".

```

tdata = dataMissing(11:18,mdl.PredictorNames); % Keep only the predictors retained in the model
% Set some missing values
tdata.CustAge(1) = NaN;
tdata.ResStatus(2) = missing;
tdata.EmpStatus(3) = missing;
tdata.CustIncome(4) = NaN;
% Set some invalid values
tdata.CustAge(5) = -100;
tdata.ResStatus(6) = 'House';
tdata.EmpStatus(7) = 'Freelancer';
tdata.CustIncome(8) = -1;
disp(tdata)

```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
NaN	Tenant	Unknown	34000	44	Yes	119.8
48	<undefined>	Unknown	44000	14	Yes	403.62
65	Home Owner	<undefined>	48000	6	No	111.88
44	Other	Unknown	NaN	35	No	436.41
-100	Other	Employed	46000	16	Yes	162.21
33	House	Employed	36000	36	Yes	845.02

39	Tenant	Freelancer	34000	40	Yes	756.26
24	Home Owner	Employed	-1	19	Yes	449.61

Score the new data and see how points are assigned for missing `CustAge` and `ResStatus`, because we have an explicit bin with points for `<missing>`. However, for `EmpStatus` and `CustIncome` the `score` function sets the points to `NaN`.

```
[Scores,Points] = score(sc,ldata);
disp(Scores)
```

```
481.2231
520.8353
NaN
NaN
551.7922
487.9588
NaN
NaN
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	NaN	96.969	51.132	50.914	89.922
69.636	90.828	58.807	NaN	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	NaN	67.893	61.858	75.622	63.028
54.062	73.248	86.937	NaN	61.061	75.622	89.922

Use the name-value argument `'Missing'` in `formatpoints` to choose how to assign points to missing values for predictors that do not have an explicit `<missing>` bin. In this example, use the `'MinPoints'` option for the `'Missing'` argument. The minimum points for `EmpStatus` in the scorecard displayed above are 58.8072, and for `CustIncome` the minimum points are 29.3753.

```
sc = formatpoints(sc,'Missing','MinPoints');
[ Scores,Points] = score(sc,ldata);
disp(Scores)
```

```
481.2231
520.8353
517.7532
451.3405
551.7922
487.9588
449.3577
470.2267
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	58.807	96.969	51.132	50.914	89.922

69.636	90.828	58.807	29.375	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	58.807	67.893	61.858	75.622	63.028
54.062	73.248	86.937	29.375	61.061	75.622	89.922

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `sc =`

```
formatpoints(sc, 'BasePoints', true, 'Round', 'AllPoints', 'WorstAndBestScores',
[100, 700])
```

Note `ShiftAndSlope`, `PointsOddsAndPDO`, and `WorstAndBestScores` are scaling methods and you can use only one of these name-value pair arguments at one time. The other three name-value pair arguments (`BasePoints`, `Missing`, and `Round`) are not scaling methods and can be used together or with any one of the three scaling methods.

BasePoints — Indicator for separating base points

false (default) | logical scalar

Indicator for separating base points, specified as the comma-separated pair consisting of `'BasePoints'` and a logical scalar. If `true`, the scorecard explicitly separates base points. If `false`, the base points are spread across all variables in the `creditscorecard` object.

Data Types: char

Missing — Indicator for points assigned to missing or out-of-range information when scoring

NoScore (default) | character vector with values `NoScore`, `ZeroWOE`, `MinPoints`, and `MaxPoints`

Indicator for points assigned to missing or out-of-range information when scoring, specified as the comma-separated pair consisting of `'Missing'` and a character vector with a value for `NoScore`, `ZeroWOE`, `MinPoints`, or `MaxPoints`, where:

- `NoScore` — Missing and out-of-range data do not get points assigned and points are set to `NaN`. Also, the total score is set to `NaN`.
- `ZeroWOE` — Missing or out-of-range data get assigned a zero Weight-of-Evidence (WOE) value.

- **MinPoints** — Missing or out-of-range data get the minimum possible points for that predictor. This penalizes the score if higher scores are better.
- **MaxPoints** — Missing or out-of-range data get the maximum possible points for that predictor. This penalizes the score if lower scores are better.

Note When using the `creditscorecard` name-value argument `'BinMissingData'` with a value of `true`, missing data for numeric and categorical predictors is binned in a separate bin labeled `<missing>`. The `<missing>` bin only contains missing values for a predictor and does not contain invalid or out-of-range values for a predictor.

Data Types: `char`

Round — Indicator whether to round points or scores

`'None'` (default) | character vector with values `'AllPoints'`, `'FinalScore'`

Indicator whether to round points or scores, specified as the comma-separated pair consisting of `'Round'` and a character vector with values `'AllPoints'`, `'FinalScore'` or `'None'`, where:

- **None** — No rounding is applied.
- **AllPoints** — Apply rounding to each predictor's points before adding up the total score.
- **FinalScore** — Round the final score only (rounding is applied after all points are added up).

For more information and an example of using the `'Round'` name-value pair argument, see “Rounding and Default Probabilities” on page 15-1825.

Data Types: `char`

ShiftAndSlope — Indicator for shift and slope scaling parameters

`[0,1]` (default) | numeric array with two elements `[Shift,Slope]`

Indicator for shift and slope scaling parameters for the credit scorecard, specified as the comma-separated pair consisting of `'ShiftAndSlope'` and a numeric array with two elements `[Shift, Slope]`. Slope cannot be zero. The `ShiftAndSlope` values are used scale the scoring model.

Note `ShiftAndSlope`, `PointsOddsAndPDO`, and `WorstAndBestScores` are scaling methods and you can use only one of these name-value pair arguments at one time. The other three name-value pair arguments (`BasePoints`, `Missing`, and `Round`) are not scaling methods and can be used together or with any one of the three scaling methods.

To remove a previous scaling and revert to unscaled scores, set `ShiftAndSlope` to `[0,1]`.

Data Types: `double`

PointsOddsAndPDO — Indicator for target points for given odds and double odds level

numeric array with three elements `[Points,Odds,PDO]`

Indicator for target points (`Points`) for a given odds level (`Odds`) and the desired number of points to double the odds (`PDO`), specified as the comma-separated pair consisting of `'PointsOddsAndPDO'` and a numeric array with three elements `[Points,Odds,PDO]`. `Odds` must be a positive number. The `PointsOddsAndPDO` values are used to find scaling parameters for the scoring model.

Note The points to double the odds (PDO) may be positive or negative, depending on whether higher scores mean lower risk, or vice versa.

`ShiftAndSlope`, `PointsOddsAndPDO`, and `WorstAndBestScores` are scaling methods and you can use only one of these name-value pair arguments at one time. The other three name-value pair arguments (`BasePoints`, `Missing`, and `Round`) are not scaling methods and can be used together or with any one of the three scaling methods.

To remove a previous scaling and revert to unscaled scores, set `ShiftAndSlope` to `[0,1]`.

Data Types: double

WorstAndBestScores — Indicator for worst (highest risk) and best (lowest risk) scores in scorecard

numeric array with two elements [`WorstScore`, `BestScore`]

Indicator for worst (highest risk) and best (lowest risk) scores in the scorecard, specified as the comma-separated pair consisting of '`WorstAndBestScores`' and a numeric array with two elements [`WorstScore`, `BestScore`]. `WorstScore` and `BestScore` must be different values. These `WorstAndBestScores` values are used to find scaling parameters for the scoring model.

Note `WorstScore` means the riskiest score, and its value could be lower or higher than the 'best' score. In other words, the 'minimum' score may be the 'worst' score or the 'best' score, depending on the desired scoring scale.

`ShiftAndSlope`, `PointsOddsAndPDO`, and `WorstAndBestScores` are scaling methods and you can use only one of these name-value pair arguments at one time. The other three name-value pair arguments (`BasePoints`, `Missing`, and `Round`) are not scaling methods and can be used together or with any one of the three scaling methods.

To remove a previous scaling and revert to unscaled scores, set `ShiftAndSlope` to `[0,1]`.

Data Types: double

Output Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model returned as an updated `creditscorecard` object. For more information on using the `creditscorecard` object, see `creditscorecard`.

Algorithms

The score of an individual i is given by the formula

$$\text{Score}(i) = \text{Shift} + \text{Slope} * (b_0 + b_1 * \text{WOE}_1(i) + b_2 * \text{WOE}_2(i) + \dots + b_p * \text{WOE}_p(i))$$

where b_j is the coefficient of the j th variable in the model, and $\text{WOE}_j(i)$ is the Weight of Evidence (WOE) value for the i th individual corresponding to the j th model variable. `Shift` and `Slope` are scaling constants further discussed below. The scaling constant can be controlled with `formatpoints`.

If the data for individual i is in the i -th row of a given dataset, to compute a score, the $\text{data}(i,j)$ is binned using existing binning maps, and converted into a corresponding Weight of Evidence value $\text{WOE}_j(i)$. Using the model coefficients, the unscaled score is computed as

$$s = b_0 + b_1 \cdot \text{WOE}_1(i) + \dots + b_p \cdot \text{WOE}_p(i).$$

For simplicity, assume in the description above that the j -th variable in the model is the j -th column in the data input, although, in general, the order of variables in a given dataset does not have to match the order of variables in the model, and the dataset could have additional variables that are not used in the model.

The formatting options can be controlled using `formatpoints`. When the base points are reported separately (see the `formatpoints` parameter `BasePoints`), the base points are given by

$$\text{Base Points} = \text{Shift} + \text{Slope} \cdot b_0,$$

and the points for the j -th predictor, i -th row are given by

$$\text{Points_ji} = \text{Slope} \cdot (b_j \cdot \text{WOE}_j(i)).$$

By default, the base points are not reported separately, in which case

$$\text{Points_ji} = (\text{Shift} + \text{Slope} \cdot b_0) / p + \text{Slope} \cdot (b_j \cdot \text{WOE}_j(i)),$$

where p is the number of predictors in the scorecard model.

By default, no rounding is applied to the points by the `score` function (`Round` is `None`). If `Round` is set to `AllPoints` using `formatpoints`, then the points for individual i for variable j are given by

$$\text{points if rounding is 'AllPoints': } \text{round}(\text{Points_ji})$$

and, if base points are reported separately, the are also rounded. This yields integer-valued points per predictor, hence also integer-valued scores. If `Round` is set to `FinalScore` using `formatpoints`, then the points per predictor are not rounded, and only the final score is rounded

$$\text{score if rounding is 'FinalScore': } \text{round}(\text{Score}(i)).$$

Regarding the scaling parameters, the `Shift` parameter, and the `Slope` parameter can be set directly with the `ShiftAndSlope` parameter of `formatpoints`. Alternatively, you can use the `formatpoints` parameter for `WorstAndBestScores`. In this case, the parameters `Shift` and `Slope` are found internally by solving the system

$$\begin{aligned} \text{Shift} + \text{Slope} \cdot s_{\min} &= \text{WorstScore}, \\ \text{Shift} + \text{Slope} \cdot s_{\max} &= \text{BestScore}, \end{aligned}$$

where `WorstScore` and `BestScore` are the first and second elements in the `formatpoints` parameter for `WorstAndBestScores` and $s_{\min}$ and $s_{\max}$ are the minimum and maximum possible unscaled scores:

$$\begin{aligned} s_{\min} &= b_0 + \min(b_1 \cdot \text{WOE}_1) + \dots + \min(b_p \cdot \text{WOE}_p), \\ s_{\max} &= b_0 + \max(b_1 \cdot \text{WOE}_1) + \dots + \max(b_p \cdot \text{WOE}_p). \end{aligned}$$

A third alternative to scale scores is the `PointsOddsAndPDO` parameter in `formatpoints`. In this case, assume that the unscaled score s gives the log-odds for a row, and the `Shift` and `Slope` parameters are found by solving the following system

$$\begin{aligned} \text{Points} &= \text{Shift} + \text{Slope} \cdot \log(\text{Odds}) \\ \text{Points} + \text{PDO} &= \text{Shift} + \text{Slope} \cdot \log(2 \cdot \text{Odds}) \end{aligned}$$

where `Points`, `Odds`, and `PDO` ("points to double the odds") are the first, second, and third elements in the `PointsOddsAndPDO` parameter.

Whenever a given dataset has a missing or out-of-range value data (i,j), the points for predictor j , for individual i , are set to `NaN` by default, which results in a missing score for that row (a `NaN` score). Using the `Missing` parameter for `formatpoints`, you can modify this behavior and set the corresponding Weight-of-Evidence (WOE) value to zero, or set the points to the minimum points, or the maximum points for that predictor.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `bindata` | `fitmodel` | `displaypoints` | `score` | `setmodel` | `probdefault` | `validatemodel`

Topics

"Case Study for Credit Scorecard Analysis" on page 8-71

"Credit Scorecard Modeling with Missing Values" on page 8-57

"Troubleshooting Credit Scorecard Results" on page 8-64

"Credit Scorecard Modeling Workflow" on page 8-52

"About Credit Scorecards" on page 8-49

displaypoints

Return points per predictor per bin

Syntax

```
PointsInfo = displaypoints(sc)
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
[PointsInfo,MinScore,MaxScore] = displaypoints( ___,Name,Value)
```

Description

`PointsInfo = displaypoints(sc)` returns a table of points for all bins of all predictor variables used in the `creditscorecard` object after a linear logistic regression model is fit using `fitmodel` to the Weight of Evidence data. The `PointsInfo` table displays information on the predictor name, bin labels, and the corresponding points per bin.

`[PointsInfo,MinScore,MaxScore] = displaypoints(sc)` returns a table of points for all bins of all predictor variables used in the `creditscorecard` object after a linear logistic regression model is fit (`fitmodel`) to the Weight of Evidence data. The `PointsInfo` table displays information on the predictor name, bin labels, and the corresponding points per bin and `displaypoints`. In addition, the optional `MinScore` and `MaxScore` values are returned.

`[PointsInfo,MinScore,MaxScore] = displaypoints( ___,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Display Unscaled Points

This example shows how to use `displaypoints` after a model is fitted to compute the unscaled points per bin, for a given predictor in the `creditscorecard` model.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the 'IDVar' argument in the `creditscorecard` function to indicate that 'CustID' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data,'IDVar','CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06

3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Display unscaled points for predictors retained in the fitting model.

`PointsInfo = displaypoints(sc)`

PointsInfo=37x3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.15894
{'CustAge' }	{'[33,37)' }	-0.14036
{'CustAge' }	{'[37,40)' }	-0.060323
{'CustAge' }	{'[40,46)' }	0.046408
{'CustAge' }	{'[46,48)' }	0.21445
{'CustAge' }	{'[48,58)' }	0.23039
{'CustAge' }	{'[58,Inf]' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252
{'ResStatus' }	{'Home Owner' }	0.12696
{'ResStatus' }	{'Other' }	0.37641
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.076317
{'EmpStatus' }	{'Employed' }	0.31449
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	-0.45716
:		

`displaypoints` always displays a '<missing>' bin for each predictor. The value of the '<missing>' bin comes from the initial `creditscorecard` object, and the '<missing>' bin is set to NaN whenever the scorecard model has no information on how to assign points to missing data.

To configure the points for the '`<missing>`' bin, you must use the initial `creditscorecard` object. For predictors that have missing values in the training set, the points for the '`<missing>`' bin are estimated from the data if the '`BinMissingData`' name-value pair argument is set to `true` using `creditscorecard`. When the '`BinMissingData`' parameter is set to `false`, or when the data contains no missing values in the training set, use the '`Missing`' name-value pair argument in `formatpoints` to indicate how to assign points to the missing data.

Display Unscaled Points When Using Missing Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

Number of rows: 1200

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

Number of missing values CustAge: 30

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

Number of missing values ResStatus: 40

Use `creditscorecard` with the name-value argument `'BinMissingData'` set to `true` to bin the missing numeric or categorical data in a separate bin. Apply automatic binning.

```
sc = creditscorecard(dataMissing, 'IDVar', 'CustID', 'BinMissingData', true);
sc = autobinning(sc);
```

```
disp(sc)
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'Uti
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 1
    IDVar: 'CustID'

```

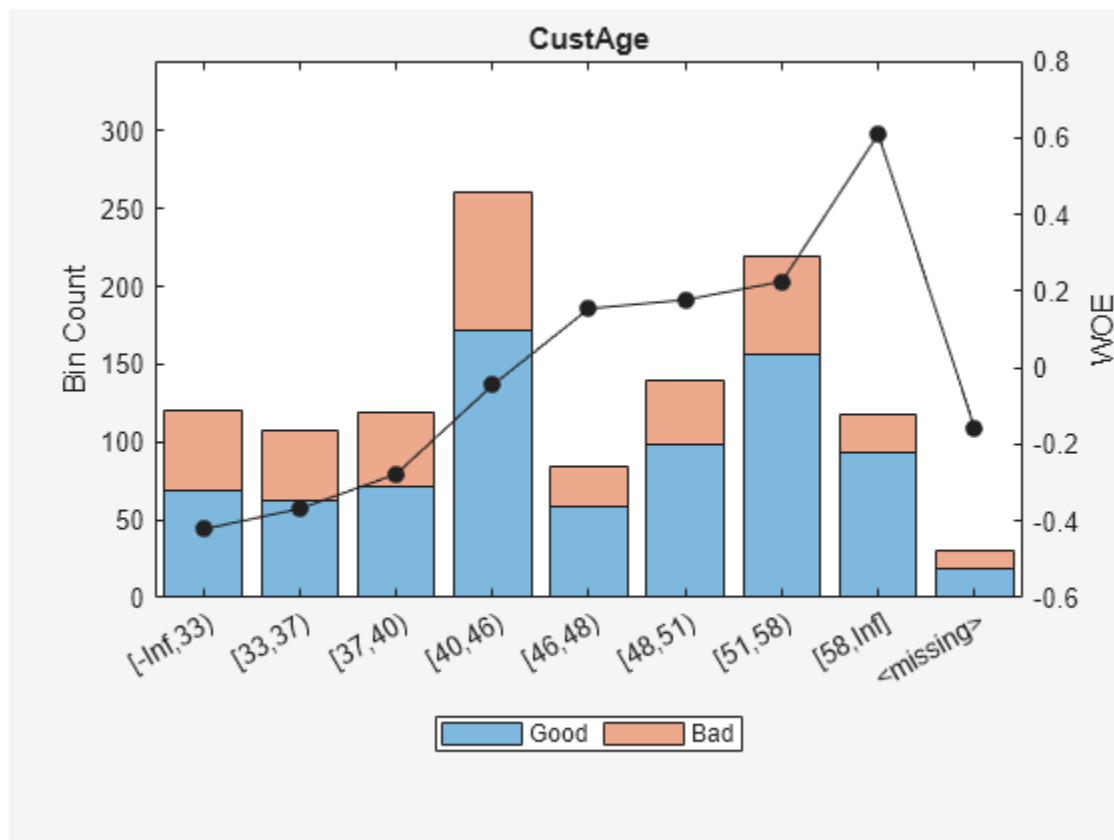
```
PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'Tr
Data: [1200x11 table]
```

Display and plot bin information for numeric data for 'CustAge' that includes missing data in a separate bin labelled <missing>.

```
[bi,cp] = bininfo(sc,'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'} }	69	52	1.3269	-0.42156	0.018993
{'[33,37)'} }	63	45	1.4	-0.36795	0.012839
{'[37,40)'} }	72	47	1.5319	-0.2779	0.0079824
{'[40,46)'} }	172	89	1.9326	-0.04556	0.0004549
{'[46,48)'} }	59	25	2.36	0.15424	0.0016199
{'[48,51)'} }	99	41	2.4146	0.17713	0.0035449
{'[51,58)'} }	157	62	2.5323	0.22469	0.0088407
{'[58,Inf]'} }	93	25	3.72	0.60931	0.032198
{'<missing>'} }	19	11	1.7273	-0.15787	0.00063885
{'Totals' }	803	397	2.0227	NaN	0.087112

```
plotbins(sc,'CustAge')
```

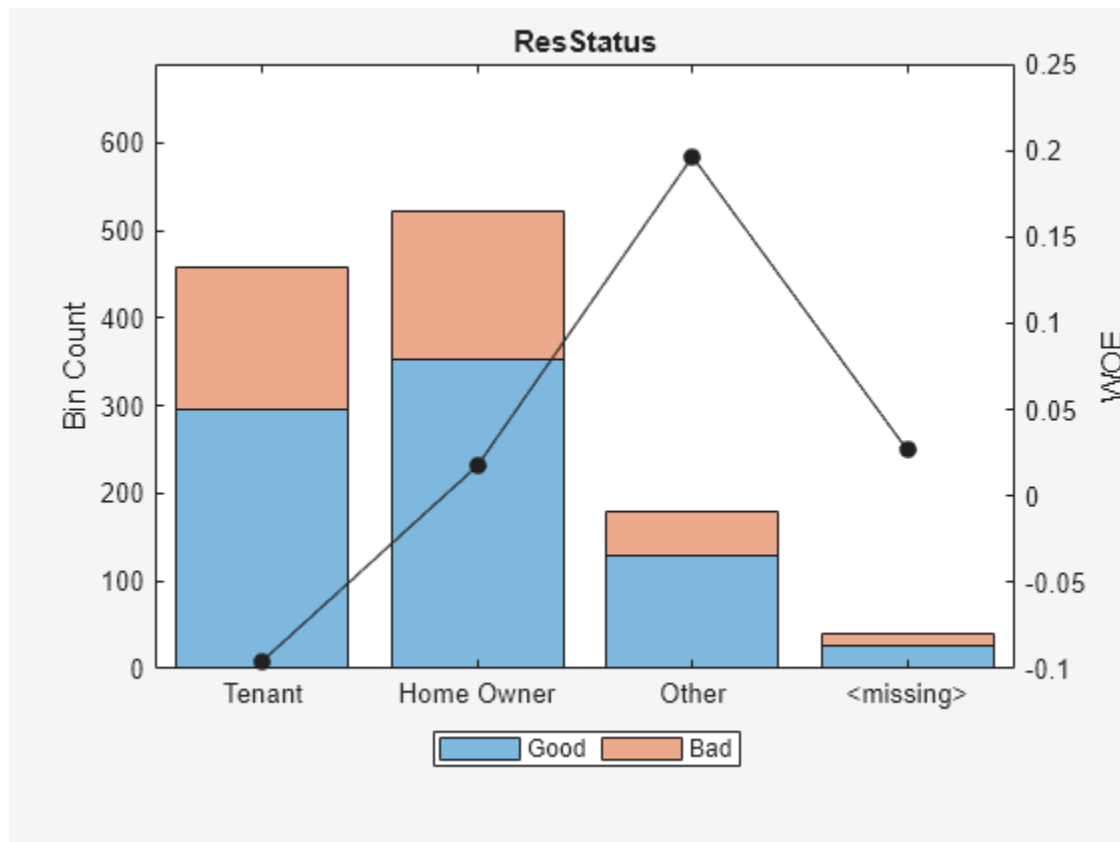


Display and plot bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



For the 'CustAge' and 'ResStatus' predictors, there is missing data (NaNs and <undefined>) in the training data, and the binning process estimates a WOE value of -0.15787 and 0.026469 respectively for missing data in these predictors, as shown above.

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). For predictors that have missing data, there is an explicit <missing> bin, with a corresponding WOE value computed from the data. When using `fitmodel`, the corresponding WOE value for the <missing> bin is applied when performing the WOE transformation.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1442.8477, Chi2Stat = 4.4974731, PValue = 0.033944979
6. Adding ResStatus, Deviance = 1438.9783, Chi2Stat = 3.86941, PValue = 0.049173805
7. Adding OtherCC, Deviance = 1434.9751, Chi2Stat = 4.0031966, PValue = 0.045414057

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70229	0.063959	10.98	4.7498e-28
CustAge	0.57421	0.25708	2.2335	0.025513
ResStatus	1.3629	0.66952	2.0356	0.04179
EmpStatus	0.88373	0.2929	3.0172	0.002551
CustIncome	0.73535	0.2159	3.406	0.00065929
TmWBank	1.1065	0.23267	4.7556	1.9783e-06
OtherCC	1.0648	0.52826	2.0156	0.043841
AMBalance	1.0446	0.32197	3.2443	0.0011775

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 88.5, p-value = 2.55e-16

Display unscaled points for predictors retained in the fitting model (to scale points use `formatpoints`).

`PointsInfo = displaypoints(sc)`

PointsInfo=38x3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.14173
{'CustAge' }	{'[33,37)' }	-0.11095
{'CustAge' }	{'[37,40)' }	-0.059244
{'CustAge' }	{'[40,46)' }	0.074167
{'CustAge' }	{'[46,48)' }	0.1889
{'CustAge' }	{'[48,51)' }	0.20204
{'CustAge' }	{'[51,58)' }	0.22935
{'CustAge' }	{'[58,Inf]' }	0.45019
{'CustAge' }	{'<missing>' }	0.0096749
{'ResStatus' }	{'Tenant' }	-0.029778
{'ResStatus' }	{'Home Owner' }	0.12425
{'ResStatus' }	{'Other' }	0.36796
{'ResStatus' }	{'<missing>' }	0.1364
{'EmpStatus' }	{'Unknown' }	-0.075948
{'EmpStatus' }	{'Employed' }	0.31401
{'EmpStatus' }	{'<missing>' }	NaN
:		

Notice that points for the <missing> bin for `CustAge` and `ResStatus` are explicitly shown. These points are computed from the WOE value for the <missing> bin and the logistic model coefficients.

For predictors that have no missing data in the training set, there is no explicit <missing> bin, and by default the points are set to NaN for missing data, and they lead to a score of NaN when running `score`. For predictors that have no explicit <missing> bin, use the name-value argument 'Missing' in `formatpoints` to indicate how missing data should be treated for scoring purposes.

Display Scaled Points

This example shows how to use `formatpoints` after a model is fitted to format scaled points, and then use `displaypoints` to display the scaled points per bin, for a given predictor in the `creditscorecard` model.

Points become scaled when a range is defined. Specifically, a linear transformation from the unscaled to the scaled points is necessary. This transformation is defined either by supplying a shift and slope or by specifying the worst and best scores possible. (For more information, see `formatpoints`.)

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the 'IDVar' argument in the `creditscorecard` function to indicate that 'CustID' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding `CustIncome`, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding `TmWBank`, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding `AMBalance`, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding `EmpStatus`, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding `CustAge`, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding `ResStatus`, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding `OtherCC`, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179

TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Use the `formatpoints` function to scale providing the 'Worst' and 'Best' score values. The range provided below is a common score range.

```
sc = formatpoints(sc, 'WorstAndBestScores', [300 850]);
```

Display the points information again to verify that the points are now scaled and also display the scaled minimum and maximum scores.

```
[PointsInfo, MinScore, MaxScore] = displaypoints(sc)
```

PointsInfo=37×3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	46.396
{'CustAge' }	{'[33,37)' }	48.727
{'CustAge' }	{'[37,40)' }	58.772
{'CustAge' }	{'[40,46)' }	72.167
{'CustAge' }	{'[46,48)' }	93.256
{'CustAge' }	{'[48,58)' }	95.256
{'CustAge' }	{'[58,Inf]' }	126.46
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	62.421
{'ResStatus' }	{'Home Owner' }	82.276
{'ResStatus' }	{'Other' }	113.58
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	56.765
{'EmpStatus' }	{'Employed' }	105.81
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	8.9706
:		

```
MinScore =
300.0000
```

```
MaxScore =
850.0000
```

Notice that, as expected, the values of `MinScore` and `MaxScore` correspond to the worst and best possible scores.

Separate the Base Points From the Total Points

This example shows how to use `displaypoints` after a model is fitted to separate the base points from the rest of the points assigned to each predictor variable. The name-value pair argument

'BasePoints' in the `formatpoints` function is a boolean that serves this purpose. By default, the base points are spread across all variables in the scorecard.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the 'IDVar' argument in the `creditscorecard` function to indicate that 'CustID' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Use the `formatpoints` function to separate the base points by providing the 'BasePoints' name-value pair argument.

```
sc = formatpoints(sc, 'BasePoints', true);
```

Display the base points, separated out from the other points, for predictors retained in the fitting model.

```
PointsInfo = displaypoints(sc)
```

PointsInfo=38x3 table

Predictors	Bin	Points
{'BasePoints' }	{'BasePoints' }	0.70239
{'CustAge' }	{'[-Inf,33)' }	-0.25928
{'CustAge' }	{'[33,37)' }	-0.24071
{'CustAge' }	{'[37,40)' }	-0.16066
{'CustAge' }	{'[40,46)' }	-0.053933
{'CustAge' }	{'[46,48)' }	0.11411
{'CustAge' }	{'[48,58)' }	0.13005
{'CustAge' }	{'[58,Inf]' }	0.37866
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.13159
{'ResStatus' }	{'Home Owner' }	0.026616
{'ResStatus' }	{'Other' }	0.27607
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.17666
{'EmpStatus' }	{'Employed' }	0.21415
{'EmpStatus' }	{'<missing>' }	NaN
:	:	:

Display Points After Modifying Bin Labels

This example shows how to use `displaypoints` after a model is fitted and the `modifybins` function is used to provide user-defined bin labels for a numeric predictor.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument in the `creditscorecard` function to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data,'IDVar','CustID');
```

Perform automatic binning to bin for all predictors.

```
sc = autobinning(sc);
```

Fit a linear regression model using default parameters.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Use the `displaypoints` function to display point information.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

PointsInfo=37×3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.15894
{'CustAge' }	{'[33,37)' }	-0.14036
{'CustAge' }	{'[37,40)' }	-0.060323
{'CustAge' }	{'[40,46)' }	0.046408
{'CustAge' }	{'[46,48)' }	0.21445
{'CustAge' }	{'[48,58)' }	0.23039
{'CustAge' }	{'[58,Inf]' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252
{'ResStatus' }	{'Home Owner' }	0.12696
{'ResStatus' }	{'Other' }	0.37641
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.076317
{'EmpStatus' }	{'Employed' }	0.31449
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	-0.45716
⋮		

```
MinScore =  
-1.3100
```

```
MaxScore =  
3.0726
```

Use the `modifybins` function to specify user-defined bin labels for 'CustAge' so that the bin ranges are described in natural language.

```
labels = {'Up to 32', '33 to 36', '37 to 39', '40 to 45', '46 to 47', '48 to 57', 'At least 58'};  
sc = modifybins(sc, 'CustAge', 'BinLabels', labels);
```

Rerun `displaypoints` to verify the updated bin labels.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc)
```

```
PointsInfo=37x3 table
```

Predictors	Bin	Points
{'CustAge' }	{'Up to 32' }	-0.15894
{'CustAge' }	{'33 to 36' }	-0.14036
{'CustAge' }	{'37 to 39' }	-0.060323
{'CustAge' }	{'40 to 45' }	0.046408
{'CustAge' }	{'46 to 47' }	0.21445
{'CustAge' }	{'48 to 57' }	0.23039
{'CustAge' }	{'At least 58' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252
{'ResStatus' }	{'Home Owner' }	0.12696
{'ResStatus' }	{'Other' }	0.37641
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.076317
{'EmpStatus' }	{'Employed' }	0.31449
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	-0.45716
:		

```
MinScore =  
-1.3100
```

```
MaxScore =  
3.0726
```

Compute the Predictor Weights

This example shows how to use a credit scorecard to compute the weights of the predictors. The weights of the predictors are determined from the range of points of each predictor, divided by the total range of points for the scorecard. The points for the scorecard not only take into consideration the betas, but also implicitly the binning of the predictor values and the corresponding weights of evidence.

Create a scorecard.

```
load CreditCardData.mat  
sc = creditscorecard(data,'IDVar','CustID');  
sc = autobinning(sc);  
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBal
```

Distribution = Binomial

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Compute scorecard points and the MinPts and MaxPts scores.

```
sc = formatpoints(sc, 'PointsOddsAndPD0', [500 2 50]);
[PointsTable, MinPts, MaxPts] = displaypoints(sc);
PtsRange = MaxPts - MinPts;
disp(PointsTable(1:10, :));
```

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	52.821
{'CustAge' }	{'[33,37)' }	54.161
{'CustAge' }	{'[37,40)' }	59.934
{'CustAge' }	{'[40,46)' }	67.633
{'CustAge' }	{'[46,48)' }	79.755
{'CustAge' }	{'[48,58)' }	80.905
{'CustAge' }	{'[58,Inf]' }	98.838
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	62.031
{'ResStatus' }	{'Home Owner' }	73.444

```
fprintf('Min points: %g, Max points: %g\n', MinPts, MaxPts);
```

Min points: 355.505, Max points: 671.64

Compute the predictor weights.

```
Predictor = unique(PointsTable.Predictors, 'stable');
NumPred = length(Predictor);
Weight = zeros(NumPred, 1);
for ii=1:NumPred
    Ind = cellfun(@(x) strcmpi(Predictor{ii}, x), PointsTable.Predictors);
    MaxPtsPred = max(PointsTable.Points(Ind));
    MinPtsPred = min(PointsTable.Points(Ind));
    Weight(ii) = 100*(MaxPtsPred - MinPtsPred)/PtsRange;
end

PredictorWeights = table(Predictor, Weight);
PredictorWeights(end+1, :) = PredictorWeights(end, :);
```

```
PredictorWeights.Predictor{end} = 'Total';
PredictorWeights.Weight(end) = sum(Weight);
disp(PredictorWeights)
```

Predictor	Weight
{'CustAge' }	14.556
{'ResStatus' }	9.302
{'EmpStatus' }	8.9174
{'CustIncome' }	20.401
{'TmWBank' }	25.884
{'OtherCC' }	7.9885
{'AMBalance' }	12.951
{'Total' }	100

The weights are defined as the range of points for the predictor divided by the range of points for the scorecard.

Display Points for creditcorecard Object That Contains Missing Data

To create a `creditcorecard` object using the `CreditCardData.mat` file, load the data (using a dataset from Refaat 2011). Using the `dataMissing` dataset, set the `'BinMissingData'` indicator to `true`.

```
load CreditCardData.mat
sc = creditcorecard(dataMissing, 'BinMissingData', true);
```

Use autobinning with the `creditcorecard` object.

```
sc = autobinning(sc);
```

The binning map or rules for categorical data are summarized in a "category grouping" table, returned as an optional output. By default, each category is placed in a separate bin. Here is the information for the predictor `ResStatus`.

```
[bi, cg] = bininfo(sc, 'ResStatus')
```

```
bi=5x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____
{'Tenant'   }    296    161    1.8385   -0.095463    0.0035249
{'Home Owner'}    352    171    2.0585    0.017549    0.00013382
{'Other'    }    128     52    2.4615     0.19637     0.0055808
{'<missing>' }     27     13    2.0769    0.026469    2.3248e-05
{'Totals'   }    803    397    2.0227         NaN     0.0092627
```

```
cg=3x2 table
      Category      BinNumber
      _____
{'Tenant'   }         1
{'Home Owner'}         2
```

```
{'Other'      }      3
```

To group categories 'Tenant' and 'Other', modify the category grouping table `cg`, so the bin number for 'Other' is the same as the bin number for 'Tenant'. Then use `modifybins` to update the `creditscorecard` object.

```
cg.BinNumber(3) = 2;
sc = modifybins(sc, 'ResStatus', 'Catg', cg);
```

Display the updated bin information using `bininfo`. Note that the bin labels has been updated and that the bin membership information is contained in the category grouping `cg`.

```
[bi,cg] = bininfo(sc, 'ResStatus')
```

```
bi=4x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____
{'Group1'  }      296      161      1.8385     -0.095463      0.0035249
{'Group2'  }      480      223      2.1525      0.062196      0.0022419
{'<missing>'}       27       13      2.0769      0.026469      2.3248e-05
{'Totals'  }      803      397      2.0227           NaN      0.00579
```

```
cg=3x2 table
      Category      BinNumber
      _____
{'Tenant'      }          1
{'Home Owner'  }          2
{'Other'       }          2
```

Use `formatpoints` with the 'Missing' name-value pair argument to indicate that missing data is assigned 'maxpoints'.

```
sc = formatpoints(sc, 'BasePoints', true, 'Missing', 'maxpoints', 'WorstAndBest', [300 800]);
```

Use `fitmodel` to fit the model.

```
sc = fitmodel(sc, 'VariableSelection', 'fullmodel', 'Display', 'Off');
```

Then use `displaypoints` (Risk Management Toolbox) with the `creditscorecard` object to return a table of points for all bins of all predictor variables used in the `compactCreditScorecard` object. By setting the `displaypoints` (Risk Management Toolbox) name-value pair argument for 'ShowCategoricalMembers' to `true`, all the members contained in each individual group are displayed.

```
[PointsInfo,MinScore,MaxScore] = displaypoints(sc, 'ShowCategoricalMembers', true)
```

```
PointsInfo=51x3 table
      Predictors      Bin      Points
      _____
{'BasePoints'  } {'BasePoints'}      535.25
{'CustID'      } {'[-Inf,121)'}      12.085
{'CustID'      } {'[121,241)'}      5.4738
```



```

{'CustID'      } {'[241,1081]'} -1.4061
{'CustID'      } {'[1081,Inf]'} -7.2217
{'CustID'      } {'<missing>'} 12.085
{'CustAge'     } {'[-Inf,33)'} -25.973
{'CustAge'     } {'[33,37)'} -22.67
{'CustAge'     } {'[37,40)'} -17.122
{'CustAge'     } {'[40,46)'} -2.8071
{'CustAge'     } {'[46,48)'} 9.5034
{'CustAge'     } {'[48,51)'} 10.913
{'CustAge'     } {'[51,58)'} 13.844
{'CustAge'     } {'[58,Inf]'} 37.541
{'CustAge'     } {'<missing>'} -9.7271
{'TmAtAddress' } {'[-Inf,23)'} -9.3683
:

```

```

MinScore =
300.0000

```

```

MaxScore =
800.0000

```

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

```

Example: [PointsInfo,MinScore,MaxScore] = displaypoints(sc,
'ShowCategoricalMembers',true)

```

ShowCategoricalMembers — Indicator for how to display bins labels of categories that were grouped together

false (default) | true or false

Indicator for how to display bins labels of categories that were grouped together, specified as the comma-separated pair consisting of `'ShowCategoricalMembers'` and a logical scalar with a value of true or false.

By default, when `'ShowCategoricalMembers'` is false, bin labels are displayed as `Group1`, `Group2`, ..., `Groupn`, or if the bin labels were modified in `creditscorecard`, then the user-defined bin label names are displayed.

If `'ShowCategoricalMembers'` is true, all the members contained in each individual group are displayed.

Data Types: `logical`

Output Arguments

PointsInfo — One row per bin, per predictor, with the corresponding points

table

One row per bin, per predictor, with the corresponding points, returned as a table. For example:

Predictors	Bin	Points
Predictor_1	Bin_11	Points_11
Predictor_1	Bin_12	Points_12
Predictor_1	Bin_13	Points_13
	...	...
Predictor_1	'<missing>'	NaN (Default)
Predictor_2	Bin_21	Points_21
Predictor_2	Bin_22	Points_22
Predictor_2	Bin_23	Points_23
	...	...
Predictor_2	'<missing>'	NaN (Default)
Predictor_j	Bin_ji	Points_ji
	...	...
Predictor_j	'<missing>'	NaN (Default)

`displaypoints` always displays a '<missing>' bin for each predictor. The value of the '<missing>' bin comes from the initial `creditscorecard` object, and the '<missing>' bin is set to NaN whenever the scorecard model has no information on how to assign points to missing data.

To configure the points for the '<missing>' bin, you must use the initial `creditscorecard` object. For predictors that have missing values in the training set, the points for the '<missing>' bin are estimated from the data if the 'BinMissingData' name-value pair argument for is set to `true` using `creditscorecard`. When the 'BinMissingData' parameter is set to `false`, or when the data contains no missing values in the training set, use the 'Missing' name-value pair argument in `formatpoints` to indicate how to assign points to the missing data.

Another option is to use `fillmissing` to specify replacement "fill" values for predictors with a NaN or <undefined> value. If you use `fillmissing`, then the `displaypoints` '<missing>' row has the same points as the bin associated with the fill value.

When base points are reported separately (see `formatpoints`), the first row of the returned `PointsInfo` table contains the base points.

MinScore — Minimum possible total score

scalar

Minimum possible total score, returned as a scalar.

Note Minimum score is the lowest possible total score in the mathematical sense, independently of whether a low score means high risk or low risk.

MaxScore — Maximum possible total score

scalar

Maximum possible total score, returned as a scalar.

Note Maximum score is the highest possible total score in the mathematical sense, independently of whether a high score means high risk or low risk.

Algorithms

The points for predictor j and bin i are, by default, given by

$$\text{Points_ji} = (\text{Shift} + \text{Slope} * b_0) / p + \text{Slope} * (b_j * \text{WOE}_j(i))$$

where b_j is the model coefficient of predictor j , p is the number of predictors in the model, and $\text{WOE}_j(i)$ is the Weight of Evidence (WOE) value for the i -th bin corresponding to the j -th model predictor. Shift and Slope are scaling constants.

When the base points are reported separately (see the `formatpoints` name-value pair argument `BasePoints`), the base points are given by

$$\text{Base Points} = \text{Shift} + \text{Slope} * b_0,$$

and the points for the j -th predictor, i -th row are given by

$$\text{Points_ji} = \text{Slope} * (b_j * \text{WOE}_j(i)).$$

By default, the base points are not reported separately.

The minimum and maximum scores are:

$$\begin{aligned} \text{MinScore} &= \text{Shift} + \text{Slope} * b_0 + \min(\text{Slope} * b_1 * \text{WOE}_1) + \dots + \min(\text{Slope} * b_p * \text{WOE}_p), \\ \text{MaxScore} &= \text{Shift} + \text{Slope} * b_0 + \max(\text{Slope} * b_1 * \text{WOE}_1) + \dots + \max(\text{Slope} * b_p * \text{WOE}_p). \end{aligned}$$

Use `formatpoints` to control the way points are scaled, rounded, and whether the base points are reported separately. See `formatpoints` for more information on format parameters and for details and formulas on these formatting options.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `fillmissing` | `modifybins` | `bindata` | `fitmodel` | `formatpoints` | `score` | `setmodel` | `probdefault` | `validatemodel`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71
“Credit Scorecard Modeling with Missing Values” on page 8-57
“Troubleshooting Credit Scorecard Results” on page 8-64
“Credit Scorecard Modeling Workflow” on page 8-52
“About Credit Scorecards” on page 8-49

fitmodel

Fit logistic regression model to Weight of Evidence (WOE) data

Syntax

```
sc = fitmodel(sc)

[sc,mdl] = fitmodel(sc)
[sc,mdl] = fitmodel( ___,Name,Value)
```

Description

`sc = fitmodel(sc)` fits a logistic regression model to the Weight of Evidence (WOE) data and stores the model predictor names and corresponding coefficients in the `creditscorecard` object.

`fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic or manual binning process. The response variable is mapped so that "Good" is 1, and "Bad" is 0. This implies that higher (unscaled) scores correspond to better (less risky) individuals (smaller probability of default).

Alternatively, you can use `setmodel` to provide names of the predictors that you want in the logistic regression model, along with their corresponding coefficients.

`[sc,mdl] = fitmodel(sc)` fits a logistic regression model to the Weight of Evidence (WOE) data and stores the model predictor names and corresponding coefficients in the `creditscorecard` object. `fitmodel` returns an updated `creditscorecard` object and a `GeneralizedLinearModel` object containing the fitted model.

`fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic or manual binning process. The response variable is mapped so that "Good" is 1, and "Bad" is 0. This implies that higher (unscaled) scores correspond to better (less risky) individuals (smaller probability of default).

Alternatively, you can use `setmodel` to provide names of the predictors that you want in the logistic regression model, along with their corresponding coefficients.

`[sc,mdl] = fitmodel( ___,Name,Value)` fits a logistic regression model to the Weight of Evidence (WOE) data using optional name-value pair arguments and stores the model predictor names and corresponding coefficients in the `creditscorecard` object. Using name-value pair arguments, you can select which Generalized Linear Model to fit the data. `fitmodel` returns an updated `creditscorecard` object and a `GeneralizedLinearModel` object containing the fitted model.

Examples

Fit a Stepwise Logistic Model

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID')

sc =
  creditscorecard with properties:

      GoodLabel: 0
      ResponseVar: 'status'
      WeightsVar: ''
      VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
      NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
      CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
      BinMissingData: 0
      IDVar: 'CustID'
      PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
      Data: [1200x11 table]
```

Perform automatic binning.

```
sc = autobinning(sc)

sc =
  creditscorecard with properties:

      GoodLabel: 0
      ResponseVar: 'status'
      WeightsVar: ''
      VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
      NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
      CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
      BinMissingData: 0
      IDVar: 'CustID'
      PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
      Data: [1200x11 table]
```

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default).

```
sc = fitmodel(sc);

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769
```

```
Generalized linear regression model:
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

```
Estimated Coefficients:
      Estimate      SE      tStat      pValue
```

(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Fit a Stepwise Logistic Model For a `creditscorecard` Object Containing Weights

Use the `CreditCardData.mat` file to load the data (`dataWeights`) that contains a column (`RowWeights`) for the weights (using a dataset from Refaat 2011).

load `CreditCardData`

Create a `creditscorecard` object using the optional name-value pair argument for `'WeightsVar'`.

```
sc = creditscorecard(dataWeights, 'IDVar', 'CustID', 'WeightsVar', 'RowWeights')
```

sc =

creditscorecard with properties:

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: 'RowWeights'
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    Data: [1200x12 table]

```

Perform automatic binning.

```
sc = autobinning(sc)
```

sc =

creditscorecard with properties:

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: 'RowWeights'
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0

```

```
IDVar: 'CustID'
PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
Data: [1200x12 table]
```

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). When the optional name-value pair argument `'WeightsVar'` is used to specify observation (sample) weights, the `mdl` output uses the weighted counts with `stepwiseglm` and `fitglm`.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 764.3187, Chi2Stat = 15.81927, PValue = 6.968927e-05
2. Adding TmWBank, Deviance = 751.0215, Chi2Stat = 13.29726, PValue = 0.0002657942
3. Adding AMBalance, Deviance = 743.7581, Chi2Stat = 7.263384, PValue = 0.007037455

Generalized linear regression model:

```
logit(status) ~ 1 + CustIncome + TmWBank + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70642	0.088702	7.964	1.6653e-15
CustIncome	1.0268	0.25758	3.9862	6.7132e-05
TmWBank	1.0973	0.31294	3.5063	0.0004543
AMBalance	1.0039	0.37576	2.6717	0.0075464

1200 observations, 1196 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 36.4, p-value = 6.22e-08

Fit a Logistic Model with All Predictors

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
```

```
sc = creditscorecard(data,'IDVar','CustID')
```

```
sc =
```

```
creditscorecard with properties:
```

```
GoodLabel: 0
ResponseVar: 'status'
WeightsVar: ''
VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
BinMissingData: 0
IDVar: 'CustID'
```



```
PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilRate'}
Data: [1200x11 table]
```

Perform automatic binning.

```
sc = autobinning(sc, 'Algorithm', 'EqualFrequency')
```

```
sc =
```

```
creditscorecard with properties:
```

```
    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilRate'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilRate'}
    Data: [1200x11 table]
```

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. Set the `VariableSelection` name-value pair argument to `FullModel` to specify that all predictors must be included in the fitted logistic regression model.

```
sc = fitmodel(sc, 'VariableSelection', 'FullModel');
```

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + TmAtAddress + ResStatus + EmpStatus + CustIncome + TmWBank + AMBalance + UtilRate
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70262	0.063862	11.002	3.734e-28
CustAge	0.57683	0.27064	2.1313	0.033062
TmAtAddress	1.0653	0.55233	1.9287	0.053762
ResStatus	1.4189	0.65162	2.1775	0.029441
EmpStatus	0.89916	0.29217	3.0776	0.002087
CustIncome	0.77506	0.21942	3.5323	0.0004119
TmWBank	1.0826	0.26583	4.0727	4.648e-05
OtherCC	1.1354	0.52827	2.1493	0.031612
AMBalance	0.99315	0.32642	3.0425	0.0023459
UtilRate	0.16723	0.55745	0.29999	0.76419

1200 observations, 1190 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 85.6, p-value = 1.25e-14

Fit a Stepwise Logistic Model When Using Missing Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

```
Number of rows: 1200
```

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

```
Number of missing values CustAge: 30
```

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

```
Number of missing values ResStatus: 40
```

Use `creditscorecard` with the name-value argument `'BinMissingData'` set to `true` to bin the missing numeric or categorical data in a separate bin.

```
sc = creditscorecard(dataMissing,'IDVar','CustID','BinMissingData',true);
sc = autobinning(sc);
disp(sc)
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
  ResponseVar: 'status'
   WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBBalance' 'UtilizationRate'}
CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
  BinMissingData: 1
         IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
          Data: [1200x11 table]
```

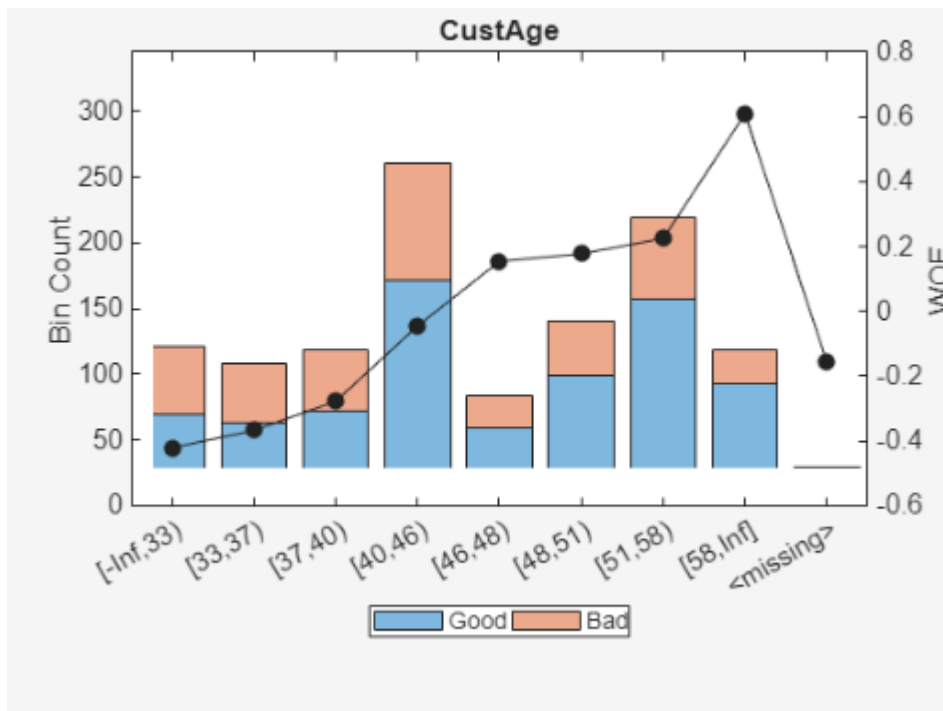
Display and plot bin information for numeric data for `'CustAge'` that includes missing data in a separate bin labelled `<missing>`.

```
[bi,cp] = bininfo(sc,'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'} {'33,50'] {'50,60'] {'60,68'] {'68,75'] {'75,Inf']	69 52 52 52 52 52	52 52 52 52 52 52	1.3269 1.3269 1.3269 1.3269 1.3269 1.3269	-0.42156 -0.42156 -0.42156 -0.42156 -0.42156 -0.42156	0.018993 0.018993 0.018993 0.018993 0.018993 0.018993

{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549
{ '[46,48)' }	59	25	2.36	0.15424	0.0016199
{ '[48,51)' }	99	41	2.4146	0.17713	0.0035449
{ '[51,58)' }	157	62	2.5323	0.22469	0.0088407
{ '[58,Inf]' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.087112

```
plotbins(sc, 'CustAge')
```

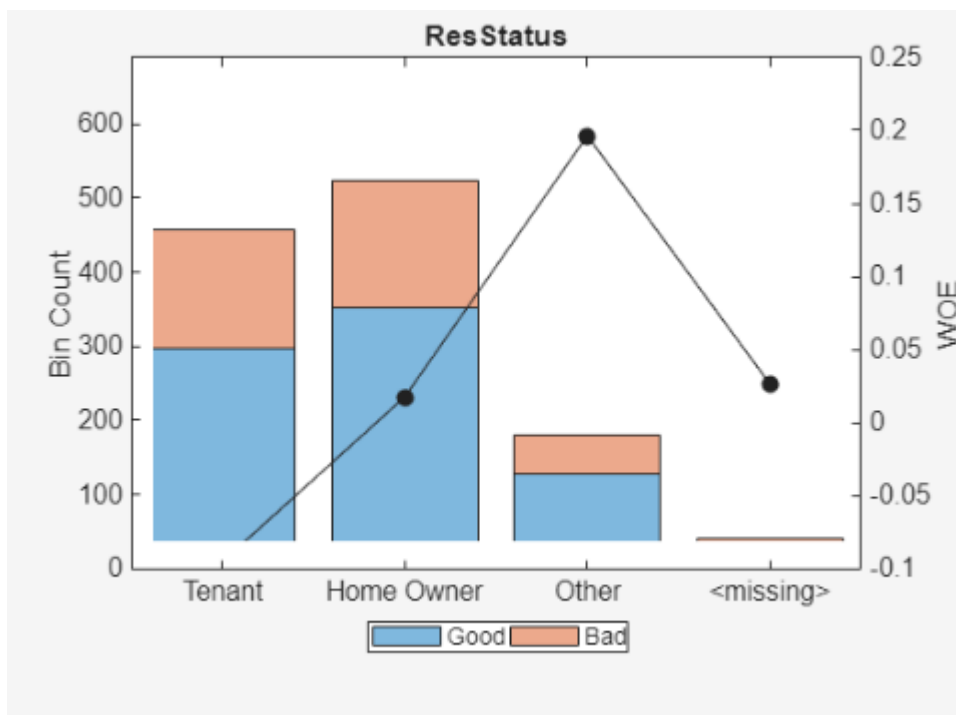


Display and plot bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). For predictors that have missing data, there is an explicit `<missing>` bin, with a corresponding WOE value computed from the data. When using `fitmodel`, the corresponding WOE value for the `<missing>` bin is applied when performing the WOE transformation. For example, a missing value for customer age (`CustAge`) is replaced with `-0.15787` which is the WOE value for the `<missing>` bin for the `CustAge` predictor. However, when `'BinMissingData'` is false, a missing value for `CustAge` remains as missing (`NaN`) when applying the WOE transformation.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding `CustIncome`, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding `TmWBank`, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding `AMBalance`, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding `EmpStatus`, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding `CustAge`, Deviance = 1442.8477, Chi2Stat = 4.4974731, PValue = 0.033944979
6. Adding `ResStatus`, Deviance = 1438.9783, Chi2Stat = 3.86941, PValue = 0.049173805
7. Adding `OtherCC`, Deviance = 1434.9751, Chi2Stat = 4.0031966, PValue = 0.045414057

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70229	0.063959	10.98	4.7498e-28
CustAge	0.57421	0.25708	2.2335	0.025513
ResStatus	1.3629	0.66952	2.0356	0.04179

EmpStatus	0.88373	0.2929	3.0172	0.002551
CustIncome	0.73535	0.2159	3.406	0.00065929
TmWBank	1.1065	0.23267	4.7556	1.9783e-06
OtherCC	1.0648	0.52826	2.0156	0.043841
AMBalance	1.0446	0.32197	3.2443	0.0011775

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 88.5, p-value = 2.55e-16

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[sc,mdl] = fitmodel(sc,'VariableSelection','FullModel')`

PredictorVars — Predictor variables for fitting creditscorecard object

all predictors in the `creditscorecard` object (default) | cell array of character vectors

Predictor variables for fitting the `creditscorecard` object, specified as the comma-separated pair consisting of `'PredictorVars'` and a cell array of character vectors. When provided, the `creditscorecard` object property `PredictorVars` is updated. Note that the order of predictors in the original dataset is enforced, regardless of the order in which `'PredictorVars'` is provided. When not provided, the predictors used to create the `creditscorecard` object (by using `creditscorecard`) are used.

Data Types: `cell`

VariableSelection — Variable selection method to fit logistic regression model

`'Stepwise'` (default) | character vector with values `'Stepwise'`, `'FullModel'`

The variable selection method to fit the logistic regression model, specified as the comma-separated pair consisting of `'VariableSelection'` and a character vector with values `'Stepwise'` or `'FullModel'`:

- **Stepwise** — Uses a stepwise selection method which calls the Statistics and Machine Learning Toolbox function `stepwiseglm`. Only variables in `PredictorVars` can potentially become part of the model and uses the `StartingModel` name-value pair argument to select the starting model.
- **FullModel** — Fits a model with all predictor variables in the `PredictorVars` name-value pair argument and calls `fitglm`.

Note Only variables in the `PredictorVars` property of the `creditscorecard` object can potentially become part of the logistic regression model and only linear terms are included in this model with no interactions or any other higher-order terms.

The response variable is mapped so that “Good” is 1 and “Bad” is 0.

Data Types: char

StartingModel — Initial model for Stepwise variable selection

'Constant' (default) | character vector with values 'Constant', 'Linear'

Initial model for the Stepwise variable selection method, specified as the comma-separated pair consisting of 'StartingModel' and a character vector with values 'Constant' or 'Linear'. This option determines the initial model (constant or linear) that the Statistics and Machine Learning Toolbox function `stepwiseglm` starts with.

- **Constant** — Starts the stepwise method with an empty (constant only) model.
- **Linear** — Starts the stepwise method from a full (all predictors in) model.

Note `StartingModel` is used only for the Stepwise option of `VariableSelection` and has no effect for the `FullModel` option of `VariableSelection`.

Data Types: char

Display — Indicator to display model information at command line

'On' (default) | character vector with values 'On', 'Off'

Indicator to display model information at command line, specified as the comma-separated pair consisting of 'Display' and a character vector with value 'On' or 'Off'.

Data Types: char

Output Arguments

sc — Credit scorecard model

`creditscorecard` object

Credit scorecard model, returned as an updated `creditscorecard` object. The `creditscorecard` object contains information about the model predictors and coefficients used to fit the WOE data. For more information on using the `creditscorecard` object, see `creditscorecard`.

mdl — Fitted logistic model

`GeneralizedLinearModel` object

Fitted logistic model, returned as an object of type `GeneralizedLinearModel` containing the fitted model. For more information on a `GeneralizedLinearModel` object, see `GeneralizedLinearModel`.

Note When creating the `creditscorecard` object with `creditscorecard`, if the optional name-value pair argument `WeightsVar` was used to specify observation (sample) weights, then `mdl` uses the weighted counts with `stepwiseglm` and `fitglm`.

More About

Using fitmodel with Weights

When observation weights are provided in the credit scorecard `data`, the weights are used to calibrate the model coefficients.

The underlying Statistics and Machine Learning Toolbox functionality for `stepwiseglm` and `fitglm` supports observation weights. The weights also affect the logistic model through the WOE values. The WOE transformation is applied to all predictors before fitting the logistic model. The observation weights directly impact the WOE values. For more information, see “Using bininfo with Weights” on page 15-1958 and “Credit Scorecard Modeling Using Observation Weights” on page 8-55.

Therefore, the credit scorecard points and final score depend on the observation weights through both the logistic model coefficients and the WOE values.

Models

A logistic regression model is used in the `creditscorecard` object.

For the model, the probability of being “Bad” is defined as: $\text{ProbBad} = \exp(-s) / (1 + \exp(-s))$.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`fitConstrainedModel` | `creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `bindata` | `displaypoints` | `formatpoints` | `score` | `stepwiseglm` | `fitglm` | `setmodel` | `probdefault` | `validatemodel` | `GeneralizedLinearModel`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Credit Scorecard Modeling with Missing Values” on page 8-57

“Troubleshooting Credit Scorecard Results” on page 8-64

“Credit Scorecard Modeling Workflow” on page 8-52

“About Credit Scorecards” on page 8-49

“Credit Scorecard Modeling Using Observation Weights” on page 8-55

“What Are Generalized Linear Models?”

fitConstrainedModel

Fit logistic regression model to Weight of Evidence (WOE) data subject to constraints on model coefficients

Syntax

```
[sc,mdl] = fitConstrainedModel(sc)
[sc,mdl] = fitConstrainedModel( ___,Name,Value)
```

Description

`[sc,mdl] = fitConstrainedModel(sc)` fits a logistic regression model to the Weight of Evidence (WOE) data subject to equality, inequality, or bound constraints on the model coefficients. `fitConstrainedModel` stores the model predictor names and corresponding coefficients in an updated `creditscorecard` object `sc` and returns the `GeneralizedLinearModel` object `mdl` which contains the fitted model.

`[sc,mdl] = fitConstrainedModel( ___,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Credit Scorecards with Constrained Logistic Regression Coefficients

To compute scores for a `creditscorecard` object with constraints for equality, inequality, or bounds on the coefficients of the logistic regression model, use `fitConstrainedModel`. Unlike `fitmodel`, `fitConstrainedModel` solves for both the unconstrained and constrained problem. The current solver used to minimize an objective function for `fitConstrainedModel` is `fmincon`, from the Optimization Toolbox™.

This example has three main sections. First, `fitConstrainedModel` is used to solve for the coefficients in the unconstrained model. Then, `fitConstrainedModel` demonstrates how to use several types of constraints. Finally, `fitConstrainedModel` uses bootstrapping for the significance analysis to determine which predictors to reject from the model.

Create the `creditscorecard` Object and Bin data

```
load CreditCardData.mat
sc = creditscorecard(data,'IDVar','CustID');
sc = autobinning(sc);
```

Unconstrained Model Using `fitConstrainedModel`

Solve for the unconstrained coefficients using `fitConstrainedModel` with default values for the input parameters. `fitConstrainedModel` uses the internal optimization solver `fmincon` from the Optimization Toolbox™. If you do not set any constraints, `fmincon` treats the model as an unconstrained optimization problem. The default parameters for the `LowerBound` and `UpperBound` are `-Inf` and `+Inf`, respectively. For the equality and inequality constraints, the default is an empty numeric array.


```
[sc1,mdl1] = fitConstrainedModel(sc);
coeff1 = mdl1.Coefficients.Estimate;
disp(mdl1.Coefficients);
```

	Estimate
(Intercept)	0.70246
CustAge	0.6057
TmAtAddress	1.0381
ResStatus	1.3794
EmpStatus	0.89648
CustIncome	0.70179
TmWBank	1.1132
OtherCC	1.0598
AMBalance	1.0572
UtilRate	-0.047597

Unlike `fitmodel` which gives p -values, when using `fitConstrainedModel`, you must use bootstrapping to find out which predictors are rejected from the model, when subject to constraints. This is illustrated in the "Significance Bootstrapping" section.

Using `fitmodel` to Compare the Results and Calibrate the Model

`fitmodel` fits a logistic regression model to the Weight-of-Evidence (WOE) data and there are no constraints. You can compare the results from the "Unconstrained Model Using `fitConstrainedModel`" section with those of `fitmodel` to verify that the model is well calibrated.

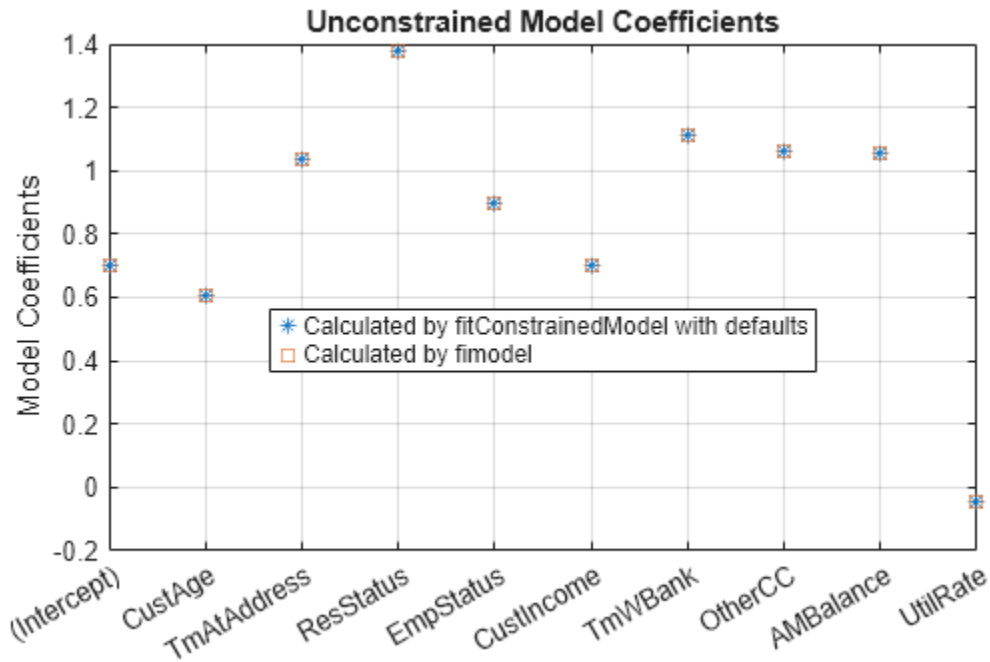
Now, solve the unconstrained problem by using `fitmodel`. Note that `fitmodel` and `fitConstrainedModel` use different solvers. While `fitConstrainedModel` uses `fmincon`, `fitmodel` uses `stepwiseglm` by default. To include all predictors from the start, set the 'VariableSelection' name-value pair argument of `fitmodel` to 'fullmodel'.

```
[sc2,mdl2] = fitmodel(sc,'VariableSelection','fullmodel','Display','off');
coeff2 = mdl2.Coefficients.Estimate;
disp(mdl2.Coefficients);
```

	Estimate	SE	tStat	pValue
(Intercept)	0.70246	0.064039	10.969	5.3719e-28
CustAge	0.6057	0.24934	2.4292	0.015131
TmAtAddress	1.0381	0.94042	1.1039	0.26963
ResStatus	1.3794	0.6526	2.1137	0.034538
EmpStatus	0.89648	0.29339	3.0556	0.0022458
CustIncome	0.70179	0.21866	3.2095	0.0013295
TmWBank	1.1132	0.23346	4.7683	1.8579e-06
OtherCC	1.0598	0.53005	1.9994	0.045568
AMBalance	1.0572	0.36601	2.8884	0.0038718
UtilRate	-0.047597	0.61133	-0.077858	0.93794

```
figure
plot(coeff1,'*')
hold on
plot(coeff2,'s')
xticklabels(mdl1.Coefficients.Properties.RowNames)
ylabel('Model Coefficients')
title('Unconstrained Model Coefficients')
```

```
legend({'Calculated by fitConstrainedModel with defaults', 'Calculated by fimodel'}, 'Location', 'b',
grid on
```



As both the tables and the plot show, the model coefficients match. You can be confident that this implementation of `fitConstrainedModel` is well calibrated.

Constrained Model

In the constrained model approach, you solve for the values of the coefficients b_i of the logistic model, subject to constraints. The supported constraints are bound, equality, or inequality. The coefficients maximize the likelihood-of-default function defined, for observation i , as:

$$L_i = p(\text{Default}_i)^{y_i} \times (1 - p(\text{Default}_i))^{1 - y_i}$$

where:

- $p(\text{Default}_i) = \frac{1}{1 + e^{-b\mathbf{x}_i}}$
- $\mathbf{b} = [b_1 \ b_2 \ \dots \ b_K]$ is an unknown model coefficient
- $\mathbf{x}_i = [x_{i1} \ x_{i2} \ \dots \ x_{iK}]$ is the predictor values at observation i
- y_i is the response value; a value of 1 represents default and a value of 0 represents non-default

This formula is for non-weighted data. When observation i has weight w_i , it means that there are w_i as many observations i . Therefore, the probability that default occurs at observation i is the product of the probabilities of default:

$$p_i = p(\text{Default}_i)^{y_i} * p(\text{Default}_i)^{y_i} * \dots * p(\text{Default}_i)^{y_i} = p(\text{Default}_i)^{w_i * y_i}$$

w_i times

Likewise, the probability of non-default for weighted observation i is:

$$\hat{p}_i = p(\sim\text{Default}_i)^{1-y_i} * p(\sim\text{Default}_i)^{1-y_i} * \dots * p(\sim\text{Default}_i)^{1-y_i} = (1 - p(\text{Default}_i))^{w_i * (1 - y_i)}$$

w_i times

For weighted data, if there is default at a given observation i whose weight is w_i , it is as if there was a w_i count of that one observation, and all of them either all default, or all non-default. w_i may or may not be an integer.

Therefore, for the weighted data, the likelihood-of-default function for observation i in the first equation becomes

$$L_i = p(\text{Default}_i)^{w_i * y_i} \times (1 - p(\text{Default}_i))^{w_i * (1 - y_i)}$$

By assumption, all defaults are independent events, so the objective function is

$$L = L_1 \times L_2 \times \dots \times L_N$$

or, in more convenient logarithmic terms:

$$\log(L) = \sum_{i=1}^N w_i * [y_i \log(p(\text{Default}_i)) + (1 - y_i) \log(1 - p(\text{Default}_i))]$$

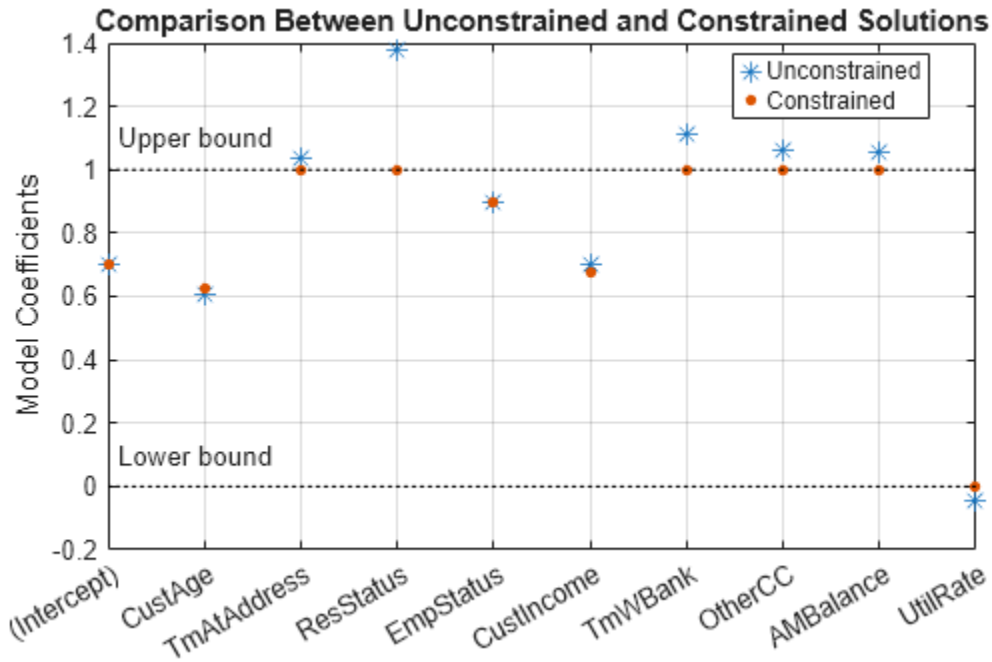
Apply Constraints on the Coefficients

After calibrating the unconstrained model as described in the "Unconstrained Model Using fitConstrainedModel" section, you can solve for the model coefficients subject to constraints. You can choose lower and upper bounds such that $0 \leq b_i \leq 1$, $\forall i = 1 \dots K$, except for the intercept. Also, since the customer age and customer income are somewhat correlated, you can also use additional constraints on their coefficients, for example, $|b_{\text{CusAge}} - b_{\text{CustIncome}}| < 0.1$. The coefficients corresponding to the predictors 'CustAge' and 'CustIncome' in this example are b_2 and b_6 , respectively.

```
K = length(sc.PredictorVars);
lb = [-Inf;zeros(K,1)];
ub = [Inf;ones(K,1)];
AIneq = [0 -1 0 0 0 1 0 0 0 0;0 -1 0 0 0 -1 0 0 0 0];
bIneq = [0.05;0.05];
Options = optimoptions('fmincon','SpecifyObjectiveGradient',true,'Display','off');
[sc3,mdl] = fitConstrainedModel(sc,'AInequality',AIneq,'bInequality',bIneq,...
    'LowerBound',lb,'UpperBound',ub,'Options',Options);

figure
plot(coeff1,'*','MarkerSize',8)
hold on
plot(mdl.Coefficients.Estimate,',' , 'MarkerSize',12)
line(xlim,[0 0],'color','k','linestyle',':')
line(xlim,[1 1],'color','k','linestyle',':')
text(1.1,0.1,'Lower bound')
text(1.1,1.1,'Upper bound')
grid on

xticklabels(mdl.Coefficients.Properties.RowNames)
ylabel('Model Coefficients')
title('Comparison Between Unconstrained and Constrained Solutions')
legend({'Unconstrained','Constrained'},'Location','best')
```



Significance Bootstrapping

For the unconstrained problem, standard formulas are available for computing p -values, which you use to evaluate which coefficients are significant and which are to be rejected. However, for the constrained problem, standard formulas are not available, and the derivation of formulas for significance analysis is complicated. A practical alternative is to perform significance analysis through *bootstrapping*.

In the bootstrapping approach, when using `fitConstrainedModel`, you set the name-value argument `'Bootstrap'` to `true` and chose a value for the name-value argument `'BootstrapIter'`. Bootstrapping means that N_{iter} samples (with replacement) from the original observations are selected. In each iteration, `fitConstrainedModel` solves for the same constrained problem as the "Constrained Model" section. `fitConstrainedModel` obtains several values (solutions) for each coefficient b_i and you can plot these as a boxplot or histogram. Using the boxplot or histogram, you can examine the median values to evaluate whether the coefficients are away from zero and how much the coefficients deviate from their means.

```
lb = [-Inf;zeros(K,1)];
ub = [Inf;ones(K,1)];
AIneq = [0 -1 0 0 0 1 0 0 0 0;0 1 0 0 0 -1 0 0 0 0];
bIneq = [0.05;0.05];
c0 = zeros(K,1);
Niter = 100;
Options = optimoptions('fmincon','SpecifyObjectiveGradient',true,'Display','off');
rng('default')

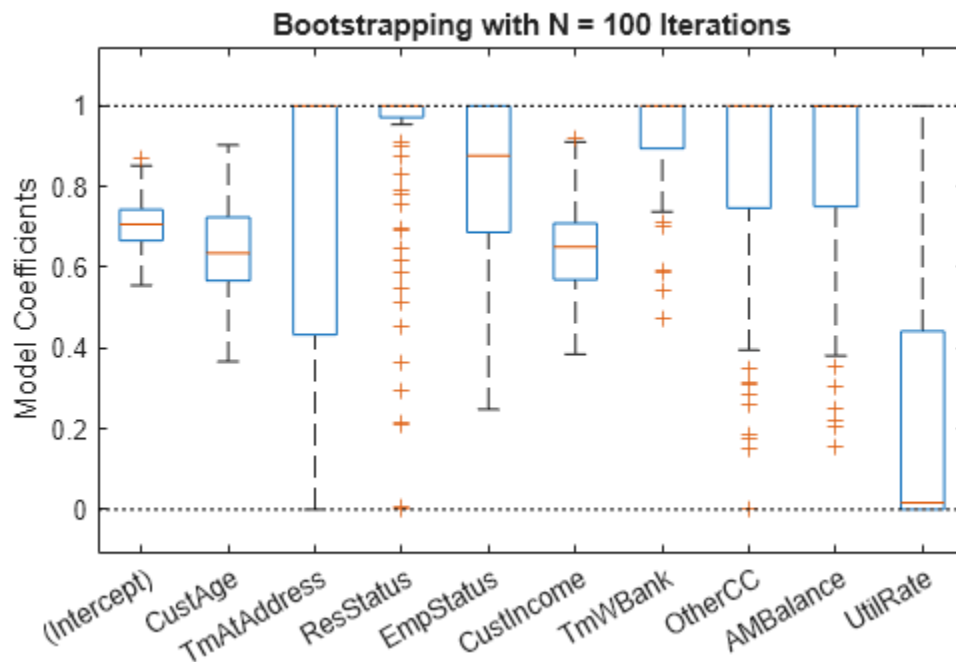
[sc,mdl] = fitConstrainedModel(sc,'AInequality',AIneq,'bInequality',bIneq,...
    'LowerBound',lb,'UpperBound',ub,'Bootstrap',true,'BootstrapIter',Niter,'Options',Options);

figure
boxplot(mdl.Bootstrap.Matrix,mdl.Coefficients.Properties.RowNames)
hold on
```

```

line(xlim,[0 0],'color','k','linestyle',':')
line(xlim,[1 1],'color','k','linestyle',':')
title('Bootstrapping with N = 100 Iterations')
ylabel('Model Coefficients')

```



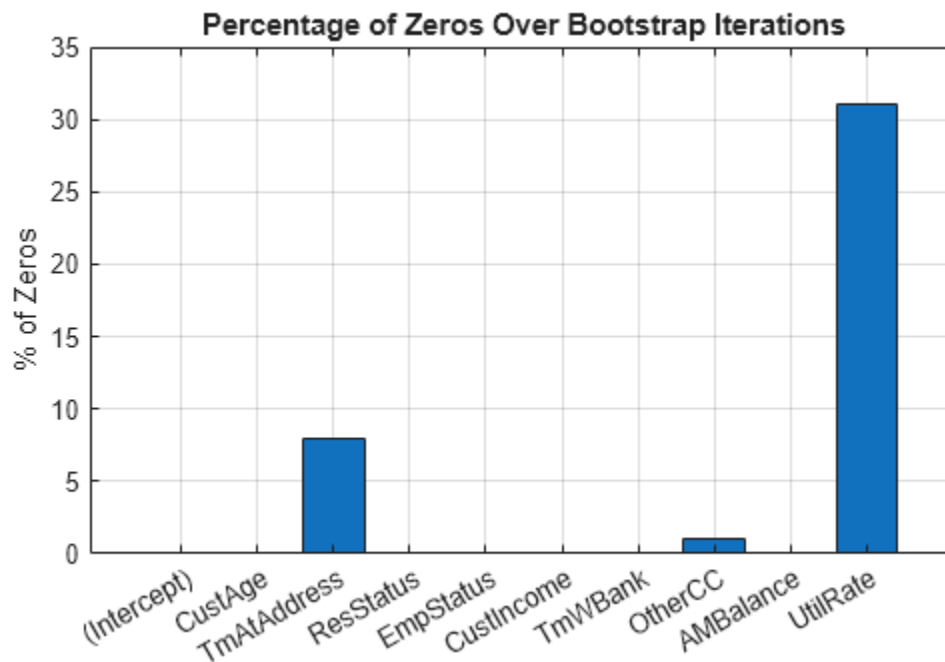
The solid red lines in the boxplot indicate that the median values and the bottom and top edges are for the 25th and 75th percentiles. The "whiskers" are the minimum and maximum values, not including outliers. The dotted lines represent the lower and upper bound constraints on the coefficients. In this example, the coefficients cannot be negative, by construction.

To help decide which predictors to keep in the model, assess the proportion of times each coefficient is zero.

```

Tol = 1e-6;
figure
bar(100*sum mdl.Bootstrap.Matrix <= Tol)/NIter)
ylabel('% of Zeros')
title('Percentage of Zeros Over Bootstrap Iterations')
xticklabels mdl.Coefficients.Properties.RowNames)
grid on

```



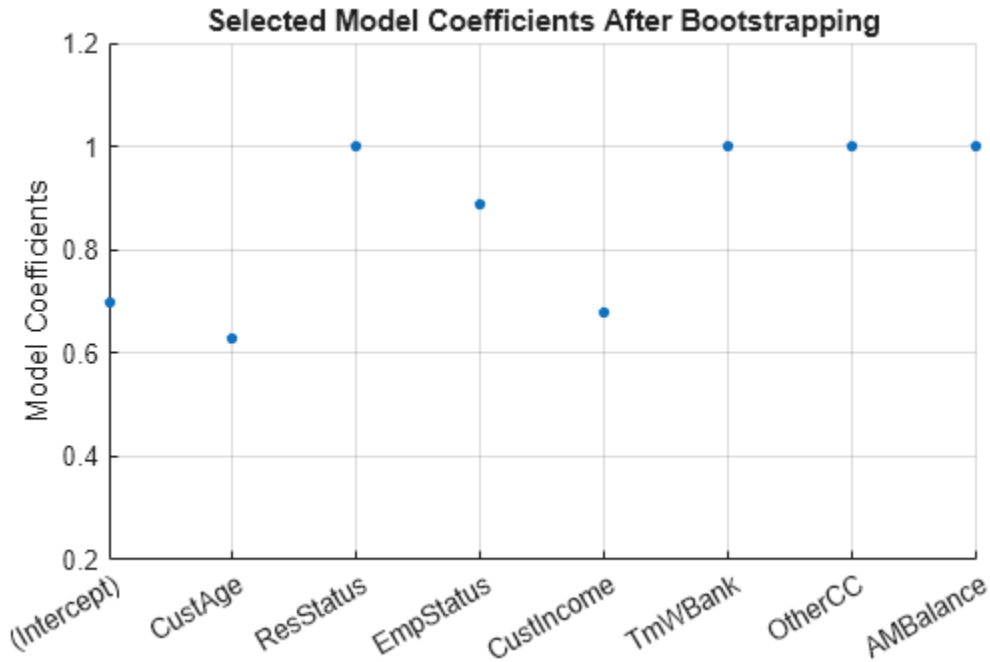
Based on the plot, you can reject 'UtilRate' since it has the highest number of zero values. You can also decide to reject 'TmAtAddress' since it shows a peak, albeit small.

Set the Corresponding Coefficients to Zero

To set the corresponding coefficients to zero, set their upper bound to zero and solve the model again using the original data set.

```
ub(3) = 0;
ub(end) = 0;
[sc,mdl] = fitConstrainedModel(sc,'AInequality',AIneq,'bInequality',bIneq,'LowerBound',lb,'UpperBound',ub);
Ind = (abs(mdl.Coefficients.Estimate) <= Tol);
ModelCoeff = mdl.Coefficients.Estimate(~Ind);
ModelPreds = mdl.Coefficients.Properties.RowNames(~Ind)';

figure
hold on
plot(ModelCoeff,','MarkerSize',12)
ylim([0.2 1.2])
ylabel('Model Coefficients')
xticklabels(ModelPreds)
title('Selected Model Coefficients After Bootstrapping')
grid on
```



Set Constrained Coefficients Back Into the creditscorecard

Now that you have solved for the constrained coefficients, use `setmodel` to set the model's coefficients and predictors. Then you can compute the (unscaled) points.

```
ModelPreds = ModelPreds(2:end);
sc = setmodel(sc,ModelPreds,ModelCoeff);
p = displaypoints(sc);
```

```
disp(p)
```

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.16725
{'CustAge' }	{'[33,37)' }	-0.14811
{'CustAge' }	{'[37,40)' }	-0.065607
{'CustAge' }	{'[40,46)' }	0.044404
{'CustAge' }	{'[46,48)' }	0.21761
{'CustAge' }	{'[48,58)' }	0.23404
{'CustAge' }	{'[58,Inf]' }	0.49029
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	0.0044307
{'ResStatus' }	{'Home Owner' }	0.11932
{'ResStatus' }	{'Other' }	0.30048
{'ResStatus' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.077028
{'EmpStatus' }	{'Employed' }	0.31459
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	-0.43795
{'CustIncome' }	{'[29000,33000)' }	-0.097814
{'CustIncome' }	{'[33000,35000)' }	0.053667
{'CustIncome' }	{'[35000,40000)' }	0.081921

{ 'CustIncome' }	{ '[40000,42000)' }	}	0.092364
{ 'CustIncome' }	{ '[42000,47000)' }	}	0.23932
{ 'CustIncome' }	{ '[47000,Inf]' }	}	0.42477
{ 'CustIncome' }	{ '<missing>' }	}	NaN
{ 'TmWBank' }	{ '[-Inf,12)' }	}	-0.15547
{ 'TmWBank' }	{ '[12,23)' }	}	-0.031077
{ 'TmWBank' }	{ '[23,45)' }	}	-0.021091
{ 'TmWBank' }	{ '[45,71)' }	}	0.36703
{ 'TmWBank' }	{ '[71,Inf]' }	}	0.86888
{ 'TmWBank' }	{ '<missing>' }	}	NaN
{ 'OtherCC' }	{ 'No' }	}	-0.16832
{ 'OtherCC' }	{ 'Yes' }	}	0.15336
{ 'OtherCC' }	{ '<missing>' }	}	NaN
{ 'AMBalance' }	{ '[-Inf,558.88)' }	}	0.34418
{ 'AMBalance' }	{ '[558.88,1254.28)' }	}	-0.012745
{ 'AMBalance' }	{ '[1254.28,1597.44)' }	}	-0.057879
{ 'AMBalance' }	{ '[1597.44,Inf]' }	}	-0.19896
{ 'AMBalance' }	{ '<missing>' }	}	NaN

Using the unscaled points, you can follow the remainder of the “Credit Scorecard Modeling Workflow” on page 8-52 to compute scores and probabilities of default and to validate the model.

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[sc,mdl] = fitConstrainedModel(sc,'LowerBound',2,'UpperBound',100)`

PredictorVars — Predictor variables for fitting creditscorecard object

all predictors in the `creditscorecard` object (default) | cell array of character vectors

Predictor variables for fitting the `creditscorecard` object, specified as the comma-separated pair consisting of `'PredictorVars'` and a cell array of character vectors. If you provide predictor variables, then the function updates the `creditscorecard` object property `PredictorVars`. The order of predictors in the original dataset is enforced, regardless of the order in which `'PredictorVars'` is provided. When not provided, the predictors used to create the `creditscorecard` object (by using `creditscorecard`) are used.

Data Types: `cell`

LowerBound — Lower bound

-Inf (default) | scalar | vector

Lower bound, specified as the comma-separated pair consisting of 'LowerBound' and a scalar or a real vector of length $N+1$, where N is the number of model coefficients in the `creditscorecard` object.

Data Types: double

UpperBound — Upper bound

Inf (default) | scalar | vector

Upper bound, specified as the comma-separated pair consisting of 'UpperBound' and a scalar or a real vector of length $N+1$, where N is the number of model coefficients in the `creditscorecard` object.

Data Types: double

AInequality — Matrix of linear inequality constraints

[] (default) | matrix

Matrix of linear inequality constraints, specified as the comma-separated pair consisting of 'AInequality' and a real M -by- $N+1$ matrix, where M is the number of constraints and N is the number of model coefficients in the `creditscorecard` object.

Data Types: double

bInequality — Vector of linear inequality constraints

[] (default) | vector

Vector of linear inequality constraints, specified as the comma-separated pair consisting of 'bInequality' and a real M -by-1 vector, where M is the number of constraints.

Data Types: double

AEquality — Matrix of linear equality constraints

[] (default) | matrix

Matrix of linear equality constraints, specified as the comma-separated pair consisting of 'AEquality' and a real M -by- $N+1$ matrix, where M is the number of constraints and N is the number of model coefficients in the `creditscorecard` object.

Data Types: double

bEquality — Vector of linear equality constraints

[] (default) | vector

Vector of linear equality constraints, specified as the comma-separated pair consisting of 'bEquality' and a real M -by-1 vector, where M is the number of constraints.

Data Types: double

Bootstrap — Indicator that bootstrapping defines the solution accuracy

false (default) | logical with a value of true or false

Indicator that bootstrapping defines the solution accuracy, specified as the comma-separated pair consisting of 'Bootstrap' and a logical with a value of true or false.

Data Types: logical

BootstrapIter — Number of bootstrapping iterations

100 (default) | positive integer

Number of bootstrapping iterations, specified as the comma-separated pair consisting of 'BootstrapIter' and a positive integer.

Data Types: double

Options — optimoptions object`optimoptions('fmincon','SpecifiedObjectiveGradient',true,'Display','off')`
(default) | object

`optimoptions` object, specified as the comma-separated pair consisting of 'Options' and an `optimoptions` object. You can create the object by using `optimoptions` from Optimization Toolbox.

Data Types: object

Output Arguments**sc — Credit scorecard model**`creditscorecard` object

Credit scorecard model, returned as an updated `creditscorecard` object. The `creditscorecard` object contains information about the model predictors and coefficients that fit the WOE data. For more information on using the `creditscorecard` object, see `creditscorecard`.

mdl — Fitted logistic model`GeneralizedLinearModel` object

Fitted logistic model, returned as a `GeneralizedLinearModel` object containing the fitted model. For more information on a `GeneralizedLinearModel` object, see `GeneralizedLinearModel`.

Note If you specify the optional `WeightsVar` argument when creating a `creditscorecard` object, then `mdl` uses the weighted counts with `stepwiseglm` and `fitglm`.

The `mdl` structure has the following fields:

- `Coefficients` is a table in which the `RowNames` property contains the names of the model coefficients and has a single column, 'Estimate', containing the solution.
- `Bootstrap` exists when 'Bootstrap' is set to `true`, and has two fields:
 - `CI` contains the 95% confidence interval for the solution.
 - `Matrix` an `NIter`-by-`N` matrix of coefficients, where `NIter` is the number of bootstrap iterations and `N` is the number of model coefficients.
- `Solver` has three fields:
 - `Options` additional information on the algorithm and solution.
 - `ExitFlag` contains an integer that codes the reason why the solver stopped. For more information, see `fmincon`.
 - `Output` is a structure with additional information on the optimization process.

More About

Model Coefficients

When you use `fitConstrainedModel` to solve for the model coefficients, the function solves for the same number of parameters as predictor variables you specify, plus one additional coefficient for the intercept.

The first coefficient corresponds to the intercept. If you provide predictor variables using the 'PredictorVars' optional input argument, then `fitConstrainedModel` updates the `creditscorecard` object property `PredictorsVars`. The order of predictors in the original dataset is enforced, regardless of the order in which 'PredictorVars' is provided. When not provided, the predictors used to create the `creditscorecard` object (by using `creditscorecard`) are used.

Calibration

The constrained model is first calibrated such that, when unconstrained, the solution is identical, within a certain tolerance, to the solution given by `fitmodel`, with the 'fullmodel' choice for the name-value argument 'VariableSelection'.

As an exercise, you can test the calibration by leaving all name-value parameters of `fitConstrainedModel` to their default values. The solutions are identical to within a 10^{-6} to 10^{-5} tolerance.

Calibration with Weights and Missing Data

If the credit scorecard data contains observation weights, the `fitConstrainedModel` function uses the weights to calibrate the model coefficients.

For credit scorecard data with no missing data and no weights, the likelihood function for observation i is

$$L_i = p(\text{Default}_i) \times \left(\frac{1}{1 + e^{-bx_i}} \right)$$

where $p(\text{Default}_i) = \frac{1}{1 + e^{-bx_i}}$

where:

- $b = [b_1 \ b_2 \dots b_K]$ is for unknown model coefficients
- $x_i = [x_{i1} \ x_{i2} \dots x_{iK}]$ is the predictor values at observation i
- y_i is the response value of 1 (the default) or a value of 0.

When observation i has weight w_i , it means that there are w_i observations. Because of the independence of defaults between observations, the probability that there is default at observation i is the product of the probabilities of default

$$p_i = p(\text{Default}_i) * p(\text{Default}_i) * \dots * p(\text{Default}_i) = p(\text{Default}_i)^{w_i \text{ times}}$$

Likewise, the probability of non-default for weighted observation i is

$$\hat{p}_i = p(\sim) * p(\sim) * \dots * p(\sim) = (1 - p(\text{Default}_i))^{w_i \text{ times}}$$

For weighted data, if there is default at a given observation i whose weight is w_i , it is as if there was w_i defaults of that one observation, and all of them either all default, or all non-default. w_i may or may not be an integer. Therefore, the likelihood function for observation i becomes

$$L_i = p(\text{Default}_i)^{w_i * y_i} \times (1 - p(\text{Default}_i))^{w_i * (1 - y_i)}$$

Likewise, for data with missing observations (NaN, <undefined>, or “Missing”), the model is calibrated by comparing the unconstrained case with results given by `fitglm`. Where the data contains missing observations, the WOE input matrix has NaN values. The NaN values do not pose any issue for `fitglm` (unconstrained), or `fmincon` (constrained). The only edge case is if all observations of a given predictor are missing, in which case, that predictor is discarded from the model.

Bootstrapping

Bootstrapping is a method for estimating the accuracy of the solution obtained after iterating the objective function `NIter` times.

When 'Bootstrap' is set to true, the `fitConstrainedModel` function performs sampling with replacement of the WOE values and is passed to the objective function. At the end of the iterative process, the solutions are stored in a `NIter-by-N+1` matrix, where `N` is the number of model coefficients.

The 95% confidence interval (CI) returned in the output structure `mdl.Bootstrap` contains the values of the coefficients at the 25th and 97.5th percentiles.

Models

A logistic regression model is used in the `creditscorecard` object.

For the model, the probability of being “Bad” is given by $\text{ProbBad} = \exp(-s) / (1 + \exp(-s))$.

Version History

Introduced in R2019a

References

- [1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.
- [2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`fitmodel` | `creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `bindata` | `displaypoints` | `formatpoints` | `score` | `stepwiseglm` | `fitglm` | `fmincon` | `setmodel` | `probdefault` | `validatemodel` | `GeneralizedLinearModel`

Topics

- “Case Study for Credit Scorecard Analysis” on page 8-71
- “Credit Scorecard Modeling with Missing Values” on page 8-57
- “Troubleshooting Credit Scorecard Results” on page 8-64
- “Credit Scorecard Modeling Workflow” on page 8-52
- “About Credit Scorecards” on page 8-49

“Credit Scorecard Modeling Using Observation Weights” on page 8-55
“What Are Generalized Linear Models?”

setmodel

Set model predictors and coefficients

Syntax

```
sc = setmodel(sc,ModelPredictors,ModelCoefficients)
```

Description

`sc = setmodel(sc,ModelPredictors,ModelCoefficients)` sets the predictors and coefficients of a linear logistic regression model fitted outside the `creditscorecard` object and returns an updated `creditscorecard` object. The predictors and coefficients are used for the computation of scorecard points. Use `setmodel` in lieu of `fitmodel`, which fits a linear logistic regression model, because `setmodel` offers increased flexibility. For example, when a model fitted with `fitmodel` needs to be modified, you can use `setmodel`. For more information, see “Workflows for Using setmodel” on page 15-1895.

Note When using `setmodel`, the following assumptions apply:

- The model coefficients correspond to a linear logistic regression model (where only linear terms are included in the model and there are no interactions or any other higher-order terms).
 - The model was previously fitted using Weight of Evidence (WOE) data with the response mapped so that ‘Good’ is 1 and ‘Bad’ is 0.
-

Examples

Modify a GLM Model Fitted with `fitmodel`

This example shows how to use `setmodel` to make modifications to a logistic regression model initially fitted using the `fitmodel` function, and then set the new logistic regression model predictors and coefficients back into the `creditscorecard` object.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID')

sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustI
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
```

```

BinMissingData: 0
IDVar: 'CustID'
PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBa
Data: [1200x11 table]

```

Perform automatic binning.

```
sc = autobinning(sc);
```

The standard workflow is to use the `fitmodel` function to fit a logistic regression model using a stepwise method. However, `fitmodel` only supports limited options regarding the stepwise procedure. You can use the optional `mdl` output argument from `fitmodel` to get a copy of the fitted `GeneralizedLinearModel` object, to later modify.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Suppose you want to include, or "force," the predictor `'UtilRate'` in the logistic regression model, even though the stepwise method did not include it in the fitted model. You can add `'UtilRate'` to the logistic regression model using the `GeneralizedLinearModel` object `mdl` directly.

```
mdl = mdl.addTerms('UtilRate')
```

```
mdl =
```

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60843	0.24936	2.44	0.014687
ResStatus	1.3773	0.6529	2.1096	0.034896
EmpStatus	0.88556	0.29303	3.0221	0.0025103
CustIncome	0.70146	0.2186	3.2089	0.0013324
TmWBank	1.1071	0.23307	4.7503	2.0316e-06
OtherCC	1.0882	0.52918	2.0563	0.03975
AMBalance	1.0413	0.36557	2.8483	0.004395
UtilRate	0.013157	0.60864	0.021618	0.98275

1200 observations, 1191 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 5.26e-16

Use `setmodel` to update the model predictors and model coefficients in the `creditscorecard` object. The `ModelPredictors` input argument does not explicitly include a string for the intercept. However, the `ModelCoefficients` input argument does have the intercept information as its first element.

```
ModelPredictors = mdl.PredictorNames
```

```
ModelPredictors = 8×1 cell
```

```
{'CustAge' }
{'ResStatus' }
{'EmpStatus' }
{'CustIncome' }
{'TmWBank' }
{'OtherCC' }
{'AMBalance' }
{'UtilRate' }
```

```
ModelCoefficients = mdl.Coefficients.Estimate
```

```
ModelCoefficients = 9×1
```

```
0.7024
0.6084
1.3773
0.8856
0.7015
1.1071
1.0882
1.0413
0.0132
```

```
sc = setmodel(sc,ModelPredictors,ModelCoefficients);
```

Verify that 'UtilRate' is part of the scorecard predictors by displaying the scorecard points.

```
pi = displaypoints(sc)
```


Predictors

Predictors

{ 'CustAge' }	{ '[-Inf,33)' }	-0.17152
{ 'CustAge' }	{ '[33,37)' }	-0.15295
{ 'CustAge' }	{ '[37,40)' }	-0.072892
{ 'CustAge' }	{ '[40,46)' }	0.033856
{ 'CustAge' }	{ '[46,48)' }	0.20193
{ 'CustAge' }	{ '[48,58)' }	0.21787
{ 'CustAge' }	{ '[58,Inf]' }	0.46652
{ 'CustAge' }	{ '<missing>' }	NaN
{ 'ResStatus' }	{ 'Tenant' }	-0.043826
{ 'ResStatus' }	{ 'Home Owner' }	0.11442
{ 'ResStatus' }	{ 'Other' }	0.36394
{ 'ResStatus' }	{ '<missing>' }	NaN
{ 'EmpStatus' }	{ 'Unknown' }	-0.088843
{ 'EmpStatus' }	{ 'Employed' }	0.30193
{ 'EmpStatus' }	{ '<missing>' }	NaN
{ 'CustIncome' }	{ '[-Inf,29000)' }	-0.46956

•
•
•

```
sc = autobinning(sc);
```

The logistic regression model needs to be fit with Weight of Evidence (WOE) data. The WOE transformation is a special case of binning, since the data first needs to be binned, and then the binned information is mapped to the corresponding WOE values. This transformation is done using the `bindata` function. `bindata` has an argument that prepares the data for the model fitting step. By setting the `bindata` name-value pair argument for `'OutputType'` to `WOEModelInput`:

- All predictors are converted to WOE values.
- The output contains only predictors and response (no `'IDVar'` or any unused variables).
- Predictors with infinite or undefined (NaN) WOE values are discarded.
- The response values are mapped so that "Good" is 1 and "Bad" is 0 (this implies that higher unscaled scores correspond to better, less risky customers).

```
bd = bindata(sc, 'OutputType', 'WOEModelInput');
```

For example, the first ten rows in the original data for the variables `'CustAge'`, `'ResStatus'`, `'CustIncome'`, and `'status'` (response variable) look like this:

```
data(1:10, {'CustAge' 'ResStatus' 'CustIncome' 'status'})
```

```
ans=10x4 table
```

CustAge	ResStatus	CustIncome	status
53	Tenant	50000	0
61	Home Owner	52000	0
47	Tenant	37000	0
50	Home Owner	53000	0
68	Home Owner	53000	0
65	Home Owner	48000	0
34	Home Owner	32000	1
50	Other	51000	0
50	Tenant	52000	1
49	Home Owner	53000	1

Here is how the same ten rows look after calling `bindata` with the name-value pair argument `'OutputType'` set to `'WOEModelInput'`:

```
bd(1:10, {'CustAge' 'ResStatus' 'CustIncome' 'status'})
```

```
ans=10x4 table
```

CustAge	ResStatus	CustIncome	status
0.21378	-0.095564	0.47972	1
0.62245	0.019329	0.47972	1
0.18758	-0.095564	-0.026696	1
0.21378	0.019329	0.47972	1
0.62245	0.019329	0.47972	1
0.62245	0.019329	0.47972	1
-0.39568	0.019329	-0.29217	0
0.21378	0.20049	0.47972	1
0.21378	-0.095564	0.47972	0
0.21378	0.019329	0.47972	0

Fit a logistic linear regression model using a stepwise method with the Statistics and Machine Learning Toolbox™ function `stepwiseglm`, but use a nondefault value for the 'PEnter' and 'PRemove' optional arguments. The predictors 'ResStatus' and 'OtherCC' would normally be included in the logistic linear regression model using default options for the stepwise procedure.

```
mdl = stepwiseglm(bd,'constant','Distribution','binomial',...
'Upper','linear','PEnter',0.025,'PRemove',0.05)
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306

```
mdl =
Generalized linear regression model:
    logit(status) ~ 1 + CustAge + EmpStatus + CustIncome + TmWBank + AMBalance
    Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70263	0.063759	11.02	3.0544e-28
CustAge	0.57265	0.2482	2.3072	0.021043
EmpStatus	0.88356	0.29193	3.0266	0.002473
CustIncome	0.70399	0.21781	3.2321	0.001229
TmWBank	1.1	0.23185	4.7443	2.0924e-06
AMBalance	1.0313	0.32007	3.2221	0.0012724

1200 observations, 1194 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 81.4, p-value = 4.18e-16

Use `setmodel` to update the model predictors and model coefficients in the `creditscorecard` object. The `ModelPredictors` input argument does not explicitly include a string for the intercept. However, the `ModelCoefficients` input argument does have the intercept information as its first element.

```
ModelPredictors = mdl.PredictorNames
```

```
ModelPredictors = 5x1 cell
```

```
{'CustAge' }
{'EmpStatus' }
{'CustIncome' }
{'TmWBank' }
{'AMBalance' }
```

```
ModelCoefficients = mdl.Coefficients.Estimate
```

```
ModelCoefficients = 6x1
```

```
0.7026
0.5726
0.8836
0.7040
```

```
1.1000
1.0313
```

```
sc = setmodel(sc,ModelPredictors,ModelCoefficients);
```

Verify that the desired model predictors are part of the scorecard predictors by displaying the scorecard points.

```
pi = displaypoints(sc)
```

pi=30x3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.10354
{'CustAge' }	{'[33,37)' }	-0.086059
{'CustAge' }	{'[37,40)' }	-0.010713
{'CustAge' }	{'[40,46)' }	0.089757
{'CustAge' }	{'[46,48)' }	0.24794
{'CustAge' }	{'[48,58)' }	0.26294
{'CustAge' }	{'[58,Inf]' }	0.49697
{'CustAge' }	{'<missing>' }	NaN
{'EmpStatus' }	{'Unknown' }	-0.035716
{'EmpStatus' }	{'Employed' }	0.35417
{'EmpStatus' }	{'<missing>' }	NaN
{'CustIncome' }	{'[-Inf,29000)' }	-0.41884
{'CustIncome' }	{'[29000,33000)' }	-0.065161
{'CustIncome' }	{'[33000,35000)' }	0.092353
{'CustIncome' }	{'[35000,40000)' }	0.12173
{'CustIncome' }	{'[40000,42000)' }	0.13259
:		

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a creditscorecard object. Use creditscorecard to create a creditscorecard object.

ModelPredictors — Predictor names included in fitted model

cell array of character vectors with predictor values

{'PredictorName1','PredictorName2',...}

Predictor names included in the fitted model, specified as a cell array of character vectors as {'PredictorName1','PredictorName2',...}. The predictor names must match predictor variable names in the creditscorecard object.

Note Do not include a character vector for the constant term in ModelPredictors, setmodel internally handles the '(Intercept)' term based on the number of model coefficients (see ModelCoefficients).

Data Types: cell

ModelCoefficients — Model coefficients corresponding to model predictors

numeric array with values [coeff1,coeff2,...]

Model coefficients corresponding to the model predictors, specified as a numeric array of model coefficients, [coeff1,coeff2,...]. If N is the number of predictor names provided in `ModelPredictors`, the size of `ModelCoefficients` can be N or $N+1$. If `ModelCoefficients` has $N+1$ elements, then the first coefficient is used as the '(Intercept)' of the fitted model. Otherwise, the '(Intercept)' is set to 0.

Data Types: double

Output Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, returned as an updated `creditscorecard` object. The `creditscorecard` object contains information about the model predictors and coefficients of the fitted model. For more information on using the `creditscorecard` object, see `creditscorecard`.

More About

Workflows for Using setmodel

When using `setmodel`, there are two possible workflows to set the final model predictors and model coefficients into a `creditscorecard` object.

The first workflow is:

- Use `fitmodel` to get the optional output argument `mdl`. This is a `GeneralizedLinearModel` object and you can add and remove terms, or modify the parameters of the stepwise procedure. Only linear terms can be in the model (no interactions or any other higher-order terms).
- Once the `GeneralizedLinearModel` object is satisfactory, set the final model predictors and model coefficients into the `creditscorecard` object using the `setmodel` input arguments for `ModelPredictors` and `ModelCoefficients`.

An alternate workflow is:

- Obtain the Weight of Evidence (WOE) data using `bindata`. Use the 'WOEModelInput' option for the 'OutputType' name-value pair argument in `bindata` to ensure that:
 - The predictors data is transformed to WOE.
 - Only predictors whose bins have finite WOE values are included.
 - The response variable is placed in the last column.
 - The response variable is mapped ("Good" is 1 and "Bad" is 0).
- Use the data from the previous step to fit a linear logistic regression model (only linear terms in the model, no interactions, or any other higher-order terms). See, for example, `stepwiseglm`.
- Once the `GeneralizedLinearModel` object is satisfactory, set the final model predictors and model coefficients into the `creditscorecard` object using the `setmodel` input arguments for `ModelPredictors` and `ModelCoefficients`.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `bindata` | `displaypoints` | `formatpoints` | `score` | `stepwiseglm` | `fitglm` | `fitmodel` | `probdefault` | `validatemodel` | `GeneralizedLinearModel`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Troubleshooting Credit Scorecard Results” on page 8-64

“Credit Scorecard Modeling Workflow” on page 8-52

“About Credit Scorecards” on page 8-49

bindata

Binned predictor variables

Syntax

```
bdata = bindata(sc)
bdata = bindata(sc,data)
bdata = bindata(sc,Name,Value)
```

Description

`bdata = bindata(sc)` binned predictor variables returned as a table. This is a table of the same size as the data input, but only the predictors specified in the `creditscorecard` object's `PredictorVars` property are binned and the remaining ones are unchanged.

`bdata = bindata(sc,data)` returns a table of binned predictor variables. `bindata` returns a table of the same size as the `creditscorecard` data, but only the predictors specified in the `creditscorecard` object's `PredictorVars` property are binned and the remaining ones are unchanged.

`bdata = bindata(sc,Name,Value)` binned predictor variables returned as a table using optional name-value pair arguments. This is a table of the same size as the data input, but only the predictors specified in the `creditscorecard` object's `PredictorVars` property are binned and the remaining ones are unchanged.

Examples

Bin `creditscorecard` Data as Bin Numbers, Categories, or WOE Values

This example shows how to use the `bindata` function to simply bin or discretize data.

Suppose bin ranges of

- '0 to 30'
- '31 to 50'
- '51 and up'

are determined for the age variable (via manual or automatic binning). If a data point with age 41 is given, binning this data point means placing it in the bin for 41 years old, which is the second bin, or the '31 to 50' bin. Binning is then the mapping from the original data, into discrete groups or bins. In this example, you can say that a 41-year old is mapped into bin number 2, or that it is binned into the '31 to 50' category. If you know the Weight of Evidence (WOE) value for each of the three bins, you could also replace the data point 41 with the WOE value corresponding to the second bin. `bindata` supports the three binning formats just mentioned:

- Bin number (where the `'OutputType'` name-value pair argument is set to `'BinNumber'`); this is the default option, and in this case, 41 is mapped to bin 2.

- Categorical (where the 'OutputType' name-value pair argument is set to 'Categorical'); in this case, 41 is mapped to the '31 to 50' bin.
- WOE value (where the 'OutputType' name-value pair argument is set to 'WOE'); in this case, 41 is mapped to the WOE value of bin number 2.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the 'IDVar' argument to indicate that 'CustID' contains ID information and should not be included as a predictor variable.

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID')

sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        Data: [1200x11 table]
```

Perform automatic binning.

```
sc = autobinning(sc);
```

Show the bin information for 'CustAge'.

```
bininfo(sc, 'CustAge')
```

```
ans=8x6 table
    Bin      Good    Bad    Odds      WOE      InfoValue
    _____
    {'[-Inf,33)'}    70     53    1.3208    -0.42622    0.019746
    {'[33,37)'}      64     47    1.3617    -0.39568    0.015308
    {'[37,40)'}      73     47    1.5532    -0.26411    0.0072573
    {'[40,46)'}     174     94    1.8511   -0.088658    0.001781
    {'[46,48)'}      61     25     2.44     0.18758    0.0024372
    {'[48,58)'}     263    105    2.5048     0.21378    0.013476
    {'[58,Inf]'}      98     26    3.7692     0.62245     0.0352
    {'Totals'}      803    397    2.0227         NaN    0.095205
```

These are the first 10 age values in the original data, used to create the `creditscorecard` object.

```
data(1:10, 'CustAge')
```

```
ans=10x1 table
    CustAge
    _____
```

```
53
```



```

61
47
50
68
65
34
50
50
49

```

Bin scorecard data into bin numbers (default behavior).

```
bdata = bindata(sc);
```

According to the bin information, the first age should be mapped into the fourth bin, the second age into the fifth bin, etc. These are the first 10 binned ages, in bin-number format.

```
bdata(1:10, 'CustAge')
```

```
ans=10×1 table
    CustAge
```

```

6
7
5
6
7
7
2
6
6
6

```

Bin the scorecard data and show their bin labels. To do this, set the `bindata` name-value pair argument for `'OutputType'` to `'Categorical'`.

```
bdata = bindata(sc, 'OutputType', 'Categorical');
```

These are the first 10 binned ages, in categorical format.

```
bdata(1:10, 'CustAge')
```

```
ans=10×1 table
    CustAge
```

```

[48,58)
[58,Inf]
[46,48)
[48,58)
[58,Inf]
[58,Inf]
[33,37)
[48,58)
[48,58)

```

```
[48,58)
```

Convert the scorecard data to WOE values. To do this, set the `bindata` name-value pair argument for `'OutputType'` to `'WOE'`.

```
bdata = bindata(sc, 'OutputType', 'WOE');
```

These are the first 10 binned ages, in WOE format. The ages are mapped to the WOE values that are internally displayed using the `bininfo` function.

```
bdata(1:10, 'CustAge')
```

```
ans=10×1 table
    CustAge
```

```

0.21378
0.62245
0.18758
0.21378
0.62245
0.62245
-0.39568
0.21378
0.21378
0.21378
```

Bin Additional "Test" Data

This example shows how to use the `bindata` function's optional input for the data to bin. If not provided, `bindata` bins the `creditscorecard` training data. However, if a different dataset needs to be binned, for example, some "test" data, this can be passed into `bindata` as an optional input.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

```
load CreditCardData
```

```
sc = creditscorecard(data, 'IDVar', 'CustID')
```

```
sc =
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustI
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'T
    Data: [1200×11 table]
```

Perform automatic binning.

```
sc = autobinning(sc);
```

Show the bin information for 'CustAge'.

```
bininfo(sc, 'CustAge')
```

```
ans=8x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ---      ---      ---      ---      ---      ---
{' [-Inf,33) ' }      70      53      1.3208     -0.42622     0.019746
{' [33,37) '   }      64      47      1.3617     -0.39568     0.015308
{' [37,40) '   }      73      47      1.5532     -0.26411     0.0072573
{' [40,46) '   }     174      94      1.8511    -0.088658     0.001781
{' [46,48) '   }      61      25       2.44      0.18758     0.0024372
{' [48,58) '   }     263     105      2.5048      0.21378     0.013476
{' [58,Inf] '  }      98      26      3.7692      0.62245      0.0352
{' Totals '    }     803     397      2.0227         NaN     0.095205
```

For the purpose of illustration, take a few rows from the original data as "test" data and display the first 10 age values in the test data.

```
tdata = data(101:110,:);
tdata(1:10, 'CustAge')
```

```
ans=10x1 table
      CustAge
      ---
      34
      59
      64
      61
      28
      65
      55
      37
      49
      51
```

Convert the test data to WOE values. To do this, set the bindata name-value pair argument for 'OutputType' to 'WOE', passing the test data (tdata) as an optional input.

```
bdata = bindata(sc,tdata, 'OutputType', 'WOE')
```

```
bdata=10x11 table
      CustID      CustAge      TmAtAddress      ResStatus      EmpStatus      CustIncome      TmWBank      Other
      ---      ---      ---      ---      ---      ---      ---      ---
      101     -0.39568     -0.087767     -0.095564      0.2418     -0.011271      0.76889      0.09
      102      0.62245      0.14288      0.019329     -0.19947      0.20579     -0.13107     -0.2
      103      0.62245      0.02263      0.019329      0.2418      0.47972     -0.12109      0.09
      104      0.62245      0.02263     -0.095564      0.2418      0.47972     -0.12109      0.09
      105     -0.42622      0.02263      0.019329      0.2418     -0.06843      0.76889      0.09
      106      0.62245      0.02263      0.019329     -0.19947      0.20579     -0.13107      0.09
```

107	0.21378	-0.087767	-0.095564	0.2418	0.47972	0.26704	0.095564
108	-0.26411	-0.087767	0.019329	-0.19947	-0.29217	-0.13107	0.095564
109	0.21378	-0.087767	-0.095564	0.2418	-0.026696	-0.13107	0.095564
110	0.21378	-0.087767	0.019329	0.2418	0.20579	-0.13107	0.095564

These are the first 10 binned ages, in WOE format. The ages are mapped to the WOE values displayed internally by `bininfo`.

```
bdata(1:10, 'CustAge')
```

```
ans=10x1 table
CustAge
```

```

-0.39568
0.62245
0.62245
0.62245
-0.42622
0.62245
0.21378
-0.26411
0.21378
0.21378
```

Bin Additional "Test" Data When Using the 'BinMissingData' Option

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data with missing values. The variables `CustAge` and `ResStatus` have missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherBank
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

Use `creditscorecard` with the name-value argument `'BinMissingData'` set to `true` to bin the missing numeric or categorical data in a separate bin. Apply automatic binning.

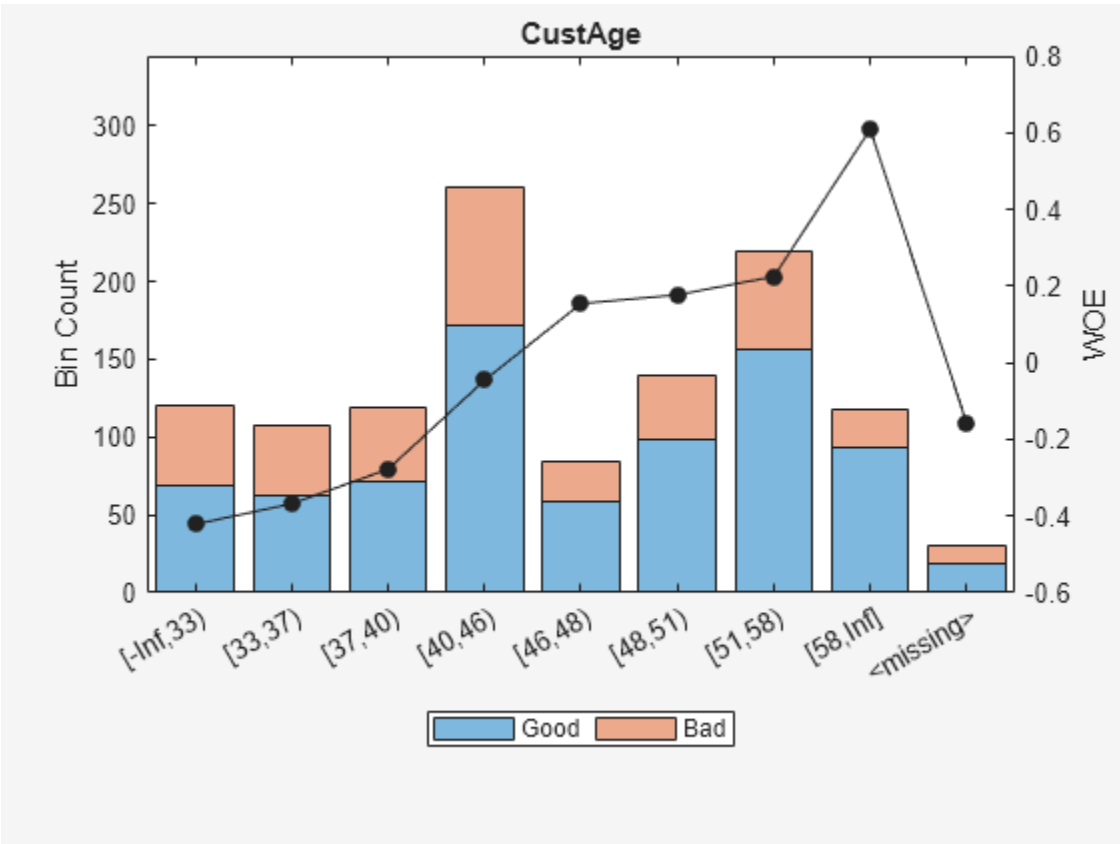
```
sc = creditscorecard(dataMissing, 'IDVar', 'CustID', 'BinMissingData', true);
sc = autobin(sc);
```

```
disp(sc)
```

```

creditscorecard with properties:
    GoodLabel: 0
    ResponseVar: 'status'
```

15-1903

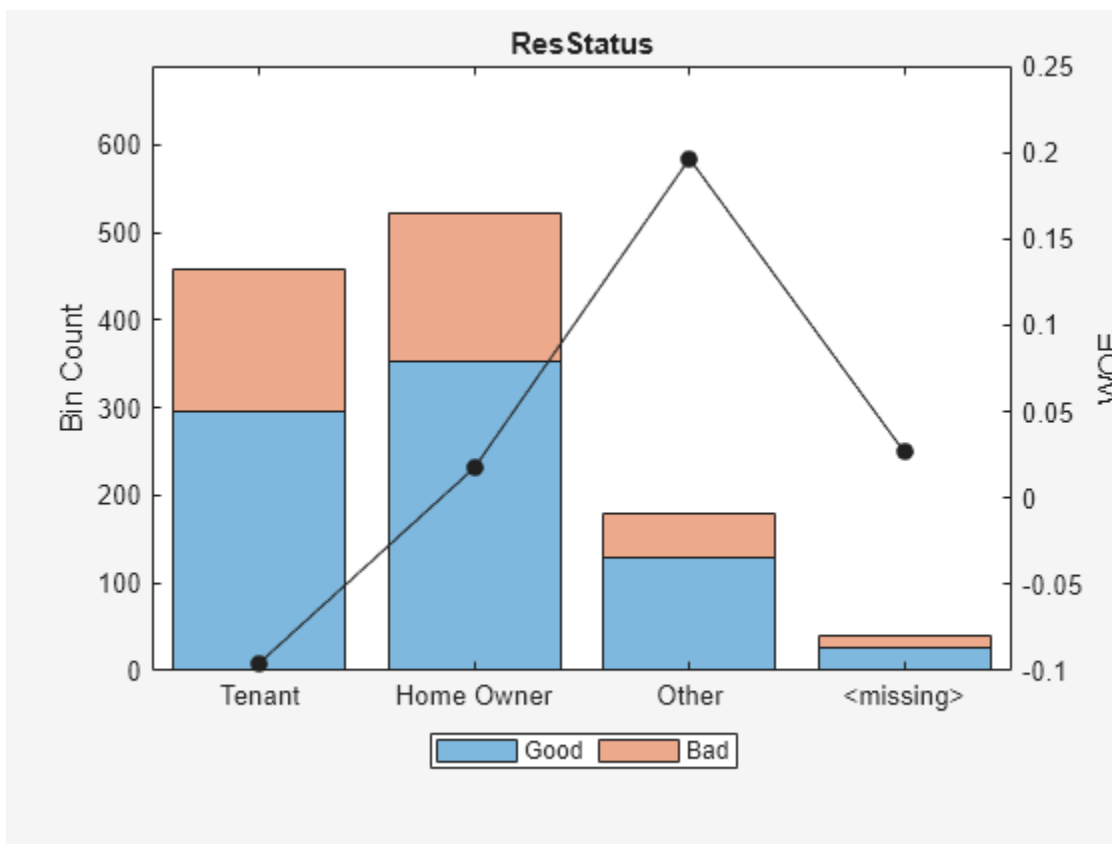


Display and plot bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');  
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



For the 'CustAge' and 'ResStatus' predictors, there is missing data (NaNs and <undefined>) in the training data, and the binning process estimates a WOE value of -0.15787 and 0.026469 respectively for missing data in these predictors, as shown above.

For the purpose of illustration, take a few rows from the original data as test data and introduce some missing data.

```
tdata = dataMissing(11:14,:);
tdata.CustAge(1) = NaN;
tdata.TmAtAddress(2) = NaN;
tdata.ResStatus(3) = missing;
tdata.EmpStatus(4) = missing;
disp(tdata)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	0
11	NaN	24	Tenant	Unknown	34000	44	
12	48	NaN	Other	Unknown	44000	14	
13	65	63	<undefined>	Unknown	48000	6	
14	44	75	Other	<undefined>	41000	35	

Convert the test data to WOE values. To do this, set the bindata name-value pair argument for 'OutputType' to 'WOE', passing the test data tdata as an optional input.

```
bdata = bindata(sc,tdata,'OutputType','WOE');
disp(bdata)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
11	-0.15787	0.02263	-0.095463	-0.19947	-0.06843	-0.12109	0.095463
12	0.17713	NaN	0.19637	-0.19947	0.20579	-0.13107	0.095463
13	0.60931	0.02263	0.026469	-0.19947	0.47972	-0.25547	-0.095463
14	-0.04556	0.02263	0.19637	NaN	-0.011271	-0.12109	-0.095463

For the 'CustAge' and 'ResStatus' predictors, because there is missing data in the training data, the missing values in the test data get mapped to the WOE value estimated for the <missing> bin. Therefore, a missing value for 'CustAge' is replaced with -0.15787, and a missing value for 'ResStatus' is replaced with 0.026469.

For 'TmAtAddress' and 'EmpStatus', the training data has no missing values, therefore there is no bin for missing data, and there is no way to estimate a WOE value for missing data. Therefore, for these predictors, the WOE transformation leaves missing values as missing (that is, sets a WOE value of NaN).

These rules apply when 'OutputType' is set to 'WOE' or 'WOEModelInput'. The rationale is that if a data-based WOE value exists for missing data, it should be used for the WOE transformation and for subsequent steps (for example, fitting a logistic model or scoring).

On the other hand, when 'OutputType' is set to 'BinNumber' or 'Categorical', bindata leaves missing values as missing, since this allows you to subsequently treat the missing data as you see fit.

For example, when 'OutputType' is set to 'BinNumber', missing values are set to NaN:

```
bdata = bindata(sc,tdata,'OutputType','BinNumber');
disp(bdata)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
11	NaN	2	1	1	3	3	2
12	6	NaN	3	1	6	2	2
13	8	2	NaN	1	7	1	1
14	4	2	3	NaN	5	3	1

And when 'OutputType' is set to 'Categorical', missing values are set to '<undefined>':

```
bdata = bindata(sc,tdata,'OutputType','Categorical');
disp(bdata)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank
11	<undefined>	[23,83)	Tenant	Unknown	[33000,35000)	[23,45000)
12	[48,51)	<undefined>	Other	Unknown	[42000,47000)	[12,23000)
13	[58,Inf]	[23,83)	<undefined>	Unknown	[47000,Inf]	[-Inf,23000)
14	[40,46)	[23,83)	Other	<undefined>	[40000,42000)	[23,45000)

Apply a Weight of Evidence (WOE) Transformation to Data

bindata supports the following types of WOE transformation:

- Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Use the `'IDVar'` argument to indicate that `'CustID'` contains ID information and should not be included as a predictor variable.

Perform automatic binning.

Show the bin information for 'CustAge'.

ans=8*6 table					
Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'}	70	53	1.3208	-0.42622	0.019746
{'[33,37)'}	64	47	1.3617	-0.39568	0.015308
{'[37,40)'}	73	47	1.5532	-0.26411	0.0072573
{'[40,46)'}	174	94	1.8511	-0.088658	0.001781
{'[46,48)'}	61	25	2.44	0.18758	0.0024372
{'[48,58)'}	263	105	2.5048	0.21378	0.013476
{'[58,Inf]'}	98	26	3.7692	0.62245	0.0352
{'Totals'}	803	397	2.0227	NaN	0.095205

These are the first 10 age values in the original data, used to create the `creditscorecard` object.

```
data(1:10, 'CustAge')
```

```
ans=10x1 table
```

```
  CustAge
```

```
-----  
    53  
    61  
    47  
    50  
    68  
    65  
    34  
    50  
    50  
    49
```

Convert the test data to WOE values. To do this, set the `bindata` name-value pair argument for `'OutputType'` to `'WOE'`.

```
bdata = bindata(sc, 'OutputType', 'WOE');
```

These are the first 10 binned ages, in WOE format. The ages are mapped to the WOE values displayed internally by `bininfo`.

```
bdata(1:10, 'CustAge')
```

```
ans=10x1 table
```

```
  CustAge
```

```
-----  
 0.21378  
 0.62245  
 0.18758  
 0.21378  
 0.62245  
 0.62245  
-0.39568  
 0.21378  
 0.21378  
 0.21378
```

These are the first 10 binned ages, in WOE format. The ages are mapped to the WOE values displayed internally by `bininfo`.

```
bdata(1:10, 'CustAge')
```

```
ans=10x1 table
```

```
  CustAge
```

```
-----  
 0.21378  
 0.62245  
 0.18758  
 0.21378
```

```

0.62245
0.62245
-0.39568
0.21378
0.21378
0.21378

```

The size of the original data and the size of `bdata` output are the same because `bindata` leaves unused variables (such as 'IDVar') unchanged and in place.

```
whos data bdata
```

Name	Size	Bytes	Class	Attributes
bdata	1200x11	109307	table	
data	1200x11	85035	table	

The response values are the same in the original data and in the binned data because, by default, `bindata` does not modify response values.

```
disp([data.status(1:10) bdata.status(1:10)])
```

```

0      0
0      0
0      0
0      0
0      0
0      0
1      1
0      0
1      1
1      1

```

When fitting a logistic regression model with WOE data, set the 'OutputType' name-value pair argument to 'WOEModelInput'.

```
bdata = bindata(sc, 'OutputType', 'WOEModelInput');
```

The binned predictor data is the same as when the 'OutputType' name-value pair argument is set to 'WOE'.

```
bdata(1:10, 'CustAge')
```

```
ans=10x1 table
    CustAge
```

```

0.21378
0.62245
0.18758
0.21378
0.62245
0.62245
-0.39568
0.21378
0.21378
0.21378

```

However, the size of the original data and the size of `bdata` output are different. This is because `bindata` removes unused variables (such as `'IDVar'`).

whos `data bdata`

Name	Size	Bytes	Class	Attributes
<code>bdata</code>	1200x10	99455	table	
<code>data</code>	1200x11	85035	table	

The response values are also modified in this case and are mapped so that "Good" is 1 and "Bad" is 0.

```
disp([data.status(1:10) bdata.status(1:10)])
```

```

0      1
0      1
0      1
0      1
0      1
0      1
1      0
0      1
1      0
1      0

```

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

data — Data to bin given the rules set in `creditscorecard` object

table

Data to bin given the rules set in the `creditscorecard` object, specified using a table. By default, `data` is set to the `creditscorecard` object's raw data.

Before creating a `creditscorecard` object, perform a data preparation task to have an appropriately structured data as input to a `creditscorecard` object.

Data Types: `table`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `bdata = bindata(sc, 'OutputType', 'WOE', 'ResponseFormat', 'Mapped')`

OutputType — Output format

'BinNumber' (default) | character vector with values 'BinNumber', 'Categorical', 'WOE'

Output format, specified as the comma-separated pair consisting of 'OutputType' and a character vector with the following values:

- **BinNumber** — Returns the bin numbers corresponding to each observation.
- **Categorical** — Returns the bin label corresponding to each observation.
- **WOE** — Returns the Weight of Evidence (WOE) corresponding to each observation.
- **WOEModelInput** — Use this option when fitting a model. This option:
 - Returns the Weight of Evidence (WOE) corresponding to each observation.
 - Returns predictor variables, but no IDVar or unused variables are included in the output.
 - Discards any predictors whose bins have Inf or NaN WOE values.
 - Includes the mapped response variable as the last column.

Note When the bindata name-value pair argument 'OutputType' is set to 'WOEModelInput', the bdata output only contains the columns corresponding to predictors whose bins do not have Inf or NaN Weight of Evidence (WOE) values, and bdata includes the mapped response as the last column.

Missing data (if any) are included in the bdata output as missing data as well, and do not influence the rules to discard predictors when 'OutputType' is set to 'WOEModelInput'.

Data Types: char

ResponseFormat — Response values format

'RawData' (default) | character vector with values 'RawData', 'Mapped'

Response values format, specified as the comma-separated pair consisting of 'ResponseFormat' and a character vector with the following values:

- **RawData** — The response variable is copied unchanged into the bdata output.
- **Mapped** — The response values are modified (if necessary) so that "Good" is mapped to 1, and "Bad" is mapped to 0.

Data Types: char

Output Arguments

bdata — Binned predictor variables

table

Binned predictor variables, returned as a table. This is a table of the same size (see exception in the following Note) as the data input, but only the predictors specified in the creditcorecard object's PredictorVars property are binned and the remaining ones are unchanged.

Note When the bindata name-value pair argument 'OutputType' is set to 'WOEModelInput', the bdata output only contains the columns corresponding to predictors whose bins do not have Inf or NaN Weight of Evidence (WOE) values, and bdata includes the mapped response as the last column.

Missing data (if any) are included in the bdata output as missing data as well, and do not influence the rules to discard predictors when 'OutputType' is set to 'WOEModelInput'.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `plotbins` | `modifybins` | `fitmodel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `probdefault` | `validatemodel`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Credit Scorecard Modeling with Missing Values” on page 8-57

“Troubleshooting Credit Scorecard Results” on page 8-64

“Credit Scorecard Modeling Workflow” on page 8-52

“About Credit Scorecards” on page 8-49

plotbins

Plot histogram counts for predictor variables

Syntax

```
plotbins(sc,PredictorName)
hFigure = plotbins(sc,PredictorName)
hFigure = plotbins( ___,Name,Value)
```

Description

`plotbins(sc,PredictorName)` plots histogram counts for given predictor variables. When a predictor's bins are modified using `modifybins` or `autobinning`, rerun `plotbins` to update the figure to reflect the change.

`hFigure = plotbins(sc,PredictorName)` returns a handle to the figure. `plotbins` plots histogram counts for given predictor variables. When a predictor's bins are modified using `modifybins` or `autobinning`, rerun `plotbins` to update the figure to reflect the change.

`hFigure = plotbins( ___,Name,Value)` returns a handle to the figure. `plotbins` plots histogram counts for given predictor variables using optional name-value pair arguments. When a predictor's bins are modified using `modifybins` or `autobinning`, rerun `plotbins` to update the figure to reflect the change.

Examples

Plot a Histogram for Bin Information

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Perform automatic binning for the `PredictorName` input argument for `CustIncome` using the defaults for the algorithm `Monotone`.

```
sc = autobinning(sc, 'CustIncome')
```

```
sc =
    creditscorecard with properties:
```

```
    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn...'
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan...'
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
```

```
PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome'}
Data: [1200x11 table]
```

Use `bininfo` to display the autobinned data.

```
[bi, cp] = bininfo(sc, 'CustIncome')
```

```
bi=8x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____  _____  _____  _____  _____  _____
{'[-Inf,29000)'}    53       58    0.91379   -0.79457    0.06364
{'[29000,33000)'}   74       49    1.5102   -0.29217    0.0091366
{'[33000,35000)'}   68       36    1.8889   -0.06843    0.00041042
{'[35000,40000)'}  193       98    1.9694   -0.026696   0.00017359
{'[40000,42000)'}   68       34         2   -0.011271   1.0819e-05
{'[42000,47000)'}  164       66    2.4848    0.20579    0.0078175
{'[47000,Inf]'}    183       56    3.2679    0.47972    0.041657
{'Totals'}         803      397    2.0227         NaN    0.12285
```

```
cp = 6x1
```

```
29000
33000
35000
40000
42000
47000
```

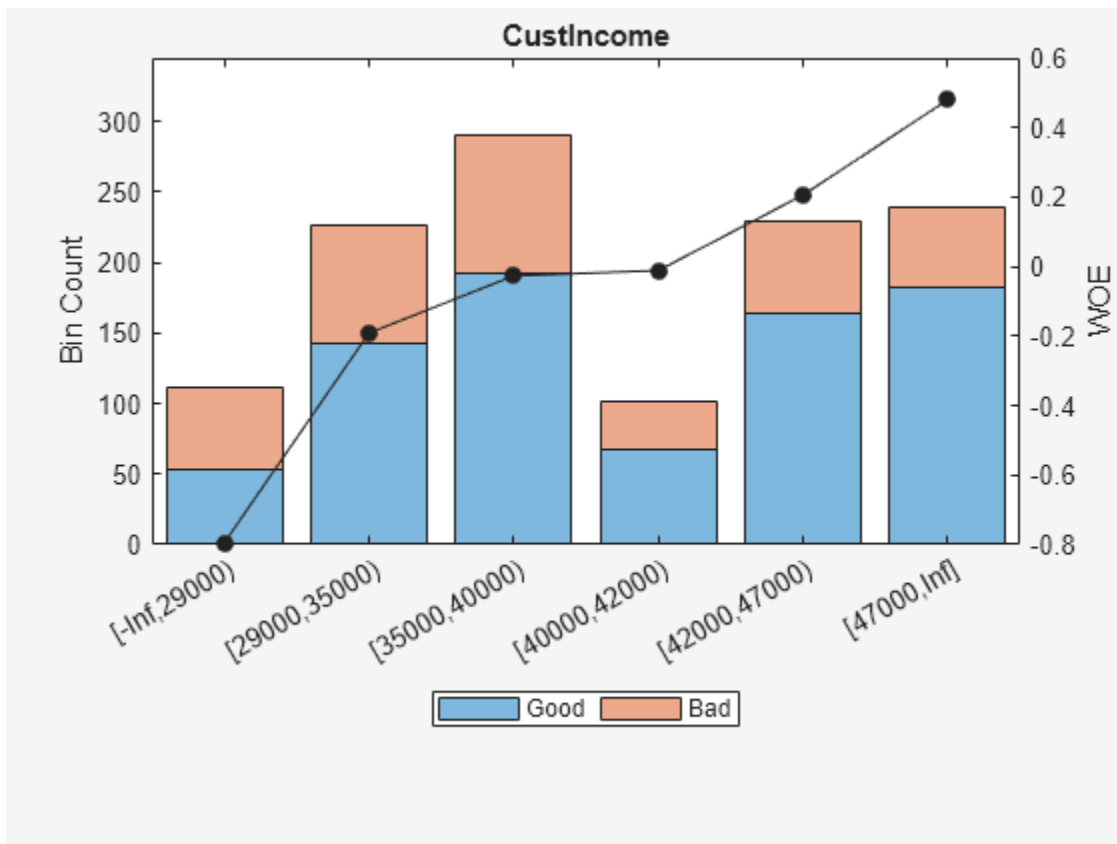
Manually remove the second cut point (the boundary between the second and third bins) to merge bins two and three. Use the `modifybins` function to update the scorecard and then display updated bin information.

```
cp(2) = [];
sc = modifybins(sc, 'CustIncome', 'CutPoints', cp);
bi = bininfo(sc, 'CustIncome')
```

```
bi=7x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____  _____  _____  _____  _____  _____
{'[-Inf,29000)'}    53       58    0.91379   -0.79457    0.06364
{'[29000,35000)'}  142       85    1.6706   -0.19124    0.0071274
{'[35000,40000)'}  193       98    1.9694   -0.026696   0.00017359
{'[40000,42000)'}   68       34         2   -0.011271   1.0819e-05
{'[42000,47000)'}  164       66    2.4848    0.20579    0.0078175
{'[47000,Inf]'}    183       56    3.2679    0.47972    0.041657
{'Totals'}         803      397    2.0227         NaN    0.12043
```

Plot the histogram count for updated bin information for the PredictorName called `CustIncome`.

```
plotbins(sc, 'CustIncome')
```

Plot a Histogram for Bin Information Using Name-Value Pair Arguments

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Perform automatic binning for the `PredictorName` input argument for `CustIncome` using the defaults for the algorithm `Monotone`.

```
sc = autobinning(sc, 'CustIncome')
```

```
sc =
creditscorecard with properties:

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
```

Data: [1200×11 table]

Use `bininfo` to display the autobinned data.

```
[bi, cp] = bininfo(sc, 'CustIncome')
```

bi=8×6 table

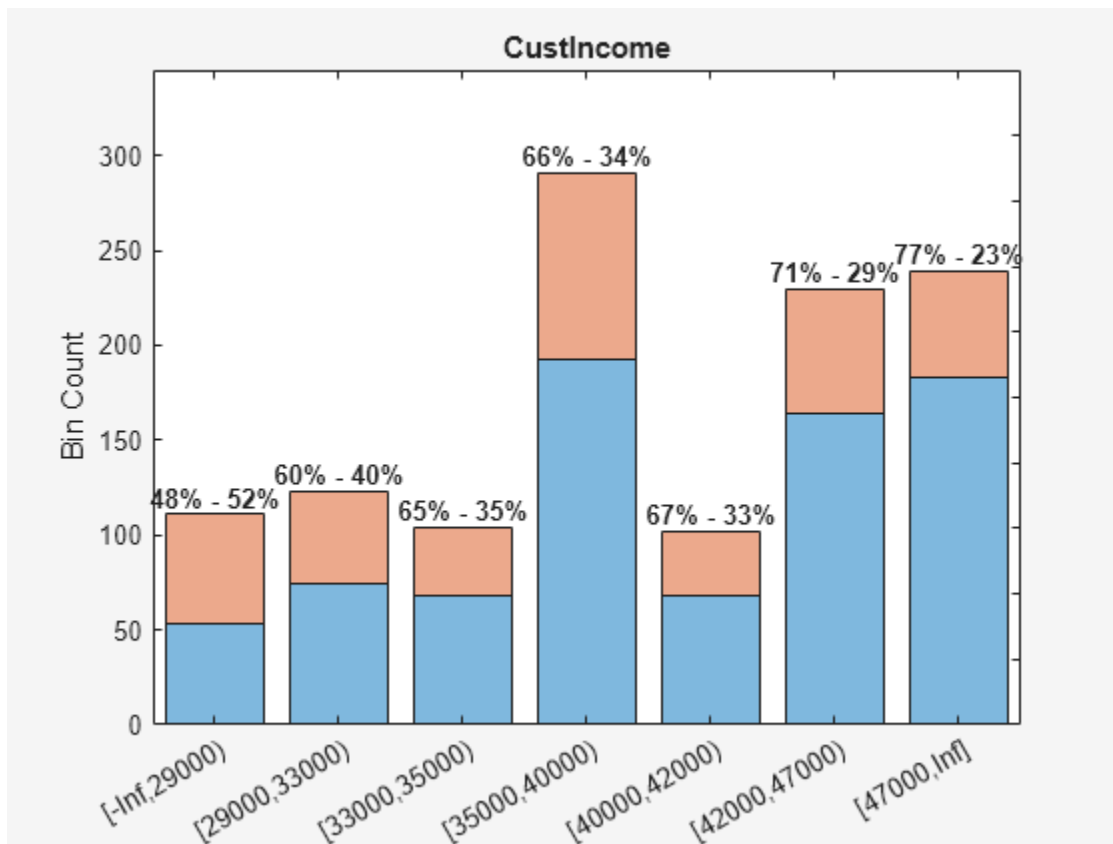
Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,29000)'} }	53	58	0.91379	-0.79457	0.06364
{'[29000,33000)'} }	74	49	1.5102	-0.29217	0.0091366
{'[33000,35000)'} }	68	36	1.8889	-0.06843	0.00041042
{'[35000,40000)'} }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)'} }	68	34	2	-0.011271	1.0819e-05
{'[42000,47000)'} }	164	66	2.4848	0.20579	0.0078175
{'[47000,Inf]'} }	183	56	3.2679	0.47972	0.041657
{'Totals' }	803	397	2.0227	NaN	0.12285

cp = 6×1

```
29000
33000
35000
40000
42000
47000
```

Plot the bin information for `CustIncome` without the Weight of Evidence (WOE) line and without a legend by setting the `'WOE'` and `'Legend'` name-value arguments to `'Off'`. Also, set the `'BinText'` name-value pair argument to `'PercentRows'` to show as text over the plot bars for the proportion of "Good" and "Bad" within each bin, that is, the probability of "Good" and "Bad" within each bin.

```
plotbins(sc, 'CustIncome', 'WOE', 'Off', 'Legend', 'Off', 'BinText', 'PercentRows')
```



Plot Bin Information When Using Missing Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

```
Number of rows: 1200
```

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

```
Number of missing values CustAge: 30
```

Number of missing values ResStatus: 40

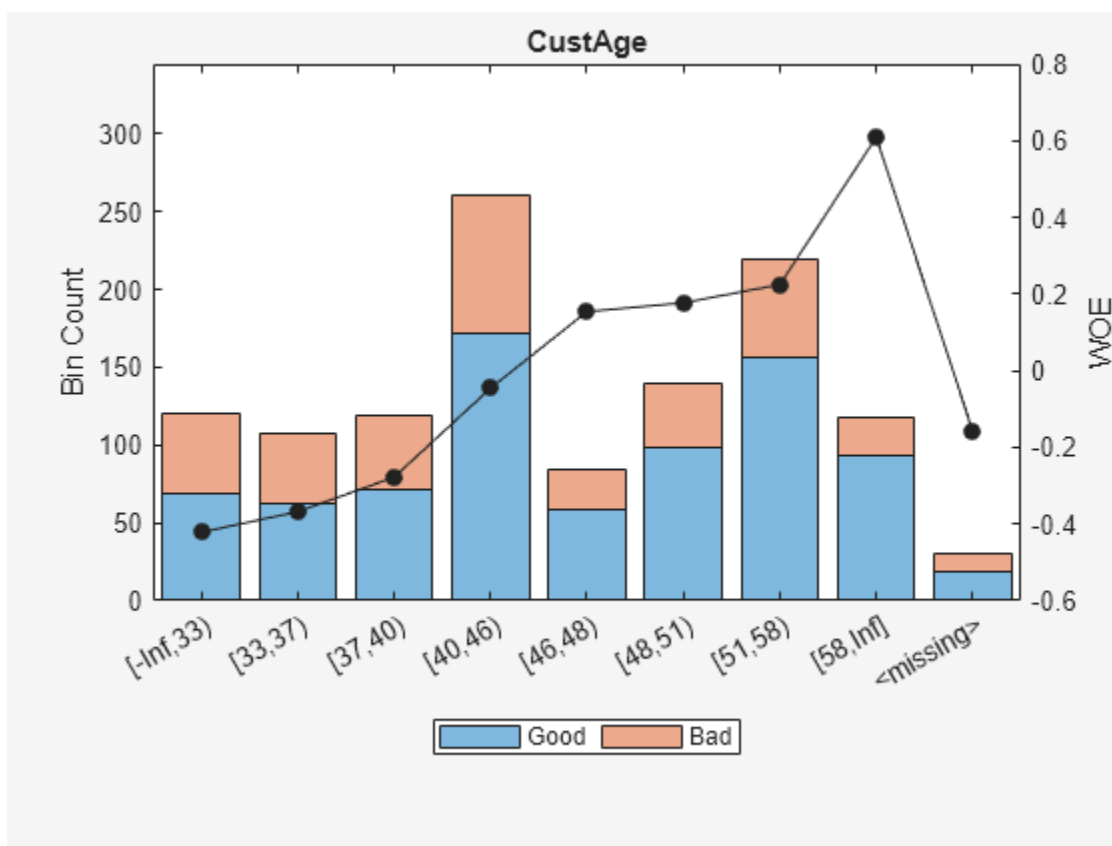
```
sc = creditscorecard(dataMissing, 'IDVar', 'CustID', 'BinMissingData', true);
sc = autobinning(sc);
```

```
creditscorecard with properties:
```

Display and plot bin information for numeric data for 'CustAge' that includes missing data in a separate bin labelled `<missing>`.

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[-Inf,33)' }	69	52	1.3269	-0.42156	0.018993
{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549
{ '[46,48)' }	59	25	2.36	0.15424	0.0016199
{ '[48,51)' }	99	41	2.4146	0.17713	0.0035449
{ '[51,58)' }	157	62	2.5323	0.22469	0.0088407
{ '[58,Inf)' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.087112

15-1918

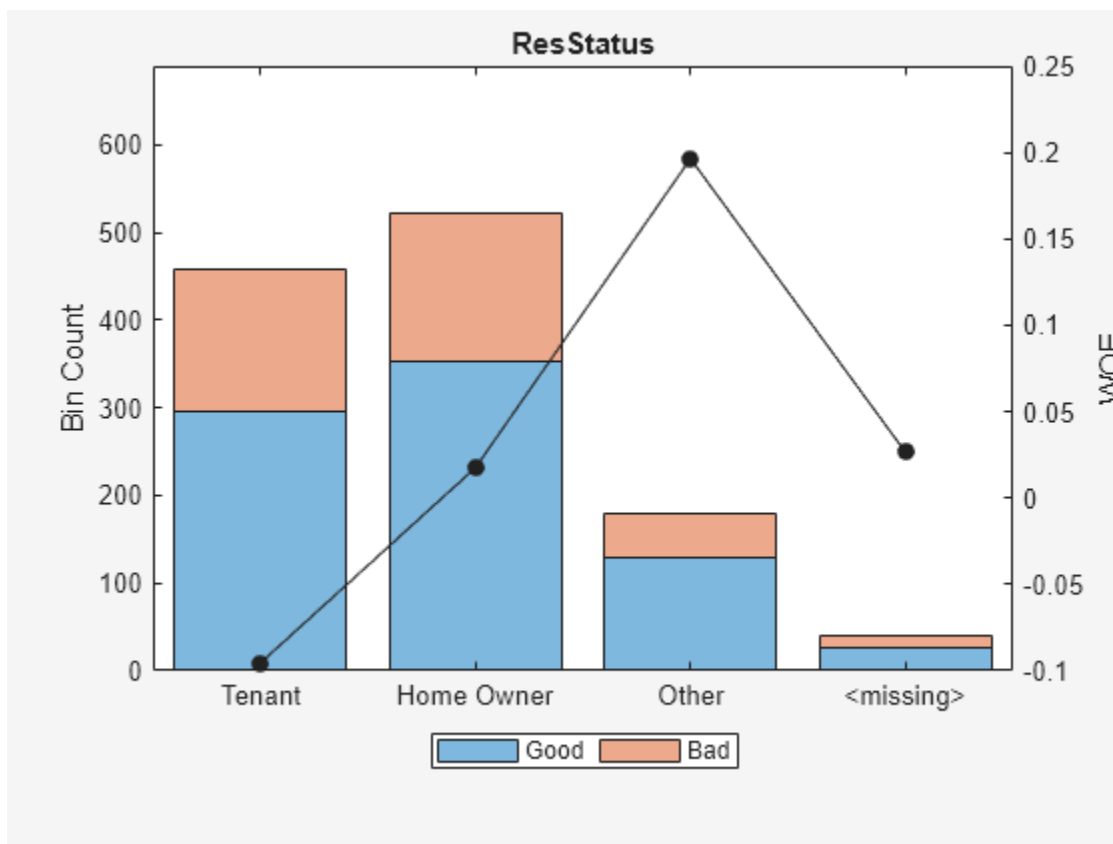


Display and plot bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

PredictorName — Name of one or more predictors to plot

character vector with predictor name | cell array of character vectors with predictor names

Name of one or more predictors to plot, specified using a character vector or cell array of character vectors containing one or more names of the predictors.

Data Types: `char` | `cell`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `plotbins(sc,PredictorName,'BinText','Count','WOE','On')`

BinText — Information to display on top of plotted bin counts

'None' (default) | character vector with values 'None', 'Count', 'PercentRows', 'PercentCols', 'PercentTotal'

Information to display on top of plotted bin counts, specified as the comma-separated pair consisting of 'BinText' and a character vector with values:

- **None** — No text is displayed on top of the bins.
- **Count** — For each bin, displays the count for “Good” and “Bad.”
- **PercentRows** — For each bin, displays the count for “Good” and “Bad” as a percentage of the number of observations in the bin.
- **PercentCols** — For each bin, displays the count for “Good” and “Bad” as a percentage of the total “Good” and total “Bad” in the entire sample.
- **PercentTotal** — For each bin, displays the count for “Good” and “Bad” as a percentage of the total number of observations in the entire sample.

Data Types: char

WOE — Indicator for Weight of Evidence (WOE)

'On' (default) | character vector with values 'On', 'Off'

Indicator for Weight of Evidence (WOE) line, specified as the comma-separated pair consisting of 'WOE' and a character vector with values On or Off. When set to On, the WOE line is plotted on top of the histogram.

Data Types: char

Legend — Indicator for legend on plot

'On' (default) | character vector with values 'On', 'Off'

Indicator for legend on the plot, specified as the comma-separated pair consisting of 'Legend' and a character vector with values On or Off.

Data Types: char

Output Arguments**hFigure — Figure handle for histogram plot for predictor variables**

figure object

Figure handle for histogram plot for predictor variables, returned as figure object or array of figure objects if more than one PredictorName is specified as an input.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `bininfo` | `predictorinfo` | `modifypredictor` | `bindata` | `modifybins` | `fitmodel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `probdefault` | `validatemodel`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71
“Credit Scorecard Modeling with Missing Values” on page 8-57
“Troubleshooting Credit Scorecard Results” on page 8-64
“Credit Scorecard Modeling Workflow” on page 8-52
“About Credit Scorecards” on page 8-49

modifybins

Modify predictor's bins

Syntax

```
sc = modifybins(sc,PredictorName,Name,Value)
```

Description

`sc = modifybins(sc,PredictorName,Name,Value)` manually modifies predictor bins for numeric predictors or categorical predictors using optional name-value pair arguments. For numeric predictors, minimum value, maximum value, and cut points can be specified. For categorical predictors, category groupings can be specified. Bin labels can be specified for both types of predictors.

Examples

Modify Predictor Bins for Numeric Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

The predictor `CustIncome` is numeric. By default, each value of a predictor is placed in a separate bin.

```
bi = bininfo(sc,'CustIncome')
```

```
bi=46x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ---      ---      ---      ---      ---      ---
{'18000'}      2       3      0.66667     -1.1099     0.0056227
{'19000'}      1       2       0.5      -1.3976     0.0053002
{'20000'}      4       2       2      -0.011271     6.3641e-07
{'21000'}      6       3       2      -0.011271     9.5462e-07
{'22000'}      4       2       2      -0.011271     6.3641e-07
{'23000'}      4       4       1     -0.70442     0.0035885
{'24000'}      5       5       1     -0.70442     0.0044856
{'25000'}      4       9      0.44444     -1.5153     0.026805
{'26000'}      4      11      0.36364     -1.716      0.038999
{'27000'}      6       6       1     -0.70442     0.0053827
{'28000'}     13      11      1.1818     -0.53736     0.0061896
{'29000'}     11      10       1.1     -0.60911     0.0069988
{'30000'}     18      16      1.125     -0.58664     0.010493
{'31000'}     24       8       3      0.39419     0.0038382
{'32000'}     21      15      1.4     -0.36795     0.0042797
{'33000'}     35      19      1.8421     -0.093509     0.00039951
```

```

:
```

Use `modifybins` to set a minimum value of 0, and cut points every 10000, from 20000 to 60000. Display updated bin information, including cut points.

```

sc = modifybins(sc, 'CustIncome', 'MinValue', 0, 'CutPoints', 20000:10000:60000);
[bi, cp] = bininfo(sc, 'CustIncome')
```

```

bi=7x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____
{'[0,20000)'}      3       5       0.6     -1.2152     0.010765
{'[20000,30000)'}  61      63     0.96825    -0.73668     0.060942
{'[30000,40000)'} 324     173     1.8728    -0.076967    0.0024846
{'[40000,50000)'} 304     123     2.4715     0.20042     0.013781
{'[50000,60000)'} 103      32     3.2188     0.46457     0.022144
{'[60000,Inf]'}    8       1       8         1.375     0.010235
{'Totals'}        803    397     2.0227      NaN     0.12035
```

```

cp = 5x1
```

```

20000
30000
40000
50000
60000
```

The first and last bins contain very few points. To merge the first bin into the second one, remove the first cut point. Similarly, to merge the last bin into the second-to-last one, remove the last cut point. Then use `modifybins` to update the scorecard, and display updated bin information.

```

cp(1)=[];
cp(end)=[];
sc = modifybins(sc, 'CustIncome', 'CutPoints', cp);
bi = bininfo(sc, 'CustIncome')
```

```

bi=5x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____
{'[0,30000)'}      64      68     0.94118    -0.76504     0.070065
{'[30000,40000)'} 324     173     1.8728    -0.076967    0.0024846
{'[40000,50000)'} 304     123     2.4715     0.20042     0.013781
{'[50000,Inf]'}   111      33     3.3636     0.5086     0.028028
{'Totals'}        803    397     2.0227      NaN     0.11436
```

Modify Predictor Bins for Categorical Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

The binning map or rules for categorical data are summarized in a "category grouping" table, returned as an optional output. By default, each category is placed in a separate bin. Here is the information for the predictor `ResStatus`.

```
[bi,cg] = bininfo(sc, 'ResStatus')
```

```
bi=4x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ----      -
      {'Home Owner'} 365    177    2.0621    0.019329    0.0001682
      {'Tenant' }    307    167    1.8383   -0.095564    0.0036638
      {'Other'  }    131     53    2.4717    0.20049    0.0059418
      {'Totals'  }    803   397    2.0227         NaN    0.0097738
```

```
cg=3x2 table
      Category      BinNumber
      ----      -
      {'Home Owner'}      1
      {'Tenant' }      2
      {'Other'  }      3
```

To group categories 'Tenant' and 'Other', modify the category grouping table `cg`, so the bin number for 'Other' is the same as the bin number for 'Tenant'. Then use `modifybins` to update the scorecard.

```
cg.BinNumber(3) = 2;
sc = modifybins(sc, 'ResStatus', 'CatGrouping', cg);
```

Display the updated bin information. Note that the bin labels has been updated and that the bin membership information is contained in the category grouping `cg`.

```
[bi,cg] = bininfo(sc, 'ResStatus')
```

```
bi=3x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ----      -
      {'Group1'} 365    177    2.0621    0.019329    0.0001682
      {'Group2'} 438    220    1.9909   -0.015827    0.00013772
      {'Totals'} 803    397    2.0227         NaN    0.00030592
```

```
cg=3x2 table
      Category      BinNumber
      ----      -
      {'Home Owner'}      1
      {'Tenant' }      2
      {'Other'  }      2
```

Merge Bins for Numerical and Categorical Predictors

Create a `creditscorecard` object (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID', 'GoodLabel', 0);
```

For the numerical predictor `CustAge`, use the `modifybins` function to set the following cut points:

```
cp = [25 37 49 65];
sc = modifybins(sc, 'CustAge', 'CutPoints', cp, 'MinValue', 0, 'MaxValue', 75);
bininfo(sc, 'CustAge')
```

```
ans=6x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____
{'[0,25)'}         9         8         1.125      -0.58664      0.0052464
{'[25,37)'}       125        92         1.3587      -0.39789      0.030268
{'[37,49)'}       340       183         1.8579     -0.084959      0.0031898
{'[49,65)'}       298       108         2.7593       0.31054      0.030765
{'[65,75]'}        31         6         5.1667       0.93781      0.022031
{'Totals' }       803       397         2.0227         NaN         0.0915
```

Use the `modifybins` function to merge the 2nd and 3rd bins.

```
sc = modifybins(sc, 'CustAge', 'CutPoints', cp([1 3 4]));
bininfo(sc, 'CustAge')
```

```
ans=5x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____
{'[0,25)'}         9         8         1.125     -0.58664      0.0052464
{'[25,49)'}       465       275         1.6909     -0.17915      0.020355
{'[49,65)'}       298       108         2.7593       0.31054      0.030765
{'[65,75]'}        31         6         5.1667       0.93781      0.022031
{'Totals' }       803       397         2.0227         NaN         0.078397
```

Display bin information for the categorical predictor `ResStatus`.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

```
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____
{'Home Owner'}    365       177         2.0621       0.019329      0.0001682
{'Tenant' }       307       167         1.8383     -0.095564      0.0036638
{'Other' }        131         53         2.4717       0.20049      0.0059418
{'Totals' }       803       397         2.0227         NaN         0.0097738
```

Use the `modifybins` function to merge categories 2 and 3.

ans=3x6 table					
Bin	Good	Bad	Odds	WOE	InfoValue
{'Group1'}	365	177	2.0621	0.019329	0.0001682
{'Group2'}	438	220	1.9909	-0.015827	0.00013772
{'Totals'}	803	397	2.0227	NaN	0.00030592

Create a `creditscorecard` object (using a dataset from Refaat 2011).

For the numerical predictor `TmAtAddress`, use the `modifybins` function to set the following cut points:

ans=5*6 table					
Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,30)' }	330	154	2.1429	0.057722	0.0013305
{ '[30,80)' }	379	201	1.8856	-0.070187	0.0024086
{ '[80,120)' }	78	36	2.1667	0.068771	0.00044396
{ '[120,210]' }	16	6	2.6667	0.27641	0.0013301
{ 'Totals' }	803	397	2.0227	NaN	0.0055131

15-1927

```
sc = modifybins(sc, 'TmAtAddress', 'CutPoints', [cp(1) 50 cp(2:end)]);
bininfo(sc, 'TmAtAddress')
```

```
ans=6×6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____
{'[0,30)'}      330      154      2.1429      0.057722      0.0013305
{'[30,50)'}      211      104      2.0288      0.0030488      2.4387e-06
{'[50,80)'}      168      97      1.732      -0.15517      0.005449
{'[80,120)'}      78      36      2.1667      0.068771      0.00044396
{'[120,210]'}      16      6      2.6667      0.27641      0.0013301
{'Totals'}      803      397      2.0227      NaN      0.0085559
```

Display bin information for the categorical predictor ResStatus.

```
[bi,cg] = bininfo(sc, 'ResStatus')
```

```
bi=4×6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____
{'Home Owner'}      365      177      2.0621      0.019329      0.0001682
{'Tenant'}      307      167      1.8383      -0.095564      0.0036638
{'Other'}      131      53      2.4717      0.20049      0.0059418
{'Totals'}      803      397      2.0227      NaN      0.0097738
```

```
cg=3×2 table
      Category      BinNumber
      _____      _____
{'Home Owner'}      1
{'Tenant'}      2
{'Other'}      3
```

Use the modifybins function to merge categories 2 and 3.

```
cg.BinNumber(3) = 2;
sc = modifybins(sc, 'ResStatus', 'CatGrouping', cg);
bininfo(sc, 'ResStatus')
```

```
ans=3×6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____
{'Group1'}      365      177      2.0621      0.019329      0.0001682
{'Group2'}      438      220      1.9909      -0.015827      0.00013772
{'Totals'}      803      397      2.0227      NaN      0.00030592
```

Use the modifybins function to split bin 2 and put Other under bin 3.

```
cg.BinNumber(3) = 3;
sc = modifybins(sc, 'ResStatus', 'CatGrouping', cg);
[bi,cg] = bininfo(sc, 'ResStatus')
```

bi=4×6 table

Bin	Good	Bad	Odds	WOE	InfoValue
{'Home Owner'}	365	177	2.0621	0.019329	0.0001682
{'Tenant' }	307	167	1.8383	-0.095564	0.0036638
{'Other' }	131	53	2.4717	0.20049	0.0059418
{'Totals' }	803	397	2.0227	NaN	0.0097738

cg=3×2 table

Category	BinNumber
{'Home Owner'}	1
{'Tenant' }	2
{'Other' }	3

Modify Bin Labels

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Use `modifybins` to reset the minimum value and create three bins for the predictor `CustIncome` and display updated bin information.

```
sc = modifybins(sc, 'CustIncome', 'MinValue', 0, 'CutPoints', [30000 50000]);
bi = bininfo(sc, 'CustIncome')
```

bi=4×6 table

Bin	Good	Bad	Odds	WOE	InfoValue
{'[0,30000)'} }	64	68	0.94118	-0.76504	0.070065
{'[30000,50000)'} }	628	296	2.1216	0.047762	0.0017421
{'[50000,Inf]'} }	111	33	3.3636	0.5086	0.028028
{'Totals' }	803	397	2.0227	NaN	0.099836

Modify the bin labels and display updated bin information.

```
NewLabels = {'Up to 30k', '30k to 50k', '50k and more'};
sc = modifybins(sc, 'CustIncome', 'BinLabels', NewLabels);
bi = bininfo(sc, 'CustIncome')
```

bi=4×6 table

Bin	Good	Bad	Odds	WOE	InfoValue
{'Up to 30k' }	64	68	0.94118	-0.76504	0.070065
{'30k to 50k' }	628	296	2.1216	0.047762	0.0017421
{'50k and more' }	111	33	3.3636	0.5086	0.028028

```
{'Totals'      }      803      397      2.0227      NaN      0.099836
```

Bin labels should be the last bin-modification step. As in this example, user-defined bin labels often contain information about the cut points, minimum, or maximum values for numeric data, or information about category groupings for categorical data. To prevent situations where user-defined labels and cut points are inconsistent (and labels are misleading), the `creditscorecard` object overrides user-defined labels every time the bins are modified using `modifybins`.

To illustrate `modifybins` overriding user-defined labels every time the bins are modified, reset the first cut point to 31000 and display updated bin information. Note that the bin labels are reset to their default format and accurately reflect the change in the cut points.

```
sc = modifybins(sc, 'CustIncome', 'CutPoints', [31000 50000]);
bi = bininfo(sc, 'CustIncome')
```

bi=4x6 table

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,31000)' }	82	84	0.97619	-0.72852	0.079751
{ '[31000,50000)' }	610	280	2.1786	0.074251	0.0040364
{ '[50000,Inf]' }	111	33	3.3636	0.5086	0.028028
{ 'Totals' }	803	397	2.0227	NaN	0.11182

Modify Bin Information When Using Missing Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

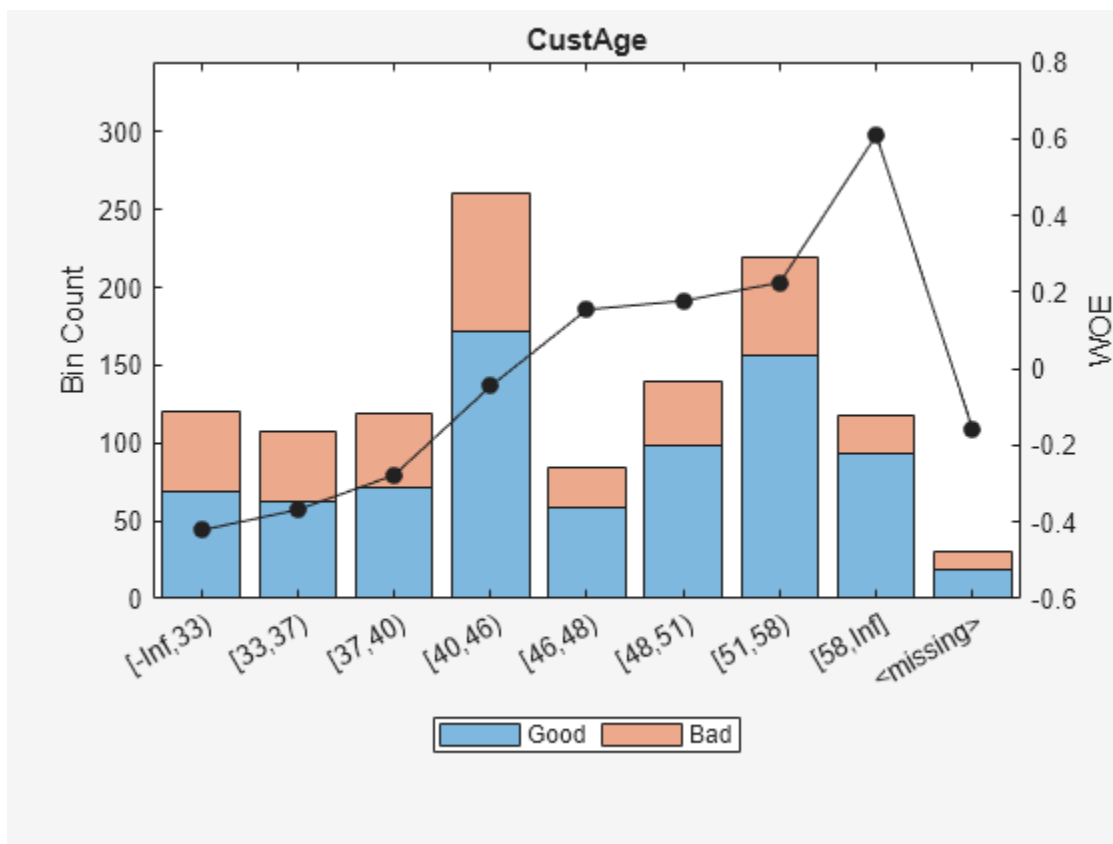
Number of rows: 1200

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

Number of missing values CustAge: 30

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

Number of missing values ResStatus: 40

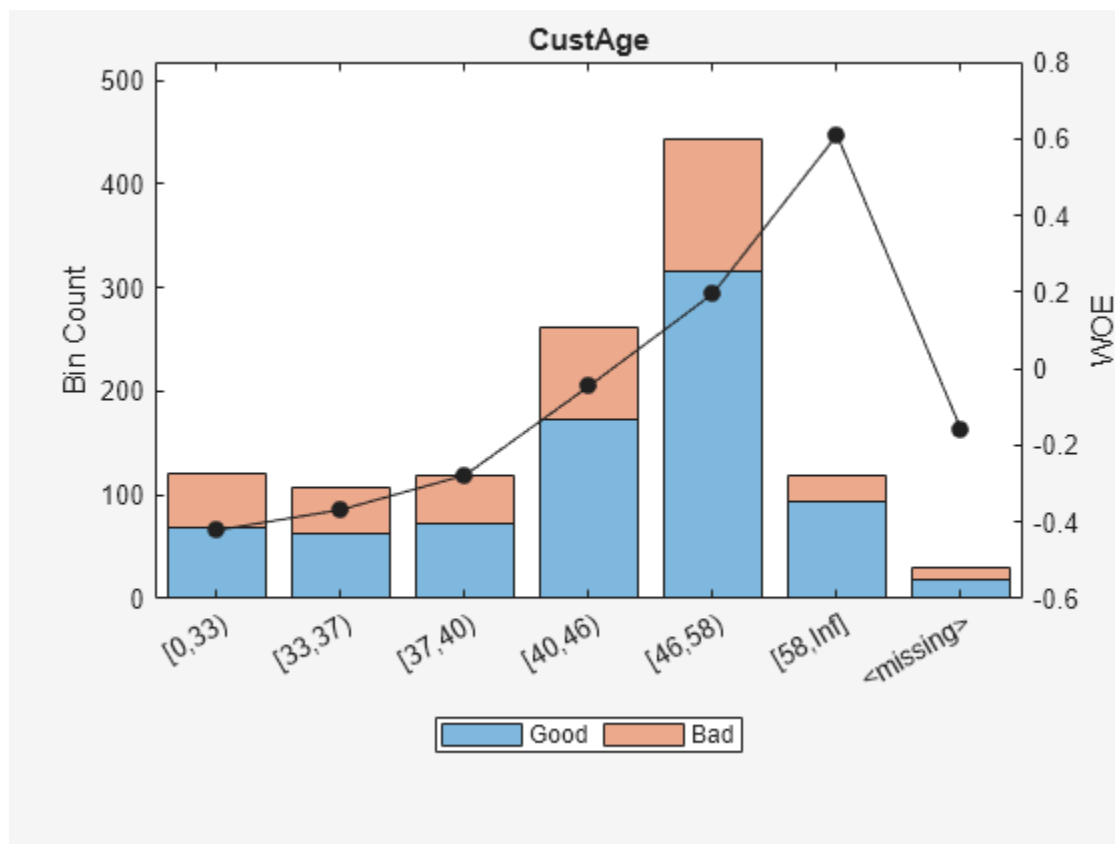


For the numeric predictor `CustAge`, remove cut points 48 and 51 and then use `modifybins` to define a 'MinValue' of 0 to manually change the binning and notice that this does not affect the data in the `<missing>` bin and the `<missing>` bin remains at the end.

```
cp(cp==48) = [];
cp(cp==51) = [];
sc = modifybins(sc, 'CustAge', 'CutPoints', cp, 'MinValue', 0);
bi = bininfo(sc, 'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,33)' }	69	52	1.3269	-0.42156	0.018993
{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549
{ '[46,58)' }	315	128	2.4609	0.19612	0.013701
{ '[58,Inf]' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.086808

```
plotbins(sc, 'CustAge')
```

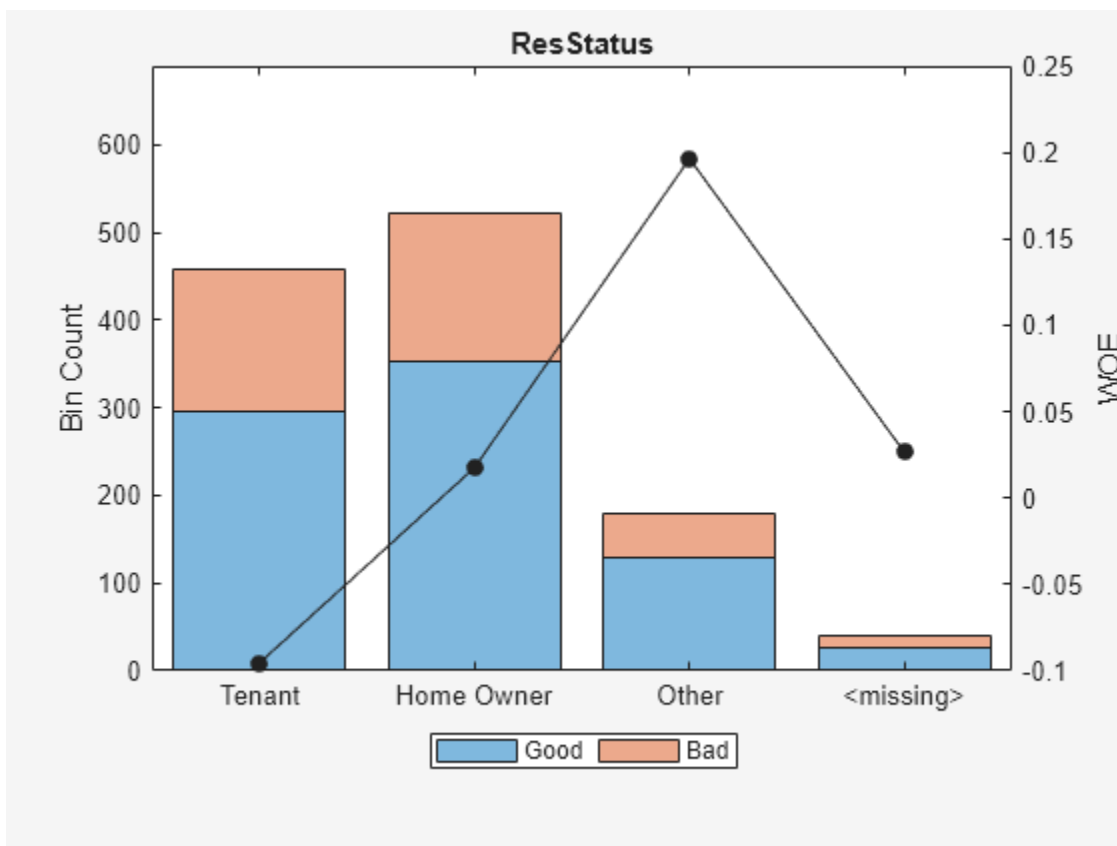


Display bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



For the categorical predictor `ResStatus`, use `modifybins` to manually merge 'HomeOwner' and 'Other' into a single group by assigning the same bin number to these categories. Notice that this does not affect the data in the <missing> bin and the <missing> bin remains at the end.

```
cg.BinNumber(3) = 2;
sc = modifybins(sc, 'ResStatus', 'CatGrouping', cg);
[bi, cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Group1' }	296	161	1.8385	-0.095463	0.0035249
{'Group2' }	480	223	2.1525	0.062196	0.0022419
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.00579

```
disp(cg)
```

Category	BinNumber
{'Tenant' }	1
{'Home Owner' }	2
{'Other' }	2

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

PredictorName — Name of predictor

character vector

Name of predictor, specified as a character vector containing the name of the predictor. `PredictorName` is case-sensitive.

Data Types: `char`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `sc = modifybins(sc,PredictorName,'MinValue',10,'CutPoints',[23, 44, 66, 88])`

MinValue — Minimum acceptable value (numeric predictors only)

-Inf (default) | numeric

Minimum acceptable value, specified as the comma-separated pair consisting of `'MinValue'` and a numeric value (for numeric predictors only). Values below this number are considered out of range.

Data Types: `double`

MaxValue — Maximum acceptable value (numeric predictors only)

Inf (default) | numeric

Maximum acceptable value, specified as the comma-separated pair consisting of `'MaxValue'` and a numeric value (for numeric predictors only). Values above this number are considered out of range.

Data Types: `double`

CutPoints — Split points between bins

each observed value of the predictor is placed in a separate bin (default) | nondecreasing numeric array

Split points between bins, specified as the comma-separated pair consisting of `'CutPoints'` and a nondecreasing numeric array. If there are `NumBins` bins, there are $n = \text{NumBins} - 1$ cut points so that $C_1, C_2, \dots, C_n$ describe the bin boundaries with the following convention:

- The first bin includes any values $\geq \text{MinValue}$, but $< C_1$.
- The second bin includes any values $\geq C_1$, but $< C_2$.
- The last bin includes any values $\geq C_n$, and $\leq \text{MaxValue}$.

Note Cut points do not include `MinValue` or `MaxValue`.

By default, cut points are defined so that each observed value of the predictor is placed in a separate bin. If the sorted observed values are $V_1, \dots, V_M$, the default cut points are $V_2, \dots, V_M$, which define M bins.

Data Types: double

CatGrouping — Table with two columns named `Category` and `BinNumber`

each category is placed in a separate bin (default) | table with two columns named `Category` and `BinNumber`

Table with two columns named `Category` and `BinNumber`, specified as the comma-separated pair consisting of `'CatGrouping'` and a table, where the first column contains an exhaustive list of categories for the predictor, and the second column contains the bin number to which each category belongs.

By default, each category is placed in a separate bin. If the observed categories are `'Cat1' ... 'CatM'`, the default category grouping is as follows.

Category	BinNumber
'Cat1 '	1
'Cat2 '	2
...	...
'CatM'	M

Data Types: double

BinLabels — Bin labels for each bin

automatically generated bin labels depending on the predictor's type (default) | cell array of character vectors

Bin labels for each bin, specified as the comma-separated pair consisting of `'BinLabels'` and a cell array of character vectors with bin label names.

Note `'BinLabels'` does not support a value of `<missing>`.

Bin labels are used to tag the bins in different object functions such as `bininfo`, `plotbins`, and `displaypoints`. A `creditscorecard` object automatically sets default bins whenever bins are modified. The default format for bin labels depends on the predictor's type.

The format for `BinLabels` is:

- Numeric data — Before any manual or automatic modification of the predictor bins, there is a bin for each observed predictor value by default. In that case, the bin labels simply show the predictor values. Once the predictor bins have been modified, there are nondefault values for `MinValue` or `MaxValue`, or nondefault cut points $C_1, C_2, \dots, C_n$. In that case, the bin labels are:
 - Bin 1 label: `' [MinValue, C1) '`
 - Bin 2 label: `' [C1, C2) '`

- Last bin label: ' [Cn, MaxValue] '

For example, if there are three bins, MinValue is 0 and MaxValue is 40, and cut point 1 is 20 and cut point 2 is 30, then the corresponding three bin labels are:

```
' [0,20) '
' [20,30) '
' [30,40] '
```

- Categorical data — For categorical data, before any modification of the predictor bins, there is one bin per category. In that case, the bin labels simply show the predictor categories. Once the bins have been modified, the labels are set to 'Group1', 'Group2', and so on, for bin 1, bin 2, and so on, respectively. For example, suppose that we have the following category grouping

Category	BinNumber
'Cat1 '	1
'Cat2 '	2
'Cat3 '	2

Bin 1 contains 'Cat1' only and its bin label is set to 'Group1'. Bin 2 contains 'Cat2' and 'Cat3' and its bin label is set to 'Group2'.

Tip Using BinLabels should be the last step (if needed) in modifying bins. BinLabels definitions are overridden each time that the bins are modified using the modifybins or autobinning functions.

Data Types: cell

Output Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, returned as an updated creditscorecard object. For more information on using the creditscorecard object, see creditscorecard.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

creditscorecard | autobinning | bininfo | predictorinfo | modifypredictor | bindata | plotbins | fitmodel | displaypoints | formatpoints | score | setmodel | probdefault | validatemodel

Topics

"Case Study for Credit Scorecard Analysis" on page 8-71

"Credit Scorecard Modeling with Missing Values" on page 8-57

"Troubleshooting Credit Scorecard Results" on page 8-64

"Credit Scorecard Modeling Workflow" on page 8-52

"About Credit Scorecards" on page 8-49

modifypredictor

Set properties of credit scorecard predictors

Syntax

```
sc = modifypredictor(sc,PredictorName)
sc = modifypredictor( ___,Name,Value)
```

Description

sc = modifypredictor(sc,PredictorName) sets the properties of the credit scorecard predictors.

sc = modifypredictor(___,Name,Value) sets the properties of the credit scorecard predictors using optional name-value pair arguments.

Examples

Modify a Predictor to Change the Predictor Type from Numeric to Categorical

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). In practice, categorical data many times is represented with numeric values. To show the case where categorical data is given as numeric data, the data for the variable 'ResStatus' is intentionally converted to numeric values.

```
load CreditCardData
data.ResStatus = double(data.ResStatus);
sc = creditscorecard(data,'IDVar','CustID')

sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBANK' 'OtherCC'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'ResStatus' 'CustIncome' 'TmWBank' 'AMBANK' 'OtherCC'}
        CategoricalPredictors: {'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBANK' 'OtherCC'}
        Data: [1200x11 table]
```

```
[T,Stats] = predictorinfo(sc,'ResStatus')
```

T=1x4 table	PredictorType	LatestBinning	LatestFillMissingType	LatestFillMissing

```
ResStatus      {'Numeric'}      {'Original Data'}      {'Original'}      {0x0 double}
```

Stats=4x1 table	
	Value
Min	1
Max	3
Mean	1.7017
Std	0.71833

Note that 'ResStatus' appears as part of the NumericPredictors property. Assume that you want 'ResStatus' to be treated as categorical data. For example, you may want to allow automatic binning algorithms to reorder the categories. Use `modifypredictor` to change the 'PredictorType' of the PredictorName 'ResStatus' from numeric to categorical.

```
sc = modifypredictor(sc, 'ResStatus', 'PredictorType', 'Categorical')
```

```
sc =
  creditscorecard with properties:

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBBalance' 'Uti
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'T
    Data: [1200x11 table]
```

```
[T,Stats] = predictorinfo(sc,'ResStatus')
```

	PredictorType	Ordinal	LatestBinning	LatestFillMissingType	LatestFillMissingValue
ResStatus	{'Categorical'}	false	{'Original Data'}	{'Original'}	{'Original'}

Stats=3x1	table
	Count
C1	542
C2	474
C3	184

Notice that 'ResStatus' now appears as part of the 'Categorical' predictors.

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

PredictorName — Predictor name

character vector | cell array of character vectors

Predictor name, specified using a character vector or cell array of character vectors containing the names of the credit scorecard predictors. `PredictorName` is case-sensitive.

Data Types: `char` | `cell`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `sc = modifypredictor(sc, {'CustAge', 'CustIncome'}, 'PredictorType', 'Categorical', 'Ordinal', true)`

PredictorType — Predictor type that one or more predictors are converted to

`''` no conversion occurs (default) | character vector with values `'Numeric'`, `'Categorical'`

Predictor type that one or more predictors are converted to, specified as the comma-separated pair consisting of `'PredictorType'` and a character vector. Possible values are:

- `''` — No conversion occurs.
- `'Numeric'` — The predictor data specified by `PredictorName` is converted to numeric.
- `'Categorical'` — The predictor data specified by `PredictorName` is converted to categorical.

Data Types: `char`

Ordinal — Indicator for whether predictors being converted to categorical are ordinal

`false` (default) | logical with values `true`, `false`

Indicator for whether predictors being converted to categorical or existing categorical predictors are treated as ordinal data, specified as the comma-separated pair consisting of `'Ordinal'` and a logical with values `true` or `false`.

Note This optional input parameter is only used for predictors of type `'Categorical'`.

Data Types: `logical`

Output Arguments

sc — Credit scorecard model

`creditscorecard` object

Credit scorecard model, returned as an updated `creditscorecard` object.

Version History

Introduced in R2015b

See Also

`creditscorecard` | `modifybins` | `predictorinfo` | `bininfo`

Summary of credit scorecard predictor properties

```
[T,Stats] = predictorinfo(sc,PredictorName)
```

[T,Stats] = predictorinfo(sc,PredictorName) returns a summary of credit scorecard predictor properties and some basic predictor statistics.

Obtain Information for a Specified PredictorName

```
load CreditCardData
sc = creditcorecard(data, 'IDVar', 'CustID')

sc =
    creditcorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustInqRatio' 'CustInqYr' 'CustInqMo' 'CustInqDay' 'CustInqNight' 'CustInqWeekend' 'CustInqHoliday' 'CustInqOther' 'CustInqTotal'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilRate' 'CustInqRatio' 'CustInqYr' 'CustInqMo' 'CustInqDay' 'CustInqNight' 'CustInqWeekend' 'CustInqHoliday' 'CustInqOther' 'CustInqTotal'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustInqRatio' 'CustInqYr' 'CustInqMo' 'CustInqDay' 'CustInqNight' 'CustInqWeekend' 'CustInqHoliday' 'CustInqOther' 'CustInqTotal'}
        Data: [1200x11 table]
```

```
[T,Stats] = predictorinfo(sc, 'CustAge')
```

	PredictorType	LatestBinning	LatestFillMissingType	LatestFillMissingValue
CustAge	{'Numeric'}	{'Original Data'}	{'Original'}	{0x0 double}

Value

```

Min      21
Max      74
Mean    45.174
Std      9.8302

```

Obtain the predictor statistics for the PredictorName of ResStatus.

```
[T,Stats] = predictorinfo(sc,'ResStatus')
```

T=1×5 table

	PredictorType	Ordinal	LatestBinning	LatestFillMissingType	LatestFillMissingValue
ResStatus	{'Categorical'}	false	{'Original Data'}	{'Original'}	{'Original'}

Stats=3×1 table

	Count
Home Owner	542
Tenant	474
Other	184

Obtain Information for a Specified PredictorName After Using fillmissing

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```

load CreditCardData
sc = creditscorecard(dataMissing,'BinMissingData',true,'IDVar','CustID');
sc = autobinning(sc);

```

Use `fillmissing` to replace missing values for the `CustAge` predictor with a value of 38.

```
sc = fillmissing(sc,'CustAge','constant',38);
```

Obtain the predictor statistics for the PredictorName of CustAge.

```
[T,Stats] = predictorinfo(sc,'CustAge')
```

T=1×4 table

	PredictorType	LatestBinning	LatestFillMissingType	LatestFillMissingValue
CustAge	{'Numeric'}	{'Automatic / Monotone'}	{'Constant'}	{38}

Stats=4×1 table

	Value
Min	21
Max	74

```
Mean    44.932
Std     9.7436
```

Use `fillmissing` to replace missing values for the `ResStatus` predictor with a mode value.

```
sc = fillmissing(sc, 'ResStatus', 'mode');
```

Obtain the predictor statistics for the `PredictorName` of `ResStatus`.

```
[T, Stats] = predictorinfo(sc, 'ResStatus')
```

`T=1×5 table`

	PredictorType	Ordinal	LatestBinning	LatestFillMissingType
ResStatus	{'Categorical'}	false	{'Automatic / Monotone'}	{'Mode'}

`Stats=3×1 table`

	Count
Tenant	457
Home Owner	563
Other	180

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

PredictorName — Predictor name

character vector

Predictor name, specified using a character vector containing the names of the credit scorecard predictor of interest. `PredictorName` is case-sensitive.

Data Types: char

Output Arguments

T — Summary information for specified predictor

table

Summary information for specified predictor, returned as table with the following columns:

- `'PredictorType'` — `'Numeric'` or `'Categorical'`.
- `'Ordinal'` — For categorical predictors, a boolean indicating whether it is ordinal.
- `'LatestBinning'` — Character vector indicating the last applied algorithm for the input argument `PredictorName`. The values are:

- 'Original Data' — When no binning is applied to the predictor.
- 'Automatic / BinningName' — Where 'BinningName' is one of the following: Monotone, Equal Width, or Equal Frequency.
- 'Manual' — After each call of `modifybins`, where either 'CutPoints', 'CatGrouping', 'MinValue', or 'MaxValue' are modified.
- 'LatestFillMissingType' — If `fillmissing` has been applied to the predictor, the value of the `Statistics` argument for `fillmissing` is displayed. If the predictor does not have any missing data, then the fill type is 'Original'.
- 'LatestFillMissingValue' — If `fillmissing` has been applied to the predictor, the fill value is displayed. If the predictor does not have any missing data, then the fill value is [].

The predictor's name is used as a row name in the table that is returned.

Stats — Summary statistics for the input PredictorName

table

Summary statistics for the input `PredictorName`, returned as a table. The corresponding value is stored in the 'Value' column.

The table's row names indicate the relevant statistics for numeric predictors:

- 'Min' — Minimum value in the sample.
- 'Max' — Maximum value in the sample.
- 'Mean' — Mean value in the sample.
- 'Std' — Standard deviation of the sample.

Note For data types other than 'double' or 'single', numeric precision may be lost for the standard deviation. Data types other than 'double' or 'single' are cast as 'double' before computing the standard deviation.

For categorical predictors, the row names contain the names of the categories, with corresponding total count in the 'Count' column.

Version History

Introduced in R2015b

See Also

`creditscorecard` | `modifybins` | `modifypredictor` | `bininfo` | `fillmissing`

bininfo

Return predictor's bin information

Syntax

```
bi = bininfo(sc,PredictorName)
bi = bininfo( ___,Name,Value)

[bi,bm] = bininfo(sc,PredictorName,Name,Value)

[bi,bm,mv] = bininfo(sc,PredictorName,Name,Value)
```

Description

`bi = bininfo(sc,PredictorName)` returns information at bin level, such as frequencies of "Good," "Bad," and bin statistics for the predictor specified in `PredictorName`.

`bi = bininfo( ___,Name,Value)` adds optional name-value arguments.

`[bi,bm] = bininfo(sc,PredictorName,Name,Value)` adds optional name-value arguments. `bininfo` also optionally returns the binning map (`bm`) or bin rules in the form of a vector of cut points for numeric predictors, or a table of category groupings for categorical predictors.

`[bi,bm,mv] = bininfo(sc,PredictorName,Name,Value)` returns information at bin level, such as frequencies of "Good," "Bad," and bin statistics for the predictor specified in `PredictorName` using optional name-value pair arguments. `bininfo` optionally returns the binning map or bin rules in the form of a vector of cut points for numeric predictors, or a table of category groupings for categorical predictors. In addition, optional name-value pair arguments `mv` returns a numeric array containing the minimum and maximum values, as set (or defined) by the user. The `mv` output argument is set to an empty array for categorical predictors.

Examples

Display Bin Information Using Default Options

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Display bin information for the categorical predictor `ResStatus`.

```
bi = bininfo(sc,'ResStatus')
```

```
bi=4x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____
{'Home Owner'}  365      177      2.0621      0.019329      0.0001682
```

```

{'Tenant'      }    307    167    1.8383    -0.095564    0.0036638
{'Other'       }    131     53    2.4717     0.20049    0.0059418
{'Totals'      }    803    397    2.0227         NaN    0.0097738

```

Display Bin Information For a creditcorecard Object Containing Weights

Use the CreditCardData.mat file to load the data (dataWeights) that contains a column (RowWeights) for the weights (using a dataset from Refaat 2011).

load CreditCardData

Create a creditcorecard object using the optional name-value pair argument for 'WeightsVar'.

```
sc = creditcorecard(dataWeights, 'WeightsVar', 'RowWeights')
```

```
sc =
```

```
creditcorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: 'RowWeights'
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalan'
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan'
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome'
    Data: [1200x12 table]

```

Display bin information for the numerical predictor 'CustIncome'. When the optional name-value pair argument 'WeightsVar' is used to specify observation (sample) weights, the bi table contains weighted counts.

```
bi = bininfo(sc, 'CustIncome');
bi(1:10,:)
```

```
ans=10x6 table
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'18000'}	0.94515	1.496	0.63179	-1.1667	0.0059198
{'19000'}	0.47588	0.80569	0.59065	-1.2341	0.0034716
{'20000'}	2.1671	1.4636	1.4806	-0.31509	0.00061392
{'21000'}	3.2522	0.88064	3.693	0.59889	0.0021303
{'22000'}	1.5438	1.2714	1.2142	-0.51346	0.0012913
{'23000'}	1.787	2.7529	0.64913	-1.1397	0.010509
{'24000'}	3.4111	2.2538	1.5135	-0.29311	0.00082663
{'25000'}	2.2333	6.1383	0.36383	-1.7186	0.042642
{'26000'}	2.1246	4.4754	0.47474	-1.4525	0.024526
{'27000'}	3.1058	3.528	0.88032	-0.83501	0.0082144

Display Bin Information Using Name-Value Arguments

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Display customized bin information for the categorical predictor `ResStatus`, keeping only the `WOE` column. The Weight-of-Evidence (WOE) is defined bin by bin, but there is no concept of "total WOE", therefore the last element in the `'Totals'` row is set to `NaN`.

```
bi = bininfo(sc, 'ResStatus', 'Statistics', 'WOE');
disp(bi)
```

Bin	Good	Bad	WOE
{ 'Home Owner' }	365	177	0.019329
{ 'Tenant' }	307	167	-0.095564
{ 'Other' }	131	53	0.20049
{ 'Totals' }	803	397	NaN

Display customized bin information for the categorical predictor `ResStatus`, keeping only the `Odd`s and `WOE` columns, without the `Totals` row.

```
bi = bininfo(sc, 'ResStatus', 'Statistics', {'Odd', 'WOE'}, 'Totals', 'Off');
disp(bi)
```

Bin	Good	Bad	Odd	WOE
{ 'Home Owner' }	365	177	2.0621	0.019329
{ 'Tenant' }	307	167	1.8383	-0.095564
{ 'Other' }	131	53	2.4717	0.20049

Display information value, entropy, Gini, and chi square statistics. For more information on these statistics, see "Statistics for a Credit Scorecard" on page 15-1957.

For information value, entropy and Gini, the value reported at a bin level is the contribution of the bin to the total value. The total information value, entropy, and Gini measures are in the `'Totals'` row.

For chi square, if there are N bins, the first $N-1$ values in the `'Chi2'` column report pairwise chi square statistics for contiguous bins. For example, this pairwise measure is also used by the `'Merge'` algorithm in autobinning to determine if two contiguous bins should be merged. In this example, the first value in the `'Chi2'` column (1.0331) is the chi square statistic of bins 1 and 2 (`'Home Owner'` and `'Tenant'`), and the second value in the column (2.5103) is the chi square statistic of bins 2 and 3 (`'Tenant'` and `'Other'`). There are no more pairwise chi square values to compute in this example, so the third element of the `'Chi2'` column is set to `NaN`. The chi square value reported in the `'Totals'` row is the chi square statistic computed over all bins.

```
bi = bininfo(sc, 'ResStatus', 'Statistics', {'InfoValue', 'Entropy', 'Gini', 'Chi2'});
disp(bi)
```

Bin	Good	Bad	InfoValue	Entropy	Gini	Chi2
-----	------	-----	-----------	---------	------	------

```

{'Home Owner'} 365 177 0.0001682 0.91138 0.43984 1.0331
{'Tenant' } 307 167 0.0036638 0.93612 0.45638 2.5103
{'Other' } 131 53 0.0059418 0.86618 0.41015 NaN
{'Totals' } 803 397 0.0097738 0.91422 0.44182 2.5549

```

Display Bin Information and Binning Map for Categorical Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

The binning map or rules for categorical data are summarized in a "category grouping" table, returned as an optional output. By default, each category is placed in a separate bin. Here is the information for the predictor `ResStatus`.

```
[bi,cg] = bininfo(sc,'ResStatus')
```

```
bi=4x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
-----
{'Home Owner'} 365    177    2.0621    0.019329    0.0001682
{'Tenant' } 307    167    1.8383   -0.095564    0.0036638
{'Other' } 131     53    2.4717    0.20049    0.0059418
{'Totals' } 803    397    2.0227         NaN    0.0097738
```

```
cg=3x2 table
      Category      BinNumber
-----
{'Home Owner'}      1
{'Tenant' }      2
{'Other' }      3
```

To group categories `Tenant` and `Other`, modify the category grouping table `cg` so that the bin number for `Other` is the same as the bin number for `Tenant`. Then use the `modifybins` function to update the scorecard.

```
cg.BinNumber(3) = 2;
sc = modifybins(sc,'ResStatus','CatGrouping',cg);
```

Display the updated bin information. The bin labels have been updated and that the bin membership information is contained in the category grouping `cg`.

```
[bi,cg] = bininfo(sc,'ResStatus')
```

```
bi=3x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
-----
{'Group1'} 365    177    2.0621    0.019329    0.0001682
{'Group2'} 438    220    1.9909   -0.015827    0.00013772
```

```
{'Totals'}      803      397      2.0227      NaN      0.00030592
```

```
cg=3x2 table
      Category      BinNumber
      _____      _____
{'Home Owner'}      1
{'Tenant' }          2
{'Other' }           2
```

Display Bin Information and Binning Map for Numeric Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

The predictor `CustIncome` is numeric. By default, each value of the predictor is placed in a separate bin.

```
bi = bininfo(sc, 'CustIncome')
```

```
bi=46x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      _____      _____      _____      _____      _____      _____
{'18000'}      2        3      0.66667      -1.1099      0.0056227
{'19000'}      1        2        0.5        -1.3976      0.0053002
{'20000'}      4        2        2        -0.011271      6.3641e-07
{'21000'}      6        3        2        -0.011271      9.5462e-07
{'22000'}      4        2        2        -0.011271      6.3641e-07
{'23000'}      4        4        1        -0.70442      0.0035885
{'24000'}      5        5        1        -0.70442      0.0044856
{'25000'}      4        9      0.44444      -1.5153      0.026805
{'26000'}      4       11      0.36364      -1.716       0.038999
{'27000'}      6        6        1        -0.70442      0.0053827
{'28000'}     13       11      1.1818      -0.53736      0.0061896
{'29000'}     11       10      1.1        -0.60911      0.0069988
{'30000'}     18       16      1.125      -0.58664      0.010493
{'31000'}     24        8        3        0.39419      0.0038382
{'32000'}     21       15      1.4        -0.36795      0.0042797
{'33000'}     35       19      1.8421     -0.093509      0.00039951
      ⋮
```

Reduce the number of bins using the `autobinning` function (the `modifybins` function can also be used).

```
sc = autobinning(sc, 'CustIncome');
```

Display the updated bin information. The binning map or rules for numeric data are summarized as "cut points," returned as an optional output (`cp`).

```
[bi,cp] = bininfo(sc,'CustIncome')
```

```
bi=8×6 table
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,29000)'} }	53	58	0.91379	-0.79457	0.06364
{'[29000,33000)'} }	74	49	1.5102	-0.29217	0.0091366
{'[33000,35000)'} }	68	36	1.8889	-0.06843	0.00041042
{'[35000,40000)'} }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)'} }	68	34	2	-0.011271	1.0819e-05
{'[42000,47000)'} }	164	66	2.4848	0.20579	0.0078175
{'[47000,Inf]'} }	183	56	3.2679	0.47972	0.041657
{'Totals' }	803	397	2.0227	NaN	0.12285

```
cp = 6×1
```

```
29000
33000
35000
40000
42000
47000
```

Manually remove the second cut point (the boundary between the second and third bins) to merge bins two and three. Use the `modifybins` function to update the scorecard.

```
cp(2) = [];
sc = modifybins(sc,'CustIncome','CutPoints',cp,'MinValue',0);
```

Display the updated bin information.

```
[bi,cp,mv] = bininfo(sc,'CustIncome')
```

```
bi=7×6 table
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[0,29000)'} }	53	58	0.91379	-0.79457	0.06364
{'[29000,35000)'} }	142	85	1.6706	-0.19124	0.0071274
{'[35000,40000)'} }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)'} }	68	34	2	-0.011271	1.0819e-05
{'[42000,47000)'} }	164	66	2.4848	0.20579	0.0078175
{'[47000,Inf]'} }	183	56	3.2679	0.47972	0.041657
{'Totals' }	803	397	2.0227	NaN	0.12043

```
cp = 5×1
```

```
29000
35000
40000
42000
47000
```

```
mv = 1×2
```

0 Inf

Note, it is recommended to avoid having bins with frequencies of zero because they lead to infinite or undefined (NaN) statistics. Use the `modifybins` or `autobinning` functions to modify bins.

Display Bin Information for Missing Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the dataMissing with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

Number of rows: 1200

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

Number of missing values CustAge: 30

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

Number of missing values ResStatus: 40

Use `creditscorecard` with the name-value argument `'BinMissingData'` set to `true` to bin the missing data in a separate bin.

```
sc = creditscorecard(dataMissing, 'IDVar', 'CustID', 'BinMissingData', true);
sc = autobinning(sc);
```

```
disp(sc)
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 1
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
    Data: [1200x11 table]

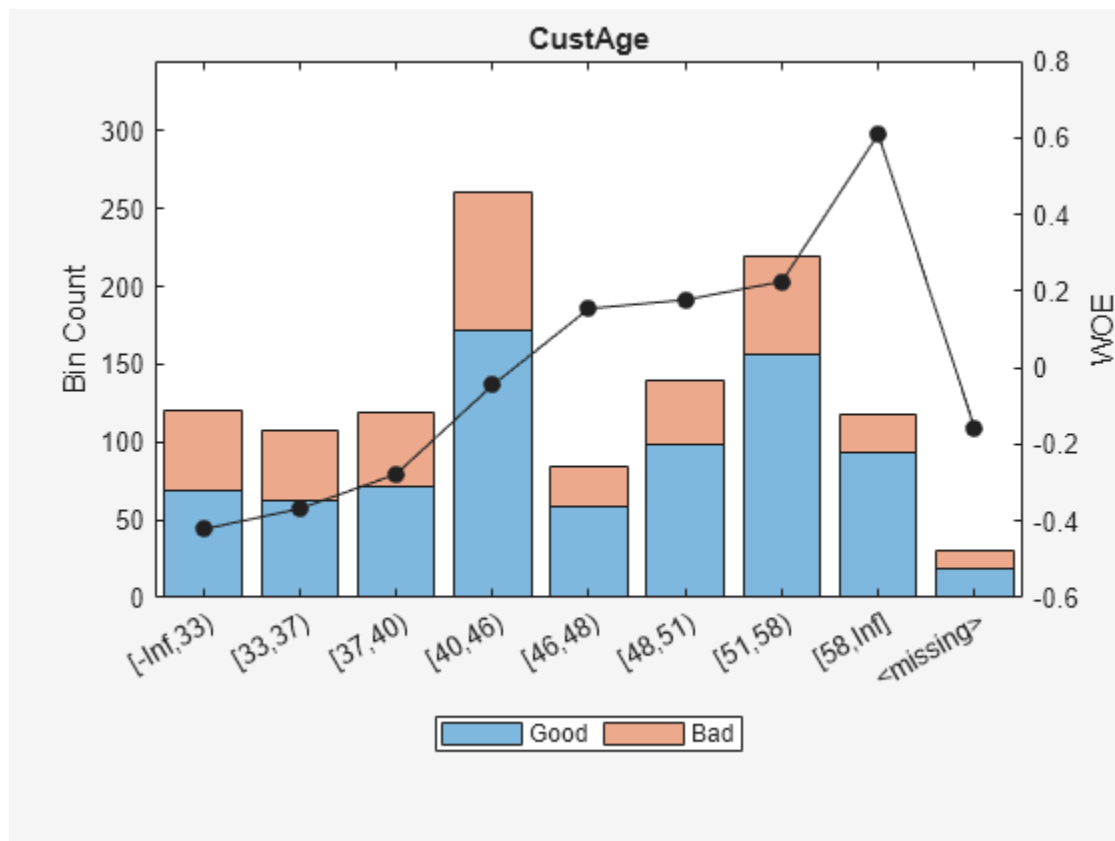
```

Display bin information for numeric data for 'CustAge' that includes missing data in a separate bin labelled <missing>.

```
bi = bininfo(sc, 'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'} }	69	52	1.3269	-0.42156	0.018993
{'[33,37)'} }	63	45	1.4	-0.36795	0.012839
{'[37,40)'} }	72	47	1.5319	-0.2779	0.0079824
{'[40,46)'} }	172	89	1.9326	-0.04556	0.0004549
{'[46,48)'} }	59	25	2.36	0.15424	0.0016199
{'[48,51)'} }	99	41	2.4146	0.17713	0.0035449
{'[51,58)'} }	157	62	2.5323	0.22469	0.0088407
{'[58,Inf]'} }	93	25	3.72	0.60931	0.032198
{'<missing>'} }	19	11	1.7273	-0.15787	0.00063885
{'Totals'} }	803	397	2.0227	NaN	0.087112

```
plotbins(sc, 'CustAge')
```



Display bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

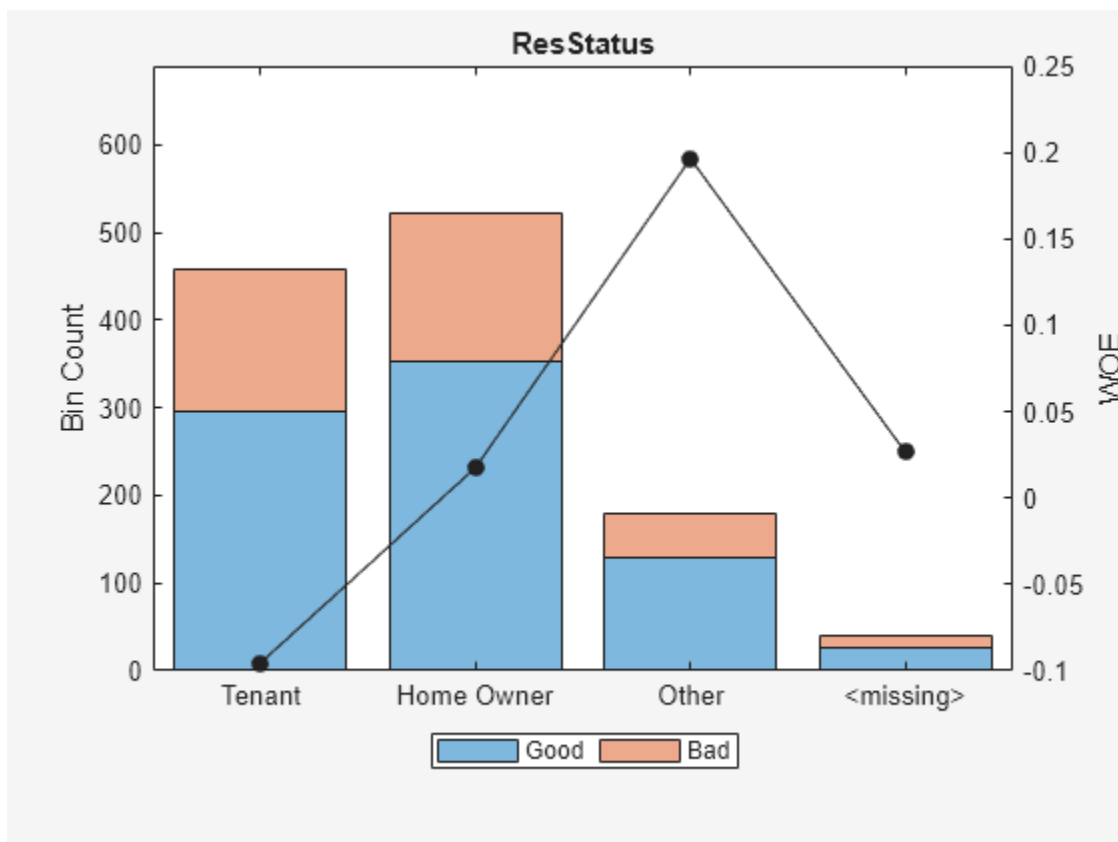

Bin	Good	Bad	Odds	WOE	InfoValue
{'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{'Other' }	128	52	2.4615	0.19637	0.0055808
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.0092627

disp(cg)

Category	BinNumber
{'Tenant' }	1
{'Home Owner' }	2
{'Other' }	3

Note that the category grouping table does not include <missing> because this is a reserved bin and users cannot interact directly with the <missing> bin.

plotbins(sc, 'ResStatus')



Input Arguments

sc — Credit scorecard model
creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

PredictorName — Predictor name

character vector

Predictor name, specified using a character vector containing the name of the predictor. `PredictorName` is case-sensitive.

Data Types: `char`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `bi = bininfo(sc, PredictorName, 'Statistics', 'WOE', 'Totals', 'On')`

Statistics — List of statistics to include for bin information

`{'Odds', 'WOE', 'InfoValue'}` (default) | character vector with values `'Odds', 'WOE', 'InfoValue', 'Entropy', 'Gini', 'Chi2'` | cell array of character vectors with values `'Odds', 'WOE', 'InfoValue', 'Entropy', 'Gini', 'Chi2'`

List of statistics to include in the bin information, specified as the comma-separated pair consisting of `'Statistics'` and a character vector or a cell array of character vectors. For more information, see “Statistics for a Credit Scorecard” on page 15-1957. Possible values are:

- `'Odds'` — Odds information is the ratio of “Goods” over “Bads.”
- `'WOE'` — Weight of Evidence. The WOE Statistic measures the deviation between the distribution of “Goods” and “Bads.”
- `'InfoValue'` — Information value. Closely tied to the WOE, it is a statistic used to determine how strong a predictor is to use in the fitting model. It measures how strong the deviation is between the distributions of “Goods” and “Bads.” However, bins with only “Good” or only “Bad” observations do lead to an infinite Information Value. Consider modifying the bins in those cases by using `modifybins` or `autobinning`.
- `'Entropy'` — Entropy is a measure of unpredictability contained in the bins. The more the number of “Goods” and “Bads” differ within the bins, the lower the entropy.
- `'Gini'` — Measure of statistical dispersion or inequality within a sample of data.
- `'Chi2'` — Measure of statistical difference and independence between groups.

Note Avoid having bins with frequencies of zero because they lead to infinite or undefined (NaN) statistics. Use `modifybins` or `autobinning` to modify bins.

Data Types: `char` | `cell`

Totals — Indicator to include row of totals at bottom information table

`'On'` (default) | character vector with values `'On', 'Off'`

Indicator to include a row of totals at the bottom of the information table, specified as the comma-separated pair consisting of 'Totals' and a character vector with values On or Off.

Data Types: char

Output Arguments

bi — Bin information

table

Bin information, returned as a table. The bin information table contains one row per bin and a row of totals. The columns contain bin descriptions, frequencies of “Good” and “Bad,” and bin statistics. Avoid having bins with frequencies of zero because they lead to infinite or undefined (NaN) statistics. Use `modifybins` or `autobinning` to modify bins.

Note When creating the `creditscorecard` object with `creditscorecard`, if the optional name-value pair argument `WeightsVar` was used to specify observation (sample) weights, then the `bi` table contains weighted counts.

bm — Binning map or rules

vector of cut points for numeric predictors | table of category groupings for categorical predictors

Binning map or rules, returned as a vector of cut points for numeric predictors, or a table of category groupings for categorical predictors. For more information, see `modifybins`.

mv — Binning minimum and maximum values

numeric array

Binning minimum and maximum values (as set or defined by the user), returned as a numeric array. The `mv` output argument is set to an empty array for categorical predictors.

More About

Statistics for a Credit Scorecard

Weight of Evidence (WOE) is a measure of the difference of the distribution of “Goods” and “Bads” within a bin.

Suppose the predictor's data takes on M possible values $b_1, \dots, b_M$. For binned data, M is a small number. The response takes on two values, “Good” and “Bad.” The frequency table of the data is given by:

	Good	Bad	Total
b_1 :	n_{11}	n_{12}	n_1
b_2 :	n_{21}	n_{22}	n_2
b_M :	n_{M1}	n_{M2}	n_M
Total:	n_{Good}	n_{Bad}	n_{Total}

The Weight of Evidence (WOE) is defined for each data value b_i as

$$\text{WOE}(i) = \log((n_{i1}/n_{\text{Good}})/(n_{i2}/n_{\text{Bad}})).$$

If you define

$$p_{\text{Good}}(i) = n_{i1}/n_{\text{Good}}, \quad p_{\text{Bad}}(i) = n_{i2}/n_{\text{Bad}}$$

then $p_{\text{Good}}(i)$ is the proportion of “Good” observations that take on the value b_i , and similarly for $p_{\text{Bad}}(i)$. In other words, $p_{\text{Good}}(i)$ gives the distribution of good observations over the M observed values of the predictor, and similarly for $p_{\text{Bad}}(i)$. With this, an equivalent formula for the WOE is

$$\text{WOE}(i) = \log(p_{\text{Good}}(i)/p_{\text{Bad}}(i)).$$

Using the same frequency table, the odds for row i are defined as

$$\text{Odds}(i) = n_{i1} / n_{i2},$$

and the odds for the sample are defined as

$$\text{OddsTotal} = n_{\text{Good}} / n_{\text{Bad}}.$$

For each row i , you can also compute its contribution to the total Information Value, given by

$$\text{InfoValue}(i) = (p_{\text{Good}}(i) - p_{\text{Bad}}(i)) * \text{WOE}(i),$$

and the total Information Value is simply the sum of all the $\text{InfoValue}(i)$ terms. (A `nansum` is returned to discard contributions from rows with no observations at all.)

Likewise, for each row i , we can compute its contribution to the total Entropy, given by

$$\text{Entropy}(i) = -1/\log(2)*(n_{i1}/n_i*\log(n_{i1}/n_i)+n_{i2}/n_i*\log(n_{i2}/n_i)),$$

and the total Entropy is simply the weighted sum of the row entropies,

$$\text{Entropy} = \sum(n_i/n_{\text{Total}} * \text{Entropy}(i)), \quad i = 1 \dots M.$$

Chi2 is computed pairwise for each pair of bins and measures the statistical difference between two groups when splitting or merging bins and is defined as:

$$\text{Chi2} = \sum(\sum((A_{ij} - E_{ij})^2/E_{ij}, \quad j=1..k), \quad i=m,m+1).$$

For more information on splitting and merging bins, see `Split` on page 15-1981 and `Merge` on page 15-1984.

Gini ratio is a measure of the parent node, that is, of the given bins/categories prior to splitting or merging. The Gini ratio is defined as:

$$\text{Gr} = 1 - G_{\text{hat}}/G_p$$

G_{hat} is the weighted Gini measure for the current split or merge:

$$G_{\text{hat}} = \sum((n_j/N) * G_j, \quad j=1..m).$$

For more information on splitting and merging bins, see `Split` on page 15-1981 and `Merge` on page 15-1984.

Using `bininfo` with Weights

When observation weights are defined using the optional `WeightsVar` argument when creating a `creditscorecard` object, instead of counting the rows that are good or bad in each bin, the `bininfo` function accumulates the weight of the rows that are good or bad in each bin.

The “frequencies” reported are no longer the basic “count” of rows, but the “cumulative weight” of the rows that are good or bad and fall in a particular bin. Once these “weighted frequencies” are known, all other relevant statistics (Good, Bad, Odds, WOE, and InfoValue) are computed with the usual formulas. For more information, see “Credit Scorecard Modeling Using Observation Weights” on page 8-55.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

[2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `autobinning` | `predictorinfo` | `modifypredictor` | `modifybins` | `bindata` | `plotbins` | `fitmodel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `probdefault` | `validatemodel`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Credit Scorecard Modeling with Missing Values” on page 8-57

“Troubleshooting Credit Scorecard Results” on page 8-64

“Credit Scorecard Modeling Workflow” on page 8-52

“About Credit Scorecards” on page 8-49

“Credit Scorecard Modeling Using Observation Weights” on page 8-55

autobinning

Perform automatic binning of given predictors

Syntax

```
sc = autobinning(sc)
sc = autobinning(sc,PredictorNames)
sc = autobinning( ___,Name,Value)
```

Description

`sc = autobinning(sc)` performs automatic binning of all predictors.

Automatic binning finds binning maps or rules to bin numeric data and to group categories of categorical data. The binning rules are stored in the `creditscorecard` object. To apply the binning rules to the `creditscorecard` object data, or to a new dataset, use `bindata`.

`sc = autobinning(sc,PredictorNames)` performs automatic binning of the predictors given in `PredictorNames`.

Automatic binning finds binning maps or rules to bin numeric data and to group categories of categorical data. The binning rules are stored in the `creditscorecard` object. To apply the binning rules to the `creditscorecard` object data, or to a new dataset, use `bindata`.

`sc = autobinning( ___,Name,Value)` performs automatic binning of the predictors given in `PredictorNames` using optional name-value pair arguments. See the name-value argument `Algorithm` for a description of the supported binning algorithms.

Automatic binning finds binning maps or rules to bin numeric data and to group categories of categorical data. The binning rules are stored in the `creditscorecard` object. To apply the binning rules to the `creditscorecard` object data, or to a new dataset, use `bindata`.

Examples

Perform Automatic Binning Using the Defaults

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning using the default options. By default, `autobinning` bins all predictors and uses the `Monotone` algorithm.

```
sc = autobinning(sc);
```

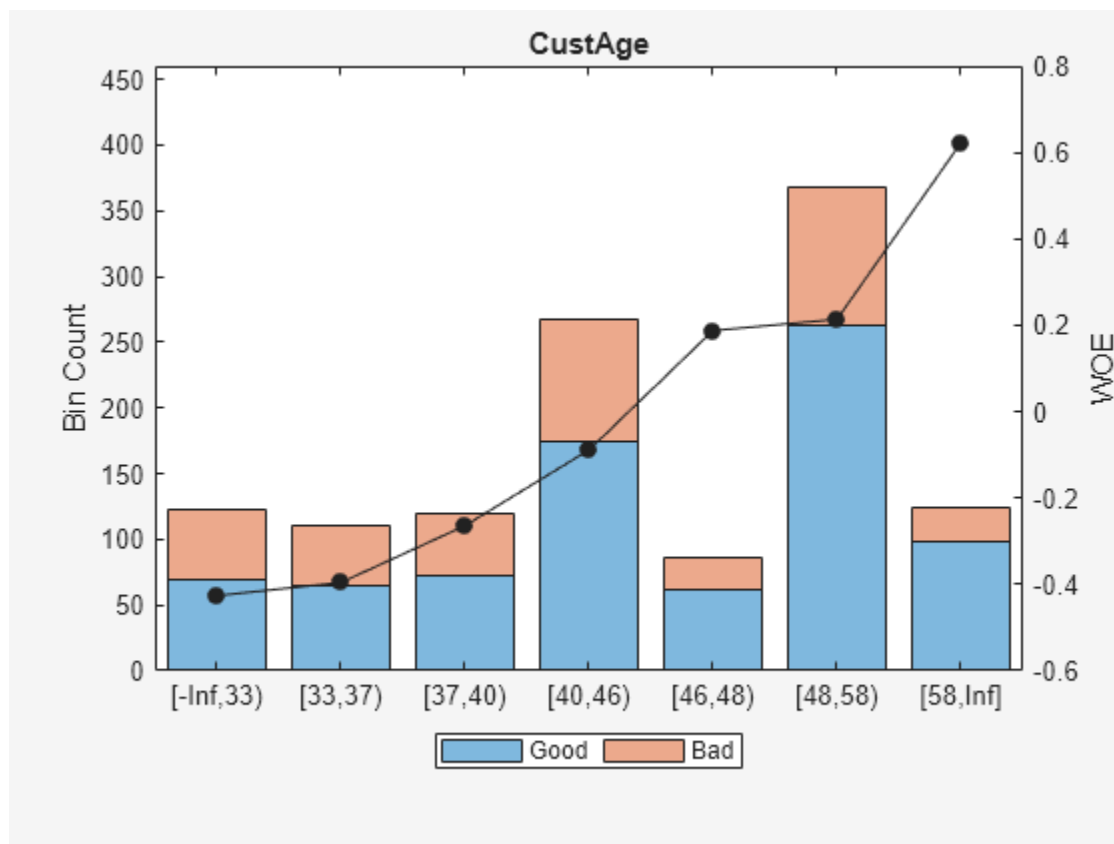
Use `bininfo` to display the binned data for the predictor `CustAge`.

```
bi = bininfo(sc, 'CustAge')
```

bi=8x6 table Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'} }	70	53	1.3208	-0.42622	0.019746
{'[33,37)'} }	64	47	1.3617	-0.39568	0.015308
{'[37,40)'} }	73	47	1.5532	-0.26411	0.0072573
{'[40,46)'} }	174	94	1.8511	-0.088658	0.001781
{'[46,48)'} }	61	25	2.44	0.18758	0.0024372
{'[48,58)'} }	263	105	2.5048	0.21378	0.013476
{'[58,Inf]'} }	98	26	3.7692	0.62245	0.0352
{'Totals' }	803	397	2.0227	NaN	0.095205

Use `plotbins` to display the histogram and WOE curve for the predictor `CustAge`.

```
plotbins(sc, 'CustAge')
```



Perform Automatic Binning with a Named Predictor Using the Defaults

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Perform automatic binning for the predictor `CustIncome` using the default options. By default, autobinning uses the Monotone algorithm.

```
sc = autobinning(sc, 'CustIncome');
```

Use `bininfo` to display the binned data.

```
bi = bininfo(sc, 'CustIncome')
```

```
bi=8x6 table
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,29000)'} }	53	58	0.91379	-0.79457	0.06364
{'[29000,33000)'} }	74	49	1.5102	-0.29217	0.0091366
{'[33000,35000)'} }	68	36	1.8889	-0.06843	0.00041042
{'[35000,40000)'} }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)'} }	68	34	2	-0.011271	1.0819e-05
{'[42000,47000)'} }	164	66	2.4848	0.20579	0.0078175
{'[47000,Inf]'} }	183	56	3.2679	0.47972	0.041657
{'Totals' }	803	397	2.0227	NaN	0.12285

Perform Automatic Binning Using Two Name-Value Pair Arguments

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Perform automatic binning for the predictor `CustIncome` using the Monotone algorithm with the initial number of bins set to 20. This example explicitly sets both the `Algorithm` and the `AlgorithmOptions` name-value arguments.

```
AlgoOptions = {'InitialNumBins',20};
sc = autobinning(sc, 'CustIncome', 'Algorithm', 'Monotone', 'AlgorithmOptions', ...
    AlgoOptions);
```

Use `bininfo` to display the binned data. Here, the cut points, which delimit the bins, are also displayed.

```
[bi,cp] = bininfo(sc, 'CustIncome')
```

```
bi=11x6 table
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,19000)'} }	2	3	0.66667	-1.1099	0.0056227
{'[19000,29000)'} }	51	55	0.92727	-0.77993	0.058516
{'[29000,31000)'} }	29	26	1.1154	-0.59522	0.017486
{'[31000,34000)'} }	80	42	1.9048	-0.060061	0.0003704
{'[34000,35000)'} }	33	17	1.9412	-0.041124	7.095e-05
{'[35000,40000)'} }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)'} }	68	34	2	-0.011271	1.0819e-05
{'[42000,43000)'} }	39	16	2.4375	0.18655	0.001542

{ '[43000,47000)' }	125	50	2.5	0.21187	0.0062972
{ '[47000,Inf]' }	183	56	3.2679	0.47972	0.041657
{ 'Totals' }	803	397	2.0227	NaN	0.13175

```
cp = 9×1
```

```
19000
29000
31000
34000
35000
40000
42000
43000
47000
```

Perform Automatic Binning Using Multiple Name-Value Pair Arguments

This example shows how to use the autobinning default Monotone algorithm and the `AlgorithmOptions` name-value pair arguments associated with the Monotone algorithm. The `AlgorithmOptions` for the Monotone algorithm are three name-value pair parameters: 'InitialNumBins', 'Trend', and 'SortCategories'. 'InitialNumBins' and 'Trend' are applicable for numeric predictors and 'Trend' and 'SortCategories' are applicable for categorical predictors.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning for the numeric predictor `CustIncome` using the Monotone algorithm with 20 bins. This example explicitly sets both the `Algorithm` argument and the `AlgorithmOptions` name-value arguments for 'InitialNumBins' and 'Trend'.

```
AlgoOptions = {'InitialNumBins', 20, 'Trend', 'Increasing'};

sc = autobinning(sc, 'CustIncome', 'Algorithm', 'Monotone', ...
    'AlgorithmOptions', AlgoOptions);
```

Use `bininfo` to display the binned data.

```
bi = bininfo(sc, 'CustIncome')
```

```
bi=11×6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
    _____
    {'[-Inf,19000)' }      2       3    0.66667    -1.1099    0.0056227
    {'[19000,29000)' }     51      55    0.92727    -0.77993    0.058516
    {'[29000,31000)' }     29      26    1.1154    -0.59522    0.017486
    {'[31000,34000)' }     80      42    1.9048    -0.060061   0.0003704
    {'[34000,35000)' }     33      17    1.9412    -0.041124   7.095e-05
```

{ '[35000,40000]' }	193	98	1.9694	-0.026696	0.00017359
{ '[40000,42000]' }	68	34	2	-0.011271	1.0819e-05
{ '[42000,43000]' }	39	16	2.4375	0.18655	0.001542
{ '[43000,47000]' }	125	50	2.5	0.21187	0.0062972
{ '[47000,Inf]' }	183	56	3.2679	0.47972	0.041657
{ 'Totals' }	803	397	2.0227	NaN	0.13175

Perform Automatic Binning for Multiple Predictors

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID');
```

Perform automatic binning for the predictor `CustIncome` and `CustAge` using the default Monotone algorithm with `AlgorithmOptions` for `InitialNumBins` and `Trend`.

```
AlgoOptions = {'InitialNumBins',20,'Trend','Increasing'};

sc = autobinning(sc,{'CustAge','CustIncome'},'Algorithm','Monotone',...
    'AlgorithmOptions',AlgoOptions);
```

Use `bininfo` to display the binned data.

```
bi1 = bininfo(sc, 'CustIncome')
```

```
bi1=11x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ---      ---      ---      ---      ---      ---
{ '[-Inf,19000]' }      2      3      0.66667      -1.1099      0.0056227
{ '[19000,29000]' }     51     55      0.92727      -0.77993      0.058516
{ '[29000,31000]' }     29     26      1.1154      -0.59522      0.017486
{ '[31000,34000]' }     80     42      1.9048      -0.060061     0.0003704
{ '[34000,35000]' }     33     17      1.9412      -0.041124     7.095e-05
{ '[35000,40000]' }    193     98      1.9694      -0.026696     0.00017359
{ '[40000,42000]' }     68     34      2      -0.011271     1.0819e-05
{ '[42000,43000]' }     39     16      2.4375      0.18655      0.001542
{ '[43000,47000]' }    125     50      2.5      0.21187      0.0062972
{ '[47000,Inf]' }     183     56      3.2679      0.47972      0.041657
{ 'Totals' }         803    397      2.0227      NaN      0.13175
```

```
bi2 = bininfo(sc, 'CustAge')
```

```
bi2=8x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ---      ---      ---      ---      ---      ---
{ '[-Inf,35)' }      93     76      1.2237      -0.50255      0.038003
{ '[35,40)' }     114     71      1.6056      -0.2309      0.0085141
{ '[40,42)' }      52     30      1.7333      -0.15437      0.0016687
{ '[42,44)' }      58     32      1.8125      -0.10971      0.00091888
{ '[44,47)' }      97     51      1.902      -0.061533     0.00047174
```

```

{'[47,62)'} } 333 130 2.5615 0.23619 0.020605
{'[62,Inf]'} } 56 7 8 1.375 0.071647
{'Totals'} } 803 397 2.0227 NaN 0.14183

```

Perform Automatic Binning for a Categorical Predictor Using the Defaults

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data);
```

Perform automatic binning for the predictor that is a categorical predictor called `ResStatus` using the default options. By default, `autobinning` uses the Monotone algorithm.

```
sc = autobinning(sc, 'ResStatus');
```

Use `bininfo` to display the binned data.

```
bi = bininfo(sc, 'ResStatus')
```

```
bi=4x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ---      ---      ---      ---      ---      ---
{'Tenant' } 307 167 1.8383 -0.095564 0.0036638
{'Home Owner'} 365 177 2.0621 0.019329 0.0001682
{'Other' } 131 53 2.4717 0.20049 0.0059418
{'Totals' } 803 397 2.0227 NaN 0.0097738
```

Perform Automatic Binning for a Categorical Predictor Using Name-Value Pair Arguments

This example shows how to modify the data (for this example only) to illustrate binning categorical predictors using the Monotone algorithm.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
```

Add two new categories and updating the response variable.

```
newdata = data;
rng('default'); %for reproducibility
Predictor = 'ResStatus';
Status = newdata.status;
NumObs = length(newdata.(Predictor));
Ind1 = randi(NumObs,100,1);
Ind2 = randi(NumObs,100,1);
newdata.(Predictor)(Ind1) = 'Subtenant';
newdata.(Predictor)(Ind2) = 'CoOwner';
```

```
Status(Ind1) = randi(2,100,1)-1;  
Status(Ind2) = randi(2,100,1)-1;
```

```
newdata.status = Status;
```

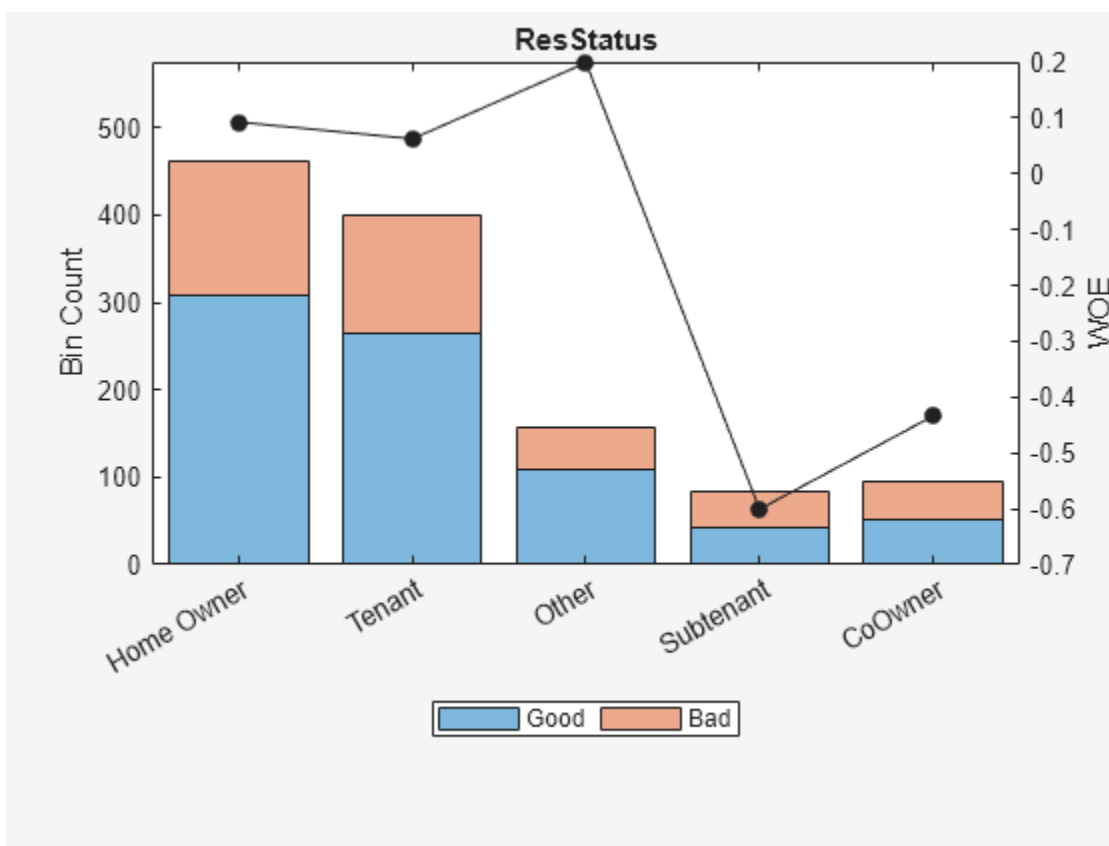
Update the `creditscorecard` object using the `newdata` and plot the bins for a later comparison.

```
scnew = creditscorecard(newdata,'IDVar','CustID');  
[bi,cg] = bininfo(scnew,Predictor)
```

```
bi=6×6 table  
      Bin      Good      Bad      Odds      WOE      InfoValue  
_____  
{ 'Home Owner' }    308    154         2    0.092373    0.0032392  
{ 'Tenant'       }    264    136    1.9412    0.06252    0.0012907  
{ 'Other'        }    109     49    2.2245    0.19875    0.0050386  
{ 'Subtenant'    }     42     42         1   -0.60077    0.026813  
{ 'CoOwner'      }     52     44    1.1818   -0.43372    0.015802  
{ 'Totals'       }    775    425    1.8235         NaN    0.052183
```

```
cg=5×2 table  
      Category      BinNumber  
_____  
{ 'Home Owner' }         1  
{ 'Tenant'       }         2  
{ 'Other'        }         3  
{ 'Subtenant'    }         4  
{ 'CoOwner'      }         5
```

```
plotbins(scnew,Predictor)
```



Perform automatic binning for the categorical Predictor using the default Monotone algorithm with the AlgorithmOptions name-value pair arguments for 'SortCategories' and 'Trend'.

```
AlgoOptions = {'SortCategories','Goods','Trend','Increasing'};
```

```
scnew = autobinning(scnew,Predictor,'Algorithm','Monotone',...  
    'AlgorithmOptions',AlgoOptions);
```

Use bininfo to display the bin information. The second output parameter 'cg' captures the bin membership, which is the bin number that each group belongs to.

```
[bi,cg] = bininfo(scnew,Predictor)
```

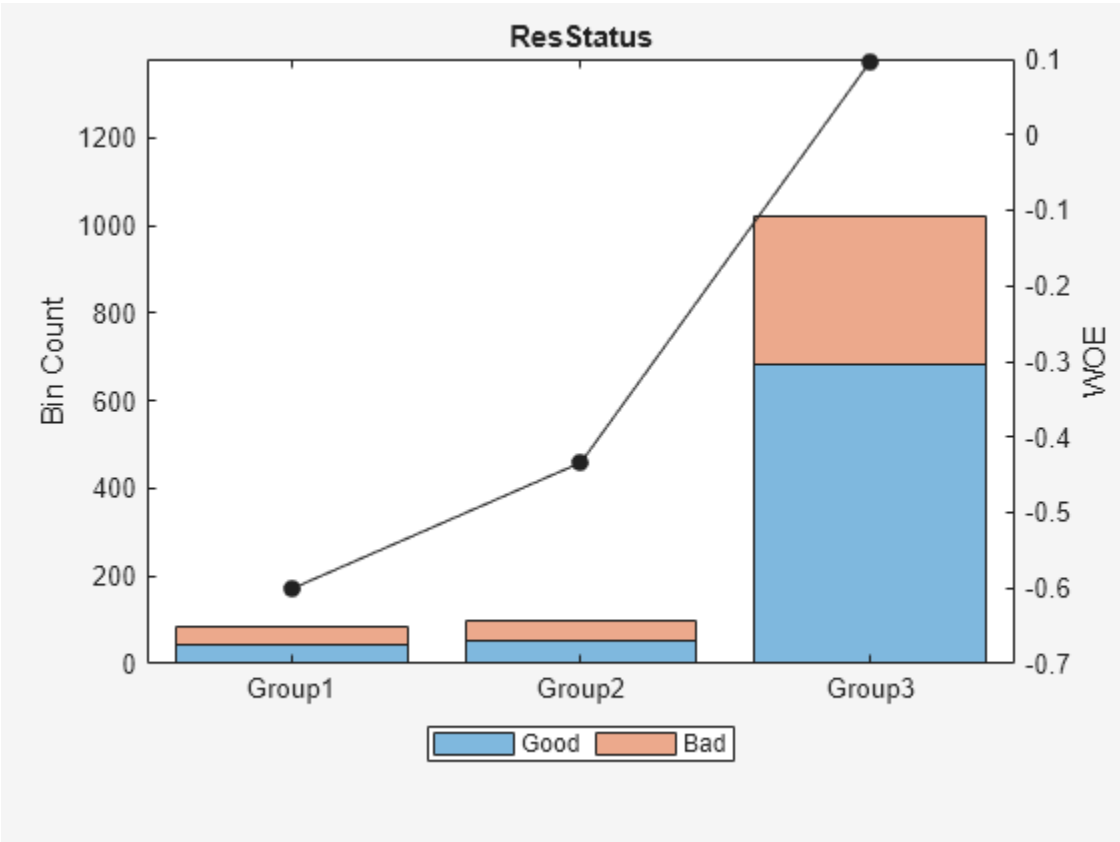
```
bi=4x6 table
    Bin      Good      Bad      Odds      WOE      InfoValue
    _____
    {'Group1'}    42      42         1    -0.60077    0.026813
    {'Group2'}    52      44    1.1818    -0.43372    0.015802
    {'Group3'}   681     339    2.0088     0.096788    0.0078459
    {'Totals'}   775     425    1.8235         NaN    0.05046
```

```
cg=5x2 table
    Category      BinNumber
    _____
    {'Subtenant' }         1
```

```
{ 'CoOwner'    }      2
{ 'Other'      }      3
{ 'Tenant'     }      3
{ 'Home Owner' }      3
```

Plot bins and compare with the histogram plotted pre-binning.

```
plotbins(scnew,Predictor)
```



Perform Automatic Binning When Using Missing Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Oth
1	53	62	<undefined>	Unknown	50000	55	Ye
2	61	22	Home Owner	Employed	52000	25	Ye
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Ye
5	68	56	Home Owner	Employed	53000	14	Ye

Number of rows: 1200

Number of missing values CustAge: 30

Number of missing values ResStatus: 40

```
sc = creditscorecard(dataMissing, 'BinMissingData', true);  
disp(sc)
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 1
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    Data: [1200x11 table]

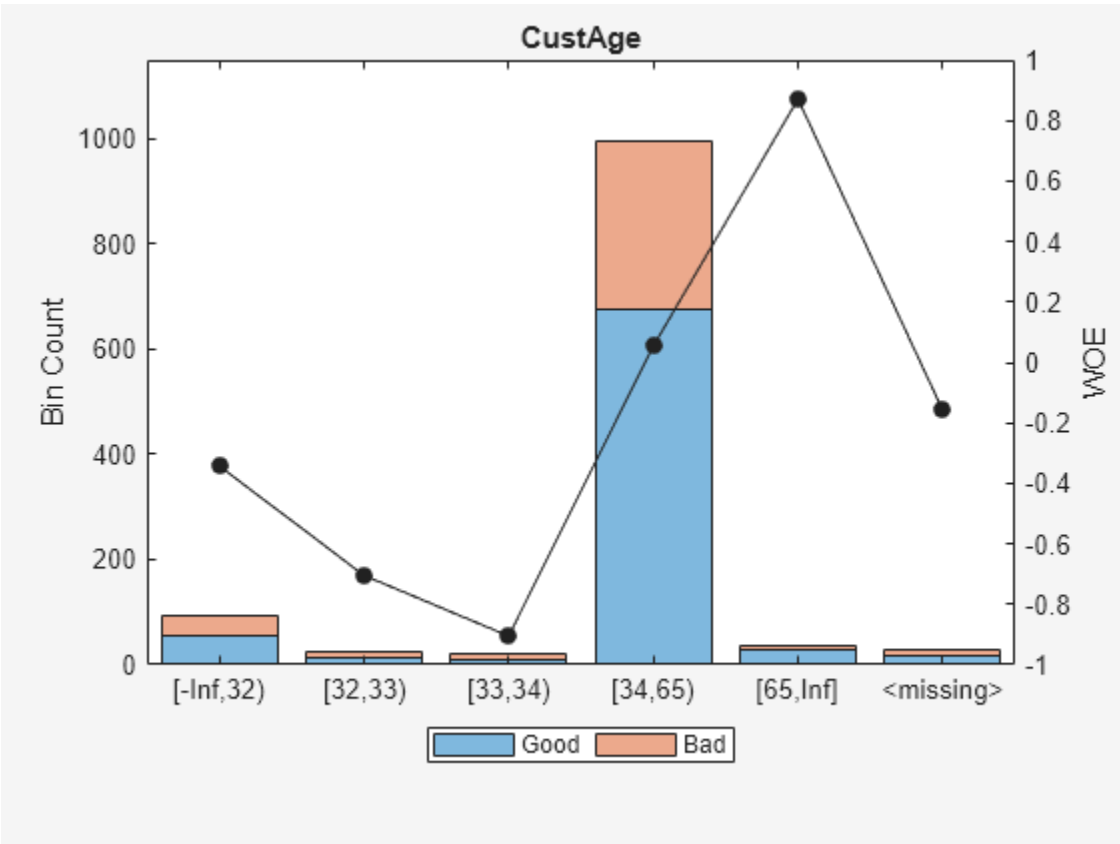
```

```
sc = autobinching(sc, 'Algorithm', 'Merge');
```

```
[bi,cp] = bininfo(sc,'CustAge');  
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,32)'} {'[32,33)'} {'[33,34)'} {'[34,65)'} {'[65,Inf]'} {'<missing>'} {'Totals'}	56 13 9 677 29 19 803	39 13 11 317 6 11 397	1.4359 1 0.81818 2.1356 4.8333 1.7273 2.0227	-0.34263 -0.70442 -0.90509 0.054351 0.87112 -0.15787 NaN	0.0097643 0.011663 0.014934 0.002424 0.018295 0.00063885 0.057718

15-1969

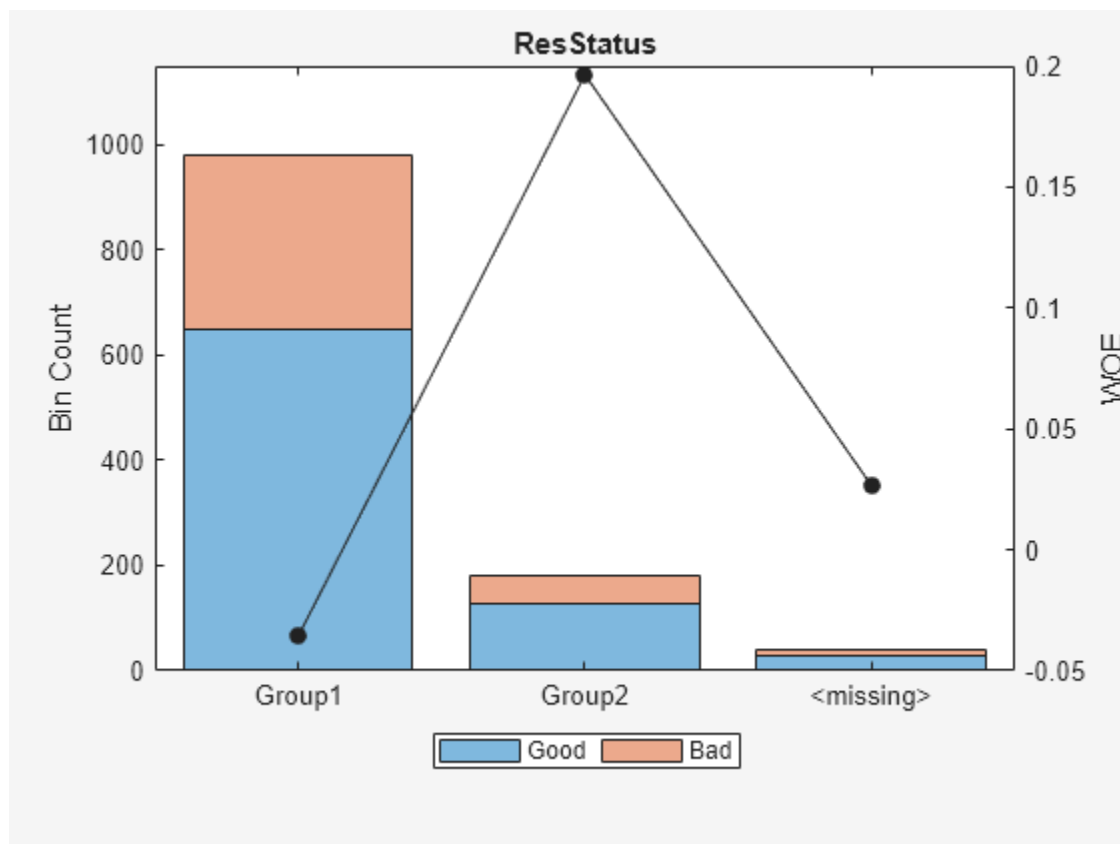


Display bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing> and this is the last bin. No matter what binning algorithm is used in autobinning, the algorithm operates on the non-missing data and the bin for the <missing> categorical values for a predictor is always the last bin.

```
[bi,cg] = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Group1' }	648	332	1.9518	-0.035663	0.0010449
{'Group2' }	128	52	2.4615	0.19637	0.0055808
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.0066489

```
plotbins(sc, 'ResStatus')
```

Perform Automatic Binning Using the Split Algorithm

This example demonstrates using the 'Split' algorithm with categorical and numeric predictors. Load the CreditCardData.mat dataset and modify so that it contains four categories for the predictor 'ResStatus' to demonstrate how the split algorithm works.

```
load CreditCardData.mat
x = data.ResStatus;
Ind = find(x == 'Tenant');
Nx = length(Ind);
x(Ind(1:floor(Nx/3))) = 'Subletter';
data.ResStatus = x;
```

Create a creditcorecard and use bininfo to display the 'Statistics'.

```
sc = creditcorecard(data, 'IDVar', 'CustID');
[bil,cgl] = bininfo(sc, 'ResStatus', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bil)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Home Owner' }	365	177	2.0621	0.019329	0.0001682
{ 'Tenant' }	204	112	1.8214	-0.1048	0.0029415
{ 'Other' }	131	53	2.4717	0.20049	0.0059418

```

{'Subletter' } 103    55    1.8727    -0.077023    0.00079103
{'Totals'    } 803    397    2.0227         NaN    0.0098426

```

```
disp(cg1)
```

Category	BinNumber
'Home Owner'	1
'Tenant'	2
'Other'	3
'Subletter'	4

Using the Split Algorithm with a Categorical Predictor

Apply presorting to the 'ResStatus' category using the default sorting by 'Odds' and specify the 'Split' algorithm.

```

sc = autobinning(sc, 'ResStatus', 'Algorithm', 'split', 'AlgorithmOptions', ...
    {'Measure', 'gini', 'SortCategories', 'odds', 'Tolerance', 1e4});
[bi2, cg2] = bininfo(sc, 'ResStatus', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi2)

```

Bin	Good	Bad	Odds	WOE	InfoValue
'Group1'	803	397	2.0227	0	0
'Totals'	803	397	2.0227	NaN	0

```
disp(cg2)
```

Category	BinNumber
'Tenant'	1
'Subletter'	1
'Home Owner'	1
'Other'	1

Using the Split Algorithm with a Numeric Predictor

To demonstrate a split for the numeric predictor, 'TmAtAddress', first use autobinning with the default 'Monotone' algorithm.

```

sc = autobinning(sc, 'TmAtAddress');
bi3 = bininfo(sc, 'TmAtAddress', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi3)

```

Bin	Good	Bad	Odds	WOE	InfoValue
'[-Inf,23)'	239	129	1.8527	-0.087767	0.0023963
'[23,83)'	480	232	2.069	0.02263	0.00030269
'[83,Inf]'	84	36	2.3333	0.14288	0.00199
'Totals'	803	397	2.0227	NaN	0.004689

Then use autobinning with the 'Split' algorithm.

```
sc = autobinning(sc, 'TmAtAddress', 'Algorithm', 'Split');
bi4 = bininfo(sc, 'TmAtAddress', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi4)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,4)'} }	20	12	1.6667	-0.19359	0.0010299
{'[4,5)'} }	4	7	0.57143	-1.264	0.015991
{'[5,23)'} }	215	110	1.9545	-0.034261	0.00031973
{'[23,33)'} }	130	39	3.3333	0.49955	0.0318
{'[33,Inf]'} }	434	229	1.8952	-0.065096	0.0023664
{'Totals' }	803	397	2.0227	NaN	0.051507

Perform Automatic Binning Using the Merge Algorithm

Load the CreditCardData.mat dataset. This example demonstrates using the 'Merge' algorithm with categorical and numeric predictors.

```
load CreditCardData.mat
```

Using the Merge Algorithm with a Categorical Predictor

To merge a categorical predictor, create a creditscorecard using default sorting by 'Odds' and then use bininfo on the categorical predictor 'ResStatus'.

```
sc = creditscorecard(data, 'IDVar', 'CustID');
[bi1, cg1] = bininfo(sc, 'ResStatus', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi1);
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Home Owner'}	365	177	2.0621	0.019329	0.0001682
{'Tenant' }	307	167	1.8383	-0.095564	0.0036638
{'Other' }	131	53	2.4717	0.20049	0.0059418
{'Totals' }	803	397	2.0227	NaN	0.0097738

```
disp(cg1);
```

Category	BinNumber
{'Home Owner'}	1
{'Tenant' }	2
{'Other' }	3

Use autobinning and specify the 'Merge' algorithm.

```
sc = autobinning(sc, 'ResStatus', 'Algorithm', 'Merge');
[bi2, cg2] = bininfo(sc, 'ResStatus', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi2)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Group1'}	672	344	1.9535	-0.034802	0.0010314

```

{'Group2'}    131    53    2.4717    0.20049    0.0059418
{'Totals'}    803   397    2.0227         NaN    0.0069732

```

```
disp(cg2)
```

Category	BinNumber
{'Tenant' }	1
{'Home Owner' }	1
{'Other' }	2

Using the Merge Algorithm with a Numeric Predictor

To demonstrate a merge for the numeric predictor, 'TmAtAddress', first use autobinning with the default 'Monotone' algorithm.

```

sc = autobinning(sc, 'TmAtAddress');
bi3 = bininfo(sc, 'TmAtAddress', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi3)

```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,23)'} }	239	129	1.8527	-0.087767	0.0023963
{'[23,83)'} }	480	232	2.069	0.02263	0.00030269
{'[83,Inf]'} }	84	36	2.3333	0.14288	0.00199
{'Totals' }	803	397	2.0227	NaN	0.004689

Then use autobinning with the 'Merge' algorithm.

```

sc = autobinning(sc, 'TmAtAddress', 'Algorithm', 'Merge');
bi4 = bininfo(sc, 'TmAtAddress', 'Statistics', {'Odds', 'WOE', 'InfoValue'});
disp(bi4)

```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,28)'} }	303	152	1.9934	-0.014566	8.0646e-05
{'[28,30)'} }	27	2	13.5	1.8983	0.054264
{'[30,98)'} }	428	216	1.9815	-0.020574	0.00022794
{'[98,106)'} }	11	13	0.84615	-0.87147	0.016599
{'[106,Inf]'} }	34	14	2.4286	0.18288	0.0012942
{'Totals' }	803	397	2.0227	NaN	0.072466

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. Use `creditscorecard` to create a `creditscorecard` object.

PredictorNames — Predictor or predictors names to automatically bin

character vector | cell array of character vectors

Predictor or predictors names to automatically bin, specified as a character vector or a cell array of character vectors containing the name of the predictor or predictors. `PredictorNames` are case-

sensitive and when no `PredictorNames` are defined, all predictors in the `PredictorVars` property of the `creditscorecard` object are binned.

Data Types: `char` | `cell`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `sc = autobinning(sc, 'Algorithm', 'EqualFrequency')`

Algorithm — Algorithm selection

`'Monotone'` (default) | character vector with values `'Monotone'`, `'Split'`, `'Merge'`, `'EqualFrequency'`, `'EqualWidth'`

Algorithm selection, specified as the comma-separated pair consisting of `'Algorithm'` and a character vector indicating which algorithm to use. The same algorithm is used for all predictors in `PredictorNames`. Possible values are:

- `'Monotone'` — (default) Monotone Adjacent Pooling Algorithm (MAPA), also known as Maximum Likelihood Monotone Coarse Classifier (MLMCC). Supervised optimal binning algorithm that aims to find bins with a monotone Weight-Of-Evidence (WOE) trend. This algorithm assumes that only neighboring attributes can be grouped. Thus, for categorical predictors, categories are sorted before applying the algorithm (see `'SortCategories'` option for `AlgorithmOptions`). For more information, see “Monotone” on page 15-1979.
- `'Split'` — Supervised binning algorithm, where a measure is used to split the data into bins. The measures supported by `'Split'` are `gini`, `chi2`, `infovalue`, and `entropy`. The resulting split must be such that the gain in the information function is maximized. For more information on these measures, see `AlgorithmOptions` and “Split” on page 15-1981.
- `'Merge'` — Supervised automatic binning algorithm, where a measure is used to merge bins into buckets. The measures supported by `'Merge'` are `chi2`, `gini`, `infovalue`, and `entropy`. The resulting merging must be such that any pair of adjacent bins is statistically different from each other, according to the chosen measure. For more information on these measures, see `AlgorithmOptions` and “Merge” on page 15-1984.
- `'EqualFrequency'` — Unsupervised algorithm that divides the data into a predetermined number of bins that contain approximately the same number of observations. This algorithm is also known as “equal height” or “equal depth.” For categorical predictors, categories are sorted before applying the algorithm (see `'SortCategories'` option for `AlgorithmOptions`). For more information, see “Equal Frequency” on page 15-1986.
- `'EqualWidth'` — Unsupervised algorithm that divides the range of values in the domain of the predictor variable into a predetermined number of bins of “equal width.” For numeric data, the width is measured as the distance between bin edges. For categorical data, width is measured as the number of categories within a bin. For categorical predictors, categories are sorted before applying the algorithm (see `'SortCategories'` option for `AlgorithmOptions`). For more information, see “Equal Width” on page 15-1987.

Data Types: `char`

AlgorithmOptions — Algorithm options for selected Algorithm

`{'InitialNumBins',10,'Trend','Auto','SortCategories','Odds'}` for Monotone (default) | cell array with `{'OptionName','OptionValue'}` for Algorithm options

Algorithm options for the selected Algorithm, specified as the comma-separated pair consisting of 'AlgorithmOptions' and a cell array. Possible values are:

- For Monotone algorithm:
 - `{'InitialNumBins',n}` — Initial number (*n*) of bins (default is 10). 'InitialNumBins' must be an integer > 2. Used for numeric predictors only.
 - `{'Trend','TrendOption'}` — Determines whether the Weight-Of-Evidence (WOE) monotonic trend is expected to be increasing or decreasing. The values for 'TrendOption' are:
 - 'Auto' — (Default) Automatically determines if the WOE trend is increasing or decreasing.
 - 'Increasing' — Look for an increasing WOE trend.
 - 'Decreasing' — Look for a decreasing WOE trend.

The value of the optional input parameter 'Trend' does not necessarily reflect that of the resulting WOE curve. The parameter 'Trend' tells the algorithm to “look for” an increasing or decreasing trend, but the outcome may not show the desired trend. For example, the algorithm cannot find a decreasing trend when the data actually has an increasing WOE trend. For more information on the 'Trend' option, see “Monotone” on page 15-1979.

- `{'SortCategories','SortOption'}` — Used for categorical predictors only. Used to determine how the predictor categories are sorted as a preprocessing step before applying the algorithm. The values of 'SortOption' are:
 - 'Odds' — (default) The categories are sorted by order of increasing values of odds, defined as the ratio of “Good” to “Bad” observations, for the given category.
 - 'Goods' — The categories are sorted by order of increasing values of “Good.”
 - 'Bads' — The categories are sorted by order of increasing values of “Bad.”
 - 'Totals' — The categories are sorted by order of increasing values of total number of observations (“Good” plus “Bad”).
 - 'None' — No sorting is applied. The existing order of the categories is unchanged before applying the algorithm. (The existing order of the categories can be seen in the category grouping optional output from `bininfo`.)

For more information, see Sort Categories on page 15-1987

- For Split algorithm:
 - `{'InitialNumBins',n}` — Specifies an integer that determines the number (*n* > 0) of bins that the predictor is initially binned into before splitting. Valid for numeric predictors only. Default is 50.
 - `{'Measure',MeasureName}` — Specifies the measure where '*MeasureName*' is one of the following: 'Gini' (default), 'Chi2', 'InfoValue', or 'Entropy'.
 - `{'MinBad',n}` — Specifies the minimum number *n* (*n* ≥ 0) of Bads per bin. The default value is 1, to avoid pure bins.
 - `{'MaxBad',n}` — Specifies the maximum number *n* (*n* ≥ 0) of Bads per bin. The default value is Inf.

- `{'MinGood',  $n$ }` — Specifies the minimum number n ($n \geq 0$) of Goods per bin. The default value is 1, to avoid pure bins.
- `{'MaxGood',  $n$ }` — Specifies the maximum number n ($n \geq 0$) of Goods per bin. The default value is Inf.
- `{'MinCount',  $n$ }` — Specifies the minimum number n ($n \geq 0$) of observations per bin. The default value is 1, to avoid empty bins.
- `{'MaxCount',  $n$ }` — Specifies the maximum number n ($n \geq 0$) of observations per bin. The default value is Inf.
- `{'MaxNumBins',  $n$ }` — Specifies the maximum number n ($n \geq 2$) of bins resulting from the splitting. The default value is 5.
- `{'Tolerance',  $Tol$ }` — Specifies the minimum gain (>0) in the information function, during the iteration scheme, to select the cut-point that maximizes the gain. The default is $1e4$.
- `{'Significance',  $n$ }` — Significance level threshold for the chi-square statistic, above which splitting happens. Values are in the interval $[0, 1]$. Default is 0.9 (90% significance level).
- `{'SortCategories', 'SortOption'}` — Used for categorical predictors only. Used to determine how the predictor categories are sorted as a preprocessing step before applying the algorithm. The values of 'SortOption' are:
 - 'Goods' — The categories are sorted by order of increasing values of "Good."
 - 'Bads' — The categories are sorted by order of increasing values of "Bad."
 - 'Odds' — (default) The categories are sorted by order of increasing values of odds, defined as the ratio of "Good" to "Bad" observations, for the given category.
 - 'Totals' — The categories are sorted by order of increasing values of total number of observations ("Good" plus "Bad").
 - 'None' — No sorting is applied. The existing order of the categories is unchanged before applying the algorithm. (The existing order of the categories can be seen in the category grouping optional output from `bininfo`.)

For more information, see Sort Categories on page 15-1987

- For Merge algorithm:
 - `{'InitialNumBins',  $n$ }` — Specifies an integer that determines the number ($n > 0$) of bins that the predictor is initially binned into before merging. Valid for numeric predictors only. Default is 50.
 - `{'Measure',  $MeasureName$ }` — Specifies the measure where ' $MeasureName$ ' is one of the following: 'Chi2' (default), 'Gini', 'InfoValue', or 'Entropy'.
 - `{'MinNumBins',  $n$ }` — Specifies the minimum number n ($n \geq 2$) of bins that result from merging. The default value is 2.
 - `{'MaxNumBins',  $n$ }` — Specifies the maximum number n ($n \geq 2$) of bins that result from merging. The default value is 5.
 - `{'Tolerance',  $n$ }` — Specifies the minimum threshold below which merging happens for the information value and entropy statistics. Valid values are in the interval $(0, 1)$. Default is $1e3$.
 - `{'Significance',  $n$ }` — Significance level threshold for the chi-square statistic, below which merging happens. Values are in the interval $[0, 1]$. Default is 0.9 (90% significance level).
 - `{'SortCategories', 'SortOption'}` — Used for categorical predictors only. Used to determine how the predictor categories are sorted as a preprocessing step before applying the algorithm. The values of 'SortOption' are:

- 'Goods' — The categories are sorted by order of increasing values of "Good."
- 'Bads' — The categories are sorted by order of increasing values of "Bad."
- 'Odds' — (default) The categories are sorted by order of increasing values of odds, defined as the ratio of "Good" to "Bad" observations, for the given category.
- 'Totals' — The categories are sorted by order of increasing values of total number of observations ("Good" plus "Bad").
- 'None' — No sorting is applied. The existing order of the categories is unchanged before applying the algorithm. (The existing order of the categories can be seen in the category grouping optional output from `bininfo`.)

For more information, see Sort Categories on page 15-1987

- For `EqualFrequency` algorithm:
 - {'NumBins', *n*} — Specifies the desired number (*n*) of bins. The default is {'NumBins', 5} and the number of bins must be a positive number.
 - {'SortCategories', 'SortOption'} — Used for categorical predictors only. Used to determine how the predictor categories are sorted as a preprocessing step before applying the algorithm. The values of 'SortOption' are:
 - 'Odds' — (default) The categories are sorted by order of increasing values of odds, defined as the ratio of "Good" to "Bad" observations, for the given category.
 - 'Goods' — The categories are sorted by order of increasing values of "Good."
 - 'Bads' — The categories are sorted by order of increasing values of "Bad."
 - 'Totals' — The categories are sorted by order of increasing values of total number of observations ("Good" plus "Bad").
 - 'None' — No sorting is applied. The existing order of the categories is unchanged before applying the algorithm. (The existing order of the categories can be seen in the category grouping optional output from `bininfo`.)

For more information, see Sort Categories on page 15-1987

- For `EqualWidth` algorithm:
 - {'NumBins', *n*} — Specifies the desired number (*n*) of bins. The default is {'NumBins', 5} and the number of bins must be a positive number.
 - {'SortCategories', 'SortOption'} — Used for categorical predictors only. Used to determine how the predictor categories are sorted as a preprocessing step before applying the algorithm. The values of 'SortOption' are:
 - 'Odds' — (default) The categories are sorted by order of increasing values of odds, defined as the ratio of "Good" to "Bad" observations, for the given category.
 - 'Goods' — The categories are sorted by order of increasing values of "Good."
 - 'Bads' — The categories are sorted by order of increasing values of "Bad."
 - 'Totals' — The categories are sorted by order of increasing values of total number of observations ("Good" plus "Bad").
 - 'None' — No sorting is applied. The existing order of the categories is unchanged before applying the algorithm. (The existing order of the categories can be seen in the category grouping optional output from `bininfo`.)

For more information, see Sort Categories on page 15-1987


```
Example: sc =
autobinning(sc, 'CustAge', 'Algorithm', 'Monotone', 'AlgorithmOptions',
{'Trend', 'Increasing'})
```

Data Types: cell

Display — Indicator to display information on status of the binning process at command line

'Off' (default) | character vector with values 'On', 'Off'

Indicator to display the information on status of the binning process at command line, specified as the comma-separated pair consisting of 'Display' and a character vector with a value of 'On' or 'Off'.

Data Types: char

Output Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, returned as an updated `creditscorecard` object containing the automatically determined binning maps or rules (cut points or category groupings) for one or more predictors. For more information on using the `creditscorecard` object, see `creditscorecard`.

Note If you have previously used the `modifybins` function to manually modify bins, these changes are lost when running `autobinning` because all the data is automatically binned based on internal `autobinning` rules.

More About

Predictor

In the context of a credit scorecard, a predictor a variable or feature used in the credit scoring model to estimate the likelihood of a borrower defaulting on a loan or credit obligation.

Predictors are statistical variables that have been identified as having a significant correlation with credit risk. They are used in predictive modeling to forecast future behavior based on historical data.

Monotone

The 'Monotone' algorithm is an implementation of the Monotone Adjacent Pooling Algorithm (MAPA), also known as Maximum Likelihood Monotone Coarse Classifier (MLMCC); see Anderson or Thomas in the “References” on page 15-1988.

Preprocessing

During the preprocessing phase, preprocessing of numeric predictors consists in applying equal frequency binning, with the number of bins determined by the 'InitialNumBins' parameter (the default is 10 bins). The preprocessing of categorical predictors consists in sorting the categories according to the 'SortCategories' criterion (the default is to sort by odds in increasing order). Sorting is not applied to ordinal predictors. See the “Sort Categories” on page 15-1987 definition or the description of `AlgorithmOptions` option for 'SortCategories' for more information.

Main Algorithm

The following example illustrates how the 'Monotone' algorithm arrives at cut points for numeric data.

Bin	Good	Bad	Iteration1	Iteration2	Iteration3	Iteration4
'[- Inf,33000) '	127	107	0.543			
'[33000,38 000) '	194	90	0.620	0.683		
'[38000,42 000) '	135	78	0.624	0.662		
'[42000,47 000) '	164	66	0.645	0.678	0.713	
'[47000,In f] '	183	56	0.669	0.700	0.740	0.766

Initially, the numeric data is preprocessed with an equal frequency binning. In this example, for simplicity, only the five initial bins are used. The first column indicates the equal frequency bin ranges, and the second and third columns have the “Good” and “Bad” counts per bin. (The number of observations is 1,200, so a perfect equal frequency binning would result in five bins with 240 observations each. In this case, the observations per bin do not match 240 exactly. This is a common situation when the data has repeated values.)

Monotone finds break points based on the cumulative proportion of “Good” observations. In the 'Iteration1' column, the first value (0.543) is the number of “Good” observations in the first bin (127), divided by the total number of observations in the bin (127+107). The second value (0.620) is the number of “Good” observations in bins 1 and 2, divided by the total number of observations in bins 1 and 2. And so forth. The first cut point is set where the minimum of this cumulative ratio is found, which is in the first bin in this example. This is the end of iteration 1.

Starting from the second bin (the first bin after the location of the minimum value in the previous iteration), cumulative proportions of “Good” observations are computed again. The second cut point is set where the minimum of this cumulative ratio is found. In this case, it happens to be in bin number 3, therefore bins 2 and 3 are merged.

The algorithm proceeds the same way for two more iterations. In this particular example, in the end it only merges bins 2 and 3. The final binning has four bins with cut points at 33,000, 42,000, and 47,000.

For categorical data, the only difference is that the preprocessing step consists in reordering the categories. Consider the following categorical data:

Bin	Good	Bad	Odds
'Home Owner'	365	177	2.062
'Tenant'	307	167	1.838
'Other'	131	53	2.474

The preprocessing step, by default, sorts the categories by 'Odds'. (See the “Sort Categories” on page 15-1987 definition or the description of `AlgorithmOptions` option for 'SortCategories' for more information.) Then, it applies the same steps described above, shown in the following table:

Bin	Good	Bad	Odds	Iteration1	Iteration2	Iteration3
'Tenant'	307	167	1.838	0.648		
'Home Owner'	365	177	2.062	0.661	0.673	
'Other'	131	53	2.472	0.669	0.683	0.712

In this case, the Monotone algorithm would not merge any categories. The only difference, compared with the data before the application of the algorithm, is that the categories are now sorted by 'Odds'.

In both the numeric and categorical examples above, the implicit 'Trend' choice is 'Increasing'. (See the description of `AlgorithmOptions` option for the 'Monotone' 'Trend' option.) If you set the trend to 'Decreasing', the algorithm looks for the maximum (instead of the minimum) cumulative ratios to determine the cut points. In that case, at iteration 1, the maximum would be in the last bin, which would imply that all bins should be merged into a single bin. Binning into a single bin is a total loss of information and has no practical use. Therefore, when the chosen trend leads to a single bin, the Monotone implementation rejects it, and the algorithm returns the bins found after the preprocessing step. This state is the initial equal frequency binning for numeric data and the sorted categories for categorical data. The implementation of the Monotone algorithm by default uses a heuristic to identify the trend ('Auto' option for 'Trend').

Split

Split is a supervised automatic binning algorithm, where a measure is used to split the data into buckets. The supported measures are `gini`, `chi2`, `infovalue`, and `entropy`.

Internally, the split algorithm proceeds as follows:

- 1 All categories are merged into a single bin.
- 2 At the first iteration, all potential cutpoint indices are tested to see which one results in the maximum increase in the information function (Gini, InfoValue, Entropy, or Chi2). That cutpoint is then selected, and the bin is split.
- 3 The same procedure is reiterated for the next sub-bins.
- 4 The algorithm stops when the maximum number of bins is reached or when the splitting does not result in any additional change in the information change function.

The following table for a categorical predictor summarizes the values of the change function at each iteration. In this example, 'Gini' is the measure of choice, such that the goal is to see a decrease of the Gini measure at each iteration.

Iteration 0 Bin Number	Member	Gini	Iteration 1 Bin Number	Member	Gini	Iteration 2 Bin Number	Member	Gini
1	'Tenant'		1	'Tenant'		1	'Tenant'	0.45638
1	'Subletter'		1	'Subletter'	0.44789	1	'Subletter'	

Iteration 0 Bin Number	Member	Gini	Iteration 1 Bin Number	Member	Gini	Iteration 2 Bin Number	Member	Gini
1	'Home Owner'		1	'Home Owner'		2	'Home Owner'	0.43984
1	'Other'		2	'Other'	0.41015	3	'Other'	0.41015
Total Gini		0.442765			0.442102			0.441822
Relative Change		0			0.001498			0.002128

The relative change at iteration i is with respect to the Gini measure of the entire bins at iteration $i-1$. The final result corresponds to that from the last iteration which, in this example, is iteration 2.

The following table for a numeric predictor summarizes the values of the change function at each iteration. In this example, 'Gini' is the measure of choice, such that the goal is to see a decrease of the Gini measure at each iteration. Since most numeric predictors in datasets contain many bins, there is a preprocessing step where the data is pre-binned into 50 equal-frequency bins. This makes the pool of valid cutpoints to choose from for splitting smaller and more manageable.

Iteration 0 Bin Number	Member	Gini	Iteration 1 Bin Number	Gini	Iteration 2 Bin Number	Gini	Iteration 3 Bin Number	Gini
1	'21 '		' [- Inf, 47] '	0.473897	' [- Inf, 47] '	0.473897	' [- Inf, 35] '	0.494941
1	'22 '		' [47, In f] '	0.385238	' [47, 61] '	0.407072	' [35, 47] '	0.463201
1	'23 '				' [61, In f] '	0.208795	' [47, 61] '	0.407072
1	'74 '			0			' [61, In f] '	0.208795
Total Gini		0.442765		0.435035		0.432048		0.430511
Relative Change		0		0.01746		0.006867		0.0356

The resulting split must be such that the information function (content) increases. As such, the best split is the one that results in the maximum information gain. The information functions supported are:

- Gini: Each split results in an increase in the Gini Ratio, defined as:

$$G_r = 1 - G_{\text{hat}}/G_p$$

G_p is the Gini measure of the parent node, that is, of the given bins/categories prior to splitting. G_{hat} is the weighted Gini measure for the current split:

$$G_{\text{hat}} = \text{Sum}((n_j/N) * \text{Gini}(j), j=1..m)$$

where

n_j is the total number of observations in the j th bin.

N is the total number of observations in the dataset.

m is the number of splits for the given variable.

$Gini(j)$ is the Gini measure for the j th bin.

The Gini measure for the split/node j is:

$$Gini(j) = 1 - (G_j^2 + B_j^2) / (n_j)^2$$

where G_j, B_j = Number of Goods and Bads for bin j .

- **InfoValue:** The information value for each split results in an increase in the total information. The split that is retained is the one which results in the maximum gain, within the acceptable gain tolerance. The Information Value (IV) for a given observation j is defined as:

$$IV = \sum (pG_i - pB_i) * \log(pG_i / pB_i), i=1..n$$

where

pG_i is the distribution of Goods at observation i , that is $Goods(i) / Total_Goods$.

pB_i is the distribution of Bads at observation i , that is $Bads(i) / Total_Bads$.

n is the total number of bins.

- **Entropy:** Each split results in a decrease in entropy variance defined as:

$$E = -\sum (n_i * E_i, i=1..n)$$

where

n_i is the total count for bin i , that is $(n_i = G_i + B_i)$.

E_i is the entropy for row (or bin) i , defined as:

$$E_i = -\sum (G_i * \log_2(G_i / n_i) + B_i * \log_2(B_i / n_i)) / N, i=1..n$$

- **Chi2:** Chi2 is computed pairwise for each pair of bins and measures the statistical difference between two groups. Splitting is selected at a point (cutpoint or category indexing) where the maximum Chi2 value is:

$$Chi2 = \sum (\sum ((A_{ij} - E_{ij})^2 / E_{ij}), j=1..k), i=m, m+1$$

where

m takes values from $1 \dots n-1$, where n is the number of bins.

k is the number of classes. Here $k = 2$ for the (Goods, Bads).

A_{ij} is the number of observations in bin i , j th class.

E_{ij} is the expected frequency of A_{ij} , which is equal to $(R_i * C_j) / N$.

R_i is the number of observations in bin i , which is equal to $\sum (A_{ij}, j=1..k)$.

C_j is the number of observations in the j th class, which is equal to $\sum (A_{ij}, i = m, m+1)$.

N is the total number of observations, which is equal to $\text{sum}(C_j, j=1..k)$.

The Chi2 measure for the entire sample (as opposed to the pairwise Chi2 measure for adjacent bins) is:

$$\text{Chi2} = \text{sum}(\text{sum}((A_{ij} - E_{ij})^2 / E_{ij}, j=1..k), i=1..n)$$

Merge

Merge is a supervised automatic binning algorithm, where a measure is used to merge bins into buckets. The supported measures are chi2, gini, infovalue, and entropy.

Internally, the merge algorithm proceeds as follows:

- 1 All categories are initially in separate bins.
- 2 The user selected information function (Chi2, Gini, InfoValue or Entropy) is computed for any pair of adjacent bins.
- 3 At each iteration, the pair with the smallest information change measured by the selected information function is merged.
- 4 The merging continues until either:
 - a All pairwise information values are greater than the threshold set by the significance level or the relative change is smaller than the tolerance.
 - b If at the end, the number of bins is still greater than the MaxNumBins allowed, merging is forced until there are at most MaxNumBins bins. Similarly, merging stops when there are only MinNumBins bins.
- 5 For categorical, original bins/categories are pre-sorted according to the sorting of choice set by the user. For numeric data, the data is preprocessed to get InitialNumBins bins of equal frequency before the merging algorithm starts.

The following table for a categorical predictor summarizes the values of the change function at each iteration. In this example, 'Chi2' is the measure of choice. The default sorting by Odds is applied as a preprocessing step. The Chi2 value reported below at row i is for bins i and $i+1$. The significance level is 0.9 (90%), so that the inverse Chi2 value is 2.705543. This is the threshold below which adjacent pairs of bins are merged. The minimum number of bins is 2.

Iteration 0 Bin Number	Member	Chi2	Iteration 1 Bin Number	Member	Chi2	Iteration 2 Bin Number	Member	Chi2
1	'Tenant'	1.007613	1	'Tenant'	0.795920	1	'Tenant'	
2	'Subletter'	0.257347	2	'Subletter'		1	'Subletter'	
3	'Home Owner'	1.566330	2	'Home Owner'	1.522914	1	'Home Owner'	1.797395
4	'Other'		3	'Other'		2	'Other'	
Total Chi2		2.573943			2.317717			1.797395

The following table for a numeric predictor summarizes the values of the change function at each iteration. In this example, 'Chi2' is the measure of choice.

Iteration 0 Bin Number	Chi2	Iteration 1 Bins	Chi2		Final Iteration Bins	Chi2
'[- Inf,22]'	0.11814	'[- Inf,22]'	0.11814		'[- Inf,33]'	8.4876
'[22,23]'	1.6464	'[22,23]'	1.6464		'[33, 48]'	7.9369
...		...			'[48,64]'	9.956
'[58,59]'	0.311578	'[58,59]'	0.27489		'[64,65]'	9.6988
'[59,60]'	0.068978	'[59,61]'	1.8403		'[65,Inf]'	NaN
'[60,61]'	1.8709	'[61,62]'	5.7946	...		
'[61,62]'	5.7946	...				
...		'[69,70]'	6.4271			
'[69,70]'	6.4271	'[70,Inf]'	NaN			
'[70,Inf]'	NaN					
Total Chi2	67.467		67.399			23.198

The resulting merging must be such that any pair of adjacent bins is statistically different from each other, according to the chosen measure. The measures supported for Merge are:

- Chi2: Chi2 is computed pairwise for each pair of bins and measures the statistical difference between two groups. Merging is selected at a point (cutpoint or category indexing) where the maximum Chi2 value is:

$$\text{Chi2} = \sum(\sum((A_{ij} - E_{ij})^2/E_{ij}, j=1..k), i=m,m+1)$$

where

m takes values from 1 ... n-1, and n is the number of bins.

k is the number of classes. Here k = 2 for the (Goods, Bads).

A_{ij} is the number of observations in bin i, jth class.

E_{ij} is the expected frequency of A_{ij} , which is equal to $(R_i * C_j) / N$.

R_i is the number of observations in bin i, which is equal to $\sum(A_{ij}, j=1..k)$.

C_j is the number of observations in the jth class, which is equal to $\sum(A_{ij}, i = m,m+1)$.

N is the total number of observations, which is equal to $\sum(C_j, j=1..k)$.

The Chi2 measure for the entire sample (as opposed to the pairwise Chi2 measure for adjacent bins) is:

$$\text{Chi2} = \sum(\sum((A_{ij} - E_{ij})^2/E_{ij}, j=1..k), i=1..n)$$

- Gini: Each merge results in a decrease in the Gini Ratio, defined as:

$$G_r = 1 - G_{\text{hat}}/G_p$$

G_p is the Gini measure of the parent node, that is, of the given bins/categories prior to merging.
 G_{hat} is the weighted Gini measure for the current merge:

$$G_{\text{hat}} = \text{Sum}((n_j/N) * \text{Gini}(j), j=1..m)$$

where

n_j is the total number of observations in the j th bin.

N is the total number of observations in the dataset.

m is the number of merges for the given variable.

$\text{Gini}(j)$ is the Gini measure for the j th bin.

The Gini measure for the merge/node j is:

$$\text{Gini}(j) = 1 - (G_j^2 + B_j^2) / (n_j)^2$$

where G_j, B_j = Number of Goods and Bads for bin j .

- **InfoValue:** The information value for each merge will result in a decrease in the total information. The merge that is retained is the one which results in the minimum gain, within the acceptable gain tolerance. The Information Value (IV) for a given observation j is defined as:

$$IV = \text{sum}((pG_i - pB_i) * \log(pG_i/pB_i), i=1..n)$$

where

pG_i is the distribution of Goods at observation i , that is $\text{Goods}(i)/\text{Total_Goods}$.

pB_i is the distribution of Bads at observation i , that is $\text{Bads}(i)/\text{Total_Bads}$.

n is the total number of bins.

- **Entropy:** Each merge results in an increase in entropy variance defined as:

$$E = -\text{sum}(n_i * E_i, i=1..n)$$

where

n_i is the total count for bin i , that is $(n_i = G_i + B_i)$.

E_i is the entropy for row (or bin) i , defined as:

$$E_i = -\text{sum}(G_i * \log_2(G_i/n_i) + B_i * \log_2(B_i/n_i)) / N, \\ i=1..n$$

Note When using the Merge algorithm, if there are pure bins (bins that have either zero count of Goods or zero count of Bads), the statistics such as Information Value and Entropy have non-finite values. To account for this, a frequency shift of .5 is applied for computing various statistics whenever the algorithm finds pure bins.

Equal Frequency

Unsupervised algorithm that divides the data into a predetermined number of bins that contain approximately the same number of observations.

EqualFrequency is defined as:

Let $v[1], v[2], \dots, v[N]$ be the sorted list of different values or categories observed in the data. Let $f[i]$ be the frequency of $v[i]$. Let $F[k] = f[1] + \dots + f[k]$ be the cumulative sum of frequencies up to the k th sorted value. Then $F[N]$ is the same as the total number of observations.

Define $\text{AvgFreq} = F[N] / \text{NumBins}$, which is the ideal average frequency per bin after binning. The n th cut point index is the index k such that the distance $\text{abs}(F[k] - n * \text{AvgFreq})$ is minimized.

This rule attempts to match the cumulative frequency up to the n th bin. If a single value contains too many observations, equal frequency bins are not possible, and the above rule yields less than NumBins total bins. In that case, the algorithm determines NumBins bins by breaking up bins, in the order in which the bins were constructed.

The preprocessing of categorical predictors consists in sorting the categories according to the 'SortCategories' criterion (the default is to sort by odds in increasing order). Sorting is not applied to ordinal predictors. See the "Sort Categories" on page 15-1987 definition or the description of AlgorithmOptions option for 'SortCategories' for more information.

Equal Width

Unsupervised algorithm that divides the range of values in the domain of the predictor variable into a predetermined number of bins of "equal width." For numeric data, the width is measured as the distance between bin edges. For categorical data, width is measured as the number of categories within a bin.

The EqualWidth option is defined as:

For numeric data, if MinValue and MaxValue are the minimum and maximum data values, then

$$\text{Width} = (\text{MaxValue} - \text{MinValue}) / \text{NumBins}$$

and the CutPoints are set to $\text{MinValue} + \text{Width}$, $\text{MinValue} + 2 * \text{Width}$, ... $\text{MaxValue} - \text{Width}$. If a MinValue or MaxValue have not been specified using the modifybins function, the EqualWidth option sets MinValue and MaxValue to the minimum and maximum values observed in the data.

For categorical data, if there are NumCats numbers of original categories, then

$$\text{Width} = \text{NumCats} / \text{NumBins},$$

and set cut point indices to the rounded values of Width , $2 * \text{Width}$, ..., $\text{NumCats} - \text{Width}$, plus 1.

The preprocessing of categorical predictors consists in sorting the categories according to the 'SortCategories' criterion (the default is to sort by odds in increasing order). Sorting is not applied to ordinal predictors. See the "Sort Categories" on page 15-1987 definition or the description of AlgorithmOptions option for 'SortCategories' for more information.

Sort Categories

As a preprocessing step for categorical data, 'Monotone', 'EqualFrequency', and 'EqualWidth' support the 'SortCategories' input. This serves the purpose of reordering the categories before applying the main algorithm. The default sorting criterion is to sort by 'Odds'. For example, suppose that the data originally looks like this:

Bin	Good	Bad	Odds
'Home Owner'	365	177	2.062
'Tenant'	307	167	1.838
'Other'	131	53	2.472

After the preprocessing step, the rows would be sorted by 'Odds' and the table looks like this:

Bin	Good	Bad	Odds
'Tenant'	307	167	1.838
'Home Owner'	365	177	2.062
'Other'	131	53	2.472

The three algorithms only merge adjacent bins, so the initial order of the categories makes a difference for the final binning. The 'None' option for 'SortCategories' would leave the original table unchanged. For a description of the sorting criteria supported, see the description of the `AlgorithmOptions` option for 'SortCategories'.

Upon the construction of a scorecard, the initial order of the categories, before any algorithm or any binning modifications are applied, is the order shown in the first output of `bininfo`. If the bins have been modified (either manually with `modifybins` or automatically with `autobinning`), use the optional output (`cg`, 'category grouping') from `bininfo` to get the current order of the categories.

The 'SortCategories' option has no effect on categorical predictors for which the 'Ordinal' parameter is set to true (see the 'Ordinal' input parameter in MATLAB categorical arrays for `categorical`). Ordinal data has a natural order, which is honored in the preprocessing step of the algorithms by leaving the order of the categories unchanged. Only categorical predictors whose 'Ordinal' parameter is false (default option) are subject to reordering of categories according to the 'SortCategories' criterion.

Using autobinning with Weights

When observation weights are defined using the optional `WeightsVar` argument when creating a `creditscorecard` object, instead of counting the rows that are good or bad in each bin, the autobinning function accumulates the weight of the rows that are good or bad in each bin.

The “frequencies” reported are no longer the basic “count” of rows, but the “cumulative weight” of the rows that are good or bad and fall in a particular bin. Once these “weighted frequencies” are known, all other relevant statistics (`Good`, `Bad`, `Odds`, `WOE`, and `InfoValue`) are computed with the usual formulas. For more information, see “Credit Scorecard Modeling Using Observation Weights” on page 8-55.

Version History

Introduced in R2014b

References

[1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.

- [2] Kerber, R. "ChiMerge: Discretization of Numeric Attributes." *AAAI-92 Proceedings*. 1992.
- [3] Liu, H., et. al. *Data Mining, Knowledge, and Discovery*. Vol 6. Issue 4. October 2002, pp. 393-423.
- [4] Refaat, M. *Data Preparation for Data Mining Using SAS*. Morgan Kaufmann, 2006.
- [5] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.
- [6] Thomas, L., et al. *Credit Scoring and Its Applications*. Society for Industrial and Applied Mathematics, 2002.

See Also

creditscorecard | bininfo | predictorinfo | modifypredictor | modifybins | bindata | plotbins | fitmodel | displaypoints | formatpoints | score | setmodel | probdefault | validatemodel

Topics

- "Case Study for Credit Scorecard Analysis" on page 8-71
- "Credit Scorecard Modeling with Missing Values" on page 8-57
- "Troubleshooting Credit Scorecard Results" on page 8-64
- "Credit Scorecard Modeling Workflow" on page 8-52
- "About Credit Scorecards" on page 8-49
- "Credit Scorecard Modeling Using Observation Weights" on page 8-55

probdefault

Likelihood of default for given data set

Syntax

```
pd = probdefault(sc)
pd = probdefault(sc,data)
```

Description

`pd = probdefault(sc)` computes the probability of default for `sc`, the data used to build the `creditscorecard` object.

`pd = probdefault(sc,data)` computes the probability of default for a given data set specified using the optional argument `data`.

By default, the data used to build the `creditscorecard` object are used. You can also supply input data, to which the same computation of probability of default is applied.

Examples

Compute Probability for Default Using Credit ScoreCard Data

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID')

sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustIncome2' 'CustIncome3' 'CustIncome4' 'CustIncome5' 'CustIncome6' 'CustIncome7' 'CustIncome8' 'CustIncome9' 'CustIncome10' 'CustIncome11' 'CustIncome12' 'CustIncome13' 'CustIncome14' 'CustIncome15' 'CustIncome16' 'CustIncome17' 'CustIncome18' 'CustIncome19' 'CustIncome20' 'CustIncome21' 'CustIncome22' 'CustIncome23' 'CustIncome24' 'CustIncome25' 'CustIncome26' 'CustIncome27' 'CustIncome28' 'CustIncome29' 'CustIncome30' 'CustIncome31' 'CustIncome32' 'CustIncome33' 'CustIncome34' 'CustIncome35' 'CustIncome36' 'CustIncome37' 'CustIncome38' 'CustIncome39' 'CustIncome40' 'CustIncome41' 'CustIncome42' 'CustIncome43' 'CustIncome44' 'CustIncome45' 'CustIncome46' 'CustIncome47' 'CustIncome48' 'CustIncome49' 'CustIncome50' 'CustIncome51' 'CustIncome52' 'CustIncome53' 'CustIncome54' 'CustIncome55' 'CustIncome56' 'CustIncome57' 'CustIncome58' 'CustIncome59' 'CustIncome60' 'CustIncome61' 'CustIncome62' 'CustIncome63' 'CustIncome64' 'CustIncome65' 'CustIncome66' 'CustIncome67' 'CustIncome68' 'CustIncome69' 'CustIncome70' 'CustIncome71' 'CustIncome72' 'CustIncome73' 'CustIncome74' 'CustIncome75' 'CustIncome76' 'CustIncome77' 'CustIncome78' 'CustIncome79' 'CustIncome80' 'CustIncome81' 'CustIncome82' 'CustIncome83' 'CustIncome84' 'CustIncome85' 'CustIncome86' 'CustIncome87' 'CustIncome88' 'CustIncome89' 'CustIncome90' 'CustIncome91' 'CustIncome92' 'CustIncome93' 'CustIncome94' 'CustIncome95' 'CustIncome96' 'CustIncome97' 'CustIncome98' 'CustIncome99' 'CustIncome100'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'Utilization' 'CustIncome2' 'CustIncome3' 'CustIncome4' 'CustIncome5' 'CustIncome6' 'CustIncome7' 'CustIncome8' 'CustIncome9' 'CustIncome10' 'CustIncome11' 'CustIncome12' 'CustIncome13' 'CustIncome14' 'CustIncome15' 'CustIncome16' 'CustIncome17' 'CustIncome18' 'CustIncome19' 'CustIncome20' 'CustIncome21' 'CustIncome22' 'CustIncome23' 'CustIncome24' 'CustIncome25' 'CustIncome26' 'CustIncome27' 'CustIncome28' 'CustIncome29' 'CustIncome30' 'CustIncome31' 'CustIncome32' 'CustIncome33' 'CustIncome34' 'CustIncome35' 'CustIncome36' 'CustIncome37' 'CustIncome38' 'CustIncome39' 'CustIncome40' 'CustIncome41' 'CustIncome42' 'CustIncome43' 'CustIncome44' 'CustIncome45' 'CustIncome46' 'CustIncome47' 'CustIncome48' 'CustIncome49' 'CustIncome50' 'CustIncome51' 'CustIncome52' 'CustIncome53' 'CustIncome54' 'CustIncome55' 'CustIncome56' 'CustIncome57' 'CustIncome58' 'CustIncome59' 'CustIncome60' 'CustIncome61' 'CustIncome62' 'CustIncome63' 'CustIncome64' 'CustIncome65' 'CustIncome66' 'CustIncome67' 'CustIncome68' 'CustIncome69' 'CustIncome70' 'CustIncome71' 'CustIncome72' 'CustIncome73' 'CustIncome74' 'CustIncome75' 'CustIncome76' 'CustIncome77' 'CustIncome78' 'CustIncome79' 'CustIncome80' 'CustIncome81' 'CustIncome82' 'CustIncome83' 'CustIncome84' 'CustIncome85' 'CustIncome86' 'CustIncome87' 'CustIncome88' 'CustIncome89' 'CustIncome90' 'CustIncome91' 'CustIncome92' 'CustIncome93' 'CustIncome94' 'CustIncome95' 'CustIncome96' 'CustIncome97' 'CustIncome98' 'CustIncome99' 'CustIncome100'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustIncome2' 'CustIncome3' 'CustIncome4' 'CustIncome5' 'CustIncome6' 'CustIncome7' 'CustIncome8' 'CustIncome9' 'CustIncome10' 'CustIncome11' 'CustIncome12' 'CustIncome13' 'CustIncome14' 'CustIncome15' 'CustIncome16' 'CustIncome17' 'CustIncome18' 'CustIncome19' 'CustIncome20' 'CustIncome21' 'CustIncome22' 'CustIncome23' 'CustIncome24' 'CustIncome25' 'CustIncome26' 'CustIncome27' 'CustIncome28' 'CustIncome29' 'CustIncome30' 'CustIncome31' 'CustIncome32' 'CustIncome33' 'CustIncome34' 'CustIncome35' 'CustIncome36' 'CustIncome37' 'CustIncome38' 'CustIncome39' 'CustIncome40' 'CustIncome41' 'CustIncome42' 'CustIncome43' 'CustIncome44' 'CustIncome45' 'CustIncome46' 'CustIncome47' 'CustIncome48' 'CustIncome49' 'CustIncome50' 'CustIncome51' 'CustIncome52' 'CustIncome53' 'CustIncome54' 'CustIncome55' 'CustIncome56' 'CustIncome57' 'CustIncome58' 'CustIncome59' 'CustIncome60' 'CustIncome61' 'CustIncome62' 'CustIncome63' 'CustIncome64' 'CustIncome65' 'CustIncome66' 'CustIncome67' 'CustIncome68' 'CustIncome69' 'CustIncome70' 'CustIncome71' 'CustIncome72' 'CustIncome73' 'CustIncome74' 'CustIncome75' 'CustIncome76' 'CustIncome77' 'CustIncome78' 'CustIncome79' 'CustIncome80' 'CustIncome81' 'CustIncome82' 'CustIncome83' 'CustIncome84' 'CustIncome85' 'CustIncome86' 'CustIncome87' 'CustIncome88' 'CustIncome89' 'CustIncome90' 'CustIncome91' 'CustIncome92' 'CustIncome93' 'CustIncome94' 'CustIncome95' 'CustIncome96' 'CustIncome97' 'CustIncome98' 'CustIncome99' 'CustIncome100'}
        Data: [1200x11 table]
```

Perform automatic binning using the default options. By default, `autobinning` uses the `Monotone` algorithm.

```
sc = autobinning(sc);
```

Fit the model.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Compute the probability of default.

```
pd = probdefault(sc);
disp(pd(1:15,:))
```

```
0.2503
0.1878
0.3173
0.1711
0.1895
0.1307
0.5218
0.2848
0.2612
0.3047
0.3418
0.2237
0.2793
0.3615
0.1653
```

Compute Probability for Default Using Credit ScoreCard Data When Using the 'BinMissingData' Option

This example describes both the assignment of points for missing data when the 'BinMissingData' option is set to `true`, and the corresponding computation of probabilities of default.

- Predictors that have missing data in the training set have an explicit bin for `<missing>` with corresponding points in the final scorecard. These points are computed from the Weight-of-Evidence (WOE) value for the `<missing>` bin and the logistic model coefficients. For scoring purposes, these points are assigned to missing values and to out-of-range values, and the final score is mapped to a probability of default when using `probdefault`.
- Predictors with no missing data in the training set have no `<missing>` bin, therefore no WOE can be estimated from the training data. By default, the points for missing and out-of-range values are set to `NaN`, and this leads to a score of `NaN` when running `score`. For predictors that have no explicit `<missing>` bin, use the name-value argument 'Missing' in `formatpoints` to indicate how missing data should be treated for scoring purposes. The final score is then mapped to a probability of default when using `probdefault`.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

Use `creditscorecard` with the name-value argument 'BinMissingData' set to `true` to bin the missing numeric or categorical data in a separate bin. Apply automatic binning.

```
sc = creditscorecard(dataMissing,'IDVar','CustID','BinMissingData',true);
sc = autobinning(sc);
```

```
disp(sc)
```

```
creditscorecard with properties:
```

```

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBBalance' 'UtilizationRate'}
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 1
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'OtherCC'}
    Data: [1200x11 table]
```

Set a minimum value of 0 for `CustAge` and `CustIncome`. With this, any negative age or income information becomes invalid or "out-of-range". For scoring and probability of default computations, out-of-range values are given the same points as missing values.

```
sc = modifybins(sc, 'CustAge', 'MinValue', 0);
sc = modifybins(sc, 'CustIncome', 'MinValue', 0);
```

Display bin information for numeric data for 'CustAge' that includes missing data in a separate bin labelled <missing>.

```
bi = bininfo(sc, 'CustAge');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,33)' }	69	52	1.3269	-0.42156	0.018993
{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549
{ '[46,48)' }	59	25	2.36	0.15424	0.0016199
{ '[48,51)' }	99	41	2.4146	0.17713	0.0035449
{ '[51,58)' }	157	62	2.5323	0.22469	0.0088407
{ '[58,Inf]' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.087112

Display bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
bi = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{ 'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{ 'Other' }	128	52	2.4615	0.19637	0.0055808
{ '<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{ 'Totals' }	803	397	2.0227	NaN	0.0092627

For the 'CustAge' and 'ResStatus' predictors, there is missing data (NaNs and <undefined>) in the training data, and the binning process estimates a WOE value of -0.15787 and 0.026469 respectively for missing data in these predictors, as shown above.

For EmpStatus and CustIncome there is no explicit bin for missing values because the training data has no missing values for these predictors.

```
bi = bininfo(sc, 'EmpStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ 'Unknown' }	396	239	1.6569	-0.19947	0.021715
{ 'Employed' }	407	158	2.5759	0.2418	0.026323
{ 'Totals' }	803	397	2.0227	NaN	0.048038

```
bi = bininfo(sc, 'CustIncome');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
-----	------	-----	------	-----	-----------

{ '[0,29000)' }	53	58	0.91379	-0.79457	0.06364
{ '[29000,33000)' }	74	49	1.5102	-0.29217	0.0091366
{ '[33000,35000)' }	68	36	1.8889	-0.06843	0.00041042
{ '[35000,40000)' }	193	98	1.9694	-0.026696	0.00017359
{ '[40000,42000)' }	68	34	2	-0.011271	1.0819e-05
{ '[42000,47000)' }	164	66	2.4848	0.20579	0.0078175
{ '[47000,Inf]' }	183	56	3.2679	0.47972	0.041657
{ 'Totals' }	803	397	2.0227	NaN	0.12285

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). For predictors that have missing data, there is an explicit `<missing>` bin, with a corresponding WOE value computed from the data. When using `fitmodel`, the corresponding WOE value for the `<missing>` bin is applied when performing the WOE transformation.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1442.8477, Chi2Stat = 4.4974731, PValue = 0.033944979
6. Adding ResStatus, Deviance = 1438.9783, Chi2Stat = 3.86941, PValue = 0.049173805
7. Adding OtherCC, Deviance = 1434.9751, Chi2Stat = 4.0031966, PValue = 0.045414057

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70229	0.063959	10.98	4.7498e-28
CustAge	0.57421	0.25708	2.2335	0.025513
ResStatus	1.3629	0.66952	2.0356	0.04179
EmpStatus	0.88373	0.2929	3.0172	0.002551
CustIncome	0.73535	0.2159	3.406	0.00065929
TmWBank	1.1065	0.23267	4.7556	1.9783e-06
OtherCC	1.0648	0.52826	2.0156	0.043841
AMBalance	1.0446	0.32197	3.2443	0.0011775

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 88.5, p-value = 2.55e-16

Scale the scorecard points by the "points, odds, and points to double the odds (PDO)" method using the `'PointsOddsAndPDO'` argument of `formatpoints`. Suppose that you want a score of 500 points to have odds of 2 (twice as likely to be good than to be bad) and that the odds double every 50 points (so that 550 points would have odds of 4).

Display the scorecard showing the scaled points for predictors retained in the fitting model.

```
sc = formatpoints(sc, 'PointsOddsAndPDO', [500 2 50]);
PointsInfo = displaypoints(sc)
```


PointsInfo=38x3 table

Predictors	Bin	Points
{'CustAge' }	{'[0,33)' }	54.062
{'CustAge' }	{'[33,37)' }	56.282
{'CustAge' }	{'[37,40)' }	60.012
{'CustAge' }	{'[40,46)' }	69.636
{'CustAge' }	{'[46,48)' }	77.912
{'CustAge' }	{'[48,51)' }	78.86
{'CustAge' }	{'[51,58)' }	80.83
{'CustAge' }	{'[58,Inf]' }	96.76
{'CustAge' }	{'<missing>' }	64.984
{'ResStatus' }	{'Tenant' }	62.138
{'ResStatus' }	{'Home Owner' }	73.248
{'ResStatus' }	{'Other' }	90.828
{'ResStatus' }	{'<missing>' }	74.125
{'EmpStatus' }	{'Unknown' }	58.807
{'EmpStatus' }	{'Employed' }	86.937
{'EmpStatus' }	{'<missing>' }	NaN
:		

Notice that points for the <missing> bin for CustAge and ResStatus are explicitly shown (as 64.9836 and 74.1250, respectively). These points are computed from the WOE value for the <missing> bin and the logistic model coefficients.

For predictors that have no missing data in the training set, there is no explicit <missing> bin. By default the points are set to NaN for missing data, and they lead to a score of NaN when running score. For predictors that have no explicit <missing> bin, use the name-value argument 'Missing' in formatpoints to indicate how missing data should be treated for scoring purposes.

For the purpose of illustration, take a few rows from the original data as test data and introduce some missing data. Also introduce some invalid, or out-of-range, values. For numeric data, values below the minimum (or above the maximum) allowed are considered invalid, such as a negative value for age (recall 'MinValue' was earlier set to 0 for CustAge and CustIncome). For categorical data, invalid values are categories not explicitly included in the scorecard, for example, a residential status not previously mapped to scorecard categories, such as "House", or a meaningless string such as "abc123".

```

tdata = dataMissing(11:18,mdl.PredictorNames); % Keep only the predictors retained in the model
% Set some missing values
tdata.CustAge(1) = NaN;
tdata.ResStatus(2) = missing;
tdata.EmpStatus(3) = missing;
tdata.CustIncome(4) = NaN;
% Set some invalid values
tdata.CustAge(5) = -100;
tdata.ResStatus(6) = 'House';
tdata.EmpStatus(7) = 'Freelancer';
tdata.CustIncome(8) = -1;
disp(tdata)

```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
NaN	Tenant	Unknown	34000	44	Yes	119.8

48	<undefined>	Unknown	44000	14	Yes	403.62
65	Home Owner	<undefined>	48000	6	No	111.88
44	Other	Unknown	NaN	35	No	436.41
-100	Other	Employed	46000	16	Yes	162.21
33	House	Employed	36000	36	Yes	845.02
39	Tenant	Freelancer	34000	40	Yes	756.26
24	Home Owner	Employed	-1	19	Yes	449.61

Score the new data and see how points are assigned for missing `CustAge` and `ResStatus`, because we have an explicit bin with points for `<missing>`. However, for `EmpStatus` and `CustIncome` the score function sets the points to `NaN`. The corresponding probabilities of default are also set to `NaN`.

```
[Scores,Points] = score(sc,tdata);
disp(Scores)
```

```
481.2231
520.8353
    NaN
    NaN
551.7922
487.9588
    NaN
    NaN
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	NaN	96.969	51.132	50.914	89.922
69.636	90.828	58.807	NaN	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	NaN	67.893	61.858	75.622	63.028
54.062	73.248	86.937	NaN	61.061	75.622	89.922

```
pd = probdefault(sc,tdata);
disp(pd)
```

```
0.3934
0.2725
    NaN
    NaN
0.1961
0.3714
    NaN
    NaN
```

Use the name-value argument `'Missing'` in `formatpoints` to choose how to assign points to missing values for predictors that do not have an explicit `<missing>` bin. In this example, use the `'MinPoints'` option for the `'Missing'` argument. The minimum points for `EmpStatus` in the scorecard displayed above are 58.8072, and for `CustIncome` the minimum points are 29.3753. All rows now have a score and a corresponding probability of default.

```
sc = formatpoints(sc,'Missing','MinPoints');
[ Scores,Points ] = score(sc,tdata);
disp(Scores)
```

```

481.2231
520.8353
517.7532
451.3405
551.7922
487.9588
449.3577
470.2267

```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	58.807	96.969	51.132	50.914	89.922
69.636	90.828	58.807	29.375	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	58.807	67.893	61.858	75.622	63.028
54.062	73.248	86.937	29.375	61.061	75.622	89.922

```

pd = probdefault(sc,tdata);
disp(pd)

```

```

0.3934
0.2725
0.2810
0.4954
0.1961
0.3714
0.5022
0.4304

```

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. To create this object, use `creditscorecard`.

data — Dataset to apply probability of default rules

table

(Optional) Dataset to apply probability of default rules, specified as a MATLAB table, where each row corresponds to individual observations. The data must contain columns for each of the predictors in the `creditscorecard` object.

Data Types: `table`

Output Arguments

pd — Probability of default

array

Probability of default, returned as a NumObs-by-1 numerical array of default probabilities.

More About

Default Probability

After the unscaled scores are computed (see “Algorithms for Computing and Scaling Scores” on page 15-1812), the probability of the points being “Good” is represented by the following formula:

$$\text{ProbGood} = 1. / (1 + \exp(-\text{UnscaledScores}))$$

Thus, the probability of default is

$$\text{pd} = 1 - \text{ProbGood}$$

Version History

Introduced in R2015a

References

[1] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

`creditscorecard` | `bininfo` | `predictorinfo` | `modifypredictor` | `modifybins` | `bindata` | `plotbins` | `fitmodel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `validatemodel` | `table`

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71

“Troubleshooting Credit Scorecard Results” on page 8-64

“Credit Scorecard Modeling Workflow” on page 8-52

“About Credit Scorecards” on page 8-49

validatemodel

Validate quality of credit scorecard model

Syntax

```
Stats = validatemodel(sc)
Stats = validatemodel(sc,data)
[Stats,T] = validatemodel(sc,Name,Value)
[Stats,T,hf] = validatemodel(sc,Name,Value)
```

Description

`Stats = validatemodel(sc)` validates the quality of the `creditscorecard` model.

By default, the data used to build the `creditscorecard` object is used. You can also supply input data to which the validation is applied.

`Stats = validatemodel(sc,data)` validates the quality of the `creditscorecard` model for a given data set specified using the optional argument `data`.

`[Stats,T] = validatemodel(sc,Name,Value)` validates the quality of the `creditscorecard` model using the optional name-value pair arguments, and returns `Stats` and `T` outputs.

`[Stats,T,hf] = validatemodel(sc,Name,Value)` validates the quality of the `creditscorecard` model using the optional name-value pair arguments, and returns the figure handle `hf` to the CAP, ROC, and KS plots.

Examples

Validate a Credit Scorecard Model

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data, 'IDVar','CustID')

sc =
    creditscorecard with properties:
        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustIncome2' 'CustIncome3' 'CustIncome4' 'CustIncome5' 'CustIncome6' 'CustIncome7' 'CustIncome8' 'CustIncome9' 'CustIncome10' 'CustIncome11' 'CustIncome12' 'CustIncome13' 'CustIncome14' 'CustIncome15' 'CustIncome16' 'CustIncome17' 'CustIncome18' 'CustIncome19' 'CustIncome20' 'CustIncome21' 'CustIncome22' 'CustIncome23' 'CustIncome24' 'CustIncome25' 'CustIncome26' 'CustIncome27' 'CustIncome28' 'CustIncome29' 'CustIncome30' 'CustIncome31' 'CustIncome32' 'CustIncome33' 'CustIncome34' 'CustIncome35' 'CustIncome36' 'CustIncome37' 'CustIncome38' 'CustIncome39' 'CustIncome40' 'CustIncome41' 'CustIncome42' 'CustIncome43' 'CustIncome44' 'CustIncome45' 'CustIncome46' 'CustIncome47' 'CustIncome48' 'CustIncome49' 'CustIncome50' 'CustIncome51' 'CustIncome52' 'CustIncome53' 'CustIncome54' 'CustIncome55' 'CustIncome56' 'CustIncome57' 'CustIncome58' 'CustIncome59' 'CustIncome60' 'CustIncome61' 'CustIncome62' 'CustIncome63' 'CustIncome64' 'CustIncome65' 'CustIncome66' 'CustIncome67' 'CustIncome68' 'CustIncome69' 'CustIncome70' 'CustIncome71' 'CustIncome72' 'CustIncome73' 'CustIncome74' 'CustIncome75' 'CustIncome76' 'CustIncome77' 'CustIncome78' 'CustIncome79' 'CustIncome80' 'CustIncome81' 'CustIncome82' 'CustIncome83' 'CustIncome84' 'CustIncome85' 'CustIncome86' 'CustIncome87' 'CustIncome88' 'CustIncome89' 'CustIncome90' 'CustIncome91' 'CustIncome92' 'CustIncome93' 'CustIncome94' 'CustIncome95' 'CustIncome96' 'CustIncome97' 'CustIncome98' 'CustIncome99' 'CustIncome100'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate' 'CreditLimit' 'CreditLimit2' 'CreditLimit3' 'CreditLimit4' 'CreditLimit5' 'CreditLimit6' 'CreditLimit7' 'CreditLimit8' 'CreditLimit9' 'CreditLimit10' 'CreditLimit11' 'CreditLimit12' 'CreditLimit13' 'CreditLimit14' 'CreditLimit15' 'CreditLimit16' 'CreditLimit17' 'CreditLimit18' 'CreditLimit19' 'CreditLimit20' 'CreditLimit21' 'CreditLimit22' 'CreditLimit23' 'CreditLimit24' 'CreditLimit25' 'CreditLimit26' 'CreditLimit27' 'CreditLimit28' 'CreditLimit29' 'CreditLimit30' 'CreditLimit31' 'CreditLimit32' 'CreditLimit33' 'CreditLimit34' 'CreditLimit35' 'CreditLimit36' 'CreditLimit37' 'CreditLimit38' 'CreditLimit39' 'CreditLimit40' 'CreditLimit41' 'CreditLimit42' 'CreditLimit43' 'CreditLimit44' 'CreditLimit45' 'CreditLimit46' 'CreditLimit47' 'CreditLimit48' 'CreditLimit49' 'CreditLimit50' 'CreditLimit51' 'CreditLimit52' 'CreditLimit53' 'CreditLimit54' 'CreditLimit55' 'CreditLimit56' 'CreditLimit57' 'CreditLimit58' 'CreditLimit59' 'CreditLimit60' 'CreditLimit61' 'CreditLimit62' 'CreditLimit63' 'CreditLimit64' 'CreditLimit65' 'CreditLimit66' 'CreditLimit67' 'CreditLimit68' 'CreditLimit69' 'CreditLimit70' 'CreditLimit71' 'CreditLimit72' 'CreditLimit73' 'CreditLimit74' 'CreditLimit75' 'CreditLimit76' 'CreditLimit77' 'CreditLimit78' 'CreditLimit79' 'CreditLimit80' 'CreditLimit81' 'CreditLimit82' 'CreditLimit83' 'CreditLimit84' 'CreditLimit85' 'CreditLimit86' 'CreditLimit87' 'CreditLimit88' 'CreditLimit89' 'CreditLimit90' 'CreditLimit91' 'CreditLimit92' 'CreditLimit93' 'CreditLimit94' 'CreditLimit95' 'CreditLimit96' 'CreditLimit97' 'CreditLimit98' 'CreditLimit99' 'CreditLimit100'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustIncome2' 'CustIncome3' 'CustIncome4' 'CustIncome5' 'CustIncome6' 'CustIncome7' 'CustIncome8' 'CustIncome9' 'CustIncome10' 'CustIncome11' 'CustIncome12' 'CustIncome13' 'CustIncome14' 'CustIncome15' 'CustIncome16' 'CustIncome17' 'CustIncome18' 'CustIncome19' 'CustIncome20' 'CustIncome21' 'CustIncome22' 'CustIncome23' 'CustIncome24' 'CustIncome25' 'CustIncome26' 'CustIncome27' 'CustIncome28' 'CustIncome29' 'CustIncome30' 'CustIncome31' 'CustIncome32' 'CustIncome33' 'CustIncome34' 'CustIncome35' 'CustIncome36' 'CustIncome37' 'CustIncome38' 'CustIncome39' 'CustIncome40' 'CustIncome41' 'CustIncome42' 'CustIncome43' 'CustIncome44' 'CustIncome45' 'CustIncome46' 'CustIncome47' 'CustIncome48' 'CustIncome49' 'CustIncome50' 'CustIncome51' 'CustIncome52' 'CustIncome53' 'CustIncome54' 'CustIncome55' 'CustIncome56' 'CustIncome57' 'CustIncome58' 'CustIncome59' 'CustIncome60' 'CustIncome61' 'CustIncome62' 'CustIncome63' 'CustIncome64' 'CustIncome65' 'CustIncome66' 'CustIncome67' 'CustIncome68' 'CustIncome69' 'CustIncome70' 'CustIncome71' 'CustIncome72' 'CustIncome73' 'CustIncome74' 'CustIncome75' 'CustIncome76' 'CustIncome77' 'CustIncome78' 'CustIncome79' 'CustIncome80' 'CustIncome81' 'CustIncome82' 'CustIncome83' 'CustIncome84' 'CustIncome85' 'CustIncome86' 'CustIncome87' 'CustIncome88' 'CustIncome89' 'CustIncome90' 'CustIncome91' 'CustIncome92' 'CustIncome93' 'CustIncome94' 'CustIncome95' 'CustIncome96' 'CustIncome97' 'CustIncome98' 'CustIncome99' 'CustIncome100'}
        Data: [1200x11 table]
```

Perform automatic binning using the default options. By default, autobinning uses the Monotone algorithm.

```
sc = autobinning(sc);
```

Fit the model.

```
sc = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1441.994, Chi2Stat = 5.3511754, PValue = 0.020708306
6. Adding ResStatus, Deviance = 1437.8756, Chi2Stat = 4.118404, PValue = 0.042419078
7. Adding OtherCC, Deviance = 1433.707, Chi2Stat = 4.1686018, PValue = 0.041179769

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70239	0.064001	10.975	5.0538e-28
CustAge	0.60833	0.24932	2.44	0.014687
ResStatus	1.377	0.65272	2.1097	0.034888
EmpStatus	0.88565	0.293	3.0227	0.0025055
CustIncome	0.70164	0.21844	3.2121	0.0013179
TmWBank	1.1074	0.23271	4.7589	1.9464e-06
OtherCC	1.0883	0.52912	2.0569	0.039696
AMBalance	1.045	0.32214	3.2439	0.0011792

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 89.7, p-value = 1.4e-16

Format the unscaled points.

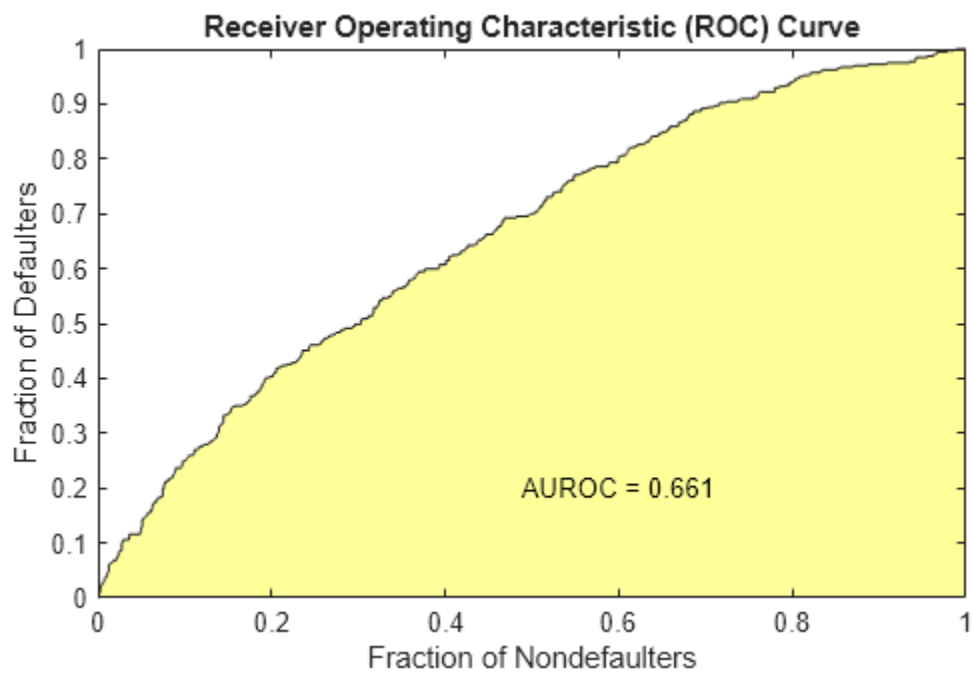
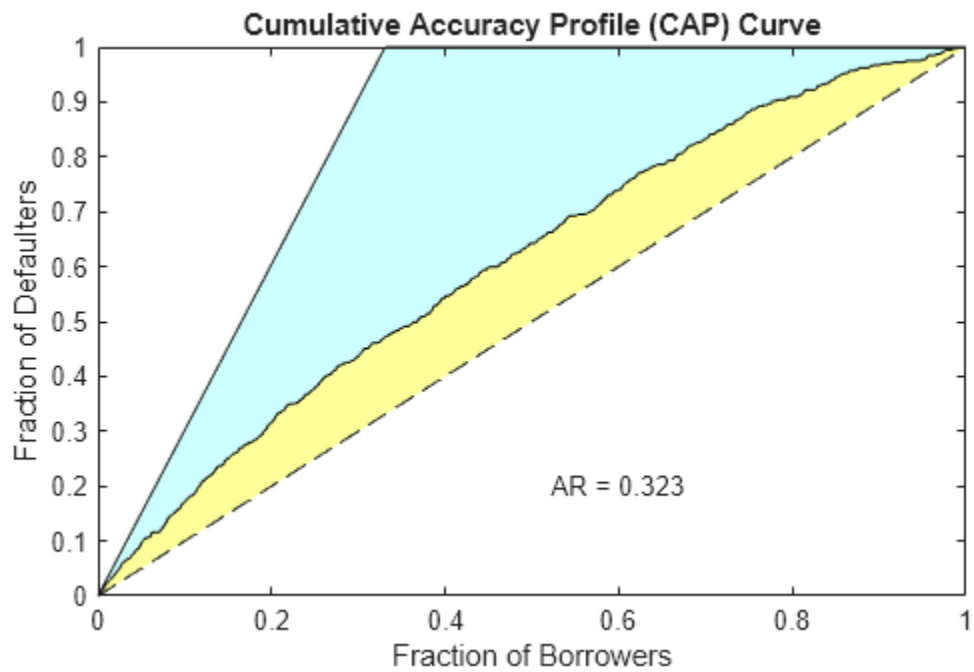
```
sc = formatpoints(sc, 'PointsOddsAndPD0', [500,2,50]);
```

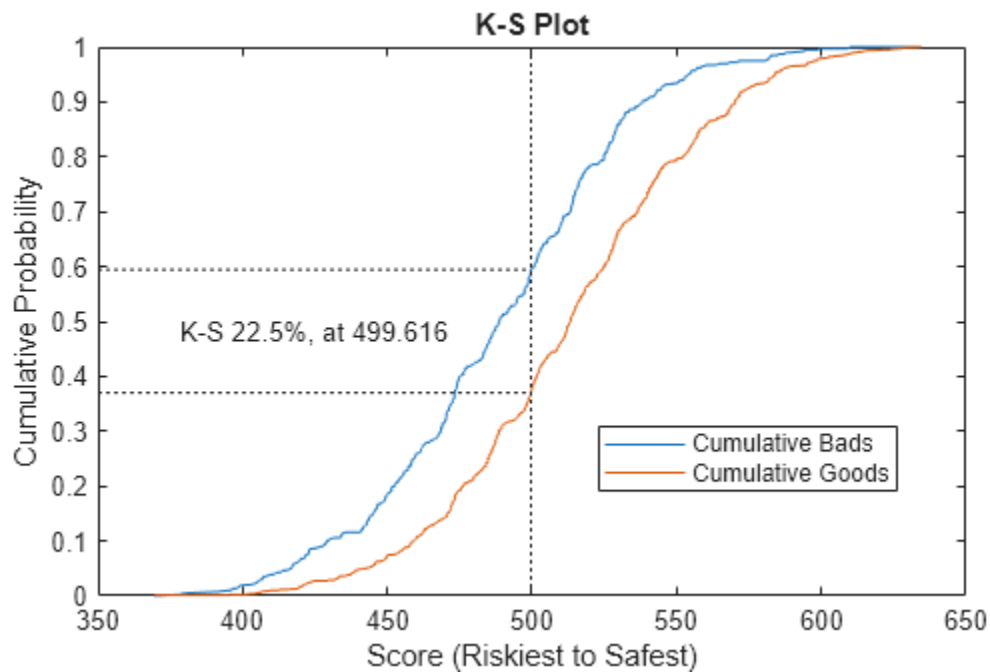
Score the data.

```
scores = score(sc);
```

Validate the credit scorecard model by generating the CAP, ROC, and KS plots.

```
[Stats,T] = validatemodel(sc, 'Plot', {'CAP', 'ROC', 'KS'});
```





```
disp(Stats)
```

Measure	Value
{'Accuracy Ratio' }	0.32258
{'Area under ROC curve' }	0.66129
{'KS statistic' }	0.2246
{'KS score' }	499.62

```
disp(T(1:15,:))
```

Scores	ProbDefault	TrueBads	FalseBads	TrueGoods	FalseGoods	Sensitivity	
369.54	0.75313	0	1	802	397	0	0
378.19	0.73016	1	1	802	396	0.0025189	0
380.28	0.72444	2	1	802	395	0.0050378	0
391.49	0.69234	3	1	802	394	0.0075567	0
395.57	0.68017	4	1	802	393	0.010076	0
396.14	0.67846	4	2	801	393	0.010076	0
396.45	0.67752	5	2	801	392	0.012594	0
398.61	0.67094	6	2	801	391	0.015113	0
398.68	0.67072	7	2	801	390	0.017632	0
401.33	0.66255	8	2	801	389	0.020151	0
402.66	0.65842	8	3	800	389	0.020151	0
404.25	0.65346	9	3	800	388	0.02267	0
404.73	0.65193	9	4	799	388	0.02267	0
405.53	0.64941	11	4	799	386	0.027708	0
405.7	0.64887	11	5	798	386	0.027708	0

Validate a Credit Score Card Model With Weights

Use the `CreditCardData.mat` file to load the data (`dataWeights`) that contains a column (`RowWeights`) for the weights (using a dataset from Refaat 2011).

load `CreditCardData`

Create a `creditscorecard` object using the optional name-value pair argument for `'WeightsVar'`.

```
sc = creditscorecard(dataWeights, 'IDVar', 'CustID', 'WeightsVar', 'RowWeights')
```

```
sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: 'RowWeights'
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        Data: [1200x12 table]
```

Perform automatic binning.

```
sc = autobinning(sc)
```

```
sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: 'RowWeights'
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        Data: [1200x12 table]
```

Fit the model.

```
sc = fitmodel(sc);
```

1. Adding `CustIncome`, Deviance = 764.3187, Chi2Stat = 15.81927, PValue = 6.968927e-05
2. Adding `TmWBank`, Deviance = 751.0215, Chi2Stat = 13.29726, PValue = 0.0002657942
3. Adding `AMBalance`, Deviance = 743.7581, Chi2Stat = 7.263384, PValue = 0.007037455

Generalized linear regression model:

```
logit(status) ~ 1 + CustIncome + TmWBank + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

Estimate	SE	tStat	pValue
----------	----	-------	--------

(Intercept)	0.70642	0.088702	7.964	1.6653e-15
CustIncome	1.0268	0.25758	3.9862	6.7132e-05
TmWBank	1.0973	0.31294	3.5063	0.0004543
AMBalance	1.0039	0.37576	2.6717	0.0075464

1200 observations, 1196 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 36.4, p-value = 6.22e-08

Format the unscaled points.

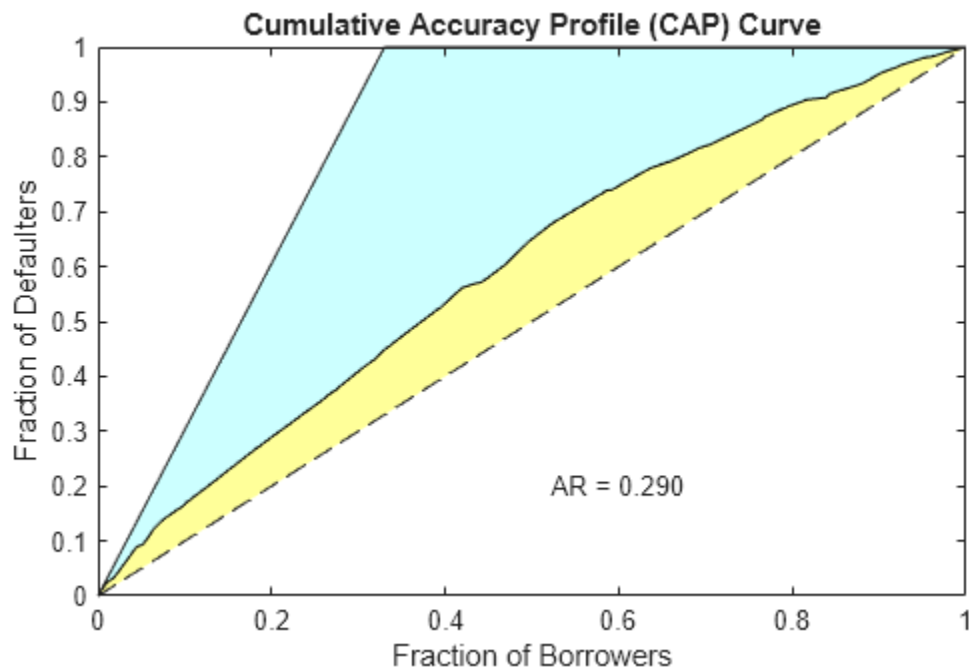
```
sc = formatpoints(sc, 'PointsOddsAndPD0',[500,2,50]);
```

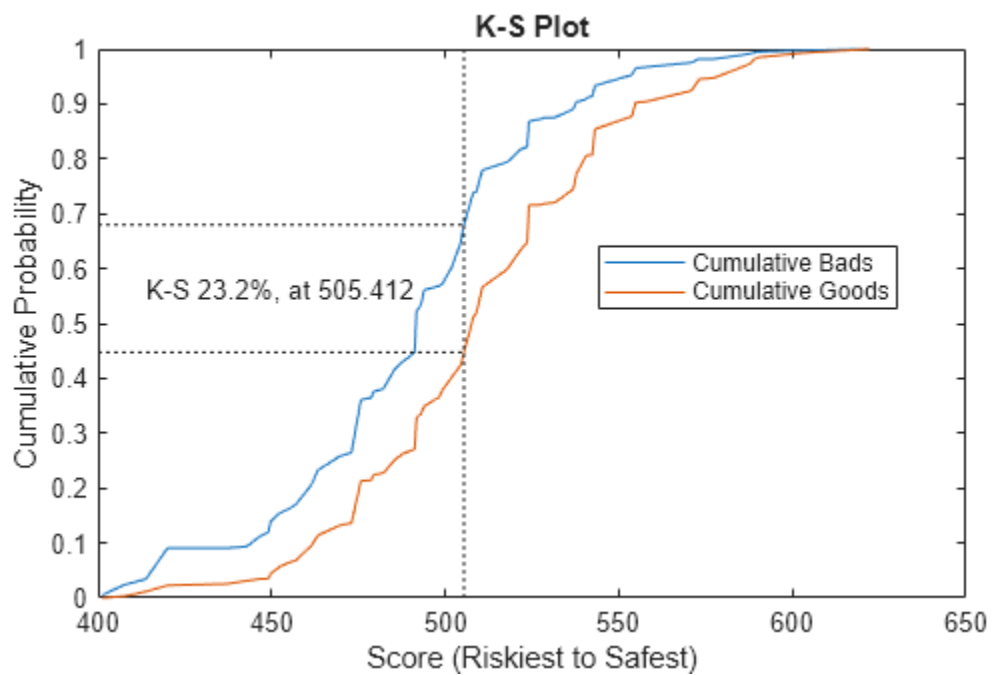
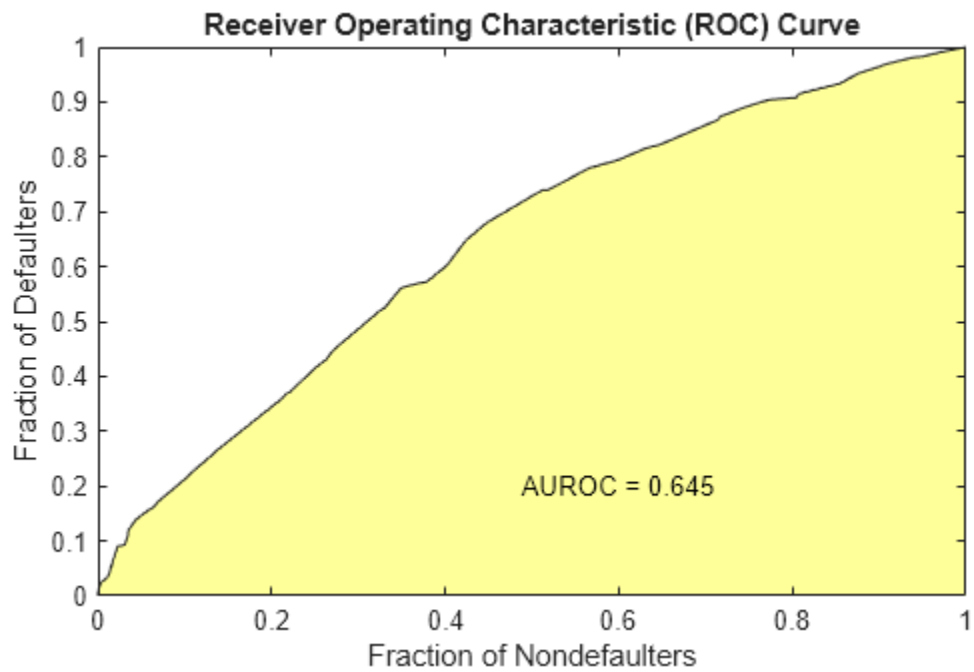
Score the data.

```
scores = score(sc);
```

Validate the credit scorecard model by generating the CAP, ROC, and KS plots. When the optional name-value pair argument 'WeightsVar' is used to specify observation (sample) weights, the T table uses statistics, sums, and cumulative sums that are weighted counts.

```
[Stats,T] = validatemodel(sc,'Plot',{'CAP','ROC','KS'});
```





Stats

Stats=4x2 table

Measure	Value
{'Accuracy Ratio' }	0.28972
{'Area under ROC curve' }	0.64486
{'KS statistic' }	0.23215

```
{'KS score'          }      505.41
```

```
T(1:10,:)
```

```
ans=10×9 table
```

Scores	ProbDefault	TrueBads	FalseBads	TrueGoods	FalseGoods	Sensitivity
401.34	0.66253	1.0788	0	411.95	201.95	0.0053135
407.59	0.64289	4.8363	1.2768	410.67	198.19	0.023821
413.79	0.62292	6.9469	4.6942	407.25	196.08	0.034216
420.04	0.60236	18.459	9.3899	402.56	184.57	0.090918
437.27	0.544	18.459	10.514	401.43	184.57	0.090918
442.83	0.52481	18.973	12.794	399.15	184.06	0.093448
446.19	0.51319	22.396	14.15	397.8	180.64	0.11031
449.08	0.50317	24.325	14.405	397.54	178.71	0.11981
449.73	0.50095	28.246	18.049	393.9	174.78	0.13912
452.44	0.49153	31.511	23.565	388.38	171.52	0.1552

Validate a Credit Score Card Model When Using the 'BinMissingData' Option

This example describes both the assignment of points for missing data when the 'BinMissingData' option is set to `true`, and the corresponding computation of model validation statistics.

- Predictors that have missing data in the training set have an explicit bin for `<missing>` with corresponding points in the final scorecard. These points are computed from the Weight-of-Evidence (WOE) value for the `<missing>` bin and the logistic model coefficients. For scoring purposes, these points are assigned to missing values and to out-of-range values, and the final score is used to compute model validation statistics with `validatemodel`.
- Predictors with no missing data in the training set have no `<missing>` bin, therefore no WOE can be estimated from the training data. By default, the points for missing and out-of-range values are set to `NaN`, and this leads to a score of `NaN` when running `score`. For predictors that have no explicit `<missing>` bin, use the name-value argument 'Missing' in `formatpoints` to indicate how missing data should be treated for scoring purposes. The final score is used to compute model validation statistics with `validatemodel`.

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the `dataMissing` with missing values.

```
load CreditCardData.mat
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Other
1	53	62	<undefined>	Unknown	50000	55	Ye
2	61	22	Home Owner	Employed	52000	25	Ye
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Ye
5	68	56	Home Owner	Employed	53000	14	Ye

Use `creditscorecard` with the name-value argument 'BinMissingData' set to `true` to bin the missing numeric or categorical data in a separate bin. Apply automatic binning.

```
disp(sc)
```

```
GoodLabel: 0
ResponseVar: 'status'
WeightsVar: ''
VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustInq' 'CustSocSec' 'CustMktSegment' 'CustMktSegment2' 'CustMktSegment3' 'CustMktSegment4' 'CustMktSegment5' 'CustMktSegment6' 'CustMktSegment7' 'CustMktSegment8' 'CustMktSegment9' 'CustMktSegment10' 'CustMktSegment11' 'CustMktSegment12' 'CustMktSegment13' 'CustMktSegment14' 'CustMktSegment15' 'CustMktSegment16' 'CustMktSegment17' 'CustMktSegment18' 'CustMktSegment19' 'CustMktSegment20' 'CustMktSegment21' 'CustMktSegment22' 'CustMktSegment23' 'CustMktSegment24' 'CustMktSegment25' 'CustMktSegment26' 'CustMktSegment27' 'CustMktSegment28' 'CustMktSegment29' 'CustMktSegment30' 'CustMktSegment31' 'CustMktSegment32' 'CustMktSegment33' 'CustMktSegment34' 'CustMktSegment35' 'CustMktSegment36' 'CustMktSegment37' 'CustMktSegment38' 'CustMktSegment39' 'CustMktSegment40' 'CustMktSegment41' 'CustMktSegment42' 'CustMktSegment43' 'CustMktSegment44' 'CustMktSegment45' 'CustMktSegment46' 'CustMktSegment47' 'CustMktSegment48' 'CustMktSegment49' 'CustMktSegment50' 'CustMktSegment51' 'CustMktSegment52' 'CustMktSegment53' 'CustMktSegment54' 'CustMktSegment55' 'CustMktSegment56' 'CustMktSegment57' 'CustMktSegment58' 'CustMktSegment59' 'CustMktSegment60' 'CustMktSegment61' 'CustMktSegment62' 'CustMktSegment63' 'CustMktSegment64' 'CustMktSegment65' 'CustMktSegment66' 'CustMktSegment67' 'CustMktSegment68' 'CustMktSegment69' 'CustMktSegment70' 'CustMktSegment71' 'CustMktSegment72' 'CustMktSegment73' 'CustMktSegment74' 'CustMktSegment75' 'CustMktSegment76' 'CustMktSegment77' 'CustMktSegment78' 'CustMktSegment79' 'CustMktSegment80' 'CustMktSegment81' 'CustMktSegment82' 'CustMktSegment83' 'CustMktSegment84' 'CustMktSegment85' 'CustMktSegment86' 'CustMktSegment87' 'CustMktSegment88' 'CustMktSegment89' 'CustMktSegment90' 'CustMktSegment91' 'CustMktSegment92' 'CustMktSegment93' 'CustMktSegment94' 'CustMktSegment95' 'CustMktSegment96' 'CustMktSegment97' 'CustMktSegment98' 'CustMktSegment99'}
NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate' 'CreditLimit' 'CreditScore' 'CreditHistory' 'CreditBureau' 'CreditBureau2' 'CreditBureau3' 'CreditBureau4' 'CreditBureau5' 'CreditBureau6' 'CreditBureau7' 'CreditBureau8' 'CreditBureau9' 'CreditBureau10' 'CreditBureau11' 'CreditBureau12' 'CreditBureau13' 'CreditBureau14' 'CreditBureau15' 'CreditBureau16' 'CreditBureau17' 'CreditBureau18' 'CreditBureau19' 'CreditBureau20' 'CreditBureau21' 'CreditBureau22' 'CreditBureau23' 'CreditBureau24' 'CreditBureau25' 'CreditBureau26' 'CreditBureau27' 'CreditBureau28' 'CreditBureau29' 'CreditBureau30' 'CreditBureau31' 'CreditBureau32' 'CreditBureau33' 'CreditBureau34' 'CreditBureau35' 'CreditBureau36' 'CreditBureau37' 'CreditBureau38' 'CreditBureau39' 'CreditBureau40' 'CreditBureau41' 'CreditBureau42' 'CreditBureau43' 'CreditBureau44' 'CreditBureau45' 'CreditBureau46' 'CreditBureau47' 'CreditBureau48' 'CreditBureau49' 'CreditBureau50' 'CreditBureau51' 'CreditBureau52' 'CreditBureau53' 'CreditBureau54' 'CreditBureau55' 'CreditBureau56' 'CreditBureau57' 'CreditBureau58' 'CreditBureau59' 'CreditBureau60' 'CreditBureau61' 'CreditBureau62' 'CreditBureau63' 'CreditBureau64' 'CreditBureau65' 'CreditBureau66' 'CreditBureau67' 'CreditBureau68' 'CreditBureau69' 'CreditBureau70' 'CreditBureau71' 'CreditBureau72' 'CreditBureau73' 'CreditBureau74' 'CreditBureau75' 'CreditBureau76' 'CreditBureau77' 'CreditBureau78' 'CreditBureau79' 'CreditBureau80' 'CreditBureau81' 'CreditBureau82' 'CreditBureau83' 'CreditBureau84' 'CreditBureau85' 'CreditBureau86' 'CreditBureau87' 'CreditBureau88' 'CreditBureau89' 'CreditBureau90' 'CreditBureau91' 'CreditBureau92' 'CreditBureau93' 'CreditBureau94' 'CreditBureau95' 'CreditBureau96' 'CreditBureau97' 'CreditBureau98' 'CreditBureau99'}
CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
BinMissingData: 1
IDVar: 'CustID'
PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustInq' 'CustSocSec' 'CustMktSegment' 'CustMktSegment2' 'CustMktSegment3' 'CustMktSegment4' 'CustMktSegment5' 'CustMktSegment6' 'CustMktSegment7' 'CustMktSegment8' 'CustMktSegment9' 'CustMktSegment10' 'CustMktSegment11' 'CustMktSegment12' 'CustMktSegment13' 'CustMktSegment14' 'CustMktSegment15' 'CustMktSegment16' 'CustMktSegment17' 'CustMktSegment18' 'CustMktSegment19' 'CustMktSegment20' 'CustMktSegment21' 'CustMktSegment22' 'CustMktSegment23' 'CustMktSegment24' 'CustMktSegment25' 'CustMktSegment26' 'CustMktSegment27' 'CustMktSegment28' 'CustMktSegment29' 'CustMktSegment30' 'CustMktSegment31' 'CustMktSegment32' 'CustMktSegment33' 'CustMktSegment34' 'CustMktSegment35' 'CustMktSegment36' 'CustMktSegment37' 'CustMktSegment38' 'CustMktSegment39' 'CustMktSegment40' 'CustMktSegment41' 'CustMktSegment42' 'CustMktSegment43' 'CustMktSegment44' 'CustMktSegment45' 'CustMktSegment46' 'CustMktSegment47' 'CustMktSegment48' 'CustMktSegment49' 'CustMktSegment50' 'CustMktSegment51' 'CustMktSegment52' 'CustMktSegment53' 'CustMktSegment54' 'CustMktSegment55' 'CustMktSegment56' 'CustMktSegment57' 'CustMktSegment58' 'CustMktSegment59' 'CustMktSegment60' 'CustMktSegment61' 'CustMktSegment62' 'CustMktSegment63' 'CustMktSegment64' 'CustMktSegment65' 'CustMktSegment66' 'CustMktSegment67' 'CustMktSegment68' 'CustMktSegment69' 'CustMktSegment70' 'CustMktSegment71' 'CustMktSegment72' 'CustMktSegment73' 'CustMktSegment74' 'CustMktSegment75' 'CustMktSegment76' 'CustMktSegment77' 'CustMktSegment78' 'CustMktSegment79' 'CustMktSegment80' 'CustMktSegment81' 'CustMktSegment82' 'CustMktSegment83' 'CustMktSegment84' 'CustMktSegment85' 'CustMktSegment86' 'CustMktSegment87' 'CustMktSegment88' 'CustMktSegment89' 'CustMktSegment90' 'CustMktSegment91' 'CustMktSegment92' 'CustMktSegment93' 'CustMktSegment94' 'CustMktSegment95' 'CustMktSegment96' 'CustMktSegment97' 'CustMktSegment98' 'CustMktSegment99'}
Data: [1200x11 table]
```

```
sc = modifybins(sc, 'CustAge', 'MinValue', 0);  
sc = modifybins(sc, 'CustIncome', 'MinValue', 0);
```

```
bi = bininfo(sc, 'CustAge');  
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{ '[0,33)' }	69	52	1.3269	-0.42156	0.018993
{ '[33,37)' }	63	45	1.4	-0.36795	0.012839
{ '[37,40)' }	72	47	1.5319	-0.2779	0.0079824
{ '[40,46)' }	172	89	1.9326	-0.04556	0.0004549
{ '[46,48)' }	59	25	2.36	0.15424	0.0016199
{ '[48,51)' }	99	41	2.4146	0.17713	0.0035449
{ '[51,58)' }	157	62	2.5323	0.22469	0.0088407
{ '[58,Inf]' }	93	25	3.72	0.60931	0.032198
{ '<missing>' }	19	11	1.7273	-0.15787	0.00063885
{ 'Totals' }	803	397	2.0227	NaN	0.087112

```
bi = bininfo(sc, 'ResStatus');  
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{'Other' }	128	52	2.4615	0.19637	0.0055808
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.0092627

For the 'CustAge' and 'ResStatus' predictors, there is missing data (NaNs and <undefined>) in the training data, and the binning process estimates a WOE value of -0.15787 and 0.026469 respectively for missing data in these predictors, as shown above.

For EmpStatus and CustIncome there is no explicit bin for missing values, because the training data has no missing values for these predictors.

```
bi = bininfo(sc, 'EmpStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Unknown' }	396	239	1.6569	-0.19947	0.021715
{'Employed' }	407	158	2.5759	0.2418	0.026323
{'Totals' }	803	397	2.0227	NaN	0.048038

```
bi = bininfo(sc, 'CustIncome');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[0,29000)' }	53	58	0.91379	-0.79457	0.06364
{'[29000,33000)' }	74	49	1.5102	-0.29217	0.0091366
{'[33000,35000)' }	68	36	1.8889	-0.06843	0.00041042
{'[35000,40000)' }	193	98	1.9694	-0.026696	0.00017359
{'[40000,42000)' }	68	34	2	-0.011271	1.0819e-05
{'[42000,47000)' }	164	66	2.4848	0.20579	0.0078175
{'[47000,Inf]' }	183	56	3.2679	0.47972	0.041657
{'Totals' }	803	397	2.0227	NaN	0.12285

Use `fitmodel` to fit a logistic regression model using Weight of Evidence (WOE) data. `fitmodel` internally transforms all the predictor variables into WOE values, using the bins found with the automatic binning process. `fitmodel` then fits a logistic regression model using a stepwise method (by default). For predictors that have missing data, there is an explicit <missing> bin, with a corresponding WOE value computed from the data. When using `fitmodel`, the corresponding WOE value for the <missing> bin is applied when performing the WOE transformation.

```
[sc,mdl] = fitmodel(sc);
```

1. Adding CustIncome, Deviance = 1490.8527, Chi2Stat = 32.588614, PValue = 1.1387992e-08
2. Adding TmWBank, Deviance = 1467.1415, Chi2Stat = 23.711203, PValue = 1.1192909e-06
3. Adding AMBalance, Deviance = 1455.5715, Chi2Stat = 11.569967, PValue = 0.00067025601
4. Adding EmpStatus, Deviance = 1447.3451, Chi2Stat = 8.2264038, PValue = 0.0041285257
5. Adding CustAge, Deviance = 1442.8477, Chi2Stat = 4.4974731, PValue = 0.033944979
6. Adding ResStatus, Deviance = 1438.9783, Chi2Stat = 3.86941, PValue = 0.049173805
7. Adding OtherCC, Deviance = 1434.9751, Chi2Stat = 4.0031966, PValue = 0.045414057

Generalized linear regression model:

```
logit(status) ~ 1 + CustAge + ResStatus + EmpStatus + CustIncome + TmWBank + OtherCC + AMBalance
Distribution = Binomial
```

Estimated Coefficients:

	Estimate	SE	tStat	pValue
(Intercept)	0.70229	0.063959	10.98	4.7498e-28

CustAge	0.57421	0.25708	2.2335	0.025513
ResStatus	1.3629	0.66952	2.0356	0.04179
EmpStatus	0.88373	0.2929	3.0172	0.002551
CustIncome	0.73535	0.2159	3.406	0.00065929
TmWBank	1.1065	0.23267	4.7556	1.9783e-06
OtherCC	1.0648	0.52826	2.0156	0.043841
AMBalance	1.0446	0.32197	3.2443	0.0011775

1200 observations, 1192 error degrees of freedom

Dispersion: 1

Chi^2-statistic vs. constant model: 88.5, p-value = 2.55e-16

Scale the scorecard points by the "points, odds, and points to double the odds (PDO)" method using the '`PointsOddsAndPDO`' argument of `formatpoints`. Suppose that you want a score of 500 points to have odds of 2 (twice as likely to be good than to be bad) and that the odds double every 50 points (so that 550 points would have odds of 4).

Display the scorecard showing the scaled points for predictors retained in the fitting model.

```
sc = formatpoints(sc, 'PointsOddsAndPDO', [500 2 50]);
PointsInfo = displaypoints(sc)
```

PointsInfo=38x3 table

Predictors	Bin	Points
{'CustAge' }	{ '[0,33)' }	54.062
{'CustAge' }	{ '[33,37)' }	56.282
{'CustAge' }	{ '[37,40)' }	60.012
{'CustAge' }	{ '[40,46)' }	69.636
{'CustAge' }	{ '[46,48)' }	77.912
{'CustAge' }	{ '[48,51)' }	78.86
{'CustAge' }	{ '[51,58)' }	80.83
{'CustAge' }	{ '[58,Inf]' }	96.76
{'CustAge' }	{ '<missing>' }	64.984
{'ResStatus' }	{ 'Tenant' }	62.138
{'ResStatus' }	{ 'Home Owner' }	73.248
{'ResStatus' }	{ 'Other' }	90.828
{'ResStatus' }	{ '<missing>' }	74.125
{'EmpStatus' }	{ 'Unknown' }	58.807
{'EmpStatus' }	{ 'Employed' }	86.937
{'EmpStatus' }	{ '<missing>' }	NaN
:		

Notice that points for the `<missing>` bin for `CustAge` and `ResStatus` are explicitly shown (as 64.9836 and 74.1250, respectively). These points are computed from the WOE value for the `<missing>` bin, and the logistic model coefficients.

For predictors that have no missing data in the training set, there is no explicit `<missing>` bin. By default the points are set to `NaN` for missing data, and they lead to a score of `NaN` when running `score`. For predictors that have no explicit `<missing>` bin, use the name-value argument '`Missing`' in `formatpoints` to indicate how missing data should be treated for scoring purposes.

For the purpose of illustration, take a few rows from the original data as test data and introduce some missing data. Also introduce some invalid, or out-of-range, values. For numeric data, values below the

minimum (or above the maximum) allowed are considered invalid, such as a negative value for age (recall 'MinValue' was earlier set to 0 for CustAge and CustIncome). For categorical data, invalid values are categories not explicitly included in the scorecard, for example, a residential status not previously mapped to scorecard categories, such as "House", or a meaningless string such as "abc123".

This is a very small validation data set, only used to illustrate the scoring of rows with missing and out-of-range values, and its relationship with model validation.

```
tdata = dataMissing(11:18,mdl.PredictorNames); % Keep only the predictors retained in the model
tdata.status = dataMissing.status(11:18); % Copy the response variable value, needed for validation
% Set some missing values
tdata.CustAge(1) = NaN;
tdata.ResStatus(2) = missing;
tdata.EmpStatus(3) = missing;
tdata.CustIncome(4) = NaN;
% Set some invalid values
tdata.CustAge(5) = -100;
tdata.ResStatus(6) = 'House';
tdata.EmpStatus(7) = 'Freelancer';
tdata.CustIncome(8) = -1;
disp(tdata)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance	sta
NaN	Tenant	Unknown	34000	44	Yes	119.8	
48	<undefined>	Unknown	44000	14	Yes	403.62	0
65	Home Owner	<undefined>	48000	6	No	111.88	0
44	Other	Unknown	NaN	35	No	436.41	0
-100	Other	Employed	46000	16	Yes	162.21	0
33	House	Employed	36000	36	Yes	845.02	0
39	Tenant	Freelancer	34000	40	Yes	756.26	3
24	Home Owner	Employed	-1	19	Yes	449.61	0

Score the new data and see how points are assigned for missing CustAge and ResStatus, because we have an explicit bin with points for <missing>. However, for EmpStatus and CustIncome the score function sets the points to NaN.

The validation results are unreliable, the scores with NaN values are kept (see the validation table ValTable below), but it is unclear what impact these NaN values have in the validation statistics (ValStats). This is a very small validation data set, but NaN scores could still influence the validation results on a larger data set.

```
[Scores,Points] = score(sc,tdata);
disp(Scores)
```

```
481.2231
520.8353
NaN
NaN
551.7922
487.9588
NaN
NaN
```

```
disp(Points)
```


CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	NaN	96.969	51.132	50.914	89.922
69.636	90.828	58.807	NaN	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	NaN	67.893	61.858	75.622	63.028
54.062	73.248	86.937	NaN	61.061	75.622	89.922

```
[ValStats,ValTable] = validatemodel(sc,tdata);
disp(ValStats)
```

Measure	Value
{'Accuracy Ratio' }	0.16667
{'Area under ROC curve'}	0.58333
{'KS statistic' }	0.5
{'KS score' }	481.22

```
disp(ValTable)
```

Scores	ProbDefault	TrueBads	FalseBads	TrueGoods	FalseGoods	Sensitivity
NaN	NaN	0	1	5	2	0
NaN	NaN	0	2	4	2	0
NaN	NaN	1	2	4	1	0.5
NaN	NaN	1	3	3	1	0.5
481.22	0.39345	2	3	3	0	1
487.96	0.3714	2	4	2	0	1
520.84	0.2725	2	5	1	0	1
551.79	0.19605	2	6	0	0	1

Use the name-value argument 'Missing' in formatpoints to choose how to assign points to missing values for predictors that do not have an explicit <missing> bin. In this example, use the 'MinPoints' option for the 'Missing' argument. The minimum points for EmpStatus in the scorecard displayed above are 58.8072 and for CustIncome the minimum points are 29.3753.

The validation results are no longer influenced by NaN values, since all rows now have a score.

```
sc = formatpoints(sc,'Missing','MinPoints');
[Scores,Points] = score(sc,tdata);
disp(Scores)
```

```
481.2231
520.8353
517.7532
451.3405
551.7922
487.9588
449.3577
470.2267
```

```
disp(Points)
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
64.984	62.138	58.807	67.893	61.858	75.622	89.922
78.86	74.125	58.807	82.439	61.061	75.622	89.922
96.76	73.248	58.807	96.969	51.132	50.914	89.922
69.636	90.828	58.807	29.375	61.858	50.914	89.922
64.984	90.828	86.937	82.439	61.061	75.622	89.922
56.282	74.125	86.937	70.107	61.858	75.622	63.028
60.012	62.138	58.807	67.893	61.858	75.622	63.028
54.062	73.248	86.937	29.375	61.061	75.622	89.922

```
[ValStats,ValTable] = validateModel(sc,tdata);
disp(ValStats)
```

Measure	Value
{'Accuracy Ratio' }	0.66667
{'Area under ROC curve' }	0.83333
{'KS statistic' }	0.66667
{'KS score' }	481.22

```
disp(ValTable)
```

Scores	ProbDefault	TrueBads	FalseBads	TrueGoods	FalseGoods	Sensitivity
449.36	0.50223	1	0	6	1	0.5
451.34	0.49535	1	1	5	1	0.5
470.23	0.43036	1	2	4	1	0.5
481.22	0.39345	2	2	4	0	1
487.96	0.3714	2	3	3	0	1
517.75	0.28105	2	4	2	0	1
520.84	0.2725	2	5	1	0	1
551.79	0.19605	2	6	0	0	1

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object. To create this object, use `creditscorecard`.

data — Validation data

table

(Optional) Validation data, specified as a MATLAB table, where each table row corresponds to individual observations. The **data** must contain columns for each of the predictors in the credit scorecard model. The columns of data can be any one of the following data types:

- Numeric
- Logical
- Cell array of character vectors

- Character array
- Categorical
- String
- String array

In addition, the table must contain a binary response variable.

Note When observation weights are defined using the optional `WeightsVar` name-value pair argument when creating a `creditscorecard` object, the weights stored in the `WeightsVar` column are used when validating the model on the training data. If a different validation data set is provided using the optional `data` input, observation weights for the validation data must be included in a column whose name matches `WeightsVar`, otherwise unit weights are used for the validation data. For more information, see “Using `validatemodel` with `Weights`” on page 15-2016.

Data Types: table

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `sc = validatemodel(sc,data,'AnalysisLevel','Deciles','Plot','CAP')`

AnalysisLevel — Type of analysis level

'Scores' (default) | character vector with values 'Deciles', 'Scores'

Type of analysis level, specified as the comma-separated pair consisting of 'AnalysisLevel' and a character vector with one of the following values:

- 'Scores' — Returns the statistics (`Stats`) at the observation level. Scores are sorted from riskiest to safest, and duplicates are removed.
- 'Deciles' — Returns the statistics (`Stats`) at decile level. Scores are sorted from riskiest to safest and binned with their corresponding statistics into 10 deciles (10%, 20%, ..., 100%).

Data Types: char

Plot — Type of plot

'None' (default) | character vector with values 'None', 'CAP', 'ROC', 'KS' | cell array of character vectors with values 'None', 'CAP', 'ROC', 'KS'

Type of plot, specified as the comma-separated pair consisting of 'Plot' and a character vector with one of the following values:

- 'None' — No plot is displayed.
- 'CAP' — Cumulative Accuracy Profile. Plots the fraction of borrowers up to score “s” versus the fraction of defaulters up to score “s” ('PctObs' versus 'Sensitivity' columns of T optional output argument). For more details, see “Cumulative Accuracy Profile (CAP)” on page 15-2015.

- **'ROC'** — Receiver Operating Characteristic. Plots the fraction of non-defaulters up to score “s” versus the fraction of defaulters up to score “s” (**'FalseAlarm'** versus **'Sensitivity'** columns of T optional output argument). For more details, see “Receiver Operating Characteristic (ROC)” on page 15-2015.
- **'KS'** — Kolmogorov-Smirnov. Plots each score “s” versus the fraction of defaulters up to score “s,” and also versus the fraction of non-defaulters up to score “s” (**'Scores'** versus both **'Sensitivity'** and **'FalseAlarm'** columns of the optional output argument T). For more details, see “Kolmogorov-Smirnov statistic (KS)” on page 15-2016.

Tip For the Kolmogorov-Smirnov statistic option, you can enter **'KS'** or **'K-S'**.

Data Types: char | cell

Output Arguments

Stats — Validation measures

table

Validation measures, returned as a 4-by-2 table. The first column, **'Measure'**, contains the names of the following measures:

- Accuracy ratio (AR)
- Area under the ROC curve (AUROC)
- The KS statistic
- KS score

The second column, **'Value'**, contains the values corresponding to these measures.

T — Validation statistics data

array

Validation statistics data, returned as an N-by-9 table of validation statistics data, sorted, by score, from riskiest to safest. When **AnalysisLevel** is set to **'Deciles'**, N is equal to 10. Otherwise, N is equal to the total number of unique scores, that is, scores without duplicates.

The table T contains the following nine columns, in this order:

- **'Scores'** — Scores sorted from riskiest to safest. The data in this row corresponds to all observations up to, and including the score in this row.
- **'ProbDefault'** — Probability of default for observations in this row. For deciles, the average probability of default for all observations in the given decile is reported.
- **'TrueBads'** — Cumulative number of “bads” up to, and including, the corresponding score.
- **'FalseBads'** — Cumulative number of “goods” up to, and including, the corresponding score.
- **'TrueGoods'** — Cumulative number of “goods” above the corresponding score.
- **'FalseGoods'** — Cumulative number of “bads” above the corresponding score.
- **'Sensitivity'** — Fraction of defaulters (or the cumulative number of “bads” divided by total number of “bads”). This is the distribution of “bads” up to and including the corresponding score.
- **'FalseAlarm'** — Fraction of non-defaulters (or the cumulative number of “goods” divided by total number of “goods”). This is the distribution of “goods” up to and including the corresponding score.

- 'PctObs' — Fraction of borrowers, or the cumulative number of observations, divided by total number of observations up to and including the corresponding score.

Note When creating the `creditscorecard` object with `creditscorecard`, if the optional name-value pair argument `WeightsVar` was used to specify observation (sample) weights, then the T table uses statistics, sums, and cumulative sums that are weighted counts.

hf — Handle to the plotted measures

figure handle

Figure handle to plotted measures, returned as a figure handle or array of handles. When `Plot` is set to 'None', `hf` is an empty array.

More About

Cumulative Accuracy Profile (CAP)

CAP is generally a concave curve and is also known as the Gini curve, Power curve, or Lorenz curve.

The scores of given observations are sorted from riskiest to safest. For a given fraction *M* (0% to 100%) of the total borrowers, the height of the CAP curve is the fraction of defaulters whose scores are less than or equal to the maximum score of the fraction *M*, also known as “Sensitivity.”

The area under the CAP curve, known as the AUCAP, is then compared to that of the perfect or “ideal” model, leading to the definition of a summary index known as the accuracy ratio (*AR*) or the Gini coefficient:

$$AR = \frac{A_R}{A_P}$$

where A_R is the area between the CAP curve and the diagonal, and A_P is the area between the perfect model and the diagonal. This represents a “random” model, where scores are assigned randomly and therefore the proportion of defaulters and non-defaulters is independent of the score. The perfect model is the model for which all defaulters are assigned the lowest scores, and therefore, perfectly discriminates between defaulters and nondefaulters. Thus, the closer to unity *AR* is, the better the scoring model.

Receiver Operating Characteristic (ROC)

To find the receiver operating characteristic (ROC) curve, the proportion of defaulters up to a given score “s,” or “Sensitivity,” is computed.

This proportion is known as the true positive rate (TPR). Additionally, the proportion of nondefaulters up to score “s,” or “False Alarm Rate,” is also computed. This proportion is also known as the false positive rate (FPR). The ROC curve is the plot of the “Sensitivity” vs. the “False Alarm Rate.” Computing the ROC curve is similar to computing the equivalent of a confusion matrix at each score level.

Similar to the CAP, the ROC has a summary statistic known as the area under the ROC curve (AUROC). The closer to unity, the better the scoring model. The accuracy ratio (*AR*) is related to the area under the curve by the following formula:

$$AR = 2(AUROC) - 1$$

Kolmogorov-Smirnov statistic (KS)

The Kolmogorov-Smirnov (KS) plot, also known as the fish-eye graph, is a common statistic used to measure the predictive power of scorecards.

The KS plot shows the distribution of defaulters and the distribution of non-defaulters on the same plot. For the distribution of defaulters, each score “s” is plotted versus the proportion of defaulters up to “s,” or “Sensitivity.” For the distribution of non-defaulters, each score “s” is plotted versus the proportion of non-defaulters up to “s,” or “False Alarm.” The statistic of interest is called the KS statistic and is the maximum difference between these two distributions (“Sensitivity” minus “False Alarm”). The score at which this maximum is attained is also of interest.

Using `validateModel` with Weights

Model validation statistics incorporate observation weights when these are provided by the user.

Without weights, the validation statistics are based on how many good and bad observations fall below a particular score. On the other hand, when observation weights are provided, the weight (not the count) is accumulated for the good and the bad observations that fall below a particular score.

When observation weights are defined using the optional `WeightsVar` name-value pair argument when creating a `creditscorecard` object, the weights stored in the `WeightsVar` column are used when validating the model on the training data. When a different validation data set is provided using the optional `data` input, observation weights for the validation data must be included in a column whose name matches `WeightsVar`, otherwise unit weights are used for the validation data set.

Not only the validation statistics, but also the credit scorecard scores themselves depend on the observation weights of the training data. For more information, see “Using `fitModel` with Weights” on page 15-1873 and “Credit Scorecard Modeling Using Observation Weights” on page 8-55.

Version History

Introduced in R2015a

References

- [1] “Basel Committee on Banking Supervision: Studies on the Validation of Internal Rating Systems.” Working Paper No. 14, February 2005.
- [2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.
- [3] Loeffler, G. and Posch, P. N. *Credit Risk Modeling Using Excel and VBA*. Wiley Finance, 2007.

See Also

`creditscorecard` | `bininfo` | `predictorinfo` | `modifypredictor` | `modifybins` | `bindata` | `plotbins` | `fitmodel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `probdefault` | `table`

Topics

- “Case Study for Credit Scorecard Analysis” on page 8-71
- “Credit Scorecard Modeling with Missing Values” on page 8-57
- “Troubleshooting Credit Scorecard Results” on page 8-64

"Credit Scorecard Modeling Workflow" on page 8-52

"About Credit Scorecards" on page 8-49

"Credit Scorecard Modeling Using Observation Weights" on page 8-55

fillmissing

Replace missing values for credit scorecard predictors

Syntax

```
sc = fillmissing(sc,PredictorNames,Statistics)
sc = fillmissing( ___,ConstantValue)
```

Description

`sc = fillmissing(sc,PredictorNames,Statistics)` replaces missing values of the predictor `PredictorNames` with values defined by `Statistics` and returns an updated credit scorecard object (`sc`). Standard missing data is defined as follows:

- NaN for numeric arrays
- <undefined> for categorical arrays

Note If you run `fillmissing` after binning a predictor, the existing cutpoints and bin edges are preserved and the "Good" and "Bad" counts from the <missing> bin are added to the corresponding bin.

`sc = fillmissing( ___,ConstantValue)` uses arguments from the previous syntax and a value for a `ConstantValue` to replace missing values.

Examples

Fill Missing Data in a creditcorecard Object

This example shows how to use `fillmissing` to replace missing values in the `CustAge` and `ResStatus` predictors with user-defined values. For additional information on alternative approaches for "treating" missing data, see "Credit Scorecard Modeling with Missing Values" on page 8-57.

Load the credit scorecard data and use `dataMissing` for the training data.

```
load CreditCardData.mat
disp(head(dataMissing));
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	Othe
1	53	62	<undefined>	Unknown	50000	55	Ye
2	61	22	Home Owner	Employed	52000	25	Ye
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Ye
5	68	56	Home Owner	Employed	53000	14	Ye
6	65	13	Home Owner	Employed	48000	59	Ye
7	34	32	Home Owner	Unknown	32000	26	Ye
8	50	57	Other	Employed	51000	33	No

Create a `creditscorecard` object with `'BinMissingData'` set to `true`.

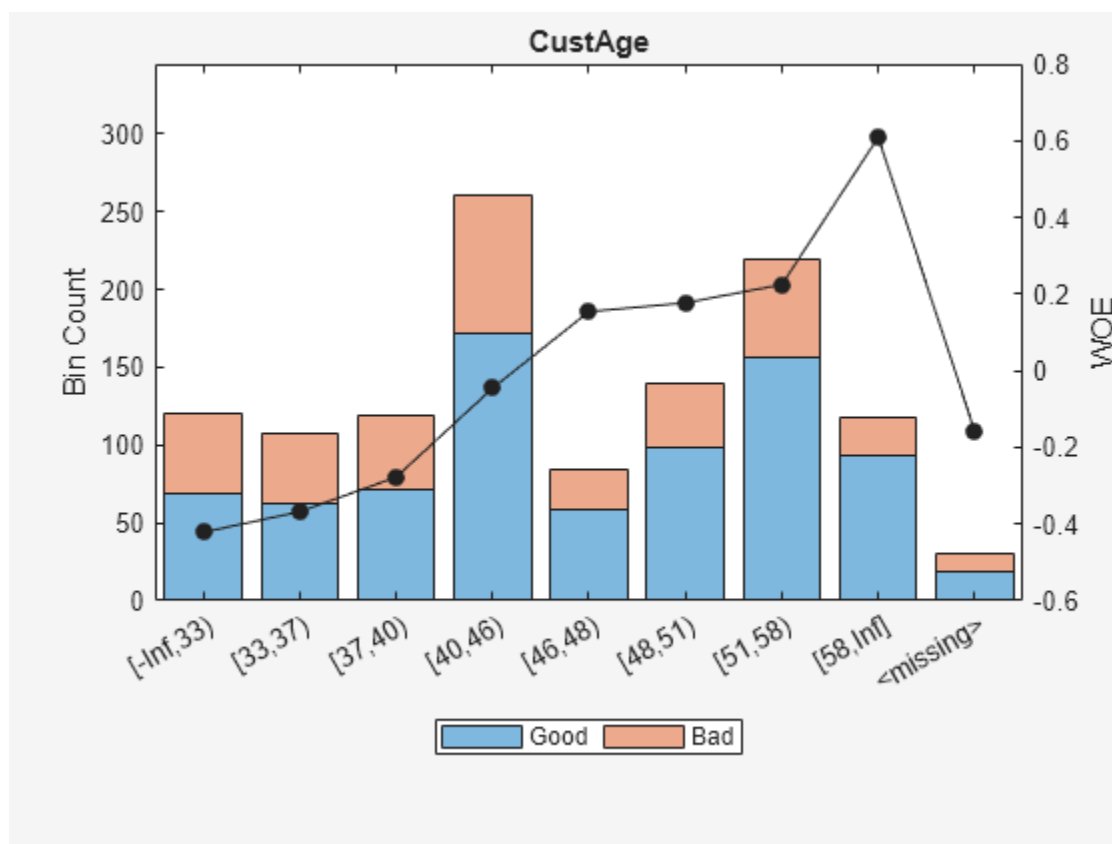
```
sc = creditscorecard(dataMissing, 'BinMissingData', true);
sc = autobinning(sc);
```

Use `bininfo` and `plotbins` to display the `CustAge` and `ResStatus` predictors with missing data.

```
bininfo(sc, 'CustAge')
```

```
ans=10x6 table
      Bin      Good      Bad      Odds      WOE      InfoValue
      ---      ---      ---      ---      ---      ---
{' [-Inf,33) ' }      69      52      1.3269     -0.42156      0.018993
{' [33,37) ' }      63      45       1.4     -0.36795      0.012839
{' [37,40) ' }      72      47      1.5319     -0.2779      0.0079824
{' [40,46) ' }     172      89      1.9326     -0.04556      0.0004549
{' [46,48) ' }      59      25       2.36      0.15424      0.0016199
{' [48,51) ' }      99      41      2.4146      0.17713      0.0035449
{' [51,58) ' }     157      62      2.5323      0.22469      0.0088407
{' [58,Inf] ' }      93      25       3.72      0.60931      0.032198
{' <missing> ' }      19      11      1.7273     -0.15787      0.00063885
{' Totals ' }     803     397      2.0227         NaN      0.087112
```

```
plotbins(sc, 'CustAge');
```

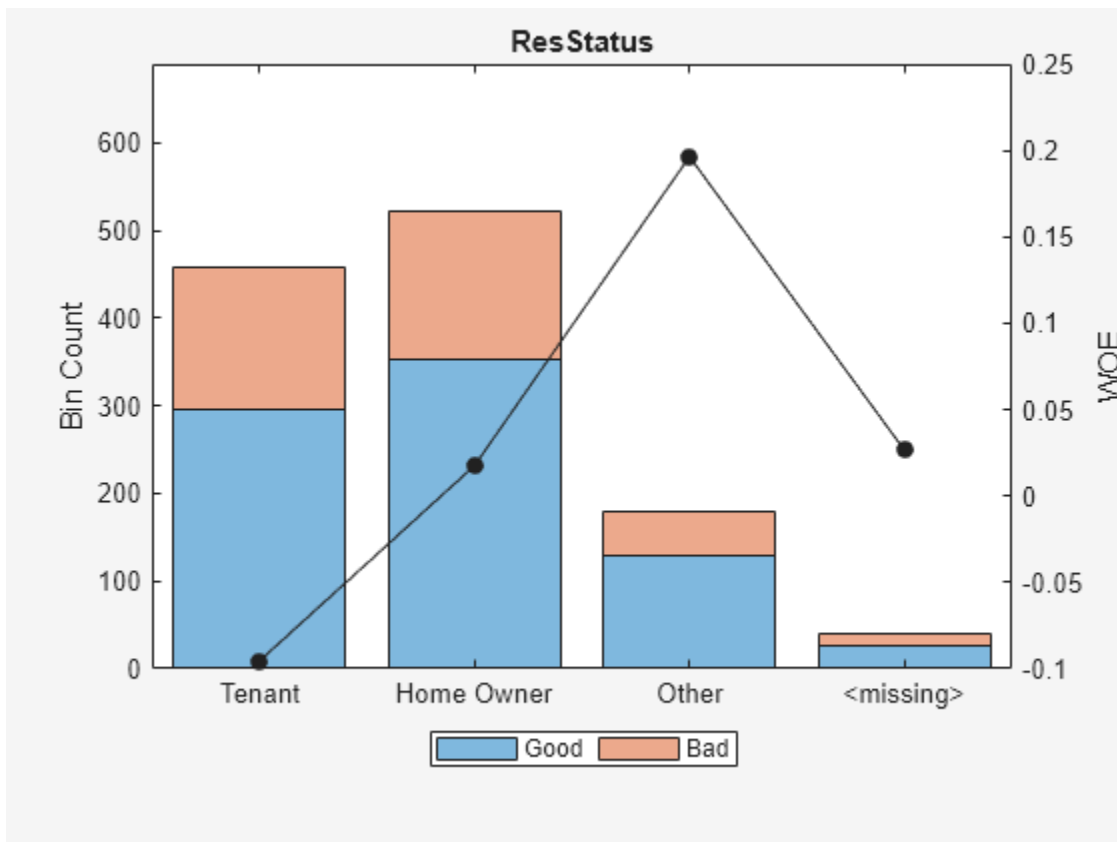


```
bininfo(sc, 'ResStatus')
```

ans=5x6 table

Bin	Good	Bad	Odds	WOE	InfoValue
{'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{'Other' }	128	52	2.4615	0.19637	0.0055808
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus');
```



Use `fillmissing` to replace NaN values in `CustAge` with the median value and to replace the `<missing>` values in `ResStatus` with 'Tenant'. Use `predictorinfo` to verify the filled values.

```
sc = fillmissing(sc,{'CustAge'},'median');
sc = fillmissing(sc,{'ResStatus'},'constant','Tenant');
predictorinfo(sc,'CustAge')
```

ans=1x4 table

	PredictorType	LatestBinning	LatestFillMissingType	LatestFillM
CustAge	{'Numeric'}	{'Automatic / Monotone'}	{'Median'}	{[4

```
predictorinfo(sc, 'ResStatus')
```

ans=1x5 table

	PredictorType	Ordinal	LatestBinning	LatestFillMissingType
ResStatus	{'Categorical'}	false	{'Automatic / Monotone'}	{'Constant'}

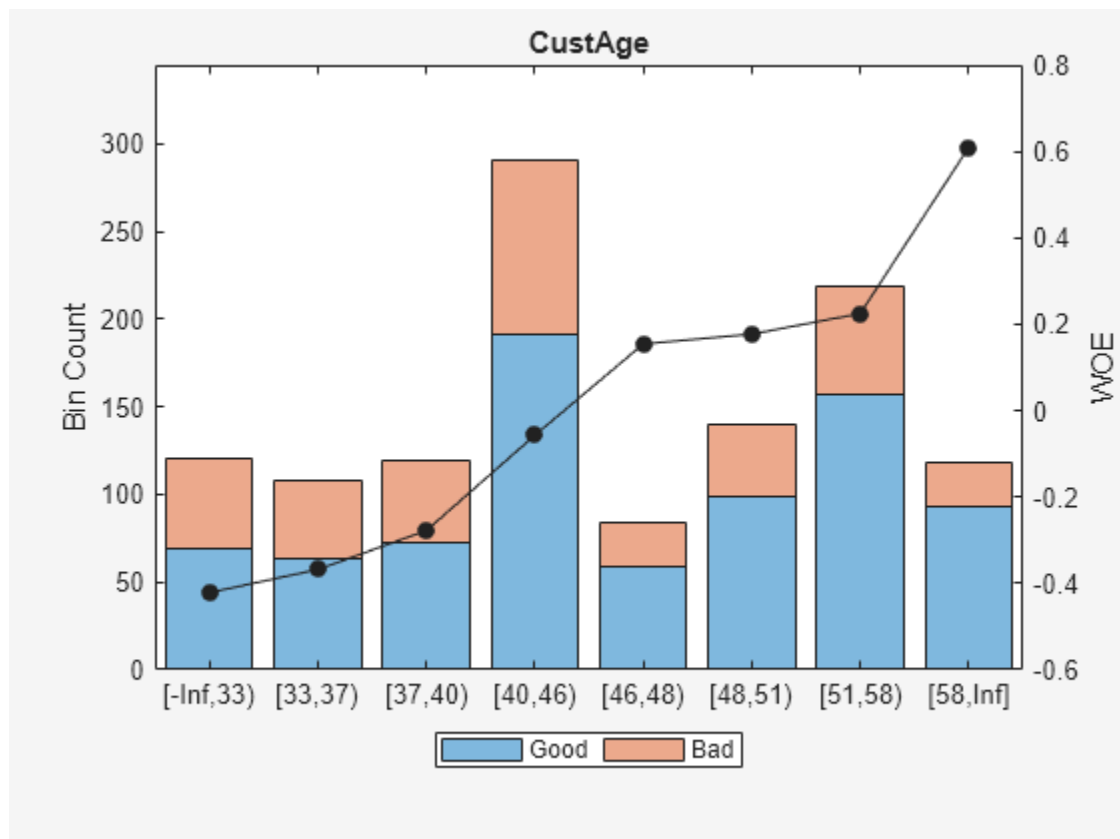
Use `bininfo` and `plotbins` to display the `CustAge` and `ResStatus` predictors to verify that the missing data has been replaced with the values defined by `fillmissing`.

```
bininfo(sc, 'CustAge')
```

ans=9x6 table

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'} }	69	52	1.3269	-0.42156	0.018993
{'[33,37)'} }	63	45	1.4	-0.36795	0.012839
{'[37,40)'} }	72	47	1.5319	-0.2779	0.0079824
{'[40,46)'} }	191	100	1.91	-0.057315	0.0008042
{'[46,48)'} }	59	25	2.36	0.15424	0.0016199
{'[48,51)'} }	99	41	2.4146	0.17713	0.0035449
{'[51,58)'} }	157	62	2.5323	0.22469	0.0088407
{'[58,Inf]'} }	93	25	3.72	0.60931	0.032198
{'Totals' }	803	397	2.0227	NaN	0.086822

```
plotbins(sc, 'CustAge');
```

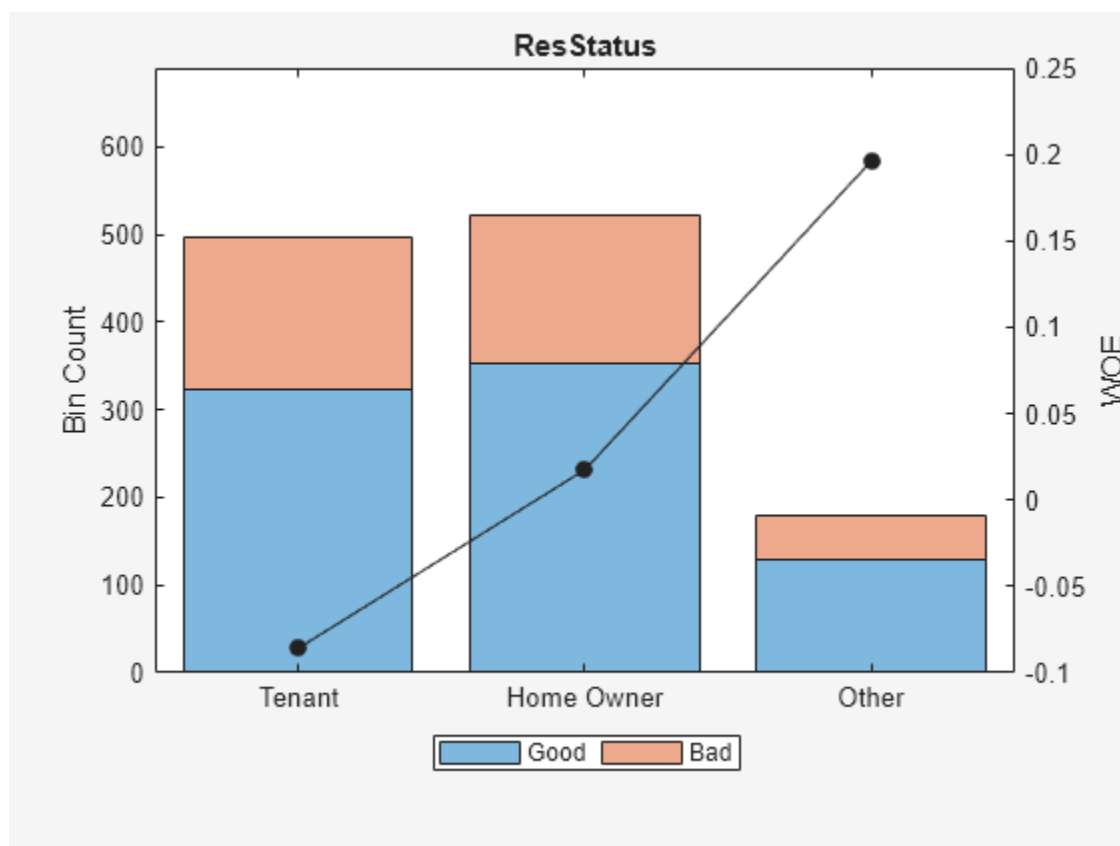


```
bininfo(sc, 'ResStatus')
```

```
ans=4x6 table
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Tenant' }	323	174	1.8563	-0.085821	0.0030935
{'Home Owner'}	352	171	2.0585	0.017549	0.00013382
{'Other' }	128	52	2.4615	0.19637	0.0055808
{'Totals' }	803	397	2.0227	NaN	0.0088081

```
plotbins(sc, 'ResStatus');
```



Use `fitmodel` and then run `formatpoints`, `displaypoints`, and `score`.

```
sc = fitmodel(sc, 'Display', 'off');
sc = formatpoints(sc, 'WorstAndBest', [300 800]);
t = displaypoints(sc)
```

```
t=31x3 table
```

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33) ' }	72.565
{'CustAge' }	{'[33,37) ' }	76.588
{'CustAge' }	{'[37,40) ' }	83.346

```

{'CustAge' } {'[40,46)' } 99.902
{'CustAge' } {'[46,48)' } 115.78
{'CustAge' } {'[48,51)' } 117.5
{'CustAge' } {'[51,58)' } 121.07
{'CustAge' } {'[58,Inf]' } 149.93
{'CustAge' } {'<missing>' } 99.902
{'EmpStatus' } {'Unknown' } 79.64
{'EmpStatus' } {'Employed' } 133.98
{'EmpStatus' } {'<missing>' } NaN
{'CustIncome' } {'[-Inf,29000)' } 21.926
{'CustIncome' } {'[29000,33000)' } 73.949
{'CustIncome' } {'[33000,35000)' } 97.117
{'CustIncome' } {'[35000,40000)' } 101.44
:

```

When a validation data set has missing values and you use `fillmissing` with the training dataset, the missing values in the validation data set are assigned the same points as the corresponding bins containing the filled values.

As the table shows, the '`<missing>`' bin for the `CustAge` predictor is assigned the same points as the '`[40,46)`' bin because the missing data is filled with the median value 45.

The points assigned to the '`<missing>`' bin for the `EmpStatus` predictor are `NaN` because `fillmissing` is not used for that predictor. The assigned points are decided by the default '`NoScore`' for the '`Missing`' name-value pair argument in `formatpoints`.

Create a test validation data set (`tdata`) and add missing values.

```

tdata = data(1:10,:);
tdata.CustAge(1) = NaN;
tdata.ResStatus(2) = missing;
[scr,pts] = score(sc,tdata)

```

```
scr = 10×1
```

```

566.7335
611.2547
584.5130
628.7876
609.7148
671.1048
403.6413
551.9461
575.9874
524.4789

```

```
pts=10×5 table
```

CustAge	EmpStatus	CustIncome	TmWBank	AMBalance
99.902	79.64	153.88	145.38	87.933
149.93	133.98	153.88	85.531	87.933
115.78	133.98	101.44	145.38	87.933
117.5	133.98	153.88	83.991	139.44
149.93	133.98	153.88	83.991	87.933
149.93	133.98	153.88	145.38	87.933

76.588	79.64	73.949	85.531	87.933
117.5	133.98	153.88	85.531	61.06
117.5	79.64	153.88	85.531	139.44
117.5	79.64	153.88	85.531	87.933

Handling of Missing Values in Validation Data Sets

This example shows different possibilities for handling missing data in validation data.

When scoring data from a validation data set, you have several options. If you choose to do nothing, the points assigned to the missing data are NaN, which comes from the default 'NoScore' for the 'Missing' name-value pair argument in `formatpoints`.

If you want to score missing values of all the predictors with one consistent metric, you can use the options 'ZeroWOE', 'MinPoints', or 'MaxPoints' for the 'Missing' name-value pair argument in `formatpoints`.

```
load CreditCardData.mat
sc = creditscorecard(data);
predictorinfo(sc, 'CustAge')
```

ans=1x4 table

	PredictorType	LatestBinning	LatestFillMissingType	LatestFillMissingValue
CustAge	{'Numeric'}	{'Original Data'}	{'Original'}	{0x0 double}

```
predictorinfo(sc, 'ResStatus')
```

ans=1x5 table

	PredictorType	Ordinal	LatestBinning	LatestFillMissingType	LatestFillMissingValue
ResStatus	{'Categorical'}	false	{'Original Data'}	{'Original'}	{0x0 double}

```
sc = autobinning(sc);
sc = fitmodel(sc, 'display', 'off');
```

```
displaypoints(sc)
```

ans=37x3 table

Predictors	Bin	Points
{'CustAge' }	{'[-Inf,33)' }	-0.15894
{'CustAge' }	{'[33,37)' }	-0.14036
{'CustAge' }	{'[37,40)' }	-0.060323
{'CustAge' }	{'[40,46)' }	0.046408
{'CustAge' }	{'[46,48)' }	0.21445
{'CustAge' }	{'[48,58)' }	0.23039
{'CustAge' }	{'[58,Inf]' }	0.479
{'CustAge' }	{'<missing>' }	NaN
{'ResStatus' }	{'Tenant' }	-0.031252

```

{'ResStatus' } {'Home Owner' } 0.12696
{'ResStatus' } {'Other' } 0.37641
{'ResStatus' } {'<missing>' } NaN
{'EmpStatus' } {'Unknown' } -0.076317
{'EmpStatus' } {'Employed' } 0.31449
{'EmpStatus' } {'<missing>' } NaN
{'CustIncome' } {'[-Inf,29000)' } -0.45716
:

```

```

sc = formatpoints(sc, 'Missing', 'minpoints', 'WorstAndBestScores', [300 850]);
displaypoints(sc)

```

```

ans=37×3 table
    Predictors      Bin      Points
    _____  _____  _____
{'CustAge' } {'[-Inf,33)' } 46.396
{'CustAge' } {'[33,37)' } 48.727
{'CustAge' } {'[37,40)' } 58.772
{'CustAge' } {'[40,46)' } 72.167
{'CustAge' } {'[46,48)' } 93.256
{'CustAge' } {'[48,58)' } 95.256
{'CustAge' } {'[58,Inf]' } 126.46
{'CustAge' } {'<missing>' } 46.396
{'ResStatus' } {'Tenant' } 62.421
{'ResStatus' } {'Home Owner' } 82.276
{'ResStatus' } {'Other' } 113.58
{'ResStatus' } {'<missing>' } 62.421
{'EmpStatus' } {'Unknown' } 56.765
{'EmpStatus' } {'Employed' } 105.81
{'EmpStatus' } {'<missing>' } 56.765
{'CustIncome' } {'[-Inf,29000)' } 8.9706
:

```

The value of -32.5389 for the <missing> bin of 'CustAge' comes from the 'minPoints' argument for formatpoints.

```

[scr,pts] = score(sc,dataMissing(1:5,:))

```

```

scr = 5×1

```

```

602.0394
648.1988
560.5569
613.5595
646.8109

```

```

pts=5×7 table
    CustAge    ResStatus    EmpStatus    CustIncome    TmWBank    OtherCC    AMBalance
    _____  _____  _____  _____  _____  _____  _____
95.256      62.421      56.765      121.18      116.05      86.224      64.15
126.46      82.276      105.81      121.18      62.107      86.224      64.15
93.256      62.421      105.81      76.585      116.05      42.287      64.15
46.396      82.276      105.81      121.18      60.719      86.224      110.96

```

126.46 82.276 105.81 121.18 60.719 86.224 64.15

Alternatively, you can score missing data for each individual predictor with a different statistic based on that predictor's information. To do so, use `fillmissing` for a `creditscorecard` object.

```
load CreditCardData.mat
sc = creditscorecard(data);
sc = fillmissing(sc, 'CustAge', 'constant', 35);
predictorinfo(sc, 'CustAge')
```

```
ans=1x4 table
      PredictorType LatestBinning LatestFillMissingType LatestFillMissingValue
      _____
CustAge {'Numeric'} {'Original Data'} {'Constant'} {[35]}
```

```
sc = fillmissing(sc, 'ResStatus', 'Mode');
predictorinfo(sc, 'ResStatus')
```

```
ans=1x5 table
      PredictorType Ordinal LatestBinning LatestFillMissingType LatestFillMissingValue
      _____
ResStatus {'Categorical'} false {'Original Data'} {'Mode'}
```

```
sc = autobinning(sc);
sc = fitmodel(sc, 'display', 'off');
sc = formatpoints(sc, 'Missing', 'minpoints', 'WorstAndBestScores', [300 850]);
```

```
displaypoints(sc)
```

```
ans=37x3 table
      Predictors Bin Points
      _____
{'CustAge' } {'[-Inf,33)'} 46.396
{'CustAge' } {'[33,37)'} 48.727
{'CustAge' } {'[37,40)'} 58.772
{'CustAge' } {'[40,46)'} 72.167
{'CustAge' } {'[46,48)'} 93.256
{'CustAge' } {'[48,58)'} 95.256
{'CustAge' } {'[58,Inf]'} 126.46
{'CustAge' } {'<missing>'} 48.727
{'ResStatus' } {'Tenant'} 62.421
{'ResStatus' } {'Home Owner'} 82.276
{'ResStatus' } {'Other'} 113.58
{'ResStatus' } {'<missing>'} 82.276
{'EmpStatus' } {'Unknown'} 56.765
{'EmpStatus' } {'Employed'} 105.81
{'EmpStatus' } {'<missing>'} 56.765
{'CustIncome' } {'[-Inf,29000)'} 8.9706
⋮
```

The value of `<missing>` for `'CustAge'` comes from the fill value of 35 even though the training data has no missing values.


```
disp(dataMissing(1:5,:));
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
[scr,pts] = score(sc,dataMissing(1:5,:))
```

```
scr = 5×1
```

```
621.8943
648.1988
560.5569
615.8904
646.8109
```

```
pts=5×7 table
```

CustAge	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC	AMBalance
95.256	82.276	56.765	121.18	116.05	86.224	64.15
126.46	82.276	105.81	121.18	62.107	86.224	64.15
93.256	62.421	105.81	76.585	116.05	42.287	64.15
48.727	82.276	105.81	121.18	60.719	86.224	110.96
126.46	82.276	105.81	121.18	60.719	86.224	64.15

Input Arguments

sc — Credit scorecard model

creditscorecard object

Credit scorecard model, specified as a `creditscorecard` object.

PredictorNames — Name of creditscorecard predictor whose missing data is filled

character vector | string | cell array of character vectors | string array

Name of `creditscorecard` predictor to fill missing data for, specified as a scalar character vector, scalar string, cell array of character vectors, or string array.

Data Types: `char` | `string` | `cell`

Statistics — Statistic to use to fill missing data for predictors

character vector with a value of 'mean', 'median', 'mode', 'original', or 'constant' | string with a value of "mean", "median", "mode", "original", or "constant"

Statistic to use to fill missing data for the predictors, specified as a character vector or string with one of the following values.

- 'mean' — Replace missing data with the average or mean value. The option is valid only for numeric data. The 'mean' calculates the weighted mean of the predictor by referring to the

predictor column and the `Weights` column from the `creditscorecard` object. For more information, see “Weighted Mean” on page 15-2028.

- `'median'` — Replace missing data with the median value. Valid for numeric and ordinal data. The `'median'` calculates the weighted median of the predictor by referring to the predictor column and the `Weights` column from the `creditscorecard` object. For more information, see “Weighted Median” on page 15-2029.
- `'mode'` — Replace missing data with the mode. Valid for numeric and both nominal and ordinal categorical data. The `'mode'` calculates the weighted mode of the predictor by referring to the predictor column and the `Weights` column from the `creditscorecard` object. For more information, see “Weighted Mode” on page 15-2029.
- `'original'` — Set the missing data for numeric and categorical predictors back to its original value: `NaN` if numeric, `<undefined>` or `<missing>` if categorical.
- `'constant'` — Set the missing data for numeric and categorical predictors to a constant value that you specify in the optional argument for `ConstantValue`.

Data Types: `char` | `string`

ConstantValue — Value to fill missing entries in predictors specified in `PredictorNames`

[] (default) | numeric | character vector | string | cell array of character vectors | string array

(Optional) Value to fill missing entries in predictors specified in `PredictorNames`, specified as a numeric value, character vector, string, or cell array of character vectors.

Note You can use `ConstantValue` only if you set the `Statistics` argument to `'constant'`.

Data Types: `char` | `double` | `string` | `cell`

Output Arguments

sc — Updated `creditscorecard`

`creditscorecard` object

Updated `creditscorecard` object, returned as an object.

More About

Weighted Mean

The weighted mean is similar to an ordinary mean except that instead of each of the data points contributing equally to the final average, some data points contribute more than others.

The weighted mean for a nonempty finite multiset of data (x) with corresponding nonnegative weights (w) is

$$\bar{x} = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

Weighted Median

The weighted median is the 50% weighted percentile, where the percentage in the total weight is counted instead of the total number.

For n distinct ordered elements (x) positive weights (w) such that $\sum_{i=1}^n w_i = 1$, the weighted median is the element x_k :

$$\sum_{i=1}^{k-1} w_i \leq \frac{1}{2} \text{ and } \sum_{i=k+1}^n w_i \leq \frac{1}{2}$$

In the case where the respective weights of both elements border the midpoint of the set of weights without encapsulating it, each element defines a partition equal to 1/2. These elements are referred to as the lower weighted median and upper weighted median. The weighted median is chosen based on which element keeps the partitions most equal. This median is always the weighted median with the lowest weight. In the event that the upper and lower weighted medians are equal, the lower weighted median is accepted.

Weighted Mode

The weighted mode of a set of weighted data values is the value that appears most often.

The mode of a sample is the element that occurs most often in the collection. For example, the mode of the sample [1, 3, 6, 6, 6, 6, 7, 7, 12, 12, 17] is 6.

Version History

Introduced in R2020a

References

- [1] "Basel Committee on Banking Supervision: Studies on the Validation of Internal Rating Systems." Working Paper No. 14, February 2005.
- [2] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.
- [3] Loeffler, G. and Posch, P. N. *Credit Risk Modeling Using Excel and VBA*. Wiley Finance, 2007.

See Also

`creditscorecard` | `bininfo` | `predictorinfo` | `modifypredictor` | `modifybins` | `bindata` | `plotbins` | `fitmodel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `probdefault` | `validatemodel` | `table`

Topics

- "Case Study for Credit Scorecard Analysis" on page 8-71
- "Credit Scorecard Modeling with Missing Values" on page 8-57
- "Troubleshooting Credit Scorecard Results" on page 8-64
- "Credit Scorecard Modeling Workflow" on page 8-52
- "About Credit Scorecards" on page 8-49
- "Credit Scorecard Modeling Using Observation Weights" on page 8-55

creditscorecard

Create `creditscorecard` object to build credit scorecard model

Description

Build a credit scorecard model by creating a `creditscorecard` object and specify input data in a table format.

After creating a `creditscorecard` object, you can use the associated object functions to bin the data and perform logistic regression analysis to develop a credit scorecard model to guide credit decisions. This workflow shows how to develop a credit scorecard model.

- 1 Use `screenpredictors` from Risk Management Toolbox to pare down a potentially large set of predictors to a subset that is most predictive of the credit score card response variable. Use this subset of predictors when creating the `creditscorecard` object.
- 2 Create a `creditscorecard` object (see “Create `creditscorecard`” on page 15-2030 and “Properties” on page 15-2033).
- 3 Bin the data using `autobinning`.
- 4 Fit a logistic regression model using `fitmodel` or `fitConstrainedModel`.
- 5 Review and format the credit scorecard points using `displaypoints` and `formatpoints`. At this point in the workflow, if you have a license for Risk Management Toolbox, you have the option to create a `compactCreditScorecard` object (`csc`) using the `compact` function. You can then use the following functions `displaypoints`, `score`, and `probdefault` from the Risk Management Toolbox with the `csc` object.
- 6 Score the data using `score`.
- 7 Calculate the probabilities of default for the data using `probdefault`.
- 8 Validate the quality of the credit scorecard model using `validatemodel`.

For more detailed information on this workflow, see “Credit Scorecard Modeling Workflow” on page 8-52.

Creation

Syntax

```
sc = creditscorecard(data)
sc = creditscorecard( ___, Name, Value)
```

Description

`sc = creditscorecard(data)` creates a `creditscorecard` object by specifying `data`. The credit scorecard model, returned as a `creditscorecard` object, contains the binning maps or rules (cut points or category groupings) for one or more predictors.

`sc = creditscorecard( ___, Name, Value)` sets Properties on page 15-2033 using name-value pairs and any of the arguments in the previous syntax. For example, `sc =`

`creditscorecard(data, 'GoodLabel', 0, 'IDVar', 'CustID', 'ResponseVar', 'status', 'PredictorVars', {'CustAge', 'CustIncome'}, 'WeightsVar', 'RowWeights', 'BinMissingData', true).` You can specify multiple name-value pairs.

Note To use observation (sample) weights in the credit scorecard workflow, when creating a `creditscorecard` object, you must use the optional name-value pair `WeightsVar` to define which column in the data contains the weights.

Input Arguments

data — Data for `creditscorecard` object

table

Data for the `creditscorecard` object, specified as a MATLAB table, where each column of data can be any one of the following data types:

- Numeric
- Logical
- Cell array of character vectors
- Character array
- Categorical
- String

In addition, the table must contain a binary response variable. Before creating a `creditscorecard` object, perform a data preparation task to have appropriately structured data as input to a `creditscorecard` object. The `data` input sets the `Data` on page 15-0 property.

Data Types: table

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `sc =`

```
creditscorecard(data, 'GoodLabel', 0, 'IDVar', 'CustID', 'ResponseVar', 'status', 'PredictorVars',
{'CustAge', 'CustIncome'}, 'WeightsVar', 'RowWeights', 'BinMissingData', true)
```

GoodLabel — Indicator for which of two possible values in response variable correspond to “Good” observations

set to the response value with the highest count (default) | character vector | numeric scalar | logical

Indicator for which of the two possible values in the response variable correspond to “Good” observations, specified as the comma-separated pair consisting of `'GoodLabel'` and a numeric scalar, logical, or character vector. The `GoodLabel` name-value pair argument sets the `GoodLabel` on page 15-0 property.

When specifying `GoodLabel`, follow these guidelines.

If Response Variable is...	GoodLabel Must be...
numeric	numeric
logical	logical or numeric
cell array of character vectors	character vector
character array	character vector
categorical	character vector

If not specified, `GoodLabel` is set to the response value with the highest count. However, if the optional `WeightsVar` argument is provided when creating the `creditscorecard` object, then counts are replaced with weighted frequencies. For more information, see “Credit Scorecard Modeling Using Observation Weights” on page 8-55.

`GoodLabel` can only be set when creating the `creditscorecard` object. This parameter cannot be set using dot notation.

Data Types: `char` | `double`

IDVar — Variable name used as ID or tag for observations

empty character vector `''` (default) | character vector

Variable name used as ID or tag for the observations, specified as the comma-separated pair consisting of `'IDVar'` and a character vector. The `IDVar` data could be an ordinal number (for example, 1,2,3...), a Social Security number. This is provided as a convenience to remove this column from the predictor variables. `IDVar` is case-sensitive. The `IDVar` name-value pair argument sets the `IDVar` on page 15-0 property.

You can set this optional parameter using the `creditscorecard` function or by using dot notation at the command line, as follows.

Example: `sc.IDVar = 'CustID'`

Data Types: `char`

ResponseVar — Response variable name

last column of the data input (default) | character vector

Response variable name, specified as the comma-separated pair consisting of `'ResponseVar'` and a character vector. The response variable data must be binary the "Good" or "Bad" indicator. The `ResponseVar` name-value pair argument sets the `ResponseVar` on page 15-0 property.

If not specified, `ResponseVar` is set to the last column of the data input. `ResponseVar` can only be set when creating the `creditscorecard` object using the `creditscorecard` function. `ResponseVar` is case-sensitive.

Data Types: `char`

WeightsVar — Weights variable name

empty character vector `''` (default) | character vector

Weights variable name, specified as the comma-separated pair consisting of `'WeightsVar'` and a character vector to indicate which column name in the `data` table contains the row weights. `WeightsVar` is case-sensitive. The `WeightsVar` name-value pair argument sets the `WeightsVar` on

page 15-0 property, and this property can only be set at the creation of a `creditscorecard` object. If the name-value pair argument `WeightsVar` is not specified when creating a `creditscorecard` object, then observation weights are set to unit weights by default.

The `WeightsVar` values are used in the credit scorecard workflow by `autobinning`, `bininfo`, `fitmodel`, and `validatemodel`. For more information, see “Credit Scorecard Modeling Using Observation Weights” on page 8-55.

Data Types: `char`

BinMissingData — Indicates if missing data is removed or displayed in a separate bin
`false` (missing data removed) (default) | logical with value `true` or `false`

Indicates if missing data is removed or displayed in a separate bin, specified as the comma-separated pair consisting of 'BinMissingdata' and a logical scalar with a value of `true` or `false`. If `BinMissingData` is `true`, the missing data for a predictor is displayed in a separate bin labeled `<missing>`.

Data Types: `logical`

Output Arguments

sc — Credit scorecard model
`creditscorecard` object

Credit scorecard model, returned as a `creditscorecard` object.

Properties

GoodLabel — Indicator for which of the two possible values in the response variable correspond to “Good” observations

set to the response value with the highest count (default) | character vector | numeric scalar | logical

Indicator for which of the two possible values in the response variable correspond to “Good” observations. When specifying `GoodLabel`, follow these guidelines:

If Response Variable is...	GoodLabel must be:
numeric	numeric
logical	logical or numeric
cell array of character vectors	character vector
character array	character vector
categorical	character vector

If not specified, `GoodLabel` is set to the response value with the highest count. This property can only be set with an optional parameter when creating a `creditscorecard` object. This property cannot be set using dot notation.

Data Types: `char` | `double` | `logical`

ResponseVar — Name of the response variable

last column of the data input (default) | character vector

Name of the response variable, specified as a character vector. The response variable data must be binary, the “Good” or “Bad” indicator. If not specified, `ResponseVar` is set to the last column of the

data input. This property can only be set with an optional parameter when creating a `creditscorecard` object. `ResponseVar` is case-sensitive.

Data Types: `char`

WeightsVar — Name of the variable used as ID or tag for weights

empty character vector `''` (default) | character vector

Name of the variable used as ID or tag to indicate which column name in the data table contains the row weights, specified as a character vector. This property can be set as an optional parameter (`WeightsVar`) when creating a `creditscorecard` object. `WeightsVar` is case-sensitive.

Data Types: `char`

VarNames — All variable names from the data input

`VarNames` come directly from data input to `creditscorecard` object (default)

This property is read-only.

`VarNames` is a cell array of character vectors containing the names of all variables in the data. The `VarNames` come directly from the data input to the `creditscorecard` object. `VarNames` is case-sensitive.

Data Types: `cell`

NumericPredictors — Name of numeric predictors

empty character vector `''` (default) | character vector

Name of numeric predictors, specified as a character vector. This property cannot be set by using dot notation at the command line. It can only be modified using the `modifypredictor` function.

Data Types: `char`

CategoricalPredictors — Name of categorical predictors

empty character vector `''` (default) | character vector

Name of categorical predictors, specified as a character vector. This property cannot be set by using dot notation at the command line. It can only be modified using the `modifypredictor` function.

Data Types: `char`

BinMissingData — Indicates if missing data is removed or displayed in a separate bin

`false` (missing data removed) (default) | logical with value `true` or `false`

Indicates if missing data is removed or displayed in a separate bin, specified as the comma-separated pair consisting of `'BinMissingdata'` and a logical scalar with a value of `true` or `false`. If `BinMissingData` is `true`, the missing data for a predictor is displayed in a separate bin labeled `<missing>`. For more information on working with missing data, see “Credit Scorecard Modeling with Missing Values” on page 8-57.

Data Types: `logical`

IDVar — Name of the variable used as ID or tag for the observations

empty character vector `''` (default) | character vector

Name of the variable used as ID or tag for the observations, specified as a character vector. This property can be set as an optional parameter when creating a `creditscorecard` object or by using dot notation at the command line. `IDVar` is case-sensitive.

Example: `sc.IDVar = 'CustID'`

Data Types: `char`

PredictorVars — Predictor variable names

set difference between `VarNames` and `{IDVar,ResponseVar}` (default) | cell array of character vectors containing names

Predictor variable names, specified using a cell array of character vectors containing names. By default, when you create a `creditscorecard` object, all variables are predictors except for `IDVar` and `ResponseVar`. This property can be modified using a name-value pair argument for the `fitmodel` function or by using dot notation. `PredictorVars` is case-sensitive and the predictor variable name cannot be the same as the `IDVar` or `ResponseVar`.

Example: `sc.PredictorVars = {'CustID','CustIncome'}`

Data Types: `cell`

Data — Data used to create the creditscorecard object

table

Data used to create the `creditscorecard` object, specified as a table when creating a `creditscorecard` object. In the `Data` property, categorical predictors are stored as categorical arrays.

Example: `sc.Data(1:10,:)`

Data Types: `table`

creditscorecard Property	Set/Modify Property from Command Line Using creditscorecard Function	Modify Property Using Dot Notation	Property Not User-Defined and Value Is Defined Internally
<code>Data</code>	No	No	Yes, copy of data input
<code>IDVar</code>	Yes	Yes	No, but the user specifies this
<code>VarNames</code>	No	No	Yes
<code>ResponseVar</code>	Yes	No	If not specified, set to last column of data input
<code>WeightsVar</code>	No	No	Yes
<code>GoodLabel</code>	Yes	No	If not specified, set to response value with highest count
<code>PredictorVars</code>	Yes (also modifiable using <code>fitmodel</code> function)	Yes	Yes, but the user can modify this
<code>NumericPredictors</code>	No (can only be modified using <code>modifypredictor</code> function)	No	Yes, but the user can modify this

creditscorecard Property	Set/Modify Property from Command Line Using creditscorecard Function	Modify Property Using Dot Notation	Property Not User-Defined and Value Is Defined Internally
CategoricalPredictors	No (can only be modified using modifypredictor function)	No	Yes, but the user can modify this
BinMissingData	Yes	No	False by default, but the user can modify this

Object Functions

autobinning	Perform automatic binning of given predictors
bininfo	Return predictor's bin information
predictorinfo	Summary of credit scorecard predictor properties
modifypredictor	Set properties of credit scorecard predictors
fillmissing	Replace missing values for credit scorecard predictors
modifybins	Modify predictor's bins
bindata	Binned predictor variables
plotbins	Plot histogram counts for predictor variables
fitmodel	Fit logistic regression model to Weight of Evidence (WOE) data
fitConstrainedModel	Fit logistic regression model to Weight of Evidence (WOE) data subject to constraints on model coefficients
setmodel	Set model predictors and coefficients
displaypoints	Return points per predictor per bin
formatpoints	Format scorecard points and scaling
score	Compute credit scores for given data
probdefault	Likelihood of default for given data set
validatemodel	Validate quality of credit scorecard model
compact	Create compact credit scorecard

Examples

Create a creditscorecard Object

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data)
```

```
sc =
    creditscorecard with properties:
```

```
    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalan'
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalan'
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
```

```

BinMissingData: 0
IDVar: ''
PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
Data: [1200×11 table]

```

Create a creditscorecard Object Containing Weights

Use the CreditCardData.mat file to load the data (dataWeights) that contains a column (RowWeights) for the weights (using a dataset from Refaat 2011).

```
load CreditCardData
```

Create a creditscorecard object using the optional name-value pair argument for 'WeightsVar'.

```
sc = creditscorecard(dataWeights, 'WeightsVar', 'RowWeights')
```

```

sc =
creditscorecard with properties:

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: 'RowWeights'
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalanc
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    Data: [1200×12 table]

```

Display creditscorecard Object Properties

Create a creditscorecard object using the CreditCardData.mat file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
```

```
sc = creditscorecard(data)
```

```

sc =
creditscorecard with properties:

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalanc
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn

```

Data: [1200x11 table]

To display the `creditscorecard` object properties, use dot notation.

sc.PredictorVars

```
ans = 1x10 cell
    'CustID'    'CustAge'    'TmAtAddress'    'ResStatus'    'EmpStatus'    'CustIncor
```

sc.VarNames

```
ans = 1x11 cell
    'CustID'    'CustAge'    'TmAtAddress'    'ResStatus'    'EmpStatus'    'CustIncor
```

Change a Property Value for a creditcard Object

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data)
```

```
sc =
  creditscorecard with properties:

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalanc
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: ''
    PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    Data: [1200x11 table]
```

Since the `IDVar` property has public access, you can change its value at the command line.

```
sc.IDVar = 'CustID'
```

```
sc =
  creditcorecard with properties:

    GoodLabel: 0
    ResponseVar: 'status'
    WeightsVar: ''
    VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
    NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBBalance' 'Uti
    CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
    BinMissingData: 0
    IDVar: 'CustID'
    PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'T
```

```
Data: [1200×11 table]
```

Create a creditscorecard Object

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011).

```
load CreditCardData
sc = creditscorecard(data)
```

```
sc =
    creditscorecard with properties:
```

```

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustInconvenience' 'CustSatisfaction' 'CustComplaint' 'CustChurn' 'CustRecommendation'}
        NumericPredictors: {'CustID' 'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'CustSatisfaction' 'CustComplaint' 'CustChurn' 'CustRecommendation'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: ''
        PredictorVars: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'CustInconvenience' 'CustSatisfaction' 'CustComplaint' 'CustChurn' 'CustRecommendation'}
        Data: [1200×11 table]
```

In this example, the default values for the properties `ResponseVar`, `PredictorVars` and `GoodLabel` are assigned when this object is created. By default, the property `ResponseVar` is set to the variable name that is in the last column of the input data ('`status`' in this example). The property `PredictorVars` contains the names of all the variables that are in `VarNames`, but excludes `IDVar` and `ResponseVar`. Also, by default in the previous example, `GoodLabel` is set to 0, since it is the value in the response variable (`ResponseVar`) with the highest count.

Display the `creditscorecard` object properties using dot notation.

```
sc.PredictorVars
```

```
ans = 1×10 cell
    {'CustID'}    {'CustAge'}    {'TmAtAddress'}    {'ResStatus'}    {'EmpStatus'}    {'CustIncome'}
```

```
sc.VarNames
```

```
ans = 1×11 cell
    {'CustID'}    {'CustAge'}    {'TmAtAddress'}    {'ResStatus'}    {'EmpStatus'}    {'CustIncome'}
```

Since `IDVar` and `PredictorVars` have public access, you can change their values at the command line.

```
sc.IDVar = 'CustID'
```

```
sc =
    creditscorecard with properties:
```

```

        GoodLabel: 0
```

```

        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'Uti
CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'T
        Data: [1200×11 table]

sc.PredictorVars = {'CustIncome','ResStatus','AMBalance'}

sc =
    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
NumericPredictors: {'CustIncome' 'AMBalance'}
CategoricalPredictors: {'ResStatus'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'ResStatus' 'CustIncome' 'AMBalance'}
        Data: [1200×11 table]

disp(sc)

    creditscorecard with properties:

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIn
NumericPredictors: {'CustIncome' 'AMBalance'}
CategoricalPredictors: {'ResStatus'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'ResStatus' 'CustIncome' 'AMBalance'}
        Data: [1200×11 table]

```

Create a creditscorecard Object and Set GoodLabel and ResponseVar

Create a `creditscorecard` object using the `CreditCardData.mat` file to load the data (using a dataset from Refaat 2011). Then use name-value pair arguments for `creditscorecard` to define `GoodLabel` and `ResponseVar`.

```

load CreditCardData
sc = creditscorecard(data, 'IDVar', 'CustID', 'GoodLabel', 0, 'ResponseVar', 'status')

sc =
    creditscorecard with properties:

        GoodLabel: 0

```

```

        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
        BinMissingData: 0
        IDVar: 'CustID'
        PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}
        Data: [1200x11 table]

```

GoodLabel and ResponseVar can only be set (enforced) when creating a creditscorecard object using creditscorecard.

Create a creditscorecard Object and Set the 'BinMissingData' Property

Create a creditscorecard object using the CreditCardData.mat file to load the dataMissing with missing values.

```
load CreditCardData
head(dataMissing,5)
```

CustID	CustAge	TmAtAddress	ResStatus	EmpStatus	CustIncome	TmWBank	OtherCC
1	53	62	<undefined>	Unknown	50000	55	Yes
2	61	22	Home Owner	Employed	52000	25	Yes
3	47	30	Tenant	Employed	37000	61	No
4	NaN	75	Home Owner	Employed	53000	20	Yes
5	68	56	Home Owner	Employed	53000	14	Yes

```
fprintf('Number of rows: %d\n',height(dataMissing))
```

```
Number of rows: 1200
```

```
fprintf('Number of missing values CustAge: %d\n',sum(ismissing(dataMissing.CustAge)))
```

```
Number of missing values CustAge: 30
```

```
fprintf('Number of missing values ResStatus: %d\n',sum(ismissing(dataMissing.ResStatus)))
```

```
Number of missing values ResStatus: 40
```

Use creditscorecard with the name-value argument 'BinMissingData' set to true to bin the missing data in a separate bin.

```
sc = creditscorecard(dataMissing,'IDVar','CustID','BinMissingData',true);
sc = autobinning(sc);
disp(sc)
```

```
creditscorecard with properties:
```

```

        GoodLabel: 0
        ResponseVar: 'status'
        WeightsVar: ''
        VarNames: {'CustID' 'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'TmWBank' 'AMBalance' 'UtilizationRate'}

```

```

NumericPredictors: {'CustAge' 'TmAtAddress' 'CustIncome' 'TmWBank' 'AMBalance' 'Uti
CategoricalPredictors: {'ResStatus' 'EmpStatus' 'OtherCC'}
BinMissingData: 1
IDVar: 'CustID'
PredictorVars: {'CustAge' 'TmAtAddress' 'ResStatus' 'EmpStatus' 'CustIncome' 'Tr
Data: [1200x11 table]

```

Display bin information for numeric data for 'CustAge' that includes missing data in a separate bin labelled <missing>.

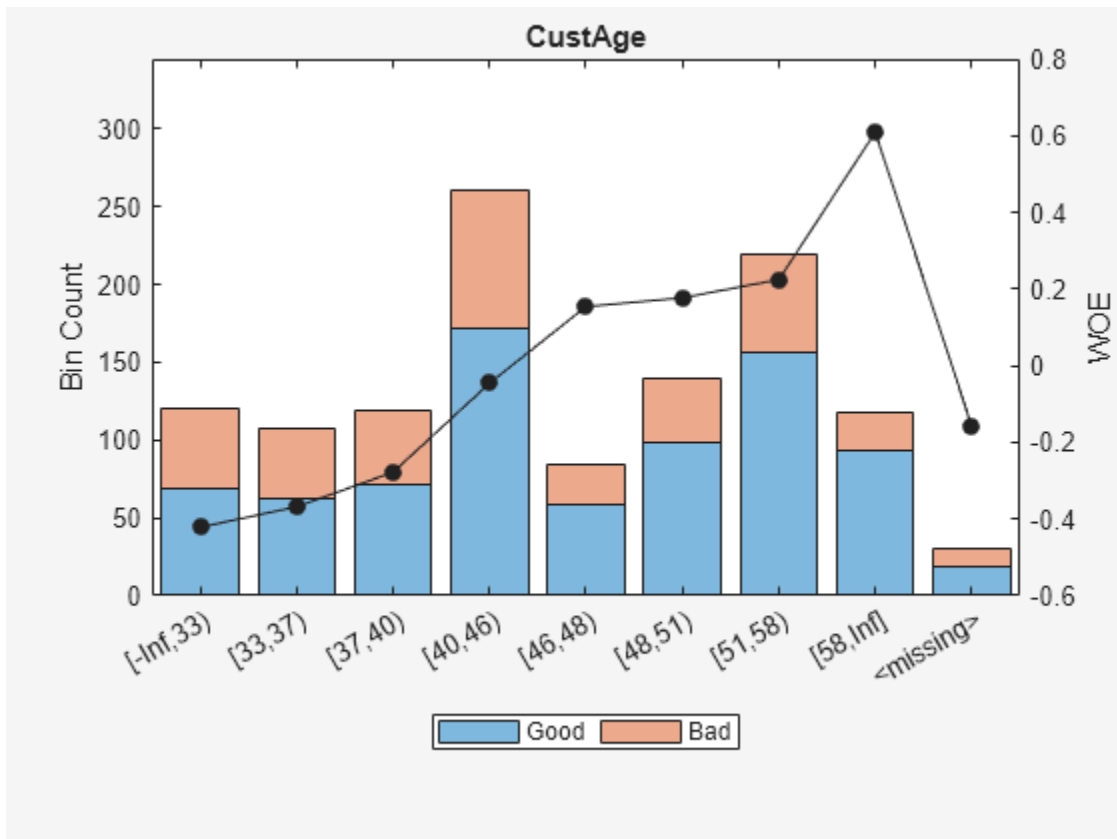
```

bi = bininfo(sc, 'CustAge');
disp(bi)

```

Bin	Good	Bad	Odds	WOE	InfoValue
{'[-Inf,33)'} }	69	52	1.3269	-0.42156	0.018993
{'[33,37)'} }	63	45	1.4	-0.36795	0.012839
{'[37,40)'} }	72	47	1.5319	-0.2779	0.0079824
{'[40,46)'} }	172	89	1.9326	-0.04556	0.0004549
{'[46,48)'} }	59	25	2.36	0.15424	0.0016199
{'[48,51)'} }	99	41	2.4146	0.17713	0.0035449
{'[51,58)'} }	157	62	2.5323	0.22469	0.0088407
{'[58,Inf]'} }	93	25	3.72	0.60931	0.032198
{'<missing>'} }	19	11	1.7273	-0.15787	0.00063885
{'Totals' }	803	397	2.0227	NaN	0.087112

```
plotbins(sc, 'CustAge')
```

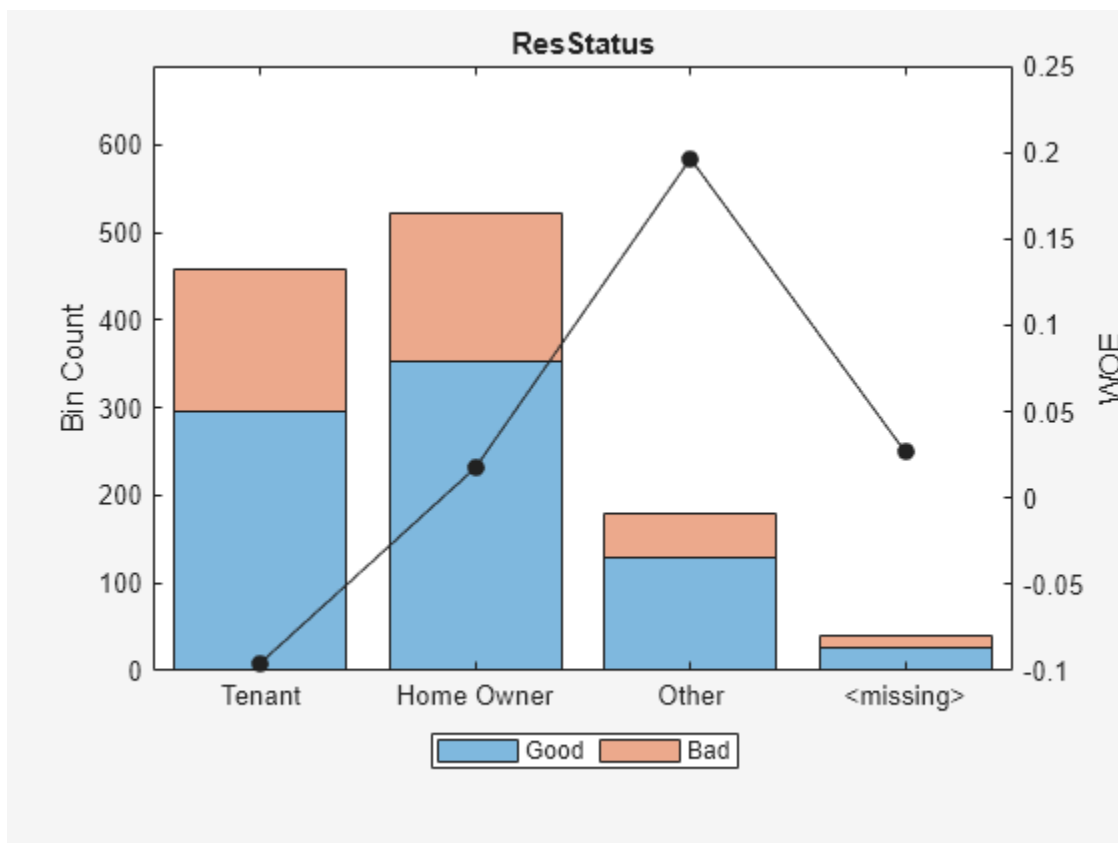


Display bin information for categorical data for 'ResStatus' that includes missing data in a separate bin labelled <missing>.

```
bi = bininfo(sc, 'ResStatus');
disp(bi)
```

Bin	Good	Bad	Odds	WOE	InfoValue
{'Tenant' }	296	161	1.8385	-0.095463	0.0035249
{'Home Owner' }	352	171	2.0585	0.017549	0.00013382
{'Other' }	128	52	2.4615	0.19637	0.0055808
{'<missing>' }	27	13	2.0769	0.026469	2.3248e-05
{'Totals' }	803	397	2.0227	NaN	0.0092627

```
plotbins(sc, 'ResStatus')
```



Version History

Introduced in R2014b

References

- [1] Anderson, R. *The Credit Scoring Toolkit*. Oxford University Press, 2007.
- [2] Refaat, M. *Data Preparation for Data Mining Using SAS*. Morgan Kaufmann, 2006.

[3] Refaat, M. *Credit Risk Scorecards: Development and Implementation Using SAS*. lulu.com, 2011.

See Also

Functions

`screenpredictors` | `autobinning` | `modifybins` | `bindata` | `bininfo` | `fillmissing` | `predictorinfo` | `modifypredictor` | `plotbins` | `fitmodel` | `fitConstrainedModel` | `displaypoints` | `formatpoints` | `score` | `setmodel` | `validatemodel` | `probdefault` | `table`

Apps

Binning Explorer

Topics

“Case Study for Credit Scorecard Analysis” on page 8-71
“Credit Scorecards with Constrained Logistic Regression Coefficients” on page 8-91
“Credit Scorecard Modeling with Missing Values” on page 8-57
“Credit Scoring Using Logistic Regression and Decision Trees” (Risk Management Toolbox)
“Use Reject Inference Techniques with Credit Scorecards” (Risk Management Toolbox)
“compactCreditScorecard Object Workflow” (Risk Management Toolbox)
“Troubleshooting Credit Scorecard Results” on page 8-64
“Bin Data to Create Credit Scorecards Using Binning Explorer” (Risk Management Toolbox)
“Explore Fairness Metrics for Credit Scoring Model” (Risk Management Toolbox)
“Bias Mitigation in Credit Scoring by Reweighting” (Risk Management Toolbox)
“Bias Mitigation in Credit Scoring by Disparate Impact Removal” (Risk Management Toolbox)
“Interpretability and Explainability for Credit Scoring” (Risk Management Toolbox)
“Credit Scorecard Modeling Workflow” on page 8-52
“About Credit Scorecards” on page 8-49
“Credit Scorecard Modeling Using Observation Weights” on page 8-55
“Overview of Binning Explorer” (Risk Management Toolbox)

External Websites

Credit Risk Modeling with MATLAB (53 min 10 sec)

addBusinessCalendar

Add business calendar awareness to timetables

Syntax

```
TT = addBusinessCalendar(TT)
TT = addBusinessCalendar( ___,Name=Value)
```

Description

`TT = addBusinessCalendar(TT)` adds business calendar awareness to an input timetable `TT` by setting a custom property for the output timetable `TT`.

`TT = addBusinessCalendar( ___,Name=Value)` sets additional options specified by one or more name-value arguments, using any of the input arguments in the previous syntax. For example, `TT = addBusinessCalendar(TT,Holidays=H)` replaces the default holidays stored in `Data_NYSE_Closures.mat` with the list of holidays `H`.

Examples

Add Business Calendar for Calculating Period-Over-Period Rolling Returns

This example shows how to add a business calendar when you calculate period-over-period (PoP) rolling differences for 5 years of simulated daily prices. For each date in timetable `TT`, the difference represents the PoP difference of the corresponding price compared to the price one period earlier.

Use `holidays` to indicate the holidays in `holidays.m` for the simulation period.

```
H = holidays(datetime(2014,1,1),datetime(2018,12,31));
```

Simulate daily prices for three assets.

```
t = (datetime(2014,1,1):caldays:datetime(2018,12,31))';
rng(200,"twister")
Price = 100 + 0.1*(0:numel(t) - 1)'.*cumsum(randn(numel(t),1)/100);
Price = round(Price*100)/100;
Price2 = round(Price*94)/100;
Price3 = round(Price*88)/100;
```

```
TT = timetable(Price,Price2,Price3,RowTimes=t);
head(TT,15)
```

Time	Price	Price2	Price3
01-Jan-2014	100	94	88
02-Jan-2014	100	94	88
03-Jan-2014	100	94	88
04-Jan-2014	100	94	88
05-Jan-2014	100.01	94.01	88.01
06-Jan-2014	100.01	94.01	88.01

07-Jan-2014	100.02	94.02	88.02
08-Jan-2014	100.02	94.02	88.02
09-Jan-2014	100.04	94.04	88.04
10-Jan-2014	100.06	94.06	88.05
11-Jan-2014	100.08	94.08	88.07
12-Jan-2014	100.11	94.1	88.1
13-Jan-2014	100.11	94.1	88.1
14-Jan-2014	100.12	94.11	88.11
15-Jan-2014	100.12	94.11	88.11

Use `addBusinessCalendar` to indicate the holidays for the simulation period in the timetable `TT`.

```
TT = addBusinessCalendar(TT,Holidays=H);
```

Use `rollingreturns` with the 'Method' name-value pair argument set to "difference" to compute the differences, that is, the period-over-period changes.

```
deltas = rollingreturns(TT,Period=calweeks(1),Method="difference");
head(deltas,15)
```

Time	Price_Difference_1w	Price2_Difference_1w	Price3_Difference_1w
01-Jan-2014	NaN	NaN	NaN
02-Jan-2014	NaN	NaN	NaN
03-Jan-2014	NaN	NaN	NaN
04-Jan-2014	NaN	NaN	NaN
05-Jan-2014	NaN	NaN	NaN
06-Jan-2014	NaN	NaN	NaN
07-Jan-2014	NaN	NaN	NaN
08-Jan-2014	NaN	NaN	NaN
09-Jan-2014	0.04	0.04	0.04
10-Jan-2014	0.06	0.06	0.05
11-Jan-2014	0.08	0.08	0.07
12-Jan-2014	0.11	0.1	0.1
13-Jan-2014	0.1	0.09	0.09
14-Jan-2014	0.1	0.09	0.09
15-Jan-2014	0.1	0.09	0.09

Input Arguments

TT — Input timetable to update with business calendar awareness

timetable

Input timetable to update with business calendar awareness, specified as a timetable.

Data Types: timetable

Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `TT = addBusinessCalendar(TT,Holidays=H)` replaces the default holidays stored in `Data_NYSE_Closures.mat` with the list of holidays `H`

Holidays — Alternate holidays and market closure dates

datetime vector

Alternate holidays and market closure dates, specified as a datetime vector. The dates in `Holidays` must be whole dates without `HH:MM:SS` components. No business is conducted on the dates in `Holidays`.

By default, `Holidays` is the New York Stock Exchange (NYSE) holidays and market closure dates. For more details, load the default holidays in `Data_NYSE_Closures.mat` and inspect the `NYSE` variable, or, if you have a Financial Toolbox license, see `holidays` and `isbusday`.

Tip If you have a Financial Toolbox license, you can generate alternate holiday schedules by using the `createholidays` function and performing this procedure:

- 1 Generate a new `holidays` function using `createholidays`.
 - 2 Call the new `holidays` function to get the list of holidays.
 - 3 Specify the alternate holidays to `addBusinessCalendar` by using the `Holidays` name-value argument.
-

Data Types: `datetime`

Weekends — Alternate weekend days on which no business is conducted

[1 0 0 0 0 0 1] or ["Sunday" "Saturday"] (default) | logical vector | string vector

Alternate weekend days on which no business is conducted, specified as a length 7 logical vector or a string vector.

For a logical vector, `true` (1) entries indicate a weekend day and `false` (0) entries indicate a weekday, where entries correspond to Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, and Saturday.

Example: `Weekends=[1 0 0 0 0 1 1]` specifies that business is not conducted on Fridays through Sundays.

For a string vector, entries explicitly list the weekend days.

Example: `Weekends=["Friday" "Saturday" "Sunday"]`

Tip If business is conducted seven days per week, set `Weekends` to [0 0 0 0 0 0 0].

Data Types: `logical`

Output Arguments

TT — Updated timetable TT with added business calendar awareness by a custom property
timetable

Updated timetable `TT` with added business calendar awareness by the custom property `BusinessCalendar`, returned as a `timetable`.

The custom property `BusinessCalendar` contains a data structure that contains a field `IsBusinessDay` that stores a callable function (`F`). The function `F` accepts a datetime matrix (`D`) and returns a logical indicator matrix (`I`) of the same size: `I = F(D)`. `true (1)` elements of `I` indicate that the corresponding element of `D` occurs on a business day; `false (0)` elements of `I` indicate otherwise.

Access the callable function `F` by using `F = TT.Properties.CustomProperties.BusinessCalendar.IsBusinessDay`.

Version History

Introduced in R2020b

See Also

`isbusday` | `rollingreturns` | `timetable`

rollingreturns

Period-over-period rolling returns or differences from prices

Syntax

```
returns = rollingreturns(TT)
returns = rollingreturns( ___,Name,Value)
```

Description

`returns = rollingreturns(TT)` calculates period-over-period (PoP) returns or differences from corresponding prices. For each date in timetable `TT`, the return represents the PoP return of the corresponding price compared to the price one `Period` earlier.

`returns = rollingreturns( ___,Name,Value)` specifies options using one or more optional name-value pair arguments in addition to the input arguments in the previous syntax.

Examples

Year-over-Year Returns from Daily Prices with Business Calendar Awareness

This example shows how to compute year-over-year rolling returns for five years of simulated daily prices and also includes business calendar awareness.

Simulate five years of daily prices and store the result in the timetable `TT`. Then, use `isbusday` to retain only data for New York Stock Exchange (NYSE) business dates.

```
rng(200,'twister')
time = (datetime(2014,1,1):caldays:datetime(2018,12,31))';
price = 100 + 0.1*(0:numel(time) - 1)'.*cumsum(randn(numel(time),1)/100);
price = round(price*100)/100; % Round prices to the nearest penny
TT = timetable(price,'RowTimes',time,'VariableNames',{'Prices'});
TT = TT(isbusday(TT.Properties.RowTimes),:); % Retain only NYSE business days
head(TT,10)
```

Time	Prices
02-Jan-2014	100
03-Jan-2014	100
06-Jan-2014	100.01
07-Jan-2014	100.02
08-Jan-2014	100.02
09-Jan-2014	100.04
10-Jan-2014	100.06
13-Jan-2014	100.11
14-Jan-2014	100.12
15-Jan-2014	100.12

Use `addBusinessCalendar` to add NYSE business calendar awareness. The business calendar logic determines if the date of the previous period is a business date, and if it is not, then the most recent

business day preceding that date is found. For example, since 21-May-2016 is a Saturday and 22-May-2016 is a Sunday, year-over-year prices for Monday 22-May-2017 are compared to Friday 20-May-2016.

```
TT = addBusinessCalendar(TT); % Add NYSE business calendar
```

Compute the year-over-year returns and display the last few prices and corresponding returns.

```
returns = rollingreturns(TT, 'Period', calyears);
tail([TT returns])
```

Time	Prices	Prices_Return_1y
19-Dec-2018	212.68	0.16941
20-Dec-2018	215.54	0.19024
21-Dec-2018	217.66	0.18648
24-Dec-2018	221.42	0.20882
26-Dec-2018	224.81	0.21473
27-Dec-2018	222.17	0.19897
28-Dec-2018	224.63	0.19142
31-Dec-2018	224.37	0.19206

Year-over-Year Returns from Monthly Prices with End-of-Month Dates

Economic data is often reported on the last day of each month or quarter. So end-of-month ambiguities can arise when computing period-over-period returns for periods that exceed the periodicity at which data is reported.

Simulate five years of daily prices and store the result in the timetable TT.

```
rng(200, 'twister')
time = (datetime(2014,1,1):caldays:datetime(2018,12,31))';
price = 100 + 0.1*(0:numel(time) - 1)'.*cumsum(randn(numel(time),1)/100);
price = round(price*100)/100; % Round prices to the nearest penny
TT = timetable(price, 'RowTimes', time, 'VariableNames', {'Prices'});
head(TT,10)
```

Time	Prices
01-Jan-2014	100
02-Jan-2014	100
03-Jan-2014	100
04-Jan-2014	100
05-Jan-2014	100.01
06-Jan-2014	100.01
07-Jan-2014	100.02
08-Jan-2014	100.02
09-Jan-2014	100.04
10-Jan-2014	100.06

Create a new timetable by sampling TT on the last day of each month to mimic monthly reporting.


```
monthEndDates = dateshift(TT.Time(1):calmonths:TT.Time(end), 'end', 'month');
TT = TT(monthEndDates,:); % Sample TT at end-of-month dates
head(TT,10)
```

Time	Prices
31-Jan-2014	100.47
28-Feb-2014	100.93
31-Mar-2014	102
30-Apr-2014	102.28
31-May-2014	103.22
30-Jun-2014	103.92
31-Jul-2014	102.2
31-Aug-2014	104.79
30-Sep-2014	103.11
31-Oct-2014	105.29

Display a subset of the dates and compare a direct calculation of the dates in previous months to those shifted to the end of the month in which the previous period occurs.

```
dates = timerange(datetime(2016,2,29),datetime(2017,2,28), 'month');
[TT.Time(dates) (TT.Time(dates) - calyears) dateshift(TT.Time(dates) - calyears, 'end', 'month')]
```

```
ans = 13x3 datetime
29-Feb-2016 28-Feb-2015 28-Feb-2015
31-Mar-2016 31-Mar-2015 31-Mar-2015
30-Apr-2016 30-Apr-2015 30-Apr-2015
31-May-2016 31-May-2015 31-May-2015
30-Jun-2016 30-Jun-2015 30-Jun-2015
31-Jul-2016 31-Jul-2015 31-Jul-2015
31-Aug-2016 31-Aug-2015 31-Aug-2015
30-Sep-2016 30-Sep-2015 30-Sep-2015
31-Oct-2016 31-Oct-2015 31-Oct-2015
30-Nov-2016 30-Nov-2015 30-Nov-2015
31-Dec-2016 31-Dec-2015 31-Dec-2015
31-Jan-2017 31-Jan-2016 31-Jan-2016
28-Feb-2017 28-Feb-2016 29-Feb-2016
```

Examine these results and notice that the dates in the second and third columns of the last row differ. Specifically, when the current date in the first column is 28-Feb-2017 the dates in the second and third columns differ because 2016 is a leap year. More generally, the dates differ whenever the month of the previous period has more days than the current month for which returns are computed. In this example, end-of-months dates present the following ambiguity. When the current date of interest is 28-Feb-2017, should subtracting one calendar year produce 28-Feb-2016 or 29-Feb-2016?

The correct answer depends on the application, and both approaches are valid use cases. This problem is exacerbated, for example, when working with end-of-monthly price data and computing month-over-month returns. To address the end-of-month ambiguity, the `rollingreturns` function supports an `EndOfMonth` flag.

```
returns = rollingreturns(TT, 'Period', calyears, 'EndOfMonth', true);
```

The `EndOfMonth` flag ensures that the `rollingreturns` function uses the correct end-of-month date of each calendar month. In this example, the return on 28-Feb-2017 is correctly computed from the price reported 29-Feb-2016 rather than 28-Feb-2016.

```
[TT(dates,:) returns(dates,:)]
```

```
ans=13x2 timetable
```

Time	Prices	Prices_Return_1y
29-Feb-2016	135.59	0.21671
31-Mar-2016	138.47	0.25052
30-Apr-2016	131.44	0.11598
31-May-2016	129.34	0.083068
30-Jun-2016	133.86	0.077865
31-Jul-2016	132.78	0.046253
31-Aug-2016	140.32	0.11871
30-Sep-2016	136.52	0.087549
31-Oct-2016	141.27	0.10652
30-Nov-2016	140.76	0.1053
31-Dec-2016	135.96	0.057643
31-Jan-2017	129.52	0.0099025
28-Feb-2017	136.36	0.0056789

Input Arguments

TT — Input timetable of prices

timetable

Input timetable of prices, specified as a `timetable`. The timetable TT must satisfy the following conditions:

- All observations in TT must be associated with whole dates specified as datetimes with no HH:MM:SS time component (no time-of-day component).
- TT dates must be sorted in ascending order.
- TT must have no duplicate dates.
- Each variable in TT must contain either a single numeric vector or a numeric matrix of prices. For example, suppose TT contains three variables of daily prices.

Time	Price1	Price2	Prices	
24-Dec-2018	221.42	442.84	221.42	442.84
25-Dec-2018	220.62	441.24	220.62	441.24
26-Dec-2018	224.81	449.62	224.81	449.62
27-Dec-2018	222.17	444.34	222.17	444.34
28-Dec-2018	224.63	449.26	224.63	449.26
29-Dec-2018	225.36	450.72	225.36	450.72
30-Dec-2018	226.73	453.46	226.73	453.46
31-Dec-2018	224.37	448.74	224.37	448.74

The corresponding daily returns are formatted as three returns for the three price variables.

Time	Price1_Return	Price2_Return	Prices_Return	
24-Dec-2018	NaN	NaN	NaN	NaN
25-Dec-2018	-0.003613	-0.003613	-0.003613	-0.003613
26-Dec-2018	0.018992	0.018992	0.018992	0.018992
27-Dec-2018	-0.011743	-0.011743	-0.011743	-0.011743

28-Dec-2018	0.011073	0.011073	0.011073	0.011073
29-Dec-2018	0.003249	0.003249	0.003249	0.003249
30-Dec-2018	0.006079	0.006079	0.006079	0.006079
31-Dec-2018	-0.010409	-0.010409	-0.010409	-0.010409

Note To include business-calendar-awareness and account for nonbusiness days (for example, weekends, holidays, market closures), you must first use the `addBusinessCalendar` function to populate a custom property for the input `TT`. For example, to add business calendar logic to include only NYSE business days, you can use `TT = addBusinessCalendar(TT)`.

Data Types: `timetable`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `returns = rollingreturns(TT, 'Period', calweeks(1), 'EndOfMonth', true, 'Method', 'continuous')`

Period — Period to compute period-over-period returns

`TT.Properties.TimeStep` (default) | scalar calendar duration

Period to compute period-over-period returns, specified as the comma-separated pair consisting of `'EndOfMonthRule'` and a scalar calendar duration (for example, `caldays`, `calweeks`, or `calmonths`).

The default is the time step defined in `TT` (`TT.Properties.TimeStep`), but only when `TT.Properties.TimeStep` is not `NaN`. If `TT.Properties.TimeStep` is `NaN`, then `Period` is required.

Data Types: `double`

EndOfMonth — End-of-month flag indicates whether prices at current date are compared to last date of month for previous Period

`false` (default) | value of `true` or `false`

End-of-month flag indicates whether the prices at the current date are compared to last date of month for the previous `Period`, specified as the comma-separated pair consisting of `'EndOfMonth'` and a scalar logical value of `true` or `false`.

- If you set `EndOfMonth` to `true` (logical 1), meaning that the current prices are compared to end-of-month prices of the previous `Period`.
- If you set `EndOfMonth` to `false` (logical 0), meaning that the current prices are compared to prices recorded on the actual date of the previous `Period`.

Note The `EndOfMonth` flag is intended to address end-of-month date calculations when computing the dates of a previous `Period` one or more months in the past.

For example, suppose you have monthly prices reported at the end of each month and want to compute year-over-year returns (that is, `Period = calyears(1)`). When the current date of interest is 28-Feb-2017, setting `EndOfMonth = true` (logical 1) ensures that returns computed for 28-Feb-2017 compare the prices on 28-Feb-2017 to those on 29-Feb-2016 rather than 28-Feb-2016.

Similarly, suppose you have monthly prices reported at the end of each month and want to compute month-over-month returns (that is, `Period = calmonths(1)`). When the current date of interest is 30-Apr-2020, setting `EndOfMonth = true` (logical 1) ensures that returns computed for 30-Apr-2020 compare the prices on 30-Apr-2020 to those on 31-Mar-2020 rather than 30-Mar-2020.

Data Types: `logical`

Method — Method for computing returns from prices

'simple' (default) | character vector with value 'simple', 'continuous', or 'difference'

Method for computing returns from prices, specified as the comma-separated pair consisting of 'Method' and a scalar character vector.

- 'simple' — Compute simple (proportional) returns: $R(t) = P(t)/P(t\text{-period}) - 1$.
- 'continuous' — Compute continuous (logarithmic) returns: $R(t) = \log(P(t)/P(t\text{-period}))$.
- 'difference' — Compute differences (period-over-period changes): $R(t) = P(t) - P(t\text{-period})$

Data Types: `char`

Output Arguments

returns — Period-over-period decimal returns or differences

`timetable`

Period-over-period decimal returns or differences, returned as a timetable of the same size and format as the input argument `TT`. The returns or differences in row t are associated with the t th date in `TT` and represent the return or difference of the t th price $P(t)$ relative to the price in the previous period $P(t\text{-period})$. If the date in the previous period is not found in `TT`, then the result is `NaN` to indicate a missing value.

Variable names in the output append `_Return` or `_Difference` to the variable names in `TT` for returns and differences, respectively, followed by the period used in the period-over-period results. For example, if `TT` has a variable named `ABC` and week-over-week returns are computed for a `Period` of `calweeks(1)`, the corresponding output variable is named `ABC_Returns_1w`.

`rollingreturns` is an aggregation function in which the frequency at which prices are recorded must equal or exceed that at which returns or differences are computed. For example, daily prices can be used to compute daily, weekly, or monthly returns, but computing daily returns from weekly or monthly prices generally makes no sense.

Algorithms

Period-over-period results are computed for every date in `TT` as follows:

- 1** For each date t in TT , the date t -period is computed.
 - If date t -period is a business date, then this date is the date "one period ago"
 - If date t -period is not a business date, then each calendar day preceding t -period is examined repeatedly until a business date is found, at which point this date is the date "one period ago," or the preceding date occurs prior to the first date in TT , at which point no previous business date exists in TT .
- 2** If the date "one period ago" is found in TT , then the corresponding price $P(t\text{-period})$ is recorded and the return $R(t)$ is computed. However, if the date "one period ago" is not found in TT , then the previous price $P(t\text{-period})$ is assumed missing (that is, an implicit **NaN**), and the return $R(t) = \text{NaN}$.
- 3** The previous steps are repeated until the date t -period precedes the first date found in TT , at which point the algorithm terminates.

Version History

Introduced in **R2020b**

See Also

`addBusinessCalendar` | `adjustedClosingPrices` | `periodicreturns` | `timetable`

simByMilstein

Simulate diagonal diffusion for BM, GBM, CEV, HWV, SDEDDO, SDELD, or SDEMRD sample paths by Milstein approximation

Syntax

```
[Paths,Times,Z] = simByMilstein(MDL,NPeriods)
[Paths,Times,Z] = simByMilstein( __,Name=Value)
```

Description

`[Paths,Times,Z] = simByMilstein(MDL,NPeriods)` simulates `NTrials` sample paths of `NVars` state variables driven by the BM, GBM, CEV, HWV, SDEDDO, SDELD, or SDEMRD process sources of risk over `NPeriods` consecutive observation periods, approximating continuous-time by the Milstein approximation.

`simByMilstein` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`[Paths,Times,Z] = simByMilstein( __,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein Scheme Using GBM Model

This example shows how to use `simByMilstein` with a GBM model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead of pseudo random numbers.

Create a `univariategbm` object to represent the model: $dX_t = 0.25X_t dt + 0.3X_t dW_t$.

```
GBM_obj = gbm(0.25, 0.3) % (B = Return, Sigma)

GBM_obj =
  Class GBM: Generalized Geometric Brownian Motion
  -----
  Dimensions: State = 1, Brownian = 1
  -----
  StartTime: 0
  StartState: 1
  Correlation: 1
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
```

```
Simulation: simulation method/function simByEuler
Return: 0.25
Sigma: 0.3
```

gbm objects display the parameter B as the more familiar Return.

Perform a quasi-Monte Carlo simulation by using `simByMilstein` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z] = simByMilstein(GBM_obj,10,ntrials=4096,montecarlomethod="quasi",quasisequence="s
```

Input Arguments

MDL — Stochastic differential equation model

BM object | GBM object | CEV object | HWV object | SDEDDO object | SDELD object | SDEMRO object

Stochastic differential equation model, specified as a BM, GBM, CEV, HWV, SDEDDO, SDELD, or SDEMRO object. You can use the following to create a MDL object:

- `bm`
- `gbm`
- `cev`
- `hvw`
- `sdeddo`
- `sdelld`
- `sdemrd`

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

```
Example: [Paths,Times,Z] =
simByMilstein(GBM_obj,NPeriods,DeltaTime=dt,Method="reflection")
```

Method — Method to handle negative values

'basic' (default) | character vector with values 'basic', 'absorption', 'reflection', 'partial-truncation', 'full-truncation', or 'higham-mao' | string with values "basic", "absorption", "reflection", "partial-truncation", "full-truncation", or "higham-mao"

Method to handle negative values, specified as `Method` and a character vector or string with a supported value.

Data Types: `char` | `string`

NTrials — Simulated trials (sample paths) of NPeriods observations each

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: `double`

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of `'DeltaTime'` and a scalar or a `NPeriods`-by-1 column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps within each time increment dt (specified as DeltaTime)

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilestonein` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByMilestonein` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

Antithetic — Flag to indicate whether simByMilestonein uses antithetic sampling to generate Gaussian random variates

`False` (no antithetic sampling) (default) | logical with values `True` or `False`

Flag that indicates whether `simByMilestonein` uses antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes). This argument is specified as `Antithetic` and a scalar logical flag with a value of `True` or `False`.

When you specify `True`, `simByMilestonein` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1, 3, 5, ...) correspond to the primary Gaussian paths.
- Even trials (2, 4, 6, ...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilestonein` ignores the value of `Antithetic`.

Data Types: `logical`

Z — Direct specification of the dependent random noise process used to generate Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the `SDE` object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process used to generate the Brownian motion vector (Wiener process) that drives the simulation. This argument is specified as `Z` and a function or as an `(NPeriods × NSteps)-by-NBrowns-by-NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns-by-1` column vector, and you must call it with two inputs:

- A real-valued scalar observation time t .
 - An `NVars-by-1` state vector X_t .
-

Data Types: `double` | `function`

StorePaths — Flag that indicates how the output array `Paths` is stored and returned

`True` (default) | logical with values `True` or `False`

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar logical flag with a value of `True` or `False`.

- If `StorePaths` is `True` (the default value) or is unspecified, `simByMilstein` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is `False` (logical 0), `simByMilstein` returns the `Paths` output array as an empty matrix.

Data Types: `logical`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see `Z`), `simByMilstein` ignores the value of `MonteCarloMethod`.

Data Types: `string` | `char`

QuasiSequence — Low discrepancy sequence to drive stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as **QuasiSequence** and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If **MonteCarloMethod** option is not specified or specified as "standard", **QuasiSequence** is ignored.

If you specify an input noise process (see **Z**), **simByMilstein** ignores the value of **QuasiSequence**.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as **BrownianMotionMethod** and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the **Z** input argument, **BrownianMotionMethod** is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the **MonteCarloMethod** using pseudo random numbers. However, the performance differs between the two when the **MonteCarloMethod** option "quasi" is introduced, with faster convergence seen for "brownian-bridge" construction option and the fastest convergence when using the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

simByEuler makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments of the form, specified as **Processes** and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByMilstein` function runs processing functions at each interpolation time. They must accept the current interpolation time t , and the current state vector X_t , and return a state vector that may be an adjustment to the input state.

If you specify more than one processing function, `simByMilstein` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When the input flag `StorePaths = False`, `simByEuler` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates used to generate Brownian motion vector

array

Dependent random variates used to generate the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time series array.

More About

Milstein Method

The Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The Milstein method adds a correction term to the Euler-Maruyama method that takes into account the second-order derivative of the SDE. This correction term improves the accuracy of the approximation, especially for SDEs with non-linearities.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another of the same expected value, but smaller variance. In a typical Monte Carlo simulation, each sample path is

independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of any other pair, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

This function simulates any vector-valued SDE of the form

$$dX_t = F(t, X_t)dt + G(t, X_t)dW_t$$

where:

- X is an $NVars$ -by-1 state vector of process variables (for example, short rates or equity prices) to simulate.
- W is an $NBrowns$ -by-1 Brownian motion vector.
- F is an $NVars$ -by-1 vector-valued drift-rate function.
- G is an $NVars$ -by- $NBrowns$ matrix-valued diffusion-rate function.

`simByEuler` simulates `NTrials` sample paths of `NVars` correlated state variables driven by `NBrowns` Brownian motion sources of risk over `NPeriods` consecutive observation periods, using the Euler approach to approximate continuous-time stochastic processes.

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Version History

Introduced in R2023a

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

`simulate` | `simByMilstein2`

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

"Performance Considerations" on page 14-65

simByMilstein

Simulate CIR process sample paths by Milstein approximation

Syntax

```
[Paths,Times,Z] = simByMilstein(MDL,NPeriods)
[Paths,Times,Z] = simByMilstein( ___,Name=Value)
```

Description

`[Paths,Times,Z] = simByMilstein(MDL,NPeriods)` simulates `NTrials` sample paths of `NVARS` state variables driven by the CIR process sources of risk over `NPeriods` consecutive observation periods, approximating continuous-time Cox-Ingersoll-Ross (CIR) by the Milstein approximation.

`simByMilstein` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`[Paths,Times,Z] = simByMilstein( ___,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with simByMilstein Scheme Using CIR Model

This example shows how to use `simByMilstein` with a CIR model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead of pseudo random numbers.

The Cox-Ingersoll-Ross (CIR) short rate class derives directly from SDE with mean-reverting drift

$$(SDEM\text{RD}): dX_t = S(t)[L(t) - X_t]dt + D(t, X_t^{\frac{1}{2}})V(t)dW$$

where D is a diagonal matrix whose elements are the square root of the corresponding element of the state vector.

Create a `cir` object to represent the model: $dX_t = 0.2(0.1 - X_t)dt + 0.05X_t^{\frac{1}{2}}dW$.

```
cir_obj = cir(0.2, 0.1, 0.05) % (Speed, Level, Sigma)
```

```
cir_obj =  
    Class CIR: Cox-Ingersoll-Ross
```

```

-----
Dimensions: State = 1, Brownian = 1
-----
StartTime: 0
StartState: 1
Correlation: 1
    Drift: drift rate function F(t,X(t))
    Diffusion: diffusion rate function G(t,X(t))
Simulation: simulation method/function simByEuler
    Sigma: 0.05
    Level: 0.1
    Speed: 0.2

```

Define the quasi-Monte Carlo simulation using the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z] = simByMilstein(cir_obj,10,ntrials=4096,method="basic",montecarlomethod="quasi",q
```

Input Arguments

MDL — Stochastic differential equation model

CIR object

Stochastic differential equation model, specified as a `cir` object. You can create a `bates` object using `cir`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1,...,NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z] = simByMilstein(CIR_obj,NPeriods,DeltaTime=dt,Method="reflection")`

Method — Method to handle negative values

'basic' (default) | character vector with values 'basic', 'absorption', 'reflection', 'partialtruncation', 'fulltruncation', or 'higham-mao' | string with values "basic", "absorption", "reflection", "partialtruncation", "fulltruncation", or "higham-mao"

Method to handle negative values, specified as `Method` and a character vector or string with a supported value.

Note The `Method` argument is only supported when using a CIR object. For more information on creating a CIR object, see `cir`.

Data Types: `char` | `string`

NTrials — Simulated trials (sample paths) of NPeriods observations each

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: `double`

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of `'DeltaTime'` and a scalar or a `NPeriods`-by-1 column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps within each time increment dt (specified as `DeltaTime`)

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilestonein` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps` - 1 intermediate points. Although `simByMilestonein` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

Antithetic — Flag to indicate whether `simByMilestonein` uses antithetic sampling to generate Gaussian random variates

`False` (no antithetic sampling) (default) | logical with values `True` or `False`

Flag that indicates whether `simByMilestonein` uses antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes). This argument is specified as `Antithetic` and a scalar logical flag with a value of `True` or `False`.

When you specify `True`, `simByMilestonein` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1,3,5,...) correspond to the primary Gaussian paths.
- Even trials (2,4,6,...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilstein` ignores the value of `Antithetic`.

Data Types: `logical`

Z — Direct specification of the dependent random noise process used to generate Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the `SDE` object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process used to generate the Brownian motion vector (Wiener process) that drives the simulation. This argument is specified as `Z` and a function or as an `(NPeriods × NSteps)-by-NBrowns-by-NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns-by-1` column vector, and you must call it with two inputs:

- A real-valued scalar observation time t .
 - An `NVars-by-1` state vector X_t .
-

Data Types: `double` | `function`

StorePaths — Flag that indicates how the output array `Paths` is stored and returned

`True` (default) | logical with values `True` or `False`

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar logical flag with a value of `True` or `False`.

- If `StorePaths` is `True` (the default value) or is unspecified, `simByMilstein` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is `False` (logical 0), `simByMilstein` returns the `Paths` output array as an empty matrix.

Data Types: `logical`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see `Z`), `simByMilstein` ignores the value of `MonteCarloMethod`.

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive the stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as `QuasiSequence` and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If `MonteCarloMethod` option is not specified or specified as "standard", `QuasiSequence` is ignored.

If you specify an input noise process (see `Z`), `simByMilstein` ignores the value of `QuasiSequence`.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as `BrownianMotionMethod` and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the `Z` input argument, `BrownianMotionMethod` is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the `MonteCarloMethod` using pseudo random numbers. However, the performance differs between the two when the `MonteCarloMethod` option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

`simByEuler` makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments of the form, specified as **Processes** and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The **simByMilstein** function runs processing functions at each interpolation time. They must accept the current interpolation time t , and the current state vector X_t , and return a state vector that may be an adjustment to the input state.

If you specify more than one processing function, **simByMilstein** invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of **Paths** is the transpose of the state vector X_t at time t . When the input flag **StorePaths** = **False**, **simByEuler** returns **Paths** as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of **Times** is associated with the corresponding row of **Paths**.

Z — Dependent random variates used to generate Brownian motion vector

array

Dependent random variates used to generate the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time series array.

More About

Milstein Method

The Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The Milstein method adds a correction term to the Euler-Maruyama method that takes into account the second-order derivative of the SDE. This correction term improves the accuracy of the approximation, especially for SDEs with non-linearities.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another of the same expected value, but smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of any other pair, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

This function simulates any vector-valued SDE of the form

$$dX_t = F(t, X_t)dt + G(t, X_t)dW_t$$

where:

- X is an $NVars$ -by-1 state vector of process variables (for example, short rates or equity prices) to simulate.
- W is an $NBrowns$ -by-1 Brownian motion vector.
- F is an $NVars$ -by-1 vector-valued drift-rate function.
- G is an $NVars$ -by- $NBrowns$ matrix-valued diffusion-rate function.

`simByEuler` simulates $NTrials$ sample paths of $NVars$ correlated state variables driven by $NBrowns$ Brownian motion sources of risk over $NPeriods$ consecutive observation periods, using the Euler approach to approximate continuous-time stochastic processes.

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Version History

Introduced in R2023a

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

`simulate` | `cir` | `simByMilstein2`

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

"Performance Considerations" on page 14-65

simByMilstein

Simulate Heston process sample paths by Milstein approximation

Syntax

```
[Paths,Times,Z] = simByMilstein(MDL,NPeriods)
[Paths,Times,Z] = simByMilstein( ___,Name=Value)
```

Description

`[Paths,Times,Z] = simByMilstein(MDL,NPeriods)` simulates `NTrials` sample paths of Heston bivariate models driven by two `NBrowns` Brownian motion sources of risk approximating continuous-time stochastic processes by the Milstein approximation.

`simByMilstein` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`[Paths,Times,Z] = simByMilstein( ___,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein Scheme Using Heston Model

This example shows how to use `simByMilstein` with a Heston model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead pseudo random numbers.

Define the parameters for the heston object.

```
Return = 0.03;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;

AssetPrice = 80;
V0 = 0.04;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a heston object.

```
Heston = heston(Return,Speed,Level,Volatility,startstate=StartState,correlation=Correlation)
```

```
Heston =
  Class HESTON: Heston Bivariate Stochastic Volatility
  -----
  Dimensions: State = 2, Brownian = 2
  -----
  StartTime: 0
  StartState: 2x1 double array
  Correlation: 2x2 double array
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Return: 0.03
  Speed: 1
  Level: 0.05
  Volatility: 0.2
```

Perform a quasi-Monte Carlo simulation by using `simByMilstein` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time] = simByMilstein(Heston,10,ntrials=4096,MonteCarloMethod="quasi",QuasiSequence="sobo")
```

Input Arguments

MDL — Stochastic differential equation model

Heston object

Stochastic differential equation model, specified as a `heston` object. You can create a `heston` object using `heston`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z] = simByMilstein(Heston_obj,NPeriods,NTrials=10,DeltaTime=dt)`

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as `DeltaTime` and a scalar or an `NPeriods-by-1` column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: double

NSteps — Number of intermediate time steps within each time increment dt

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilstein` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByMilstein` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: double

Antithetic — Flag to use antithetic sampling to generate Gaussian random variates

false (no antithetic sampling) (default) | logical with values true or false

Flag to use antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes), specified as `Antithetic` and a scalar numeric or logical 1 (true) or 0 (false).

When you specify true, `simByEuler` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1, 3, 5, ...) correspond to the primary Gaussian paths.
- Even trials (2, 4, 6, ...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilstein` ignores the value of `Antithetic`.

Data Types: logical

Z — Direct specification of the dependent random noise process for generating Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the SDE object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process for generating the Brownian motion vector (Wiener process) that drives the simulation, specified as `Z` and a function or as an (`NPeriods` $\times$ `NSteps`)-by-`NBrowns`-by-`NTrials` three-dimensional array of dependent random variates.

Note If you specify Z as a function, it must return an N Browns-by-1 column vector, and you must call it with two inputs:

- A real-valued scalar observation time t
 - An N Vars-by-1 state vector X_t
-

Data Types: double | function

StorePaths — Flag that indicates how Paths is stored and returned

true (default) | logical with values true or false

Flag that indicates how the output array Paths is stored and returned, specified as StorePaths and a scalar numeric or logical 1 (true) or 0 (false).

- If StorePaths is true (the default value) or is unspecified, simByMilstein returns Paths as a three-dimensional time-series array.
- If StorePaths is false (logical 0), simByMilstein returns Paths as an empty matrix.

Data Types: logical

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as MonteCarloMethod and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see Z), simByMilstein ignores the value of MonteCarloMethod.

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive the stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as QuasiSequence and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If MonteCarloMethod option is not specified or specified as "standard", QuasiSequence is ignored.

If you specify an input noise process (see Z), simByMilstein ignores the value of QuasiSequence.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as `BrownianMotionMethod` and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the `Z` input argument, `BrownianMotionMethod` is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the `MonteCarloMethod` using pseudo random numbers. However, the performance differs between the two when the `MonteCarloMethod` option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

`simByMilstein` makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as `Processes` and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByMilstein` function runs processing functions at each interpolation time. The functions must accept the current interpolation time t , and the current state vector X_t and return a state vector that can be an adjustment to the input state.

If you specify more than one processing function, `simByMilstein` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

The end-of-period `Processes` argument allows you to terminate a given trial early. At the end of each time step, `simByMilstein` tests the state vector X_t for an all-`NaN` condition. Thus, to signal an early termination of a given trial, all elements of the state vector X_t must be `NaN`. This test enables you to define a `Processes` function to signal early termination of a trial, and offers significant performance benefits in some situations (for example, pricing down-and-out barrier options).

Data Types: cell | function

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When `StorePaths` is set to `false`, `simByMilstein` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates for generating Brownian motion vector

array

Dependent random variates for generating the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time-series array.

More About

Milstein Method

The Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The Milstein method adds a correction term to the Euler-Maruyama method that takes into account the second-order derivative of the SDE. This correction term improves the accuracy of the approximation, especially for SDEs with non-linearities.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another that has the same expected value but a smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of other pairs, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Version History

Introduced in R2023a

References

[1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

heston | simulate | simByMilstein2

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

"Performance Considerations" on page 14-65

simByMilstein

Simulate Bates process sample paths by Milstein approximation

Syntax

```
[Paths,Times,Z,N] = simByMilstein(MDL,NPeriods)
[Paths,Times,Z,N] = simByMilstein( ___,Name=Value)
```

Description

`[Paths,Times,Z,N] = simByMilstein(MDL,NPeriods)` simulates `NTrials` sample paths of Bates or Heston bivariate models driven by `NBrowns` Brownian motion sources of risk and `NJUMPS` compound Poisson processes representing the arrivals of important events over `NPeriods` consecutive observation periods, approximating continuous-time stochastic processes by the Milstein approximation.

`simByMilstein` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`[Paths,Times,Z,N] = simByMilstein( ___,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein Scheme Using Bates Model

This example shows how to use `simByMilstein` with a Bates model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead of pseudo random numbers.

Define the parameters for the bates object.

```
AssetPrice = 80;
Return = 0.03;
JumpMean = 0.02;
JumpVol = 0.08;
JumpFreq = 0.1;

V0 = 0.04;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;
Rho = -0.7;
```

```
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a bates object.

```
Bates = bates(Return, Speed, Level, Volatility, ...
             JumpFreq, JumpMean, JumpVol,startstate=StartState,correlation=Correlation)
```

```
Bates =
  Class BATES: Bates Bivariate Stochastic Volatility
  -----
  Dimensions: State = 2, Brownian = 2
  -----
  StartTime: 0
  StartState: 2x1 double array
  Correlation: 2x2 double array
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Return: 0.03
  Speed: 1
  Level: 0.05
  Volatility: 0.2
  JumpFreq: 0.1
  JumpMean: 0.02
  JumpVol: 0.08
```

Perform a quasi-Monte Carlo simulation by using `simByMilstein` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z,n] = simByMilstein(Bates,10,ntrials=4096,montecarlomethod="quasi",quasisequence="s
```

Input Arguments

MDL — Stochastic differential equation model

Bates object

Stochastic differential equation model, specified as a bates object. You can create a bates object using `bates`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z,N] =
simByMilstein(Bates_obj,NPeriods,NTrials=10,DeltaTime=dt)`

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as `DeltaTime` and a scalar or an `NPeriods-by-1` column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: double

NSteps — Number of intermediate time steps within each time increment dt

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilstein` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByMilstein` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: double

Antithetic — Flag to use antithetic sampling to generate Gaussian random variates

false (no antithetic sampling) (default) | logical with values true or false

Flag to use antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes), specified as `Antithetic` and a scalar numeric or logical 1 (true) or 0 (false).

When you specify true, `simByEuler` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1,3,5,...) correspond to the primary Gaussian paths.
- Even trials (2,4,6,...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilstein` ignores the value of `Antithetic`.

Data Types: logical

Z — Direct specification of the dependent random noise process for generating Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the SDE object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process for generating the Brownian motion vector (Wiener process) that drives the simulation, specified as `Z` and a function or as an `(NPeriods × NSteps)-by-NBrowns-by-NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns-by-1` column vector, and you must call it with two inputs:

- A real-valued scalar observation time t
 - An `NVars-by-1` state vector X_t
-

Data Types: double | function

N — Dependent random counting process for generating number of jumps

random numbers from Poisson distribution with parameter `JumpFreq` from a bates object (default) | three-dimensional array | function

Dependent random counting process for generating the number of jumps, specified as `N` and a function or an `(NPeriods × NSteps)-by-NJumps-by-NTrials` three-dimensional array of dependent random variates.

Note If you specify a function, `N` must return an `NJumps-by-1` column vector, and you must call it with two inputs: a real-valued scalar observation time t followed by an `NVars-by-1` state vector X_t .

Data Types: double | function

StorePaths — Flag that indicates how Paths is stored and returned

true (default) | logical with values true or false

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar numeric or logical 1 (true) or 0 (false).

- If `StorePaths` is true (the default value) or is unspecified, `simByMilstein` returns `Paths` as a three-dimensional time-series array.
- If `StorePaths` is false (logical 0), `simByMilstein` returns `Paths` as an empty matrix.

Data Types: logical

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers

- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see Z), `simByMilstein` ignores the value of `MonteCarloMethod`.

Data Types: `string` | `char`

QuasiSequence — Low discrepancy sequence to drive the stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as `QuasiSequence` and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If `MonteCarloMethod` option is not specified or specified as "standard", `QuasiSequence` is ignored.

If you specify an input noise process (see Z), `simByMilstein` ignores the value of `QuasiSequence`.

Data Types: `string` | `char`

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as `BrownianMotionMethod` and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the Z input argument, `BrownianMotionMethod` is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the `MonteCarloMethod` using pseudo random numbers. However, the performance differs between the two when the `MonteCarloMethod`

option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

`simByMilstein` makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as `Processes` and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByMilstein` function runs processing functions at each interpolation time. The functions must accept the current interpolation time t , and the current state vector X_t and return a state vector that can be an adjustment to the input state.

If you specify more than one processing function, `simByMilstein` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

The end-of-period `Processes` argument allows you to terminate a given trial early. At the end of each time step, `simByMilstein` tests the state vector X_t for an all-`NaN` condition. Thus, to signal an early termination of a given trial, all elements of the state vector X_t must be `NaN`. This test enables you to define a `Processes` function to signal early termination of a trial, and offers significant performance benefits in some situations (for example, pricing down-and-out barrier options).

Data Types: cell | function

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)\text{-by-}N\text{Vars-by-}N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When `StorePaths` is set to `false`, `simByMilstein` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)\text{-by-}1$ column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates for generating Brownian motion vector

array

Dependent random variates for generating the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})\text{-by-}N\text{Browns-by-}N\text{Trials}$ three-dimensional time-series array.

N — Dependent random variates for generating jump counting process vector

array

Dependent random variates used to generate the jump counting process vector, returned as an (NPeriods × NSteps)-by-NJumps-by-NTrials three-dimensional time series array.

More About

Milstein Method

The Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The Milstein method adds a correction term to the Euler-Maruyama method that takes into account the second-order derivative of the SDE. This correction term improves the accuracy of the approximation, especially for SDEs with non-linearities.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another that has the same expected value but a smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of other pairs, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Version History

Introduced in R2023a

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

bates | simulate | simByMilstein2

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71
"SDEs" on page 14-2
"SDE Models" on page 14-7
"SDE Class Hierarchy" on page 14-5
"Quasi-Monte Carlo Simulation" on page 14-63
"Performance Considerations" on page 14-65

simByMilstein

Simulate diagonal diffusion Merton sample paths by Milstein approximation

Syntax

```
[Paths,Times,Z,N] = simByMilstein(MDL,NPeriods)
[Paths,Times,Z,N] = simByMilstein( ___,Name=Value)
```

Description

[Paths,Times,Z,N] = simByMilstein(MDL,NPeriods) simulates NTrials sample paths of NVars correlated state variables driven by NBrowns Brownian motion sources of risk and NJumps compound Poisson processes representing the arrivals of important events over NPeriods consecutive observation periods. The simulation approximates the continuous-time Merton jump diffusion process by the Milstein approach.

[Paths,Times,Z,N] = simByMilstein(___,Name=Value) specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for MonteCarloMethod, QuasiSequence, and BrownianMotionMethod. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein Scheme Using Merton Model

This example shows how to use simByMilstein with a Merton model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead pseudo random numbers.

Define the parameters for the merton object.

```
AssetPrice = 80;
Return = 0.03;
Sigma = 0.16;
JumpMean = 0.02;
JumpVol = 0.08;
JumpFreq = 2;
```

Create a merton object.

```
mertonObj = merton(Return,Sigma,JumpFreq,JumpMean,JumpVol,StartState=AssetPrice)
```

```
mertonObj =
  Class MERTON: Merton Jump Diffusion
  -----
  Dimensions: State = 1, Brownian = 1
  -----
  StartTime: 0
```

```
StartState: 80
Correlation: 1
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
Simulation: simulation method/function simByEuler
  Sigma: 0.16
  Return: 0.03
  JumpFreq: 2
  JumpMean: 0.02
  JumpVol: 0.08
```

Perform a quasi-Monte Carlo simulation by using `simByMilstein` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,times] = simByMilstein(mertonObj,10,Ntrials=4096,MonteCarloMethod="quasi",QuasiSequence="S
```

Input Arguments

MDL — Stochastic differential equation model

Merton object

Stochastic differential equation model, specified as a merton object. You can create a merton object using `merton`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z,N] = simByMilstein(Merton_obj,NPeriods,NTrials=1000,DeltaTime=dt,NSteps=10)`

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as the comma-separated pair consisting of `'NTrials'` and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of `'DeltaTime'` and a scalar or a `NPeriods-by-1` column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps within each time increment

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as the comma-separated pair consisting of 'NSteps' and a positive scalar integer.

The `simByEuler` function partitions each time increment dt into `NSteps` subintervals of length dt/NSteps , and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByEuler` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

Antithetic — Flag to use antithetic sampling to generate the Gaussian random variates

false (no antithetic sampling) (default) | logical with values `true` or `false`

Flag to use antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes), specified as the comma-separated pair consisting of 'Antithetic' and a scalar numeric or logical 1 (`true`) or 0 (`false`).

When you specify `true`, `simByEuler` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1, 3, 5, ...) correspond to the primary Gaussian paths.
- Even trials (2, 4, 6, ...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByEuler` ignores the value of `Antithetic`.

Data Types: `logical`

Z — Direct specification of the dependent random noise process for generating Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the SDE object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process for generating the Brownian motion vector (Wiener process) that drives the simulation, specified as the comma-separated pair consisting of 'Z' and a function or as an (`NPeriods` × `NSteps`)-by-`NBrowns`-by-`NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns`-by-1 column vector, and you must call it with two inputs:

- A real-valued scalar observation time t
 - An `NVars`-by-1 state vector X_t
-

Data Types: double | function

N — Dependent random counting process for generating number of jumps

random numbers from Poisson distribution with parameter `JumpFreq` from merton object (default) | three-dimensional array | function

Dependent random counting process for generating the number of jumps, specified as the comma-separated pair consisting of 'N' and a function or an (NPeriods × NSteps) -by-NJumps-by-NTrials three-dimensional array of dependent random variates.

If you specify a function, N must return an NJumps-by-1 column vector, and you must call it with two inputs: a real-valued scalar observation time t followed by an NVars-by-1 state vector X_t .

Data Types: double | function

StorePaths — Flag that indicates how Paths is stored and returned

true (default) | logical with values true or false

Flag that indicates how the output array Paths is stored and returned, specified as the comma-separated pair consisting of 'StorePaths' and a scalar numeric or logical 1 (true) or 0 (false).

- If StorePaths is true (the default value) or is unspecified, `simByEuler` returns Paths as a three-dimensional time series array.
- If StorePaths is false (logical 0), `simByEuler` returns Paths as an empty matrix.

Data Types: logical

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as the comma-separated pair consisting of 'MonteCarloMethod' and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see Z and N), `simByEuler` ignores the value of `MonteCarloMethod`.

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive the stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as the comma-separated pair consisting of 'QuasiSequence' and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If `MonteCarloMethod` option is not specified or specified as "standard", `QuasiSequence` is ignored.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as the comma-separated pair consisting of 'BrownianMotionMethod' and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the `Z` input argument, `BrownianMotionMethod` is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the `MonteCarloMethod` using pseudo random numbers. However, the performance differs between the two when the `MonteCarloMethod` option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

`simByEuler` makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as the comma-separated pair consisting of 'Processes' and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByEuler` function runs processing functions at each interpolation time. The functions must accept the current interpolation time t , and the current state vector X_t and return a state vector that can be an adjustment to the input state.

If you specify more than one processing function, `simByEuler` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

The end-of-period `Processes` argument allows you to terminate a given trial early. At the end of each time step, `simByEuler` tests the state vector X_t for an all-NaN condition. Thus, to signal an early

termination of a given trial, all elements of the state vector X_t must be NaN. This test enables you to define a `Processes` function to signal early termination of a trial, and offers significant performance benefits in some situations (for example, pricing down-and-out barrier options).

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When `StorePaths` is set to `false`, `simByEuler` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates for generating Brownian motion vector

array

Dependent random variates used to generate the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time-series array.

N — Dependent random variates used for generating jump counting process vector

array

Dependent random variates for generating the jump counting process vector, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Jumps}$ -by- $N\text{Trials}$ three-dimensional time-series array.

More About

Milstein Method

The Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The Milstein method adds a correction term to the Euler-Maruyama method that takes into account the second-order derivative of the SDE. This correction term improves the accuracy of the approximation, especially for SDEs with non-linearities.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another that has the same expected value but a smaller variance. In a typical Monte Carlo simulation, each sample path is

independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of other pairs, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

This function simulates any vector-valued SDE of the following form:

$$dX_t = B(t, X_t)X_t dt + D(t, X_t)V(t, x_t)dW_t + Y(t, X_t, N_t)X_t dN_t$$

Here:

- X_t is an `NVars`-by-1 state vector of process variables.
- $B(t, X_t)$ is an `NVars`-by-`NVars` matrix of generalized expected instantaneous rates of return.
- $D(t, X_t)$ is an `NVars`-by-`NVars` diagonal matrix in which each element along the main diagonal is the corresponding element of the state vector.
- $V(t, X_t)$ is an `NVars`-by-`NVars` matrix of instantaneous volatility rates.
- dW_t is an `NBrowns`-by-1 Brownian motion vector.
- $Y(t, X_t, N_t)$ is an `NVars`-by-`NJumps` matrix-valued jump size function.
- dN_t is an `NJumps`-by-1 counting process vector.

`simByEuler` simulates `NTrials` sample paths of `NVars` correlated state variables driven by `NBrowns` Brownian motion sources of risk over `NPeriods` consecutive observation periods, using the Euler approach to approximate continuous-time stochastic processes.

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Version History

Introduced in R2023a

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

merton | simulate | simByMilstein2

Topics

“Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation” on page 14-71

“SDEs” on page 14-2

“SDE Models” on page 14-7

“SDE Class Hierarchy” on page 14-5

“Quasi-Monte Carlo Simulation” on page 14-63

“Performance Considerations” on page 14-65

simByMilstein2

Simulate BM, GBM, CEV, HWV, SDEDDO, SDELD, SDEMRD process sample paths by second order Milstein approximation

Syntax

```
[Paths,Times,Z] = simByMilstein2(MDL,NPeriods)
[Paths,Times,Z] = simByMilstein2( ___,Name=Value)
```

Description

`[Paths,Times,Z] = simByMilstein2(MDL,NPeriods)` simulates `NTrials` sample paths of `NVARS` state variables driven by the BM, GBM, CEV, HWV, SDEDDO, SDELD, or SDEMRD process sources of risk over `NPeriods` consecutive observation periods, approximating continuous-time by the second order Milstein approximation.

`simByMilstein2` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`simByMilstein2` is only valid for diagonal diffusion SDE models.

`[Paths,Times,Z] = simByMilstein2( ___,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein2 Scheme Using GBM Model

This example shows how to use `simByMilstein2` with a GBM model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead of pseudo random numbers.

Create a univariate `gbm` object to represent the model: $dX_t = 0.25X_t dt + 0.3X_t dW_t$.

```
GBM_obj = gbm(0.25, 0.3) % (B = Return, Sigma)
```

```
GBM_obj =
  Class GBM: Generalized Geometric Brownian Motion
  -----
  Dimensions: State = 1, Brownian = 1
  -----
  StartTime: 0
  StartState: 1
```

```
Correlation: 1
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Return: 0.25
  Sigma: 0.3
```

gbm objects display the parameter **B** as the more familiar **Return**.

Perform a quasi-Monte Carlo simulation by using `simByMilstein2` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z] = simByMilstein2(GBM_obj,10,ntrials=4096,montecarlomethod="quasi",quasisequence=";
```

Input Arguments

MDL — Stochastic differential equation model

BM object | GBM object | CEV object | HWV object | SDEDDO object | SDELD object | SDEM RD object

Stochastic differential equation model, specified as a BM, GBM, CEV, HWV, SDEDDO, SDELD, or SDEM RD object. You can use the following to create a MDL object:

- `bm`
- `gbm`
- `cev`
- `hvw`
- `sdeddo`
- `sdelld`
- `sdemrd`

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

```
Example: [Paths,Times,Z] =  
simByMilstein2(GBM_obj,NPeriods,DeltaTime=dt,Method="reflection")
```

Method — Method to handle negative values

'basic' (default) | character vector with values 'basic', 'absorption', 'reflection', 'partial-truncation', 'full-truncation', or 'higham-mao' | string with values "basic",

"absorption", "reflection", "partial-truncation", "full-truncation", or "higham-mao"

Method to handle negative values, specified as `Method` and a character vector or string with a supported value.

Data Types: `char` | `string`

NTrials — Simulated trials (sample paths) of NPeriods observations each

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: `double`

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of 'DeltaTime' and a scalar or a `NPeriods`-by-1 column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps within each time increment dt (specified as DeltaTime)

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilestein` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps` - 1 intermediate points. Although `simByMilstein2` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

Antithetic — Flag to indicate whether simByMilstein2 uses antithetic sampling to generate Gaussian random variates

False (no antithetic sampling) (default) | logical with values `True` or `False`

Flag that indicates whether `simByMilstein2` uses antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes). This argument is specified as `Antithetic` and a scalar logical flag with a value of `True` or `False`.

When you specify `True`, `simByMilstein2` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1,3,5,...) correspond to the primary Gaussian paths.
- Even trials (2,4,6,...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `Antithetic`.

Data Types: `logical`

Z — Direct specification of the dependent random noise process used to generate Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the `SDE` object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process used to generate the Brownian motion vector (Wiener process) that drives the simulation. This argument is specified as `Z` and a function or as an `(NPeriods × NSteps)-by-NBrowns-by-NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns-by-1` column vector, and you must call it with two inputs:

- A real-valued scalar observation time t .
 - An `NVars-by-1` state vector X_t .
-

Data Types: `double` | `function`

StorePaths — Flag that indicates how the output array `Paths` is stored and returned

`True` (default) | logical with values `True` or `False`

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar logical flag with a value of `True` or `False`.

- If `StorePaths` is `True` (the default value) or is unspecified, `simByMilstein2` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is `False` (logical 0), `simByMilstein2` returns the `Paths` output array as an empty matrix.

Data Types: `logical`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `MonteCarloMethod`.

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as QuasiSequence and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If MonteCarloMethod option is not specified or specified as "standard", QuasiSequence is ignored.

If you specify an input noise process (see Z), simByMilstein2 ignores the value of QuasiSequence.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as BrownianMotionMethod and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the Z input argument, BrownianMotionMethod is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the MonteCarloMethod using pseudo random numbers. However, the performance differs between the two when the MonteCarloMethod option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

simByEuler makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as `Processes` and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByMilstein2` function runs processing functions at each interpolation time. They must accept the current interpolation time t , and the current state vector X_t , and return a state vector that may be an adjustment to the input state.

If you specify more than one processing function, `simByMilstein2` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When the input flag `StorePaths = False`, `simByMilstein2` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates used to generate Brownian motion vector

array

Dependent random variates used to generate the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time series array.

More About

Second-Order Milstein Method

The second-order Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The second order Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The second-order Milstein method is more accurate than the Euler-Maruyama method and is often used to solve SDEs with strong or weak convergence rates.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another of the same expected value, but smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of any other pair, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

This function simulates any vector-valued SDE of the form

$$dX_t = F(t, X_t)dt + G(t, X_t)dW_t$$

where:

- X is an $N\text{Vars}$ -by-1 state vector of process variables (for example, short rates or equity prices) to simulate.
- W is an $N\text{Browns}$ -by-1 Brownian motion vector.
- F is an $N\text{Vars}$ -by-1 vector-valued drift-rate function.
- G is an $N\text{Vars}$ -by- $N\text{Browns}$ matrix-valued diffusion-rate function.

`simByEuler` simulates $N\text{Trials}$ sample paths of $N\text{Vars}$ correlated state variables driven by $N\text{Browns}$ Brownian motion sources of risk over $N\text{Periods}$ consecutive observation periods, using the Euler approach to approximate continuous-time stochastic processes.

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Further refining of the Euler scheme gives out a method with a weak order 2:

$$\begin{aligned} X_{t+1} = & X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt) + \mu'(X_t)\sigma(X_t)dI \\ & + \frac{1}{2}\left(\frac{1}{2}\sigma^2(X_t)\mu''(X_t) + \mu(X_t)\mu'(X_t)\right)dt^2 + \left(\frac{1}{2}\sigma^2(X_t)\sigma''(X_t) + \mu(X_t)\sigma'(X_t)\right)(dW_t dt - dI) \end{aligned}$$

where dI is given by the area of the triangle with base dt and height dW .

Version History

Introduced in R2023b

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

`simulate` | `simByMilstein`

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71
"SDEs" on page 14-2
"SDE Models" on page 14-7
"SDE Class Hierarchy" on page 14-5
"Quasi-Monte Carlo Simulation" on page 14-63
"Performance Considerations" on page 14-65

simByMilstein2

Simulate CIR process sample paths by second order Milstein approximation

Syntax

```
[Paths,Times,Z] = simByMilstein2(MDL,NPeriods)
[Paths,Times,Z] = simByMilstein2( ___,Name=Value)
```

Description

[Paths,Times,Z] = simByMilstein2(MDL,NPeriods) simulates NTrials sample paths of NVars state variables driven by the CIR process sources of risk over NPeriods consecutive observation periods, approximating continuous-time Cox-Ingersoll-Ross (CIR) by the second order Milstein approximation.

simByMilstein2 provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as DeltaTime approaches zero.

simByMilstein2 is only valid for diagonal diffusion SDE models.

[Paths,Times,Z] = simByMilstein2(___,Name=Value) specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for MonteCarloMethod, QuasiSequence, and BrownianMotionMethod. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with simByMilstein2 Scheme Using CIR Model

This example shows how to use simByMilstein2 with a CIR model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead of pseudo random numbers.

The Cox-Ingersoll-Ross (CIR) short rate class derives directly from SDE with mean-reverting drift

$$(SDEMMD): dX_t = S(t)[L(t) - X_t]dt + D(t, X_t^{\frac{1}{2}})V(t)dW$$

where D is a diagonal matrix whose elements are the square root of the corresponding element of the state vector.

Create a cir object to represent the model: $dX_t = 0.2(0.1 - X_t)dt + 0.05X_t^{\frac{1}{2}}dW$.

```
cir_obj = cir(0.2, 0.1, 0.05) % (Speed, Level, Sigma)
```

```

cir_obj =
  Class CIR: Cox-Ingersoll-Ross
  -----
  Dimensions: State = 1, Brownian = 1
  -----
  StartTime: 0
  StartState: 1
  Correlation: 1
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Sigma: 0.05
  Level: 0.1
  Speed: 0.2

```

Define the quasi-Monte Carlo simulation with use `simByMilstein2` using the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z] = simByMilstein2(cir_obj,10,ntrials=4096,method="basic",montecarlomethod="quasi",
```

Input Arguments

MDL — Stochastic differential equation model

CIR object

Stochastic differential equation model, specified as a `cir` object. You can create a `bates` object using `cir`.

Data Types: `object`

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: `double`

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z] = simByMilstein2(CIR_obj,NPeriods,DeltaTime=dt,Method="reflection")`

Method — Method to handle negative values

'basic' (default) | character vector with values 'basic', 'absorption', 'reflection', 'partialtruncation', 'fulltruncation', or 'higham-mao' | string with values "basic", "absorption", "reflection", "partialtruncation", "fulltruncation", or "higham-mao"

Method to handle negative values, specified as `Method` and a character vector or string with a supported value.

Note The `Method` argument is only supported when using a CIR object. For more information on creating a CIR object, see `cir`.

Data Types: `char` | `string`

NTrials — Simulated trials (sample paths) of NPeriods observations each

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: `double`

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of `'DeltaTime'` and a scalar or a `NPeriods`-by-1 column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps within each time increment dt (specified as `DeltaTime`)

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilestonein` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps` - 1 intermediate points. Although `simByMilstein2` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

Antithetic — Flag to indicate whether `simByMilstein2` uses antithetic sampling to generate Gaussian random variates

`False` (no antithetic sampling) (default) | logical with values `True` or `False`

Flag that indicates whether `simByMilstein2` uses antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes). This argument is specified as `Antithetic` and a scalar logical flag with a value of `True` or `False`.

When you specify `True`, `simByMilstein2` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1, 3, 5, ...) correspond to the primary Gaussian paths.
- Even trials (2, 4, 6, ...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `Antithetic`.

Data Types: `logical`

Z — Direct specification of the dependent random noise process used to generate Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the `SDE` object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process used to generate the Brownian motion vector (Wiener process) that drives the simulation. This argument is specified as `Z` and a function or as an `(NPeriods × NSteps)-by-NBrowns-by-NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns-by-1` column vector, and you must call it with two inputs:

- A real-valued scalar observation time t .
 - An `NVars-by-1` state vector X_t .
-

Data Types: `double` | `function`

StorePaths — Flag that indicates how the output array `Paths` is stored and returned

`True` (default) | logical with values `True` or `False`

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar logical flag with a value of `True` or `False`.

- If `StorePaths` is `True` (the default value) or is unspecified, `simByMilstein2` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is `False` (logical 0), `simByMilstein2` returns the `Paths` output array as an empty matrix.

Data Types: `logical`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `MonteCarloMethod`.

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as QuasiSequence and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If MonteCarloMethod option is not specified or specified as "standard", QuasiSequence is ignored.

If you specify an input noise process (see Z), simByMilstein2 ignores the value of QuasiSequence.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as BrownianMotionMethod and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the Z input argument, BrownianMotionMethod is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the MonteCarloMethod using pseudo random numbers. However, the performance differs between the two when the MonteCarloMethod option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

simByEuler makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as `Processes` and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByMilstein2` function runs processing functions at each interpolation time. They must accept the current interpolation time t , and the current state vector X_t , and return a state vector that may be an adjustment to the input state.

If you specify more than one processing function, `simByMilstein2` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When the input flag `StorePaths = False`, `simByMilstein2` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates used to generate Brownian motion vector

array

Dependent random variates used to generate the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time series array.

More About

Second-Order Milstein Method

The second-order Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The second order Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The second-order Milstein method is more accurate than the Euler-Maruyama method and is often used to solve SDEs with strong or weak convergence rates.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another of the same expected value, but smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of any other pair, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

This function simulates any vector-valued SDE of the form

$$dX_t = F(t, X_t)dt + G(t, X_t)dW_t$$

where:

- X is an $NVars$ -by-1 state vector of process variables (for example, short rates or equity prices) to simulate.
- W is an $NBrowns$ -by-1 Brownian motion vector.
- F is an $NVars$ -by-1 vector-valued drift-rate function.
- G is an $NVars$ -by- $NBrowns$ matrix-valued diffusion-rate function.

`simByEuler` simulates $NTrials$ sample paths of $NVars$ correlated state variables driven by $NBrowns$ Brownian motion sources of risk over $NPeriods$ consecutive observation periods, using the Euler approach to approximate continuous-time stochastic processes.

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Further refining of the Euler scheme gives out a method with a weak order 2:

$$\begin{aligned} X_{t+1} = & X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt) + \mu'(X_t)\sigma(X_t)dI \\ & + \frac{1}{2}\left(\frac{1}{2}\sigma^2(X_t)\mu''(X_t) + \mu(X_t)\mu'(X_t)\right)dt^2 + \left(\frac{1}{2}\sigma^2(X_t)\sigma''(X_t) + \mu(X_t)\sigma'(X_t)\right)(dW_t dt - dI) \end{aligned}$$

where dI is given by the area of the triangle with base dt and height dW .

Version History

Introduced in R2023a

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

`simulate` | `cir` | `simByMilstein`

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71
"SDEs" on page 14-2
"SDE Models" on page 14-7
"SDE Class Hierarchy" on page 14-5
"Quasi-Monte Carlo Simulation" on page 14-63
"Performance Considerations" on page 14-65

simByMilstein2

Simulate Heston process sample paths by second order Milstein approximation

Syntax

```
[Paths,Times,Z] = simByMilstein2(MDL,NPeriods)
[Paths,Times,Z] = simByMilstein2( ___,Name=Value)
```

Description

`[Paths,Times,Z] = simByMilstein2(MDL,NPeriods)` simulates `NTrials` sample paths of a Heston bivariate model driven by two `NBrowns` Brownian motion sources of risk approximating continuous-time stochastic processes by the second order Milstein approximation.

`simByMilstein2` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`simByMilstein2` is only valid for diagonal diffusion SDE models.

`[Paths,Times,Z] = simByMilstein2( ___,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein2 Scheme Using Heston Model

This example shows how to use `simByMilstein2` with a Heston model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead pseudo random numbers.

Define the parameters for the heston object.

```
Return = 0.03;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;

AssetPrice = 80;
V0 = 0.04;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a heston object.

```
Heston = heston(Return,Speed,Level,Volatility,startstate=StartState,correlation=Correlation)
```

```
Heston =
  Class HESTON: Heston Bivariate Stochastic Volatility
  -----
  Dimensions: State = 2, Brownian = 2
  -----
  StartTime: 0
  StartState: 2x1 double array
  Correlation: 2x2 double array
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Return: 0.03
  Speed: 1
  Level: 0.05
  Volatility: 0.2
```

Perform a quasi-Monte Carlo simulation by using `simByMilstein2` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z] = simByMilstein2(Heston,10,ntrials=4096,MonteCarloMethod="quasi",QuasiSequence="s
```

Input Arguments

MDL — Stochastic differential equation model

Heston object

Stochastic differential equation model, specified as a `heston` object. You can create a `heston` object using `heston`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z] = simByMilstein2(Heston_obj,NPeriods,NTrials=10,DeltaTime=dt)`

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as `DeltaTime` and a scalar or an `NPeriods-by-1` column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: double

NSteps — Number of intermediate time steps within each time increment dt

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilstein2` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByMilstein2` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: double

Antithetic — Flag to use antithetic sampling to generate Gaussian random variates

false (no antithetic sampling) (default) | logical with values true or false

Flag to use antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes), specified as `Antithetic` and a scalar numeric or logical 1 (true) or 0 (false).

When you specify true, `simByEuler` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1,3,5,...) correspond to the primary Gaussian paths.
- Even trials (2,4,6,...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `Antithetic`.

Data Types: logical

Z — Direct specification of the dependent random noise process for generating Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the SDE object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process for generating the Brownian motion vector (Wiener process) that drives the simulation, specified as `Z` and a function or as an (`NPeriods` ☐ `NSteps`)-by-`NBrowns`-by-`NTrials` three-dimensional array of dependent random variates.

Note If you specify `Z` as a function, it must return an `NBrowns`-by-1 column vector, and you must call it with two inputs:

- A real-valued scalar observation time t
 - An `NVars`-by-1 state vector X_t
-

Data Types: `double` | `function`

StorePaths — Flag that indicates how Paths is stored and returned

`true` (default) | logical with values `true` or `false`

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar numeric or logical 1 (`true`) or 0 (`false`).

- If `StorePaths` is `true` (the default value) or is unspecified, `simByMilstein2` returns `Paths` as a three-dimensional time-series array.
- If `StorePaths` is `false` (logical 0), `simByMilstein2` returns `Paths` as an empty matrix.

Data Types: `logical`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

`"standard"` (default) | string with values `"standard"`, `"quasi"`, or `"randomized-quasi"` | character vector with values `'standard'`, `'quasi'`, or `'randomized-quasi'`

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- `"standard"` — Monte Carlo using pseudo random numbers
- `"quasi"` — Quasi-Monte Carlo using low-discrepancy sequences
- `"randomized-quasi"` — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `MonteCarloMethod`.

Data Types: `string` | `char`

QuasiSequence — Low discrepancy sequence to drive stochastic processes

`"sobol"` (default) | string with value `"sobol"` | character vector with value `'sobol'`

Low discrepancy sequence to drive the stochastic processes, specified as `QuasiSequence` and a string or character vector with the following value:

- `"sobol"` — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If `MonteCarloMethod` option is not specified or specified as `"standard"`, `QuasiSequence` is ignored.

If you specify an input noise process (see `Z`), `simByMilstein2` ignores the value of `QuasiSequence`.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as `BrownianMotionMethod` and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the `Z` input argument, `BrownianMotionMethod` is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the `MonteCarloMethod` using pseudo random numbers. However, the performance differs between the two when the `MonteCarloMethod` option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

`simByMilstein2` makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as `Processes` and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByMilstein2` function runs processing functions at each interpolation time. The functions must accept the current interpolation time t , and the current state vector X_t and return a state vector that can be an adjustment to the input state.

If you specify more than one processing function, `simByMilstein2` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

The end-of-period `Processes` argument allows you to terminate a given trial early. At the end of each time step, `simByMilstein2` tests the state vector X_t for an all-`NaN` condition. Thus, to signal an early termination of a given trial, all elements of the state vector X_t must be `NaN`. This test enables you to define a `Processes` function to signal early termination of a trial, and offers significant performance benefits in some situations (for example, pricing down-and-out barrier options).

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When `StorePaths` is set to `false`, `simByMilstein2` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates for generating Brownian motion vector

array

Dependent random variates for generating the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time-series array.

More About

Second-Order Milstein Method

The second-order Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The second order Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The second-order Milstein method is more accurate than the Euler-Maruyama method and is often used to solve SDEs with strong or weak convergence rates.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another that has the same expected value but a smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent other pairs, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Further refining of the Euler scheme gives out a method with a weak order 2:

$$\begin{aligned} X_{t+1} = & X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt) + \mu'(X_t)\sigma(X_t)dI \\ & + \frac{1}{2}\left(\frac{1}{2}\sigma^2(X_t)\mu''(X_t) + \mu(X_t)\mu'(X_t)\right)dt^2 + \left(\frac{1}{2}\sigma^2(X_t)\sigma''(X_t) + \mu(X_t)\sigma'(X_t)\right)(dW_t dt - dI) \end{aligned}$$

where dI is given by the area of the triangle with base dt and height dW .

Version History

Introduced in R2023b

References

[1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

heston | simulate | simByMilstein

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

"Performance Considerations" on page 14-65

simByMilstein2

Simulate Bates process sample paths by second order Milstein approximation

Syntax

```
[Paths,Times,Z,N] = simByMilstein2(MDL,NPeriods)
[Paths,Times,Z,N] = simByMilstein2( ____,Name=Value)
```

Description

`[Paths,Times,Z,N] = simByMilstein2(MDL,NPeriods)` simulates `NTrials` sample paths of a Bates bivariate model driven by `NBrowns` Brownian motion sources of risk and `NJUMPS` compound Poisson processes representing the arrivals of important events over `NPeriods` consecutive observation periods, approximating continuous-time stochastic processes by the second order Milstein approximation.

`simByMilstein2` provides a discrete-time approximation of the underlying generalized continuous-time process. The simulation is derived directly from the stochastic differential equation of motion; the discrete-time process approaches the true continuous-time process only in the limit as `DeltaTime` approaches zero.

`simByMilstein2` is only valid for diagonal diffusion SDE models.

`[Paths,Times,Z,N] = simByMilstein2( ____,Name=Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein2 Scheme Using Bates Model

This example shows how to use `simByMilstein2` with a Bates model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead of pseudo random numbers.

Define the parameters for the `bates` object.

```
AssetPrice = 80;
Return = 0.03;
JumpMean = 0.02;
JumpVol = 0.08;
JumpFreq = 0.1;

V0 = 0.04;
Level = 0.05;
Speed = 1.0;
```

```
Volatility = 0.2;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a bates object.

```
Bates = bates(Return, Speed, Level, Volatility, ...
             JumpFreq, JumpMean, JumpVol,startstate=StartState,correlation=Correlation)
```

```
Bates =
  Class BATES: Bates Bivariate Stochastic Volatility
  -----
  Dimensions: State = 2, Brownian = 2
  -----
  StartTime: 0
  StartState: 2x1 double array
  Correlation: 2x2 double array
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Return: 0.03
  Speed: 1
  Level: 0.05
  Volatility: 0.2
  JumpFreq: 0.1
  JumpMean: 0.02
  JumpVol: 0.08
```

Perform a quasi-Monte Carlo simulation by using `simByMilstein2` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,time,z,n] = simByMilstein2(Bates,10,ntrials=4096,MonteCarloMethod="randomized-quasi",Quas
```

Input Arguments

MDL — Stochastic differential equation model

Bates object

Stochastic differential equation model, specified as a bates object. You can create a bates object using `bates`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z,N] = simByMilstein2(Bates_obj,NPeriods,NTrials=10,DeltaTime=dt)`

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as `NTrials` and a positive scalar integer.

Data Types: `double`

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as `DeltaTime` and a scalar or an `NPeriods-by-1` column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps within each time increment dt

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as `DeltaTime`), specified as `NSteps` and a positive scalar integer.

The `simByMilstein2` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByMilstein2` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

Antithetic — Flag to use antithetic sampling to generate Gaussian random variates

`false` (no antithetic sampling) (default) | logical with values `true` or `false`

Flag to use antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes), specified as `Antithetic` and a scalar numeric or logical `1` (`true`) or `0` (`false`).

When you specify `true`, `simByEuler` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1,3,5,...) correspond to the primary Gaussian paths.
- Even trials (2,4,6,...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see Z), `simByMilstein2` ignores the value of `Antithetic`.

Data Types: `logical`

Z — Direct specification of the dependent random noise process for generating Brownian motion vector

generates correlated Gaussian variates based on the `Correlation` member of the SDE object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process for generating the Brownian motion vector (Wiener process) that drives the simulation, specified as Z and a function or as an (NPeriods $\times$ NSteps)-by-NBrowns-by-NTrials three-dimensional array of dependent random variates.

Note If you specify Z as a function, it must return an NBrowns-by-1 column vector, and you must call it with two inputs:

- A real-valued scalar observation time t
 - An NVars-by-1 state vector X_t
-

Data Types: `double` | `function`

N — Dependent random counting process for generating number of jumps

random numbers from Poisson distribution with parameter `JumpFreq` from a `bates` object (default) | three-dimensional array | function

Dependent random counting process for generating the number of jumps, specified as N and a function or an (NPeriods $\times$ NSteps) -by-NJumps-by-NTrials three-dimensional array of dependent random variates.

Note If you specify a function, N must return an NJumps-by-1 column vector, and you must call it with two inputs: a real-valued scalar observation time t followed by an NVars-by-1 state vector X_t .

Data Types: `double` | `function`

StorePaths — Flag that indicates how Paths is stored and returned

`true` (default) | logical with values `true` or `false`

Flag that indicates how the output array `Paths` is stored and returned, specified as `StorePaths` and a scalar numeric or logical 1 (`true`) or 0 (`false`).

- If `StorePaths` is `true` (the default value) or is unspecified, `simByMilstein2` returns `Paths` as a three-dimensional time-series array.
- If `StorePaths` is `false` (logical 0), `simByMilstein2` returns `Paths` as an empty matrix.

Data Types: `logical`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as `MonteCarloMethod` and a string or character vector with one of the following values:

- `"standard"` — Monte Carlo using pseudo random numbers
- `"quasi"` — Quasi-Monte Carlo using low-discrepancy sequences
- `"randomized-quasi"` — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see Z), `simByMilstein2` ignores the value of `MonteCarloMethod`.

Data Types: `string` | `char`

QuasiSequence — Low discrepancy sequence to drive stochastic processes

`"sobol"` (default) | string with value `"sobol"` | character vector with value `'sobol'`

Low discrepancy sequence to drive the stochastic processes, specified as `QuasiSequence` and a string or character vector with the following value:

- `"sobol"` — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If `MonteCarloMethod` option is not specified or specified as `"standard"`, `QuasiSequence` is ignored.

If you specify an input noise process (see Z), `simByMilstein2` ignores the value of `QuasiSequence`.

Data Types: `string` | `char`

BrownianMotionMethod — Brownian motion construction method

`"standard"` (default) | string with value `"brownian-bridge"` or `"principal-components"` | character vector with value `'brownian-bridge'` or `'principal-components'`

Brownian motion construction method, specified as `BrownianMotionMethod` and a string or character vector with one of the following values:

- `"standard"` — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- `"brownian-bridge"` — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- `"principal-components"` — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the Z input argument, `BrownianMotionMethod` is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the MonteCarloMethod using pseudo random numbers. However, the performance differs between the two when the MonteCarloMethod option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

simByMilstein2 makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as Processes and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The simByMilstein2 function runs processing functions at each interpolation time. The functions must accept the current interpolation time t , and the current state vector X_t and return a state vector that can be an adjustment to the input state.

If you specify more than one processing function, simByMilstein2 invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

The end-of-period Processes argument allows you to terminate a given trial early. At the end of each time step, simByMilstein2 tests the state vector X_t for an all-NaN condition. Thus, to signal an early termination of a given trial, all elements of the state vector X_t must be NaN. This test enables you to define a Processes function to signal early termination of a trial, and offers significant performance benefits in some situations (for example, pricing down-and-out barrier options).

Data Types: cell | function

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an (NPeriods + 1)-by-NVars-by-NTrials three-dimensional time series array.

For a given trial, each row of Paths is the transpose of the state vector X_t at time t . When StorePaths is set to false, simByMilstein2 returns Paths as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an (NPeriods + 1)-by-1 column vector. Each element of Times is associated with the corresponding row of Paths.

Z — Dependent random variates for generating Brownian motion vector

array

Dependent random variates for generating the Brownian motion vector (Wiener processes) that drive the simulation, returned as an (NPeriods $\times$ NSteps)-by-NBrowns-by-NTrials three-dimensional time-series array.

N — Dependent random variates for generating jump counting process vector

array

Dependent random variates used to generate the jump counting process vector, returned as an (NPeriods $\times$ NSteps)-by-NJumps-by-NTrials three-dimensional time series array.

More About

Second-Order Milstein Method

The second-order Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The second order Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The second-order Milstein method is more accurate than the Euler-Maruyama method and is often used to solve SDEs with strong or weak convergence rates.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another that has the same expected value but a smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent other pairs, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Further refining of the Euler scheme gives out a method with a weak order 2:

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt) + \mu'(X_t)\sigma(X_t)dI \\ + \frac{1}{2}\left(\frac{1}{2}\sigma^2(X_t)\mu''(X_t) + \mu(X_t)\mu'(X_t)\right)dt^2 + \left(\frac{1}{2}\sigma^2(X_t)\sigma''(X_t) + \mu(X_t)\sigma'(X_t)\right)(dW_t dt - dI)$$

where dI is given by the area of the triangle with base dt and height dW .

Version History

Introduced in R2023b

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

bates | simulate | simByMilstein

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

"Performance Considerations" on page 14-65

simByMilstein2

Simulate diagonal diffusion Merton sample paths by second order Milstein approximation

Syntax

```
[Paths,Times,Z,N] = simByMilstein2(MDL,NPeriods)
[Paths,Times,Z,N] = simByMilstein2( ___,Name=Value)
```

Description

[Paths,Times,Z,N] = simByMilstein2(MDL,NPeriods) simulates NTrials sample paths of NVars correlated state variables driven by NBrowns Brownian motion sources of risk and NJumps compound Poisson processes representing the arrivals of important events over NPeriods consecutive observation periods. The simulation approximates the continuous-time Merton jump diffusion process by the second order Milstein approximation.

simByMilstein2 is only valid for diagonal diffusion SDE models.

[Paths,Times,Z,N] = simByMilstein2(___,Name=Value) specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for MonteCarloMethod, QuasiSequence, and BrownianMotionMethod. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Quasi-Monte Carlo Simulation with Milstein2 Scheme Using Merton Model

This example shows how to use simByMilstein2 with a Merton model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead pseudo random numbers.

Define the parameters for the merton object.

```
AssetPrice = 80;
Return = 0.03;
Sigma = 0.16;
JumpMean = 0.02;
JumpVol = 0.08;
JumpFreq = 2;
```

Create a merton object.

```
mertonObj = merton(Return,Sigma,JumpFreq,JumpMean,JumpVol,StartState=AssetPrice)
```

```
mertonObj =
  Class MERTON: Merton Jump Diffusion
  -----
  Dimensions: State = 1, Brownian = 1
```

```

-----
StartTime: 0
StartState: 80
Correlation: 1
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
Simulation: simulation method/function simByEuler
  Sigma: 0.16
  Return: 0.03
  JumpFreq: 2
  JumpMean: 0.02
  JumpVol: 0.08

```

Perform a quasi-Monte Carlo simulation by using `simByMilstein2` with the optional name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`.

```
[paths,times] = simByMilstein2(mertonObj,10,Ntrials=4096,MonteCarloMethod="quasi",QuasiSequence=)
```

Input Arguments

MDL — Stochastic differential equation model

Merton object

Stochastic differential equation model, specified as a merton object. You can create a merton object using `merton`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `[Paths,Times,Z,N] =`

```
simByMilstein2(Merton_obj,NPeriods,NTrials=1000,DeltaTime=dt,NSteps=10)
```

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive scalar integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as the comma-separated pair consisting of 'NTrials' and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of 'DeltaTime' and a scalar or a NPeriods-by-1 column vector.

DeltaTime represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: double

NSteps — Number of intermediate time steps within each time increment

1 (indicating no intermediate evaluation) (default) | positive scalar integer

Number of intermediate time steps within each time increment dt (specified as DeltaTime), specified as the comma-separated pair consisting of 'NSteps' and a positive scalar integer.

The `simByEuler` function partitions each time increment dt into NSteps subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at $NSteps - 1$ intermediate points. Although `simByEuler` does not report the output state vector at these intermediate points, the refinement improves accuracy by allowing the simulation to more closely approximate the underlying continuous-time process.

Data Types: double

Antithetic — Flag to use antithetic sampling to generate Gaussian random variates

false (no antithetic sampling) (default) | logical with values true or false

Flag to use antithetic sampling to generate the Gaussian random variates that drive the Brownian motion vector (Wiener processes), specified as the comma-separated pair consisting of 'Antithetic' and a scalar numeric or logical 1 (true) or 0 (false).

When you specify true, `simByEuler` performs sampling such that all primary and antithetic paths are simulated and stored in successive matching pairs:

- Odd trials (1,3,5,...) correspond to the primary Gaussian paths.
- Even trials (2,4,6,...) are the matching antithetic paths of each pair derived by negating the Gaussian draws of the corresponding primary (odd) trial.

Note If you specify an input noise process (see Z), `simByEuler` ignores the value of Antithetic.

Data Types: logical

Z — Direct specification of the dependent random noise process for generating Brownian motion vector

generates correlated Gaussian variates based on the Correlation member of the SDE object (default) | function | three-dimensional array of dependent random variates

Direct specification of the dependent random noise process for generating the Brownian motion vector (Wiener process) that drives the simulation, specified as the comma-separated pair consisting of 'Z' and a function or as an (NPeriods × NSteps)-by-NBrowns-by-NTrials three-dimensional array of dependent random variates.

Note If you specify Z as a function, it must return an NBrowns-by-1 column vector, and you must call it with two inputs:

- A real-valued scalar observation time t
- An $N\text{Vars}$ -by-1 state vector X_t

Data Types: double | function

N — Dependent random counting process for generating number of jumps

random numbers from Poisson distribution with parameter `JumpFreq` from `merton` object (default) | three-dimensional array | function

Dependent random counting process for generating the number of jumps, specified as the comma-separated pair consisting of 'N' and a function or an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Jumps}$ -by- $N\text{Trials}$ three-dimensional array of dependent random variates.

If you specify a function, N must return an $N\text{Jumps}$ -by-1 column vector, and you must call it with two inputs: a real-valued scalar observation time t followed by an $N\text{Vars}$ -by-1 state vector X_t .

Data Types: double | function

StorePaths — Flag that indicates how Paths is stored and returned

true (default) | logical with values true or false

Flag that indicates how the output array `Paths` is stored and returned, specified as the comma-separated pair consisting of 'StorePaths' and a scalar numeric or logical 1 (true) or 0 (false).

- If `StorePaths` is true (the default value) or is unspecified, `simByEuler` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is false (logical 0), `simByEuler` returns `Paths` as an empty matrix.

Data Types: logical

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as the comma-separated pair consisting of 'MonteCarloMethod' and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Note If you specify an input noise process (see `Z` and `N`), `simByEuler` ignores the value of `MonteCarloMethod`.

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as the comma-separated pair consisting of 'QuasiSequence' and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension.

Note If MonteCarloMethod option is not specified or specified as "standard", QuasiSequence is ignored.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as the comma-separated pair consisting of 'BrownianMotionMethod' and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

Note If an input noise process is specified using the Z input argument, BrownianMotionMethod is ignored.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the MonteCarloMethod using pseudo random numbers. However, the performance differs between the two when the MonteCarloMethod option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

simByEuler makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as the comma-separated pair consisting of 'Processes' and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

The `simByEuler` function runs processing functions at each interpolation time. The functions must accept the current interpolation time t , and the current state vector X_t and return a state vector that can be an adjustment to the input state.

If you specify more than one processing function, `simByEuler` invokes the functions in the order in which they appear in the cell array. You can use this argument to specify boundary conditions, prevent negative prices, accumulate statistics, plot graphs, and more.

The end-of-period `Processes` argument allows you to terminate a given trial early. At the end of each time step, `simByEuler` tests the state vector X_t for an all-`NaN` condition. Thus, to signal an early termination of a given trial, all elements of the state vector X_t must be `NaN`. This test enables you to define a `Processes` function to signal early termination of a trial, and offers significant performance benefits in some situations (for example, pricing down-and-out barrier options).

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When `StorePaths` is set to `false`, `simByMilstein2` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

Z — Dependent random variates for generating Brownian motion vector

array

Dependent random variates used to generate the Brownian motion vector (Wiener processes) that drive the simulation, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Browns}$ -by- $N\text{Trials}$ three-dimensional time-series array.

N — Dependent random variates used for generating jump counting process vector

array

Dependent random variates for generating the jump counting process vector, returned as an $(N\text{Periods} \times N\text{Steps})$ -by- $N\text{Jumps}$ -by- $N\text{Trials}$ three-dimensional time-series array.

More About

Second-Order Milstein Method

The second-order Milstein method is a numerical method for approximating solutions to stochastic differential equations (SDEs).

The second order Milstein method is an extension of the Euler-Maruyama method, which is a first-order numerical method for SDEs. The second-order Milstein method is more accurate than the Euler-Maruyama method and is often used to solve SDEs with strong or weak convergence rates.

Antithetic Sampling

Simulation methods allow you to specify a popular *variance reduction* technique called *antithetic sampling*.

This technique attempts to replace one sequence of random observations with another that has the same expected value but a smaller variance. In a typical Monte Carlo simulation, each sample path is independent and represents an independent trial. However, antithetic sampling generates sample paths in pairs. The first path of the pair is referred to as the *primary path*, and the second as the *antithetic path*. Any given pair is independent of other pairs, but the two paths within each pair are highly correlated. Antithetic sampling literature often recommends averaging the discounted payoffs of each pair, effectively halving the number of Monte Carlo trials.

This technique attempts to reduce variance by inducing negative dependence between paired input samples, ideally resulting in negative dependence between paired output samples. The greater the extent of negative dependence, the more effective antithetic sampling is.

Algorithms

This function simulates any vector-valued SDE of the following form:

$$dX_t = B(t, X_t)X_t dt + D(t, X_t)V(t, x_t)dW_t + Y(t, X_t, N_t)X_t dN_t$$

Here:

- X_t is an `NVars`-by-1 state vector of process variables.
- $B(t, X_t)$ is an `NVars`-by-`NVars` matrix of generalized expected instantaneous rates of return.
- $D(t, X_t)$ is an `NVars`-by-`NVars` diagonal matrix in which each element along the main diagonal is the corresponding element of the state vector.
- $V(t, X_t)$ is an `NVars`-by-`NVars` matrix of instantaneous volatility rates.
- dW_t is an `NBrowns`-by-1 Brownian motion vector.
- $Y(t, X_t, N_t)$ is an `NVars`-by-`NJumps` matrix-valued jump size function.
- dN_t is an `NJumps`-by-1 counting process vector.

`simByEuler` simulates `NTrials` sample paths of `NVars` correlated state variables driven by `NBrowns` Brownian motion sources of risk over `NPeriods` consecutive observation periods, using the Euler approach to approximate continuous-time stochastic processes.

Consider the process X satisfying a stochastic differential equation of the form.

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t$$

The attempt of including a term of $O(dt)$ in the drift refines the Euler scheme and results in the algorithm derived by Milstein [1].

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt)$$

Further refining of the Euler scheme gives out a method with a weak order 2:

$$X_{t+1} = X_t + \mu(X_t)dt + \sigma(X_t)dW_t + \frac{1}{2}\sigma(X_t)\sigma'(X_t)(dW_t^2 - dt) + \mu'(X_t)\sigma(X_t)dI \\ + \frac{1}{2}\left(\frac{1}{2}\sigma^2(X_t)\mu''(X_t) + \mu(X_t)\mu'(X_t)\right)dt^2 + \left(\frac{1}{2}\sigma^2(X_t)\sigma''(X_t) + \mu(X_t)\sigma'(X_t)\right)(dW_t dt - dI)$$

where dI is given by the area of the triangle with base dt and height dW .

Version History

Introduced in R2023b

References

- [1] Milstein, G.N. "A Method of Second-Order Accuracy Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 23, 1978, pp. 396-401.

See Also

merton | simulate | simByMilstein

Topics

"Price American Basket Options Using Standard Monte Carlo and Quasi-Monte Carlo Simulation" on page 14-71
 "SDEs" on page 14-2
 "SDE Models" on page 14-7
 "SDE Class Hierarchy" on page 14-5
 "Quasi-Monte Carlo Simulation" on page 14-63
 "Performance Considerations" on page 14-65

simByTransition

Simulate Bates sample paths with transition density

Syntax

```
[Paths,Times] = simByTransition(MDL,NPeriods)
[Paths,Times] = simByTransition( ___,Name,Value)
```

Description

`[Paths,Times] = simByTransition(MDL,NPeriods)` simulates `NTrials` of Bates bivariate models driven by two Brownian motion sources of risk and one compound Poisson process representing the arrivals of important events over `NPeriods` consecutive observation periods. `simByTransition` approximates continuous-time stochastic processes by the transition density.

`[Paths,Times] = simByTransition( ___,Name,Value)` specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for `MonteCarloMethod`, `QuasiSequence`, and `BrownianMotionMethod`. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Use `simByTransition` with `bates` Object

Simulate Bates sample paths with transition density.

Define the parameters for the `bates` object.

```
AssetPrice = 80;
Return = 0.03;
JumpMean = 0.02;
JumpVol = 0.08;
JumpFreq = 0.1;
V0 = 0.04;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a `bates` object.

```
batesObj = bates(Return, Speed, Level, Volatility,...
                JumpFreq, JumpMean, JumpVol,'startstate',StartState,...
                'correlation',Correlation)
```

```
batesObj =
    Class BATES: Bates Bivariate Stochastic Volatility
```

```

-----
Dimensions: State = 2, Brownian = 2
-----
StartTime: 0
StartState: 2x1 double array
Correlation: 2x2 double array
    Drift: drift rate function F(t,X(t))
    Diffusion: diffusion rate function G(t,X(t))
Simulation: simulation method/function simByEuler
    Return: 0.03
    Speed: 1
    Level: 0.05
Volatility: 0.2
    JumpFreq: 0.1
    JumpMean: 0.02
    JumpVol: 0.08

```

Define the simulation parameters.

```

nPeriods = 5; % Simulate sample paths over the next five years
Paths = simByTransition(batesObj,nPeriods);
Paths

```

```

Paths = 6x2

```

```

    80.0000    0.0400
    99.5724    0.0201
   124.2066    0.0176
    56.5051    0.1806
    80.4545    0.1732
    94.7887    0.1169

```

Quasi-Monte Carlo Simulation Using Bates Model

This example shows how to use `simByTransition` with a Bates model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead pseudo random numbers.

Define the parameters for the `bates` object.

```

AssetPrice = 80;
Return = 0.03;
JumpMean = 0.02;
JumpVol = 0.08;
JumpFreq = 0.1;
V0 = 0.04;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];

```

Create a `bates` object.

```
Bates = bates(Return, Speed, Level, Volatility,...
             JumpFreq, JumpMean, JumpVol, 'startstate', StartState,...
             'correlation', Correlation)
```

```
Bates =
  Class BATES: Bates Bivariate Stochastic Volatility
  -----
  Dimensions: State = 2, Brownian = 2
  -----
  StartTime: 0
  StartState: 2x1 double array
  Correlation: 2x2 double array
    Drift: drift rate function F(t,X(t))
    Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
    Return: 0.03
    Speed: 1
    Level: 0.05
  Volatility: 0.2
    JumpFreq: 0.1
    JumpMean: 0.02
    JumpVol: 0.08
```

Perform a quasi-Monte Carlo simulation by using `simByTransition` with the optional name-value argument for `'MonteCarloMethod'`, `'QuasiSequence'`, and `'BrownianMotionMethod'`.

```
[paths,time] = simByTransition(Bates,10,'ntrials',4096,'MonteCarloMethod','quasi','QuasiSequence')
```

Input Arguments

MDL — Stochastic differential equation model

bates object

Stochastic differential equation model, specified as a `bates` object. For more information on creating a `bates` object, see `bates`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `[Paths,Times] = simByTransition(Bates,NPeriods,'DeltaTime',dt)`

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as the comma-separated pair consisting of 'NTrials' and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of 'DeltaTime' and a scalar or `NPeriods`-by-1 column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: double

NSteps — Number of intermediate time steps

1 (indicating no intermediate evaluation) (default) | positive integer

Number of intermediate time steps within each time increment dt (defined as `DeltaTime`), specified as the comma-separated pair consisting of 'NSteps' and a positive scalar integer.

The `simByTransition` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByTransition` does not report the output state vector at these intermediate points, the refinement improves accuracy by enabling the simulation to more closely approximate the underlying continuous-time process.

Data Types: double

StorePaths — Flag for storage and return method

True (default) | logical with values True or False

Flag for storage and return method that indicates how the output array `Paths` is stored and returned, specified as the comma-separated pair consisting of 'StorePaths' and a scalar logical flag with a value of True or False.

- If `StorePaths` is True (the default value) or is unspecified, then `simByTransition` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is False (logical 0), then `simByTransition` returns the `Paths` output array as an empty matrix.

Data Types: logical

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as the comma-separated pair consisting of 'MonteCarloMethod' and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Data Types: string | char

QuasiSequence — Low discrepancy sequence to drive stochastic processes

"sobel" (default) | string with value "sobel" | character vector with value 'sobel'

Low discrepancy sequence to drive the stochastic processes, specified as the comma-separated pair consisting of 'QuasiSequence' and a string or character vector with the following value:

- "sobel" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension

Note If MonteCarloMethod option is not specified or specified as "standard", QuasiSequence is ignored.

Data Types: string | char

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as the comma-separated pair consisting of 'BrownianMotionMethod' and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the MonteCarloMethod using pseudo random numbers. However, the performance differs between the two when the MonteCarloMethod option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: string | char

Processes — Sequence of end-of-period processes or state vector adjustments

simByTransition makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as the comma-separated pair consisting of 'Processes' and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

`simByTransition` applies processing functions at the end of each observation period. The processing functions accept the current observation time t and the current state vector X_t , and return a state vector that might adjust the input state.

If you specify more than one processing function, `simByTransition` invokes the functions in the order in which they appear in the cell array.

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an $(N\text{Periods} + 1)$ -by- $N\text{Vars}$ -by- $N\text{Trials}$ three-dimensional time series array.

For a given trial, each row of `Paths` is the transpose of the state vector X_t at time t . When the input flag `StorePaths = False`, `simByTransition` returns `Paths` as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an $(N\text{Periods} + 1)$ -by-1 column vector. Each element of `Times` is associated with the corresponding row of `Paths`.

More About

Transition Density

Transition density describes the probability distribution of a state variable transitioning from one state to another over a given time interval.

Transition Density Simulation

The CIR SDE has no solution such that $r(t) = f(r(0), \dots)$.

In other words, the equation is not explicitly solvable. However, the transition density for the process is known.

The exact simulation for the distribution of $r(t_1), \dots, r(t_n)$ is that of the process at times $t_1, \dots, t_n$ for the same value of $r(0)$. The transition density for this process is known and is expressed as

$$r(t) = \frac{\sigma^2(1 - e^{-\alpha(t-u)})}{4\alpha} x_d^2 \left(\frac{4\alpha e^{-\alpha(t-u)}}{\sigma^2(1 - e^{-\alpha(t-u)})} r(u) \right), t > u$$

where

$$d \equiv \frac{4b\alpha}{\sigma^2}$$

Bates Model

Bates models are bivariate composite models.

Each Bates model consists of two coupled univariate models:

- A geometric Brownian motion (gbm) model with a stochastic volatility function and jumps.

$$dX_{1t} = B(t)X_{1t}dt + \sqrt{X_{2t}}X_{1t}dW_{1t} + Y(t)X_{1t}dN_t$$

This model usually corresponds to a price process whose volatility (variance rate) is governed by the second univariate model.

- A Cox-Ingersoll-Ross (cir) square root diffusion model.

$$dX_{2t} = S(t)[L(t) - X_{2t}]dt + V(t)\sqrt{X_{2t}}dW_{2t}$$

This model describes the evolution of the variance rate of the coupled Bates price process.

Version History

Introduced in R2020b

R2022a: Perform Quasi-Monte Carlo simulation

Behavior changed in R2022a

Perform Quasi-Monte Carlo simulation using the name-value arguments `MonteCarloMethod` and `QuasiSequence`.

R2022b: Perform Brownian bridge and principal components construction

Behavior changed in R2022b

Perform Brownian bridge and principal components construction using the name-value argument `BrownianMotionMethod`.

References

- [1] Glasserman, Paul *Monte Carlo Methods in Financial Engineering*, New York: Springer-Verlag, 2004.
- [2] Van Haastrecht, Alexander, and Antoon Pelsser. "Efficient, Almost Exact Simulation of the Heston Stochastic Volatility Model." *International Journal of Theoretical and Applied Finance*, Vol. 13, No. 01 (2010): 1-43.

See Also

`bates` | `simByEuler` | `simByQuadExp`

Topics

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

simByTransition

Simulate Heston sample paths with transition density

Syntax

```
[Paths,Times] = simByTransition(MDL,NPeriods)
[Paths,Times] = simByTransition( ___,Name,Value)
```

Description

[Paths,Times] = simByTransition(MDL,NPeriods) simulates NTrials sample paths of Heston bivariate models driven by two Brownian motion sources of risk. **simByTransition** approximates continuous-time stochastic processes by the transition density.

[Paths,Times] = simByTransition(___,Name,Value) specifies options using one or more name-value pair arguments in addition to the input arguments in the previous syntax.

You can perform quasi-Monte Carlo simulations using the name-value arguments for **MonteCarloMethod**, **QuasiSequence**, and **BrownianMotionMethod**. For more information, see “Quasi-Monte Carlo Simulation” on page 14-63.

Examples

Use simByTransition with heston Object

Simulate Heston sample paths with transition density.

Define the parameters for the heston object.

```
Return = 0.03;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;

AssetPrice = 80;
V0 = 0.04;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a heston object.

```
hestonObj = heston(Return,Speed,Level,Volatility,'startstate',StartState,'correlation',Correlation)
```

```
hestonObj =
    Class HESTON: Heston Bivariate Stochastic Volatility
    -----
    Dimensions: State = 2, Brownian = 2
    -----
    StartTime: 0
    StartState: 2x1 double array
```

```
Correlation: 2x2 double array
  Drift: drift rate function F(t,X(t))
  Diffusion: diffusion rate function G(t,X(t))
  Simulation: simulation method/function simByEuler
  Return: 0.03
  Speed: 1
  Level: 0.05
  Volatility: 0.2
```

Define the simulation parameters.

```
nPeriods = 5; % Simulate sample paths over the next five years
Paths = simByTransition(hestonObj,nPeriods);
Paths
```

```
Paths = 6x2
```

```
80.0000    0.0400
99.7718    0.0201
124.7044    0.0176
52.7914    0.1806
75.3173    0.1732
76.7572    0.1169
```

Quasi-Monte Carlo Simulation Using Heston Model

This example shows how to use `simByTransition` with a Heston model to perform a quasi-Monte Carlo simulation. Quasi-Monte Carlo simulation is a Monte Carlo simulation that uses quasi-random sequences instead pseudo random numbers.

Define the parameters for the heston object.

```
Return = 0.03;
Level = 0.05;
Speed = 1.0;
Volatility = 0.2;

AssetPrice = 80;
V0 = 0.04;
Rho = -0.7;
StartState = [AssetPrice;V0];
Correlation = [1 Rho;Rho 1];
```

Create a heston object.

```
Heston = heston(Return,Speed,Level,Volatility,'startstate',StartState,'correlation',Correlation)
```

```
Heston =
  Class HESTON: Heston Bivariate Stochastic Volatility
  -----
  Dimensions: State = 2, Brownian = 2
  -----
  StartTime: 0
  StartState: 2x1 double array
  Correlation: 2x2 double array
```

```

    Drift: drift rate function F(t,X(t))
    Diffusion: diffusion rate function G(t,X(t))
    Simulation: simulation method/function simByEuler
    Return: 0.03
    Speed: 1
    Level: 0.05
    Volatility: 0.2

```

Perform a quasi-Monte Carlo simulation by using `simByTransition` with the optional name-value argument for `'MonteCarloMethod'`, `'QuasiSequence'`, and `'BrownianMotionMethod'`.

```
[paths,time] = simByTransition(Heston,10,'ntrials',4096,'MonteCarloMethod','quasi','QuasiSequence')
```

Input Arguments

MDL — Stochastic differential equation model

heston object

Stochastic differential equation model, specified as a `heston` object. For more information on creating a `heston` object, see `heston`.

Data Types: object

NPeriods — Number of simulation periods

positive scalar integer

Number of simulation periods, specified as a positive scalar integer. The value of `NPeriods` determines the number of rows of the simulated output series.

Data Types: double

Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

```
Example: [Paths,Times] = simByTransition(Heston,NPeriods,'DeltaTime',dt)
```

NTrials — Simulated trials (sample paths)

1 (single path of correlated state variables) (default) | positive integer

Simulated trials (sample paths) of `NPeriods` observations each, specified as the comma-separated pair consisting of `'NTrials'` and a positive scalar integer.

Data Types: double

DeltaTime — Positive time increments between observations

1 (default) | scalar | column vector

Positive time increments between observations, specified as the comma-separated pair consisting of `'DeltaTime'` and a scalar or `NPeriods`-by-1 column vector.

`DeltaTime` represents the familiar dt found in stochastic differential equations, and determines the times at which the simulated paths of the output state variables are reported.

Data Types: `double`

NSteps — Number of intermediate time steps

1 (indicating no intermediate evaluation) (default) | positive integer

Number of intermediate time steps within each time increment dt (defined as `DeltaTime`), specified as the comma-separated pair consisting of '`NSteps`' and a positive scalar integer.

The `simByTransition` function partitions each time increment dt into `NSteps` subintervals of length $dt/NSteps$, and refines the simulation by evaluating the simulated state vector at `NSteps - 1` intermediate points. Although `simByTransition` does not report the output state vector at these intermediate points, the refinement improves accuracy by enabling the simulation to more closely approximate the underlying continuous-time process.

Data Types: `double`

MonteCarloMethod — Monte Carlo method to simulate stochastic processes

"standard" (default) | string with values "standard", "quasi", or "randomized-quasi" | character vector with values 'standard', 'quasi', or 'randomized-quasi'

Monte Carlo method to simulate stochastic processes, specified as the comma-separated pair consisting of '`MonteCarloMethod`' and a string or character vector with one of the following values:

- "standard" — Monte Carlo using pseudo random numbers
- "quasi" — Quasi-Monte Carlo using low-discrepancy sequences
- "randomized-quasi" — Randomized quasi-Monte Carlo

Data Types: `string` | `char`

QuasiSequence — Low discrepancy sequence to drive stochastic processes

"sobol" (default) | string with value "sobol" | character vector with value 'sobol'

Low discrepancy sequence to drive the stochastic processes, specified as the comma-separated pair consisting of '`QuasiSequence`' and a string or character vector with the following value:

- "sobol" — Quasi-random low-discrepancy sequences that use a base of two to form successively finer uniform partitions of the unit interval and then reorder the coordinates in each dimension

Note If `MonteCarloMethod` option is not specified or specified as "standard", `QuasiSequence` is ignored.

Data Types: `string` | `char`

BrownianMotionMethod — Brownian motion construction method

"standard" (default) | string with value "brownian-bridge" or "principal-components" | character vector with value 'brownian-bridge' or 'principal-components'

Brownian motion construction method, specified as the comma-separated pair consisting of '`BrownianMotionMethod`' and a string or character vector with one of the following values:

- "standard" — The Brownian motion path is found by taking the cumulative sum of the Gaussian variates.
- "brownian-bridge" — The last step of the Brownian motion path is calculated first, followed by any order between steps until all steps have been determined.
- "principal-components" — The Brownian motion path is calculated by minimizing the approximation error.

The starting point for a Monte Carlo simulation is the construction of a Brownian motion sample path (or Wiener path). Such paths are built from a set of independent Gaussian variates, using either standard discretization, Brownian-bridge construction, or principal components construction.

Both standard discretization and Brownian-bridge construction share the same variance and, therefore, the same resulting convergence when used with the `MonteCarloMethod` using pseudo random numbers. However, the performance differs between the two when the `MonteCarloMethod` option "quasi" is introduced, with faster convergence for the "brownian-bridge" construction option and the fastest convergence for the "principal-components" construction option.

Data Types: `string` | `char`

StorePaths — Flag for storage and return method

`True` (default) | logical with `True` or `False`

Flag for storage and return method that indicates how the output array `Paths` is stored and returned, specified as the comma-separated pair consisting of 'StorePaths' and a scalar logical flag with a value of `True` or `False`.

- If `StorePaths` is `True` (the default value) or is unspecified, then `simByTransition` returns `Paths` as a three-dimensional time series array.
- If `StorePaths` is `False` (logical 0), then `simByTransition` returns the `Paths` output array as an empty matrix.

Data Types: `logical`

Processes — Sequence of end-of-period processes or state vector adjustments

`simByTransition` makes no adjustments and performs no processing (default) | function | cell array of functions

Sequence of end-of-period processes or state vector adjustments, specified as the comma-separated pair consisting of 'Processes' and a function or cell array of functions of the form

$$X_t = P(t, X_t)$$

`simByTransition` applies processing functions at the end of each observation period. The processing functions accept the current observation time t and the current state vector X_t , and return a state vector that might adjust the input state.

If you specify more than one processing function, `simByTransition` invokes the functions in the order in which they appear in the cell array.

Data Types: `cell` | `function`

Output Arguments

Paths — Simulated paths of correlated state variables

array

Simulated paths of correlated state variables, returned as an (NPeriods + 1)-by-NVars-by-NTrials three-dimensional time series array.

For a given trial, each row of Paths is the transpose of the state vector X_t at time t . When the input flag StorePaths = False, simByTransition returns Paths as an empty matrix.

Times — Observation times associated with simulated paths

column vector

Observation times associated with the simulated paths, returned as an (NPeriods + 1)-by-1 column vector. Each element of Times is associated with the corresponding row of Paths.

More About

Transition Density

Transition density describes the probability distribution of a state variable transitioning from one state to another over a given time interval.

Transition Density Simulation

The CIR SDE has no solution such that $r(t) = f(r(0), \dots)$.

In other words, the equation is not explicitly solvable. However, the transition density for the process is known.

The exact simulation for the distribution of $r(t_1), \dots, r(t_n)$ is that of the process at times $t_1, \dots, t_n$ for the same value of $r(0)$. The transition density for this process is known and is expressed as

$$r(t) = \frac{\sigma^2(1 - e^{-\alpha(t-u)})}{4\alpha} x_d^2 \left(\frac{4\alpha e^{-\alpha(t-u)}}{\sigma^2(1 - e^{-\alpha(t-u)})} r(u) \right), t > u$$

where

$$d \equiv \frac{4b\alpha}{\sigma^2}$$

Heston Model

Heston models are bivariate composite models.

Each Heston model consists of two coupled univariate models:

- A geometric Brownian motion (gbm) model with a stochastic volatility function.

$$dX_{1t} = B(t)X_{1t}dt + \sqrt{X_{2t}}X_{1t}dW_{1t}$$

This model usually corresponds to a price process whose volatility (variance rate) is governed by the second univariate model.

- A Cox-Ingersoll-Ross (`cir`) square root diffusion model.

$$dX_{2t} = S(t)[L(t) - X_{2t}]dt + V(t)\sqrt{X_{2t}}dW_{2t}$$

This model describes the evolution of the variance rate of the coupled GBM price process.

Version History

Introduced in R2020b

R2022a: Perform Quasi-Monte Carlo simulation

Behavior changed in R2022a

Perform Quasi-Monte Carlo simulation using the name-value arguments `MonteCarloMethod` and `QuasiSequence`.

R2022b: Perform Brownian bridge and principal components construction

Behavior changed in R2022b

Perform Brownian bridge and principal components construction using the name-value argument `BrownianMotionMethod`.

References

- [1] Glasserman, Paul *Monte Carlo Methods in Financial Engineering*, New York: Springer-Verlag, 2004.
- [2] Van Haastrecht, Alexander, and Antoon Pelsser. "Efficient, Almost Exact Simulation of the Heston Stochastic Volatility Model." *International Journal of Theoretical and Applied Finance*, Vol. 13, No. 01 (2010): 1-43.

See Also

`simByEuler` | `simByQuadExp` | `heston`

Topics

"SDEs" on page 14-2

"SDE Models" on page 14-7

"SDE Class Hierarchy" on page 14-5

"Quasi-Monte Carlo Simulation" on page 14-63

Bibliography

Bibliography

"Bond Pricing and Yields" on page A-2
"Term Structure of Interest Rates" on page A-2
"Derivatives Pricing and Yields" on page A-3
"Portfolio Analysis" on page A-3
"Investment Performance Metrics" on page A-3
"Financial Statistics" on page A-4
"Standard References" on page A-4
"Credit Risk Analysis" on page A-5
"Portfolio Optimization" on page A-5
"Stochastic Differential Equations" on page A-6
"Life Tables" on page A-6

Note For the well-known algorithms and formulas used in Financial Toolbox software (such as how to compute a loan payment given principal, interest rate, and length of the loan), no references are given here. The references here pertain to less common formulas.

Bond Pricing and Yields

The pricing and yield formulas for fixed-income securities come from:

- [1] Golub, B.W. and L.M. Tilman. *Risk Management: Approaches for Fixed Income Markets*. Wiley, 2000.
- [2] Martellini, L., P. Priaulet, and S. Priaulet. *Fixed Income Securities*. Wiley, 2003.
- [3] Mayle, Jan. *Standard Securities Calculation Methods*. New York: Securities Industry Association, Inc. Vol. 1, 3rd ed., 1993, ISBN 1-882936-01-9. Vol. 2, 1994, ISBN 1-882936-02-7.
- [4] Tuckman, B. *Fixed Income Securities: Tools for Today's Markets*. Wiley, 2002.

In many cases these formulas compute the price of a security given yield, dates, rates, and other data. These formulas are nonlinear, however; so when solving for an independent variable within a formula, Financial Toolbox software uses Newton's method. See any elementary numerical methods textbook for the mathematics underlying Newton's method.

Term Structure of Interest Rates

The formulas and methodology for term structure functions come from:

- [5] Fabozzi, Frank J. "The Structure of Interest Rates." Ch. 6 in Fabozzi, Frank J. and T. Dossa Fabozzi, eds. *The Handbook of Fixed Income Securities*. 4th ed. New York, Irwin Professional Publishing, 1995, ISBN 0-7863-0001-9.
- [6] McEnally, Richard W. and James V. Jordan. "The Term Structure of Interest Rates." Ch. 37 in Fabozzi and Fabozzi, *ibid*.
- [7] Das, Satyajit. "Calculating Zero Coupon Rates." *Swap and Derivative Financing*. Appendix to Ch. 8, pp. 219-225, New York, Irwin Professional Publishing., 1994, ISBN 1-55738-542-4.

Derivatives Pricing and Yields

The pricing and yield formulas for derivative securities come from:

[8] Chriss, Neil A. *Black-Scholes and Beyond: Option Pricing Models*. Chicago, Irwin Professional Publishing, 1997, ISBN 0-7863-1025-1.

[9] Cox, J., S. Ross, and M. Rubenstein. "Option Pricing: A Simplified Approach." *Journal of Financial Economics*. Vol. 7, Sept. 1979, pp. 229-263.

[10] Hull, John C. *Options, Futures, and Other Derivatives*. 5th edition, Prentice Hall, 2003, ISBN 0-13-009056-5.

Portfolio Analysis

The Markowitz model is used for portfolio analysis computations. For a discussion of this model see Chapter 7 of:

[11] Bodie, Zvi, Alex Kane, and Alan J. Marcus. *Investments*. 2nd. Edition. Burr Ridge, IL, Irwin Professional Publishing, 1993, ISBN 0-256-08342-8.

Investment Performance Metrics

The risk and ratio formulas for investment performance metrics come from:

[12] Daniel Bernoulli. "Exposition of a New Theory on the Measurement of Risk." *Econometrica*. Vol. 22, No 1, January 1954, pp. 23-36 (English translation of "Specimen Theoriae Novae de Mensura Sortis." *Commentarii Academiae Scientiarum Imperialis Petropolitanae*. Tomus V, 1738, pp. 175-192).

[13] Martin Eling and Frank Schuhmacher. *Does the Choice of Performance Measure Influence the Evaluation of Hedge Funds?* Working Paper, November 2005.

[14] John Lintner. "The Valuation of Risk Assets and the Selection of Risky Investments in Stocks Portfolios and Capital Budgets." *Review of Economics and Statistics*. Vol. 47, No. 1, February 1965, pp. 13-37.

[15] Malik Magdon-Ismail, Amir F. Atiya, Amrit Pratap, and Yaser S. Abu-Mostafa. "On the Maximum Drawdown of a Brownian Motion." *Journal of Applied Probability*. Volume 41, Number 1, March 2004, pp. 147-161.

[16] Malik Magdon-Ismail and Amir Atiya. "Maximum Drawdown." <https://www.risk.net/risk-magazine>, October 2004.

[17] Harry Markowitz. "Portfolio Selection." *Journal of Finance*. Vol. 7, No. 1, March 1952, pp. 77-91.

[18] Harry Markowitz. *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, 1959.

[19] Jan Mossin. "Equilibrium in a Capital Asset Market." *Econometrica*. Vol. 34, No. 4, October 1966, pp. 768-783.

[20] Christian S. Pedersen and Ted Rudholm-Alfvén. "Selecting a Risk-Adjusted Shareholder Performance Measure." *Journal of Asset Management*. Vol. 4, No. 3, 2003, pp. 152-172.

[21] William F. Sharpe. "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk." *Journal of Finance*. Vol. 19, No. 3, September 1964, pp. 425-442.

[22] Katerina Simons. "Risk-Adjusted Performance of Mutual Funds." *New England Economic Review*. September/October 1998, pp. 34-48.

Financial Statistics

The discussion of computing statistical values for portfolios containing missing data elements derives from the following references:

[23] Little, Roderick J.A. and Donald B. Rubin. *Statistical Analysis with Missing Data*. 2nd Edition. John Wiley & Sons, Inc., 2002.

[24] Meng, Xiao-Li, and Donald B. Rubin. "Maximum Likelihood Estimation via the ECM Algorithm." *Biometrika*. Vol. 80, No. 2, 1993, pp. 267-278.

[25] Sexton, Joe and Anders Rygh Swensen. "ECM Algorithms That Converge at the Rate of EM." *Biometrika*. Vol. 87, No. 3, 2000, pp. 651-662.

[26] Dempster, A.P., N.M. Laird, and Donald B. Rubin. "Maximum Likelihood from Incomplete Data via the EM Algorithm." *Journal of the Royal Statistical Society. Series B*, Vol. 39, No. 1, 1977, pp. 1-37.

Standard References

Standard references include:

[27] Addendum to Securities Industry Association, *Standard Securities Calculation Methods: Fixed Income Securities Formulas for Analytic Measures*. Vol. 2, Spring 1995. This addendum explains and clarifies the end-of-month rule.

[28] Brealey, Richard A. and Stewart C. Myers. *Principles of Corporate Finance*. New York, McGraw-Hill. 4th ed., 1991, ISBN 0-07-007405-4.

[29] Daigler, Robert T. *Advanced Options Trading*. Chicago, Probus Publishing Co., 1994, ISBN 1-55738-552-1.

[30] *A Dictionary of Finance*. Oxford, Oxford University Press., 1993, ISBN 0-19-285279-5.

[31] Fabozzi, Frank J. and T. Dossa Fabozzi, eds. *The Handbook of Fixed-Income Securities*. 4th Edition. Burr Ridge, IL, Irwin, 1995, ISBN 0-7863-0001-9.

[32] Fitch, Thomas P. *Dictionary of Banking Terms*. 2nd Edition. Hauppauge, NY, Barron's. 1993, ISBN 0-8120-1530-4.

[33] Hill, Richard O., Jr. *Elementary Linear Algebra*. Orlando, FL, Academic Press. 1986, ISBN 0-12-348460-X.

[34] Luenberger, David G. *Investment Science*. Oxford University Press, 1998. ISBN 0195108094.

[35] Marshall, John F. and Vipul K. Bansal. *Financial Engineering: A Complete Guide to Financial Innovation*. New York, New York Institute of Finance. 1992, ISBN 0-13-312588-2.

[36] Sharpe, William F. *Macro-Investment Analysis*. An "electronic work-in-progress" published on the World Wide Web, 1995, at <https://www.stanford.edu/~wfsarpe/mia/mia.htm>.

[37] Sharpe, William F. and Gordon J. Alexander. *Investments*. Englewood Cliffs, NJ: Prentice-Hall. 4th ed., 1990, ISBN 0-13-504382-4.

[38] Stigum, Marcia, with Franklin Robinson. *Money Market and Bond Calculations*. Richard D. Irwin., 1996, ISBN 1-55623-476-7.

Credit Risk Analysis

The credit rating and estimation transition probabilities come from:

[39] Altman, E. "Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy." *Journal of Finance*. Vol. 23, No. 4, (Sept., 1968), pp. 589-609.

[40] Basel Committee on Banking Supervision, *International Convergence of Capital Measurement and Capital Standards: A Revised Framework, Bank for International Settlements (BIS)*. comprehensive version, June 2006.

[41] Hanson, S. and T. Schuermann. "Confidence Intervals for Probabilities of Default." *Journal of Banking & Finance*. Vol. 30(8), Elsevier, August 2006, pp. 2281-2301.

[42] Jafry, Y. and T. Schuermann. "Measurement, Estimation and Comparison of Credit Migration Matrices." *Journal of Banking & Finance*. Vol. 28(11), Elsevier, November 2004, pp. 2603-2639.

[43] Löffler, G. and P. N. Posch. *Credit Risk Modeling Using Excel and VBA*. West Sussex, England: Wiley Finance, 2007.

[44] Schuermann, T. "Credit Migration Matrices." in E. Melnick and B. Everitt (eds.), *Encyclopedia of Quantitative Risk Analysis and Assessment*. Wiley, 2008.

Credit Derivatives

Beumee, J., D. Brigo, D. Schiemert, and G. Stoye. "Charting a Course Through the CDS Big Bang." *Fitch Solutions, Quantitative Research*. Global Special Report. April 7, 2009.

Hull, J., and A. White. "Valuing Credit Default Swaps I: No Counterparty Default Risk." *Journal of Derivatives*. Vol. 8, pp. 29-40.

O'Kane, D. and S. Turnbull. "Valuation of Credit Default Swaps." *Lehman Brothers, Fixed Income Quantitative Credit Research*. April, 2003.

O'Kane, D. *Modelling Single-name and Multi-name Credit Derivatives*. Wiley Finance, 2008, pp. 156-169.

Portfolio Optimization

The Markowitz model is used for portfolio optimization computations.

[45] Kelley, J. E. "The Cutting-Plane Method for Solving Convex Programs." *Journal of the Society for Industrial and Applied Mathematics*. Vol. 8, No. 4, December 1960, pp. 703-712.

[46] Markowitz, H. "Portfolio Selection." *Journal of Finance*. Vol. 7, No. 1, March 1952, pp. 77-91.

- [47] Markowitz, H. M. *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, Inc., 1959.
- [48] Rockafellar, R. T. and S. Uryasev. "Optimization of Conditional Value-at-Risk." *Journal of Risk*. Vol. 2, No. 3, Spring 2000, pp. 21-41.
- [49] Rockafellar, R. T. and S. Uryasev. "Conditional Value-at-Risk for General Loss Distributions." *Journal of Banking and Finance*. Vol. 26, 2002, pp. 1443-1471.
- [50] Konno, H. and H. Yamazaki. "Mean-Absolute Deviation Portfolio Optimization Model and Its Application to Tokyo Stock Market." *Management Science*. Vol. 37, No. 5, May 1991, pp. 519-531.
- [51] Cornuejols, A. and R. Tütüncü. *Optimization Methods in Finance*. Cambridge University Press, 2007.

Stochastic Differential Equations

The SDE formulas come from:

- [52] Aït-Sahalia, Y. "Testing Continuous-Time Models of the Spot Interest Rate." *The Review of Financial Studies*. Spring 1996, Vol. 9, No. 2, pp. 385-426.
- [53] Aït-Sahalia, Y. "Transition Densities for Interest Rate and Other Nonlinear Diffusions." *The Journal of Finance*. Vol. 54, No. 4, August 1999.
- [54] Glasserman, P. *Monte Carlo Methods in Financial Engineering*. Springer-Verlag, New York, 2004.
- [55] Hull, J. C. *Options, Futures, and Other Derivatives*. 5th edition, Englewood Cliffs, NJ: Prentice Hall, 2002.
- [56] Johnson, N. L., S. Kotz, and N. Balakrishnan. *Continuous Univariate Distributions*. Vol. 2, 2nd ed. New York: John Wiley & Sons, 1995.
- [57] Shreve, S. E. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer-Verlag, New York, 2004.

Life Tables

The Life Table formulas come from:

- [58] Arias, E. "United States Life Tables." *National Vital Statistics Reports, U.S. Department of Health and Human Services*. Vol. 62, No. 7, 2009.
- [59] Carriere, F. "Parametric Models for Life Tables." *Transactions of the Society of Actuaries*. Vol. 44, 1992, pp. 77-99.
- [60] Gompertz, B. "On the Nature of the Function Expressive of the Law of Human Mortality, and on a New Mode of Determining the Value of Life Contingencies." *Philosophical Transactions of the Royal Society*. Vol. 115, 1825, pp. 513-582.
- [61] Heligman, L. M. A., and J. H. Pollard. "The Age Pattern of Mortality." *Journal of the Institute of Actuaries*. Vol. 107, Pt. 1, 1980, pp. 49-80.

- [62] Makeham, W. M. "On the Law of Mortality and the Construction of Annuity Tables." *Journal of the Institute of Actuaries*. Vol. 8, 1860. pp. 301–310.
- [63] Siler, W. "A Competing-Risk Model for Animal Mortality." *Ecology*. Vol. 60, pp. 750–757, 1979.
- [64] Siler, W. "Parameters of Mortality in Human Populations with Widely Varying Life Spans." *Statistics in Medicine*. Vol. 2, 1983, pp. 373–380.

